

# claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664.jsonl`  
- SHA-256: `38ef80915af3cfcaa202238c97486faff15db805bbab84fd4b8c56bfa12a2a12`  
- Source modified: `2026-07-07T04:00:20+00:00`  
- Imported at: `2026-07-08T15:59:13+00:00`  
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

## Transcript

### 1. user (2026-07-06T05:54:19.047Z)

Hi. Can you ramp up and review <https://github.com/avidullu/telegram-video-filter/pulls> thoroughly. Make sure to sync the repo to remote head before executing

### 2. assistant (2026-07-06T05:54:28.698Z)

I'll start by ramping up: locating the repo, checking memory/handoff context, and syncing to remote head. Let me explore in parallel.

### 3. user (2026-07-06T05:54:35.058Z)

```
total 196  
drwxr-xr-x 1 avidu 197609      0 Jul  6 10:47 .  
drwxr-xr-x 1 avidu 197609      0 Jul  5 22:04 ..  
-rw-r--r-- 1 avidu 197609 122880 Jul  6 10:47 .coverage  
-rw-r--r-- 1 avidu 197609    733 Jul  3 23:33 .env  
-rw-r--r-- 1 avidu 197609    730 Jul  3 22:46 .env.example  
drwxr-xr-x 1 avidu 197609      0 Jul  6 10:49 .git  
-rw-r--r-- 1 avidu 197609    827 Jul  3 22:36 .gitignore  
drwxr-xr-x 1 avidu 197609      0 Jul  4 08:36 .mypy_cache  
-rw-r--r-- 1 avidu 197609    645 Jul  3 22:46 .pre-commit-config.yaml  
drwxr-xr-x 1 avidu 197609      0 Jul  3 22:43 .pytest_cache  
drwxr-xr-x 1 avidu 197609      0 Jul  4 08:36 .ruff_cache  
drwxr-xr-x 1 avidu 197609      0 Jul  3 22:41 .venv  
drwxr-xr-x 1 avidu 197609      0 Jul  4 15:06 data  
-rw-r--r-- 1 avidu 197609    662 Jul  3 22:46 docker-compose.yml  
-rw-r--r-- 1 avidu 197609   1005 Jul  4 00:39 Dockerfile  
drwxr-xr-x 1 avidu 197609      0 Jul  6 10:23 docs  
-rw-r--r-- 1 avidu 197609   3373 Jul  4 00:47 pyproject.toml  
-rw-r--r-- 1 avidu 197609   3354 Jul  4 14:26 README.md  
drwxr-xr-x 1 avidu 197609      0 Jul  4 14:26 scripts  
drwxr-xr-x 1 avidu 197609      0 Jul  3 22:40 src  
drwxr-xr-x 1 avidu 197609      0 Jul  6 10:23 tests  
---TELEGRAM GIT---  
origin https://github.com/avidullu/telegram-video-filter.git (fetch)  
origin https://github.com/avidullu/telegram-video-filter.git (push)  
---BADMINTON---  
total 13  
drwxr-xr-x 1 avidu 197609      0 Jul  2 11:41 .  
drwxr-xr-x 1 avidu 197609      0 Jul  5 22:04 ..  
drwxr-xr-x 1 avidu 197609      0 Jul  2 22:08 .claude  
-rw-r--r-- 1 avidu 197609    600 Jul  2 11:41 AGENTS.md  
drwxr-xr-x 1 avidu 197609      0 Jun 30 03:00 output
```

### 4. user (2026-07-06T05:54:37.852Z)

```
---PROJECTS DIR---
_p3_scratch
Agent Sessions
badminton-highlight-indexer
khelsutra-guru
muneem-ai
Telegram
Vairagi
---gh version---
gh version 2.92.0 (2026-04-28)
---gh auth---
github.com
? Logged in to github.com account avidullu (keyring)
- Active account: true
- Git operations protocol: https
- Token: gho_*****
- Token scopes: 'gist', 'read:org', 'repo', 'workflow'
```

## 5. user (2026-07-06T05:54:40.008Z)

```
Exit code 1
---MEMORY badminton---
---MEMORY.md badminton---
```

## 6. assistant (2026-07-06T05:54:48.551Z)

The repo is at `C:/Users/avidu/Projects/Telegram`. gh is authenticated as `avidullu`. Let me check its git state, sync to remote, list PRs, and look for any handoff/memory.

## 7. user (2026-07-06T05:54:53.813Z)

```
---BRANCH---
feat/track-b-domain-schema
---STATUS---
```

### feat/track-b-domain-schema...origin/feat/track-b-domain-schema

```
---LOG (local HEAD)---
2771285 feat(domain): provider-neutral schema ? sources/content_items/policies (ADR 0011, Track B)
f6c9322 docs(adr): accept 0011 provider-neutral domain schema (Track B decision) (#34)
5ffc25b feat(identity): split users into accounts + telegram_sessions (ADR 0010, Track A) (#33)
0e21d36 revert: remove accidental file (tooling mistake ? will use issue-comment tool)
54cdb3c noop
```

## 8. user (2026-07-06T05:54:55.878Z)

```
---Telegram memory dir---
---handoff root---
---any project memory dirs---
```

## 9. user (2026-07-06T05:54:58.782Z)

```
---OPEN PRs---
[{"author":{"id":"MDQ6VXNlcjE2NzQ4Nzc=","is_bot":false,"login":"avidullu","name":"Avi"},"baseRefName":"main","createdAt":"2026-07-06T05:19:48Z","headRefName":"feat/track-b-domain-schema","isDraft":false,"number":35,"title":"feat(domain): provider-neutral schema ? sources/content_items/policies (ADR 0011, Track B)"}]
```

## 10. assistant (2026-07-06T05:55:08.222Z)

One open PR: **##35** (`feat/track-b-domain-schema` ? `main`), and I'm currently on that branch. No prior handoff/memory exists for this repo, so this is a fresh ramp-up. Let me sync to remote and gather context.

## 11. user (2026-07-06T05:55:16.428Z)

```
---AHEAD/BEHIND current vs origin---
0      0
(left=origin ahead, right=local ahead)
---main vs origin/main---
0      0
---origin/main log---
f6c9322 docs(adr): accept 0011 provider-neutral domain schema (Track B decision) (#34)
5ffc25b feat(identity): split users into accounts + telegram_sessions (ADR 0010, Track A) (#33)
0e21d36 revert: remove accidental file (tooling mistake ? will use issue-comment tool)
54cdb3c noop
b817b63 docs(adr): accept 0010 identity/session separation (Track A decision) (#32)
```

## 12. assistant (2026-07-06T05:55:25.950Z)

Fully synced ? clean tree, current branch and `main` both match origin. PR #35 is one commit on top of `main`. Now let me pull the PR contents and repo conventions to ramp up.

## 13. user (2026-07-06T05:55:32.455Z)

```
===PR #35 META===
{"additions":1299,"body":"## What\n\nImplements **Track B (#27)** of the provider-neutral umbrella (#25), following ADR 0011 (Option 1A: tiered `dedup_key`; Option 2A: `policies` generalizes `filter_config`), accepted via PR #34.\n\nThis is the implementation PR promised in #34's body once the two decisions landed.\n\n## Schema (6 new tables)\n\n| Table | Populated by | Purpose |\n|---|---|---|\n| `sources` | ? backfilled from `channels` | Generalizes channels (`provider`, `provider_source_id`, `watched`) |\n| `content_items` | ? backfilled from `videos` | Generalizes videos (`dedup_key`, `provider_message_id`) |\n| `policies` | ? backfilled from `filter_config` | Generalizes filter_config (`name`, `is_default`, `config_json`) |\n| `destinations` | ? empty | Track C (#28) populates |\n| `routing_rules` | ? empty | Track C populates |\n| `delivery_attempts` | ? empty | Track C populates |\n\nThe three empty tables are the **contract Track C codes against** ? landed now so the ADR is complete and Track C doesn't need a migration.\n\n## Dedup (Option 1A ? tiered lookup)\n\nPure `compute_dedup_key(provider, **metadata)` implements the decision tree:\n\n1. `document_id` set ? `\"tg:doc:{id}\"` (Tier-1)\n2. all four fingerprint fields set ? `\"tg:fp:{size:dur:h:w}\"` (Tier-2)\n3. else ? `\"tg:msg:{source_id}:{message_id}\"` (Tier-0 fallback)\n\nEach branch is **guarded** so partial metadata never collapses to a degenerate key (e.g. `tg:fp:0:0:0:0`) ? the ADR 0011 review fix. `UNIQUE(account_id, dedup_key)` replaces the old Tier-1 `document_id` index.\n\n## Migration `_003_domain_schema` (idempotent, integer-preserving)\n\n1. Backfill `sources` from `channels` (`watched=1` per ADR 0009 backfill).\n2. Backfill `content_items` from `videos`, computing `dedup_key` in SQL via the 3-branch `CASE`.\n3. Backfill `policies` from `filter_config` (one `default` policy per account, `is_default=1`).\n4. Rename `channels`/`videos`/`filter_config` ? `*_deprecated` (one-release rollback net, ADR 0010 pattern).\n\n**Idempotency fix:** `_apply_schema` drops empty legacy-table recreations on subsequent `init_db` runs (the `_SCHEMA` `CREATE TABLE IF NOT EXISTS` recreates them after `user_version` is bumped, so the migration never re-runs to clean them up).\n\n## Backwards-compat shims (Track-A pattern)\n\nLegacy repos rewritten as shims over the new tables ? `telethon_client.py`, `backfill.py`, `commands.py` keep working without a flag day.\n\n| Legacy function | Now reads/writes |\n|---|---|\n| `upsert_channel` / `get_channel` | `sources` |\n| `get_video` / `get_video_by_*` / `insert_video` | `content_items` (+ `compute_dedup_key`) |\n| `load_filter_config` / `save_filter_config` | `policies` (default policy) |\n| `mark_forwarded` | no-op (delivery state ? Track C) |\n\n`mark_forwarded` is a no-op because `content_items` has no `forwarded` column ? per-destination delivery state moves to `delivery_attempts` in Track C. For Phase 1 this is safe: forwards are idempotent, and `matched` (set at insert time) is the surviving signal.\n\n## Verification\n\n`n434` passed @ 96.63% coverage (gate: 80%)\n\nruff check . ? clean\n\nruff format --check ? clean (on changed files)\n\n17 new tests:\n\n**`compute_dedup_key` matrix (9 cases):** all three branches, partial-metadata guard (missing `width` ? Tier-0, not degenerate `tg:fp`), empty-string `document_id`, case-insensitive provider prefix, future-provider prefix, missing-required-fields `ValueError`.\n\n**Source/ContentItem/Policy repos:** round-trip, conflict-update, dedup_key uniqueness enforcement, default-policy lookup.\n\n**Migration 003 backfill:** seeds rows hitting all three dedup branches on a legacy DB, asserts no two rows collapse to the same `dedup_key`.\n\n**Idempotency:** second `init_db` is a no-op.\n\n## Scope discipline\n\n? **Creates:** `sources`, `content_items`, `policies` (+ backfill).\n\n? **Creates empty:** `destinations`,
```

```
`routing_rules`, `delivery_attempts`.\n- ? **Shims:** legacy `channels`/`videos`/`filter_config`  
repos keep working.\n- ? **Not in this track:** delivery logic, routing logic, command changes  
(Tracks C/D/E).\n\n## Closes\n\nCloses #27 (Track B). Unblocks #28 (Track C), #29 (Track D), #30  
(Track E).\n\n## Related\n\n- ADR 0011 ? the accepted decision (PR #34).\n- ADR 0010 ?  
identity/session split (this PR's `account_id` FKs point at `accounts.id`).\n- ADR 0006 ?  
content dedup (generalized by Option 1A).\n- ADR 0009 ? selective scanning + routing  
( `sources.watched`, `routing_rules` ).", "changedFiles":4, "deletions":150, "mergeStateStatus":"CLEA  
N", "mergeable":"MERGEABLE", "number":35, "reviewDecision": "", "statusCheckRollup": [], "title": "feat(  
domain): provider-neutral schema ? sources/content_items/policies (ADR 0011, Track B)"}  
  
===FILES CHANGED===  
src/tg_video_filter/db.py  
src/tg_video_filter/models.py  
tests/test_db.py  
tests/test_delivery.py
```

#### 14. user (2026-07-06T05:55:34.616Z)

===README===

## telegram-video-filter

A Telethon-based **userbot** that logs in as your own Telegram account, watches new video messages across all the channels/groups you subscribe to, applies configurable filters (duration, resolution, file size), and **forwards matching videos to your Saved Messages** in real time.

> ?? This project automates a **user** account (a "userbot") via MTProto. Keep > volumes reasonable and respect Telegram's Terms of Service. You are > responsible for your own account. See [`docs/adr/0001-userbot-over-botapi.md`](docs/adr/0001-userbot-over-botapi.md).

## Status

? **v1 complete** ? all 6 build phases merged via PR. The app runs end-to-end: bootstrap a user account ? manager watches their channels ? filters videos ? forwards matches to Saved Messages. See [`docs/plan.md`](docs/plan.md) for the build history and [`docs/runbook.md`](docs/runbook.md) to deploy. phased build plan and current status.

## What it does (v1)

- Watches every channel/group your account is a member of ? no per-channel setup.
- Reads video metadata **without downloading** the file (duration, width/height, size, MIME).
- Applies an **AND** of your filters, e.g.:
  - duration between 5s and 2h,
  - height ? 1080 (1080p or better),
  - size ? 2 GiB.
- Forwards each match to your **Saved Messages** in real time, or to a **channel you own** if you set `delivery_target`` (per-user).
- **Never forwards the same video twice** ? three-tier content dedup (message instance ? Telegram `document.id`` ? metadata fingerprint) means a video cross-posted to 5 channels is delivered once. See [`docs/adr/0006-content-dedup.md`](docs/adr/0006-content-dedup.md).
- Supports multiple users (<100) on a single host, each with its own session, filters, delivery mode/target, and dedup state.

## What it does NOT do yet (v2+)

- Public-channel `/add @channel`` join, manual-add ingestion.
- Scheduled digest delivery and on-demand `/list`` / `/search``.
- ffmpeg-based deep inspection (`/inspect``).
- Codec / keyword / MIME filters.

## Documentation

- [`docs/plan.md`](docs/plan.md) ? phased build plan, scope, verification.
- [`docs/design.md`](docs/design.md) ? architecture, data model, control flow.
- [`docs/runbook.md`](docs/runbook.md) ? deploy, onboard users, rotate sessions, handle flood waits.
- [`docs/adr/`](docs/adr/) ? Architecture Decision Records:
  - [0001 ? Userbot over Bot API](docs/adr/0001-userbot-over-botapi.md)
  - [0002 ? SQLite over PostgreSQL](docs/adr/0002-sqlite-over-postgres.md)
  - [0003 ? Metadata-first probing](docs/adr/0003-metadata-first-probing.md)
  - [0004 ? Pluggable delivery modes](docs/adr/0004-pluggable-delivery-modes.md)
  - [0005 ? Per-user delivery target](docs/adr/0005-per-user-delivery-target.md)

## Quick start

See the [runbook](docs/runbook.md) for the full guide. In short:

```
```bash
cp .env.example .env          # set TG_API_ID, TG_API_HASH from my.telegram.org
docker compose up -d --build
python -m tg_video_filter.bootstrap # interactive one-time login per user
docker compose restart app
```
```

## Tech stack

Python 3.11+ · [Telethon](https://docs.telethon.dev/) ·aiosqlite (SQLite) ·  
pydantic / pydantic-settings · Docker · pytest · ruff.

## License

TBD.

```
===repo CLAUDE.md / AGENTS.md===
ls: cannot access 'CLAUDE.md': No such file or directory
ls: cannot access 'AGENTS.md': No such file or directory
ls: cannot access 'CONTRIBUTING.md': No such file or directory
---
```

### 15. user (2026-07-06T05:55:34.695Z)

```
===TREE (src/tests/docs)===
src/tg_video_filter/backfill.py
src/tg_video_filter/bootstrap.py
src/tg_video_filter/commands.py
src/tg_video_filter/config.py
src/tg_video_filter/db.py
src/tg_video_filter/delivery.py
src/tg_video_filter/filters/base.py
src/tg_video_filter/filters/content_type.py
src/tg_video_filter/filters/duration.py
src/tg_video_filter/filters/resolution.py
src/tg_video_filter/filters/size.py
src/tg_video_filter/filters/__init__.py
src/tg_video_filter/filters/__pycache__/base.cpython-313.pyc
src/tg_video_filter/filters/__pycache__/content_type.cpython-313.pyc
src/tg_video_filter/filters/__pycache__/duration.cpython-313.pyc
src/tg_video_filter/filters/__pycache__/resolution.cpython-313.pyc
src/tg_video_filter/filters/__pycache__/size.cpython-313.pyc
src/tg_video_filter/filters/__pycache__/__init__.cpython-313.pyc
src/tg_video_filter/manager.py
src/tg_video_filter/models.py
src/tg_video_filter/telethon_client.py
```

src/tg\_video\_filter/\_\_init\_\_.py  
src/tg\_video\_filter/\_\_main\_\_.py  
src/tg\_video\_filter/\_\_pycache\_\_/backfill.cpython-313.pyc  
src/tg\_video\_filter/\_\_pycache\_\_/bootstrap.cpython-313.pyc  
src/tg\_video\_filter/\_\_pycache\_\_/commands.cpython-313.pyc  
src/tg\_video\_filter/\_\_pycache\_\_/config.cpython-313.pyc  
src/tg\_video\_filter/\_\_pycache\_\_/db.cpython-313.pyc  
src/tg\_video\_filter/\_\_pycache\_\_/delivery.cpython-313.pyc  
src/tg\_video\_filter/\_\_pycache\_\_/manager.cpython-313.pyc  
src/tg\_video\_filter/\_\_pycache\_\_/models.cpython-313.pyc  
src/tg\_video\_filter/\_\_pycache\_\_/telethon\_client.cpython-313.pyc  
src/tg\_video\_filter/\_\_pycache\_\_/\_\_init\_\_.cpython-313.pyc  
src/tg\_video\_filter/\_\_pycache\_\_/\_\_main\_\_.cpython-313.pyc  
tests/conftest.py  
tests/test\_backfill.py  
tests/test\_bootstrap.py  
tests/test\_commands.py  
tests/test\_config.py  
tests/test\_db.py  
tests/test\_delivery.py  
tests/test\_filters\_content\_type.py  
tests/test\_filters\_duration.py  
tests/test\_filters\_resolution.py  
tests/test\_filters\_size.py  
tests/test\_filter\_engine.py  
tests/test\_main.py  
tests/test\_manager.py  
tests/test\_telethon\_extraction.py  
tests/\_\_pycache\_\_/conftest.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/conftest.cpython-313-pytest-9.0.3.pyc  
tests/\_\_pycache\_\_/test\_backfill.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/test\_backfill.cpython-313-pytest-9.0.3.pyc  
tests/\_\_pycache\_\_/test\_bootstrap.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/test\_bootstrap.cpython-313-pytest-9.0.3.pyc  
tests/\_\_pycache\_\_/test\_commands.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/test\_config.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/test\_config.cpython-313-pytest-9.0.3.pyc  
tests/\_\_pycache\_\_/test\_db.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/test\_db.cpython-313-pytest-9.0.3.pyc  
tests/\_\_pycache\_\_/test\_delivery.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/test\_delivery.cpython-313-pytest-9.0.3.pyc  
tests/\_\_pycache\_\_/test\_filters\_content\_type.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/test\_filters\_duration.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/test\_filters\_duration.cpython-313-pytest-9.0.3.pyc  
tests/\_\_pycache\_\_/test\_filters\_resolution.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/test\_filters\_resolution.cpython-313-pytest-9.0.3.pyc  
tests/\_\_pycache\_\_/test\_filters\_size.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/test\_filters\_size.cpython-313-pytest-9.0.3.pyc  
tests/\_\_pycache\_\_/test\_filter\_engine.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/test\_filter\_engine.cpython-313-pytest-9.0.3.pyc  
tests/\_\_pycache\_\_/test\_main.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/test\_main.cpython-313-pytest-9.0.3.pyc  
tests/\_\_pycache\_\_/test\_manager.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/test\_manager.cpython-313-pytest-9.0.3.pyc  
tests/\_\_pycache\_\_/test\_telethon\_extraction.cpython-313-pytest-8.4.2.pyc  
tests/\_\_pycache\_\_/test\_telethon\_extraction.cpython-313-pytest-9.0.3.pyc  
docs/adr/0001-userbot-over-botapi.md  
docs/adr/0002-sqlite-over-postgres.md  
docs/adr/0003-metadata-first-probing.md  
docs/adr/0004-pluggable-delivery-modes.md  
docs/adr/0005-per-user-delivery-target.md  
docs/adr/0006-content-dedup.md  
docs/adr/0007-inline-config-commands.md  
docs/adr/0008-bot-api-onboarding-front-door.md  
docs/adr/0009-selective-channel-scanning.md

```
docs/adr/0010-identity-and-session-separation.md
docs/adr/0011-provider-neutral-domain-schema.md
docs/design.md
docs/plan.md
docs/runbook.md
```

```
===pyproject===
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "tg-video-filter"
version = "0.1.0"
description = "Telethon userbot that filters Telegram channel videos by duration, resolution,
and size, and forwards matches to Saved Messages."
readme = "README.md"
requires-python = ">=3.11"
license = { text = "TBD" }
authors = [{ name = "avidullu" }]
keywords = ["telegram", "telethon", "userbot", "video", "filter"]
classifiers = [
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
    "Operating System :: OS Independent",
    "Private :: Do Not Upload",
]
```

## Runtime dependencies. Pinned to compatible ranges; bump deliberately.

```
dependencies = [
    "telethon>=1.36,<2.0",
    "aiosqlite>=0.20,<1.0",
    "pydantic>=2.7,<3.0",
    "pydantic-settings>=2.3,<3.0",
    "python-dotenv>=1.0,<2.0",
    # Native crypto acceleration for Telethon (optional but strongly recommended;
    # without it Telethon falls back to slower pure-Python encryption).
    "cryptg>=0.4,<1.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.2,<9.0",
    "pytest-asyncio>=0.23,<1.0",
    "pytest-cov>=5.0,<7.0",
    "ruff>=0.5,<0.7",
    "pre-commit>=3.7,<5.0",
]
```

```
[project.urls]
Repository = "https://github.com/avidullu/telegram-video-filter"
Documentation = "https://github.com/avidullu/telegram-video-filter/blob/main/docs/design.md"
```

## Tell hatchling where the package lives (src layout).

```
[tool.hatch.build.targets.wheel]
packages = ["src/tg_video_filter"]
```

## --- pytest -----

```
[tool.pytest.ini_options]
minversion = "8.0"
testpaths = ["tests"]
asyncio_mode = "auto"
```

```

addopts = [
    "-ra",
    "--strict-markers",
    "--strict-config",
    "--cov=tg_video_filter",
    "--cov-report=term-missing",
]
filterwarnings = [
    "error",
    # Telethon/aiosqlite may emit benign DeprecationWarnings; surface but don't fail.
    "default::DeprecationWarning:telethon.*",
]

```

### --- ruff (lint + format) -----

```

[tool.ruff]
target-version = "py311"
line-length = 100
src = ["src", "tests"]
extend-exclude = ["data"]

[tool.ruff.lint]
select = [
    "E",      # pycodestyle errors
    "W",      # pycodestyle warnings
    "F",      # pyflakes
    "I",      # isort
    "B",      # flake8-bugbear
    "UP",     # pyupgrade
    "SIM",    # flake8-simplify
    "RUF",    # ruff-specific
]
ignore = [
    "E501",   # line length handled by formatter
]

[tool.ruff.lint.per-file-ignores]
"tests/**" = ["B017"] # allow pytest.raises(Exception) patterns in tests
"src/tg_video_filter/backfill.py" = ["RUF006"] # asyncio.create_task stored in named var; GC
keeps ref

[tool.ruff.format]
docstring-code-format = true

```

### --- coverage floor -----

```

[tool.coverage.run]
branch = true
source = ["src/tg_video_filter"]

[tool.coverage.report]
exclude_lines = [
    "pragma: no cover",
    "raise NotImplementedError",
    "if TYPE_CHECKING:",
    "if __name__ == '__main__':",
]

```

**Final gate: 80%. Interactive/Telethon-glue entrypoints are marked  
`pragma: no cover` (they require a live Telegram connection or stdin).**

fail\_under = 80

**16. assistant (2026-07-06T05:55:56.442Z)**

Substantial PR: **\*\*+1299/?150 across 4 files\*\*** (`db.py`, `models.py`, `test\_db.py`,



`test\_delivery.py`), implementing ADR 0011's provider-neutral schema. Note `statusCheckRollup` is empty ? **\*\*no CI on GitHub\*\***, so I must verify tests/lint/coverage locally per the pre-LGTM gate. Let me gather the diff, ADRs, and CI config.

## 17. user (2026-07-06T05:56:04.578Z)

Exit code 2

1767 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr35.diff

---CI workflows---

ls: cannot access '.github/workflows/': No such file or directory

## 18. user (2026-07-06T05:56:04.616Z)

```
1      # ADR 0011: Provider-neutral domain schema (Track B)
2
3      - **Status**: Accepted (Question 1 ? Option 1A; Question 2 ? Option 2A; PR #34)
4      - **Date**: 2026-07-06
5      - **Decision owner**: project owner
6      - **Tracking issue**: #27 (Track B of the provider-neutral umbrella, #25)
7      - **Depends on**: ADR 0010 (Track A ? `accounts` + `telegram_sessions`, merged in #33)
8      - **Blocks**: #28 (Track C ? delivery/routing), #29 (Track D ? commands), and indirectly
#30 (Track E)
9
10     ## Context
11
12     ADR 0010 split `users` into `accounts` (product identity) + `telegram_sessions`
13     (MTPProto credential). That unblocked the *identity* dimension. This ADR
14     tackles the **content dimension**: how to represent sources, content items,
15     policies, destinations, routes, and delivery attempts in a provider-neutral
16     way, so that the system can ingest from Telegram today and from WhatsApp
17     (or any future provider) later without another schema rewrite.
18
19     ### What we have today (post-Track-A)
20
21     | Table | Meaning | Provider-coupling |
22     |---|---|---|
23     | `accounts` | Product identity (ADR 0010) | provider-neutral (`provider` column) |
24     | `telegram_sessions` | Optional MTPProto session (ADR 0010) | Telegram-specific by
design |
25     | `channels` | A Telegram channel/chat an account is a member of | **Telegram-specific**
(`channel_id` is a Telegram id) |
26     | `videos` | A seen Telegram video message + metadata | **Telegram-specific**
(`message_id`, `document_id` are Telegram ids) |
27     | `filter_config` | Per-account JSON blob of `FilterConfig` | provider-neutral in shape,
but scoped per-account not per-source |
28
29     The dedup logic (ADR 0006) is **3-tier and Telegram-specific**:
30     - Tier 0: `(user_id, channel_id, message_id)` ? same Telegram message.
31     - Tier 1: `(user_id, document_id)` ? same Telegram file forwarded across channels.
32     - Tier 2: `(user_id, size_bytes, duration_s, height, width)` ? re-upload fingerprint.
33
34     ### What we need
35
36     The umbrella (#25) introduces six concepts: `Source`, `ContentItem`, `Policy`,
37     `Destination`, `Route` (`routing_rules`), and `DeliveryAttempt`. This ADR
38     defines the schema for each, the **two open design questions** that gate the
39     schema, and the migration path from the current tables.
40
41     ## The two decisions I need from you
42
43     ### Question 1: `ContentItem.dedup_key` ? how does provider-neutral dedup relate to ADR
0006?
44
45     The current 3-tier dedup is Telegram-specific (`document_id` is a Telegram
```

```

46 concept). A provider-neutral `ContentItem` needs a dedup story that works
47 across providers. Three options:
48
49 ##### Option 1A (recommended): tiered lookup that wraps ADR 0006's tiers
50
51 Keep ADR 0006's three tiers but generalize them behind a single
52 `dedup_key` column on `content_items`:
53
54 ```
55 content_items.dedup_key TEXT NOT NULL
56 ```
57
58 The *value* of `dedup_key` is computed by an explicit decision tree at
59 ingestion time, provider by provider. The tree is ordered strongest-key
60 first and each branch is guarded so partial metadata can never collapse
61 to a degenerate key* (e.g. `tg:fp:0:0:0:0`):
62
63 - Telegram:
64   1. If `document_id` is present ? `dedup_key = "tg:doc:{document_id}"`.
65   2. Else if all four fingerprint fields (`size_bytes`, `duration_s`,
66      `height`, `width`) are present ? `dedup_key = "tg:fp:{size}:{dur}:{h}:{w}"`.
67   3. Else ? `dedup_key = "tg:msg:{source_id}:{provider_message_id}"`
68      (Tier 0; the source/channel id is required because Telegram
69      `message_id` is only unique within a channel, not across sources).
70 - Future WhatsApp: `dedup_key = "wa:msg:{message_id}"` (WhatsApp message
71    ids are globally unique per account) or `wa:sha256:{blob_hash}"` if WA
72    exposes a content hash.
73
74 The guard in step 2 (all four fields required) is what prevents the
75 partial-metadata collision the first draft of this ADR allowed. The pure
76 `compute_dedup_key(provider, *, document_id, source_id, provider_message_id,
77 size_bytes, duration_s, height, width)` function that implements this tree
78 gets its own unit tests in the implementation PR, including cases for
79 partial/null metadata.
80
81 A partial unique index on `(account_id, dedup_key) WHERE dedup_key IS NOT NULL`
82 replaces the current Tier-1 index and enforces per-account dedup at the DB
83 level, regardless of provider.
84
85 Why recommended:
86 - Zero new bandwidth ? all keys are computed from metadata already extracted
87   (honors ADR 0003's no-download principle).
88 - The provider prefix (`tg:` / `wa:`) makes cross-provider collisions
89   structurally impossible.
90 - Existing `videos` rows migrate cleanly: `dedup_key` is computed from their
91   existing `document_id` / metadata columns during the migration.
92 - Supersedes ADR 0006 cleanly: the three tiers become one column with
93   provider-specific derivation rules, documented in the migration.
94
95 Costs:
96 - The migration must compute `dedup_key` for every existing `videos` row
97   (cheap ? pure SQL string building, no I/O).
98 - The `dedup_key` derivation logic lives in code (a pure function per
99   provider), not in the schema. This is a feature, but it means the
100  derivation rules need their own unit tests.
101
102 ##### Option 1B: content hash (SHA-256 of downloaded bytes)
103
104 `dedup_key = sha256(downloaded_content)`. Cryptographically strong,
105 provider-agnostic, catches re-encodes that 1A misses.
106
107 Rejected ? violates ADR 0003 (metadata-first, no download in the hot
108 path). Downloading every incoming video to hash it would multiply bandwidth
109 and latency by orders of magnitude. Could be added later as an optional
110 deep-inspect stage (deferred to Track F per #31).

```

```

111
112 ##### Option 1C: provider-assigned only (drop Tier 2)
113
114 `dedup_key` = "tg:doc:{document_id}"` always; no metadata fingerprint
115 fallback. Simpler, but loses Tier-2 dedup for re-uploads that get a new
116 `document.id` ? a real and common case (someone re-encodes and re-uploads
117 the same video to a different channel).
118
119 **Recommendation: 1A.** It preserves the dedup quality of ADR 0006, generalizes cleanly,
120 and honors the no-download constraint.
121 ---
122
123 ### Question 2: `Policy` ? existing `filter_config` ? rename, or wrap?
124
125 The codebase already has `filter_config.config_json` (one row per account,
126 holding a `FilterConfig` JSON blob ? see `models.py` and ADR 0005). The
127 umbrella introduces a `Policy` concept. They overlap. Two options:
128
129 ##### Option 2A (recommended): `Policy` *generalizes* `filter_config` (rename + extend)
130
131 `policies` is a new table; `filter_config` is migrated into it. One account
132 can own multiple policies (enables per-route filter overrides from ADR
133 0009), with a `DEFAULT` policy` per account for the no-rules-configured case.
134
135 ```
136 policies (
137     id                INTEGER PRIMARY KEY AUTOINCREMENT,
138     account_id        INTEGER NOT NULL REFERENCES accounts(id),
139     name              TEXT NOT NULL,                -- e.g. "default", "reggae-route"
140     config_json       TEXT NOT NULL,                -- the FilterConfig blob (Phase 1)
141     is_default        INTEGER NOT NULL DEFAULT 0,    -- one per account (UNIQUE partial
idx)
142     created_at       TEXT NOT NULL,
143     updated_at       TEXT NOT NULL,
144     UNIQUE (account_id, name)
145 );
146 ```
147
148 **Phase 1** (this track): `config_json` holds exactly today's `FilterConfig`
149 JSON ? the rename is purely structural. **Phase 2** (Track F, deferred):
150 `config_json` grows to include topic preferences and security rules, but
151 that's behind the same column so no further migration.
152
153 Migration: for each `filter_config` row, insert a `policies` row with
154 `name='default'`, `is_default=1`, `config_json=<the existing blob>`. Drop
155 `filter_config` after the migration (keep `filter_config_deprecated` for one
156 release, mirroring ADR 0010's safety-net pattern).
157
158 **Why recommended:**
159 - `Policy` becomes the single home for "what to filter/enrich" ? no second
160 overlapping concept. Today's `FilterConfig` is just the Phase-1 shape.
161 - Enables ADR 0009's per-route filter overrides (`routing_rules.policy_id`)
162 without a second migration later.
163 - The existing `db.load_filter_config(account_id)` / `save_filter_config`
164 become thin shims over `policies WHERE account_id=? AND is_default=1`,
165 mirroring the Track-A `User` shim pattern ? backwards-compatible.
166
167 **Costs:**
168 - One more table migration. `filter_config` ? `policies` is a rename+extend.
169 - The `FilterConfig` Pydantic model stays (it's the Phase-1 config shape);
170 `Policy` is the *container* (name, owner, default flag) around it. Two
171 models, not one ? but they compose cleanly.
172
173 ##### Option 2B: keep `filter_config` as-is; `Policy` is a wrapping layer

```

```

174
175 Don't touch `filter_config`. `policies` references a `filter_config` row
176 plus adds enrichment/security stages as separate columns.
177
178 **Rejected** ? creates two overlapping concepts (`filter_config` for the
179 v1 rules, `policies` for "everything else"), which is exactly the
180 conflation the umbrella exists to dissolve. We'd be back here in a year
181 merging them.
182
183 **Recommendation: 2A.** One concept, one table, phased growth via the same JSON column.
184
185 ---
186
187 ## Proposed schema (assuming 1A + 2A)
188
189 ### `sources` (generalizes `channels`)
190
191 A configured place we ingest from. Maps 1:1 to today's per-account
192 `channels` rows for Telegram; future providers add their own source kinds.
193
194 ```
195 sources (
196     id                INTEGER PRIMARY KEY AUTOINCREMENT,
197     account_id        INTEGER NOT NULL REFERENCES accounts(id),
198     provider          TEXT NOT NULL DEFAULT 'telegram',
199     provider_source_id TEXT NOT NULL,          -- Telegram channel_id; future: WA group id
200     title             TEXT,
201     kind              TEXT NOT NULL DEFAULT 'channel', -- 'channel' | 'group' | 'feed'
202     watched           INTEGER NOT NULL DEFAULT 0,      -- ADR 0009 selective scanning
203     seen_first_at     TEXT NOT NULL,
204     last_seen_at      TEXT,
205     UNIQUE (account_id, provider, provider_source_id)
206 );
207 ```
208
209 `channels.user_id` (now ? `accounts.id`) maps to `sources.account_id`;
210 `channels.channel_id` maps to `sources.provider_source_id`. The `watched`
211 flag (ADR 0009, deferred to Track E) is included here so Track E doesn't
212 need another migration.
213
214 ### `content_items` (generalizes `videos`)
215
216 A normalized item discovered from any source. Phase 1 stores only videos
217 (matches today's `videos`); the columns are provider-neutral but the
218 Phase-1 migration only populates video fields.
219
220 ```
221 content_items (
222     id                INTEGER PRIMARY KEY AUTOINCREMENT,
223     account_id        INTEGER NOT NULL REFERENCES accounts(id),
224     source_id         INTEGER NOT NULL REFERENCES sources(id),
225     provider          TEXT NOT NULL DEFAULT 'telegram',
226     provider_message_id TEXT NOT NULL,          -- Telegram message_id; future: WA message
227     content_type       TEXT NOT NULL DEFAULT 'video', -- 'video' | 'link' | 'article' |
228     ...
229     text              TEXT,                    -- caption / post text (nullable)
230     link_url          TEXT,                    -- for link/article posts
231     media_ref         TEXT,                    -- provider media reference (file path,
232     blob id)
233     document_id       TEXT,                    -- Telegram document.id (kept for TG-
234     specific lookups)
235     duration_s        INTEGER,
236     width             INTEGER,
237     height            INTEGER,

```

```

235     size_bytes      INTEGER,
236     mime_type       TEXT,
237     dedup_key        TEXT NOT NULL,          -- ADR 0011 Question 1 (Option 1A)
238     matched          INTEGER NOT NULL DEFAULT 0,
239     seen_at          TEXT NOT NULL,
240     -- Tier-0 equivalent: provider_message_id is only unique within a source
241     -- (a Telegram message_id is per-channel), so source_id is required to
242     -- avoid false collisions across sources for the same account.
243     UNIQUE (account_id, source_id, provider_message_id),
244     UNIQUE (account_id, dedup_key)           -- Tier-1/2 equivalent (Option
1A)
245 );
246 ```
247
248 **Note on `forwarded`:** dropped from `content_items`. Per-destination
249 delivery state moves to `delivery_attempts` (Track C, #28). For Phase 1
250 (before Track C lands), `content_items.matched` alone is sufficient ?
251 delivery still goes to the single `delivery_target`, and the boolean
252 `forwarded` is retained temporarily on a *deprecated* `videos`-compat view
253 so the v1 delivery path keeps working. Track C removes it.
254
255 ### `policies` (generalizes `filter_config`) ? Option 2A
256
257 As specified in Question 2 above.
258
259 ### `destinations` (new ? Track C will populate)
260
261 Defined here so the schema is complete, but **Track C (#28) implements the
262 delivery layer**. Phase-1 presence is the table only (no delivery logic).
263
264 ```
265 destinations (
266     id            INTEGER PRIMARY KEY AUTOINCREMENT,
267     account_id    INTEGER NOT NULL REFERENCES accounts(id),
268     provider      TEXT NOT NULL DEFAULT 'telegram',
269     target        TEXT NOT NULL,          -- 'me', '@username', chat id (ADR 0005)
270     kind          TEXT NOT NULL DEFAULT 'saved', -- 'saved' | 'channel' | 'chat'
271     title         TEXT,
272     is_output     INTEGER NOT NULL DEFAULT 0,    -- ADR 0009: distinguishes output
channels
273     created_at    TEXT NOT NULL,
274     UNIQUE (account_id, provider, target)
275 );
276 ```
277
278 ### `routing_rules` (new ? Track C will populate)
279
280 Per ADR 0009 (canonical name locked in issue #25).
281
282 ```
283 routing_rules (
284     id            INTEGER PRIMARY KEY AUTOINCREMENT,
285     account_id    INTEGER NOT NULL REFERENCES accounts(id),
286     source_id     INTEGER NOT NULL REFERENCES sources(id),
287     policy_id     INTEGER,                -- nullable ? use account's default policy
288     destination_id INTEGER NOT NULL REFERENCES destinations(id),
289     enabled       INTEGER NOT NULL DEFAULT 1,
290     priority      INTEGER NOT NULL DEFAULT 0,
291     created_at    TEXT NOT NULL,
292     UNIQUE (account_id, source_id, destination_id)
293 );
294 ```
295
296 ### `delivery_attempts` (new ? Track C will populate)
297

```

```

298 Per #28's idempotency requirement (unique index on
299 `(content_item_id, destination_id)`).
300
301 ```
302 delivery_attempts (
303     id                INTEGER PRIMARY KEY AUTOINCREMENT,
304     content_item_id   INTEGER NOT NULL REFERENCES content_items(id),
305     destination_id    INTEGER NOT NULL REFERENCES destinations(id),
306     route_id          INTEGER REFERENCES routing_rules(id),
307     status             TEXT NOT NULL DEFAULT 'pending', --
'pending' || 'delivered' || 'failed' || 'retrying'
308     attempt_count     INTEGER NOT NULL DEFAULT 0,
309     last_error         TEXT,
310     created_at        TEXT NOT NULL,
311     delivered_at      TEXT,
312     next_retry_at     TEXT,
313     UNIQUE (content_item_id, destination_id)           -- ADR #28 idempotency
314 );
315 ```
316
317 ## Migration strategy (integer-preserving, phased)
318
319 This track lands sources, content_items, and policies only.
320 destinations, routing_rules, delivery_attempts are defined in the
321 schema (so the ADR is complete) but their tables are created empty and
322 populated by Track C. This keeps Track B focused and reviewable.
323
324 ### Migration _migration_003_domain_schema (idempotent)
325
326 1. Create sources, content_items, policies, destinations,
327 routing_rules, delivery_attempts (all IF NOT EXISTS).
328 2. Backfill sources from channels:
329 provider='telegram', provider_source_id=channel_id, watched=1
330 (ADR 0009 backfill: already-seen channels stay watched).
331 3. Backfill content_items from videos, computing dedup_key via the
332 explicit decision tree from Option 1A (every branch reachable, no
333 COALESCE(...,0) collapsing partial metadata to a degenerate key):
334 - If document_id IS NOT NULL:
335     dedup_key = 'tg:doc:' || document_id.
336 - Else if size_bytes IS NOT NULL AND duration_s IS NOT NULL AND height IS NOT NULL
AND width IS NOT NULL:
337     dedup_key = 'tg:fp:' || size_bytes || ':' || duration_s || ':' || height || ':' ||
width.
338 - Else (Tier-0 fallback; partial or missing media metadata):
339     dedup_key = 'tg:msg:' || source_id || ':' || provider_message_id,
340     where source_id is the joined sources.id for the row's legacy
341     channel_id and provider_message_id is the legacy message_id.
342
343 The implementation PR must include migration tests that seed rows hitting
344 each branch (including all-fingerprint-fields-null) and assert no two
345 rows collapse to the same dedup_key.
346 4. Backfill policies from filter_config: one row per account,
347 name='default', is_default=1, config_json copied verbatim.
348 5. Rename (don't drop) channels ? channels_deprecated,
349 videos ? videos_deprecated, filter_config ? filter_config_deprecated.
350 Kept for one release as a rollback safety net (ADR 0010 pattern).
351
352 The legacy videos/channels/filter_config tables and their
353 db.load_filter_config / upsert_channel / insert_video repos stay
354 working as shims over the new tables (same pattern as the Track-A User
355 shim), so existing call sites in telethon_client.py, backfill.py, and
356 commands.py keep working without a flag day.
357
358 ## Consequences
359

```

```

360  **Positive**
361  - Provider-neutral schema: `sources`, `content_items`, `policies`,
362    `destinations`, `routing_rules`, `delivery_attempts` all carry a
363    `provider` column or provider-prefixed keys. Adding WhatsApp later is a
364    new `provider='whatsapp'` value, not a new schema.
365  - `dedup_key` (Option 1A) generalizes ADR 0006 without losing any dedup
366    quality and without downloading anything.
367  - `Policy` (Option 2A) gives a single home for filter + future enrichment
368    config, enabling ADR 0009's per-route overrides with no second migration.
369  - Backwards-compatible via shims, mirroring the Track-A pattern that
370    already passed review.
371
372  **Negative**
373  - Six new tables; the schema grows significantly. Each is small and
374    single-purpose, but the surface area is larger.
375  - The dedup-key derivation logic is in code, not the schema ? needs its
376    own unit tests (a `compute_dedup_key(provider, **metadata)` pure function).
377  - Phase 1 carries some dead-on-arrival columns (`content_items.link_url`,
378    `text`) that aren't populated until later tracks. This is intentional
379    (avoid future migrations) but means some columns are NULL until then.
380
381  ## Alternatives considered
382
383  - **Do nothing; keep `channels`/`videos`/`filter_config` and special-case
384    providers in code.** Rejected ? same conflation argument as ADR 0010.
385    Telegram-specific ids in column names (`channel_id`, `message_id`,
386    `document_id`) would leak into every future provider's code path.
387  - **Land all six tables' *logic* in this track.** Rejected ? Track B is
388    already the largest track. Splitting schema (Track B) from delivery
389    logic (Track C) keeps each PR reviewable. The empty `destinations` /
390    `routing_rules` / `delivery_attempts` tables in this track are the
391    contract Track C codes against.
392  - **`content_items` as a view over `videos` (no migration).** Rejected ?
393    views don't give us the `provider` column or the `dedup_key` index, and
394    SQLite view support for writes is limited. A real table is needed.
395
396  ## Open questions for the decision owner
397
398  1. **Question 1** ? `dedup_key`: 1A (tiered, recommended), 1B (hash,
399    rejected), or 1C (provider-assigned only).
400  2. **Question 2** ? `Policy` ? `filter_config`: 2A (generalize, recommended)
401    or 2B (wrap, rejected).
402  3. **Dead columns in Phase 1** ? acceptable to land `content_items.link_url`
403    / `text` as NULL until later tracks populate them? Recommendation: yes
404    (avoids a future `ALTER TABLE`).
405  4. **Deprecated-table retention** ? one release (default), matching ADR 0010?
406    Recommendation: yes.
407
408  ## References
409
410  - ADR 0003 ? metadata-first probing (the no-download constraint that rules
411    out Option 1B's content hashing).
412  - ADR 0005 ? per-user delivery target (generalized by `destinations` /
413    `routing_rules`).
414  - ADR 0006 ? content dedup (generalized by `content_items.dedup_key`,
415    Option 1A).
416  - ADR 0009 ? selective scanning + multiplexed routing (depends on
417    `sources.watched` and `routing_rules`, both defined here).
418  - ADR 0010 ? identity/session separation (this ADR's `account_id` FKs point
419    at `accounts.id`).
420  - Issue #27 ? Track B (this ADR's tracking issue).
421  - Issue #25 ? umbrella; defines the six concepts and the implementation order.
422

```

Excellent ? ADR 0011 gives me the full spec to review against. The diff wrote 1767 lines (the exit 2 was just the missing ``.github/workflows/``). Let me confirm CI config, then kick off the local verification gate in the background while I read the code.

## 20. user (2026-07-06T05:56:25.812Z)

---pre-commit---

**See <https://pre-commit.com> for hook docs.**

**Install once: pre-commit install**

**Run manually: pre-commit run --all-files**

repos:

- repo: <https://github.com/pre-commit/pre-commit-hooks>  
rev: v4.6.0  
hooks:
  - id: trailing-whitespace
  - id: end-of-file-fixer
  - id: check-yaml
  - id: check-toml
  - id: check-added-large-files  
args: ["--maxkb=512"]
  - id: check-merge-conflict
  - id: detect-private-key

- repo: <https://github.com/astral-sh/ruff-pre-commit>  
rev: v0.5.7  
hooks:
  - id: ruff  
args: [--fix, --exit-non-zero-on-fix]
  - id: ruff-format

---any CI anywhere---

ls: cannot access ``.github``: No such file or directory

---mypy config?---

## 21. user (2026-07-06T05:56:29.885Z)

`.venv/Scripts/python.exe`

Python 3.13.13

pytest+ruff import ok

---installed?---

pytest 8.4.2

ruff 0.6.9

## 22. assistant (2026-07-06T05:56:44.177Z)

No CI on GitHub and no mypy configured ? the gate here is `**pytest+coverage (floor 80%)*`, `**ruff check**`, `**ruff format --check**` (matching pre-commit). Tooling is present (pytest 8.4.2, ruff 0.6.9). Let me run the full verification gate in the background while I read the code.

## 23. user (2026-07-06T05:56:50.205Z)

Command running in background with ID: bzc5er02v. Output is being written to:

`C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bzc5er02v.output`. You will be notified when it completes. To check interim output, use Read on that file path.

## 24. user (2026-07-06T05:56:50.595Z)

```
1      """SQLite persistence layer (aiosqlite).
2
3      Schema, migrations, and repository functions. See ``docs/design.md`` §3 for
4      the legacy data model and ``docs/adr/0011-provider-neutral-domain-schema.md``
5      for the provider-neutral ``sources`` / ``content_items`` / ``policies`` /
```



```

6     ``destinations`` / ``routing_rules`` / ``delivery_attempts`` schema (ADR 0011).
7
8     The repository functions take an ``aiosqlite.Connection`` so tests can inject an
9     in-memory database (``aiosqlite.connect(":memory:")``).
10    """
11
12    from __future__ import annotations
13
14    from collections.abc import Awaitable, Callable, Iterable
15    from datetime import UTC, datetime
16
17    import aiosqlite
18
19    from tg_video_filter.models import (
20        Account,
21        Channel,
22        ContentItem,
23        FilterConfig,
24        Policy,
25        Source,
26        TelegramSession,
27        User,
28        VideoRecord,
29    )
30
31    # --- DDL -----
32
33    _SCHEMA = """
34    CREATE TABLE IF NOT EXISTS accounts (
35        id            INTEGER PRIMARY KEY,
36        telegram_id   INTEGER UNIQUE,
37        provider      TEXT NOT NULL DEFAULT 'telegram',
38        display_name  TEXT,
39        created_at    TEXT NOT NULL
40    );
41
42    CREATE TABLE IF NOT EXISTS telegram_sessions (
43        account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
44        phone         TEXT,
45        session_path  TEXT NOT NULL,
46        enabled       INTEGER NOT NULL DEFAULT 1,
47        created_at    TEXT NOT NULL,
48        last_login_at TEXT
49    );
50
51    CREATE TABLE IF NOT EXISTS channels (
52        id            INTEGER PRIMARY KEY AUTOINCREMENT,
53        user_id       INTEGER NOT NULL REFERENCES accounts(id),
54        channel_id    INTEGER NOT NULL,
55        title         TEXT,
56        seen_first_at TEXT NOT NULL,
57        UNIQUE (user_id, channel_id)
58    );
59
60    CREATE TABLE IF NOT EXISTS videos (
61        id            INTEGER PRIMARY KEY AUTOINCREMENT,
62        user_id       INTEGER NOT NULL REFERENCES accounts(id),
63        channel_id    INTEGER NOT NULL,
64        message_id    INTEGER NOT NULL,
65        document_id   INTEGER,
66        duration_s    INTEGER,
67        width         INTEGER,
68        height        INTEGER,
69        size_bytes    INTEGER,
70        mime_type     TEXT,

```

```

71         matched      INTEGER NOT NULL DEFAULT 0,
72         forwarded     INTEGER NOT NULL DEFAULT 0,
73         seen_at       TEXT NOT NULL,
74         UNIQUE (user_id, channel_id, message_id)
75     );
76
77     CREATE TABLE IF NOT EXISTS filter_config (
78         user_id        INTEGER PRIMARY KEY REFERENCES accounts(id),
79         config_json    TEXT NOT NULL
80     );
81
82     -- Provider-neutral domain tables (ADR 0011, Track B). ``sources``,
83     -- ``content_items``, and ``policies`` are populated by migration 003 (and by
84     -- the shims below); ``destinations`` / ``routing_rules`` / ``delivery_attempts``
85     -- are created empty here so the schema is complete ? Track C (#28) implements
86     -- the delivery logic that populates them.
87
88     CREATE TABLE IF NOT EXISTS sources (
89         id              INTEGER PRIMARY KEY AUTOINCREMENT,
90         account_id      INTEGER NOT NULL REFERENCES accounts(id),
91         provider        TEXT NOT NULL DEFAULT 'telegram',
92         provider_source_id TEXT NOT NULL,
93         title           TEXT,
94         kind            TEXT NOT NULL DEFAULT 'channel',
95         watched         INTEGER NOT NULL DEFAULT 0,
96         seen_first_at   TEXT NOT NULL,
97         last_seen_at    TEXT,
98         UNIQUE (account_id, provider, provider_source_id)
99     );
100
101     CREATE TABLE IF NOT EXISTS content_items (
102         id              INTEGER PRIMARY KEY AUTOINCREMENT,
103         account_id      INTEGER NOT NULL REFERENCES accounts(id),
104         source_id       INTEGER NOT NULL REFERENCES sources(id),
105         provider        TEXT NOT NULL DEFAULT 'telegram',
106         provider_message_id TEXT NOT NULL,
107         content_type    TEXT NOT NULL DEFAULT 'video',
108         text            TEXT,
109         link_url        TEXT,
110         media_ref       TEXT,
111         document_id     TEXT,
112         duration_s      INTEGER,
113         width           INTEGER,
114         height          INTEGER,
115         size_bytes      INTEGER,
116         mime_type       TEXT,
117         dedup_key       TEXT NOT NULL,
118         matched         INTEGER NOT NULL DEFAULT 0,
119         seen_at         TEXT NOT NULL,
120         UNIQUE (account_id, source_id, provider_message_id),
121         UNIQUE (account_id, dedup_key)
122     );
123
124     CREATE TABLE IF NOT EXISTS policies (
125         id              INTEGER PRIMARY KEY AUTOINCREMENT,
126         account_id      INTEGER NOT NULL REFERENCES accounts(id),
127         name            TEXT NOT NULL,
128         config_json     TEXT NOT NULL,
129         is_default      INTEGER NOT NULL DEFAULT 0,
130         created_at      TEXT NOT NULL,
131         updated_at      TEXT NOT NULL,
132         UNIQUE (account_id, name)
133     );
134
135     CREATE TABLE IF NOT EXISTS destinations (

```

```

136         id            INTEGER PRIMARY KEY AUTOINCREMENT,
137         account_id    INTEGER NOT NULL REFERENCES accounts(id),
138         provider      TEXT NOT NULL DEFAULT 'telegram',
139         target        TEXT NOT NULL,
140         kind          TEXT NOT NULL DEFAULT 'saved',
141         title         TEXT,
142         is_output     INTEGER NOT NULL DEFAULT 0,
143         created_at    TEXT NOT NULL,
144         UNIQUE (account_id, provider, target)
145     );
146
147     CREATE TABLE IF NOT EXISTS routing_rules (
148         id            INTEGER PRIMARY KEY AUTOINCREMENT,
149         account_id    INTEGER NOT NULL REFERENCES accounts(id),
150         source_id     INTEGER NOT NULL REFERENCES sources(id),
151         policy_id     INTEGER REFERENCES policies(id),
152         destination_id INTEGER NOT NULL REFERENCES destinations(id),
153         enabled       INTEGER NOT NULL DEFAULT 1,
154         priority      INTEGER NOT NULL DEFAULT 0,
155         created_at    TEXT NOT NULL,
156         UNIQUE (account_id, source_id, destination_id)
157     );
158
159     CREATE TABLE IF NOT EXISTS delivery_attempts (
160         id            INTEGER PRIMARY KEY AUTOINCREMENT,
161         content_item_id INTEGER NOT NULL REFERENCES content_items(id),
162         destination_id INTEGER NOT NULL REFERENCES destinations(id),
163         route_id      INTEGER REFERENCES routing_rules(id),
164         status        TEXT NOT NULL DEFAULT 'pending',
165         attempt_count INTEGER NOT NULL DEFAULT 0,
166         last_error    TEXT,
167         created_at    TEXT NOT NULL,
168         delivered_at  TEXT,
169         next_retry_at TEXT,
170         UNIQUE (content_item_id, destination_id)
171     );
172     """
173
174     # Note: the ``idx_videos_user_document`` partial unique index is created by
175     # migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
176     # which is present on fresh tables but only added to legacy v1 DBs by the
177     # migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
178     # DBs that don't yet have the column.
179
180
181     async def init_db(db_path: str) -> None:
182         """Open the database at ``db_path`` and create/migrate the schema.
183
184         Safe to call on every startup. Applies the base schema (``IF NOT EXISTS``)
185         then runs any pending migrations via the ``user_version`` ladder.
186         """
187         async with aiosqlite.connect(db_path) as conn:
188             await _apply_schema(conn)
189
190
191     async def _apply_schema(conn: aiosqlite.Connection) -> None:
192         # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
193         # and a busy timeout so concurrent access waits instead of failing immediately.
194         await conn.execute("PRAGMA journal_mode = WAL")
195         await conn.execute("PRAGMA foreign_keys = ON")
196         await conn.execute("PRAGMA busy_timeout = 5000")
197         await conn.executescript(_SCHEMA)
198         # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
199         # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
200         # migration 002's FK rebuild and migration 003's backfill on legacy DBs).

```

```

201     # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
202     # any *empty* recreations of those legacy tables from a subsequent
203     # ``init_db`` are dropped here so they don't shadow the renamed copies.
204     if await _table_exists(conn, "channels_deprecated"):
205         await conn.execute("DROP TABLE IF EXISTS channels")
206         await conn.execute("DROP TABLE IF EXISTS videos")
207         await conn.execute("DROP TABLE IF EXISTS filter_config")
208     await _apply_migrations(conn)
209     await conn.commit()
210
211
212     # --- migrations -----
213     #
214     # Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
215     # Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
216     # the source of truth for *fresh* databases; the migrations below reconcile
217     # *existing* databases that were created by an older schema version.
218     #
219     # To add a new migration:
220     #   1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
221     #   2. Implement ``async def _migration_NN(conn) -> None`` (must be idempotent).
222     #   3. Add a test in ``tests/test_db.py`` that seeds an old schema, runs
223     #       ``init_db``, and asserts the new structure.
224
225     _MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
226
227
228     def _migration(
229         version: int,
230     ) -> Callable[
231         [Callable[[aiosqlite.Connection], Awaitable[None]]],
232         Callable[[aiosqlite.Connection], Awaitable[None]],
233     ]:
234         """Register a migration that brings the DB up to ``user_version == version``."""
235
236         def decorator(
237             fn: Callable[[aiosqlite.Connection], Awaitable[None]],
238         ) -> Callable[[aiosqlite.Connection], Awaitable[None]]:
239             _MIGRATIONS.append((version, fn))
240             return fn
241
242         return decorator
243
244
245     # --- provider-neutral dedup key (ADR 0011, Option 1A) -----
246     #
247     # Pure function: computes the ``content_items.dedup_key`` value from metadata
248     # already extracted at ingestion time (no download ? honors ADR 0003). The
249     # decision tree is ordered strongest-key first; each branch is guarded so
250     # partial metadata can never collapse to a degenerate key (e.g. ``tg:fp:0:0:0:0``).
251     # See ``compute_dedup_key_test`` in ``tests/test_db.py`` for the full matrix.
252
253
254     def compute_dedup_key(
255         *,
256         provider: str,
257         document_id: str | None = None,
258         source_id: int | str | None = None,
259         provider_message_id: str | None = None,
260         size_bytes: int | None = None,
261         duration_s: int | None = None,
262         height: int | None = None,
263         width: int | None = None,
264     ) -> str:
265         """Compute the provider-neutral ``dedup_key`` for a ``ContentItem``.

```

```

266
267 Implements the ADR 0011 Option 1A decision tree:
268
269 - **Telegram** (`provider == "telegram"`):
270     1. If `document_id` is set ? ``"tg:doc:{document_id}"``.
271     2. Else if **all four** fingerprint fields (size, duration, height, width)
272        are set ? ``"tg:fp:{size}:{dur}:{h}:{w}"``.
273     3. Else ? ``"tg:msg:{source_id}:{provider_message_id}"`` (Tier-0 fallback;
274        `source_id` is required because Telegram `message_id` is only
275        unique within a channel).
276 - **Other providers** (`provider != "telegram"`): fall back to the Tier-0
277    ``{provider}:msg:{source_id}:{provider_message_id}"`` shape. Future
278    providers add their own branches (e.g. ``"wa:sha256:{blob_hash}"``) here.
279
280 Raises ``ValueError`` if the fallback branch is reached without the fields
281 it requires (`source_id` + `provider_message_id`) ? a degenerate
282 `dedup_key` is worse than a loud failure at ingestion time.
283 """
284 p = provider.lower()
285 if p == "telegram":
286     if document_id is not None and document_id != "":
287         return f"tg:doc:{document_id}"
288     # All four fingerprint fields are required to avoid a degenerate
289     # `tg:fp:0:0:0:0` key when some metadata is missing.
290     if None not in (size_bytes, duration_s, height, width):
291         return f"tg:fp:{size_bytes}:{duration_s}:{height}:{width}"
292     # Tier-0 fallback.
293     if source_id is None or provider_message_id is None:
294         raise ValueError(
295             "Telegram dedup_key fallback requires source_id and provider_message_id"
296         )
297     return f"tg:msg:{source_id}:{provider_message_id}"
298 # Future-proof default for providers without a dedicated branch.
299 if source_id is None or provider_message_id is None:
300     raise ValueError(
301         f"dedup_key for provider={provider!r} requires source_id and
302 provider_message_id"
303     )
304 return f"{p}:msg:{source_id}:{provider_message_id}"
305
306
307 async def _apply_migrations(conn: aiosqlite.Connection) -> None:
308     """Run every registered migration whose version > current `user_version`."""
309     async with conn.execute("PRAGMA user_version") as cur:
310         row = await cur.fetchone()
311         current = int(row[0]) if row else 0
312
313     for version, fn in _MIGRATIONS:
314         if version <= current:
315             continue
316         await fn(conn)
317         await conn.execute(f"PRAGMA user_version = {version}")
318
319
320 async def _column_exists(conn: aiosqlite.Connection, table: str, column: str) -> bool:
321     """Return True if `column` exists on `table` (async, PRAGMA table_info)."""
322     async with conn.execute(f"PRAGMA table_info({table})") as cur:
323         rows = await cur.fetchall()
324         return any(row[1] == column for row in rows)
325
326
327 @_migration(1)
328 async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:

```

```

329     """Add ``document_id`` column + partial unique index (ADR 0006 content dedup).
330
331     Idempotent: safe on fresh DBs (column/index already present) and on v1 DBs
332     created before this migration existed.
333     """
334     if not await _column_exists(conn, "videos", "document_id"):
335         await conn.execute("ALTER TABLE videos ADD COLUMN document_id INTEGER")
336     # Create the partial unique index for both fresh and migrated DBs. Uses
337     # IF NOT EXISTS so it is safe on DBs where it was already created.
338     await conn.execute(
339         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
340         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
341     )
342
343
344 @migration(2)
345 async def _migration_002_identity_split(conn: aiosqlite.Connection) -> None:
346     """Split the legacy ``users`` table into ``accounts`` + ``telegram_sessions``.
347
348     ADR 0010. Integer-preserving: ``accounts.id`` copies ``users.id`` and the
349     data in ``channels`` / ``videos`` / ``filter_config`` is copied verbatim,
350     so their integer ``user_id`` values still resolve to the same identity (now
351     via ``accounts.id``). The legacy ``users`` table is renamed to
352     ``users_deprecated`` and kept for one release as a rollback safety net; a
353     follow-up migration drops it.
354
355     FK contract: ``channels`` / ``videos`` / ``filter_config`` are rebuilt with
356     ``REFERENCES accounts(id)`` so migrated DBs end up with the *same* FK shape
357     as fresh DBs. (A naive rename of ``users`` would leave SQLite rewriting
358     those FKs to point at ``users_deprecated``, diverging from fresh installs
359     and breaking inserts for new accounts ? see PR #33 review.)
360
361     Idempotent: a no-op once the legacy ``users`` table has been renamed.
362     """
363     # If the legacy users table is gone, this migration has already run
364     # (or the DB is fresh and only has accounts). Nothing to do.
365     if not await _table_exists(conn, "users"):
366         return
367
368     # Build accounts from the legacy users table. accounts.id := users.id.
369     # (The empty accounts table created by _SCHEMA on a legacy DB is dropped
370     # and recreated here so the INSERT can populate it cleanly.)
371     await conn.execute("DROP TABLE IF EXISTS accounts")
372     await conn.execute(
373         """
374         CREATE TABLE accounts (
375             id            INTEGER PRIMARY KEY,
376             telegram_id   INTEGER UNIQUE,
377             provider      TEXT NOT NULL DEFAULT 'telegram',
378             display_name  TEXT,
379             created_at    TEXT NOT NULL
380         )
381         """
382     )
383     await conn.execute(
384         """
385         INSERT INTO accounts (id, telegram_id, provider, created_at)
386         SELECT id, id, 'telegram', created_at FROM users
387         """
388     )
389
390     # Build telegram_sessions, one row per legacy userbot user.
391     # (Same drop-and-recreate as accounts: _SCHEMA created an empty copy.)
392     await conn.execute("DROP TABLE IF EXISTS telegram_sessions")
393     await conn.execute(

```

```

394         """
395         CREATE TABLE telegram_sessions (
396             account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
397             phone         TEXT,
398             session_path  TEXT NOT NULL,
399             enabled       INTEGER NOT NULL DEFAULT 1,
400             created_at    TEXT NOT NULL,
401             last_login_at TEXT
402         )
403         """
404     )
405     # session_path is built from the *telegram_id* (= old users.id) to match
406     # the convention telethon_client.build_client used pre-migration. The
407     # sessions_dir prefix is prepended at read time (Settings.sessions_dir)
408     # so this migration does not need to know the runtime path.
409     await conn.execute(
410         """
411         INSERT INTO telegram_sessions (account_id, phone, session_path, enabled,
created_at)
412         SELECT id, phone, 'user_' || id || '.session', enabled, created_at
413         FROM users
414         """
415     )
416
417     # Rebuild channels / videos / filter_config so their FK constraints point
418     # at accounts(id) instead of the legacy users(id). SQLite's
419     # ALTER TABLE RENAME rewrites FK references in *other* tables to follow
420     # the rename ? so a naive rename of users ? users_deprecated would leave
421     # these tables referencing users_deprecated(id), diverging from fresh DBs
422     # (which reference accounts(id)) and breaking inserts for new accounts.
423     # The rebuild: rename the data-holding legacy table out of the way, create
424     # the new one with the correct FK, copy the data back, drop the renamed copy.
425     for table, new_ddl, copy_cols in (
426         (
427             "channels",
428             """
429             CREATE TABLE channels (
430                 id            INTEGER PRIMARY KEY AUTOINCREMENT,
431                 user_id       INTEGER NOT NULL REFERENCES accounts(id),
432                 channel_id    INTEGER NOT NULL,
433                 title         TEXT,
434                 seen_first_at TEXT NOT NULL,
435                 UNIQUE (user_id, channel_id)
436             )
437             """,
438             "id, user_id, channel_id, title, seen_first_at",
439         ),
440         (
441             "videos",
442             """
443             CREATE TABLE videos (
444                 id            INTEGER PRIMARY KEY AUTOINCREMENT,
445                 user_id       INTEGER NOT NULL REFERENCES accounts(id),
446                 channel_id    INTEGER NOT NULL,
447                 message_id    INTEGER NOT NULL,
448                 document_id   INTEGER,
449                 duration_s    INTEGER,
450                 width         INTEGER,
451                 height        INTEGER,
452                 size_bytes    INTEGER,
453                 mime_type     TEXT,
454                 matched       INTEGER NOT NULL DEFAULT 0,
455                 forwarded     INTEGER NOT NULL DEFAULT 0,
456                 seen_at       TEXT NOT NULL,
457                 UNIQUE (user_id, channel_id, message_id)

```

```

458         )
459         """
460         "id, user_id, channel_id, message_id, document_id, duration_s, width, "
461         "height, size_bytes, mime_type, matched, forwarded, seen_at",
462     ),
463     (
464         "filter_config",
465         """
466         CREATE TABLE filter_config (
467             user_id      INTEGER PRIMARY KEY REFERENCES accounts(id),
468             config_json  TEXT NOT NULL
469         )
470         """
471         "user_id, config_json",
472     ),
473 ):
474     await conn.execute(f"ALTER TABLE {table} RENAME TO {table}__legacy_data")
475     await conn.execute(new_ddl)
476     await conn.execute(
477         f"INSERT INTO {table} ({copy_cols}) SELECT {copy_cols} FROM
{table}__legacy_data"
478     )
479     await conn.execute(f"DROP TABLE {table}__legacy_data")
480
481     # The partial unique index for document_id dedup (created by migration 001
482     # against the legacy videos table) was dropped along with the legacy copy
483     # above. Recreate it on the new videos table.
484     await conn.execute(
485         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
486         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
487     )
488
489     # Rename the legacy users table (do NOT drop ? kept as a rollback safety
490     # net for one release per ADR 0010). The data-holding channels/videos/
491     # filter_config tables were already rebuilt above with FKs to accounts(id),
492     # so renaming users no longer retargets any live FK.
493     await conn.execute("ALTER TABLE users RENAME TO users_deprecated")
494
495
496 @migration(3)
497 async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
498     """Land the provider-neutral domain tables (ADR 0011, Track B).
499
500     Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
501     ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
502     DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
503     them), then backfills the three populated ones from the legacy
504     ``channels`` / ``videos`` / ``filter_config`` tables and renames the
505     legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
506     pattern). Idempotent: a no-op once the legacy tables have been renamed.
507     """
508     # Idempotency gate: if the legacy videos table is gone, this migration has
509     # already run (or the DB is fresh and _SCHEMA's legacy tables were already
510     # dropped by _apply_schema post-003). Nothing to backfill.
511     if not await _table_exists(conn, "videos"):
512         return
513
514     # 1. Backfill sources from channels. ADR 0009 backfill: already-seen
515     # channels stay watched (watched=1). provider_source_id is the Telegram
516     # channel_id (cast to TEXT for the provider-neutral column).
517     await conn.execute(
518         """
519         INSERT OR IGNORE INTO sources
520             (account_id, provider, provider_source_id, title, kind, watched,
521             seen_first_at, last_seen_at)

```



```

522         SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
523                1, seen_first_at, seen_first_at
524     FROM channels
525     """
526 )
527
528 # 2. Backfill content_items from videos, computing dedup_key in pure SQL.
529 #   The three-branch decision tree (Option 1A):
530 #       a) document_id IS NOT NULL           ? 'tg:doc:' || document_id
531 #       b) all four fingerprint fields set   ? 'tg:fp:' || size:dur:h:w
532 #       c) else (Tier-0 fallback)           ? 'tg:msg:' || source_id:msg_id
533 #   The ``CASE`` is ordered strongest-first and each branch is guarded so
534 #   partial metadata never collapses to a degenerate key. source_id is
535 #   resolved by joining the channel's provider_source_id back to sources.id
536 #   (Telegram message_id is per-channel, so source_id is required).
537 await conn.execute(
538     """
539     INSERT OR IGNORE INTO content_items
540         (account_id, source_id, provider, provider_message_id, content_type,
541          document_id, duration_s, width, height, size_bytes, mime_type,
542          dedup_key, matched, seen_at)
543     SELECT
544         v.user_id,
545         s.id,
546         'telegram',
547         CAST(v.message_id AS TEXT),
548         'video',
549         CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
550         v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
551         CASE
552             WHEN v.document_id IS NOT NULL
553             THEN 'tg:doc:' || v.document_id
554             WHEN v.size_bytes IS NOT NULL AND v.duration_s IS NOT NULL
555                  AND v.height IS NOT NULL AND v.width IS NOT NULL
556             THEN 'tg:fp:' || v.size_bytes || ':' || v.duration_s || ':'
557                  || v.height || ':' || v.width
558             ELSE 'tg:msg:' || s.id || ':' || v.message_id
559         END,
560         v.matched,
561         v.seen_at
562     FROM videos v
563     JOIN sources s
564         ON s.account_id = v.user_id
565        AND s.provider = 'telegram'
566        AND s.provider_source_id = CAST(v.channel_id AS TEXT)
567     """
568 )
569
570 # 3. Backfill policies from filter_config: one 'default' policy per account
571 #   (Option 2A). is_default=1 so the legacy load_filter_config shim reads
572 #   this row. UNIQUE(account_id, name) dedups idempotent re-runs.
573 await conn.execute(
574     """
575     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
576                                    created_at, updated_at)
577     SELECT user_id, 'default', config_json, 1, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP
578     FROM filter_config
579     """
580 )
581
582 # 4. Rename (don't drop) the legacy tables ? kept for one release as a
583 #   rollback safety net (ADR 0010 pattern).
584 await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
585 await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
586 await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")

```

```

587
588
589 async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
590     """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
591     async with conn.execute(
592         "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
593     ) as cur:
594         return await cur.fetchone() is not None
595
596
597 # --- helpers -----
598
599
600 def _now() -> datetime:
601     """Timezone-aware UTC 'now', used for policy timestamps."""
602     return datetime.now(UTC)
603
604
605 def _row_to_account(row: aiosqlite.Row) -> Account:
606     return Account(
607         id=row["id"],
608         telegram_id=row["telegram_id"],
609         provider=row["provider"],
610         display_name=row["display_name"],
611         created_at=row["created_at"], # stored as ISO string; pydantic parses
612     )
613
614
615 def _row_to_telegram_session(row: aiosqlite.Row) -> TelegramSession:
616     return TelegramSession(
617         account_id=row["account_id"],
618         phone=row["phone"],
619         session_path=row["session_path"],
620         enabled=bool(row["enabled"]),
621         created_at=row["created_at"],
622         last_login_at=row["last_login_at"],
623     )
624
625
626 def _account_session_to_user(account: Account, session: TelegramSession | None) -> User:
627     """Project an (account, optional session) pair onto the legacy ``User`` view.
628
629     Used by the deprecated ``upsert_user``/``get_user``/``list_enabled_users``
630     shims so pre-ADR-0010 callers and tests keep working. A bot-only account
631     (no session) maps to a ``User`` with ``enabled=False`` so the legacy
632     "list enabled users" semantics exclude it ? matching the structural rule
633     that bot-only users are not runnable Telethon clients.
634     """
635     if session is None:
636         return User(
637             id=account.telegram_id if account.telegram_id is not None else account.id,
638             phone=None,
639             enabled=False,
640             created_at=account.created_at,
641         )
642     return User(
643         id=account.telegram_id if account.telegram_id is not None else account.id,
644         phone=session.phone,
645         enabled=session.enabled,
646         created_at=session.created_at,
647     )
648
649
650 def _row_to_channel(row: aiosqlite.Row) -> Channel:
651     return Channel(

```

```

652         id=row["id"],
653         user_id=row["user_id"],
654         channel_id=row["channel_id"],
655         title=row["title"],
656         seen_first_at=row["seen_first_at"],
657     )
658
659
660 def _row_to_video(row: aiosqlite.Row) -> VideoRecord:
661     return VideoRecord(
662         id=row["id"],
663         user_id=row["user_id"],
664         channel_id=row["channel_id"],
665         message_id=row["message_id"],
666         document_id=row["document_id"],
667         duration_s=row["duration_s"],
668         width=row["width"],
669         height=row["height"],
670         size_bytes=row["size_bytes"],
671         mime_type=row["mime_type"],
672         matched=bool(row["matched"]),
673         forwarded=bool(row["forwarded"]),
674         seen_at=row["seen_at"],
675     )
676
677
678 def _row_to_source(row: aiosqlite.Row) -> Source:
679     return Source(
680         id=row["id"],
681         account_id=row["account_id"],
682         provider=row["provider"],
683         provider_source_id=row["provider_source_id"],
684         title=row["title"],
685         kind=row["kind"],
686         watched=bool(row["watched"]),
687         seen_first_at=row["seen_first_at"],
688         last_seen_at=row["last_seen_at"],
689     )
690
691
692 def _row_to_content_item(row: aiosqlite.Row) -> ContentItem:
693     return ContentItem(
694         id=row["id"],
695         account_id=row["account_id"],
696         source_id=row["source_id"],
697         provider=row["provider"],
698         provider_message_id=row["provider_message_id"],
699         content_type=row["content_type"],
700         text=row["text"],
701         link_url=row["link_url"],
702         media_ref=row["media_ref"],
703         document_id=row["document_id"],
704         duration_s=row["duration_s"],
705         width=row["width"],
706         height=row["height"],
707         size_bytes=row["size_bytes"],
708         mime_type=row["mime_type"],
709         dedup_key=row["dedup_key"],
710         matched=bool(row["matched"]),
711         seen_at=row["seen_at"],
712     )
713
714
715 def _row_to_policy(row: aiosqlite.Row) -> Policy:
716     return Policy(

```

```

717         id=row["id"],
718         account_id=row["account_id"],
719         name=row["name"],
720         config_json=row["config_json"],
721         is_default=bool(row["is_default"]),
722         created_at=row["created_at"],
723         updated_at=row["updated_at"],
724     )
725
726
727 # --- account + session repos (ADR 0010) -----
728
729
730 async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:
731     """Insert or update an account row; returns the persisted Account."""
732     await conn.execute(
733         """INSERT INTO accounts (id, telegram_id, provider, display_name, created_at)
734         VALUES (?, ?, ?, ?, ?)
735         ON CONFLICT(id) DO UPDATE SET
736             telegram_id=excluded.telegram_id,
737             provider=excluded.provider,
738             display_name=excluded.display_name""",
739         (
740             account.id,
741             account.telegram_id,
742             account.provider,
743             account.display_name,
744             account.created_at.isoformat(),
745         ),
746     )
747     await conn.commit()
748     return await get_account(conn, account.id) # type: ignore[return-value]
749
750
751 async def get_account(conn: aiosqlite.Connection, account_id: int) -> Account | None:
752     conn.row_factory = aiosqlite.Row
753     async with conn.execute("SELECT * FROM accounts WHERE id = ?", (account_id,)) as cur:
754         row = await cur.fetchone()
755         return _row_to_account(row) if row else None
756
757
758 async def get_account_by_telegram_id(
759     conn: aiosqlite.Connection, telegram_id: int
760 ) -> Account | None:
761     """Resolve a Bot API ``from.id`` (or userbot account id) to an Account."""
762     conn.row_factory = aiosqlite.Row
763     async with conn.execute("SELECT * FROM accounts WHERE telegram_id = ?",
764 (telegram_id,)) as cur:
765         row = await cur.fetchone()
766         return _row_to_account(row) if row else None
767
768
769 async def upsert_telegram_session(
770     conn: aiosqlite.Connection, session: TelegramSession
771 ) -> TelegramSession:
772     """Insert or update a telegram_sessions row; returns the persisted session."""
773     await conn.execute(
774         """INSERT INTO telegram_sessions
775         (account_id, phone, session_path, enabled, created_at, last_login_at)
776         VALUES (?, ?, ?, ?, ?, ?)
777         ON CONFLICT(account_id) DO UPDATE SET
778             phone=excluded.phone,
779             session_path=excluded.session_path,
780             enabled=excluded.enabled,

```

```

780         last_login_at=excluded.last_login_at"",
781     (
782         session.account_id,
783         session.phone,
784         session.session_path,
785         int(session.enabled),
786         session.created_at.isoformat(),
787         session.last_login_at.isoformat() if session.last_login_at else None,
788     ),
789 )
790 await conn.commit()
791 conn.row_factory = aiosqlite.Row
792 async with conn.execute(
793     "SELECT * FROM telegram_sessions WHERE account_id = ?", (session.account_id,)
794 ) as cur:
795     row = await cur.fetchone()
796     # The row must exist ? we just upserted it by PK (account_id). Assert the
797     # invariant explicitly so mypy can narrow Row | None ? Row without a
798     # type-ignore comment masking a real future regression.
799     assert row is not None
800     return _row_to_telegram_session(row)
801
802
803 async def get_telegram_session(
804     conn: aiosqlite.Connection, account_id: int
805 ) -> TelegramSession | None:
806     conn.row_factory = aiosqlite.Row
807     async with conn.execute(
808         "SELECT * FROM telegram_sessions WHERE account_id = ?", (account_id,)
809     ) as cur:
810         row = await cur.fetchone()
811         return _row_to_telegram_session(row) if row else None
812
813
814 async def list_runnable_sessions(
815     conn: aiosqlite.Connection,
816 ) -> list[tuple[Account, TelegramSession]]:
817     """Return (account, session) pairs for sessions the manager should start.
818
819     This is the ADR 0010 structural rule: ``manager.run`` iterates this list,
820     not ``accounts``. A bot-only user has no ``telegram_sessions`` row and is
821     therefore inherently invisible to the Telethon supervisor ? the "don't
822     start a client for bot-only users" invariant is enforced by query shape.
823     """
824     conn.row_factory = aiosqlite.Row
825     async with conn.execute(
826         """SELECT a.*, s.phone AS s_phone, s.session_path, s.enabled AS s_enabled,
827             s.created_at AS s_created_at, s.last_login_at
828             FROM accounts a
829             JOIN telegram_sessions s ON s.account_id = a.id
830             WHERE s.enabled = 1"""
831     ) as cur:
832         rows = await cur.fetchall()
833     return [
834         (
835             _row_to_account(r),
836             TelegramSession(
837                 account_id=r["id"],
838                 phone=r["s_phone"],
839                 session_path=r["session_path"],
840                 enabled=bool(r["s_enabled"]),
841                 created_at=r["s_created_at"],
842                 last_login_at=r["last_login_at"],
843             ),
844         )

```

```

845         for r in rows
846     ]
847
848
849 # --- legacy User shims (backwards-compatible view over accounts + sessions) --
850
851
852 async def upsert_user(conn: aiosqlite.Connection, user: User) -> User:
853     """Deprecated shim: upsert the (account, session) pair implied by a ``User``.
854
855     Maps the legacy v1 ``User`` onto the ADR 0010 schema: ``User.id`` becomes
856     both ``accounts.id`` and ``accounts.telegram_id`` (legacy users are
857     Telegram userbot accounts), and a ``telegram_sessions`` row is created so
858     the manager can run them. ``User.enabled`` maps to ``session.enabled``.
859     """
860     account = Account(id=user.id, telegram_id=user.id, created_at=user.created_at)
861     await upsert_account(conn, account)
862     await upsert_telegram_session(
863         conn,
864         TelegramSession(
865             account_id=user.id,
866             phone=user.phone,
867             session_path=f"user_{user.id}.session",
868             enabled=user.enabled,
869             created_at=user.created_at,
870         ),
871     )
872     return await get_user(conn, user.id) # type: ignore[return-value]
873
874
875 async def get_user(conn: aiosqlite.Connection, user_id: int) -> User | None:
876     """Deprecated shim: project the (account, session) pair back to a ``User``."""
877     account = await get_account(conn, user_id)
878     if account is None:
879         return None
880     session = await get_telegram_session(conn, user_id)
881     return _account_session_to_user(account, session)
882
883
884 async def list_enabled_users(conn: aiosqlite.Connection) -> list[User]:
885     """Deprecated shim: list users that have an enabled MTPROTO session.
886
887     Equivalent to projecting :func:`list_runnable_sessions` onto the legacy
888     ``User`` shape. Bot-only accounts (no session) are excluded ? matching the
889     pre-ADR-0010 "list enabled users" semantics where every row was runnable.
890     """
891     pairs = await list_runnable_sessions(conn)
892     return [_account_session_to_user(a, s) for a, s in pairs]
893
894
895 # --- Source repos (ADR 0011) -----
896
897
898 async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
899     """Insert a source row, or update title/watched on conflict.
900
901     The conflict key is ``(account_id, provider, provider_source_id)`` per the
902     ADR 0011 schema. Returns the persisted row.
903     """
904     await conn.execute(
905         """INSERT INTO sources
906             (account_id, provider, provider_source_id, title, kind, watched,
907              seen_first_at, last_seen_at)
908             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
909             ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET

```

```

910         title=excluded.title,
911         watched=excluded.watched,
912         last_seen_at=excluded.last_seen_at"",
913     (
914         source.account_id,
915         source.provider,
916         source.provider_source_id,
917         source.title,
918         source.kind,
919         int(source.watched),
920         source.seen_first_at.isoformat(),
921         (source.last_seen_at.isoformat() if source.last_seen_at else None),
922     ),
923 )
924 await conn.commit()
925 conn.row_factory = aiosqlite.Row
926 async with conn.execute(
927     """SELECT * FROM sources
928        WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
929        (source.account_id, source.provider, source.provider_source_id),
930 ) as cur:
931     row = await cur.fetchone()
932     assert row is not None # just upserted by the conflict key
933     return _row_to_source(row)
934
935
936 async def get_source_by_provider_id(
937     conn: aiosqlite.Connection, account_id: int, provider: str, provider_source_id: str
938 ) -> Source | None:
939     """Resolve a source by its provider-native id."""
940     conn.row_factory = aiosqlite.Row
941     async with conn.execute(
942         """SELECT * FROM sources
943            WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
944            (account_id, provider, provider_source_id),
945        ) as cur:
946            row = await cur.fetchone()
947            return _row_to_source(row) if row else None
948
949
950 # --- ContentItem repos (ADR 0011) -----
951
952
953 async def insert_content_item(conn: aiosqlite.Connection, item: ContentItem) ->
ContentItem:
954     """Insert a content item. Raises ``IntegrityError`` on a dedup collision.
955
956     The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
957     unique constraints guard against duplicates:
958     - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
959     - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
960     """
961     cur = await conn.execute(
962         """INSERT INTO content_items
963            (account_id, source_id, provider, provider_message_id, content_type,
964             text, link_url, media_ref, document_id, duration_s, width, height,
965             size_bytes, mime_type, dedup_key, matched, seen_at)
966            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
967         (
968             item.account_id,
969             item.source_id,
970             item.provider,
971             item.provider_message_id,
972             item.content_type,
973             item.text,

```

```

974         item.link_url,
975         item.media_ref,
976         item.document_id,
977         item.duration_s,
978         item.width,
979         item.height,
980         item.size_bytes,
981         item.mime_type,
982         item.dedup_key,
983         int(item.matched),
984         item.seen_at.isoformat(),
985     ),
986 )
987 await conn.commit()
988 return item.model_copy(update={"id": cur.lastrowid})
989
990
991 async def get_content_item_by_message(
992     conn: aiosqlite.Connection,
993     account_id: int,
994     source_id: int,
995     provider_message_id: str,
996 ) -> ContentItem | None:
997     """Tier-0 lookup by the (account, source, provider-message-id) natural key."""
998     conn.row_factory = aiosqlite.Row
999     async with conn.execute(
1000         """SELECT * FROM content_items
1001            WHERE account_id = ? AND source_id = ? AND provider_message_id = ?""",
1002         (account_id, source_id, provider_message_id),
1003     ) as cur:
1004         row = await cur.fetchone()
1005     return _row_to_content_item(row) if row else None
1006
1007
1008 async def get_content_item_by_dedup_key(
1009     conn: aiosqlite.Connection, account_id: int, dedup_key: str
1010 ) -> ContentItem | None:
1011     """Provider-neutral dedup lookup by ``dedup_key`` (Option 1A).
1012
1013     Replaces the Tier-1 (``document_id``) and Tier-2 (fingerprint) lookups with
1014     a single query: the ``dedup_key`` already encodes whichever tier was
1015     applicable at ingestion time.
1016
1017     """
1018     conn.row_factory = aiosqlite.Row
1019     async with conn.execute(
1020         "SELECT * FROM content_items WHERE account_id = ? AND dedup_key = ?",
1021         (account_id, dedup_key),
1022     ) as cur:
1023         row = await cur.fetchone()
1024     return _row_to_content_item(row) if row else None
1025
1026 # --- Policy repos (ADR 0011, Option 2A) -----
1027
1028
1029 async def upsert_policy(conn: aiosqlite.Connection, policy: Policy) -> Policy:
1030     """Insert or update a policy row by ``(account_id, name)``.
1031
1032     Exactly one policy per account should carry ``is_default=True``; callers
1033     are responsible for flipping the previous default before flipping a new
1034     one (enforced by a partial unique index landed by future work, not in the
1035     schema ? SQLite ``CHECK`` can't express "at most one per group").
1036
1037     """
1038     await conn.execute(
1039         """INSERT INTO policies (account_id, name, config_json, is_default,

```



```

1039             created_at, updated_at)
1040     VALUES (?, ?, ?, ?, ?, ?)
1041     ON CONFLICT(account_id, name) DO UPDATE SET
1042         config_json=excluded.config_json,
1043         is_default=excluded.is_default,
1044         updated_at=excluded.updated_at""",
1045     (
1046         policy.account_id,
1047         policy.name,
1048         policy.config_json,
1049         int(policy.is_default),
1050         policy.created_at.isoformat(),
1051         policy.updated_at.isoformat(),
1052     ),
1053 )
1054 await conn.commit()
1055 conn.row_factory = aiosqlite.Row
1056 async with conn.execute(
1057     "SELECT * FROM policies WHERE account_id = ? AND name = ?",
1058     (policy.account_id, policy.name),
1059 ) as cur:
1060     row = await cur.fetchone()
1061     assert row is not None
1062     return _row_to_policy(row)
1063
1064
1065 async def get_default_policy(conn: aiosqlite.Connection, account_id: int) -> Policy |
None:
1066     """Return the account's default policy (`is_default=1`)."""
1067     conn.row_factory = aiosqlite.Row
1068     async with conn.execute(
1069         "SELECT * FROM policies WHERE account_id = ? AND is_default = 1",
1070         (account_id,),
1071     ) as cur:
1072         row = await cur.fetchone()
1073         return _row_to_policy(row) if row else None
1074
1075
1076 # --- legacy Channel shim (backwards-compatible view over sources) -----
1077
1078
1079 async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1080     """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.
1081
1082     ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1083     reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
1084     New code should call :func:`upsert_source` directly.
1085     """
1086     src = await upsert_source(
1087         conn,
1088         Source(
1089             account_id=channel.user_id,
1090             provider="telegram",
1091             provider_source_id=str(channel.channel_id),
1092             title=channel.title,
1093             kind="channel",
1094             watched=True, # back-compat: pre-ADR-0011 channels were all watched
1095             seen_first_at=channel.seen_first_at,
1096         ),
1097     )
1098     return _source_to_channel(src)
1099
1100
1101 async def get_channel(conn: aiosqlite.Connection, user_id: int, channel_id: int) ->
Channel | None:

```

```

1102     """Deprecated shim: project the :class:`Source` row back to a legacy Channel."""
1103     src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1104     return _source_to_channel(src) if src else None
1105
1106
1107 def _source_to_channel(src: Source) -> Channel:
1108     """Project a provider-neutral :class:`Source` onto the legacy :class:`Channel`."""
1109     return Channel(
1110         id=src.id,
1111         user_id=src.account_id,
1112         channel_id=int(src.provider_source_id),
1113         title=src.title,
1114         seen_first_at=src.seen_first_at,
1115     )
1116
1117
1118 # --- legacy VideoRecord shims (backwards-compatible view over content_items) -
1119
1120
1121 async def get_video(
1122     conn: aiosqlite.Connection, user_id: int, channel_id: int, message_id: int
1123 ) -> VideoRecord | None:
1124     """Deprecated shim: Tier-0 lookup against ``content_items`` (ADR 0011).
1125
1126     Resolves the legacy ``(user_id, channel_id, message_id)`` key to a
1127     :class:`Source` + :class:`ContentItem` and projects the latter onto a
1128     :class:`VideoRecord`. New code should call :func:`get_content_item_by_message`.
1129     """
1130     src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1131     if src is None:
1132         return None
1133     item = await get_content_item_by_message(conn, user_id, src.id, str(message_id))
1134     return _content_item_to_video(item, src, user_id, channel_id) if item else None
1135
1136
1137 async def get_video_by_document_id(
1138     conn: aiosqlite.Connection, user_id: int, document_id: int
1139 ) -> VideoRecord | None:
1140     """Deprecated shim: Tier-1 lookup via ``compute_dedup_key`` + ``content_items``."""
1141     dedup_key = compute_dedup_key(provider="telegram", document_id=str(document_id))
1142     item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1143     if item is None:
1144         return None
1145     src = await get_source_by_id(conn, item.source_id)
1146     if src is None:
1147         return None
1148     return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1149
1150
1151 async def get_video_by_content_fingerprint(
1152     conn: aiosqlite.Connection,
1153     user_id: int,
1154     size_bytes: int,
1155     duration_s: int,
1156     height: int,
1157     width: int,
1158 ) -> VideoRecord | None:
1159     """Deprecated shim: Tier-2 lookup via ``compute_dedup_key`` + ``content_items``."""
1160     dedup_key = compute_dedup_key(
1161         provider="telegram",
1162         size_bytes=size_bytes,
1163         duration_s=duration_s,
1164         height=height,
1165         width=width,
1166     )

```

```

1167     item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1168     if item is None:
1169         return None
1170     src = await get_source_by_id(conn, item.source_id)
1171     if src is None:
1172         return None
1173     return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1174
1175
1176 async def get_source_by_id(conn: aiomysqlite.Connection, source_id: int) -> Source | None:
1177     """Resolve a source by its surrogate PK (used by the legacy video shims)."""
1178     conn.row_factory = aiomysqlite.Row
1179     async with conn.execute("SELECT * FROM sources WHERE id = ?", (source_id,)) as cur:
1180         row = await cur.fetchone()
1181     return _row_to_source(row) if row else None
1182
1183
1184 async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
1185     """Deprecated shim: insert into ``content_items`` (ADR 0011).
1186
1187     Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
1188     if absent), computes the provider-neutral ``dedup_key`` via
1189     :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1190     ``VideoRecord`` shape is reconstructed from the inserted row.
1191     """
1192     src = await upsert_source(
1193         conn,
1194         Source(
1195             account_id=video.user_id,
1196             provider="telegram",
1197             provider_source_id=str(video.channel_id),
1198             kind="channel",
1199             watched=True,
1200         ),
1201     )
1202     assert src.id is not None # upsert_source populates the surrogate PK
1203     item = ContentItem(
1204         account_id=video.user_id,
1205         source_id=src.id,
1206         provider="telegram",
1207         provider_message_id=str(video.message_id),
1208         content_type="video",
1209         document_id=str(video.document_id) if video.document_id is not None else None,
1210         duration_s=video.duration_s,
1211         width=video.width,
1212         height=video.height,
1213         size_bytes=video.size_bytes,
1214         mime_type=video.mime_type,
1215         dedup_key=compute_dedup_key(
1216             provider="telegram",
1217             document_id=(str(video.document_id) if video.document_id is not None else
1218             None),
1219             source_id=src.id,
1220             provider_message_id=str(video.message_id),
1221             size_bytes=video.size_bytes,
1222             duration_s=video.duration_s,
1223             height=video.height,
1224             width=video.width,
1225         ),
1226         matched=video.matched,
1227         seen_at=video.seen_at,
1228     )
1229     item = await insert_content_item(conn, item)
1230     return _content_item_to_video(item, src, video.user_id, video.channel_id)

```

```

1231
1232 async def mark_forwarded(conn: aiomysqlite.Connection, video_id: int) -> None:
1233     """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).
1234
1235     Per-destination delivery state moves to ``delivery_attempts`` (Track C,
1236     #28). For Phase 1 (before Track C lands) there is no equivalent column on
1237     ``content_items`` ? the ``matched`` flag alone is sufficient because v1
1238     delivery still targets the single ``delivery_target`` and retried delivery
1239     is harmless (forwards are idempotent). This shim is retained so existing
1240     call sites in ``delivery.py`` keep working without a flag day.
1241     """
1242     # content_items has no ``forwarded`` column post-ADR-0011; nothing to write.
1243     # The row's ``matched`` flag (set at insert time) is the surviving signal.
1244     return None
1245
1246
1247 def _content_item_to_video(
1248     item: ContentItem, src: Source, user_id: int, channel_id: int
1249 ) -> VideoRecord:
1250     """Project a :class:`ContentItem` onto the legacy :class:`VideoRecord`.
1251
1252     ``forwarded`` is always ``False`` post-ADR-0011 (the column is dropped;
1253     per-destination delivery state moves to Track C). Existing callers read it
1254     only to decide whether to skip re-delivery; since forwards are idempotent
1255     the ``False`` default is safe for Phase 1.
1256     """
1257     return VideoRecord(
1258         id=item.id,
1259         user_id=user_id,
1260         channel_id=channel_id,
1261         message_id=int(item.provider_message_id),
1262         document_id=int(item.document_id) if item.document_id is not None else None,
1263         duration_s=item.duration_s,
1264         width=item.width,
1265         height=item.height,
1266         size_bytes=item.size_bytes,
1267         mime_type=item.mime_type,
1268         matched=item.matched,
1269         forwarded=False,
1270         seen_at=item.seen_at,
1271     )
1272
1273
1274 # --- filter config shims (backwards-compatible view over policies) -----
1275
1276
1277 async def load_filter_config(conn: aiomysqlite.Connection, user_id: int) -> FilterConfig:
1278     """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1279
1280     ADR 0011 (Option 2A): ``filter_config`` is renamed to
1281     ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1282     instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1283     """
1284     policy = await get_default_policy(conn, user_id)
1285     if policy is None:
1286         return FilterConfig()
1287     return FilterConfig.model_validate_json(policy.config_json)
1288
1289
1290 async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1291     """Deprecated shim: upsert the account's default :class:`Policy`."""
1292     existing = await get_default_policy(conn, user_id)
1293     await upsert_policy(
1294         conn,

```

```

1295         Policy(
1296             id=existing.id if existing is not None else None,
1297             account_id=user_id,
1298             name="default",
1299             config_json=cfg.model_dump_json(),
1300             is_default=True,
1301             created_at=(existing.created_at if existing is not None else _now()),
1302             updated_at=_now(),
1303         ),
1304     )
1305
1306
1307 # --- bulk helpers (used by the manager on startup) -----
1308
1309
1310 async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1311     conn: aiosqlite.Connection, user_id: int, limit: int = 100
1312 ) -> list[VideoRecord]:
1313     """Deprecated shim: list the account's content items projected onto VideoRecord."""
1314     conn.row_factory = aiosqlite.Row
1315     async with conn.execute(
1316         """SELECT ci.*, s.provider_source_id AS src_pid
1317            FROM content_items ci
1318            JOIN sources s ON s.id = ci.source_id
1319            WHERE ci.account_id = ?
1320            ORDER BY ci.id DESC LIMIT ?""",
1321         (user_id, limit),
1322     ) as cur:
1323         rows = await cur.fetchall()
1324     return [
1325         VideoRecord(
1326             id=r["id"],
1327             user_id=user_id,
1328             channel_id=int(r["src_pid"]),
1329             message_id=int(r["provider_message_id"]),
1330             document_id=(int(r["document_id"]) if r["document_id"] is not None else
1331 None),
1332             duration_s=r["duration_s"],
1333             width=r["width"],
1334             height=r["height"],
1335             size_bytes=r["size_bytes"],
1336             mime_type=r["mime_type"],
1337             matched=bool(r["matched"]),
1338             forwarded=False,
1339             seen_at=r["seen_at"],
1340         )
1341         for r in rows
1342     ]
1343
1344 __all__ = [
1345     "init_db",
1346     "compute_dedup_key",
1347     # Account + session repos (ADR 0010)
1348     "upsert_account",
1349     "get_account",
1350     "get_account_by_telegram_id",
1351     "upsert_telegram_session",
1352     "get_telegram_session",
1353     "list_runnable_sessions",
1354     # Provider-neutral domain repos (ADR 0011)
1355     "upsert_source",
1356     "get_source_by_provider_id",
1357     "get_source_by_id",
1358     "insert_content_item",

```

```

1359     "get_content_item_by_message",
1360     "get_content_item_by_dedup_key",
1361     "upsert_policy",
1362     "get_default_policy",
1363     # Legacy User shims (backwards-compatible)
1364     "upsert_user",
1365     "get_user",
1366     "list_enabled_users",
1367     # Legacy Channel / Video / config shims (ADR 0011)
1368     "upsert_channel",
1369     "get_channel",
1370     "get_video",
1371     "get_video_by_document_id",
1372     "get_video_by_content_fingerprint",
1373     "insert_video",
1374     "mark_forwarded",
1375     "load_filter_config",
1376     "save_filter_config",
1377     "list_videos_for_user",
1378 ]
1379
1380
1381 def __dir__() -> Iterable[str]:
1382     return __all__
1383

```

## 25. assistant (2026-07-06T05:58:16.689Z)

Now the full `db.py` is in view and I have concrete concerns to verify (especially a possible **Tier2 dedup regression**). Let me read the models, the v1 dedup call sites, the tests, and the base `db.py` to confirm behavior ? and check the gate log.

## 26. user (2026-07-06T05:58:17.638Z)

```

1     """Pydantic data models for users, channels, videos, and filter config.
2
3     These models are the in-memory representation used by the repository layer
4     (`db.py`) and the filter engine (`filters/`). They are deliberately
5     decoupled from Telethon's own types so the filter engine stays pure and
6     unit-testable without a live Telegram connection.
7     """
8
9     from __future__ import annotations
10
11     from datetime import UTC, datetime
12     from enum import Enum
13
14     from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
15
16
17     def _utcnow() -> datetime:
18         """Timezone-aware UTC 'now', used as a default factory."""
19         return datetime.now(UTC)
20
21
22     class DeliveryMode(str, Enum):
23         """How a matched video is delivered to the user.
24
25         - ``FORWARD``: forward the video itself to Saved Messages (re-uploads).
26         - ``LINK``: send a text message with a t.me link + metadata summary.
27         - ``BOTH``: forward the video AND send the link message.
28         """
29
30         FORWARD = "forward"
31         LINK = "link"

```

```

32     BOTH = "both"
33
34
35     class Account(BaseModel):
36         """A product identity ? the owner of config, channels, routes, and content.
37
38         Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
39         is the bridge used by the Bot API front-door (``from.id``) and by the
40         legacy userbot path; it is nullable so future providers that don't have a
41         Telegram id (e.g. WhatsApp) can still own an ``Account``.
42         """
43
44         model_config = ConfigDict(frozen=True)
45
46         id: int = Field(description="Stable surrogate PK; the owner key for all downstream
tables")
47         telegram_id: int | None = Field(
48             default=None,
49             description="Telegram user id (bot from.id OR userbot account id). UNIQUE when
set.",
50         )
51         provider: str = Field(default="telegram", description="Provider namespace; future:
'whatsapp'")
52         display_name: str | None = None
53         created_at: datetime = Field(default_factory=_utcnow)
54
55
56     class TelegramSession(BaseModel):
57         """An optional MTPROTO (Telethon) session linked to an :class:`Account`.
58
59         A bot-only user has an ``Account`` row but no ``TelegramSession`` row;
60         that is what makes them structurally invisible to ``manager.run``
61         (ADR 0010). ``session_path`` is data-driven so the file-location
62         convention is explicit rather than derived from the PK at runtime.
63         """
64
65         model_config = ConfigDict(frozen=True)
66
67         account_id: int = Field(description="FK ? accounts.id; PK (one session per account
in v1)")
68         phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
69         session_path: str = Field(description="File-backed Telethon session path
(user_<id>.session)")
70         enabled: bool = Field(
71             default=True,
72             description="If False, the manager skips this session (not the account)",
73         )
74         created_at: datetime = Field(default_factory=_utcnow)
75         last_login_at: datetime | None = None
76
77
78     class User(BaseModel):
79         """Deprecated: legacy v1 model conflating identity + MTPROTO session.
80
81         Retained as a backwards-compatible view over the new ``accounts`` +
82         ``telegram_sessions`` tables (ADR 0010). New code should use
83         :class:`Account` + :class:`TelegramSession` directly. The ``db`` layer
84         maps ``User`` ? (``Account``, ``TelegramSession``) so existing callers
85         and tests keep working without a flag day.
86         """
87
88         model_config = ConfigDict(frozen=True)
89
90         id: int = Field(description="Telegram user id (== accounts.telegram_id for legacy

```

```

rows)")
91     phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
92     enabled: bool = Field(default=True, description="If False, the manager skips this
session")
93     created_at: datetime = Field(default_factory=_utcnow)
94
95
96     class Channel(BaseModel):
97         """A channel/group a user is a member of. Populated lazily as messages arrive.
98
99         Deprecated: a provider-neutral view over :class:`Source` (ADR 0011). New
100         code should use :class:`Source` directly; the ``db`` layer maps
101         ``Channel`` ? ``Source`` so existing callers and tests keep working.
102         """
103
104         model_config = ConfigDict(frozen=True)
105
106         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
107         user_id: int
108         channel_id: int = Field(description="Telegram channel/chat id")
109         title: str | None = None
110         seen_first_at: datetime = Field(default_factory=_utcnow)
111
112
113     # --- provider-neutral domain models (ADR 0011, Track B) -----
114     #
115     # These generalize the legacy v1 ``Channel`` / ``VideoRecord`` / ``FilterConfig``
116     # shapes so the schema no longer leaks Telegram-specific ids into every future
117     # provider's code path. ``provider`` columns + provider-prefixed ``dedup_key``
118     # make cross-provider collisions structurally impossible. See ADR 0011 for the
119     # full rationale and the two decisions (Option 1A: tiered ``dedup_key``;
120     # Option 2A: ``policies`` generalizes ``filter_config``).
121
122
123     class Source(BaseModel):
124         """A configured place we ingest content from (ADR 0011).
125
126         Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127         providers add their own source kinds). ``provider_source_id`` is the
128         provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129         """
130
131         model_config = ConfigDict(frozen=True)
132
133         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134         account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135         provider: str = Field(default="telegram", description="Provider namespace")
136         provider_source_id: str = Field(
137             description="Provider-native source id (Telegram channel_id, ?)"
138         )
139         title: str | None = None
140         kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141         watched: bool = Field(
142             default=False,
143             description="ADR 0009 selective scanning: ingest from this source iff True",
144         )
145         seen_first_at: datetime = Field(default_factory=_utcnow)
146         last_seen_at: datetime | None = None
147
148
149     class ContentItem(BaseModel):
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos

```



```

153         (`content_type='video'`); `text` / `link_url` are landed now but stay
154         NULL until later tracks populate them (avoids a future `ALTER TABLE`).
155         `dedup_key` is the provider-neutral dedup key (Option 1A: tiered lookup
156         wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157         """
158
159         model_config = ConfigDict(frozen=True)
160
161         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162         account_id: int = Field(description="FK ? accounts.id")
163         source_id: int = Field(description="FK ? sources.id")
164         provider: str = Field(default="telegram", description="Provider namespace")
165         provider_message_id: str = Field(
166             description="Provider-native message id (Telegram message_id, ?)"
167         )
168         content_type: str = Field(default="video", description="'video' | 'link' |
'article'")
169         text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
170         link_url: str | None = Field(default=None, description="For link/article posts
(Phase 1: NULL)")
171         media_ref: str | None = Field(default=None, description="Provider media reference")
172         document_id: str | None = Field(
173             default=None,
174             description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175         )
176         duration_s: int | None = None
177         width: int | None = None
178         height: int | None = None
179         size_bytes: int | None = None
180         mime_type: str | None = None
181         dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
per account")
182         matched: bool = False
183         seen_at: datetime = Field(default_factory=_utcnow)
184
185
186     class Policy(BaseModel):
187         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189         Generalizes the legacy per-account `filter_config` row so one account can
190         own multiple policies (enables ADR 0009's per-route filter overrides) and so
191         future enrichment/security stages grow into the same `config_json` column.
192         Exactly one policy per account carries `is_default=True`; the legacy
193         `load_filter_config` / `save_filter_config` shims read/write that row.
194         """
195
196         model_config = ConfigDict(frozen=True)
197
198         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
199         account_id: int = Field(description="FK ? accounts.id")
200         name: str = Field(description="Policy name; 'default' for the account default
policy")
201         config_json: str = Field(
202             description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
203         )
204         is_default: bool = Field(
205             default=False, description="Exactly one per account (UNIQUE partial index)"
206         )
207         created_at: datetime = Field(default_factory=_utcnow)
208         updated_at: datetime = Field(default_factory=_utcnow)
209
210
211     class FilterConfig(BaseModel):
212         """Per-user filter configuration.

```

```

213
214 v1 rules: duration bounds, minimum height (resolution), and max file size.
215 Stored in the DB as a JSON blob (`filter_config.config_json`) so adding a
216 new field later is backwards-compatible.
217 """
218
219 # Duration bounds in seconds. None means unbounded on that side.
220 min_duration_s: int | None = Field(default=5, ge=0)
221 max_duration_s: int | None = Field(default=7200, ge=0)
222
223 # Resolution: a video passes if its height >= target_height.
224 # None disables the resolution rule.
225 target_height: int | None = Field(default=1080, ge=1)
226
227 # Max file size in mebibytes. None disables the size rule.
228 max_size_mb: int | None = Field(default=2048, ge=1)
229
230 # Content-type exclusions. GIFs and round videos are excluded by default
231 # (their duration/dimensions are usually incidental to real video filters);
232 # silent videos are off by default since some legit clips have no audio.
233 # Existing users' stored configs are unaffected ? the DB JSON blob overrides
234 # these defaults, so this only changes the behaviour for fresh installs.
235 exclude_gif: bool = Field(
236     default=True,
237     description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
238 )
239 exclude_round: bool = Field(
240     default=True,
241     description="If True, exclude circular video messages (round_message=True)",
242 )
243 exclude_nosound: bool = Field(
244     default=False,
245     description="If True, exclude videos without an audio track (nosound=True)",
246 )
247
248 # How matched videos are delivered: forward the video, send a link, or both.
249 # Stored in the same JSON blob, so adding this field is backwards-compatible
250 # (existing rows without it default to FORWARD).
251 delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)
252
253 # Where matched videos / link messages are sent. Accepts:
254 # - "me" (default): Saved Messages ? exact v1.1 behaviour.
255 # - "@username": a public channel the account is a member of.
256 # - "-100..." or a bare int-as-string: a numeric Telegram chat id
257 #   (private channels without a username).
258 # The string is passed verbatim to Telethon's forward_messages / send_message.
259 # See ADR 0005 for rationale and access requirements.
260 delivery_target: str = Field(
261     default="me",
262     min_length=1,
263     description="Telethon entity to deliver to: 'me', '@username', or a chat id",
264 )
265
266 @field_validator("delivery_target", mode="before")
267 @classmethod
268 def _coerce_delivery_target(cls, v: object) -> str:
269     """Accept int chat ids (e.g. -1001234567890) and coerce to str.
270
271     Telethon accepts both str and int as an entity, but storing as str keeps
272     the JSON blob uniform and lets '@username' / 'me' / numeric ids share
273     one field type.
274     """
275     if v is None:
276         return "me"
277     if isinstance(v, int):

```

```

278         return str(v)
279     return str(v)
280
281     @model_validator(mode="after")
282     def _check_duration_bounds(self) -> FilterConfig:
283         """Ensure min_duration_s <= max_duration_s when both are set."""
284         if (
285             self.min_duration_s is not None
286             and self.max_duration_s is not None
287             and self.min_duration_s > self.max_duration_s
288         ):
289             raise ValueError(
290                 f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s ({self.max_duration_s})"
291             )
292         return self
293
294
295     class VideoRecord(BaseModel):
296         """A seen video message + its metadata. Drives dedup and future /search.
297
298         Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).
299         New code should use :class:`ContentItem`; the ``db`` layer maps
300         ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.
301         """
302
303         model_config = ConfigDict(frozen=True)
304
305         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
306         user_id: int
307         channel_id: int = Field(description="Origin Telegram channel id")
308         message_id: int = Field(description="Telegram message id within the channel")
309         document_id: int | None = Field(
310             default=None,
311             description="Telegram document.id ? reused across forwards; Tier-1 content dedup",
312         )
313
314         # Metadata extracted from the Telethon message (may be None if absent).
315         duration_s: int | None = None
316         width: int | None = None
317         height: int | None = None
318         size_bytes: int | None = None
319         mime_type: str | None = None
320
321         matched: bool = False
322         forwarded: bool = False
323         seen_at: datetime = Field(default_factory=_utcnow)
324
325         @property
326         def dedup_key(self) -> tuple[int, int, int]:
327             """The natural key used for deduplication in the ``videos`` table."""
328             return (self.user_id, self.channel_id, self.message_id)
329
330         @property
331         def content_fingerprint(self) -> tuple[int, int, int, int] | None:
332             """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height, width).
333
334             Returns None if any field is missing (cannot fingerprint).
335             """
336             if None in (self.size_bytes, self.duration_s, self.height, self.width):
337                 return None
338             return (self.size_bytes, self.duration_s, self.height, self.width) # type: ignore[return-value]

```

```

339
340
341     # Default filter config shared by the bootstrap onboarding step.
342     DEFAULT_FILTER_CONFIG = FilterConfig()
343

```

## 27. user (2026-07-06T05:58:23.138Z)

```

===CALL SITES of dedup shims===
src/tg_video_filter/backfill.py:165:         existing = await db.get_video(conn, user.id,
channel_id, message_id)
src/tg_video_filter/backfill.py:171:         existing = await
db.get_video_by_document_id(conn, user.id, document_id)
src/tg_video_filter/backfill.py:192:         existing = await
db.get_video_by_content_fingerprint(
src/tg_video_filter/backfill.py:232:         inserted = await db.insert_video(conn,
record)
src/tg_video_filter/db.py:1370:     "get_video",
src/tg_video_filter/db.py:1371:     "get_video_by_document_id",
src/tg_video_filter/db.py:1372:     "get_video_by_content_fingerprint",
src/tg_video_filter/db.py:1373:     "insert_video",
src/tg_video_filter/db.py:1374:     "mark_forwarded",
src/tg_video_filter/delivery.py:138:     await db.mark_forwarded(conn, video_id)
src/tg_video_filter/delivery.py:156:     await db.mark_forwarded(conn, video_id)
src/tg_video_filter/telethon_client.py:7:     -> dedup via db.get_video()
src/tg_video_filter/telethon_client.py:9:     -> db.insert_video(...) +
delivery.forward_to_target(..., target=cfg.delivery_target)
src/tg_video_filter/telethon_client.py:187:     existing = await db.get_video(conn, user.id,
channel_id, message_id)
src/tg_video_filter/telethon_client.py:193:     existing = await
db.get_video_by_document_id(conn, user.id, document_id)
src/tg_video_filter/telethon_client.py:214:     existing = await
db.get_video_by_content_fingerprint(
src/tg_video_filter/telethon_client.py:263:     inserted = await db.insert_video(conn,
record)
Binary file src/tg_video_filter/__pycache__/backfill.cpython-313.pyc matches
Binary file src/tg_video_filter/__pycache__/db.cpython-313.pyc matches
Binary file src/tg_video_filter/__pycache__/delivery.cpython-313.pyc matches
Binary file src/tg_video_filter/__pycache__/telethon_client.cpython-313.pyc matches

```

```

===where dedup happens (telethon/delivery/manager)===
src/tg_video_filter/telethon_client.py:7:     -> dedup via db.get_video()
src/tg_video_filter/telethon_client.py:9:     -> db.insert_video(...) +
delivery.forward_to_target(..., target=cfg.delivery_target)
src/tg_video_filter/telethon_client.py:158:     1. extracts VideoMeta + document_id (skips if
not a video),
src/tg_video_filter/telethon_client.py:159:     2. dedups via two-tier content dedup
(document.id, then metadata fingerprint),
src/tg_video_filter/telethon_client.py:160:     plus the existing (channel, message)
instance dedup,
src/tg_video_filter/telethon_client.py:182:     document_id = _extract_document_id(message)
src/tg_video_filter/telethon_client.py:184:     # --- three-tier dedup (cheapest first) ---
src/tg_video_filter/telethon_client.py:187:     existing = await db.get_video(conn, user.id,
channel_id, message_id)
src/tg_video_filter/telethon_client.py:191:     # Tier 1: same Telegram document.id ?
forwarded/cross-posted file.
src/tg_video_filter/telethon_client.py:192:     if document_id is not None:
src/tg_video_filter/telethon_client.py:193:         existing = await
db.get_video_by_document_id(conn, user.id, document_id)
src/tg_video_filter/telethon_client.py:196:         "video_dedup_document_id user=%s
doc_id=%s channel=%s msg=%s "
src/tg_video_filter/telethon_client.py:199:         document_id,
src/tg_video_filter/telethon_client.py:207:     # Tier 2: metadata fingerprint ? same file
re-uploaded (rare but possible).
src/tg_video_filter/telethon_client.py:214:     existing = await

```

```

db.get_video_by_content_fingerprint(
src/tg_video_filter/telethon_client.py:224:             "video_dedup_fingerprint user=%s
size=%s dur=%s h=%s w=%s "
src/tg_video_filter/telethon_client.py:254:             document_id=document_id,
src/tg_video_filter/telethon_client.py:263:             inserted = await db.insert_video(conn,
record)
src/tg_video_filter/telethon_client.py:265:             # Lost a dedup race: a concurrent
delivery of the same file
src/tg_video_filter/telethon_client.py:266:             # (same document_id or same message
instance) won the insert.
src/tg_video_filter/telethon_client.py:271:             "video_dedup_race user=%s channel=%s
msg=%s doc_id=%s",
src/tg_video_filter/telethon_client.py:275:             document_id,
src/tg_video_filter/telethon_client.py:315:def _extract_document_id(message: Message) -> int |
None:
src/tg_video_filter/telethon_client.py:319:     This is the most reliable content-based dedup
key: Telegram reuses the
src/tg_video_filter/telethon_client.py:320:     same ``document.id`` when a file is forwarded
across channels.
src/tg_video_filter/telethon_client.py:334:     collapse them to a single int for dedup/storage.
The exact subclass isn't
src/tg_video_filter/delivery.py:10:``seconds + jitter``, retry once) and mark the DB row
``forwarded=1`` on success.
src/tg_video_filter/delivery.py:138:         await db.mark_forwarded(conn, video_id)
src/tg_video_filter/delivery.py:139:         log.info("video_forwarded video=%s target=%s",
video_id, target)
src/tg_video_filter/delivery.py:156:         await db.mark_forwarded(conn, video_id)

```

## 28. user (2026-07-06T05:58:24.451Z)

===GATE LOG===

## 29. assistant (2026-07-06T05:58:31.971Z)

Gate passed (exit 0). Let me read the gate results and the exact v1 dedup flow in  
`telethon\_client.py` to confirm the Tier?2 concern, plus the old `db.py` fingerprint lookup.

## 30. user (2026-07-06T05:58:34.115Z)

(Bash completed with no output)

## 31. user (2026-07-06T05:58:34.571Z)

```

180         channel_id = _peer_id(message.peer_id)
181         message_id = message.id
182         document_id = _extract_document_id(message)
183
184         # --- three-tier dedup (cheapest first) ---
185
186         # Tier 0: same (user, channel, message) instance ? already processed.
187         existing = await db.get_video(conn, user.id, channel_id, message_id)
188         if existing is not None:
189             return
190
191         # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
192         if document_id is not None:
193             existing = await db.get_video_by_document_id(conn, user.id, document_id)
194             if existing is not None:
195                 log.info(
196                     "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
197                     "original_channel=%s original_msg=%s",
198                     user.id,
199                     document_id,
200                     channel_id,

```

```

201         message_id,
202         existing.channel_id,
203         existing.message_id,
204     )
205     return
206
207 # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
208 if (
209     meta.size_bytes is not None
210     and meta.duration_s is not None
211     and meta.height is not None
212     and meta.width is not None
213 ):
214     existing = await db.get_video_by_content_fingerprint(
215         conn,
216         user.id,
217         meta.size_bytes,
218         meta.duration_s,
219         meta.height,
220         meta.width,
221     )
222     if existing is not None:
223         log.info(
224             "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
225             "channel=%s msg=%s original_channel=%s original_msg=%s",
226             user.id,
227             meta.size_bytes,
228             meta.duration_s,
229             meta.height,
230             meta.width,
231             channel_id,
232             message_id,
233             existing.channel_id,
234             existing.message_id,
235         )
236         return
237
238 # Lazily record the channel (best-effort; failures are non-fatal).
239 try:
240     await db.upsert_channel(
241         conn,
242         Channel(user_id=user.id, channel_id=channel_id,
243 title=_channel_title(event)),
244     )
245 except Exception:
246     log.warning(
247         "failed to upsert channel %s for user %s", channel_id, user.id,
248 exc_info=True
249     )
250
251 result = engine_holder[ENGINE_KEY].evaluate(meta)
252 record = VideoRecord(
253     user_id=user.id,
254     channel_id=channel_id,
255     message_id=message_id,
256     document_id=document_id,
257     duration_s=meta.duration_s,
258     width=meta.width,
259     height=meta.height,
260     size_bytes=meta.size_bytes,
261     mime_type=meta.mime_type,
262     matched=result.matched,
263 )
264 try:
265     inserted = await db.insert_video(conn, record)

```

```

264         except sqlite3.IntegrityError:
265             # Lost a dedup race: a concurrent delivery of the same file
266             # (same document_id or same message instance) won the insert.
267             # The three-tier check above already classified this as new, so
268             # the collision means another task is processing (or has
269             # processed) it ? log and skip.
270             log.info(
271                 "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
272                 user.id,
273                 channel_id,
274                 message_id,
275                 document_id,
276             )
277             return
278         log.info(
279             "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
280             user.id,
281             channel_id,
282             message_id,
283             result.matched,
284             result.reasons,
285         )
286
287         if result.matched:
288             # Build link text for LINK/BOTH modes (None for FORWARD).
289             link_text = None

```

## 32. user (2026-07-06T05:58:38.630Z)

===OLD db.py dedup fns (main)===

```

async def get_video_by_content_fingerprint(
    conn: aiomysqlite.Connection,
    user_id: int,
    size_bytes: int,
    duration_s: int,
    height: int,
    width: int,
) -> VideoRecord | None:
    """Tier-2 content dedup: metadata fingerprint match ? probably same video.

```

Catches re-uploads where Telegram assigns a new ``document.id`` but the file bytes are identical. Uses (size\_bytes, duration\_s, height, width) as a composite key ? the odds of two different videos sharing all four values are negligible for this use case.

```

"""
conn.row_factory = aiomysqlite.Row
async with conn.execute(
    """SELECT * FROM videos
    WHERE user_id = ?
    AND size_bytes = ?
    AND duration_s = ?
    AND height = ?
    AND width = ?
    LIMIT 1""",
    (user_id, size_bytes, duration_s, height, width),
) as cur:
    row = await cur.fetchone()
    return _row_to_video(row) if row else None

```

```

async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
===OLD get_video_by_document_id (main)===
async def get_video_by_document_id(
    conn: aiomysqlite.Connection, user_id: int, document_id: int
) -> VideoRecord | None:

```

```
"""Tier-1 content dedup: same Telegram file id ? same video.
```

```
Telegram reuses ``document.id`` when the same file is forwarded across
channels, so this catches cross-channel duplicates at zero cost.
"""
```

```
conn.row_factory = aiosqlite.Row
async with conn.execute(
    """SELECT * FROM videos
       WHERE user_id = ? AND document_id = ?""",
    (user_id, document_id),
) as cur:
    row = await cur.fetchone()
    return _row_to_video(row) if row else None
```

```
async def get_video_by_content_fingerprint(
```

### 33. assistant (2026-07-06T05:59:19.765Z)

**\*\*Tier2 dedup regression confirmed.\*\*** The v1 `get_video_by_content_fingerprint` did `SELECT ... WHERE size=? AND duration=? AND height=? AND width=?` ? matching a prior video` **\*\*regardless of whether it had a `document_id`\*\***. The new shim looks up `dedup_key = "tg:fp:S:D:H:W"`, but any prior item that had a `document_id` was stored as `"tg:doc:X"` ? so the fingerprint lookup misses it. Re-uploads (new `document_id`, same metadata) ? exactly Tier2's purpose ? now get delivered twice. Let me re-run the gate capturing output cleanly and read the new tests + backfill flow.

### 34. user (2026-07-06T05:59:26.344Z)

```
=== ruff check ===
All checks passed!
ruff-check-exit=0
=== ruff format --check ===
Would reformat: src\tg_video_filter\commands.py
1 file would be reformatted, 33 files already formatted
fmt-exit=1
```

### 35. user (2026-07-06T05:59:35.574Z)

\_\_\_\_\_ coverage: platform win32, python 3.13.13-final-0 \_\_\_\_\_

| Name                                        | Stmts | Miss | Branch | BrPart | Cover | Missing        |
|---------------------------------------------|-------|------|--------|--------|-------|----------------|
| -----                                       |       |      |        |        |       |                |
| src\tg_video_filter\__init__.py             | 1     | 0    | 0      | 0      | 100%  |                |
| src\tg_video_filter\__main__.py             | 7     | 0    | 0      | 0      | 100%  |                |
| src\tg_video_filter\backfill.py             | 49    | 1    | 16     | 1      | 97%   | 81             |
| src\tg_video_filter\bootstrap.py            | 23    | 0    | 0      | 0      | 100%  |                |
| src\tg_video_filter\commands.py             | 195   | 11   | 72     | 1      | 96%   | 231-233,       |
| 254-258, 288-294, 364-370                   |       |      |        |        |       |                |
| src\tg_video_filter\config.py               | 26    | 1    | 2      | 1      | 93%   | 64             |
| src\tg_video_filter\db.py                   | 256   | 8    | 40     | 3      | 96%   | 512, 651, 661, |
| 1103-1104, 1147, 1172, 1382                 |       |      |        |        |       |                |
| src\tg_video_filter\delivery.py             | 84    | 4    | 30     | 2      | 95%   | 115-117,       |
| 137->140, 188                               |       |      |        |        |       |                |
| src\tg_video_filter\filters\__init__.py     | 39    | 0    | 18     | 0      | 100%  |                |
| src\tg_video_filter\filters\base.py         | 26    | 0    | 0      | 0      | 100%  |                |
| src\tg_video_filter\filters\content_type.py | 23    | 0    | 12     | 0      | 100%  |                |
| src\tg_video_filter\filters\duration.py     | 15    | 0    | 6      | 0      | 100%  |                |
| src\tg_video_filter\filters\resolution.py   | 13    | 0    | 6      | 0      | 100%  |                |
| src\tg_video_filter\filters\size.py         | 18    | 0    | 6      | 0      | 100%  |                |
| src\tg_video_filter\manager.py              | 36    | 0    | 8      | 0      | 100%  |                |
| src\tg_video_filter\models.py               | 125   | 1    | 8      | 1      | 98%   | 276            |
| src\tg_video_filter\telethon_client.py      | 62    | 3    | 26     | 4      | 92%   | 69, 86->99,    |
| 309, 324, 355->exit                         |       |      |        |        |       |                |



```
-----
TOTAL                                998    29    250    13    97%
Required test coverage of 80.0% reached. Total coverage: 96.63%
434 passed in 5.36s
```

### 36. assistant (2026-07-06T05:59:53.404Z)

Gate results: **\*\*ruff check clean\*\*, \*\*pytest 434 passed @ 96.63%\*\*** (floor 80% ?, matches the PR's claim). But **\*\*`ruff format --check` fails on `commands.py`\*\*** ? which the PR doesn't touch, so it's pre-existing on `main` (the PR only qualified "clean on changed files"). Let me confirm that and inspect the new tests + backfill dedup flow.

### 37. user (2026-07-06T06:00:07.657Z)

```
=== is commands.py changed by PR? ===
(empty above = unchanged by PR ? format issue is pre-existing on main)
```

```
=== confirm on main directly ===
Would reformat: .commands_main_check.py
1 file would be reformatted
```

### 38. user (2026-07-06T06:00:08.343Z)

```
=== backfill.py dedup section (155-235) ===
    if meta is None:
        continue

    channel_id = _peer_id(getattr(message, "peer_id", None) or 0)
    message_id = message.id
    document_id = _extract_document_id(message) # type: ignore[arg-type]

    # --- three-tier dedup (cheapest first) ---

    # Tier 0: same (user, channel, message) instance.
    existing = await db.get_video(conn, user.id, channel_id, message_id)
    if existing is not None:
        continue

    # Tier 1: same Telegram document.id (forwarded / cross-posted file).
    if document_id is not None:
        existing = await db.get_video_by_document_id(conn, user.id, document_id)
        if existing is not None:
            log.info(
                "backfill_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
                "original_channel=%s original_msg=%s",
                user.id,
                document_id,
                channel_id,
                message_id,
                existing.channel_id,
                existing.message_id,
            )
            continue

    # Tier 2: metadata fingerprint (re-uploaded identical file).
    if (
        meta.size_bytes is not None
        and meta.duration_s is not None
        and meta.height is not None
        and meta.width is not None
    ):
        existing = await db.get_video_by_content_fingerprint(
            conn,
            user.id,
```

```

        meta.size_bytes,
        meta.duration_s,
        meta.height,
        meta.width,
    )
    if existing is not None:
        log.info(
            "backfill_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
            "channel=%s msg=%s original_channel=%s original_msg=%s",
            user.id,
            meta.size_bytes,
            meta.duration_s,
            meta.height,
            meta.width,
            channel_id,
            message_id,
            existing.channel_id,
            existing.message_id,
        )
        continue

    summary["videos_scanned"] += 1
    dialog_videos += 1

    result = engine.evaluate(meta)
    record = VideoRecord(
        user_id=user.id,
        channel_id=channel_id,
        message_id=message_id,
        document_id=document_id, # type: ignore[arg-type]
        duration_s=meta.duration_s,
        width=meta.width,
        height=meta.height,
        size_bytes=meta.size_bytes,
        mime_type=meta.mime_type,
        matched=result.matched,
    )
    inserted = await db.insert_video(conn, record)

    if result.matched:
        summary["videos_matched"] += 1

```

### 39. user (2026-07-06T06:00:08.363Z)

=== new/changed test fns ===

```

44:async def test_init_db_creates_tables(tmp_path: object) -> None:
72:async def test_upsert_and_get_user(conn: aiomysqlite.Connection) -> None:
82:async def test_get_user_returns_none_when_absent(conn: aiomysqlite.Connection) -> None:
86:async def test_upsert_user_updates_existing(conn: aiomysqlite.Connection) -> None:
96:async def test_list_enabled_users_filters_disabled(conn: aiomysqlite.Connection) -> None:
111:async def test_upsert_and_get_account(conn: aiomysqlite.Connection) -> None:
124:async def test_get_account_returns_none_when_absent(conn: aiomysqlite.Connection) -> None:
128:async def test_get_account_by_telegram_id_resolves_bot_from_id(
140:async def test_upsert_and_get_telegram_session(conn: aiomysqlite.Connection) -> None:
160:async def test_list_runnable_sessions_excludes_disabled_and_bot_only(
181:async def test_bot_only_account_has_no_session_and_is_not_a_user(
202:async def test_upsert_channel_inserts_and_is_idempotent(conn: aiomysqlite.Connection) -> None:
221:async def test_get_video_absent(conn: aiomysqlite.Connection) -> None:
225:async def test_insert_and_get_video_round_trip(conn: aiomysqlite.Connection) -> None:
252:async def test_insert_video_dedup_collision(conn: aiomysqlite.Connection) -> None:
261:async def test_mark_forwarded_is_noop_post_adr_0011(conn: aiomysqlite.Connection) -> None:
284:async def test_list_videos_for_user_orders_desc(conn: aiomysqlite.Connection) -> None:
295:async def test_video_record_dedup_key() -> None:
297:    assert v.dedup_key == (1, 2, 3)
300:async def test_video_record_content_fingerprint() -> None:

```

```

310:     assert v.content_fingerprint == (5_000_000, 120, 1080, 1920)
313:async def test_video_record_content_fingerprint_none_when_missing_field() -> None:
324:     assert v.content_fingerprint is None
327:# --- content dedup (Tiers 1 & 2) -----
330:async def test_get_video_by_document_id_hit(conn: aiomysqlite.Connection) -> None:
347:async def test_get_video_by_document_id_miss(conn: aiomysqlite.Connection) -> None:
352:async def test_insert_video_document_id_collision(conn: aiomysqlite.Connection) -> None:
353:     """Same document_id for the same user ? IntegrityError (Tier-1 dedup at DB level)."""
377:async def test_insert_video_null_document_id_no_collision(conn: aiomysqlite.Connection) ->
None:
402:async def test_get_video_by_content_fingerprint_hit(conn: aiomysqlite.Connection) -> None:
416:     found = await db.get_video_by_content_fingerprint(
428:async def test_get_video_by_content_fingerprint_miss(conn: aiomysqlite.Connection) -> None:
432:     await db.get_video_by_content_fingerprint(
447:async def test_load_filter_config_defaults_when_absent(conn: aiomysqlite.Connection) -> None:
453:async def test_save_and_load_filter_config_round_trip(conn: aiomysqlite.Connection) -> None:
463:async def test_save_filter_config_overwrites(conn: aiomysqlite.Connection) -> None:
471:async def test_filter_config_with_null_bounds_round_trips(conn: aiomysqlite.Connection) ->
None:
485:async def test_delivery_mode_round_trips(conn: aiomysqlite.Connection) -> None:
496:async def test_filter_config_rejects_inverted_duration_bounds() -> None:
506:def test_filter_config_accepts_equal_min_max_duration() -> None:
515:async def test_old_config_without_delivery_mode_defaults_to_forward(
535:async def test_delivery_target_round_trips(conn: aiomysqlite.Connection) -> None:
544:async def test_old_config_without_delivery_target_defaults_to_me(
563:def test_filter_config_coerces_int_delivery_target() -> None:
571:def test_filter_config_rejects_empty_delivery_target() -> None:
581:async def test_pragmas_are_set_on_init(conn: aiomysqlite.Connection) -> None:
595:async def test_foreign_key_enforcement_rejects_orphan_video(
608:async def test_fresh_db_has_document_id_column(conn: aiomysqlite.Connection) -> None:
612:     superseded by ``UNIQUE(account_id, dedup_key)`` on ``content_items``; the
617:     # The provider-neutral dedup uniqueness lives on content_items, not videos.
621:     # at least one unique index exists (the dedup_key uniqueness).
622:     assert any("dedup_key" in idx or "sqlite_autoindex" in idx for idx in idx_names,
idx_names
625:async def test_fresh_db_user_version_is_latest(conn: aiomysqlite.Connection) -> None:
634:async def test_migration_adds_document_id_to_legacy_db(tmp_path: object) -> None:
752:     "duration_s, width, height, size_bytes, dedup_key FROM content_items"
755:     # dedup_key falls to the Tier-0 branch (no document_id, but the
756:     # fingerprint fields are all present) ? actually the fingerprint branch
764:     assert ci[8] == "tg:fp:1000000:60:1080:1920" # fingerprint dedup_key
778:async def test_migration_002_retargeted_fks_allow_inserts_for_new_accounts(
822:async def test_migration_002_is_idempotent_on_fresh_db(tmp_path: object) -> None:
833:async def test_migration_is_idempotent(tmp_path: object) -> None:
848:async def test_document_id_unique_index_enforces_dedup(conn: aiomysqlite.Connection) -> None:
874:# --- compute_dedup_key (ADR 0011, Option 1A) -----
877:def test_compute_dedup_key_telegram_document_id_wins() -> None:
879:     key = db.compute_dedup_key(
889:     assert key == "tg:doc:12345"
892:def test_compute_dedup_key_telegram_fingerprint_when_no_document_id() -> None:
893:     """Tier-2 (fingerprint) applies when document_id is absent AND all four
894:     metadata fields are present (the guard against degenerate tg:fp:0:0:0:0)."""
895:     key = db.compute_dedup_key(
902:     assert key == "tg:fp:1000:60:1080:1920"
905:def test_compute_dedup_key_telegram_tier0_when_partial_metadata() -> None:
906:     """Tier-0 (message id) applies when fingerprint is incomplete ? prevents
908:     key = db.compute_dedup_key(
915:         width=None, # missing ? fingerprint branch skipped
917:     assert key == "tg:msg:7:42"
920:def test_compute_dedup_key_telegram_tier0_when_all_metadata_missing() -> None:
922:     key = db.compute_dedup_key(provider="telegram", source_id=7, provider_message_id="42")
923:     assert key == "tg:msg:7:42"
926:def test_compute_dedup_key_telegram_empty_document_id_falls_through() -> None:

```

Confirmed: the `commands.py` format issue is **\*\*pre-existing on `main`\*\*** (not this PR), and `backfill.py` shares the identical three-tier dedup flow ? so the Tier?2 regression hits both the live and backfill paths. Now let me read the fingerprint tests and the migration dedup test to see whether the regression is covered.

#### 41. user (2026-07-06T06:00:22.465Z)

```
327 # --- content dedup (Tiers 1 & 2) -----
328
329
330 async def test_get_video_by_document_id_hit(conn: aiomysqlite.Connection) -> None:
331     await _seed_user(conn)
332     await db.insert_video(
333         conn,
334         VideoRecord(
335             user_id=USER_ID,
336             channel_id=CHANNEL_ID,
337             message_id=1,
338             document_id=555000111,
339         ),
340     )
341     found = await db.get_video_by_document_id(conn, USER_ID, 555000111)
342     assert found is not None
343     assert found.document_id == 555000111
344     assert found.message_id == 1
345
346
347 async def test_get_video_by_document_id_miss(conn: aiomysqlite.Connection) -> None:
348     await _seed_user(conn)
349     assert await db.get_video_by_document_id(conn, USER_ID, 999999) is None
350
351
352 async def test_insert_video_document_id_collision(conn: aiomysqlite.Connection) -> None:
353     """Same document_id for the same user ? IntegrityError (Tier-1 dedup at DB
354 level)."""
355     await _seed_user(conn)
356     await db.insert_video(
357         conn,
358         VideoRecord(
359             user_id=USER_ID,
360             channel_id=CHANNEL_ID,
361             message_id=10,
362             document_id=123456789,
363         ),
364     )
365     # Same doc_id, different channel/message ? must collide on the index.
366     with pytest.raises(sqlite3.IntegrityError):
367         await db.insert_video(
368             conn,
369             VideoRecord(
370                 user_id=USER_ID,
371                 channel_id=-1009999999,
372                 message_id=99,
373                 document_id=123456789,
374             ),
375         )
376
377 async def test_insert_video_null_document_id_no_collision(conn: aiomysqlite.Connection) ->
378 None:
379     """Multiple rows with document_id=NULL do NOT collide (partial unique index)."""
380     await _seed_user(conn)
381     await db.insert_video(
382         conn,
383         VideoRecord(
```

```

383         user_id=USER_ID,
384         channel_id=CHANNEL_ID,
385         message_id=20,
386         document_id=None,
387     ),
388 )
389 # Same user, no document_id, different message ? should succeed.
390 inserted = await db.insert_video(
391     conn,
392     VideoRecord(
393         user_id=USER_ID,
394         channel_id=CHANNEL_ID,
395         message_id=21,
396         document_id=None,
397     ),
398 )
399 assert inserted.id is not None
400
401
402 async def test_get_video_by_content_fingerprint_hit(conn: aiosqlite.Connection) -> None:
403     await _seed_user(conn)
404     await db.insert_video(
405         conn,
406         VideoRecord(
407             user_id=USER_ID,
408             channel_id=CHANNEL_ID,
409             message_id=5,
410             size_bytes=10_000_000,
411             duration_s=180,
412             height=720,
413             width=1280,
414         ),
415     )
416     found = await db.get_video_by_content_fingerprint(
417         conn,
418         USER_ID,
419         10_000_000,
420         180,
421         720,
422         1280,
423     )
424     assert found is not None
425     assert found.message_id == 5
426
427
428 async def test_get_video_by_content_fingerprint_miss(conn: aiosqlite.Connection) ->
None:
429     await _seed_user(conn)
430     # No video inserted ? should be None.
431     assert (
432         await db.get_video_by_content_fingerprint(
433             conn,
434             USER_ID,
435             1,
436             1,
437             1,
438             1,
439         )
440         is None
441     )
442
443
444 # --- filter config repos -----
445
446

```

#### 42. assistant (2026-07-06T06:00:32.701Z)

The fingerprint "hit" test seeds a video with **no** `document_id`, so it never exercises the regression path. Let me write an empirical repro against the actual `db.py` to confirm the Tier2 dedup break with a concrete failing scenario.

#### 43. assistant (2026-07-06T06:00:44.928Z)

```
"""Repro: Tier-2 (fingerprint) dedup regression in ADR 0011 shims.
```

```
Scenario mirrors telethon_client.py's three-tier flow:
```

```
    original video: document_id=X, fingerprint F    (the common case ? TG videos have a doc id)
```

```
    re-upload:      document_id=Y (new), same fingerprint F
```

```
v1 behaviour: get_video_by_content_fingerprint(F) finds the original -> dedup (skip).
```

```
Post-ADR-0011: it returns None -> the re-upload is treated as new -> forwarded twice.
```

```
"""
```

```
import asyncio
```

```
import aiosqlite
```

```
from tg_video_filter import db
```

```
from tg_video_filter.models import Account, VideoRecord
```

```
USER = 42
```

```
CH = 1001
```

```
F = dict(size_bytes=10_000_000, duration_s=180, height=1080, width=1920)
```

```
async def main() -> None:
```

```
    conn = await aiosqlite.connect(":memory:")
```

```
    await db._apply_schema(conn)
```

```
    await db.upsert_account(conn, Account(id=USER, telegram_id=USER))
```

```
    # Original: HAS a document_id + full fingerprint.
```

```
    await db.insert_video(
```

```
        conn,
```

```
        VideoRecord(user_id=USER, channel_id=CH, message_id=1, document_id=111, **F),
```

```
    )
```

```
    # Re-upload arrives on a different channel with a NEW document_id, same fingerprint.
```

```
    # Tier 1 (by new document_id) correctly misses:
```

```
    t1 = await db.get_video_by_document_id(conn, USER, 222)
```

```
    # Tier 2 (by fingerprint) SHOULD find the original (v1 semantics):
```

```
    t2 = await db.get_video_by_content_fingerprint(conn, USER, **F)
```

```
    print("Tier-1 (new doc id) hit:", t1 is not None, "(expected False)")
```

```
    print("Tier-2 (fingerprint) hit:", t2 is not None, "(v1 expected True -> dedup)")
```

```
    if t2 is None:
```

```
        # Nothing stops the duplicate insert -> it gets stored & would be forwarded.
```

```
        dup = await db.insert_video(
```

```
            conn,
```

```
            VideoRecord(user_id=USER, channel_id=2002, message_id=9, document_id=222, **F),
```

```
        )
```

```
        print("Duplicate re-upload inserted as NEW row id:", dup.id,
```

```
              "-> would be delivered again (REGRESSION)")
```

```
    await conn.close()
```

```
asyncio.run(main())
```

#### 44. user (2026-07-06T06:00:44.964Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro\_dedup.py (file

state is current in your context ? no need to Read it back)

#### 45. user (2026-07-06T06:00:54.831Z)

Tier-1 (new doc id) hit: False (expected False)

Tier-2 (fingerprint) hit: False (v1 expected True -> dedup)

Duplicate re-upload inserted as NEW row id: 2 -> would be delivered again (REGRESSION)

#### 46. assistant (2026-07-06T06:01:25.266Z)

Empirically **\*\*CONFIRMED\*\***: the re-upload is not deduplicated (Tier2 returns `None`) and inserts as a new row ? it would be forwarded twice. This is a real regression from v1's advertised three-tier dedup.

I have a strong primary finding plus the local gate results. Now, per the exhaustive-review mandate, let me launch a multi-agent workflow to independently verify this and sweep the remaining dimensions (migration safety, shims, SQL/schema integrity, tests, ADR conformance) with an adversarial verification pass.

#### 47. user (2026-07-06T06:01:27.481Z)

```
=== migration dedup-branch test (700-780) ===
```

```
    rows = await cur.fetchall()
    assert rows == [(111, 111, "telegram")]
    # telegram_sessions row derived from the legacy user row.
    async with conn.execute(
        "SELECT account_id, phone, session_path, enabled FROM telegram_sessions"
    ) as cur:
        srows = await cur.fetchall()
    assert srows == [(111, "+1", "user_111.session", 1)]
```

```
# Domain schema (migration 003): legacy tables renamed to *_deprecated,
# provider-neutral tables backfilled from them.
assert await db._table_exists(conn, "sources")
assert await db._table_exists(conn, "content_items")
assert await db._table_exists(conn, "policies")
assert await db._table_exists(conn, "channels_deprecated")
assert await db._table_exists(conn, "videos_deprecated")
assert await db._table_exists(conn, "filter_config_deprecated")
assert not await db._table_exists(conn, "channels")
assert not await db._table_exists(conn, "videos")
assert not await db._table_exists(conn, "filter_config")
```

```
# FK contract (PR #33 review): the rebuilt legacy tables (now
# *_deprecated) reference accounts(id), NOT users_deprecated(id).
for tbl in ("channels_deprecated", "videos_deprecated", "filter_config_deprecated"):
    async with conn.execute(f"PRAGMA foreign_key_list({tbl})") as cur:
        fk_rows = await cur.fetchall()
        referenced_tables = {r[2] for r in fk_rows}
        assert "accounts" in referenced_tables, f"{tbl} FK does not reference accounts"
        assert (
            "users_deprecated" not in referenced_tables
        ), f"{tbl} FK still references users_deprecated (rename-rewrite bug)"
```

```
# Legacy data preserved in the deprecated tables.
async with conn.execute(
    "SELECT message_id, document_id, user_id FROM videos_deprecated"
) as cur:
    vrows = await cur.fetchall()
    assert vrows == [(1, None, 111)]
    async with conn.execute(
        "SELECT user_id, channel_id, title FROM channels_deprecated"
    ) as cur:
        crows = await cur.fetchall()
```

```

assert crows == [(111, -1001, "pre-migration")]

# Domain backfill (migration 003): sources + content_items populated.
async with conn.execute(
    "SELECT account_id, provider, provider_source_id, title, watched " "FROM sources"
) as cur:
    src_rows = await cur.fetchall()
assert src_rows == [(111, "telegram", "-1001", "pre-migration", 1)]
async with conn.execute(
    "SELECT account_id, source_id, provider_message_id, document_id, "
    "duration_s, width, height, size_bytes, dedup_key FROM content_items"
) as cur:
    ci_rows = await cur.fetchall()
# dedup_key falls to the Tier-0 branch (no document_id, but the
# fingerprint fields are all present) ? actually the fingerprint branch
# wins here since size/duration/h/w are all set.
assert len(ci_rows) == 1
ci = ci_rows[0]
assert ci[0] == 111 # account_id
assert ci[2] == "1" # provider_message_id (cast to TEXT)
assert ci[3] is None # document_id
assert ci[4:8] == (60, 1920, 1080, 1000000) # dur, w, h, size
assert ci[8] == "tg:fp:1000000:60:1080:1920" # fingerprint dedup_key

# Policy backfill (migration 003): one default policy per account.
async with conn.execute(
    "SELECT account_id, name, config_json, is_default FROM policies"
) as cur:
    pol_rows = await cur.fetchall()
assert pol_rows == [(111, "default", "{}", 1)]

# user_version bumped to the highest registered migration.
async with conn.execute("PRAGMA user_version") as cur:
    assert (await cur.fetchone())[0] == len(db._MIGRATIONS)

async def test_migration_002_retargeted_fks_allow_inserts_for_new_accounts(
    tmp_path: object,
) -> None:

```

#### 48. assistant (2026-07-06T06:01:47.961Z)

The migration test seeds only one row (single branch) ? thinner than ADR 0011 mandated ("seed rows hitting each branch, assert no two collapse"). I now have enough to launch the exhaustive review workflow. Let me run it.

#### 49. user (2026-07-06T06:02:53.340Z)

Workflow launched in background. Task ID: wbcn4k7ne  
Summary: Exhaustive multi-dimensional review of telegram-video-filter PR #35 (ADR 0011 provider-neutral schema) with adversarial verification  
Transcript dir: C:\Users\avidu\claud\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_eb99bd9f-440  
Script file: C:\Users\avidu\claud\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr35-domain-schema-wf\_eb99bd9f-440.js  
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\claud\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr35-domain-schema-wf\_eb99bd9f-440.js"} to iterate without resending the script.)  
Run ID: wf\_eb99bd9f-440  
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\claud\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr35-domain-schema-wf\_eb99bd9f-440.js", resumeFromRunId: "wf\_eb99bd9f-440"}) ? completed agents return cached results.



You will be notified when it completes. Use /workflows to watch live progress.

## 50. assistant (2026-07-06T06:03:18.231Z)

Workflow is running in the background. While it works, let me independently verify a couple of things the agents will also probe ? most importantly whether the `mark\_forwarded` no-op can cause re-delivery (is `forwarded` ever read to gate delivery?), and the timestamp-format inconsistency.

## 51. user (2026-07-06T06:03:22.300Z)

```
=== any READ of .forwarded / forwarded= to gate delivery? ===
src/tg_video_filter/db.py:72:    forwarded    INTEGER NOT NULL DEFAULT 0,
src/tg_video_filter/db.py:455:    forwarded    INTEGER NOT NULL DEFAULT 0,
src/tg_video_filter/db.py:461:    "height, size_bytes, mime_type, matched, forwarded,
seen_at",
src/tg_video_filter/db.py:673:    forwarded=bool(row["forwarded"]),
src/tg_video_filter/db.py:1233:    """Deprecated shim: no-op for the legacy ``forwarded`` flag
(ADR 0011).
src/tg_video_filter/db.py:1252:    ``forwarded`` is always ``False`` post-ADR-0011 (the column
is dropped;
src/tg_video_filter/delivery.py:10:``seconds + jitter``, retry once) and mark the DB row
``forwarded=1`` on success.
src/tg_video_filter/delivery.py:139:    log.info("video_forwarded video=%s target=%s",
video_id, target)
src/tg_video_filter/models.py:322:    forwarded: bool = False
src/tg_video_filter/telethon_client.py:320:    same ``document.id`` when a file is forwarded
across channels.
```

```
=== startup re-delivery routine? (unforwarded/pending/redeliver) ===
src/tg_video_filter/bootstrap.py:91:    2. Build a Telethon client with a file session at
``user_<pending>.session``.
src/tg_video_filter/bootstrap.py:106:    # Clean up any stale _pending session from a previous
failed/aborted run,
src/tg_video_filter/bootstrap.py:108:    for stale in sessions_dir.glob("_pending.session*"):
src/tg_video_filter/bootstrap.py:112:    pending_path = sessions_dir / "_pending"
src/tg_video_filter/bootstrap.py:113:    client = TelegramClient(str(pending_path),
settings.tg_api_id, settings.tg_api_hash)
src/tg_video_filter/bootstrap.py:138:    pending_path.with_suffix(suffix)
src/tg_video_filter/bootstrap.py:140:    else Path(str(pending_path) + suffix)
src/tg_video_filter/db.py:164:    status        TEXT NOT NULL DEFAULT 'pending',
src/tg_video_filter/db.py:185:    then runs any pending migrations via the ``user_version``
ladder.
```

```
=== delivery.py forward flow (100-160) ===
```

```
async def _send_with_retry(
    client: TelegramClient,
    send_callable: Callable[[], Awaitable[None]],
    video_id: int,
) -> bool:
    """Execute a send/forward callable with FloodWait handling. Returns success."""
    try:
        await send_callable()
    except FloodWaitError as e:
        wait = e.seconds + random.uniform(*_FLOOD_JITTER_RANGE)
        log.warning("flood_wait video=%s seconds=%s retrying_after=%.1f", video_id, e.seconds,
wait)
        await asyncio.sleep(wait)
    try:
        await send_callable()
    except Exception:
        log.exception("delivery_retry_failed video=%s", video_id)
        return False
    except Exception:
```

```

        log.exception("delivery_failed video=%s", video_id)
        return False
    return True

async def forward_to_target(
    client: TelegramClient,
    message: Message,
    conn: aiosqlite.Connection,
    video_id: int,
    target: str = "me",
) -> bool:
    """Forward ``message`` to ``target``, marking the DB row on success.

    ``target`` is a Telethon entity: ``"me"`` (Saved Messages), ``"@username"`` ,
    or a numeric chat id string. See ADR 0005.
    """
    ok = await _send_with_retry(client, lambda: client.forward_messages(target, message),
    video_id)
    if ok:
        await db.mark_forwarded(conn, video_id)
        log.info("video_forwarded video=%s target=%s", video_id, target)
    return ok

async def send_link_message(
    client: TelegramClient,
    text: str,
    conn: aiosqlite.Connection,
    video_id: int,
    target: str = "me",
) -> bool:
    """Send a link/summary text message to ``target``, marking the DB row.

    ``target`` is a Telethon entity; see :func:`forward_to_target`.
    """
    ok = await _send_with_retry(client, lambda: client.send_message(target, text), video_id)
    if ok:
        await db.mark_forwarded(conn, video_id)
        log.info("link_sent video=%s target=%s", video_id, target)
    return ok

```

## 52. assistant (2026-07-06T06:03:40.265Z)

`forwarded` is never read to gate delivery and there's no startup re-delivery query ? so the `mark\_forwarded` no-op is genuinely safe for Phase 1 (PR's claim holds). Let me confirm whether `\_row\_to\_video` / `\_row\_to\_channel` are now dead code, and settle the timestamp-format question empirically.

## 53. user (2026-07-06T06:03:48.793Z)

=== `\_row\_to\_video` / `\_row\_to\_channel` call sites (excl. defs) ===  
(if empty ? dead code)

=== timestamp parse: does pydantic accept CURRENT\_TIMESTAMP format for Policy? ===  
 parsed created\_at = datetime.datetime(2026, 7, 6, 5, 19, 48) tzinfo= None  
 isoformat roundtrip = 2026-07-06T05:19:48

## 54. assistant (2026-07-06T06:04:07.952Z)

Two more confirmed observations:

- **Dead code:** `\_row\_to\_video` / `\_row\_to\_channel` (db.py:660, 650) have no call sites left after the shims were rewritten ? low-severity cleanup.
- **Timestamp inconsistency:** migration uses SQL `CURRENT\_TIMESTAMP` ? **naive** datetime,

while everywhere else uses tz-aware ISO. Parses fine today, but a future `aware ? naive` subtraction would raise. Low/latent.

Let me settle the fresh-DB double-init idempotency directly (the subtle new `\_apply\_schema` drop logic) while the workflow runs.

#### 55. user (2026-07-06T06:04:19.020Z)

```
user_version = 3
tables = ['accounts', 'channels_deprecated', 'content_items', 'delivery_attempts',
'destinations', 'filter_config_deprecated', 'policies', 'routing_rules', 'sources',
'sqlite_sequence', 'telegram_sessions', 'videos_deprecated']
base 'channels' exists: False (expected False)
base 'videos' exists: False (expected False)
base 'filter_config' exists: False (expected False)
'channels_deprecated' exists: True (expected True)
'videos_deprecated' exists: True (expected True)
'filter_config_deprecated' exists: True (expected True)
```

#### 56. assistant (2026-07-06T06:04:27.874Z)

Fresh-DB triple-init is clean: `user\_version=3`, legacy base tables dropped, `\*\_deprecated` retained, no errors ? the new `\_apply\_schema` drop logic holds. My independent findings so far:

1. **\*\*[High] Tier22 dedup regression\*\*** (empirically confirmed) ? re-uploads with a new `document\_id` but identical fingerprint are no longer deduplicated; contradicts README's advertised 3-tier dedup and ADR 0011's "no loss of dedup quality" claim.
2. **\*\*[Low] Timestamp inconsistency\*\*** ? migration writes naive `CURRENT\_TIMESTAMP`; rest of code is tz-aware.
3. **\*\*[Low] Dead code\*\*** ? `\_row\_to\_video` / `\_row\_to\_channel` unused after shim rewrite.
4. **\*\*[Pre-existing, not this PR] `ruff format`\*\*** fails on `commands.py`.

Let me check the workflow's progress.

#### 57. user (2026-07-06T06:04:28.548Z)

<tool\_use\_error>Blocked: sleep 45 followed by: echo "waited". To wait for a condition, use Monitor with an until-loop (e.g. `until <check>; do sleep 2; done`). To wait for a command you started, use run\_in\_background: true. Do not chain shorter sleeps to work around this block.</tool\_use\_error>

#### 58. assistant (2026-07-06T06:04:31.997Z)

I'll wait for the workflow notification rather than polling. Meanwhile, let me read the `test\_delivery.py` changes to independently assess that part of the diff.

#### 59. user (2026-07-06T06:04:34.242Z)

```
=== test_delivery.py diff vs main ===
diff --git a/tests/test_delivery.py b/tests/test_delivery.py
index 61d42fa..e74206e 100644
--- a/tests/test_delivery.py
+++ b/tests/test_delivery.py
@@ -227,15 +227,25 @@ async def test_deliver_flood_then_retry(
     assert client.forward_calls == 2 # initial + 1 retry

-async def test_deliver_marks_forwarded_in_db(
+async def test_deliver_succeeds_without_forwarded_flag_post_adr_0011(
     db_with_video: tuple[aiosqlite.Connection, int],
 ) -> None:
+    """ADR 0011: delivery succeeds but ``forwarded`` stays False (no-op shim).
+
+    Post-migration-003 there is no ``forwarded`` column on ``content_items``;

```

```

+ per-destination delivery state moves to ``delivery_attempts`` in Track C.
+ For Phase 1 ``mark_forwarded`` is a no-op, so the projected
+ ``VideoRecord.forwarded`` is always False ? but delivery still returns True.
+ """
+ c, vid = db_with_video
+ client = FakeClient()
- await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.LINK, link_text="test") # type:
ignore[arg-type]
+ ok = await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.LINK, link_text="test") #
type: ignore[arg-type]
+ assert ok is True
+ row = await db.get_video(c, USER_ID, CHANNEL_ID, MSG_ID)
+ assert row is not None
- assert row.forwarded is True
+ # forwarded is always False post-ADR-0011 (the column is dropped; delivery
+ # state moves to Track C). The match is still recorded via `matched`.
+ assert row.forwarded is False

```

```

async def test_both_mode_logs_warning_on_link_failure(

```

#### 60. assistant (2026-07-06T06:04:59.273Z)

The `test\_delivery.py` change faithfully adapts the old `forwarded is True` assertion to the new no-op reality ? reasonable. Let me empirically test one more migration edge the agents are probing: whether an **orphan video** (a `videos` row whose `channel\_id` has no `channels` row) is silently dropped by the backfill's INNER JOIN + `INSERT OR IGNORE`.

#### 61. user (2026-07-06T06:05:10.177Z)

```

legacy videos count      : 2 (2 expected)
content_items migrated   : [('5', '777', 'tg:doc:777')]
MIGRATED 1 of 2 videos -> ORPHAN DROPPED (data loss)

```

#### 62. assistant (2026-07-06T06:05:31.939Z)

**Orphan-video data loss confirmed:** a legacy `videos` row whose `channel\_id` has no `channels` row (structurally possible in v1 ? `upsert\_channel` is best-effort and runs before `insert\_video`) is **silently dropped** by the backfill's INNER JOIN + `INSERT OR IGNORE`. It survives in `videos\_deprecated` (recoverable for one release) but vanishes from the live `content\_items` table ? so it's no longer dedup-tracked. Medium severity.

That's 2 confirmed bugs + several low/test-gap findings from my own pass. Let me check the workflow's progress once, then await its corroboration.

#### 63. user (2026-07-06T06:05:38.594Z)

```

-rw-r--r-- 1 avidu 197609      48 Jul  6 11:32 agent-a029016f8afd51e7d.meta.json
-rw-r--r-- 1 avidu 197609  67132 Jul  6 11:35 agent-a0ca3716211c360ea.jsonl
-rw-r--r-- 1 avidu 197609      48 Jul  6 11:34 agent-a0ca3716211c360ea.meta.json
-rw-r--r-- 1 avidu 197609 189871 Jul  6 11:34 agent-a1a6cac74f170afa1.jsonl
-rw-r--r-- 1 avidu 197609      48 Jul  6 11:32 agent-a1a6cac74f170afa1.meta.json
-rw-r--r-- 1 avidu 197609 230076 Jul  6 11:35 agent-a305eab4646413aeb.jsonl
-rw-r--r-- 1 avidu 197609      48 Jul  6 11:32 agent-a305eab4646413aeb.meta.json
-rw-r--r-- 1 avidu 197609  60733 Jul  6 11:35 agent-a5b992c9f86a3dd38.jsonl
-rw-r--r-- 1 avidu 197609      48 Jul  6 11:35 agent-a5b992c9f86a3dd38.meta.json
-rw-r--r-- 1 avidu 197609 196756 Jul  6 11:35 agent-a8308ed8ba75083d6.jsonl
-rw-r--r-- 1 avidu 197609      48 Jul  6 11:32 agent-a8308ed8ba75083d6.meta.json
-rw-r--r-- 1 avidu 197609  49079 Jul  6 11:35 agent-aa4f0d0193a24b340.jsonl
-rw-r--r-- 1 avidu 197609      48 Jul  6 11:35 agent-aa4f0d0193a24b340.meta.json
-rw-r--r-- 1 avidu 197609 206911 Jul  6 11:35 agent-ab67b20995535f4fc.jsonl
-rw-r--r-- 1 avidu 197609      48 Jul  6 11:32 agent-ab67b20995535f4fc.meta.json
-rw-r--r-- 1 avidu 197609 253630 Jul  6 11:34 agent-abf6521ed27c075a1.jsonl
-rw-r--r-- 1 avidu 197609      48 Jul  6 11:32 agent-abf6521ed27c075a1.meta.json

```

```
-rw-r--r-- 1 avidu 197609 57529 Jul 6 11:35 agent-afd0af3e039b59111.jsonl
-rw-r--r-- 1 avidu 197609 48 Jul 6 11:35 agent-afd0af3e039b59111.meta.jsonl
-rw-r--r-- 1 avidu 197609 12353 Jul 6 11:35 journal.jsonl
---latest agent files---
11
```

#### 64. assistant (2026-07-06T06:05:41.758Z)

The workflow has 6 reviewers + verifiers active (11 agent transcripts, growing). I'll await its completion notification rather than poll.

#### 65. user (2026-07-06T06:05:47.679Z)

```
2771285 feat(domain): provider-neutral schema ? sources/content_items/policies (ADR 0011, Track B)
--- confirming only PR commits on branch ---
277128521fa9e0af0d45a6a60e0da591ec90b053
```

#### 66. assistant (2026-07-06T06:05:50.764Z)

The branch cleanly contains only the single PR commit `2771285`. Now I'll wait for the workflow to finish its adversarial verification pass and report back.

#### 67. user (2026-07-06T06:05:53.784Z)

```
<tool_use_error>InputValidationError: Monitor failed due to the following issues:
The required parameter `description` is missing
An unexpected parameter `timeout_seconds` was provided
An unexpected parameter `until` was provided
```

This tool's schema was not sent to the API ? it was not in the discovered-tool set derived from message history. Without the schema in your prompt, typed parameters (arrays, numbers, booleans) get emitted as strings and the client-side parser rejects them. Load the tool first: call ToolSearch with query "select:Monitor", then retry this call. For reference, this tool's input schema is: {"\$schema":"https://json-schema.org/draft/2020-12/schema","type":"object","properties":{"description":{"description":"Short human-readable description of what you are monitoring (shown in notifications).","type":"string"},"timeout\_ms":{"description":"Kill the monitor after this deadline. Default 300000ms, max 3600000ms. Ignored when persistent is true.","default":300000,"type":"number","minimum":1000},"persistent":{"description":"Run for the lifetime of the session (no timeout). Use for session-length watches like PR monitoring or log tails. Stop with TaskStop.","default":false,"type":"boolean"},"command":{"description":"Shell command or script. Each stdout line is an event; exit ends the watch.","type":"string"},"ws":{"description":"WebSocket to open. Each text frame is an event; binary frames are reported as a placeholder line. Socket close ends the watch. Cannot be combined with command.","type":"object","properties":{"url":{"type":"string"},"protocols":{"type":"array","items":{"type":"string","pattern":"^!#\$%&'\*+.^\_`|~0-9A-Za-z-]+\$"}}},"required":["url"],"additionalProperties":false}},"required":["description","timeout\_ms","persistent"],"additionalProperties":false}</tool\_use\_error>

#### 68. assistant (2026-07-06T06:05:58.145Z)

I'll just await the workflow's completion notification instead of setting up a monitor.

#### 69. user (2026-07-06T06:06:03.403Z)

(Bash completed with no output)

#### 70. assistant (2026-07-06T06:06:19.995Z)

I've completed my own verification pass and the multi-agent workflow is finishing its adversarial verification. Here's where things stand while I await the workflow's corroboration:

**\*\*Sync & ramp-up (done):\*\*** Repo at `C:\Users\avidu\Projects\Telegram` synced to remote head

(clean tree; branch and `main` both match origin). One open PR: **\*\*#35\*\*** ? ADR 0011 provider-neutral schema (Track B), +1299/?150 across `db.py`, `models.py`, `test\_db.py`, `test\_delivery.py`. No GitHub CI exists, so I ran the gate locally.

**\*\*Local gate:\*\*** ? `ruff check` clean . ? pytest **\*\*434 passed @ 96.63%\*\*** (floor 80%) . ?? `ruff format --check` fails only on `commands.py`, which this PR doesn't touch (pre-existing on `main`).

**\*\*Confirmed findings (my pass, each reproduced):\*\***

- ? **\*\*Tier2 dedup regression\*\*** ? re-uploads with a new `document\_id` but identical fingerprint are no longer deduplicated (the fingerprint shim looks up `tg:fp:?` but doc-bearing items are stored as `tg:doc:~`). Contradicts the README's advertised 3-tier dedup.
- ? **\*\*Orphan-video silent data loss\*\*** in migration 003's backfill (INNER JOIN + `INSERT OR IGNORE` drops `videos` rows whose channel has no `channels` row).
- ? Naive-vs-aware timestamp inconsistency; dead code (`\_row\_to\_video`/`\_row\_to\_channel`); migration test covers only one dedup branch (ADR asked for all + a no-collapse assertion).

I'll wait for the workflow to finish and fold in anything it surfaces, then give you the consolidated review.

## 71. user (2026-07-06T06:14:56.376Z)

```
<task-notification>
<task-id>wbcn4k7ne</task-id>
<tool-use-id>toolu_01G7UgrVfBEJjQccmWZXBtsi</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\wbcn4k7ne.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Exhaustive multi-dimensional review of telegram-video-filter PR #35
(ADR 0011 provider-neutral schema) with adversarial verification" completed</summary>
<result>{"counts":{"raw":12,"confirmed":12,"plausible":0,"refuted":0},"confirmed":[{"dimension":
"migration","severity":"blocker","title":"Migration 003 content_items backfill silently drops
every video whose channel has no channels row (inner JOIN + INSERT OR
IGNORE)","where":"src/tg_video_filter/db.py:562","summary":"The content_items backfill inner-
joins videos to sources on channel_id, but sources is populated only from the channels table;
any legacy video whose channel_id was never inserted into channels (routine for backfill-scanned
channels) is silently discarded, so its content_item is never created.","failure_scenario":"v1
backfill.py inserts videos via db.insert_video() but never calls db.upsert_channel(), and v1
videos.channel_id has NO foreign key to channels (git main:db.py CREATE TABLE videos, bare
'channel_id INTEGER NOT NULL'). So a production v1 DB routinely holds videos for channels absent
from `channels`. On migration: sources is backfilled ONLY from channels (db.py:517-526), then
content_items is backfilled with `FROM videos v JOIN sources s ON ... s.provider_source_id =
CAST(v.channel_id AS TEXT)` (db.py:562-567). The inner JOIN drops every video whose channel has
no sources row; INSERT OR IGNORE masks it. Repro: seed users=111, channels only for -1001,
videos for -1001 (msg 1, doc 111000) AND -2002 (msg 2, doc 222000); run init_db -&gt;
videos_deprecated=2 but content_items=1 (only msg 1). get_video_by_document_id(111,222000)
returns None -&gt; the video is invisible to the live schema; dedup breaks (it re-delivers) and
after the one-release deprecated-table drop it is permanently
lost.","evidence":"src/tg_video_filter/db.py:562-567 (JOIN sources); db.py:517-526 (sources
backfilled only from channels); db.py:539 (INSERT OR IGNORE); v1 `git show
main:src/tg_video_filter/db.py` videos DDL has no FK to channels and insert_video does not
upsert a channel; `git show main:backfill.py | grep -c upsert_channel` == 0. Repro output:
'Legacy videos: 2 ... content_items rows (migrated): 1 ... DATA LOSS: 1 video(s) silently
dropped by the JOIN ... get_video_by_document_id(222000) -&gt; None'.","recommendation":"Do not
depend on the channels table for source existence. Before (or during) the content_items
backfill, backfill sources for every DISTINCT (user_id, channel_id) found in videos (INSERT OR
IGNORE INTO sources SELECT DISTINCT user_id,'telegram',CAST(channel_id AS TEXT),... FROM
videos), or LEFT JOIN and create the missing source, so no video is dropped. Add a migration
test seeding a video whose channel has no channels row and assert content_items count == videos
count.","verify_reasoning":"I read the migration and traced the full control flow, then
reproduced it. (1) v1 `videos.channel_id` has NO foreign key to `channels` ? `git show
main:db.py` CREATE TABLE videos is bare `channel_id INTEGER NOT NULL` (main db.py:277), and
`insert_video` (main db.py:670-699) inserts only into videos. (2) On main, `upsert_channel` is
called ONLY from the live-watch path `telethon_client.py:240`; the backfill path
(`backfill.py:232`) inserts videos via `db.insert_video` and never upserts a channel (0
```

occurrences of `upsert_channel` in `backfill.py`). So a v1 production DB routinely holds videos whose `channel_id` is absent from `channels` (any channel scanned by backfill but never live-watched). (3) Migration 003 backfills `sources` exclusively from `channels` (`db.py:517-526`; the only other `INSERT INTO sources` at `db.py:905` is the runtime upsert helper, not part of the migration), then backfills `content_items` with `FROM videos v JOIN sources s ON s.provider_source_id = CAST(v.channel_id AS TEXT)` (`db.py:562-567`) ? an inner JOIN ? under `INSERT OR IGNORE` (`db.py:539`). No LEFT JOIN, no fallback source creation. Every orphan-channel video is silently dropped. Repro (venv python, real `db._apply_schema` path from a `user_version=2` legacy DB seeded with account 111, a `channels` row only for -1001, and videos for both -1001 and orphan -2002): `videos_deprecated=2`, `sources=1`, `content_items=1`, and the orphan video's `content_item` (`document_id 222000`) is None. After the one-release deprecated-table drop these videos are permanently lost; in the interim dedup breaks and they can be re-delivered. This is genuinely in THIS PR's diff (migration 003 is new to `db.py` in this PR) and produces the described data loss.

["dimension":"dedup","severity":"high","title":"Tier-2 fingerprint dedup is regressed: re-uploads with a new document\_id are no longer deduped","where":"src/tg\_video\_filter/db.py:1151","summary":"get\_video\_by\_content\_fingerprint now looks up dedup\_key='tg:fp:S:D:H:W', but any prior video that had a document\_id was stored under 'tg:doc:X', so the fingerprint lookup misses it and a re-upload (identical bytes, new document.id) is admitted as new ? losing ADR 0006 Tier-2 dedup for the overwhelmingly common case (nearly all Telegram videos carry a document\_id).","failure\_scenario":"User previously saw video V in channel A: it had document\_id=111, size=5000000, dur=60, h=1080, w=1920 -& stored in content\_items with dedup\_key='tg:doc:111'. Someone re-encodes/re-uploads the identical file to channel B; Telegram assigns a NEW document\_id=222. telethon\_client.\_on\_message runs the tier chain: Tier-0 (channel B, new msg) misses; Tier-1 get\_video\_by\_document\_id(222) misses; Tier-2 get\_video\_by\_content\_fingerprint(5000000,60,1080,1920) computes/looks up dedup\_key='tg:fp:5000000:60:1080:1920' and MISSES (the prior row is under 'tg:doc:111'). All tiers report 'new', insert\_video(V') succeeds (dedup\_key 'tg:doc:222' collides with nothing), and the DUPLICATE video is stored AND forwarded to the delivery target. On main, get\_video\_by\_content\_fingerprint did `SELECT ... WHERE size/dur/h/w match` against ALL videos regardless of document\_id and would have found the prior row, returning early and suppressing the duplicate.", "evidence":"Repro on an in-memory DB: seed VideoRecord(document\_id=99999, size=5000000, dur=60, h=1080, w=1920) -& stored dedup\_key ['tg:doc:99999']; then get\_video\_by\_content\_fingerprint(5000000,60,1080,1920) returns None even though 1 row physically matches all four fingerprint columns, because the shim looks up key 'tg:fp:5000000:60:1080:1920' (`db.py:1159-1173`, `compute_dedup_key` branch order `db.py:286-291`). Full-chain repro: all three tiers miss and insert\_video succeeds -& content\_items ends with 2 rows (duplicate admitted/delivered). v1 behaviour: `git show main:src/tg_video_filter/db.py` `get_video_by_content_fingerprint = `SELECT * FROM videos WHERE user_id=? AND size_bytes=? AND duration_s=? AND height=? AND width=? LIMIT 1`` (matches ANY prior video incl. document-keyed). Caller `telethon_client.py:207-236` uses this as Tier-2. README.md:31-32 advertises 'Never forwards the same video twice ? three-tier content dedup'; docs/design.md:224-225 defines Tier-2 as catching 're-upload of an identical file with a new document.id' ? the exact case now broken. Root cause: dedup\_key is a single-tier discriminator (doc XOR fp XOR msg), so a fp-shaped lookup can never see a doc-keyed row; the 3 tiers are mutually exclusive storage keys, not 3 independent read paths as v1 had.", "recommendation":"Make the Tier-2 shim query the fingerprint COLUMNS rather than the fp-shaped dedup\_key, restoring v1 semantics: e.g. add `get_content_item_by_fingerprint(account_id, size_bytes, duration_s, height, width)` that does ``SELECT * FROM content_items WHERE account_id=? AND size_bytes=? AND duration_s=? AND height=? AND width=? LIMIT 1`` and have `get_video_by_content_fingerprint` delegate to it. (The dedup\_key UNIQUE constraint can stay as-is; it never enforced fingerprint uniqueness on main either ? Tier-2 was always a read-path check.) Also add a test that seeds a prior video WITH a document\_id and asserts the fingerprint lookup still hits ? the existing `test_get_video_by_content_fingerprint_hit` (`tests/test_db.py:402`) seeds a video with NO document\_id, so the prior row lands under 'tg:fp:...' an ... (truncated 43241 chars, full result in C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\wbcn4k7ne.output)</result>

<usage><agent\_count>18</agent\_count><subagent\_tokens>1119858</subagent\_tokens><tool\_uses>273</tool\_uses><duration\_ms>722994</duration\_ms></usage>

</task-notification>

## 72. assistant (2026-07-06T06:15:01.523Z)

The workflow completed: \*\*12 raw findings, all 12 CONFIRMED\*\* by the adversarial verifier (0 refuted). Let me read the full result to fold in everything.

### 73. user (2026-07-06T06:15:07.155Z)

Exit code 1

Traceback (most recent call last):

```
File "<string>", line 1, in <module>
import sys,json; d=json.load(sys.stdin); print('COUNTS:',d['counts']); print();
[print(f"[{i+1}] {f['severity'].upper():8} ({f['dimension']}) {f['title']}\n      where:
{f['where']}\n      summary: {f['summary']}\n") for i,f in enumerate(d['confirmed'])]
~^^^^^^^^^^^^
```

KeyError: 'counts'

### 74. user (2026-07-06T06:15:21.432Z)

Exit code 1

Traceback (most recent call last):

```
File "<string>", line 5, in <module>
print('COUNTS:',d['counts']); print()
~^^^^^^^^^^^^
```

KeyError: 'counts'

### 75. user (2026-07-06T06:15:28.385Z)

FILE STARTS WITH: '{\n "summary": "Exhaustive multi-dimensional review of telegram-video-filter  
PR #35 (ADR 0011 provider-neutral schema) w'  
LEN 78103  
counts at 281

### 76. user (2026-07-06T06:15:36.525Z)

COUNTS: {'raw': 12, 'confirmed': 12, 'plausible': 0, 'refuted': 0}

=====

[1] BLOCKER (migration) Migration 003 content\_items backfill silently drops every video whose channel has no channels row (inner JOIN + INSERT OR IGNORE)

where: src/tg\_video\_filter/db.py:562

rec : Do not depend on the channels table for source existence. Before (or during) the content\_items backfill, backfill sources for every DISTINCT (user\_id, channel\_id) found in videos (INSERT OR IGNORE INTO sources SELECT DISTINCT user\_id,'telegram',CAST(channel\_id

[2] HIGH (dedup) Tier-2 fingerprint dedup is regressed: re-uploads with a new document\_id are no longer deduped

where: src/tg\_video\_filter/db.py:1151

rec : Make the Tier-2 shim query the fingerprint COLUMNS rather than the fp-shaped dedup\_key, restoring v1 semantics: e.g. add get\_content\_item\_by\_fingerprint(account\_id, size\_bytes, duration\_s, height, width) that does `SELECT \* FROM content\_items WHERE account\_id=

[3] HIGH (migration) Migration 003 INSERT OR IGNORE silently drops distinct v1 videos that share a fingerprint (tg:fp:) dedup\_key

where: src/tg\_video\_filter/db.py:539

rec : Fingerprint collisions among pre-existing rows are data, not duplicates. Either detect collisions and fall the losing rows back to the Tier-0 tg:msg: key (which is unique per source\_id+message\_id) so no row is dropped, or log/surface a count of dropped rows in

[4] HIGH (tests) No test covers the Tier-2 cross-tier dedup case; the shim silently regresses re-upload dedup and ships uncaught

where: tests/test\_db.py:402

rec : Add a test that seeds a content item WITH a document\_id AND full fingerprint metadata, then asserts get\_video\_by\_content\_fingerprint(...) with the SAME fingerprint returns that item (mirroring v1's cross-tier catch). This test fails against the current shim an

[5] HIGH (adr\_conformance) Option 1A drops Tier-2 re-upload dedup whenever document\_id is present ? the ADR/README 'no dedup-quality loss' claim is false as implemented

where: src/tg\_video\_filter/db.py:285

rec : Either (a) store BOTH a doc key and a fingerprint key per item (e.g. a second unique dedup column / a dedup\_keys side table) and check both at ingest, or (b) have the ingest path



additionally probe the fingerprint key even when document\_id is set. Correct the

[6] MEDIUM (schema\_sql) Backfill INNER JOIN silently drops orphan videos, making them re-deliverable duplicates  
where: src/tg\_video\_filter/db.py:562  
rec : Either use a LEFT JOIN and synthesize/backfill the missing source row (e.g. INSERT the channel into sources first from DISTINCT videos.channel\_id, not only from channels), or add a post-backfill assertion that SELECT COUNT(\*) FROM content\_items equals COUNT ov

[7] LOW (migration) policies.created\_at/updated\_at written as space-separated timezone-naive CURRENT\_TIMESTAMP in migration, diverging from the ISO-8601 tz-aware format used everywhere else  
where: src/tg\_video\_filter/db.py:577  
rec : Write the migration timestamps in the same ISO-8601/UTC-aware format as the rest of the code, e.g. bind strftime('%Y-%m-%dT%H:%M:%S+00:00', 'now') or a Python \_now().isoformat() parameter, so all policies rows have a uniform tz-aware ISO representation.

[8] LOW (shims) Legacy VideoRecord shims cast provider\_source\_id / provider\_message\_id / document\_id with int(), which raises ValueError on any non-Telegram content\_item and breaks the provider-neutrality the PR ships  
where: src/tg\_video\_filter/db.py:1328  
rec : In the legacy shims, gate the int() casts on provider=='telegram' (or catch ValueError / carry the provider through) so a non-Telegram content\_item does not crash account-wide reads. The VideoRecord.channel\_id/message\_id/document\_id fields are int-typed, so th

[9] LOW (schema\_sql) Backfill fingerprint-collapse can drop a matched=1 video in favor of a matched=0 duplicate  
where: src/tg\_video\_filter/db.py:539  
rec : If the collapse is intended dedup, at minimum preserve the strongest state on conflict (e.g. backfill in matched DESC order, or after the INSERT run an UPDATE content\_items SET matched=1 WHERE dedup\_key IN (SELECT ... FROM videos\_deprecated WHERE matched=1)),

[10] LOW (tests) Delivery-side no-op test asserts only a hardcoded constant, not that the no-op is safe  
where: tests/test\_delivery.py:244  
rec : Either add `assert row.matched is True` to this delivery test to make it verify the surviving signal, or drop the redundant assertion and rely on the db-side test ? but do not present it as verifying the no-op is 'safe' when it only checks a constant.

[11] LOW (tests) Legacy-shim None-guard branches (orphan source, absent channel) are uncovered  
where: src/tg\_video\_filter/db.py:1147  
rec : Add a small test that inserts a content\_item then deletes its sources row (or otherwise forces get\_source\_by\_id to return None) and asserts the shims return None; add a get\_channel-miss test. Low priority ? the branches are defensive and near-unreachable under

[12] LOW (adr\_conformance) policies single-default UNIQUE partial index is specified by ADR 0011 but never created  
where: src/tg\_video\_filter/db.py:124  
rec : Add 'CREATE UNIQUE INDEX IF NOT EXISTS idx\_policies\_one\_default ON policies(account\_id) WHERE is\_default = 1' to \_SCHEMA (and, if legacy DBs matter, to migration 003), matching the idx\_videos\_user\_document pattern already used at db.py:338. Alternatively, if e

## 77. assistant (2026-07-06T06:16:19.934Z)

The workflow **\*\*confirmed all 12 findings (0 refuted)\*\*** and notably escalated the orphan-video issue to **\*\*BLOCKER\*\***, arguing v1's **\*backfill\*** path never calls ``upsert_channel`` ? so history-scanned videos routinely lack a ``channels`` row, making the data loss routine rather than an edge case. It also surfaced new angles. Per the "reproduce, don't trust claims" gate, let me verify the three most consequential new/escalated claims myself.

## 78. user (2026-07-06T06:16:33.405Z)

=== (A) does v1 backfill EVER upsert a channel? ===

```
>>> NO channel upsert in backfill.py ? backfill-scanned videos have NO channels row (orphan data loss is routine)
```

```
=== (C) confirm distinct-videos-sharing-fingerprint collapse on migration ===
legacy videos: 2  migrated content_items: [('10', 1, 'tg:fp:5000000:60:1080:1920')]
COLLAPSE | kept matched= [1]
```

```
=== (F) is there a UNIQUE partial index enforcing one default policy? ===
339:      "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
485:      "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
```

#### 79. assistant (2026-07-06T06:16:55.252Z)

All three verified: (A) `backfill.py` **never** creates a channels/sources row ? orphan videos are routine, not edge; (C) two distinct videos sharing a fingerprint **collapse to one** on migration (kept `matched=1` here, but ordering-dependent); (F) **no** single-default policy index exists (only `idx\_videos\_user\_document`). One more check to right-size the blocker: does anything enumerate dialogs and upsert channels at startup (which would make orphans rare)?

#### 80. user (2026-07-06T06:16:58.956Z)

```
=== startup dialog enumeration that upserts channels? ===
src/tg_video_filter/telethon_client.py:240:         await db.upsert_channel(
src/tg_video_filter/backfill.py:120:     dialogs = await client.get_dialogs()

=== where is upsert_channel called at all (whole src) ===
src/tg_video_filter/telethon_client.py:240:         await db.upsert_channel(
```

#### 81. assistant (2026-07-06T06:18:21.560Z)

Verified. `upsert\_channel` is called in **exactly one place** ? the live message handler (`telethon\_client.py:240`). Backfill enumerates dialogs (`get\_dialogs()`) but never creates channel/source rows. So any channel that was history-scanned but hasn't received a live message during the run has videos with **no** `channels` row ? all silently dropped on migration. The blocker escalation holds, and a migration that loses an unknown amount of user data with zero signal is a blocker on safety grounds regardless of exact frequency.

I now have a fully cross-verified picture. Here's the consolidated review.

---

## Code Review ? PR #35: provider-neutral domain schema (ADR 0011, Track B)

**Verdict:** ? Request changes. The implementation is careful, well-documented, and the schema/shim design is sound ? but the migration can **silently drop user data**, and the headline "three-tier dedup" is **quietly reduced to two tiers**. Both ship green because the tests don't exercise the regressed paths. Everything below was reproduced locally, not taken on faith.

### Repo sync & scope

- Synced to remote head: clean tree, `feat/track-b-domain-schema` and `main` both match `origin`; branch contains only the PR commit `2771285`.
- Diff: **+++1299/?150** across `db.py`, `models.py`, `tests/test\_db.py`, `tests/test\_delivery.py`. **No GitHub CI exists** on this repo, so I ran the gate locally.

### Local gate (my machine, `venv`, py 3.13)

```
| Check | Result |
|---|---|
| `ruff check .` | ? clean |
| `pytest` (full suite) | ? 434 passed, coverage 96.63% (floor 80%) ? matches the PR's claim |
| `ruff format --check .` | ?? fails on `commands.py` only ? pre-existing on `main`, not this PR |
```

---

## ? Blocker

**\*\*1. Migration 003 silently drops every video whose channel has no `channels` row\*\* ?**

`db.py:539-568`

The `content\_items` backfill inner-joins `videos` to `sources`, but `sources` is populated **\*\*only\*\*** from `channels`. And `backfill.py` ? the documented onboarding path ? inserts videos via `db.insert\_video()` and **\*\*never\*\*** creates a channel row (`upsert\_channel` is called in exactly one place: the live message handler `telethon\_client.py:240`). So any channel that was history-scanned but hasn't since had a *\*live\** message holds videos with no `channels` row. The `INNER JOIN` + `INSERT OR IGNORE` discards all of them with **\*\*zero signal\*\***.

Reproduced: 2 legacy videos (one orphan channel) ? `videos\_deprecated=2`, `content\_items=1`; the orphan is invisible to `get\_video\_by\_document\_id` ? re-deliverable now, and permanently lost once `\*\_deprecated` is dropped next release.

> **\*\*Fix:\*\*** backfill `sources` from `SELECT DISTINCT user\_id, channel\_id FROM videos` (not just from `channels`), or `LEFT JOIN` + synthesize the missing source. Add a post-backfill assertion / test that `COUNT(content\_items) == COUNT(videos)`.

---

## ? High

**\*\*2. Tier-2 fingerprint dedup is regressed ? re-uploads get delivered twice\*\* ?**

`db.py:1151-1173`, design at `db.py:285-291`

v1's `get\_video\_by\_content\_fingerprint` did `SELECT ... WHERE size/dur/h/w match` against **\*\*all\*\*** videos. The new shim looks up `dedup\_key='tg:fp:S:D:H:W'`, but any item that had a `document\_id` is stored under `tg:doc:X` ? so the fingerprint lookup can never see it. Since nearly all Telegram videos carry a `document\_id`, Tier-2 is effectively dead.

Reproduced end-to-end: original (doc 111, full fingerprint) stored as `tg:doc:111`; a re-upload with a **\*\*new\*\*** doc id + identical fingerprint passes all three tiers and is inserted **\*\*and forwarded again\*\***. This is the exact case `README.md` ("never forwards the same video twice ? three-tier content dedup") and `docs/design.md` Tier-2 promise to catch.

> **\*\*Root cause:\*\*** `dedup\_key` is a *\*single\** discriminator (doc XOR fp XOR msg); the three tiers became mutually-exclusive storage keys, not three independent read paths. **\*\*Fix:\*\*** have the Tier-2 shim query the fingerprint *\*columns\** on `content\_items`, not the `tg:fp:` key. Then add the missing test (see #4). This also means ADR 0011's / the PR's "generalizes ADR 0006 without losing any dedup quality" claim is **\*\*false as implemented\*\*** ? either fix the code or correct that claim honestly.

---

## ? Medium

**\*\*3. Migration collapses two *\*distinct\** v1 videos that share a fingerprint\*\* ?** `db.py:539`

Two genuinely-different videos with no `document\_id` but identical `(size,dur,h,w)` both compute `tg:fp:...`; `UNIQUE(account\_id, dedup\_key)` + `INSERT OR IGNORE` keeps one and silently drops the other (v1 kept both ? `videos` had no fingerprint uniqueness). Reproduced. Related: the survivor is whichever inserts first, so a `matched=1` row can be dropped in favor of a `matched=0` duplicate. Rare trigger, but silent. Fall the losing row back to the unique `tg:msg:` key, or at least log a dropped-row count.

## ? Low

- **\*\*4. No test covers the Tier-2 regression\*\* ?** `tests/test\_db.py:402`.

`test\_get\_video\_by\_content\_fingerprint\_hit` seeds a video with **\*\*no\*\*** `document\_id`, so it lands under `tg:fp:` and passes ? the cross-tier case (prior item *\*with\** a doc id) is untested, which is exactly why #2 shipped green.

- **\*\*5. Provider-neutrality leak\*\* ?** `db.py:1247-1271, 1310-1341`. The legacy shims `int()`-cast

`provider\_message\_id`/`document\_id`/`provider\_source\_id`; any non-Telegram `content\_item` raises `ValueError`, and `list\_videos\_for\_user` would crash account-wide. Unreachable in Phase 1 (Telegram-only) but a latent trap in the very code that's meant to be provider-neutral ? gate the casts on `provider == "telegram"`.

- \*\*6. Timestamp format inconsistency\*\* ? `db.py:577`. Migration writes `policies.created\_at/updated\_at` as naive SQL `CURRENT\_TIMESTAMP`; the rest of the code uses tz-aware ISO. Parses fine today, but a future `now(UTC)` ? `policy.created\_at` raises "naive vs aware".
- \*\*7. ADR-specified `policies` single-default index is never created\*\* ? `db.py:124-133`. ADR 0011 (schema line 141) calls for a `UNIQUE ... WHERE is\_default=1` partial index; only `idx\_videos\_user\_document` exists. Nothing structurally enforces one default per account (practically bounded only because the shims always use `name='default'`).
- \*\*8. Dead code\*\* ? `\_row\_to\_video` (`db.py:660`) and `\_row\_to\_channel` (`db.py:650`) have no call sites left after the shim rewrite.
- \*\*9. Thin tests\*\* ? the migration-003 test seeds only one row / one dedup branch (ADR asked for \*all\* branches + a "no two collapse" assertion); the delivery no-op test asserts only a constant; shim `None`-guard branches are uncovered (`db.py:512, 1103-1104, 1147, 1172`).

---

## ? Verified good (so it's not all red)

- Fresh-DB triple-`init\_db` idempotency works; the new `\_apply\_schema` drop-empty-legacy logic is correct.
- `mark\_forwarded` no-op is genuinely safe for Phase 1: `forwarded` is never read to gate delivery, and there's no startup re-delivery query ? the PR's reasoning there holds.
- `compute\_dedup\_key`'s Python tree and the SQL `CASE` are consistent (field order, guards, provider prefix).
- Migration 002's FK-retarget contract is preserved and tested.

---

**Bottom line:** Fix #1 (data-loss) and #2 (dedup regression) before merge ? both are small, surgical changes to the backfill SQL and the Tason-2 shim respectively, each with a one-line test that currently doesn't exist. The rest are cleanups that can ride along or follow up.

Want me to:

1. **Post this to the PR** (as a review requesting changes, or a plain comment)?
2. **Draft the fixes** for #1 and #2 (+ their regression tests) on a branch off `origin/feat/track-b-domain-schema`?
3. **Spin off** the low-severity items as separate tasks?

I'll hold until you say ? I won't post to GitHub or push anything without your go-ahead.

### 82. user (2026-07-06T06:44:10.312Z)

please post these comments on the PR

### 83. assistant (2026-07-06T06:44:36.152Z)

The user has authorized posting. Note: GitHub blocks the PR **author** from formally "Request changes" on their own PR, so I'll post this as a **review comment** (the content is unchanged). Let me write the body to a file (avoids shell-escaping issues) and post it.

### 84. assistant (2026-07-06T06:45:05.979Z)

## Code review ? provider-neutral domain schema (ADR 0011, Track B)

**Verdict:** ? Request changes (posting as a comment since GitHub blocks the author from formally requesting changes on their own PR).

The implementation is careful and well-documented, and the schema/shim design is sound ? but the migration can **silently drop user data**, and the headline "three-tier dedup" is **quietly reduced to two tiers**. Both ship green because the tests don't exercise the regressed paths.

Every finding below was reproduced locally against the branch, not taken from the description.

### Local gate (no CI configured on this repo)

```
| Check | Result |
|---|---|
| `ruff check .` | ? clean |
| `pytest` (full suite) | ? 434 passed, coverage **96.63%** (floor 80%) |
| `ruff format --check .` | ?? fails on `commands.py` only ? pre-existing on `main`, **not**
this PR |
```

---

## ? Blocker

### 1. Migration 003 silently drops every video whose channel has no `channels` row ? `db.py:539-568`

The `content\_items` backfill inner-joins `videos` to `sources`, but `sources` is populated **only** from `channels`. And `backfill.py` ? the documented onboarding path ? inserts videos via `db.insert\_video()` and **never** creates a channel row (`upsert\_channel` is called in exactly one place: the live message handler at `telethon\_client.py:240`). So any channel that was history-scanned but hasn't since had a *live* message holds videos with no `channels` row, and the `INNER JOIN` + `INSERT OR IGNORE` discards all of them with **zero signal**.

Reproduced: 2 legacy videos (one on an orphan channel) ? `videos\_deprecated=2`, `content\_items=1`; the orphan is invisible to `get\_video\_by\_document\_id` (so it becomes re-deliverable now) and is permanently lost once `\*\_deprecated` is dropped next release.

**Fix:** backfill `sources` from `SELECT DISTINCT user\_id, channel\_id FROM videos` (not just from `channels`), or `LEFT JOIN` and synthesize the missing source. Add a post-backfill assertion / test that `COUNT(content\_items) == COUNT(videos)`.

---

## ? High

### 2. Tier-2 fingerprint dedup is regressed ? re-uploads get delivered twice ? `db.py:1151-1173` (design at `db.py:285-291`)

v1's `get\_video\_by\_content\_fingerprint` did `SELECT ... WHERE size/dur/h/w match` against **all** videos. The new shim looks up `dedup\_key='tg:fp:S:D:H:W'`, but any item that had a `document\_id` is stored under `tg:doc:X` ? so the fingerprint lookup can never see it. Since nearly all Telegram videos carry a `document\_id`, Tier-2 is effectively dead.

Reproduced end-to-end: original (doc `111`, full fingerprint) stored as `tg:doc:111`; a re-upload with a **new** doc id + identical fingerprint passes all three tiers and is inserted **and forwarded again** ? the exact case `README.md` ("never forwards the same video twice ? three-tier content dedup") and `docs/design.md` Tier-2 promise to catch.

**Root cause:** `dedup\_key` is a *single* discriminator (doc XOR fp XOR msg); the three tiers became mutually-exclusive storage keys, not three independent read paths.  
**Fix:** have the Tier-2 shim query the fingerprint *columns* on `content\_items`, not the `tg:fp:` key; add the missing cross-tier test (see #4). Also: ADR 0011's / this PR's "generalizes ADR 0006 without losing any dedup quality" claim is **false as implemented** ? fix the code or correct the claim.

---

## ? Medium

### 3. Migration collapses two *distinct* v1 videos that share a fingerprint ? `db.py:539`

Two genuinely-different videos with no `document\_id` but identical `(size,dur,h,w)` both compute `tg:fp:...`; `UNIQUE(account\_id, dedup\_key)` + `INSERT OR IGNORE` keeps one and silently drops the other (v1 kept both ? `videos` had no fingerprint uniqueness). Reproduced. Related: the survivor is whichever inserts first, so a `matched=1` row can be dropped in favor of a `matched=0` duplicate. Rare trigger, but silent ? fall the losing row back to the unique `tg:msg:` key, or at least log a dropped-row count.

---

## ? Low

- \*\*4. No test covers the Tier-2 regression\*\* ? `tests/test\_db.py:402`. `test\_get\_video\_by\_content\_fingerprint\_hit` seeds a video with \*\*no\*\* `document\_id`, so it lands under `tg:fp:` and passes ? the cross-tier case (a prior item \*with\* a doc id) is untested, which is exactly why #2 shipped green.
- \*\*5. Provider-neutrality leak\*\* ? `db.py:1247-1271, 1310-1341`. The legacy shims `int()`-cast `provider\_message\_id`/`document\_id`/`provider\_source\_id`; any non-Telegram `content\_item` raises `ValueError`, and `list\_videos\_for\_user` would crash account-wide. Unreachable in Phase 1 (Telegram-only) but a latent trap in the very code meant to be provider-neutral ? gate the casts on `provider == "telegram"`.
- \*\*6. Timestamp format inconsistency\*\* ? `db.py:577`. The migration writes `policies.created\_at/updated\_at` as naive SQL `CURRENT\_TIMESTAMP`; the rest of the code uses tz-aware ISO. Parses fine today, but a future `now(UTC)` ? `policy.created\_at` raises "naive vs aware".
- \*\*7. ADR-specified `policies` single-default index is never created\*\* ? `db.py:124-133`. ADR 0011 (schema, line 141) calls for a `UNIQUE ... WHERE is\_default=1` partial index; only `idx\_videos\_user\_document` exists. Nothing structurally enforces one default per account (practically bounded only because the shims always use `name='default'`).
- \*\*8. Dead code\*\* ? `\_row\_to\_video` (`db.py:660`) and `\_row\_to\_channel` (`db.py:650`) have no call sites left after the shim rewrite.
- \*\*9. Thin tests\*\* ? the migration-003 test seeds only one row / one dedup branch (ADR asked for \*all\* branches + a "no two collapse" assertion); the delivery no-op test asserts only a constant; shim `None`-guard branches are uncovered (`db.py:512, 1103-1104, 1147, 1172`).

---

## ? Verified good

- Fresh-DB triple-`init\_db` idempotency works; the new `\_apply\_schema` drop-empty-legacy logic is correct.
- `mark\_forwarded` no-op is genuinely safe for Phase 1: `forwarded` is never read to gate delivery, and there's no startup re-delivery query ? the PR's reasoning holds.
- `compute\_dedup\_key`'s Python tree and the migration's SQL `CASE` are consistent (field order, guards, provider prefix).
- Migration 002's FK-retarget contract is preserved and tested.

---

**\*\*Bottom line:\*\*** fix #1 (data-loss) and #2 (dedup regression) before merge ? both are small, surgical changes (backfill SQL / Tier-2 shim) each with a one-line regression test that currently doesn't exist. The rest can ride along or follow up.

<sub>? Review generated with [Claude Code](https://claude.com/claude-code). Findings reproduced locally against branch `feat/track-b-domain-schema`; full suite green at 96.63% coverage.</sub>

### 85. user (2026-07-06T06:45:06.022Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr35\_review.md (file state is current in your context ? no need to Read it back)

### 86. user (2026-07-06T06:45:19.747Z)

---POSTED. Latest review:---

```
{"author":"avidullu","state":"COMMENTED","submittedAt":"2026-07-06T06:45:18Z","url":null}
```

### 87. assistant (2026-07-06T06:45:23.483Z)

Posted. Let me grab the direct link to confirm.

### 88. user (2026-07-06T06:45:26.438Z)

=== review comments on PR (via API) ===  
avidullu [COMMENTED] 2026-07-06T06:45:18Z  
<https://github.com/avidullu/telegram-video-filter/pull/35#pullrequestreview-4633441874>

#### 89. assistant (2026-07-06T06:45:33.702Z)

? **Posted to PR #35** ? [pullrequestreview-4633441874](<https://github.com/avidullu/telegram-video-filter/pull/35#pullrequestreview-4633441874>)

Submitted as a **review comment** (GitHub blocks the PR author from a formal "Request changes" on their own PR, so `--request-changes` would have been rejected ? the content and the ? Request-changes verdict are unchanged in the body).

The comment carries the full review: verdict + local gate results, the blocker (migration data-loss), the high (Tier-2 dedup regression), one medium, six low/cleanup items, and the verified-good list.

Want me to follow up with either of these?

1. **Draft the fixes** for #1 (backfill ``sources`` from ``DISTINCT videos.channel_id``) and #2 (Tier-2 shim queries fingerprint columns), each with the missing regression test, on a branch off ``origin/feat/track-b-domain-schema``.
2. **Spin off** the low-severity items (dead code, timestamp format, provider-neutral ``int()`` casts, missing ``policies`` default index) as separate tracked tasks.

#### 90. user (2026-07-06T07:22:52.365Z)

pls review PR comments

#### 91. assistant (2026-07-06T07:23:00.147Z)

Let me pull everything currently on the PR ? reviews, general comments, and any inline comments ? and review them.

#### 92. user (2026-07-06T07:23:07.177Z)

=== REVIEWS ===  
? avidullu [COMMENTED] 2026-07-06T06:45:18Z id=4633441874  
<https://github.com/avidullu/telegram-video-filter/pull/35#pullrequestreview-4633441874>  
body\_chars=6661

=== ISSUE (general) COMMENTS ===  
? avidullu 2026-07-06T07:16:48Z  
<https://github.com/avidullu/telegram-video-filter/pull/35#issuecomment-4890104919>  
## Review feedback addressed ? all 9 findings

All findings from the [review](#) are addressed in ``e7c188f``. Here's a point-by-point:

---

#### ? Blocker #1 ? Migration silently drops orphan-channel v

=== INLINE REVIEW COMMENTS ===  
(empty sections above = none)

#### 93. assistant (2026-07-06T07:23:15.979Z)

A new commit ``e7c188f`` was pushed and a comment posted claiming **"Review feedback addressed ? all 9 findings."** So "review PR comments" means: verify that response ? and per the gate, I don't trust the claims, I reproduce them. Let me read the full comment and sync to the new remote HEAD.

#### 94. user (2026-07-06T07:23:19.257Z)

=== FULL RESPONSE COMMENT ===

## Review feedback addressed ? all 9 findings

All findings from the [review](#) are addressed in `e7c188f`. Here's a point-by-point:

---

### ? Blocker #1 ? Migration silently drops orphan-channel videos

**\*\*Fixed.\*\*** Sources are now backfilled from `videos` (not just `channels`) via step 1b. The original SQL had a bare `MIN(v.seen\_at)` aggregate without `GROUP BY`, which collapsed the entire videos table to one row ? the orphan channel (-1002) was never produced. Fix:

```
```sql
SELECT v.user_id, 'telegram', CAST(v.channel_id AS TEXT),
       'channel', 0, MIN(v.seen_at)
FROM videos v
GROUP BY v.user_id, v.channel_id  -- ? was missing
```
```

Post-migration count assertion added: `COUNT(content\_items) == COUNT(videos)` with a `logging.warning` on mismatch.

**\*\*Repro fixed:\*\*** your 2-legacy-videos-on-orphan-channel repro now produces `content\_items=2, sources=2`.

---

### ? Blocker #2 ? Tier-2 fingerprint dedup reduced to two tiers

**\*\*Fixed.\*\*** `get\_video\_by\_content\_fingerprint()` now queries metadata **\*\*columns\*\*** on `content\_items` (`size\_bytes`, `duration\_s`, `height`, `width`) ? not the `dedup\_key`. Items stored under `tg:doc:X` still have their metadata columns populated, so the column-based query catches cross-key matches. This restores v1's pre-ADR-0011 behavior: a re-upload with a new doc id + identical fingerprint IS caught by Tier-2.

**\*\*Repro fixed:\*\*** your original (doc `111`, full fp) ? stored as `tg:doc:111`; re-upload with new doc id + same fp ? caught by column-based fingerprint query. No duplicate delivery.

---

### ? Finding #3 ? Fingerprint collision silently drops rows

**\*\*Fixed.\*\*** Migration 003 now uses a two-phase insert:

- **\*\*Phase A:\*\*** All rows land with `tg:msg:` keys (guaranteed unique via `source\_id`).
- **\*\*Phase B:\*\*** One row per `(account, document\_id)` promoted to `tg:doc:`.
- **\*\*Phase C:\*\*** One non-doc row per `(account, fingerprint)` promoted to `tg:fp:`.

Fingerprint-colliding losers stay at their `tg:msg:` key ? no silent data loss.

`UNIQUE(account\_id, dedup\_key)` enforced at promotion time, not at insert.

---

### ? Finding #4 ? No cross-tier dedup test

**\*\*Added\*\*** `test\_fingerprint\_lookup\_catches\_item\_stored\_under\_doc\_key`: a `tg:doc:111` item is found by the fingerprint column query. Also added a dense migration backfill test with 4 videos across 2 channels (one orphan), verifying all rows preserved, sources backfilled, and fp-collision keys distinct.

---

### ? Finding #5 ? Provider-neutrality leak (`int()`) crash on non-TG

**\*\*Added\*\*** `\_safe\_int\_cast(value, field\_name)` with a clear `ValueError("Cannot cast {field}={value!r} to int? Use ContentItem/Source directly for non-Telegram providers (ADR 0011).")`. All legacy shim casts (`int(provider\_source\_id)`, `int(provider\_message\_id)`, `int(document\_id)`) now use it.

---



### ? Finding #6 ? Timestamp `CURRENT\_TIMESTAMP` (naive) vs tz-aware ISO

```
**Fixed.** Migration policies backfill now uses:
```sql
strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
```

---
```

### ? Finding #7 ? Missing `policies` default partial unique index

```
**Added.** `idx_policies_account_default` created in `_SCHEMA` + re-created in migration 003
(idempotent):
```sql
CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default
ON policies(account_id) WHERE is_default = 1
```

---
```

### ? Finding #8 ? Dead code

```
**Removed** `_row_to_video` and `_row_to_channel` (no callers after shim rewrite).

---
```

### ? Finding #9 ? Thin tests

```
**Added 7 new tests:** `_safe_int_cast` (2 cases), cross-tier fingerprint lookup (3 cases),
policies partial index existence, dense migration backfill covering orphan-channel sources +
fingerprint collision.

---
```

### Verification gate (post-fix)

```
| Check | Result |
|---|---|
| `pytest` | **441 passed** (was 434), **96.52%** coverage |
| `ruff check .` | ? clean |
| `ruff format` | ? clean (changed files) |
```

### 95. user (2026-07-06T07:23:22.914Z)

```
---branch state---
0      0
(left=origin ahead, right=local ahead)
---new commits on origin branch---
---show e7c188f exists---
e7c188f fix(domain): address PR #35 review ? data-loss, dedup regression, and polish
2771285 feat(domain): provider-neutral schema ? sources/content_items/policies (ADR 0011, Track
B)
f6c9322 docs(adr): accept 0011 provider-neutral domain schema (Track B decision) (#34)
```

### 96. assistant (2026-07-06T07:24:00.275Z)

The PR branch already advanced to `e7c188f` in this shared checkout (a sibling session pushed the fix ? local HEAD is in sync with origin). The comment claims all 9 findings are fixed, but per the gate I verify, not trust. Let me confirm HEAD state and pull the actual fix diff.

### 97. user (2026-07-06T07:24:02.492Z)

```
---HEAD---
e7c188f fix(domain): address PR #35 review ? data-loss, dedup regression, and polish
---STATUS---
feat/track-b-domain-schema...origin/feat/track-b-domain-schema
?? _review_body.txt
---FILES in e7c188f---
```

```
src/tg_video_filter/db.py | 263 ++++++-----
tests/test_db.py          | 223 ++++++
2 files changed, 414 insertions(+), 72 deletions(-)
```

## 98. assistant (2026-07-06T07:24:10.651Z)

HEAD is `e7c188f`, tree clean (only an untracked `\_review\_body.txt` from the sibling session ? I'll leave it). The fix is +263 in `db.py`, +223 tests. Let me read the actual changed logic ? the two-phase migration, the fingerprint lookup, `\_safe\_int\_cast`, and the new index ? then re-run every repro and the gate.

## 99. user (2026-07-06T07:24:13.224Z)

```
=== anchors ===
136:CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default
501:async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
543:     GROUP BY v.user_id, v.channel_id
548:     # Phase A ? insert every row with a guaranteed-unique tg:msg: Tier-0 key
550:     # Phase B ? promote to tg:doc: for items with a document_id (one per doc-id group).
551:     # Phase C ? promote remaining non-doc items to tg:fp: (one per fingerprint group).
578:     # Phase B: promote one row per (account, document_id) to tg:doc:.
587:     GROUP BY account_id, document_id
592:     # Phase C: promote one non-doc row per (account, fingerprint) to tg:fp:.
608:     GROUP BY account_id, size_bytes, duration_s, height, width
631:     "CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default "
640:     async with conn.execute("SELECT COUNT(*) FROM videos") as cur:
642:     async with conn.execute("SELECT COUNT(*) FROM content_items") as cur:
679:def _safe_int_cast(value: object, field_name: str) -> int:
1227:async def get_video_by_content_fingerprint(
```

## 100. user (2026-07-06T07:24:16.225Z)

```
501     async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
502         """Land the provider-neutral domain tables (ADR 0011, Track B).
503
504         Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
505         ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
506         DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
507         them), then backfills the three populated ones from the legacy
508         ``channels`` / ``videos`` / ``filter_config`` tables and renames the
509         legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
510         pattern). Idempotent: a no-op once the legacy tables have been renamed.
511         """
512         # Idempotency gate: if the legacy videos table is gone, this migration has
513         # already run (or the DB is fresh and _SCHEMA's legacy tables were already
514         # dropped by _apply_schema post-003). Nothing to backfill.
515         if not await _table_exists(conn, "videos"):
516             return
517
518         # 1a. Backfill sources from channels first (known title + watched=1). Order
519         # matters: channels-first so title/watched data wins; the videos-based
520         # backfill below fills gaps for orphan channels (review finding #1).
521         await conn.execute(
522             """
523             INSERT OR IGNORE INTO sources
524             (account_id, provider, provider_source_id, title, kind, watched,
525              seen_first_at, last_seen_at)
526             SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
527                    1, seen_first_at, seen_first_at
528             FROM channels
529             """
530         )
531
532         # 1b. Backfill sources from videos for channels that have no channels row
```

```

533 # (e.g. backfill-only channels that were never seen via live traffic).
534 # INSERT OR IGNORE is a no-op on (account, provider, source_id) conflicts
535 # ? the channels-based row above survives with its title + watched=1.
536 await conn.execute(
537     """
538     INSERT OR IGNORE INTO sources
539         (account_id, provider, provider_source_id, kind, watched, seen_first_at)
540     SELECT v.user_id, 'telegram', CAST(v.channel_id AS TEXT),
541         'channel', 0, MIN(v.seen_at)
542     FROM videos v
543     GROUP BY v.user_id, v.channel_id
544     """
545 )
546
547 # 2. Backfill content_items from videos in two phases (review findings #2 & #3):
548 # Phase A ? insert every row with a guaranteed-unique tg:msg: Tier-0 key
549 # (no silent drops, even on fingerprint collisions).
550 # Phase B ? promote to tg:doc: for items with a document_id (one per doc-id
group).
551 # Phase C ? promote remaining non-doc items to tg:fp: (one per fingerprint
group).
552 # The non-promoted losers stay at tg:msg: (unique) ? no silent data loss.
553 await conn.execute(
554     """
555     INSERT INTO content_items
556         (account_id, source_id, provider, provider_message_id, content_type,
557         document_id, duration_s, width, height, size_bytes, mime_type,
558         dedup_key, matched, seen_at)
559     SELECT
560         v.user_id,
561         s.id,
562         'telegram',
563         CAST(v.message_id AS TEXT),
564         'video',
565         CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
566         v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
567         'tg:msg:' || s.id || ':' || v.message_id, -- Tier-0 (always unique)
568         v.matched,
569         v.seen_at
570     FROM videos v
571     JOIN sources s
572         ON s.account_id = v.user_id
573         AND s.provider = 'telegram'
574         AND s.provider_source_id = CAST(v.channel_id AS TEXT)
575     """
576 )
577
578 # Phase B: promote one row per (account, document_id) to tg:doc:.
579 await conn.execute(
580     """
581     UPDATE content_items SET dedup_key = 'tg:doc:' || document_id
582     WHERE document_id IS NOT NULL
583         AND dedup_key LIKE 'tg:msg:%'
584         AND id IN (
585         SELECT MIN(id) FROM content_items
586         WHERE document_id IS NOT NULL AND dedup_key LIKE 'tg:msg:%'
587         GROUP BY account_id, document_id
588         )
589     """
590 )
591
592 # Phase C: promote one non-doc row per (account, fingerprint) to tg:fp:.
593 await conn.execute(
594     """
595     UPDATE content_items SET dedup_key =

```

```

596         'tg:fp:' || size_bytes || ':' || duration_s || ':'
597         || height || ':' || width
598     WHERE document_id IS NULL
599         AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
600         AND height IS NOT NULL AND width IS NOT NULL
601         AND dedup_key LIKE 'tg:msg:%'
602         AND id IN (
603             SELECT MIN(id) FROM content_items
604             WHERE document_id IS NULL
605                 AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
606                 AND height IS NOT NULL AND width IS NOT NULL
607                 AND dedup_key LIKE 'tg:msg:%'
608             GROUP BY account_id, size_bytes, duration_s, height, width
609         )
610     """
611 )
612
613 # 3. Backfill policies from filter_config: one 'default' policy per account
614 #     (Option 2A). is_default=1 so the legacy load_filter_config shim reads
615 #     this row. UNIQUE(account_id, name) dedups idempotent re-runs.
616 await conn.execute(
617     """
618     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
619                                     created_at, updated_at)
620     SELECT user_id, 'default', config_json, 1,
621            strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
622            strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
623     FROM filter_config
624     """
625 )
626
627 # 4. Create the policies default-partial index (sig after backfill so the
628 #     UNIQUE constraint only applies once rows are populated). On fresh DBs
629 #     _SCHEMA creates it; on migrated DBs this is the first time.
630 await conn.execute(
631     "CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default "
632     "ON policies(account_id) WHERE is_default = 1"
633 )
634
635 # 5. Assert no silent video loss: on a legacy DB, every videos row MUST
636 #     produce a content_items row. A mismatch signals fingerprint collisions
637 #     where two distinct videos shared (size,dur,h,w) ? the losers stay at
638 #     tg:msg: keys and are still present. Log a warning so the operator can
639 #     investigate; the data is preserved in *_deprecated tables.
640 async with conn.execute("SELECT COUNT(*) FROM videos") as cur:
641     vc = (await cur.fetchone())[0]
642 async with conn.execute("SELECT COUNT(*) FROM content_items") as cur:
643     cic = (await cur.fetchone())[0]
644 if vc != cic:
645     import logging
646
647     _logger = logging.getLogger(__name__)
648     _logger.warning(
649         "migration_003: %d videos rows produced %d content_items rows "
650         "%d gap). Colliding items remain in videos_deprecated.",
651         vc,
652         cic,
653         vc - cic,
654     )
655
656 # 6. Rename (don't drop) the legacy tables ? kept for one release as a
657 #     rollback safety net (ADR 0010 pattern).
658 await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
659 await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
660 await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")

```

### 101. assistant (2026-07-06T07:24:58.261Z)

The two-phase migration looks well-designed. Let me read the fingerprint fix and `\_safe\_int\_cast`, then re-run every repro and the full gate against `e7c188f`.

### 102. user (2026-07-06T07:24:59.434Z)

```
679 def _safe_int_cast(value: object, field_name: str) -> int:
680     """Cast a provider_source_id / provider_message_id / document_id to int.
681
682     All existing data is Telegram ? these ids are numeric strings. Non-Telegram
683     providers cannot be projected onto the legacy ``Channel`` / ``VideoRecord``
684     shapes (which carry Telegram-specific int ids); this function raises a clear
685     ``ValueError`` rather than letting a cryptic ``int()`` crash propagate
686     (ADR 0011 provider-neutrality guard ? review finding #5).
687     """
688     try:
689         return int(value) # type: ignore[arg-type]
690     except (ValueError, TypeError) as exc:
691         raise ValueError(
692             f"Cannot cast {field_name}={value!r} to int. The legacy Channel/VideoRecord
693             "
694             f"types are Telegram-specific; use ContentItem/Source directly for "
695             f"non-Telegram providers (ADR 0011)."
696         ) from exc
697
698 def _row_to_account(row: aiosqlite.Row) -> Account:
699     return Account(
700         id=row["id"],
701         telegram_id=row["telegram_id"],
702         provider=row["provider"],
703         display_name=row["display_name"],
704         created_at=row["created_at"], # stored as ISO string; pydantic parses
705     )
706
707
708 def _row_to_telegram_session(row: aiosqlite.Row) -> TelegramSession:
709     return TelegramSession(
710         account_id=row["account_id"],
711         phone=row["phone"],
712         session_path=row["session_path"],
713         enabled=bool(row["enabled"]),
714         created_at=row["created_at"],
715         last_login_at=row["last_login_at"],
716     )
717
718
```

### 103. user (2026-07-06T07:25:02.287Z)

```
1227 async def get_video_by_content_fingerprint(
1228     conn: aiosqlite.Connection,
1229     user_id: int,
1230     size_bytes: int,
1231     duration_s: int,
1232     height: int,
1233     width: int,
1234 ) -> VideoRecord | None:
1235     """Tier-2 content dedup: query metadata COLUMNS, not the ``dedup_key``.
1236
1237     ADR 0011: ``dedup_key`` is a storage discriminator (``tg:doc:X`` vs
1238     ``tg:fp:...`` vs ``tg:msg:...``), NOT the read path. Items with a
1239     ``document_id`` are stored under ``tg:doc:X`` but their metadata columns
1240     are still populated ? querying the columns catches cross-key matches
```

```

1241         that a key-based lookup would miss (the Tier-2 regression: a re-upload
1242         with a new doc id + matching fingerprint must still be caught).
1243
1244         .. versionchanged:: Track B (review fix)
1245         Reverted from key-based lookup to column-based lookup, matching the v1
1246         pre-ADR-0011 behaviour that was regressed when :func:`compute_dedup_key`
1247         made the three tiers mutually-exclusive keys.
1248         """
1249         conn.row_factory = aiosqlite.Row
1250         async with conn.execute(
1251             """SELECT ci.*, s.provider_source_id AS src_pid
1252                FROM content_items ci
1253                JOIN sources s ON s.id = ci.source_id
1254                WHERE ci.account_id = ?
1255                       AND ci.size_bytes = ?
1256                       AND ci.duration_s = ?
1257                       AND ci.height = ?
1258                       AND ci.width = ?
1259                LIMIT 1""",
1260             (user_id, size_bytes, duration_s, height, width),
1261         ) as cur:
1262             row = await cur.fetchone()
1263         if row is None:
1264             return None
1265         return VideoRecord(
1266             id=row["id"],
1267             user_id=user_id,
1268             channel_id=_safe_int_cast(row["src_pid"], "provider_source_id"),
1269             message_id=_safe_int_cast(row["provider_message_id"], "provider_message_id"),
1270             document_id=(
1271                 _safe_int_cast(row["document_id"], "document_id")
1272                 if row["document_id"] is not None
1273                 else None
1274             ),
1275             duration_s=row["duration_s"],
1276             width=row["width"],
1277             height=row["height"],
1278             size_bytes=row["size_bytes"],
1279             mime_type=row["mime_type"],
1280             matched=bool(row["matched"]),
1281             forwarded=False,

```

#### 104. assistant (2026-07-06T07:25:13.262Z)

Both fixes look correct: the fingerprint lookup now queries metadata **\*\*columns\*\*** (restoring v1 semantics), and `_safe_int_cast` gives a clear error. Now the decisive step ? re-run every repro and the full gate against `e7c188f`.

#### 105. assistant (2026-07-06T07:25:35.383Z)

```

"""Re-verify all review findings against the fixed code (e7c188f)."""

```

```

import asyncio, aiosqlite
from tg_video_filter import db
from tg_video_filter.models import Account, VideoRecord

```

```

OK = "PASS"; BAD = "FAIL"

```

```

async def blocker1_orphan():
    conn = await aiosqlite.connect(":memory:")
    await conn.executescript(db._SCHEMA)
    await conn.execute("INSERT INTO accounts(id,telegram_id,provider,created_at) VALUES
(1,1,'telegram','2026-01-01T00:00:00+00:00')")
    await conn.execute("INSERT INTO channels(user_id,channel_id,title,seen_first_at) VALUES
(1,100,'has-row','2026-01-01T00:00:00+00:00')")
    await conn.execute("INSERT INTO videos(user_id,channel_id,message_id,document_id,seen_at)

```

```

VALUES (1,100,5,777,'2026-01-01T00:00:00+00:00')")
    await conn.execute("INSERT INTO videos(user_id,channel_id,message_id,document_id,seen_at)
VALUES (1,999,6,888,'2026-01-01T00:00:00+00:00')") # ORPHAN
    await conn.commit()
    await db._migration_003_domain_schema(conn)
    v = (await (await conn.execute("SELECT COUNT(*) FROM videos_deprecated")).fetchone())[0]
    ci = (await (await conn.execute("SELECT COUNT(*) FROM content_items")).fetchone())[0]
    s = (await (await conn.execute("SELECT COUNT(*) FROM sources")).fetchone())[0]
    await conn.close()
    print(f"[Blocker#1 orphan] videos={v} content_items={ci} sources={s} -> {OK if ci==v==2 and s==2 else BAD}")

async def blocker2_tier2():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    await db.upsert_account(conn, Account(id=42, telegram_id=42))
    F = dict(size_bytes=10_000_000, duration_s=180, height=1080, width=1920)
    await db.insert_video(conn, VideoRecord(user_id=42, channel_id=1001, message_id=1,
document_id=111, **F))
    t2 = await db.get_video_by_content_fingerprint(conn, 42, **F) # re-upload lookup, new doc
id
    await conn.close()
    print(f"[Blocker#2 tier2] fingerprint hit on doc-keyed item = {t2 is not None} -> {OK if t2
is not None else BAD}")

async def finding3_collapse():
    conn = await aiosqlite.connect(":memory:")
    await conn.executescript(db._SCHEMA)
    await conn.execute("INSERT INTO accounts(id,telegram_id,provider,created_at) VALUES
(1,1,'telegram','2026-01-01T00:00:00+00:00')")
    await conn.execute("INSERT INTO channels(user_id,channel_id,title,seen_first_at) VALUES
(1,100,'c','2026-01-01T00:00:00+00:00')")
    for msg, m in ((10, 1), (11, 0)): # two distinct videos, same fingerprint, no doc id
        await conn.execute("INSERT INTO videos(user_id,channel_id,message_id,document_id,duratio
n_s,width,height,size_bytes,matched,seen_at) VALUES
(1,100,?,NULL,60,1920,1080,5000000,?, '2026-01-01T00:00:00+00:00')", (msg, m))
    await conn.commit()
    await db._migration_003_domain_schema(conn)
    rows = [tuple(r) for r in await (await conn.execute("SELECT
provider_message_id,matched,dedup_key FROM content_items ORDER BY
provider_message_id")).fetchall()]
    await conn.close()
    kept_matched = any(r[1] == 1 for r in rows)
    print(f"[Finding#3 collapse] rows={rows} -> {OK if len(rows)==2 and kept_matched else BAD}")

async def finding5_safeint():
    try:
        db._safe_int_cast("wa-abc", "provider_message_id")
        print(f"[Finding#5 safeint] no raise -> {BAD}")
    except ValueError as e:
        print(f"[Finding#5 safeint] clear ValueError raised -> {OK}")

async def finding7_index():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    idx = [r[0] for r in await (await conn.execute("SELECT name FROM sqlite_master WHERE
type='index'")).fetchall()]
    has = "idx_policies_account_default" in idx
    # enforce: two defaults for same account must fail
    await db.upsert_account(conn, Account(id=7, telegram_id=7))
    await conn.execute("INSERT INTO
policies(account_id,name,config_json,is_default,created_at,updated_at) VALUES
(7,'a',{'{}',1,'x','x')")
    dup_rejected = False
    try:

```

```

        await conn.execute("INSERT INTO
policies(account_id,name,config_json,is_default,created_at,updated_at) VALUES
(7,'b','{','1','x','x')")
    except Exception:
        dup_rejected = True
    await conn.close()
    print(f"[Finding#7 index] present={has} second_default_rejected={dup_rejected} -> {OK if has
and dup_rejected else BAD}")

async def finding8_deadcode():
    gone = not hasattr(db, "_row_to_video") and not hasattr(db, "_row_to_channel")
    print(f"[Finding#8 deadcode] _row_to_video/_row_to_channel removed = {gone} -> {OK if gone
else BAD}")

async def main():
    await blocker1_orphan()
    await blocker2_tier2()
    await finding3_collapse()
    await finding5_safeint()
    await finding7_index()
    await finding8_deadcode()

asyncio.run(main())

```

### 106. user (2026-07-06T07:25:35.420Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/reverify.py (file state is current in your context ? no need to Read it back)

### 107. user (2026-07-06T07:25:41.181Z)

```

=== REGRESSION RE-VERIFY (against e7c188f) ===
[Blocker#1 orphan] videos=2 content_items=2 sources=2 -> PASS
[Blocker#2 tier2] fingerprint hit on doc-keyed item = True -> PASS
[Finding#3 collapse] rows=[('10', 1, 'tg:fp:50000000:60:1080:1920'), ('11', 0, 'tg:msg:1:11')] ->
PASS
[Finding#5 safeint] clear ValueError raised -> PASS
[Finding#7 index] present=True second_default_rejected=True -> PASS
[Finding#8 deadcode] _row_to_video/_row_to_channel removed = True -> PASS

```

### 108. user (2026-07-06T07:25:51.701Z)

```

=== GATE (fixed) ===
All checks passed!
ruff-check-exit=0
2 files already formatted
fmt-changed-files-exit=0
src\tg_video_filter\models.py          125      1      8      1    98%   276
src\tg_video_filter\telethon_client.py  62       3     26     4    92%   69, 86->99,
309, 324, 355->exit
-----
TOTAL                                1011     30    252    14    97%
Required test coverage of 80.0% reached. Total coverage: 96.52%
441 passed in 6.94s

```

### 109. assistant (2026-07-06T07:26:22.933Z)

All six regression repros now **PASS**, and the gate is green: ruff clean, **441 passed @ 96.52%** (7 new tests; coverage still far above the 80% floor). Two final checks ? confirm the new tests actually assert the cross-tier case, and that the rewritten migration is still idempotent.

### 110. user (2026-07-06T07:26:33.040Z)



```

=== new tests present? ===
252:async def test_insert_video_dedup_collision(conn: aiosqlite.Connection) -> None:
352:async def test_insert_video_document_id_collision(conn: aiosqlite.Connection) -> None:
377:async def test_insert_video_null_document_id_no_collision(conn: aiosqlite.Connection) ->
None:
595:async def test_foreign_key_enforcement_rejects_orphan_video(
892:def test_compute_dedup_key_telegram_fingerprint_when_no_document_id() -> None:
1119:async def test_migration_003_backfills_all_dedup_key_branches(tmp_path: object) -> None:
1234:async def test_fingerprint_lookup_catches_item_stored_under_doc_key(
1313:async def test_migration_003_backfills_all_three_dedup_branches_including_collision(
1442:async def test_safe_int_cast_raises_on_non_numeric(conn: aiosqlite.Connection) -> None:
1448:def test_safe_int_cast_parses_numeric_strings() -> None:

=== run the fix-specific tests ===
src\tg_video_filter\filters\size.py          18      18      6      0      0%    3-34
src\tg_video_filter\manager.py               36      36      8      0      0%    26-204
src\tg_video_filter\models.py               125      4      8      3     95%    276, 278, 289,
328
src\tg_video_filter\telethon_client.py       62      62     26      0      0%    16-358
-----
TOTAL                                     1011    667    252     12     29%
FAIL Required test coverage of 80.0% not reached. Total coverage: 29.45%
21 passed, 50 deselected in 1.10s

=== idempotency of rewritten migration (triple init) ===
triple-init user_version= 3 | base legacy dropped= True

```

### 111. assistant (2026-07-06T07:26:43.123Z)

The fix-specific subset shows **\*\*21 passed\*\*** (the coverage-FAIL line is just an artifact of running a ``-k`` subset ? the full suite already passed at 96.52%). The key regression-guard tests are present, and the rewritten migration is still idempotent. Let me confirm the cross-tier test genuinely locks in the fix.

### 112. user (2026-07-06T07:26:44.094Z)

```

1234  async def test_fingerprint_lookup_catches_item_stored_under_doc_key(
1235      conn: aiosqlite.Connection,
1236  ) -> None:
1237      """Tier-2 fingerprint lookup must catch an item stored under tg:doc:X.
1238
1239      This is the cross-tier regression test for review finding #2: a prior item
1240      with a document_id is stored as `tg:doc:111`, but its metadata columns
1241      are still populated. A later re-upload with a new doc id + identical
1242      fingerprint must be caught by the column-based fingerprint query.
1243      """
1244      await _seed_user(conn)
1245      src = await db.upsert_source(
1246          conn, Source(account_id=USER_ID, provider="telegram", provider_source_id="-1")
1247      )
1248      assert src.id is not None
1249      # Insert an item WITH a document_id ? this is stored under tg:doc:111.
1250      await db.insert_content_item(
1251          conn,
1252          ContentItem(
1253              account_id=USER_ID,
1254              source_id=src.id,
1255              provider="telegram",
1256              provider_message_id="1",
1257              document_id="111",
1258              duration_s=60,
1259              width=1920,
1260              height=1080,
1261              size_bytes=5000,
1262              dedup_key="tg:doc:111",

```

```

1263         ),
1264     )
1265     # Fingerprint lookup should find it via columns, despite the tg:doc key.
1266     found = await db.get_video_by_content_fingerprint(conn, USER_ID, 5000, 60, 1080,
1920)
1267     assert found is not None
1268     assert found.document_id == 111
1269
1270
1271     async def test_fingerprint_lookup_catches_item_stored_under_fp_key(
1272         conn: aiosqlite.Connection,
1273     ) -> None:

```

### 113. assistant (2026-07-06T07:27:24.891Z)

The cross-tier test is a genuine guard ? it stores an item under `tg:doc:111` and asserts the fingerprint lookup still finds it via columns (fails against the old shim, passes now).

I've now re-reviewed the response comment and the fix commit `e7c188f` ? **\*\*independently reproducing every claim\*\*** rather than trusting the writeup. Here's the verdict.

### Re-review of "Review feedback addressed" (commit `e7c188f`)

The branch advanced to `e7c188f` in this shared checkout (a sibling session pushed the fix; local HEAD is synced to origin). **\*\*All 9 findings are genuinely fixed ? verified, not taken on faith:\*\***

| # | Finding                 | Fix                                                                                                                     | My independent check                                      |
|---|-------------------------|-------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| 1 | Orphan-video data loss  | sources backfilled from `videos` too (`GROUP BY`), + `COUNT` parity guard                                               | repro: <b>**2/2 videos + 2 sources preserved**</b> ?      |
| 2 | Tier-2 dedup regression | fingerprint lookup queries <b>**columns**</b> , not `tg:fp` key                                                         | repro: <b>**doc-keyed item now caught**</b> ?             |
| 3 | Fingerprint collapse    | two-phase migration; losers keep unique `tg:msg:` keys                                                                  | repro: <b>**both rows kept**</b> , `matched=1` retained ? |
| 4 | Missing cross-tier test | `test_fingerprint_lookup_catches_item_stored_under_doc_key`   read it ? <b>**genuine guard**</b> , seeds `tg:doc:111` ? |                                                           |
| 5 | `int()` provider leak   | `_safe_int_cast` with clear `ValueError`                                                                                | repro: <b>**clear error raised**</b> ?                    |
| 6 | Naive timestamp         | `strftime('%T?Z', 'now')` ? tz-aware                                                                                    | verified ISO-Z ?                                          |
| 7 | Missing policies index  | `idx_policies_account_default` partial unique                                                                           | repro: <b>**present + rejects 2nd default**</b> ?         |
| 8 | Dead code               | `_row_to_video`/`_row_to_channel` removed                                                                               | `hasattr` ? <b>**gone**</b> ?                             |
| 9 | Thin tests              | +7 tests (434 ? <b>**441**</b> )   ran them: <b>**21 targeted pass**</b> ?                                              |                                                           |

**\*\*Gate on `e7c188f`:\*\*** `ruff check` clean · `ruff format` clean (changed files) · **\*\*441 passed @ 96.52%\*\*** (floor 80%) · migration still idempotent (triple-`init\_db` ? `user\_version=3`, legacy tables dropped).

**\*\*Minor, non-blocking:\*\*** coverage dipped 96.63 ? 96.52% (new defensive branches ? the never-firing `COUNT`-mismatch warning and `\_safe\_int\_cast` error path ? are partly uncovered); `import logging` sits inside the migration fn (ruff-clean, just unusual). Not worth a round-trip. Note: this repo has **\*\*no mypy\*\*** configured (not in pre-commit), so type-checking wasn't part of the gate.

**\*\*Verdict: ? the fixes are complete and correct ? this clears my earlier blockers.\*\*** By my pre-LGTM gate, verified locally: full suite green, lint/format clean, coverage above floor, and every finding resolved and independently reproduced.

Want me to **\*\*post a follow-up sign-off comment\*\*** on the PR confirming the fixes are verified (LGTM)? I'll hold until you say go ? I won't post to GitHub without your go-ahead.

### 114. user (2026-07-06T07:28:42.443Z)

yes, please post the comments on the github and also an LGTM if your analysis governs this is mergeable and wel tested

#### 115. assistant (2026-07-06T07:28:51.513Z)

My analysis supports it ? all blockers resolved and independently reproduced, suite green, lint/format clean, coverage above floor. Posting the verified sign-off now. (GitHub blocks the PR author from a formal \*Approve\*, so this goes as an LGTM review comment.)

#### 116. assistant (2026-07-06T07:29:06.663Z)

### ? LGTM ? review findings verified fixed in `e7c188f`

Re-reviewed the fix commit and **\*\*independently reproduced every claim\*\*** (didn't trust the writeup). All 9 findings are genuinely addressed.

```
| # | Finding | Fix | Independent check |
|---|-----|-----|-----|
| ? 1 | Orphan-video data loss on migration | `sources` backfilled from `videos` too (`GROUP BY user_id, channel_id`) + post-backfill `COUNT` parity guard | repro: 2/2 videos + 2 sources preserved ? |
| ? 2 | Tier-2 fingerprint dedup regression | `get_video_by_content_fingerprint` queries metadata **columns**, not the `tg:fp:` key | repro: a re-upload (new doc id, same fp) whose prior item is stored under `tg:doc:X` is now caught ? |
| ? 3 | Fingerprint collision drops distinct rows | two-phase migration (Tier-0 `tg:msg:` insert ? promote `tg:doc:`/`tg:fp:`); losers keep unique `tg:msg:` keys | repro: both distinct rows kept, `matched=1` retained ? |
| 4 | No cross-tier dedup test | `test_fingerprint_lookup_catches_item_stored_under_doc_key` ? a genuine guard (fails against the old shim) | read + ran ? |
| 5 | `int()` crash breaks provider-neutrality | `_safe_int_cast` with a clear `ValueError` | repro: clear error on non-numeric ? |
| 6 | Naive `CURRENT_TIMESTAMP` vs tz-aware ISO | `strftime('%Y-%m-%dT%H:%M:%SZ', 'now')` | verified tz-aware ? |
| 7 | Missing `policies` single-default index | `idx_policies_account_default` partial unique (`_SCHEMA` + migration) | repro: present + rejects a 2nd default ? |
| 8 | Dead code | `_row_to_video` / `_row_to_channel` removed | verified gone ? |
| 9 | Thin tests | +7 tests (434 ? 441) | ran ? |
```

#### Verification gate (local, against `e7c188f`)

```
| Check | Result |
|---|---|
| `pytest` (full suite) | ? **441 passed**, coverage **96.52%** (floor 80%) |
| `ruff check .` | ? clean |
| `ruff format --check` (changed files) | ? clean |
| migration idempotency | ? triple `init_db` ? `user_version=3`, legacy tables dropped |
```

**\*\*Notes:\*\*** coverage dipped a hair (96.63 ? 96.52%, from new defensive branches like the never-firing `COUNT`-mismatch warning) ? still far above floor; no `mypy` is configured in this repo so type-checking wasn't part of the gate; the schema/migration was verified via in-memory SQLite (no live Telegram connection needed for this change).

**\*\*LGTM ?\*\*** ? mergeable and well-tested from my side. (Posting as a comment: GitHub doesn't allow the PR author to formally Approve their own PR.)

<sub>? Verified with [Claude Code](https://claude.com/claude-code) ? findings reproduced locally against branch `feat/track-b-domain-schema` @ `e7c188f`.</sub>

#### 117. user (2026-07-06T07:29:06.700Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr35\_lgtm.md (file state is current in your context ? no need to Read it back)

#### 118. user (2026-07-06T07:29:15.616Z)

---latest review URL---  
avidullu [COMMENTED] 2026-07-06T07:29:14Z  
<https://github.com/avidullu/telegram-video-filter/pull/35#pullrequestreview-4633710005>

#### 119. assistant (2026-07-06T07:29:19.160Z)

? **\*\*LGTM sign-off posted\*\*** ?  
[pullrequestreview-4633710005](<https://github.com/avidullu/telegram-video-filter/pull/35#pullrequestreview-4633710005>)

Let me confirm the PR's merge state so I can give you a complete picture.

#### 120. user (2026-07-06T07:29:22.484Z)

state=OPEN mergeable=MERGEABLE mergeStateStatus=CLEAN (feat/track-b-domain-schema -> main)

#### 121. assistant (2026-07-06T07:29:36.796Z)

Done. Both comments are live on the PR and my analysis clears it.

### Summary

**\*\*Posted to PR #35:\*\***

1. [Initial review](<https://github.com/avidullu/telegram-video-filter/pull/35#pullrequestreview-4633441874>) ? ? request-changes (1 blocker, 1 high, + cleanups)
2. [LGTM sign-off](<https://github.com/avidullu/telegram-video-filter/pull/35#pullrequestreview-4633710005>) ? ? after verifying the fix

**\*\*Where it stands:\*\*** `state=OPEN · mergeable=MERGEABLE · mergeStateStatus=CLEAN` (`feat/track-b-domain-schema` ? `main`).

**\*\*Why I signed off\*\*** ? all 9 findings verified fixed in `e7c188f`, each \*reproduced independently\* rather than trusted:

- ? Migration data-loss ? sources now backfilled from `videos` + parity guard (2/2 preserved in repro)
- ? Tier-2 dedup regression ? column-based fingerprint lookup restored + a real cross-tier regression test
- ? Fingerprint collapse ? two-phase migration keeps every distinct row
- 6 lower-severity items all closed

**\*\*Gate (local, `e7c188f`):\*\*** 441 passed @ 96.52% (floor 80%) · ruff check + format clean · migration idempotent.

**\*\*Caveats I flagged honestly in the comment:\*\*** no `mypy` configured in this repo (type-checking wasn't part of the gate); coverage dipped 96.63?96.52% from new defensive branches (still well above floor); verified via in-memory SQLite (no live Telegram needed for a schema change). It's the author's own PR, so GitHub blocks a formal \*Approve\* ? the LGTM is a review comment.

Two housekeeping notes, no action needed unless you want it:

- There's an untracked `\_review\_body.txt` in the repo root (left by the sibling session that posted the fix) ? I left it alone.
- The pre-existing `ruff format` miss on `commands.py` is still there (on `main`, unrelated to this PR); worth a tiny cleanup PR since there's no CI to catch it. Happy to spin that off if you'd like.

#### 122. user (2026-07-06T07:30:50.249Z)

please do the house keeping work!

#### 123. assistant (2026-07-06T07:31:28.462Z)

On it. Two items: (1) the stray untracked `\_review\_body.txt`, (2) the pre-existing `commands.py` format miss. Let me inspect both before touching anything ? I didn't create the file, and I want to see exactly what `ruff format` would change.

## 124. user (2026-07-06T07:31:32.691Z)

=== current branch/tree ===

main

**main...origin/main**

?? \_review\_body.txt

=== \_review\_body.txt (size + head) ===

6811 \_review\_body.txt

--- first 15 lines ---

### Code review ? provider-neutral domain schema (ADR 0011, Track B)

**\*\*Verdict: ? Request changes\*\*** (posting as a comment since GitHub blocks the author from formally requesting changes on their own PR).

The implementation is careful and well-documented, and the schema/shim design is sound ? but the migration can **\*\*silently drop user data\*\***, and the headline "three-tier dedup" is **\*\*quietly reduced to two tiers\*\***. Both ship green because the tests don't exercise the regressed paths. Every finding below was reproduced locally against the branch, not taken from the description.

### Local gate (no CI configured on this repo)

| Check | Result |

|---|---|

| `ruff check .` | ? clean |

| `pytest` (full suite) | ? 434 passed, coverage **\*\*96.63%\*\*** (floor 80%) |

| `ruff format --check .` | ?? fails on `commands.py` only ? pre-existing on `main`, **\*\*not\*\*** this PR |

---

=== is it tracked anywhere / in .gitignore? ===

(not in .gitignore)

## 125. user (2026-07-06T07:31:37.725Z)

=== what would ruff format change in commands.py? (diff preview) ===

--- src\tg\_video\_filter\commands.py

+++ src\tg\_video\_filter\commands.py

@@ -333,7 +333,9 @@

# Friendly error for malformed /set min resolution|res.

if \_MIN\_RES\_HELP\_RE.match(text):

await \_reply(

- client, message, "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)"

+ client,

+ message,

+ "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)",

)

return

1 file would be reformatted

## 126. assistant (2026-07-06T07:32:24.983Z)

Key catch: the shared checkout is now on **\*\*`main`\*\*** (clean) ? a sibling session switched it.

Good thing I re-verified. Findings:

- `\_review\_body.txt` is a 6811-byte scratch copy of my **\*already-posted\*** review ? safe to remove.

- The `commands.py` change is purely cosmetic (wrapping one >100-char `\_reply(...)` call) ? a clean, safe format-only fix.

To avoid disturbing the shared checkout, I'll do the format fix in an isolated **\*\*git worktree\*\***

off `origin/main`, and remove the stray file from the main checkout.

#### 127. user (2026-07-06T07:32:30.811Z)

```
=== (1) remove stray scratch file ===
removed '_review_body.txt'
```

##### main...origin/main

```
=== (2) fetch + create isolated worktree off origin/main ===
323ee5a feat(domain): provider-neutral schema ? sources/content_items/policies (ADR 0011, Track B) (#35)
Preparing worktree (new branch 'chore/format-commands')
branch 'chore/format-commands' set up to track 'origin/main'.
HEAD is now at 323ee5a feat(domain): provider-neutral schema ? sources/content_items/policies (ADR 0011, Track B) (#35)
--- worktree branch ---
chore/format-commands
323ee5a feat(domain): provider-neutral schema ? sources/content_items/policies (ADR 0011, Track B) (#35)
```

#### 128. assistant (2026-07-06T07:32:46.650Z)

**\*\*PR #35 has been merged\*\*** ? `origin/main` is now `323ee5a ?(#35)`. My LGTM landed and the maintainer merged it. The worktree is correctly based on the merged main. Since #35 didn't touch `commands.py`, the format issue persists on main. Let me apply the format fix in the worktree and verify the whole repo is clean.

#### 129. user (2026-07-06T07:32:52.566Z)

```
=== confirm commands.py still unformatted on merged main ===
Would reformat: C:/Users/avidu/Projects/_wt-format-commands/src/tg_video_filter/commands.py
1 file would be reformatted
```

```
=== apply format to commands.py ONLY ===
1 file reformatted
```

```
=== whole-repo format check now clean? ===
34 files already formatted
fmt-exit=0
```

```
=== ruff lint still clean? ===
All checks passed!
```

```
=== exactly one file changed? ===
```

##### chore/format-commands...origin/main

```
M src/tg_video_filter/commands.py
```

#### 130. assistant (2026-07-06T07:33:09.141Z)

Format fix verified in isolation: whole-repo `ruff format --check` now passes, `ruff check` clean, exactly one file changed. Let me confirm the diff is purely cosmetic and run the suite against the worktree (via `PYTHONPATH`) so the sign-off is honest.

#### 131. user (2026-07-06T07:33:22.667Z)

```
=== diff (should be whitespace/wrapping only) ===
diff --git a/src/tg_video_filter/commands.py b/src/tg_video_filter/commands.py
index 6e8474c..cb35aa5 100644
--- a/src/tg_video_filter/commands.py
+++ b/src/tg_video_filter/commands.py
@@ -333,7 +333,9 @@ def make_config_command_handler(
     # Friendly error for malformed /set min resolution|res.
     if _MIN_RES_HELP_RE.match(text):
```

```

        await _reply(
-            client, message, "? Usage: `/set min resolution <N>p` (e.g. `/set min
resolution 720p`)"
+            client,
+            message,
+            "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)",
        )
    return

```

```

=== py_compile sanity ===
compile OK

```

```

=== full suite against the worktree (PYTHONPATH=worktree/src) ===

```

```

src\tg_video_filter\models.py          125      1      8      1    98%    276
src\tg_video_filter\telethon_client.py  62       3     26     4    92%    69, 86->99,
309, 324, 355->exit

```

```

-----
TOTAL                                1011     30    252     14    97%
Required test coverage of 80.0% reached. Total coverage: 96.52%
441 passed in 8.48s

```

### 132. assistant (2026-07-06T07:33:34.212Z)

Clean: cosmetic one-line wrap, compiles, **\*\*441 passed @ 96.52%\*\*** against the worktree.  
 Committing the single file, then pushing and opening a small PR ? following branch-safety  
 (explicit path, re-verify before commit).

### 133. user (2026-07-06T07:33:39.474Z)

```

=== stage explicit path only ===
=== re-verify branch + staged set ===
chore/format-commands
src\tg_video_filter\commands.py
=== commit ===
[chore/format-commands d2e56e4] style(commands): apply ruff format (wrap long _reply call)
 1 file changed, 3 insertions(+), 1 deletion(-)
=== confirm only my commit is ahead of origin/main ===
d2e56e4 style(commands): apply ruff format (wrap long _reply call)
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

```

### 134. assistant (2026-07-06T07:33:42.771Z)

Committed (`d2e56e4`, only my commit ahead of `origin/main`). Now push and open the PR.

### 135. user (2026-07-06T07:33:53.314Z)

```

=== push ===
remote:      https://github.com/avidullu/telegram-video-filter/pull/new/chore/format-commands
remote:
branch 'chore/format-commands' set up to track 'origin/chore/format-commands'.
To https://github.com/avidullu/telegram-video-filter.git
 * [new branch]      chore/format-commands -> chore/format-commands

=== open PR ===
https://github.com/avidullu/telegram-video-filter/pull/36
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

```

### 136. assistant (2026-07-06T07:34:07.365Z)

PR #36 opened for the format fix. Now clean up the worktree and verify the shared checkout is  
 left tidy.

### 137. user (2026-07-06T07:34:10.930Z)

```
=== remove worktree ===
C:/Users/avidu/Projects/Telegram 323ee5a [main]
```

```
=== main checkout state ===
main
```

**main...origin/main**

```
=== is local main cleanly behind origin/main? ===
0      0
(left=local-only, right=origin-ahead)
```

#### 138. assistant (2026-07-06T07:34:19.004Z)

Housekeeping complete and the checkout is left clean (`main` synced to `origin/main` at `323ee5a`, no stray files, worktree removed). Quick final status check on both PRs:

#### 139. user (2026-07-06T07:34:22.843Z)

PR #35: MERGED mergeable=UNKNOWN UNKNOWN ? feat(domain): provider-neutral schema ?  
sources/content\_items/policies (ADR 0011, Track B)  
PR #36: OPEN mergeable=MERGEABLE CLEAN ? style: ruff format commands.py (pre-existing format miss)

#### 140. assistant (2026-07-06T07:34:34.189Z)

All housekeeping done. ?

### Summary

**\*\*1. Stray scratch file\*\*** ? removed `\_review\_body.txt` (a 6811-byte duplicate of my already-posted review) from the repo root. The main checkout is now clean.

**\*\*2. `commands.py` format miss\*\*** ? fixed and shipped as its own PR, isolated in a git worktree so the shared checkout was never disturbed:

- [PR #36 ? `style: ruff format commands.py`](https://github.com/avidullu/telegram-video-filter/pull/36) ? **\*\*OPEN · MERGEABLE · CLEAN\*\***
- Purely cosmetic (wraps one >100-char `\_reply(...)` call). Verified: `ruff format --check .` now clean across all 34 files, `ruff check .` clean, **\*\*441 tests pass @ 96.52%\*\*** (unchanged ? formatting-only).

**\*\*Along the way:\*\*** noticed **\*\*PR #35 is now MERGED\*\*** (`origin/main` ? `323ee5a` ? (#35)) ? the maintainer merged it after my verified LGTM.

**\*\*Repo left tidy:\*\***

- Worktree removed (`git worktree list` shows only the main checkout).
- Local `main` clean and synced to `origin/main` (`0 0`, at the merged `323ee5a`).
- Branch-safety honored throughout: branched from `origin/main`, staged the single explicit path, and confirmed only my commit (`d2e56e4`) was ahead before pushing.

Nothing else outstanding. #36 is a trivial self-merge whenever you want it.

#### 141. user (2026-07-06T07:45:32.467Z)

pls review #37

#### 142. assistant (2026-07-06T07:45:41.270Z)

Let me sync to remote head and pull up PR #37.

#### 143. user (2026-07-06T07:45:47.314Z)

From https://github.com/avidullu/telegram-video-filter  
323ee5a..d806b46 main -> origin/main



---current checkout---

feat/track-c-delivery-routing

## feat/track-c-delivery-routing...origin/feat/track-c-delivery-routing

=== PR #37 META ===

```
{
  "additions": 675,
  "author": {
    "id": "MDQ6VXNlcjE2NzQ4Nzc=",
    "is_bot": false,
    "login": "avidullu",
    "name": "Avi"
  },
  "baseRefName": "main",
  "body": "## What\n\nImplements **Track C (#28)** of the provider-neutral umbrella (#25) ? destinations, routing rules, and per-destination delivery state, replacing the legacy `videos.forwarded` boolean.\n\nThe three tables were created empty by Track B (#35); this PR populates and uses them.\n\n## Models (models.py)\n\n| Model | Purpose | Natural Key |\n|---|---|---|\n| `Destination` | Delivery target (Saved Messages, @channel, chat id) | `(account_id, provider, target)` |\n| `RoutingRule` | Source ? policy ? destination mapping | `(account_id, source_id, destination_id)` |\n| `DeliveryAttempt` | Per-destination idempotent delivery record | `(content_item_id, destination_id)` |\n| `DeliveryStatus` enum | `pending` / `delivered` / `failed` / `retrying` | ? |\n\n## Migration\n\n`_004_destinations_and_routing` (idempotent)\n\n1. **Backfills default destinations** from `filter_config_deprecated.config_json` ? parses `delivery_target` JSON, defaults to `\"me\"`.\nEach account gets one destination row.\n\n2. **Creates default routing rules** ? for each source, a rule linking source ? default policy ? default destination.\n\n3. **Backfills delivery_attempts** ? for every `videos_deprecated.forwarded=1` row, creates a `delivery_attempt` with `status=delivered`.\n\n4. **Idempotent:** gate checks `COUNT(destinations) > 0`.\n\n## Repos (db.py)\n\n- `upsert_destination()` / `get_destination()` ? CRUD for destinations.\n- `record_delivery(content_item_id, destination_id, success, error)` ? replaces the `mark_forwarded` no-op. Writes per-destination delivery state. Idempotent on conflict (`attempt_count` increments, status/last_error update).\n- `get_delivery_attempt()` ? read delivery state.\n\n## Delivery layer (delivery.py)\n\n- `forward_to_target`, `send_link_message`, `deliver` now accept an optional `destination_id` kwarg.\n- When provided, a `DeliveryAttempt` is written via `record_delivery`.\n- **Backwards-compatible:** when `destination_id` is `None` (all existing callers), falls back to the legacy `mark_forwarded` no-op.\n\n## Coverage improvements (bundled)\n\n| Area | Tests added | Coverage gain |\n|---|---|---|\n| `delivery.py` FloodWait retry exhaustion | 2 tests (both-retries-fail, flood-then-generic) | 92% ? 96% |\n| `delivery.py` destination_id integration | 1 test | new code path |\n| Track C repos + migration 004 | 7 tests | 94% ? 96% |\n\n## Verification\n\n451 passed @ 95.75% (floor 80%)\n\nruff check . ? clean\n\nruff format ? clean\n\n(changed files)\n\n## Scope discipline\n\n? **Creates:** Destination/RoutingRule/DeliveryAttempt models + repos + migration.\n\n? **Populates:** destinations backfilled from `filter_config`, `delivery_attempts` from `videos_deprecated`.\n\n? **Additive:** legacy `mark_forwarded` no-op stays; `destination_id=None` is the default.\n\n? **Not in this track:** routing logic (manager/telethon_client integration), selective scanning UX, command service, Bot API adapter (Tracks D/E).\n\n## Closes\n\nCloses #28 (Track C). Unblocks #29 (Track D) and #30 (Track E).\n\n## Related\n\n- #25 ? umbrella issue\n- #35 ? Track B (created the empty tables this PR populates)\n- ADR 0011 ? provider-neutral domain schema\n- ADR 0009 ? selective scanning + multiplexed routing",
  "changedFiles": 5,
  "deletions": 9,
  "headRefName": "feat/track-c-delivery-routing",
  "isDraft": false,
  "mergeStateStatus": "UNKNOWN",
  "mergeable": "UNKNOWN",
  "number": 37,
  "state": "OPEN",
  "statusCheckRollup": [],
  "title": "feat(delivery): destinations, routing_rules, delivery_attempts (ADR 0011, Track C)"
}
```

### 144. assistant (2026-07-06T07:46:06.860Z)

The checkout is already on the PR branch `feat/track-c-delivery-routing` (synced), and `main` advanced to `d806b46` (likely includes my #36 format fix). PR #37 is \*\*Track C\*\* ? +675/?9, 5 files. No CI, so I'll run the gate locally. Let me pull the diff, commit list, and confirm the branch is current with main.

### 145. user (2026-07-06T07:46:17.404Z)

=== files changed ===

```
src/tg_video_filter/db.py
src/tg_video_filter/delivery.py
src/tg_video_filter/models.py
tests/test_db.py
tests/test_delivery.py
```

```
=== commits on branch vs origin/main ===
100abb2 feat(delivery): destinations, routing_rules, delivery_attempts (ADR 0011, Track C)

=== is branch behind main? (main ahead of branch) ===
2
(commits on main not in branch)

=== HEAD/tree ===
100abb2 feat(delivery): destinations, routing_rules, delivery_attempts (ADR 0011, Track C)
feat/track-c-delivery-routing...origin/feat/track-c-delivery-routing

830 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr37.diff
```

#### 146. assistant (2026-07-06T07:46:32.252Z)

The PR branch is one commit (`100abb2`) on a base that's now 2 commits behind main. Let me see what those 2 commits are (to check for conflicts / missing prerequisites) and confirm the branch already contains Track B's migration 003 that migration 004 depends on.

#### 147. user (2026-07-06T07:46:35.775Z)

```
=== 2 commits on main not in branch ===
d806b46 Merge pull request #36 from avidullu/chore/format-commands
d2e56e4 style(commands): apply ruff format (wrap long _reply call)

=== do those commits touch any of PR #37's files? (conflict risk) ===
src/tg_video_filter/commands.py
src/tg_video_filter/db.py
src/tg_video_filter/delivery.py
src/tg_video_filter/models.py
tests/test_db.py
tests/test_delivery.py

=== branch has migration 003 (Track B prereq) + new 004? ===
335:@_migration(1)
352:@_migration(2)
504:@_migration(3)
505:async def _migration_003_domain_schema(conn: aiomysqlite.Connection) -> None:
667:@_migration(4)
668:async def _migration_004_destinations_and_routing(conn: aiomysqlite.Connection) -> None:
```

#### 148. assistant (2026-07-06T07:46:50.713Z)

No merge conflict (main's 2 commits are just my #36 `commands.py` fix, which #37 doesn't touch), and the branch already has migration 003 so migration 004's prerequisite is present. Let me kick off the local gate in the background and read the full diff + new code.

#### 149. user (2026-07-06T07:46:56.296Z)

Command running in background with ID: bfwdxcn1. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bfwdxcn1.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 150. user (2026-07-06T07:46:57.329Z)

```
1      diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
2      index f07a767..9eed6c 100644
3      --- a/src/tg_video_filter/db.py
4      +++ b/src/tg_video_filter/db.py
5      @@ -20,8 +20,12 @@
6          Account,
7          Channel,
```

```

8         ContentItem,
9     +     DeliveryAttempt,
10    +     DeliveryStatus,
11    +     Destination,
12    +     FilterConfig,
13    +     Policy,
14    +     RoutingRule,
15    +     Source,
16    +     TelegramSession,
17    +     User,
18    @@ -660,6 +664,92 @@ async def _migration_003_domain_schema(conn: aiosqlite.Connection)
-> None:
19        await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
20
21
22    +@_migration(4)
23    +async def _migration_004_destinations_and_routing(conn: aiosqlite.Connection) -> None:
24    +    """Backfill default destinations and delivery state (ADR 0011, Track C).
25    +
26    +    Creates a default destination row for each account that has a
27    +    ``filter_config_deprecated.delivery_target`` value, then creates a default
28    +    routing rule (source ? default policy ? destination) for each source.
29    +    Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
30    +    table (one ``delivered`` attempt per previously-forwarded video to the
31    +    account's default destination).
32    +
33    +    Idempotent: skips accounts whose default destination already exists.
34    +    """
35    +    # Gate: if destinations already have data, this migration has run.
36    +    async with conn.execute("SELECT COUNT(*) FROM destinations") as cur:
37    +        if (await cur.fetchone())[0] > 0:
38    +            return
39    +
40    +    if not await _table_exists(conn, "filter_config_deprecated"):
41    +        return
42    +
43    +    # 1. Backfill default destinations from filter_config_deprecated.
44    +    #     Parse the JSON delivery_target, defaulting to "me".
45    +    await conn.execute(
46    +        """
47    +        INSERT OR IGNORE INTO destinations (account_id, provider, target, kind,
is_output, created_at)
48    +        SELECT user_id, 'telegram',
49    +               COALESCE(json_extract(config_json, '$.delivery_target'), 'me'),
50    +               CASE
51    +                   WHEN COALESCE(json_extract(config_json, '$.delivery_target'), 'me') =
'me'
52    +                       THEN 'saved'
53    +                       ELSE 'channel'
54    +                   END,
55    +               0,
56    +               strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
57    +        FROM filter_config_deprecated
58    +        """
59    +    )
60    +
61    +    # 2. Create a default routing rule per source (source ? default policy ? default
destination).
62    +    await conn.execute(
63    +        """
64    +        INSERT OR IGNORE INTO routing_rules
65    +        (account_id, source_id, policy_id, destination_id, enabled, priority,
created_at)
66    +        SELECT s.account_id, s.id, p.id, d.id, 1, 0,
67    +               strftime('%Y-%m-%dT%H:%M:%SZ', 'now')

```

```

68 +         FROM sources s
69 +         JOIN policies p ON p.account_id = s.account_id AND p.is_default = 1
70 +         JOIN destinations d ON d.account_id = s.account_id
71 +         WHERE d.target = (
72 +             SELECT COALESCE(
73 +                 json_extract(fcd.config_json, '$.delivery_target'), 'me'
74 +             )
75 +             FROM filter_config_deprecated fcd
76 +             WHERE fcd.user_id = s.account_id
77 +             LIMIT 1
78 +         )
79 +         ""
80 +     )
81 +
82 +     # 3. Backfill delivery_attempts from previously-forwarded videos.
83 +     await conn.execute(
84 +         ""
85 +         INSERT OR IGNORE INTO delivery_attempts
86 +         (content_item_id, destination_id, route_id, status, attempt_count,
87 +         created_at, delivered_at)
88 +         SELECT ci.id, d.id, rr.id, 'delivered', 1,
89 +         vd.seen_at, vd.seen_at
90 +         FROM videos_deprecated vd
91 +         JOIN content_items ci
92 +         ON ci.account_id = vd.user_id
93 +         AND ci.provider = 'telegram'
94 +         AND ci.provider_message_id = CAST(vd.message_id AS TEXT)
95 +         JOIN sources sc
96 +         ON sc.id = ci.source_id
97 +         AND sc.provider_source_id = CAST(vd.channel_id AS TEXT)
98 +         JOIN destinations d ON d.account_id = vd.user_id
99 +         JOIN routing_rules rr
100 +         ON rr.account_id = vd.user_id
101 +         AND rr.source_id = sc.id
102 +         AND rr.destination_id = d.id
103 +         WHERE vd.forwarded = 1
104 +         ""
105 +     )
106 +
107 +
108 +     async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
109 +         """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
110 +         async with conn.execute(
111 +             @@ -789,6 +879,47 @@ def _row_to_policy(row: aiosqlite.Row) -> Policy:
112 +         )
113 +
114 +
115 +     +def _row_to_destination(row: aiosqlite.Row) -> Destination:
116 +         + return Destination(
117 +             + id=row["id"],
118 +             + account_id=row["account_id"],
119 +             + provider=row["provider"],
120 +             + target=row["target"],
121 +             + kind=row["kind"],
122 +             + title=row["title"],
123 +             + is_output=bool(row["is_output"]),
124 +             + created_at=row["created_at"],
125 +         )
126 +
127 +
128 +     +def _row_to_routing_rule(row: aiosqlite.Row) -> RoutingRule:
129 +         + return RoutingRule(
130 +             + id=row["id"],
131 +             + account_id=row["account_id"],
132 +             + source_id=row["source_id"],

```

```

133 +         policy_id=row["policy_id"],
134 +         destination_id=row["destination_id"],
135 +         enabled=bool(row["enabled"]),
136 +         priority=row["priority"],
137 +         created_at=row["created_at"],
138 +     )
139 +
140 +
141 +def _row_to_delivery_attempt(row: aiosqlite.Row) -> DeliveryAttempt:
142 +    return DeliveryAttempt(
143 +        id=row["id"],
144 +        content_item_id=row["content_item_id"],
145 +        destination_id=row["destination_id"],
146 +        route_id=row["route_id"],
147 +        status=DeliveryStatus(row["status"]),
148 +        attempt_count=row["attempt_count"],
149 +        last_error=row["last_error"],
150 +        created_at=row["created_at"],
151 +        delivered_at=row["delivered_at"],
152 +        next_retry_at=row["next_retry_at"],
153 +    )
154 +
155 +
156 +    # --- account + session repos (ADR 0010) -----
157 +
158 +
159 +    @@ -1138,6 +1269,146 @@ async def get_default_policy(conn: aiosqlite.Connection,
account_id: int) -> Pol
160 +        return _row_to_policy(row) if row else None
161 +
162 +
163 +    ## --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --
164 +
165 +
166 +    +async def upsert_destination(conn: aiosqlite.Connection, dest: Destination) ->
Destination:
167 +        """Insert or update a destination row by (account, provider, target)."""
168 +        await conn.execute(
169 +            """INSERT INTO destinations
170 +                (account_id, provider, target, kind, title, is_output, created_at)
171 +                VALUES (?, ?, ?, ?, ?, ?, ?)
172 +                ON CONFLICT(account_id, provider, target) DO UPDATE SET
173 +                    kind=excluded.kind,
174 +                    title=excluded.title,
175 +                    is_output=excluded.is_output""",
176 +            (
177 +                dest.account_id,
178 +                dest.provider,
179 +                dest.target,
180 +                dest.kind,
181 +                dest.title,
182 +                int(dest.is_output),
183 +                dest.created_at.isoformat(),
184 +            ),
185 +        )
186 +        await conn.commit()
187 +        conn.row_factory = aiosqlite.Row
188 +        async with conn.execute(
189 +            """SELECT * FROM destinations
190 +                WHERE account_id = ? AND provider = ? AND target = ?""",
191 +            (dest.account_id, dest.provider, dest.target),
192 +        ) as cur:
193 +            row = await cur.fetchone()
194 +            assert row is not None
195 +            return _row_to_destination(row)

```

```

196 +
197 +
198 +async def get_destination(
199 +    conn: aiosqlite.Connection, account_id: int, provider: str, target: str
200 +) -> Destination | None:
201 +    """Resolve a destination by its (account, provider, target) natural key."""
202 +    conn.row_factory = aiosqlite.Row
203 +    async with conn.execute(
204 +        "SELECT * FROM destinations WHERE account_id = ? AND provider = ? AND target =
205 +        ?",
206 +        (account_id, provider, target),
207 +    ) as cur:
208 +        row = await cur.fetchone()
209 +        return _row_to_destination(row) if row else None
210 +
211 +async def upsert_delivery_attempt(
212 +    conn: aiosqlite.Connection, attempt: DeliveryAttempt
213 +) -> DeliveryAttempt:
214 +    """Insert or update a delivery_attempt; idempotent on (content_item,
215 +    destination)."""
216 +    await conn.execute(
217 +        """INSERT INTO delivery_attempts
218 +        (content_item_id, destination_id, route_id, status, attempt_count,
219 +        last_error, created_at, delivered_at, next_retry_at)
220 +        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
221 +        ON CONFLICT(content_item_id, destination_id) DO UPDATE SET
222 +        status=excluded.status,
223 +        attempt_count=excluded.attempt_count,
224 +        last_error=excluded.last_error,
225 +        delivered_at=excluded.delivered_at,
226 +        next_retry_at=excluded.next_retry_at""",
227 +        (
228 +            attempt.content_item_id,
229 +            attempt.destination_id,
230 +            attempt.route_id,
231 +            attempt.status.value,
232 +            attempt.attempt_count,
233 +            attempt.last_error,
234 +            attempt.created_at.isoformat(),
235 +            attempt.delivered_at.isoformat() if attempt.delivered_at else None,
236 +            attempt.next_retry_at.isoformat() if attempt.next_retry_at else None,
237 +        ),
238 +    )
239 +    await conn.commit()
240 +    conn.row_factory = aiosqlite.Row
241 +    async with conn.execute(
242 +        "SELECT * FROM delivery_attempts WHERE content_item_id = ? AND destination_id =
243 +        ?",
244 +        (attempt.content_item_id, attempt.destination_id),
245 +    ) as cur:
246 +        row = await cur.fetchone()
247 +        assert row is not None
248 +        return _row_to_delivery_attempt(row)
249 +
250 +async def record_delivery(
251 +    conn: aiosqlite.Connection,
252 +    *,
253 +    content_item_id: int,
254 +    destination_id: int,
255 +    route_id: int | None = None,
256 +    success: bool,
257 +    error: str | None = None,
258 +) -> None:

```

```

258 +     """Record a delivery outcome for one (content_item, destination) pair.
259 +
260 +     This is the post-ADR-0011 replacement for ``mark_forwarded``. On success,
261 +     sets status=delivered; on failure, increments attempt_count and records the
262 +     error. Idempotent: repeated calls update the same row.
263 +     """
264 +     status = DeliveryStatus.DELIVERED if success else DeliveryStatus.FAILED
265 +     await conn.execute(
266 +         """INSERT INTO delivery_attempts
267 +             (content_item_id, destination_id, route_id, status, attempt_count,
268 +             last_error, created_at, delivered_at)
269 +             VALUES (?, ?, ?, ?, 1, ?, strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
270 +             CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now') END)
271 +             ON CONFLICT(content_item_id, destination_id) DO UPDATE SET
272 +             status=excluded.status,
273 +             attempt_count=delivery_attempts.attempt_count + 1,
274 +             last_error=excluded.last_error,
275 +             delivered_at=CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
276 +             ELSE delivery_attempts.delivered_at END""",
277 +         (
278 +             content_item_id,
279 +             destination_id,
280 +             route_id,
281 +             status.value,
282 +             error,
283 +             success,
284 +             success,
285 +         ),
286 +     )
287 +     await conn.commit()
288 +
289 +
290 + async def get_delivery_attempt(
291 +     conn: aiomysqlite.Connection, content_item_id: int, destination_id: int
292 + ) -> DeliveryAttempt | None:
293 +     """Read a delivery attempt row by its natural key (idempotency boundary)."""
294 +     conn.row_factory = aiomysqlite.Row
295 +     async with conn.execute(
296 +         "SELECT * FROM delivery_attempts WHERE content_item_id = ? AND destination_id =
297 +         ?",
298 +         (content_item_id, destination_id),
299 +     ) as cur:
300 +         row = await cur.fetchone()
301 +         return _row_to_delivery_attempt(row) if row else None
302 +
303 + # --- legacy Channel shim (backwards-compatible view over sources) -----
304 +
305 +
306 + @@ -1479,6 +1750,12 @@ async def list_videos_for_user( # v2: used by on-demand /search;
not called in
307 +     "get_content_item_by_dedup_key",
308 +     "upsert_policy",
309 +     "get_default_policy",
310 +     # Destination / Routing / Delivery repos (ADR 0011, Track C)
311 +     "upsert_destination",
312 +     "get_destination",
313 +     "upsert_delivery_attempt",
314 +     "record_delivery",
315 +     "get_delivery_attempt",
316 +     # Legacy User shims (backwards-compatible)
317 +     "upsert_user",
318 +     "get_user",
319 + diff --git a/src/tg_video_filter/delivery.py b/src/tg_video_filter/delivery.py
320 + index 7c7a3c8..d99c22c 100644

```

```

321 --- a/src/tg_video_filter/delivery.py
322 +++ b/src/tg_video_filter/delivery.py
323 @@ -127,16 +127,31 @@ async def forward_to_target(
324     conn: aiosqlite.Connection,
325     video_id: int,
326     target: str = "me",
327 + *,
328 + destination_id: int | None = None,
329 ) -> bool:
330 -     """Forward ``message`` to ``target``, marking the DB row on success.
331 +     """Forward ``message`` to ``target``, recording delivery state.
332
333     ``target`` is a Telethon entity: ``"me"`` (Saved Messages), ``"@username"`` ,
334     or a numeric chat id string. See ADR 0005.
335 +
336 +     If ``destination_id`` is provided, a per-destination ``DeliveryAttempt``
337 +     is written (ADR 0011 Track C). Otherwise falls back to the legacy no-op
338 +     ``mark_forwarded``.
339     """
340     ok = await _send_with_retry(client, lambda: client.forward_messages(target,
341 message), video_id)
342     if ok:
343         await db.mark_forwarded(conn, video_id)
344         if destination_id is not None:
345             await db.record_delivery(
346                 conn, content_item_id=video_id, destination_id=destination_id,
347 success=True
348             )
349         else:
350             await db.mark_forwarded(conn, video_id)
351             log.info("video_forwarded video=%s target=%s", video_id, target)
352         elif destination_id is not None:
353             await db.record_delivery(
354                 conn, content_item_id=video_id, destination_id=destination_id,
355 success=False
356             )
357         return ok
358
359
360 @@ -146,15 +161,26 @@ async def send_link_message(
361     conn: aiosqlite.Connection,
362     video_id: int,
363     target: str = "me",
364 + *,
365 + destination_id: int | None = None,
366 ) -> bool:
367 -     """Send a link/summary text message to ``target``, marking the DB row.
368 +     """Send a link/summary text message to ``target``, recording delivery state.
369
370     ``target`` is a Telethon entity; see :func:`forward_to_target`.
371     """
372     ok = await _send_with_retry(client, lambda: client.send_message(target, text),
373 video_id)
374     if ok:
375         await db.mark_forwarded(conn, video_id)
376         if destination_id is not None:
377             await db.record_delivery(
378                 conn, content_item_id=video_id, destination_id=destination_id,
379 success=True
380             )
381         else:
382             await db.mark_forwarded(conn, video_id)
383             log.info("link_sent video=%s target=%s", video_id, target)
384         elif destination_id is not None:
385             await db.record_delivery(

```



```

381 +         conn, content_item_id=video_id, destination_id=destination_id,
success=False
382 +     )
383     return ok
384
385
386 @@ -167,6 +193,7 @@ async def deliver(
387     *,
388     link_text: str | None = None,
389     target: str = "me",
390 +     destination_id: int | None = None,
391 ) -> bool:
392     """Dispatch delivery based on ``mode`` to ``target``.
393
394 @@ -176,12 +203,21 @@ async def deliver(
395
396     ``target`` is a Telethon entity (``"me"`` , ``"@username"`` , chat id).
397     See ADR 0005.
398 +
399 +     ``destination_id`` is the FK to ``destinations.id`` (ADR 0011 Track C).
400 +     When provided, a per-destination ``DeliveryAttempt`` is written.
401     """
402     if mode == DeliveryMode.LINK and link_text is not None:
403         - return await send_link_message(client, link_text, conn, video_id, target)
404         + return await send_link_message(
405         +     client, link_text, conn, video_id, target, destination_id=destination_id
406         + )
407     if mode == DeliveryMode.BOTH and link_text is not None:
408         - fwd_ok = await forward_to_target(client, message, conn, video_id, target)
409         - link_ok = await send_link_message(client, link_text, conn, video_id, target)
410         + fwd_ok = await forward_to_target(
411         +     client, message, conn, video_id, target, destination_id=destination_id
412         + )
413         + link_ok = await send_link_message(
414         +     client, link_text, conn, video_id, target, destination_id=destination_id
415         + )
416         if not link_ok:
417             log.warning("link_delivery_failed_in_both_mode video=%s", video_id)
418         if not fwd_ok:
419 @@ -195,7 +231,9 @@ async def deliver(
420         mode.value,
421         link_text is None,
422     )
423     - return await forward_to_target(client, message, conn, video_id, target)
424     + return await forward_to_target(
425     +     client, message, conn, video_id, target, destination_id=destination_id
426     + )
427
428
429 __all__ = [
430 diff --git a/src/tg_video_filter/models.py b/src/tg_video_filter/models.py
431 index 0f45679..de94a75 100644
432 --- a/src/tg_video_filter/models.py
433 +++ b/src/tg_video_filter/models.py
434 @@ -208,6 +208,93 @@ class Policy(BaseModel):
435     updated_at: datetime = Field(default_factory=_utcnow)
436
437
438 +## --- Track C: destinations, routing, delivery state (ADR 0011) -----
439 +##
440 +## These tables were created empty by Track B (_SCHEMA); Track C populates
441 +## destinatinons (backfilled from filter_config.delivery_target), writes
442 +## delivery_attempts (replacing the legacy videos.forwarded boolean), and
443 +## enforces the source/destination invariant (a channel is either source or
444 +## destination, never both).

```

```

445 +
446 +
447 +class Destination(BaseModel):
448 +     """A delivery target ? where matched content is sent (ADR 0011, Track C).
449 +
450 +     Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
451 +     can have multiple named output channels. Backfilled from the existing
452 +     per-account ``delivery_target``; the default destination is the target used
453 +     when no explicit routing rule matches.
454 +     """
455 +
456 +     model_config = ConfigDict(frozen=True)
457 +
458 +     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
459 +     account_id: int = Field(description="FK ? accounts.id")
460 +     provider: str = Field(default="telegram", description="Provider namespace")
461 +     target: str = Field(description="Telethon entity: 'me', '@username', or chat id")
462 +     kind: str = Field(default="saved", description="'saved' | 'channel' | 'chat'")
463 +     title: str | None = None
464 +     is_output: bool = Field(
465 +         default=False,
466 +         description="ADR 0009: distinguishes output channels from ingest sources",
467 +     )
468 +     created_at: datetime = Field(default_factory=_utcnow)
469 +
470 +
471 +class RoutingRule(BaseModel):
472 +     """A rule that maps a source through a policy to a destination (ADR 0009).
473 +
474 +     Canonical name: ``routing_rules`` (locked in umbrella issue #25). When
475 +     ``policy_id`` is NULL, the account's default policy is used. When multiple
476 +     rules match, the highest ``priority`` wins.
477 +     """
478 +
479 +     model_config = ConfigDict(frozen=True)
480 +
481 +     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
482 +     account_id: int = Field(description="FK ? accounts.id")
483 +     source_id: int = Field(description="FK ? sources.id")
484 +     policy_id: int | None = Field(
485 +         default=None,
486 +         description="FK ? policies.id; NULL means use account default policy",
487 +     )
488 +     destination_id: int = Field(description="FK ? destinations.id")
489 +     enabled: bool = Field(default=True)
490 +     priority: int = Field(default=0, description="Higher wins when multiple rules
match")
491 +     created_at: datetime = Field(default_factory=_utcnow)
492 +
493 +
494 +class DeliveryStatus(str, Enum):
495 +     """Per-destination delivery state, replacing the legacy binary ``forwarded``."""
496 +
497 +     PENDING = "pending"
498 +     DELIVERED = "delivered"
499 +     FAILED = "failed"
500 +     RETRYING = "retrying"
501 +
502 +
503 +class DeliveryAttempt(BaseModel):
504 +     """Per-destination delivery record ? idempotent via UNIQUE index.
505 +
506 +     Replaces the legacy ``videos.forwarded`` boolean (ADR 0011). One row per
507 +     ``(content_item_id, destination_id)`` pair. On retry, ``attempt_count`` is
508 +     incremented and ``status``/``last_error`` are updated.

```

```

509 +     ""
510 +
511 +     model_config = ConfigDict(frozen=True)
512 +
513 +     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
514 +     content_item_id: int = Field(description="FK ? content_items.id")
515 +     destination_id: int = Field(description="FK ? destinations.id")
516 +     route_id: int | None = Field(default=None, description="FK ? routing_rules.id
(nullable)")
517 +     status: DeliveryStatus = Field(default=DeliveryStatus.PENDING)
518 +     attempt_count: int = Field(default=0, description="Incremented on each retry")
519 +     last_error: str | None = None
520 +     created_at: datetime = Field(default_factory=_utcnow)
521 +     delivered_at: datetime | None = None
522 +     next_retry_at: datetime | None = None
523 +
524 +
525     class FilterConfig(BaseModel):
526         """Per-user filter configuration.
527
528     diff --git a/tests/test_db.py b/tests/test_db.py
529     index de90b68..897861d 100644
530     --- a/tests/test_db.py
531     +++ b/tests/test_db.py
532     @@ -13,6 +13,8 @@
533         Account,
534         Channel,
535         ContentItem,
536     +     DeliveryStatus,
537     +     Destination,
538         FilterConfig,
539         Policy,
540         Source,
541     @@ -1449,3 +1451,205 @@ def test_safe_int_cast_parses_numeric_strings() -> None:
542         """_safe_int_cast must parse numeric strings (the common Telegram case)."""
543         assert db._safe_int_cast("-1001234567890", "provider_source_id") == -1001234567890
544         assert db._safe_int_cast("42", "provider_message_id") == 42
545     +
546     +
547     +# --- Track C: Destination / DeliveryAttempt repos (ADR 0011) -----
548     +
549     +
550     +async def test_upsert_destination_round_trip(conn: aiomysqlite.Connection) -> None:
551     +     """Insert then read back a destination."""
552     +     await _seed_user(conn)
553     +     dest = await db.upsert_destination(
554     +         conn,
555     +         Destination(account_id=USER_ID, provider="telegram", target="me",
kind="saved"),
556     +     )
557     +     assert dest.id is not None
558     +     fetched = await db.get_destination(conn, USER_ID, "telegram", "me")
559     +     assert fetched is not None
560     +     assert fetched.target == "me"
561     +     assert fetched.kind == "saved"
562     +
563     +
564     +async def test_upsert_destination_conflict_updates(conn: aiomysqlite.Connection) -> None:
565     +     """Re-upsert with same (account, provider, target) updates kind/title."""
566     +     await _seed_user(conn)
567     +     await db.upsert_destination(
568     +         conn,
569     +         Destination(account_id=USER_ID, provider="telegram", target="@ch",
kind="channel"),
570     +     )

```

```

571 +     updated = await db.upsert_destination(
572 +         conn,
573 +         Destination(
574 +             account_id=USER_ID,
575 +             provider="telegram",
576 +             target="@ch",
577 +             kind="channel",
578 +             title="My Feed",
579 +         ),
580 +     )
581 +     assert updated.title == "My Feed"
582 +     assert updated.kind == "channel"
583 +
584 +
585 + async def test_record_delivery_success(conn: aiomysqlite.Connection) -> None:
586 +     """record_delivery with success=True sets status=delivered."""
587 +     await _seed_user(conn)
588 +     src = await db.upsert_source(
589 +         conn, Source(account_id=USER_ID, provider="telegram", provider_source_id="-1")
590 +     )
591 +     dest = await db.upsert_destination(
592 +         conn, Destination(account_id=USER_ID, provider="telegram", target="me")
593 +     )
594 +     assert src.id is not None and dest.id is not None
595 +     item = await db.insert_content_item(
596 +         conn,
597 +         ContentItem(
598 +             account_id=USER_ID,
599 +             source_id=src.id,
600 +             provider="telegram",
601 +             provider_message_id="99",
602 +             dedup_key="tg:msg:1:99",
603 +         ),
604 +     )
605 +     assert item.id is not None
606 +     await db.record_delivery(conn, content_item_id=item.id, destination_id=dest.id,
607 + success=True)
608 +     attempt = await db.get_delivery_attempt(conn, item.id, dest.id)
609 +     assert attempt is not None
610 +     assert attempt.status == DeliveryStatus.DELIVERED
611 +     assert attempt.attempt_count == 1
612 +     assert attempt.delivered_at is not None
613 +
614 +
615 + async def test_record_delivery_failure(conn: aiomysqlite.Connection) -> None:
616 +     """record_delivery with success=False sets status=failed + error."""
617 +     await _seed_user(conn)
618 +     src = await db.upsert_source(
619 +         conn, Source(account_id=USER_ID, provider="telegram", provider_source_id="-1")
620 +     )
621 +     dest = await db.upsert_destination(
622 +         conn, Destination(account_id=USER_ID, provider="telegram", target="me")
623 +     )
624 +     assert src.id is not None and dest.id is not None
625 +     item = await db.insert_content_item(
626 +         conn,
627 +         ContentItem(
628 +             account_id=USER_ID,
629 +             source_id=src.id,
630 +             provider="telegram",
631 +             provider_message_id="100",
632 +             dedup_key="tg:msg:1:100",
633 +         ),
634 +     )
635 +     assert item.id is not None

```

```

635 +     await db.record_delivery(
636 +         conn,
637 +         content_item_id=item.id,
638 +         destination_id=dest.id,
639 +         success=False,
640 +         error="FloodWait(2s)",
641 +     )
642 +     attempt = await db.get_delivery_attempt(conn, item.id, dest.id)
643 +     assert attempt is not None
644 +     assert attempt.status == DeliveryStatus.FAILED
645 +     assert attempt.last_error == "FloodWait(2s)"
646 +
647 +
648 + async def test_record_delivery_is_idempotent(conn: aiosqlite.Connection) -> None:
649 +     """Repeated record_delivery updates the same row; attempt_count increments."""
650 +     await _seed_user(conn)
651 +     src = await db.upsert_source(
652 +         conn, Source(account_id=USER_ID, provider="telegram", provider_source_id="-1")
653 +     )
654 +     dest = await db.upsert_destination(
655 +         conn, Destination(account_id=USER_ID, provider="telegram", target="me")
656 +     )
657 +     assert src.id is not None and dest.id is not None
658 +     item = await db.insert_content_item(
659 +         conn,
660 +         ContentItem(
661 +             account_id=USER_ID,
662 +             source_id=src.id,
663 +             provider="telegram",
664 +             provider_message_id="101",
665 +             dedup_key="tg:msg:1:101",
666 +         ),
667 +     )
668 +     assert item.id is not None
669 +     await db.record_delivery(
670 +         conn, content_item_id=item.id, destination_id=dest.id, success=False,
671 +         error="first"
672 +     )
673 +     await db.record_delivery(conn, content_item_id=item.id, destination_id=dest.id,
674 +         success=True)
675 +     attempt = await db.get_delivery_attempt(conn, item.id, dest.id)
676 +     assert attempt is not None
677 +     assert attempt.status == DeliveryStatus.DELIVERED
678 +     assert attempt.attempt_count == 2 # failed + succeeded
679 +
680 +
681 +
682 + # --- Track C: Migration 004 -----
683 +
684 +
685 + async def test_migration_004_backfills_destinations_and_attempts(tmp_path: object) ->
686 + None:
687 +     """Migration 004 backfills default destinations from filter_config and
688 +     creates delivery_attempts for previously-forwarded videos."""
689 +     db_path = str(tmp_path) + "/mig004.db" # type: ignore[operator]
690 +     # Build a pre-ADR-0011 legacy DB with a user, channel, video, and filter_config.
691 +     legacy_schema = """
692 +     CREATE TABLE users (id INTEGER PRIMARY KEY, phone TEXT, enabled INTEGER NOT NULL
693 +     DEFAULT 1, created_at TEXT NOT NULL);
694 +     CREATE TABLE channels (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT
695 +     NULL REFERENCES users(id), channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
696 +     UNIQUE (user_id, channel_id));
697 +     CREATE TABLE videos (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL
698 +     REFERENCES users(id), channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL, duration_s
699 +     INTEGER, width INTEGER, height INTEGER, size_bytes INTEGER, mime_type TEXT, matched INTEGER NOT
700 +     NULL DEFAULT 0, forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL, UNIQUE (user_id,

```

```

channel_id, message_id));
691 + CREATE TABLE filter_config (user_id INTEGER PRIMARY KEY REFERENCES users(id),
config_json TEXT NOT NULL);
692 + """
693 + async with aiosqlite.connect(db_path) as conn:
694 +     await conn.executescript(legacy_schema)
695 +     await conn.execute("INSERT INTO users VALUES
(111, '+1', 1, '2024-01-01T00:00:00Z')")
696 +     await conn.execute(
697 +         "INSERT INTO channels (user_id, channel_id, title, seen_first_at) VALUES
(111, -1001, 'Ch', '2024-01-01T00:00:00Z')")
698 +     )
699 +     await conn.execute(
700 +         "INSERT INTO videos (user_id, channel_id, message_id, duration_s, width,
height, size_bytes, mime_type, matched, forwarded, seen_at) "
701 +         "VALUES (111, -1001, 42, 60, 1920, 1080, 1000, 'video/mp4', 1, 1,
'2024-01-01T00:00:00Z')")
702 +     )
703 +     # filter_config with @my-feed delivery_target
704 +     await conn.execute(
705 +         "INSERT INTO filter_config (user_id, config_json) VALUES (111,
\'{"delivery_target": "@my-feed", "delivery_mode": "forward"}\')")
706 +     )
707 +     await conn.commit()
708 +
709 +     await db.init_db(db_path)
710 +
711 +     async with aiosqlite.connect(db_path) as conn:
712 +         # Destination backfilled from filter_config.
713 +         async with conn.execute(
714 +             "SELECT target, kind FROM destinations WHERE account_id=111"
715 +         ) as cur:
716 +             d = await cur.fetchone()
717 +             assert d == ("@my-feed", "channel")
718 +
719 +         # Routing rule created (source ? default policy ? destination).
720 +         async with conn.execute(
721 +             "SELECT COUNT(*) FROM routing_rules WHERE account_id=111 AND enabled=1"
722 +         ) as cur:
723 +             assert (await cur.fetchone())[0] >= 1
724 +
725 +         # Delivery attempt backfilled for the forwarded video.
726 +         async with conn.execute("SELECT status, attempt_count FROM delivery_attempts")
as cur:
727 +             da = await cur.fetchone()
728 +             assert da is not None
729 +             assert da == ("delivered", 1)
730 +
731 +         # user_version bumped to latest migration.
732 +         async with conn.execute("PRAGMA user_version") as cur:
733 +             assert (await cur.fetchone())[0] == len(db._MIGRATIONS)
734 +
735 +
736 + async def test_migration_004_is_idempotent(tmp_path: object) -> None:
737 +     """Migration 004 is a no-op on second run (destinations already populated)."""
738 +     db_path = str(tmp_path) + "/idem004.db" # type: ignore[operator]
739 +     await db.init_db(db_path)
740 +     await db.init_db(db_path) # second run
741 +     async with aiosqlite.connect(db_path) as conn:
742 +         async with conn.execute("PRAGMA user_version") as cur:
743 +             assert (await cur.fetchone())[0] == len(db._MIGRATIONS)
744 +             assert await db._table_exists(conn, "destinations")
745 +             assert await db._table_exists(conn, "routing_rules")
746 +             assert await db._table_exists(conn, "delivery_attempts")
747 + diff --git a/tests/test_delivery.py b/tests/test_delivery.py

```

```

748     index e74206e..9e98191 100644
749     --- a/tests/test_delivery.py
750     +++ b/tests/test_delivery.py
751     @@ -15,7 +15,7 @@
752
753     from tg_video_filter import db
754     from tg_video_filter.delivery import build_link_text, deliver
755     -from tg_video_filter.models import DeliveryMode, User, VideoRecord
756     +from tg_video_filter.models import DeliveryMode, Destination, User, VideoRecord
757
758     USER_ID = 1
759     CHANNEL_ID = 100
760     @@ -248,6 +248,34 @@ async def
test_deliver_succeeds_without_forwarded_flag_post_adr_0011(
761         assert row.forwarded is False
762
763
764     +async def test_deliver_with_destination_id_records_delivery_attempt(
765     +    db_with_video: tuple[aiosqlite.Connection, int],
766     +) -> None:
767     +    """When destination_id is provided, record_delivery writes a DeliveryAttempt (Track
C)."""
768     +    c, vid = db_with_video
769     +    # Create a destination row for the test
770     +    dest = await db.upsert_destination(
771     +        c, Destination(account_id=USER_ID, provider="telegram", target="me")
772     +    )
773     +    assert dest.id is not None
774     +    client = FakeClient()
775     +    await deliver(
776     +        client,
777     +        _SENTINEL_MSG,
778     +        c,
779     +        vid,
780     +        DeliveryMode.LINK,
781     +        link_text="test",
782     +        target="me",
783     +        destination_id=dest.id, # type: ignore[arg-type]
784     +    )
785     +    # With destination_id, a DeliveryAttempt is written.
786     +    attempt = await db.get_delivery_attempt(c, vid, dest.id)
787     +    assert attempt is not None
788     +    assert attempt.status.value == "delivered"
789     +    assert attempt.attempt_count == 1
790     +
791     +
792     async def test_both_mode_logs_warning_on_link_failure(
793         db_with_video: tuple[aiosqlite.Connection, int],
794         caplog: pytest.LogCaptureFixture,
795     @@ -346,3 +374,35 @@ async def test_deliver_both_to_channel(
796         )
797         assert client.forward_targets == ["@curated"]
798         assert client.send_targets == ["@curated"]
799     +
800     +
801     +# --- FloodWait retry exhaustion (coverage improvement for delivery.py) -----
802     +
803     +
804     +async def test_flood_wait_retry_exhaustion(
805     +    db_with_video: tuple[aiosqlite.Connection, int], _no_sleep: None
806     +) -> None:
807     +    """When both initial send AND retry raise FloodWaitError, delivery returns
False."""
808     +    c, vid = db_with_video
809     +    client = FakeClient(

```





```
=== is upsert_delivery_attempt used anywhere (dead code)? ===
```

```
src/tg_video_filter/db.py:1756: "upsert_delivery_attempt",
```

```
=== is the 'source/destination invariant' actually enforced in code? ===
```

```
150:     is_output    INTEGER NOT NULL DEFAULT 0,
692:         INSERT OR IGNORE INTO destinations (account_id, provider, target, kind, is_output,
created_at)
890:         is_output=bool(row["is_output"]),
1231:     schema ? SQLite ``CHECK`` can't express "at most one per group").
1279:         (account_id, provider, target, kind, title, is_output, created_at)
1284:         is_output=excluded.is_output"",
1291:         int(dest.is_output),
```

### 153. assistant (2026-07-06T07:49:16.802Z)

Gate: \*\*451 passed @ 95.75%\*\* (matches PR claim; above floor). Notable: `db.py` dipped to 93%, `delivery.py` to 94% ? the uncovered `delivery.py` lines (145, 152, 181, 224) are exactly the \*\*`destination\_id` + failure\*\* recording paths (untested), and `upsert\_delivery\_attempt` is \*\*defined/exported but never called\*\* (dead). Let me confirm ruff and run repros for the BOTH-mode masking and migration edge cases.

### 154. user (2026-07-06T07:49:20.417Z)

```
=== ruff results (head of gate) ===
```

```
=== RUFF CHECK ===
```

```
All checks passed!
```

```
ruff-check-exit=0
```

```
=== RUFF FORMAT --CHECK ===
```

```
Would reformat: src\tg_video_filter\commands.py
```

```
1 file would be reformatted, 33 files already formatted
```

```
fmt-exit=1
```

```
=== PYTEST ===
```

```
..... [ 15%]
..... [ 31%]
..... [ 47%]
..... [ 63%]
```

### 155. assistant (2026-07-06T07:49:45.923Z)

```
"""Probe PR #37 (Track C) behaviors."""
```

```
import asyncio, aiosqlite, tempfile, os
```

```
from tg_video_filter import db
```

```
from tg_video_filter.models import Account, Source, ContentItem, Destination, DeliveryStatus
```

```
async def both_mode_masking():
```

```
    """BOTH mode: forward succeeds then link fails -> shared delivery_attempt row.
```

```
    Model the exact two record_delivery calls deliver() makes with one destination_id."""
```

```
    conn = await aiosqlite.connect(":memory:")
```

```
    await db._apply_schema(conn)
```

```
    await db.upsert_account(conn, Account(id=1, telegram_id=1))
```

```
    src = await db.upsert_source(conn, Source(account_id=1, provider="telegram",
provider_source_id="-1"))
```

```
    dest = await db.upsert_destination(conn, Destination(account_id=1, provider="telegram",
target="me"))
```

```
    item = await db.insert_content_item(conn, ContentItem(account_id=1, source_id=src.id,
provider="telegram", provider_message_id="5", dedup_key="tg:msg:1:5"))
```

```
    # deliver() BOTH: forward first (success), then link (failure) ? same destination_id
```

```
    await db.record_delivery(conn, content_item_id=item.id, destination_id=dest.id,
success=True)
```

```
    await db.record_delivery(conn, content_item_id=item.id, destination_id=dest.id,
success=False, error="link send failed")
```

```
    a = await db.get_delivery_attempt(conn, item.id, dest.id)
```

```
    await conn.close()
```

```
    print(f"[BOTH-mode] forward OK + link FAIL -> status={a.status.value!r}")
```

```

attempt_count={a.attempt_count} delivered_at={'set' if a.delivered_at else None}")
    print(f"          -> {'MASKS success (status=failed despite forward delivered)' if
a.status==DeliveryStatus.FAILED else 'ok'}")

async def migration_me_default():
    """Migration 004 with NO delivery_target in config -> default 'me'."""
    p = os.path.join(tempfile.mkdtemp(), "m.db")
    legacy = """
CREATE TABLE users (id INTEGER PRIMARY KEY, phone TEXT, enabled INTEGER NOT NULL DEFAULT 1,
created_at TEXT NOT NULL);
CREATE TABLE channels (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL
REFERENCES users(id), channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
UNIQUE(user_id,channel_id));
CREATE TABLE videos (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL,
channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL, duration_s INTEGER, width INTEGER,
height INTEGER, size_bytes INTEGER, mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0,
forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL,
UNIQUE(user_id,channel_id,message_id));
CREATE TABLE filter_config (user_id INTEGER PRIMARY KEY, config_json TEXT NOT NULL);
"""
    async with aiosqlite.connect(p) as conn:
        await conn.executescript(legacy)
        await conn.execute("INSERT INTO users VALUES (111,'+1',1,'2024-01-01T00:00:00Z')")
        await conn.execute("INSERT INTO channels(user_id,channel_id,title,seen_first_at) VALUES
(111,-1001,'C','2024-01-01T00:00:00Z')")
        await conn.execute("INSERT INTO
videos(user_id,channel_id,message_id,matched,forwarded,seen_at) VALUES
(111,-1001,42,1,1,'2024-01-01T00:00:00Z')")
        # config WITHOUT delivery_target
        await conn.execute("INSERT INTO filter_config VALUES (111, '{\"min_duration_s\":5}')")
        await conn.commit()
    await db.init_db(p)
    async with aiosqlite.connect(p) as conn:
        d = await (await conn.execute("SELECT target,kind FROM destinations WHERE
account_id=111")).fetchone()
        rr = (await (await conn.execute("SELECT COUNT(*) FROM routing_rules WHERE
account_id=111")).fetchone())[0]
        da = await (await conn.execute("SELECT status FROM delivery_attempts")).fetchone()
        print(f"[mig 'me' default] destination={d} routing_rules={rr} delivery_attempt={da}")
        print(f"          -> {'PASS' if d==('me','saved') and rr==1 and da==('delivered',) else
'CHECK'}")

async def migration_sources_no_filterconfig():
    """Account has channels/videos but NO filter_config row -> what happens?"""
    p = os.path.join(tempfile.mkdtemp(), "n.db")
    legacy = """
CREATE TABLE users (id INTEGER PRIMARY KEY, phone TEXT, enabled INTEGER NOT NULL DEFAULT 1,
created_at TEXT NOT NULL);
CREATE TABLE channels (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL
REFERENCES users(id), channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
UNIQUE(user_id,channel_id));
CREATE TABLE videos (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL,
channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL, matched INTEGER NOT NULL DEFAULT 0,
forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL,
UNIQUE(user_id,channel_id,message_id));
CREATE TABLE filter_config (user_id INTEGER PRIMARY KEY, config_json TEXT NOT NULL);
"""
    async with aiosqlite.connect(p) as conn:
        await conn.executescript(legacy)
        await conn.execute("INSERT INTO users VALUES (222,'+2',1,'2024-01-01T00:00:00Z')")
        await conn.execute("INSERT INTO channels(user_id,channel_id,title,seen_first_at) VALUES
(222,-2002,'C2','2024-01-01T00:00:00Z')")
        await conn.execute("INSERT INTO
videos(user_id,channel_id,message_id,matched,forwarded,seen_at) VALUES
(222,-2002,7,1,1,'2024-01-01T00:00:00Z')")

```

```

        # NO filter_config row for user 222
        await conn.commit()
    await db.init_db(p)
    async with aiosqlite.connect(p) as conn:
        dest = (await (await conn.execute("SELECT COUNT(*) FROM destinations WHERE
account_id=222")).fetchone())[0]
        rr = (await (await conn.execute("SELECT COUNT(*) FROM routing_rules WHERE
account_id=222")).fetchone())[0]
        da = (await (await conn.execute("SELECT COUNT(*) FROM
delivery_attempts")).fetchone())[0]
        pol = (await (await conn.execute("SELECT COUNT(*) FROM policies WHERE
account_id=222")).fetchone())[0]
        print(f"[mig no-filter_config] policies={pol} destinations={dest} routing_rules={rr}
delivery_attempts={da} (forwarded video exists)")

async def main():
    await both_mode_masking()
    await migration_me_default()
    await migration_sources_no_filterconfig()

asyncio.run(main())

```

#### 156. user (2026-07-06T07:49:45.955Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/reverify37.py (file state is current in your context ? no need to Read it back)

#### 157. assistant (2026-07-06T07:49:56.282Z)

`ruff format --check` flags `commands.py` again ? but that's only because this branch predates my merged #36 fix (the branch is 2 commits behind main); #37's own files are clean, and a rebase resolves it. Now let me run the behavioral repos.

#### 158. user (2026-07-06T07:49:59.714Z)

```

Exit code 1
[BOTH-mode] forward OK + link FAIL -> status='failed' attempt_count=2 delivered_at=set
-> MASKS success (status=failed despite forward delivered)
[mig 'me' default] destination=('me', 'saved') routing_rules=1 delivery_attempt=('delivered',)
-> PASS

Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\reverify37.py", line 77, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 195, in run
    return runner.run(main)
           ~~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 118, in run
    return self._loop.run_until_complete(task)
           ~~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725, in run_until_complete
    return future.result()
           ~~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\reverify37.py", line 75, in main
    await migration_sources_no_filterconfig()
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\reverify37.py", line 64, in
migration_sources_no_filterconfig

```

```

    await db.init_db(p)
File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 196, in init_db
    await _apply_schema(conn)
File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 216, in _apply_schema
    await _apply_migrations(conn)
File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 324, in
_apply_migrations
    await fn(conn)
File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 484, in
_migration_002_identity_split
    await conn.execute(
        f"INSERT INTO {table} ({copy_cols}) SELECT {copy_cols} FROM {table}__legacy_data"
    )
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 223,
in execute
    cursor = await self._execute(self._conn.execute, sql, parameters)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 160,
in _execute
    return await future
        ^^^^^^^^^^^^^^^
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 63, in
_connection_worker_thread
    result = function()
sqlite3.OperationalError: no such column: duration_s

```

#### 159. assistant (2026-07-06T07:50:17.863Z)

Two results landed before my third repro hit a **bug in my own test schema** (I omitted ``duration_s`` etc. that migration 002 copies ? not a code issue):

- **BOTH-mode masking CONFIRMED:** forward OK + link FAIL ? ``status='failed'``, ``attempt_count=2``, but ``delivered_at`` is set ? the failed link masks the successful forward.
- **Migration 'me' default: PASS** ? destination ``('me','saved')``, 1 routing rule, `delivery_attempt`delivered``.

Let me fix my repro's legacy schema and re-check the "sources but no filter\_config" edge.

#### 160. user (2026-07-06T07:50:30.344Z)

account with sources+forwarded video but NO filter\_config:  
 policies=0 destinations=0 routing\_rules=0 delivery\_attempts=0 (forwarded videos=1)  
 -> forwarded delivery history LOST (no attempt backfilled)

#### 161. assistant (2026-07-06T07:50:49.983Z)

Confirmed: an account with sources + a forwarded video but **no `filter\_config` row** gets 0 destinations, 0 routing rules, and 0 delivery\_attempts ? its forwarded-delivery history isn't backfilled, and it ends up with no delivery path at all. Let me check whether bootstrap guarantees a ``filter_config`` (which would bound this severity), then launch the exhaustive review workflow.

#### 162. user (2026-07-06T07:50:53.943Z)

```

=== does bootstrap/onboarding persist a filter_config row? ===
src/tg_video_filter/bootstrap.py:29:from tg_video_filter.models import DEFAULT_FILTER_CONFIG,
Account, TelegramSession, User
src/tg_video_filter/bootstrap.py:75:         await db.save_filter_config(conn, user.id,
DEFAULT_FILTER_CONFIG)
src/tg_video_filter/commands.py:198:     (Saved Messages). It persists config changes via
`db.save_filter_config`
src/tg_video_filter/commands.py:262:         current_cfg = await db.load_filter_config(conn,
user.id)
src/tg_video_filter/commands.py:296:         current_cfg = await db.load_filter_config(conn,
user.id)

```

```
src/tg_video_filter/commands.py:374:         cfg = await db.load_filter_config(conn, user.id)
src/tg_video_filter/commands.py:401:     cfg = await db.load_filter_config(conn, user_id)
src/tg_video_filter/commands.py:416:     await db.save_filter_config(conn, user_id, cfg)
src/tg_video_filter/manager.py:140:         cfg = await db.load_filter_config(conn,
session.account_id)
```

```
=== ADR 0011 destinations.is_output intent (for backfill is_output=0 question) ===
...
```

```
destinations (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    account_id  INTEGER NOT NULL REFERENCES accounts(id),
    provider    TEXT NOT NULL DEFAULT 'telegram',
    target      TEXT NOT NULL,          -- 'me', '@username', chat id (ADR 0005)
    kind        TEXT NOT NULL DEFAULT 'saved', -- 'saved' | 'channel' | 'chat'
    title       TEXT,
    is_output   INTEGER NOT NULL DEFAULT 0,    -- ADR 0009: distinguishes output channels
    created_at  TEXT NOT NULL,
    UNIQUE (account_id, provider, target)
);
...
```

### 163. assistant (2026-07-06T07:51:34.122Z)

Bootstrap always persists a default `filter\_config` (`bootstrap.py:75`), so the "no filter\_config" gap is bounded to LOW (unreachable via normal onboarding). I now have solid manual findings. Per the exhaustive-review mandate, let me launch a focused workflow to independently verify the BOTH-mode masking finding and sweep migration-004 SQL correctness, delivery integration, ADR conformance, and test gaps.

### 164. user (2026-07-06T07:52:24.456Z)

Workflow launched in background. Task ID: w5qexpl4p  
Summary: Exhaustive review of telegram-video-filter PR #37 (ADR 0011 Track C: destinations/routing/delivery\_attempts) with adversarial verification  
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_ae3ec722-12a  
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr37-track-c-wf\_ae3ec722-12a.js  
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr37-track-c-wf\_ae3ec722-12a.js"}) to iterate without resending the script.)  
Run ID: wf\_ae3ec722-12a  
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr37-track-c-wf\_ae3ec722-12a.js", resumeFromRunId: "wf\_ae3ec722-12a"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

### 165. assistant (2026-07-06T07:52:34.402Z)

Workflow is running. Let me confirm my "latent until Track D" framing by verifying no live caller passes `destination\_id` yet.

### 166. user (2026-07-06T07:52:38.139Z)

```
=== live callers of deliver()/forward_to_target/send_link_message ? do any pass destination_id?
===
src/tg_video_filter/telethon_client.py:9:         -> db.insert_video(...) +
deliver.forward_to_target(..., target=cfg.delivery_target)
src/tg_video_filter/telethon_client.py:299:         await deliver(
src/tg_video_filter/backfill.py:106:         ``FilterEngine``, and the ``deliver()`` dispatcher.
```

Videos already recorded  
src/tg\_video\_filter/backfill.py:244: await deliver(  
(no destination\_id in src/ non-delivery files ? live path uses None ? Track C recording is  
latent)

### 167. assistant (2026-07-06T07:52:54.701Z)

Confirmed ? the live callers (`telethon\_client.py:299`, `backfill.py:244`) call `deliver()`  
without `destination\_id`, so Track C's delivery-recording is \*\*latent until Track D\*\*.

While the workflow runs its adversarial pass, here's my current read:

**\*\*Gate (local, branch `100abb2`):\*\*** ? `ruff check` clean . ? **\*\*451 passed @ 95.75%\*\*** (floor 80%,  
matches PR claim) . migration 004 idempotent. `ruff format` flags only `commands.py` ? an  
artifact of the branch predating my merged #36 fix, not #37.

**\*\*Findings so far (reproduced):\*\***  
- ? **\*\*BOTH-mode status masking\*\*** (latent): forward succeeds + link fails ? the shared  
`delivery\_attempt` row ends `status=FAILED` with `delivered\_at` set ? the failed link masks the  
successful forward, and `attempt\_count` over-counts. The failure-recording path is also untested  
( `delivery.py:145,152,181,224` uncovered).  
- ? **\*\*Dead code\*\***: `upsert\_delivery\_attempt` is exported but never called/tested.  
- ? **\*\*Overclaiming comment\*\***: models.py says Track C "enforces the source/destination invariant"  
? nothing enforces it.  
- ? Migration edge: account with sources but no `filter\_config` gets no  
destination/routing/delivery history ? bounded LOW since bootstrap always persists one.  
- ? Coverage dipped 96.5?95.75%; typo "destinatins"; branch should rebase on main.

**\*\*Migration 004 verified correct\*\*** for the normal paths ('me' default and '@channel'). I'll  
consolidate once the workflow confirms/extends these.

### 168. user (2026-07-06T08:08:47.658Z)

```
<task-notification>
<task-id>w5qexpl4p</task-id>
<tool-use-id>toolu_012xQHGFyrg57AUzSUNagRTA</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\w5qexpl4p.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Exhaustive review of telegram-video-filter PR #37 (ADR 0011 Track C:
destinations/routing/delivery_attempts) with adversarial verification" completed</summary>
<result>{"counts":{"raw":17,"kept":15,"refuted":2},"findings":[{"dimension":"migration_004","sev
erity":"medium","verdict":"CONFIRMED","live_or_latent":"live","title":"Malformed/non-object
config_json aborts the entire migration ladder on
upgrade","where":"src/tg_video_filter/db.py:694","summary":"Migration 004 parses
filter_config_deprecated.config_json with json_extract(); if any row's config_json is not valid
JSON, json_extract raises sqlite3.OperationalError and the whole init_db upgrade fails, so the
app cannot start.","failure_scenario":"A legacy DB where any filter_config row's config_json is
not strictly-valid JSON (hand-edited, truncated on a prior crash, or written by an older/foreign
tool). On the first upgrade, _migration_004 executes step 1's INSERT ... SELECT
COALESCE(json_extract(config_json,'$.delivery_target'),'me') ... FROM filter_config_deprecated.
json_extract aborts with 'malformed JSON', _apply_migrations propagates the exception, the
ladder rolls back, and every subsequent startup re-hits the same crash. Reproduced: seeding a
filter_config row with config_json='not-json-at-all' makes init_db raise
sqlite3.OperationalError: malformed JSON (scenario G).","evidence":"Repro (scratchpad/repro5.py,
scenario G) seeds a legacy DB with config_json='not-json-at-all' and init_db() raises
sqlite3.OperationalError: malformed JSON at db.py:690 (the step-1 INSERT). Migration 003
previously only copied config_json verbatim into policies (db.py:624) and never validated it, so
004 is the first consumer to parse it. Reachability is low because the live save_filter_config
always writes cfg.model_dump_json() (db.py:1684), i.e. valid JSON ? hence medium, not
blocker.","recommendation":"Guard the parse against malformed JSON, e.g. wrap the extract in
CASE WHEN json_valid(config_json) THEN json_extract(config_json,'$.delivery_target') ELSE NULL
END (also in the routing_rules subquery at db.py:718), so a corrupt row degrades to the 'me'
default instead of aborting the whole upgrade.","verify_reasoning":"Verified end-to-end. (1)
json_extract on malformed JSON: standalone repro shows `SELECT
```

COALESCE(json\_extract(config\_json,'\$.delivery\_target'),'me')` on config\_json='not-json-at-all' raises sqlite3.OperationalError: malformed JSON. (2) Full-ladder repro (faithful to tests/test\_db.py's legacy seed, videos includes duration\_s/width/height/size\_bytes/mime\_type per harness requirement): seeding filter\_config with config\_json='not-json-at-all' makes db.init\_db() raise OperationalError: malformed JSON, while config\_json='{}' and '{"delivery\_target":"me"}' migrate cleanly. The failing statement is migration 004 step-1 INSERT at db.py:690-704. Because \_apply\_migrations runs inside \_apply\_schema before conn.commit() (db.py:216-217), the exception propagates out of init\_db and the transaction never commits, so user\_version is not bumped and every subsequent startup re-hits the same crash ? the app cannot start. (3) The finding's diff-newness claim holds: migration 003 (db.py:624) copies config\_json verbatim into policies with no parsing, so migration 004 (this PR's diff) is genuinely the first consumer to json\_extract it. (4) The same unguarded json\_extract also appears in the routing\_rules subquery at db.py:716-719, so step 2 would fail too. (5) Reachability is low: the live save path always writes cfg.model\_dump\_json() (db.py:1684), i.e. valid JSON, so a malformed row can only come from hand-editing, crash-truncation, or a foreign/older tool ? not from data this app wrote. The defect ships in this PR's diff, is LIVE-NOW (runs on any legacy upgrade), and produces exactly the described startup-abort behaviour. The suggested json\_valid()/CASE guard (both at db.py:694 and db.py:718) is a correct fix.},{\"dimension\":\"delivery\_integration\",\"severity\":\"medium\",\"verdict\":\"CONFIRMED\",\"live\_or\_latent\":\"latent\",\"title\":\"BOTH mode shares one delivery\_attempts row across forward+link: a failed link masks a successful forward (status=failed, stale delivered\_at)\",\"where\":\"src/tg\_video\_filter/delivery.py:214\",\"summary\":\"In BOTH mode, forward\_to\_target and send\_link\_message are called with the SAME destination\_id, so both record\_delivery calls target the single UNIQUE(content\_item\_id, destination\_id) row. If the forward succeeds and the link then fails, the row is UPSERTed from status=delivered to status=failed while delivered\_at (set by the successful forward) is preserved, producing a self-contradictory row that reports the whole logical delivery as failed.\",\"failure\_scenario\":\"User has delivery\_mode=BOTH. deliver() forwards the video successfully (record\_delivery success=True -> row: status=delivered, attempt\_count=1, delivered\_at=T). send\_message then raises (link fails) -> record\_delivery success=False -> ON CONFLICT sets status=failed, attempt\_count=2, keeps delivered\_at=T, last\_error='link'. A future Track-D retry/idempotency consumer reading status=failed re-delivers (re-forwards) the already-forwarded video, or reports the delivery as failed even though the video reached the user. Confirmed by repro: 'BOTH: forward-ok then link-fail -> status=failed count=2 delivered\_at=True last\_error=link'.\", \"evidence\":\"delivery.py:214-220 calls forward\_to\_target then send\_link\_message with the same destination\_id. forward\_to\_target (145-147) and send\_link\_message (174-176/181-183) both call db.record\_delivery on that (video\_id, destination\_id) pair. record\_delivery (db.py:1373-1385) UPSERTs onto UNIQUE(content\_item\_id,destination\_id) (db.py:178), overwriting status with excluded.status but keeping delivered\_at via CASE WHEN...ELSE delivery\_attempts.delivered\_at. Repro via db.\_apply\_schema + two record\_delivery calls confirms the final row is status=failed with delivered\_at still set.\",\"recommendation\":\"Model a logical delivery as one attempt per destination, not one per Telethon call. In BOTH mode either (a) record a single combined outcome after both sends (success only if both succeeded, and don't clear/keep delivered\_at inconsistently), or (b) give forward vs link distinct destination rows / an op discriminator so the link failure can't overwrite the forward's delivered status. At minimum, don't leave delivered\_at set on a status=failed row. Since destination\_id is None on all live callers today this is latent, but fix before Track D wires routing in.\",\"verify\_reasoning\":\"The finding is accurate and reproduces exactly as described. In BOTH mode, delivery.py:214-220 calls forward\_to\_target (215-217) then send\_link\_message (218-220) with the SAME destination\_id. delivery\_attempts is keyed only by UNIQUE(content\_item\_id, destination\_id) (db.py:178) with no op/route discriminator to separate the forward outcome from the link outcome, so both record\_delivery calls collapse into one row. On a forward-ok/link-fail sequence, record\_delivery (db.py:1373-1396) UPSERTs the row from status=delivered to status=failed, increments attempt\_count to 2, sets last\_error='link', and preserves delivered\_at via the ELSE branch (db.py:1384-1385). The row ends up self-contradictory: status=failed with delivered\_at still populated, reporting the whole logical delivery as failed even though the video was successfully forwarded.\\n\\nRepro (C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe against db.\_apply\_schema on aiosqlite :memory:) output: 'after forward-ok -> status=DELIVERED count=1 delivered\_at\_set=True' then 'after link-fail -> status=FAILED count=2 delivered\_at\_set=True last\_error=link'. (The trailing 'Event loop is closed' is harmless connection-teardown noise, unrelated to the result.)\\n\\nLatency is correctly characterized as LATENT: all live callers (telethon\_client/manager) pass destination\_id=None, guarded by 'if destination\_id is not None' at delivery.py:144,151,173,180, so record\_delivery is never invoked in production yet ? only tests exercise it. The wrong behavior ships in this PR's diff (delivery.py + db.py

```
record_delivery are new here) but is dormant until Track D
... (truncated 48343 chars, full result in C:\Users\avidu\AppData\Local\Temp\claude\C--Users-
avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\w5qexpl4p.output)</result>
<usage><agent_count>21</agent_count><subagent_tokens>999634</subagent_tokens><tool_uses>234</tool_uses><duration_ms>983198</duration_ms></usage>
</task-notification>
```

#### 169. assistant (2026-07-06T08:08:50.764Z)

The workflow finished: **\*\*17 raw ? 15 kept, 2 refuted\*\***, and it surfaced a new **\*\*live\*\*** finding I'd missed (malformed `config\_json` aborting the upgrade). Let me parse the full result.

#### 170. user (2026-07-06T08:08:56.299Z)

```
COUNTS: {'raw': 17, 'kept': 15, 'refuted': 2}
```

```
=====
[MEDIUM/CONFIRMED/live] (migration_004) Malformed/non-object config_json aborts the entire
migration ladder on upgrade
```

```
  where: src/tg_video_filter/db.py:694
```

```
  rec : Guard the parse against malformed JSON, e.g. wrap the extract in CASE WHEN
json_valid(config_json) THEN json_extract(config_json,'$.delivery_target') ELSE NULL END (also
in the routing_rules subquery at db.py:718), so a corrupt row degrades
```

```
[MEDIUM/CONFIRMED/latent] (delivery_integration) BOTH mode shares one delivery_attempts row
across forward+link: a failed link masks a successful forward (status=failed, stale
delivered_at)
```

```
  where: src/tg_video_filter/delivery.py:214
```

```
  rec : Model a logical delivery as one attempt per destination, not one per Telethon call. In
BOTH mode either (a) record a single combined outcome after both sends (success only if both
succeeded, and don't clear/keep delivered_at inconsistently)
```

```
[MEDIUM/CONFIRMED/latent] (delivery_integration) BOTH mode: a forward FAILURE is silently erased
when the subsequent link SUCCEEDS
```

```
  where: src/tg_video_filter/delivery.py:215
```

```
  rec : Same root fix: represent forward and link as distinct delivery facts (or a single
combined outcome that is only 'delivered' when the mode's required sends all succeeded). Do not
let a link success mask a forward failure.
```

```
[LOW/CONFIRMED/latent] (migration_004) Backfilled destination kind='channel' for numeric chat
ids; ADR distinguishes 'chat'
```

```
  where: src/tg_video_filter/db.py:695
```

```
  rec : Either extend the CASE to classify '@'-prefixed targets as 'channel' and numeric ids
as 'chat' (or leave a documented note that Track C intentionally defers kind refinement), so the
backfilled value matches the ADR vocabulary before Track E
```

```
[LOW/PLAUSIBLE/latent] (migration_004) Backfilled delivery destination has is_output=0 despite
being an output channel
```

```
  where: src/tg_video_filter/db.py:700
```

```
  rec : Set is_output=1 for the backfilled delivery destination (it is the account's output
target), or add a comment that Track E must not assume backfilled destinations carry a
meaningful is_output flag.
```

```
[LOW/CONFIRMED/latent] (migration_004) Account with legacy sources/forwarded videos but no
filter_config gets zero destinations/routes/history
```

```
  where: src/tg_video_filter/db.py:702
```

```
  rec : Optionally add a fallback that creates a default 'me' destination for any account that
has sources but no filter_config_deprecated row, so no legacy account is left routeless and its
forwarded history is still recorded; or document that thi
```

```
[LOW/CONFIRMED/latent] (delivery_integration) BOTH mode over-counts attempt_count (2 per single
logical delivery, even on full success)
```

```
  where: src/tg_video_filter/db.py:1382
```

```
  rec : Increment attempt_count once per logical delivery attempt, not per Telethon send. Tie
to the combined-outcome fix for the masking finding above.
```



[LOW/CONFIRMED/latent] (delivery\_integration) record\_delivery failure-recording branches (delivery.py:152,181 and 145,224) are uncovered/untested with destination\_id

- where: src/tg\_video\_filter/delivery.py:152
- rec : Add a test that drives forward-ok + link-fail in BOTH mode WITH destination\_id and asserts the resulting delivery\_attempts row ? which will also pin down the intended semantics for the masking finding once fixed.

[LOW/CONFIRMED/latent] (models\_adr) models.py Track-C header comment overclaims an unenforced source/destination invariant

- where: src/tg\_video\_filter/models.py:216
- rec : Reword the comment to describe intent/future work (e.g. 'Track D will need to enforce ...') rather than claiming Track C enforces it, or actually add the enforcement (a validator or a migration-time guard). As written the comment is false f

[LOW/CONFIRMED/latent] (models\_adr) Typo 'destinatins' in models.py Track-C comment

- where: src/tg\_video\_filter/models.py:214
- rec : Fix the spelling to 'destinations' and, optionally, correct 'filter\_config.delivery\_target' to 'filter\_config\_deprecated.config\_json (\$.delivery\_target)' to match \_migration\_004.

[LOW/CONFIRMED/latent] (tests) upsert\_delivery\_attempt is dead code: exported in \_\_all\_\_, zero callers, zero tests

- where: src/tg\_video\_filter/db.py:1320
- rec : Either add a direct unit test for upsert\_delivery\_attempt (insert then conflict-update, assert attempt\_count/status/delivered\_at round-trip) or drop the function and its \_\_all\_\_ entry until Track D actually needs it. Shipping an untested ex

[LOW/CONFIRMED/latent] (tests) FAILURE delivery-recording path (record\_delivery success=False via deliver) is completely untested

- where: src/tg\_video\_filter/delivery.py:151
- rec : Add a test: create a Destination, drive deliver(...FORWARD..., destination\_id=id) with FakeClient(side\_effects=[RuntimeError, RuntimeError]) (and a LINK variant), assert deliver returns False and get\_delivery\_attempt shows status=failed / a

[LOW/CONFIRMED/latent] (tests) FORWARD-mode success recording (forward\_to\_target + destination\_id) is untested

- where: src/tg\_video\_filter/delivery.py:144
- rec : Extend the destination\_id test to also cover DeliveryMode.FORWARD (and ideally BOTH), asserting a delivered DeliveryAttempt is written for the forward leg.

[LOW/CONFIRMED/latent] (tests) Migration 004 'me' default (delivery\_target key absent) is untested

- where: src/tg\_video\_filter/db.py:694
- rec : Add a migration test variant seeding a filter\_config with no delivery\_target key, asserting the destination is (target='me', kind='saved') and a matching routing\_rule + attempt are produced.

[LOW/CONFIRMED/latent] (tests) BOTH-mode + destination\_id masking (forward ok / link fail) is untested

- where: src/tg\_video\_filter/delivery.py:224
- rec : Add a test: seed a Destination, deliver(...BOTH..., destination\_id=id) with FakeClient(side\_effects=[None, RuntimeError]) (forward ok, link fail); assert deliver returns True and that the delivery\_attempt reflects the recorded outcome for t

--- REFUTED ---

(migration\_004) Idempotency gate COUNT(destinations)>0 is not crash-safe if a commit is ever introduced between migrations @ src/tg\_video\_filter/db.py:681

- why: The described partial-run trap cannot occur in this PR's diff. Verified control flow: \_apply\_migrations (db.py:315-325) issues NO per-migration commit ? only PRAGMA user\_version between migrations ? a
- (tests) Migration 004: account with forwarded videos but no filter\_config row is untested and silently drops delivery history @ src/tg\_video\_filter/db.py:685
- why: The SQL mechanism the finding describes is real (confirmed by repro), but the failure scenario it asserts ? "any account... e.g. a defaults-only user... loses all its forwarded history on upgrade" ? d

### 171. assistant (2026-07-06T08:09:08.276Z)

The workflow corroborated my findings and surfaced a valuable **new LIVE finding**: a malformed ``config_json`` in a legacy row makes migration 004's ``json_extract`` abort the whole upgrade ? bricking startup. It also caught the inverse masking case (forward fails / link succeeds ? forward failure erased) and correctly refuted 2 weak items. Per my gate, let me reproduce the new live finding myself before trusting it.

### 172. user (2026-07-06T08:09:21.200Z)

```
['valid {}'] init_db OK
  retry: OK (recovered)
['malformed'] init_db RAISED: OperationalError: malformed JSON
  retry: STILL RAISES OperationalError -> app cannot start
['valid me'] init_db OK
  retry: OK (recovered)
```

### 173. assistant (2026-07-06T08:10:13.339Z)

Reproduced and **confirmed**: a malformed ``config_json`` row makes ``init_db`` raise ``OperationalError: malformed JSON`` ? and it **still raises on every retry**, so the app can't start. Low reachability (the app only ever writes valid ``model_dump_json``), but an unrecoverable startup brick with a one-line fix.

Here's the consolidated review.

---

## Code Review ? PR #37: destinations, routing\_rules, delivery\_attempts (ADR 0011, Tra

**Verdict**: ? Solid, mergeable ? no blockers. Clean models, correct core migration, backwards-compatible delivery. I'd fix the two **medium** items below before merge (one is a cheap startup-safety guard; the other is a real logic bug that Track D will step on), but neither blocks. Everything was reproduced locally.

**Ramp-up**: synced to remote head. Branch ``feat/track-c-delivery-routing`` (one commit ``100abb2``) ? ``main``. Note **PR #35 merged** and my **#36 format fix merged** since last review; this branch is 2 commits behind main but conflict-free (main's 2 commits are just ``commands.py``, which #37 doesn't touch).

**Local gate**: ``ruff check`` clean . ? **451 passed @ 95.75%** (floor 80%, matches PR claim) . migration 004 idempotent. ``ruff format`` flags only ``commands.py`` ? because the branch predates my merged #36 fix, **not** #37 (a rebase clears it).

### ? Medium ? worth fixing before merge

**1. Malformed ``config_json`` aborts the upgrade ? unrecoverable startup brick** (LIVE) ? ``db.py:694`` (and the routing subquery ``db.py:718``)  
Migration 004 does ``json_extract(config_json, '$.delivery_target')``. If any legacy ``filter_config`` row holds non-JSON, ``json_extract`` raises ``OperationalError: malformed JSON``, the ladder rolls back, and **every subsequent startup re-hits it** ? the app can't start. Reproduced: ``config_json='not-json-at-all'`` ? ``init_db`` raises, retry still raises. Migration 003 only copied ``config_json`` verbatim, so 004 is the first consumer to parse it. *\*Reachability is low\** (the app always writes valid ``model_dump_json``), hence medium not blocker.  
> **Fix**: ``CASE WHEN json_valid(config_json) THEN json_extract(config_json, '$.delivery_target') ELSE NULL END`` at both sites ? a corrupt row degrades to the ``me`` default instead of bricking the upgrade.

**2. BOTH-mode delivery masking** (LATENT until Track D) ? ``delivery.py:214-220``, ``record_delivery`` ``db.py:~1373``  
``deliver()`` BOTH mode calls ``forward_to_target`` then ``send_link_message`` with the **same** ``destination_id``, so both ``record_delivery`` calls collide on the single ``UNIQUE(content_item_id, destination_id)`` row. Reproduced both directions:

- forward **\*\*ok\*\*** + link **\*\*fail\*\*** ? row ends ``status=failed`` with ``delivered_at`` still set (failed link masks the successful forward);
- forward **\*\*fail\*\*** + link **\*\*ok\*\*** ? the forward failure is erased.

Plus ``attempt_count`` over-counts (2 per single logical BOTH delivery). A Track-D retry/idempotency consumer reading ``status`` would re-deliver an already-forwarded video or misreport. It's dormant today (all live callers pass ``destination_id=None``) but ships in this diff.

> **\*\*Fix before Track D:\*\*** record one combined outcome per destination (delivered only if the mode's required sends all succeeded; don't leave ``delivered_at`` on a ``failed`` row), or give forward/link distinct rows. The natural place to add the missing test is now.

## ? Low / hygiene

- **\*\*Untested new paths\*\*** ? the ``destination_id`` **\*\*failure\*\*** recording branch and the FORWARD/BOTH success paths are uncovered (``delivery.py:144,145,152,181,224``); coverage dipped 96.5?95.75%. Adding the BOTH-mode test above covers most of it.
- **\*\*Dead code\*\*** ? ``upsert_delivery_attempt`` is exported in ``__all__`` but has zero callers/tests (``db.py:1320``). Test it or drop it until Track D needs it.
- **\*\*Migration edges\*\*** ? an account with legacy sources but no ``filter_config`` gets no destination/route/history (bounded: ``bootstrap.py:75`` always persists one); numeric chat ids backfill as ``kind='channel'`` where ADR distinguishes ``chat``; the output destination gets ``is_output=0``. All latent/cosmetic.
- **\*\*Overclaiming comment\*\*** ? ``models.py:216`` says Track C "enforces the source/destination invariant (a channel is either source or destination, never both)" ? nothing enforces it. Reword or enforce.
- **\*\*Nit\*\*** ? typo "destinatinsons" (``models.py:214``); comment says ``filter_config.delivery_target`` but the migration reads ``filter_config_deprecated.config_json``.
- **\*\*Rebase\*\*** on current ``main`` ? clears the ``commands.py`` format flag and de-risks the merge.

## Refuted (correctly, by the adversarial pass)

- Idempotency-gate crash-safety concern ? ``_apply_migrations`` issues no per-migration commit, so a crash rolls back atomically; the gate is safe.
- An overstated "silent history loss" framing of the no-``filter_config`` edge ? the properly-scoped low version is kept above.

## ? Verified good

- Migration 004 correct for the normal paths (``me`` default and ``@channel`` both produce destination + routing rule + backfilled ``delivered`` attempt); idempotent (tested + reproduced).
- ``record_delivery`` increment/``delivered_at`` logic is correct for the single-send (FORWARD/LINK) case.
- ``delivery.py`` is genuinely backwards-compatible ? ``destination_id=None`` preserves the existing ``mark_forwarded`` no-op for all live callers.
- Models are frozen and conform to the ADR 0011 DDL; ``DeliveryStatus`` enum matches.

---

**\*\*Bottom line:\*\*** ship-ready after the two mediums (#1 is a 1-line guard; #2 can be a "fix before Track D" with a test added now). Want me to **\*\*post this to PR #37\*\*** (review comment, as before), and/or **\*\*draft the fixes\*\*** for #1 and #2 with the missing tests?

### 174. user (2026-07-06T08:11:02.657Z)

please post this as comment and let the author fix all of them

### 175. assistant (2026-07-06T08:11:09.757Z)

Posting the full review as a comment on PR #37, framed as an actionable checklist for the author to address. (As before, it goes as a comment ? GitHub blocks the author from a formal request-changes on their own PR.)

### 176. assistant (2026-07-06T08:11:37.934Z)

## Code review ? destinations, routing\_rules, delivery\_attempts (ADR 0011, Track C)

**\*\*Verdict:** ? Solid, no blockers ? please address the items below before merge.\*\* Clean models, correct core migration, backwards-compatible delivery. Two **\*\*medium\*\*** items are worth fixing before merge (one is a cheap startup-safety guard; the other is a real logic bug Track D will step on); the rest are low/hygiene. Every finding was reproduced locally against `100abb2` (posting as a comment ? GitHub blocks the author from a formal request-changes).

### Local gate

```
| Check | Result |
|---|---|
| `pytest` (full suite) | ? 451 passed, coverage **95.75%** (floor 80%) |
| `ruff check .` | ? clean |
| `ruff format --check .` | ?? flags `commands.py` **only** because this branch predates the
merged #36 fix ? not this PR (a rebase clears it) |
| migration 004 idempotency | ? verified |
```

> Note: this branch is 2 commits behind `main` (missing the merged #36); conflict-free since #37 doesn't touch `commands.py`. A rebase clears the format flag and de-risks the merge.

---

## ? Medium ? fix before merge

### 1. Malformed `config\_json` aborts the upgrade ? unrecoverable startup brick (LIVE) ? `db.py:694` (+ routing subquery `db.py:718`)

Migration 004 runs `json\_extract(config\_json, '\$.delivery\_target')`. If any legacy `filter\_config` row holds non-JSON, `json\_extract` raises `OperationalError: malformed JSON`, the migration ladder rolls back, and **\*\*every subsequent startup re-hits it\*\*** ? the app can't start.

Reproduced: seeding `config\_json='not-json-at-all'` ? `init\_db()` raises; retry still raises. (`{}` and `{"delivery\_target":"me"}` migrate cleanly.) Migration 003 only copied `config\_json` verbatim, so 004 is the first consumer to parse it. Reachability is **\*\*low\*\*** ? the live save path always writes `cfg.model\_dump\_json()` (valid JSON) ? hence medium, not blocker.

```
**Fix:** guard both parse sites so a corrupt row degrades to the `me` default:
```sql
CASE WHEN json_valid(config_json)
      THEN json_extract(config_json, '$.delivery_target') END
...

```

### 2. BOTH-mode delivery masking (LATENT until Track D) ? `delivery.py:214-220` + `record\_delivery` (`db.py:~1373`)

In BOTH mode, `forward\_to\_target` then `send\_link\_message` are called with the **\*\*same\*\*** `destination\_id`, so both `record\_delivery` calls collide on the single `UNIQUE(content\_item\_id, destination\_id)` row. Reproduced both directions:

- forward **\*\*ok\*\*** + link **\*\*fail\*\*** ? row ends `status=failed` with `delivered\_at` still set (the failed link masks the successful forward);
- forward **\*\*fail\*\*** + link **\*\*ok\*\*** ? the forward failure is erased;
- `attempt\_count` also over-counts (2 per single logical BOTH delivery).

A future Track-D retry/idempotency consumer reading `status` would re-deliver an already-forwarded video or misreport. Dormant today (all live callers pass `destination\_id=None`), but it ships in this diff.

**\*\*Fix (before Track D wires routing):\*\*** record **\*\*one combined outcome per destination\*\*** ? `delivered` only when the mode's required sends all succeeded, and never leave `delivered\_at` set on a `failed` row ? or give the forward and link legs distinct rows / an op discriminator. Add the BOTH-mode test (below) now to pin the intended semantics.

---

## ? Low / hygiene

- **\*\*Untested new paths\*\*** ? the `destination\_id` **\*\*failure\*\***-recording branch and the FORWARD/BOTH success paths are uncovered (`delivery.py:144,145,152,181,224`); coverage dipped

96.5 ? 95.75%. A `deliver(...BOTH..., destination\_id=...)` test with  
`FakeClient(side\_effects=[None, RuntimeError])` (forward ok, link fail) covers most of it and  
locks finding #2.

- **Dead code** ? `upsert\_delivery\_attempt` is exported in `\_\_all\_\_` but has zero callers and  
zero tests (`db.py:1320`). Test it or drop it until Track D needs it.
- **Migration edges (latent/cosmetic)**:\*\* an account with legacy sources but no `filter\_config`  
row gets no destination/route/history (bounded ? `bootstrap.py:75` always persists one); numeric  
chat ids backfill as `kind='channel'` where the ADR distinguishes `chat`; the output  
destination is backfilled with `is\_output=0`.
- **Overclaiming comment** ? `models.py:216` says Track C "enforces the source/destination  
invariant (a channel is either source or destination, never both)" ? nothing in this PR enforces  
it. Reword to intent/future-work, or actually enforce it.
- **Nit** ? typo "destinatins" (`models.py:214`); the same comment says  
`filter\_config.delivery\_target` but the migration reads `filter\_config\_deprecated.config\_json`.

---

## Refuted during review (no action)

- Idempotency-gate crash-safety: `\_apply\_migrations` issues no per-migration commit, so a crash  
rolls back atomically ? the `COUNT(destinations)>0` gate is safe.
- An overstated "silent history loss" framing of the no-`filter\_config` edge ? the properly-  
scoped low version is listed above.

## ? Verified good

- Migration 004 correct for the normal paths ('me' default and '@channel' each produce a  
destination + routing rule + backfilled `delivered` attempt); idempotent.
- `record\_delivery` increment / `delivered\_at` logic is correct for the single-send (FORWARD /  
LINK) case.
- `delivery.py` is genuinely backwards-compatible ? `destination\_id=None` preserves the existing  
`mark\_forwarded` no-op for all live callers.
- Models are frozen and conform to the ADR 0011 DDL; `DeliveryStatus` enum matches.

---

## Suggested checklist

- [ ] #1 `json\_valid()` guard at `db.py:694` and `db.py:718`
- [ ] #2 combined-outcome delivery recording for BOTH mode (+ don't leave `delivered\_at` on a  
failed row)
- [ ] Test the BOTH-mode + `destination\_id` path (forward ok / link fail) and the failure-  
recording branch
- [ ] Test or remove `upsert\_delivery\_attempt`
- [ ] Fix the `models.py` invariant comment + "destinatins" typo
- [ ] Rebase on current `main`

<sub>> Reviewed with [Claude Code](https://claude.com/claude-code). Findings reproduced locally  
against `feat/track-c-delivery-routing` @ `100abb2`; full suite green at 95.75%. Multi-agent  
adversarial pass: 17 raw findings ? 15 kept, 2 refuted.</sub>

## 177. user (2026-07-06T08:11:37.972Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-  
badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr37\_review.md (file  
state is current in your context ? no need to Read it back)

## 178. user (2026-07-06T08:11:46.936Z)

---posted review URL---  
avidullu [COMMENTED] 2026-07-06T08:11:45Z  
<https://github.com/avidullu/telegram-video-filter/pull/37#pullrequestreview-4633993197>

## 179. assistant (2026-07-06T08:11:59.305Z)

? **Posted to PR #37** ? [pullrequestreview-4633993197](https://github.com/avidullu/telegram-

video-filter/pull/37#pullrequestreview-4633993197)

The full review is up as a comment for the author to work through, ending with an actionable checklist. Summary of what I left them:

**\*\*Verdict:** ? no blockers, mergeable after the fixes. **\*\* Gate verified locally:** `ruff check` clean, **\*\*451 passed @ 95.75%\*\***, migration 004 idempotent.

**\*\*? Medium (fix before merge):\*\***

1. **\*\*Malformed `config\_json` ? unrecoverable startup brick\*\*** (LIVE) ? migration 004's `json\_extract` aborts the whole upgrade on a non-JSON row and keeps failing on retry. One-line `json\_valid()` guard at `db.py:694` and `:718`.
2. **\*\*BOTH-mode delivery masking\*\*** (latent until Track D) ? forward and link share one `delivery\_attempt` row, so a failed link masks a successful forward (and vice-versa); `attempt\_count` over-counts. Fix to a combined per-destination outcome.

**\*\*? Low/hygiene:\*\*** untested failure/FORWARD paths (coverage 96.5?95.75%), dead `upsert\_delivery\_attempt`, migration edges (`kind`/`is\_output`/no-`filter\_config`), the overclaiming invariant comment, the "destinatinons" typo, and a rebase on current `main`.

I also noted the 2 findings the adversarial pass correctly **\*\*refuted\*\***, and the verified-good list, so the author knows what not to chase.

Per your instruction, I left all fixes to the author ? I didn't touch the branch. If they push fixes and you want me to re-verify (like the #35 round), just say so.

#### **180. user (2026-07-06T08:17:53.621Z)**

pls take another pass.

#### **181. assistant (2026-07-06T08:17:58.985Z)**

Let me sync and see what changed on PR #37 since my review.

#### **182. user (2026-07-06T08:18:06.809Z)**

---current checkout---

feat/track-c-delivery-routing

**feat/track-c-delivery-routing...origin/feat/track-c-delivery-routing**

?? \_review\_c.txt

=== PR #37 branch commits vs main ===

3e59ac0 fix(delivery): address PR #37 review ? json\_valid guard, BOTH-mode combined outcome  
a3e8acf feat(delivery): destinations, routing\_rules, delivery\_attempts (ADR 0011, Track C)

=== new commits since my review (after 100abb2) ===

3e59ac0 fix(delivery): address PR #37 review ? json\_valid guard, BOTH-mode combined outcome  
a3e8acf feat(delivery): destinations, routing\_rules, delivery\_attempts (ADR 0011, Track C)  
d806b46 Merge pull request #36 from avidullu/chore/format-commands  
d2e56e4 style(commands): apply ruff format (wrap long \_reply call)  
(empty = no new commits on branch)

#### **183. assistant (2026-07-06T08:18:18.881Z)**

The author **\*\*rebased on main\*\*** (Track C commit is now `a3e8acf`) and added fix commit **\*\*`3e59ac0`\*\*** ? "json\_valid guard, BOTH-mode combined outcome." There's a stray `\_review\_c.txt` again (sibling-session scratch; I'll leave it). Let me confirm HEAD, read the fix diff, and check for a response comment.

#### **184. user (2026-07-06T08:18:22.378Z)**

=== local HEAD ===

3e59ac0 fix(delivery): address PR #37 review ? json\_valid guard, BOTH-mode combined outcome

=== response comment on PR #37? ===

? avidullu 2026-07-06T08:16:57Z

## Review feedback addressed ? all findings fixed in `3e59ac0`

```
| # | Finding | Fix |
|---|---|---|
| ? 1 | Malformed JSON aborts startup | `json_valid()` guard on both `json_extract()` sites in
migration 004 ? corrupt rows degrade to `me`. Test: `cfg='not-json-at-all'` ? migrates
cleanly. |
| ? 2 | BOTH-mode delivery masking | `write_delivery_attempt` kwarg (default True). `deliver()`
sets it to False in BOTH mode, records **one** combined outcome. Status=`delivered` iff at least
one leg ok; errors concatenated; `delivered_at` only set on success. |
| ? 3 | Untested BOTH + destination_id | 2 tests: forward-ok/link-fail ? `delivered` with
combined error; both-fail ? `failed`. |
| ? 4 | Dead `upsert_delivery_attempt` | Removed from `__all__` (zero callers). |
| ? 5 | Models.py comment issues | Fixed "destinatinons" typo + softened invariant claim. |
| ? 6 | Rebase on main | Rebases cleanly on current `main` (`323ee5a` ? includes PRs #35 + #36).
|
```

### Verification (post-fix)

```
```
454 passed @ 96.10% (floor 80%)
ruff check . ? clean
ruff format ? clean
migration 004: confirmed idempotent + malformed-JSON-safe
```
```

### 185. assistant (2026-07-06T08:18:45.059Z)

The author claims all findings fixed in `3e59ac0`, including a new `write\_delivery\_attempt` kwarg for BOTH-mode combined outcomes. One thing to scrutinize: they chose `delivered` iff at least one leg ok ? I'll verify that's coherent, not just that it's "one row." Per the gate, I reproduce rather than trust. Let me read the fix diff.

### 186. user (2026-07-06T08:18:47.361Z)

=== fix commit stat ===

- Rebased on current main.
- 2 new BOTH-mode + destination\_id tests (fwd-ok/link-fail, both-fail).
- 1 malformed-JSON migration test.

Verification: 454 passed @ 96.10%, ruff clean.

```
src/tg_video_filter/db.py      | 17 ++++++--
src/tg_video_filter/delivery.py | 93 ++++++-----
src/tg_video_filter/models.py  | 7 +--
tests/test_db.py               | 42 ++++++
tests/test_delivery.py         | 60 ++++++
5 files changed, 184 insertions(+), 35 deletions(-)
```

=== FULL FIX DIFF (3e59ac0) ===

commit 3e59ac04ba9576057f50d9c1ae9d62f74acd59b8

Author: Avi Dullu <avi.dullu@gmail.com>

Date: Mon Jul 6 13:46:27 2026 +0530

fix(delivery): address PR #37 review ? json\_valid guard, BOTH-mode combined outcome

? Fix #1 ? Malformed JSON crash guard:

- Migration 004 json\_extract() calls now wrapped in json\_valid() ? a corrupt filter\_config row degrades to the 'me' default instead of aborting the migration ladder (unrecoverable startup brick).
- Test: seeds cfg='not-json-at-all' ? migrates cleanly ? 'me' default.

? Fix #2 ? BOTH-mode delivery masking:

- forward\_to\_target / send\_link\_message gain write\_delivery\_attempt kwarg (default True); set to False by deliver() in BOTH mode.
- deliver() records ONE combined DeliveryAttempt per destination after both legs complete ? no more racing two writes onto the same row.
- Combined outcome: delivered iff fwd\_ok OR link\_ok; error concatenates per-leg failures; delivered\_at set only on success.

? Other:

- Dead code: upsert\_delivery\_attempt removed from \_\_all\_\_ (no callers).
- Comment fix: models.py destinatinons typos + inaccurate invariant claim.
- Rebased on current main.
- 2 new BOTH-mode + destination\_id tests (fwd-ok/link-fail, both-fail).
- 1 malformed-JSON migration test.

Verification: 454 passed @ 96.10%, ruff clean.

```
diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
index 9eed6c..ddfc6c5 100644
--- a/src/tg_video_filter/db.py
+++ b/src/tg_video_filter/db.py
@@ -691,9 +691,17 @@ async def _migration_004_destinations_and_routing(conn:
aiosqlite.Connection) ->
    """
    INSERT OR IGNORE INTO destinations (account_id, provider, target, kind, is_output,
created_at)
    SELECT user_id, 'telegram',
-         COALESCE(json_extract(config_json, '$.delivery_target'), 'me'),
+         COALESCE(
+             CASE WHEN json_valid(config_json)
+                 THEN json_extract(config_json, '$.delivery_target') END,
+             'me'
+         ),
    CASE
-         WHEN COALESCE(json_extract(config_json, '$.delivery_target'), 'me') = 'me'
+         WHEN COALESCE(
+             CASE WHEN json_valid(config_json)
+                 THEN json_extract(config_json, '$.delivery_target') END,
+             'me'
+         ) = 'me'
+         THEN 'saved'
        ELSE 'channel'
    END,
@@ -715,7 +723,9 @@ async def _migration_004_destinations_and_routing(conn:
aiosqlite.Connection) ->
    JOIN destinations d ON d.account_id = s.account_id
    WHERE d.target = (
        SELECT COALESCE(
-         json_extract(fcd.config_json, '$.delivery_target'), 'me'
+         CASE WHEN json_valid(fcd.config_json)
+             THEN json_extract(fcd.config_json, '$.delivery_target') END,
+         'me'
        )
        FROM filter_config_deprecated fcd
        WHERE fcd.user_id = s.account_id
@@ -1753,7 +1763,6 @@ @@ __all__ = [
    # Destination / Routing / Delivery repos (ADR 0011, Track C)
    "upsert_destination",
    "get_destination",
-    "upsert_delivery_attempt",
    "record_delivery",
    "get_delivery_attempt",
    # Legacy User shims (backwards-compatible)
diff --git a/src/tg_video_filter/delivery.py b/src/tg_video_filter/delivery.py
index d99c22c..77f7c7f 100644
--- a/src/tg_video_filter/delivery.py
```



```

+++ b/src/tg_video_filter/delivery.py
@@ -129,29 +129,34 @@ async def forward_to_target(
    target: str = "me",
    *,
    destination_id: int | None = None,
+   write_delivery_attempt: bool = True,
) -> bool:
-   """Forward ``message`` to ``target``, recording delivery state.
+   """Forward ``message`` to ``target``, optionally recording delivery state.

    ``target`` is a Telethon entity: ``"me"`` (Saved Messages), ``"@username"`` ,
    or a numeric chat id string. See ADR 0005.

-   If ``destination_id`` is provided, a per-destination ``DeliveryAttempt``
-   is written (ADR 0011 Track C). Otherwise falls back to the legacy no-op
-   ``mark_forwarded``.
+   If ``destination_id`` is provided AND ``write_delivery_attempt`` is True,
+   a per-destination ``DeliveryAttempt`` is written (ADR 0011 Track C).
+   Set ``write_delivery_attempt=False`` to defer the DB write to the caller
+   (used in BOTH mode so the two legs don't race on the same row).
+   Otherwise falls back to the legacy no-op ``mark_forwarded``.
    """
    ok = await _send_with_retry(client, lambda: client.forward_messages(target, message),
video_id)
-   if ok:
-       if destination_id is not None:
-           await db.record_delivery(
-               conn, content_item_id=video_id, destination_id=destination_id, success=True
-           )
-       else:
-           await db.mark_forwarded(conn, video_id)
-           log.info("video_forwarded video=%s target=%s", video_id, target)
-   elif destination_id is not None:
+   if write_delivery_attempt and destination_id is not None:
        await db.record_delivery(
-           conn, content_item_id=video_id, destination_id=destination_id, success=False
+           conn,
+           content_item_id=video_id,
+           destination_id=destination_id,
+           success=ok,
+           error=("forward_to_target failed" if not ok else None),
        )
+   elif not write_delivery_attempt:
+       pass # caller (deliver BOTH mode) writes the combined outcome
+   elif ok:
+       await db.mark_forwarded(conn, video_id)
+   if ok:
+       log.info("video_forwarded video=%s target=%s", video_id, target)
    return ok

@@ -163,24 +168,27 @@ async def send_link_message(
    target: str = "me",
    *,
    destination_id: int | None = None,
+   write_delivery_attempt: bool = True,
) -> bool:
-   """Send a link/summary text message to ``target``, recording delivery state.
+   """Send a link/summary text message to ``target``, optionally recording delivery state.

    ``target`` is a Telethon entity; see :func:`forward_to_target`.
    """
    ok = await _send_with_retry(client, lambda: client.send_message(target, text), video_id)
-   if ok:
-       if destination_id is not None:

```

```

-         await db.record_delivery(
-             conn, content_item_id=video_id, destination_id=destination_id, success=True
-         )
-     else:
-         await db.mark_forwarded(conn, video_id)
-         log.info("link_sent video=%s target=%s", video_id, target)
-     elif destination_id is not None:
+ if write_delivery_attempt and destination_id is not None:
-         await db.record_delivery(
-             conn, content_item_id=video_id, destination_id=destination_id, success=False
+             conn,
+             content_item_id=video_id,
+             destination_id=destination_id,
+             success=ok,
+             error=("send_link_message failed" if not ok else None),
-         )
+ elif not write_delivery_attempt:
+     pass
+ elif ok:
+     await db.mark_forwarded(conn, video_id)
+ if ok:
+     log.info("link_sent video=%s target=%s", video_id, target)
+ return ok

```

```

@@ -212,17 +220,46 @@ async def deliver(
    client, link_text, conn, video_id, target, destination_id=destination_id
)
if mode == DeliveryMode.BOTH and link_text is not None:
+ # Both legs write to the same (content_item_id, destination_id) row.
+ # Defer DB writes so we record ONE combined outcome per destination
+ # (review finding #2: BOTH-mode delivery masking).
    fwd_ok = await forward_to_target(
-        client, message, conn, video_id, target, destination_id=destination_id
+        client,
+        message,
+        conn,
+        video_id,
+        target,
+        destination_id=destination_id,
+        write_delivery_attempt=False,
    )
    link_ok = await send_link_message(
-        client, link_text, conn, video_id, target, destination_id=destination_id
+        client,
+        link_text,
+        conn,
+        video_id,
+        target,
+        destination_id=destination_id,
+        write_delivery_attempt=False,
    )
    combined_ok = fwd_ok or link_ok
    if destination_id is not None:
        error = None
        if not fwd_ok:
            error = "forward failed"
        if not link_ok:
            error = f"{error}; link failed" if error else "link failed"
        await db.record_delivery(
            conn,
            content_item_id=video_id,
            destination_id=destination_id,
            success=combined_ok,
            error=error,

```

```

+         )
+         if not link_ok:
+             log.warning("link_delivery_failed_in_both_mode video=%s", video_id)
+         if not fwd_ok:
+             log.warning("forward_delivery_failed_in_both_mode video=%s", video_id)
-         return fwd_ok or link_ok
+         return combined_ok
+         # FORWARD, or fallback when link_text is missing.
+         if mode != DeliveryMode.FORWARD:
+             log.warning(
diff --git a/src/tg_video_filter/models.py b/src/tg_video_filter/models.py
index de94a75..4988efe 100644
--- a/src/tg_video_filter/models.py
+++ b/src/tg_video_filter/models.py
@@ -211,10 +211,11 @@ class Policy(BaseModel):
# --- Track C: destinations, routing, delivery state (ADR 0011) -----
#
# These tables were created empty by Track B (_SCHEMA); Track C populates
-# destinatinons (backfilled from filter_config.delivery_target), writes
+# destinations (backfilled from filter_config_deprecated.config_json), writes
# delivery_attempts (replacing the legacy videos.forwarded boolean), and
-# enforces the source/destination invariant (a channel is either source or
-# destination, never both).
+# defines the models for per-destination delivery state. The source/destination
+# invariant (a channel is either source or destination, never both) is enforced
+# at the app level by future selective-scanning logic (Track E, #30).

class Destination(BaseModel):
diff --git a/tests/test_db.py b/tests/test_db.py
index 897861d..c479c34 100644
--- a/tests/test_db.py
+++ b/tests/test_db.py
@@ -1653,3 +1653,45 @@ async def test_migration_004_is_idempotent(tmp_path: object) -> None:
    assert await db._table_exists(conn, "destinations")
    assert await db._table_exists(conn, "routing_rules")
    assert await db._table_exists(conn, "delivery_attempts")
+
+
+async def test_migration_004_handles_malformed_filter_config_json(
+    tmp_path: object,
+) -> None:
+    """Migration 004 must not crash on malformed filter_config JSON (fix #1).
+
+    A legacy filter_config row with non-JSON content must degrade to the
+    'me' default destination, not abort the migration ladder.
+
+    """
+    db_path = str(tmp_path) + "/bad_json.db" # type: ignore[operator]
+    legacy_schema = """
+    CREATE TABLE users (id INTEGER PRIMARY KEY, phone TEXT, enabled INTEGER NOT NULL DEFAULT 1,
+created_at TEXT NOT NULL);
+    CREATE TABLE channels (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL
+REFERENCES users(id), channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
+UNIQUE (user_id, channel_id));
+    CREATE TABLE videos (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL
+REFERENCES users(id), channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL, duration_s
+INTEGER, width INTEGER, height INTEGER, size_bytes INTEGER, mime_type TEXT, matched INTEGER NOT
+NULL DEFAULT 0, forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL, UNIQUE (user_id,
+channel_id, message_id));
+    CREATE TABLE filter_config (user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json
+TEXT NOT NULL);
+    """
+    async with aiosqlite.connect(db_path) as conn:
+        await conn.executescript(legacy_schema)
+        await conn.execute("INSERT INTO users VALUES (111, '+1', 1, '2024-01-01T00:00:00Z')")

```

```

+         await conn.execute(
+             "INSERT INTO channels (user_id, channel_id, title, seen_first_at) VALUES (111,
-1001, 'Ch', '2024-01-01T00:00:00Z')"
+         )
+         await conn.execute(
+             "INSERT INTO videos (user_id, channel_id, message_id, duration_s, width, height,
size_bytes, mime_type, matched, forwarded, seen_at) "
+             "VALUES (111, -1001, 1, 60, 1920, 1080, 1000, 'video/mp4', 1, 0,
'2024-01-01T00:00:00Z')"
+         )
+         # Malformed JSON ? migration 004 must not crash.
+         await conn.execute("INSERT INTO filter_config VALUES (111, 'not-json-at-all')")
+         await conn.commit()
+
+ # Must not raise ? the json_valid() guard degrades to 'me'.
+ await db.init_db(db_path)
+
+ async with aiosqlite.connect(db_path) as conn:
+     async with conn.execute(
+         "SELECT target, kind FROM destinations WHERE account_id=111"
+     ) as cur:
+         d = await cur.fetchone()
+         assert d == ("me", "saved"), f"expected ('me','saved') fallback, got {d}"
+         async with conn.execute("PRAGMA user_version") as cur:
+             assert (await cur.fetchone())[0] == len(db._MIGRATIONS)
diff --git a/tests/test_delivery.py b/tests/test_delivery.py
index 9e98191..afac256 100644
--- a/tests/test_delivery.py
+++ b/tests/test_delivery.py
@@ -276,6 +276,66 @@ async def test_deliver_with_destination_id_records_delivery_attempt(
     assert attempt.attempt_count == 1

+async def test_deliver_both_mode_with_destination_id_records_combined_outcome(
+    db_with_video: tuple[aiosqlite.Connection, int],
+) -> None:
+    """BOTH mode + destination_id: forward ok, link fail ? combined success
+    with status=delivered (at least one leg succeeded). The DB write is
+    deferred to deliver() so the two legs don't race on the same row
+    (review finding #2)."""
+    c, vid = db_with_video
+    dest = await db.upsert_destination(
+        c, Destination(account_id=USER_ID, provider="telegram", target="me")
+    )
+    assert dest.id is not None
+    # Forward succeeds, link fails (RuntimeError on send_message).
+    client = FakeClient(side_effects=[None, RuntimeError("boom")])
+    ok = await deliver(
+        client,
+        _SENTINEL_MSG,
+        c,
+        vid,
+        DeliveryMode.BOTH,
+        link_text="test",
+        target="me",
+        destination_id=dest.id, # type: ignore[arg-type]
+    )
+    assert ok is True # forward_ok OR link_ok
+    attempt = await db.get_delivery_attempt(c, vid, dest.id)
+    assert attempt is not None
+    assert attempt.status.value == "delivered"
+    assert attempt.attempt_count == 1
+    assert "link failed" in (attempt.last_error or "")
+

```

```

+async def test_deliver_both_mode_both_legs_fail_records_failed(
+    db_with_video: tuple[aiosqlite.Connection, int],
+) -> None:
+    """BOTH mode + destination_id: both legs fail ? status=failed with combined error."""
+    c, vid = db_with_video
+    dest = await db.upsert_destination(
+        c, Destination(account_id=USER_ID, provider="telegram", target="me")
+    )
+    assert dest.id is not None
+    client = FakeClient(side_effects=[RuntimeError("fwd"), RuntimeError("link")])
+    ok = await deliver(
+        client,
+        _SENTINEL_MSG,
+        c,
+        vid,
+        DeliveryMode.BOTH,
+        link_text="test",
+        target="me",
+        destination_id=dest.id, # type: ignore[arg-type]
+    )
+    assert ok is False
+    attempt = await db.get_delivery_attempt(c, vid, dest.id)
+    assert attempt is not None
+    assert attempt.status.value == "failed"
+    assert attempt.attempt_count == 1
+    assert "forward failed" in (attempt.last_error or "")
+
+    async def test_both_mode_logs_warning_on_link_failure(
+        db_with_video: tuple[aiosqlite.Connection, int],
+        caplog: pytest.LogCaptureFixture,

```

#### 187. assistant (2026-07-06T08:19:41.423Z)

Let me analyze the fix, then verify independently. The fix looks sound:

- **#1**: ``json_valid()`` wraps both ``json_extract`` sites ? corrupt rows degrade to ``'me'`` ?
- **#2**: BOTH mode now defers both legs (``write_delivery_attempt=False``) and writes **one** combined ``record_delivery`` ? ``delivered`` iff `fwd_ok OR link_ok`, errors concatenated, ``delivered_at`` only on success. The single-mode path collapsed to ``success=ok`` (also fixing the untested failure branch).
- Comment fixed, ``__all__`` entry removed, rebased.

One thing to scrutinize: ``delivered`` iff `?1 leg`` means a **forward-fail + link-ok** still records ``delivered`` ? the video never got forwarded but status says delivered. It's a deliberate, documented choice consistent with the original ``return fwd_ok or link_ok``, but worth flagging for Track D. Let me verify everything empirically.

#### 188. user (2026-07-06T08:19:54.965Z)

=== is `upsert_delivery_attempt` still DEFINED (just unexported) or removed? ===  
1330:async def upsert\_delivery\_attempt(

=== independent malformed-JSON repro (should migrate to 'me') ===

|                                               |                                                        |
|-----------------------------------------------|--------------------------------------------------------|
| <code>cfg='not-json-at-all'</code>            | <code>-&gt; OK, destination=('me', 'saved')</code>     |
| <code>cfg={'delivery_target': '@feed'}</code> | <code>-&gt; OK, destination=('feed', 'channel')</code> |
| <code>cfg='{'}</code>                         | <code>-&gt; OK, destination=('me', 'saved')</code>     |

#### 189. assistant (2026-07-06T08:20:11.096Z)

**Fix #1 verified** ? all variants migrate cleanly (malformed ? ``('me', 'saved')``, ``@feed`` ? ``('feed', 'channel')``). Note: ``upsert_delivery_attempt`` is **still defined** at ``db.py:1330`` ? only unexported, so it remains uncovered dead code (residual low). Let me run the full gate and confirm the new BOTH-mode tests.

#### 190. user (2026-07-06T08:20:26.597Z)

```
=== ruff check ===
All checks passed!
=== ruff format --check (should be clean post-rebase) ===
34 files already formatted
=== new BOTH-mode + malformed-JSON tests ===
src\tg_video_filter\models.py          162    10     8     1    90%   363-367, 377,
416, 424-426
src\tg_video_filter\teletthon_client.py  62     62    26     0     0%   16-358
-----
TOTAL                                1112   674   274    23   34%
FAIL Required test coverage of 80.0% not reached. Total coverage: 34.27%
3 passed, 108 deselected in 1.59s
=== full suite + coverage ===
-----
TOTAL                                1112    37   274    17   96%
Required test coverage of 80.0% reached. Total coverage: 96.10%
454 passed in 6.95s
```

#### 191. assistant (2026-07-06T08:20:46.854Z)

Gate green on the fix: **\*\*ruff check + format clean\*\*** (format now clean post-rebase), **\*\*454 passed @ 96.10%\*\*** (up from 95.75%), and the 3 new tests pass. Let me independently drive the BOTH-mode fix through ``deliver()`` ? including the one case the tests *\*don't\** cover (forward-fail + link-ok ? the debatable ``delivered``).

#### 192. user (2026-07-06T08:20:49.168Z)

(Bash completed with no output)

#### 193. user (2026-07-06T08:20:53.059Z)

```
116:class FakeClient:
117:    """Tracks forward_messages and send_message calls and their target args."""
119:    def __init__(self, side_effects: list[BaseException | None] | None = None) -> None:
120:        self._side_effects = list(side_effects or [])
127:    async def forward_messages(self, *a, **kw):
131:        if self._side_effects:
132:            effect = self._side_effects.pop(0)
136:    async def send_message(self, *a, **kw):
140:        if self._side_effects:
141:            effect = self._side_effects.pop(0)
159:_SENTINEL_MSG = object()
224:    client = FakeClient(side_effects=[FloodWaitError(request=None, capture=2), None])
291:    # Forward succeeds, link fails (RuntimeError on send_message).
292:    client = FakeClient(side_effects=[None, RuntimeError("boom")])
320:    client = FakeClient(side_effects=[RuntimeError("fwd"), RuntimeError("link")])
345:    # First call (forward) succeeds; second (send_message) raises.
346:    client = FakeClient(side_effects=[None, RuntimeError("link fail")])
448:        side_effects=[
464:        side_effects=[FloodWaitError(request=None, capture=1), RuntimeError("boom")]
```

#### 194. user (2026-07-06T08:21:14.835Z)

```
delivery_failed video=1
Traceback (most recent call last):
  File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\delivery.py", line 108, in
_send_with_retry
    await send_callable()
  File "<string>", line 13, in send_message
    if x: raise x
    ^^^^^^^
RuntimeError: x
```

```

link_delivery_failed_in_both_mode video=1
delivery_failed video=1
Traceback (most recent call last):
  File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\delivery.py", line 108, in
_send_with_retry
    await send_callable()
  File "<string>", line 10, in forward_messages
    if x: raise x
    ^^^^^^^
RuntimeError: x
forward_delivery_failed_in_both_mode video=1
delivery_failed video=1
Traceback (most recent call last):
  File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\delivery.py", line 108, in
_send_with_retry
    await send_callable()
  File "<string>", line 10, in forward_messages
    if x: raise x
    ^^^^^^^
RuntimeError: x
delivery_failed video=1
Traceback (most recent call last):
  File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\delivery.py", line 108, in
_send_with_retry
    await send_callable()
  File "<string>", line 13, in send_message
    if x: raise x
    ^^^^^^^
RuntimeError: y
link_delivery_failed_in_both_mode video=1
forward_delivery_failed_in_both_mode video=1
fwd OK + link FAIL: deliver()->True | rows=1 status=delivered count=1 delivered_at=set
err='link failed'
fwd FAIL+ link OK : deliver()->True | rows=1 status=delivered count=1 delivered_at=set
err='forward failed'
fwd FAIL+ link FAIL: deliver()->False | rows=1 status=failed count=1 delivered_at=None
err='forward failed; link failed'
fwd OK + link OK : deliver()->True | rows=1 status=delivered count=1 delivered_at=set err=None

```

### 195. assistant (2026-07-06T08:22:17.450Z)

Verified end-to-end. The BOTH-mode fix works exactly as designed ? \*\*always one row, no masking\*\*:

| BOTH-mode case              | `deliver()` | rows  | status    | attempt_count | delivered_at | error                         |
|-----------------------------|-------------|-------|-----------|---------------|--------------|-------------------------------|
| --- --- --- --- --- --- --- |             |       |           |               |              |                               |
| fwd OK + link FAIL          | True        | **1** | delivered | 1             | set          | `link failed`                 |
| fwd FAIL + link OK          | True        | 1     | delivered | 1             | set          | `forward failed`              |
| fwd FAIL + link FAIL        | False       | 1     | failed    | 1             | **None**     | `forward failed; link failed` |
| fwd OK + link OK            | True        | 1     | delivered | 1             | set          | None                          |

The self-contradictory row (status=failed + delivered\_at set) and the last-writer-wins masking are \*\*gone\*\* ; `delivered\_at` is set only on success ; `attempt\_count` no longer double-counts.

### Re-review: fix commit `3e59ac0`

The author rebased on `main` and pushed the fix. Every claim reproduced independently:

| #               | Finding                                                                                                                                              | Status |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| --- ----- ----- |                                                                                                                                                      |        |
| ? 1             | Malformed JSON aborts upgrade   ? **Fixed** ? `json_valid()` guards both `json_extract` sites; repro: `not-json-at-all` ? `('me','saved')`, no crash |        |
| ? 2             | BOTH-mode masking   ? **Fixed** ? one combined `record_delivery`; verified all 4 leg-outcomes (table above)                                          |        |

```
| ? 3 | Untested BOTH + `destination_id` | ? **Added** ? fwd-ok/link-fail + both-fail tests
pass; single-mode failure path now `success=ok` |
| ? 4 | Dead `upsert_delivery_attempt` | ?? **Partial** ? removed from `__all__`, but the
function body still lives at `db.py:1330` (unexported, uncovered) |
| ? 5 | Comment typo + invariant overclaim | ? **Fixed** ? "destinations", and invariant
reworded to "enforced? by future selective-scanning logic (Track E)" |
| ? 6 | Rebase on main | ? **Done** ? `ruff format --check .` now clean (34 files) |
```

**\*\*Gate on `3e59ac0`:\*\*** ? `ruff check` clean . ? `ruff format` clean . **\*\*454 passed @ 96.10%\*\***  
(up from 95.75%, floor 80%).

### Two small notes (neither blocks)

- **\*\*Residual dead code:\*\*** `upsert\_delivery\_attempt` is now unexported but still defined and untested. Either delete it or add a round-trip test ? otherwise it just drags coverage until Track D.
- **\*\*Design note for Track D (not a bug):\*\*** the chosen "delivered iff ?1 leg" means a **\*\*forward-fail + link-ok\*\*** BOTH delivery is recorded `delivered` (the video was never forwarded, only the link sent). It's deliberate and consistent with the original `return fwd\_ok or link\_ok`, and the error field records "forward failed" ? but a Track-D retry consumer reading `status=delivered` won't re-forward it. Worth a comment there.
- The low/latent migration cosmetics from my first pass that weren't on the checklist (`kind='channel'` for numeric chat ids, `is\_output=0`, the no-`filter\_config` edge) remain open ? all fine to defer to Track D/E.

**\*\*Verdict:** ? the fixes are correct and complete ? this is mergeable.\*\* By the pre-LGTM gate, verified locally: full suite green (454), lint+format clean, coverage up (96.10% ? floor), and both mediums resolved and reproduced.

Want me to **\*\*post a verified sign-off / LGTM\*\*** on PR #37 (as a comment, like #35)? I'll hold until you say go. (Also: there's a stray `\_review\_c.txt` in the checkout again from the fix session ? I can clean it up if you'd like.)

### 196. user (2026-07-06T08:23:00.034Z)

please post the comment and cleanup the stray file

### 197. assistant (2026-07-06T08:23:07.885Z)

Posting the verified LGTM and cleaning up the stray file. Let me write the sign-off comment.

### 198. assistant (2026-07-06T08:23:24.068Z)

## ? LGTM ? review findings verified fixed in `3e59ac0`

Re-reviewed the fix commit and **\*\*reproduced every claim independently\*\*** (didn't trust the writeup). Both mediums resolved; the rest addressed or reasonably deferred.

```
| # | Finding | Status |
|---|-----|-----|
| ? 1 | Malformed JSON aborts upgrade (startup brick) | ? **Fixed** ? `json_valid()` guards both
`json_extract` sites; repro: `config_json='not-json-at-all'` ? `('me','saved')`, migrates
cleanly, no crash |
| ? 2 | BOTH-mode delivery masking | ? **Fixed** ? `deliver()` defers both legs and writes
**one** combined `record_delivery`; verified all four leg-outcomes (table below) |
| ? 3 | Untested BOTH + `destination_id` | ? **Added** ? fwd-ok/link-fail + both-fail tests;
single-mode failure path now `success=ok` |
| ? 4 | Dead `upsert_delivery_attempt` | ?? **Partial** ? removed from `__all__`, but the
function body still lives at `db.py:1330` (unexported, uncovered) |
| ? 5 | Comment typo + invariant overclaim | ? **Fixed** |
| ? 6 | Rebase on main | ? **Done** ? `ruff format --check .` now clean |
```

### BOTH-mode combined outcome ? verified via `deliver()`

```
| case | `deliver()` | rows | status | attempt_count | delivered_at | error |
|---|---|---|---|---|---|---|
```



```
| fwd OK + link FAIL | True | **1** | delivered | 1 | set | `link failed` |
| fwd FAIL + link OK | True | 1 | delivered | 1 | set | `forward failed` |
| fwd FAIL + link FAIL | False | 1 | failed | 1 | **None** | `forward failed; link failed` |
| fwd OK + link OK | True | 1 | delivered | 1 | set | ? |
```

No more self-contradictory row (status=failed + delivered\_at set), no last-writer-wins masking, `delivered\_at` only on success, `attempt\_count` no longer double-counts.

#### Verification gate (local, `3e59ac0`)

```
| Check | Result |
|---|---|
| `pytest` (full suite) | ? **454 passed**, coverage **96.10%** (up from 95.75%, floor 80%) |
| `ruff check .` | ? clean |
| `ruff format --check .` | ? clean (34 files) |
| migration 004 | ? idempotent + malformed-JSON-safe |
```

#### Two small notes (neither blocks merge)

- **Residual dead code:** `upsert\_delivery\_attempt` is now unexported but still defined and untested (`db.py:1330`) ? delete it or add a round-trip test so it doesn't drag coverage until Track D needs it.
- **Design note for Track D (not a bug):** `delivered iff ?1 leg` means a **forward-fail + link-ok** BOTH delivery records `delivered` (the video wasn't forwarded, only the link sent). Deliberate and consistent with the original `return fwd\_ok or link\_ok`, and the error is recorded ? but a Track-D retry consumer reading `status=delivered` won't re-forward it. Worth a comment there.
- The low/latent migration cosmetics not on the checklist (`kind='channel'` for numeric chat ids, `is\_output=0`, no-`filter\_config` edge) remain open ? fine to defer to Track D/E.

**LGTM ?** ? mergeable and well-tested from my side. (Posting as a comment: GitHub doesn't allow the PR author to formally Approve their own PR.)

<sub>? Verified with [Claude Code](https://claude.com/claude-code) ? findings reproduced locally against `feat/track-c-delivery-routing` @ `3e59ac0`.</sub>

#### 199. user (2026-07-06T08:23:24.110Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr37\_lgtm.md (file state is current in your context ? no need to Read it back)

#### 200. user (2026-07-06T08:23:35.912Z)

=== confirm stray file is scratch (head) ===

#### Code review ? destinations, routing\_rules, delivery\_attempts (ADR 0011, Track C)

**Verdict:** ? Solid, no blockers ? please address the items below before merge. **Clean models**, correct core migration, backwards-compatible delivery. Two **medium** items are worth fixing before merge (one is a cheap startup-safety guard; the other is a real logic bug Track D will step on); the rest are low/hygiene. Every finding was reproduced locally against `100abb2` (posting as a comment ? GitHub blocks the author from a formal request-changes). (above blank = untracked)

=== post LGTM comment ===

avidullu [COMMENTED] 2026-07-06T08:23:34Z

<https://github.com/avidullu/telegram-video-filter/pull/37#pullrequestreview-4634073146>

#### 201. assistant (2026-07-06T08:23:42.981Z)

Confirmed the stray file is a scratch copy of my earlier review comment (untracked). LGTM is posted ? [pullrequestreview-4634073146](https://github.com/avidullu/telegram-video-filter/pull/37#pullrequestreview-4634073146). Now removing the stray file.

#### 202. user (2026-07-06T08:23:46.541Z)

removed '\_review\_c.txt'

--- tree state ---

## feat/track-c-delivery-routing...origin/feat/track-c-delivery-routing

--- PR #37 merge state ---

state=OPEN mergeable=MERGEABLE CLEAN

### 203. assistant (2026-07-06T08:23:56.126Z)

Both done. ?

## Summary

**\*\*Posted the verified LGTM\*\* ?**

[pullrequestreview-4634073146](https://github.com/avidullu/telegram-video-filter/pull/37#pullrequestreview-4634073146)

- Both mediums reproduced-fixed (malformed-JSON guard; BOTH-mode combined outcome verified across all four leg outcomes), gate green (\*\*454 passed @ 96.10%\*\*), ruff clean, migration idempotent). Included two non-blocking notes: the still-defined `upsert\_delivery\_attempt`, and the "delivered iff ?1 leg" semantic as a Track-D heads-up.

**\*\*Cleaned up the stray file\*\* ?** removed the untracked `\_review\_c.txt` (confirmed it was a scratch copy of my earlier review). Working tree is now clean and synced with origin.

**\*\*PR #37 state:\*\*** `OPEN · MERGEABLE · CLEAN` ? ready to merge whenever you want it.

That closes the Track C loop. If the author addresses the residual `upsert\_delivery\_attempt` dead code or you kick off Track D, just ping me for another pass.

### 204. user (2026-07-06T08:35:47.178Z)

can you review #38?

### 205. assistant (2026-07-06T08:35:49.299Z)

I'll sync and pull up PR #38.

### 206. user (2026-07-06T08:35:54.566Z)

---checkout---

feat/track-d-command-service

## feat/track-d-command-service...origin/feat/track-d-command-service

=== PR #38 META ===

```
{
  "additions": 650,
  "author": {
    "id": "MDQ6VXNlcjE2NzQ4Nzc=",
    "is_bot": false,
    "login": "avidullu",
    "name": "Avi"
  },
  "baseRefName": "main",
  "body": "## What\n\nImplements **Track D (#29)** of the provider-neutral umbrella (#25) ? extracts command parsing and config-mutation logic from the Telethon adapter into a provider-independent `CommandService`, so the same `/set`, `/show` commands work through a future Bot API front-door without duplicating business logic.\n\n## New module:\n`command_service.py`\n\n| Component | Purpose |\n|---|---|\n| `CommandResult` | dataclass |\n| `reply_text` + `config_changed` | flag ? provider-independent effect |\n| `CommandService.handle(user_id, text, conn, current_cfg)` | Parse command string, mutate config, return result |\n| All regex patterns | `/set size`, `/set length`, `/set min length`, `/set min resolution`, `/set exclude`, `/show config`, `/show commands` |\n| Unit parsers | `parse_size_mb()`, `parse_duration_s()` |\n| Formatters | `format_config()`, `format_commands()`, `_format_duration()`, `_format_size()` |\n\n**No Telethon imports** ? pure strings in, `CommandResult` out. Ready for a Bot API adapter wrapper.\n\n## Refactored:\n`commands.py` ? thin Telethon adapter\n\nThe old 407-line file is now a ~120-line adapter:\n- `make_config_command_handler()` delegates to `CommandService.handle()`\n- Telethon-specific transport stays: `message.out`, `peer_id`, `client.send_message("me", ...)`\n- **Full backwards-compat:** re-exports all regex patterns, unit parsers, formatting helpers, and `_update_config` wrapper for existing tests (`test_commands.py` unchanged, all pass)\n\n## Tests: `test_command_service.py` (20 tests, zero Telethon)\n\n| Category | Tests |\n|---|---|\n| Unit parsers | `parse_size_mb` (6 param), `parse_duration_s` (5 param) |\n| `/set size` |
```

```

success, below-minimum, invalid-format-silent |\n| `/set length` | success, below-min-duration
|\n| `/set min length` | success, above-max-duration |\n| `/set min resolution` | success,
below-min, help-on-malformed |\n| `/set exclude` | gif-on, round-off, help-on-malformed |\n|
`/show` | config, commands |\n| Noise | unrecognized returns empty |\n\n##
Verification\n\n```\n482 passed @ 97.05% (floor 80%)\nruff check . ? clean\nruff format ?
clean\n```\n\nCoverage gain: 96.52% ? 97.05% (+0.53%). `commands.py` coverage went from 96%
(inline regex paths) to 100% mapped through the service.\n\n## Scope discipline\n\n ?
**Extracted:** command parsing + config mutation ? `CommandService` (provider-independent)\n- ?
**Refactored:** `commands.py` ? thin adapter (~300 lines removed)\n- ? **Tested:** 20 new tests,
no Telethon required\n- ? **Not in this track:** Bot API adapter implementation (the
`CommandService` is ready to be wrapped ? Track D acceptance criteria satisfied)\n\n##
Closes\n\nCloses #29 (Track D).\n\n## Related\n\n- #25 ? umbrella issue\n- #35 ? Track B (domain
schema, provides the `FilterConfig` model)\n- #37 ? Track C (destinations/delivery state,
provides `record_delivery`)\n- ADR 0007 ? inline config commands\n- ADR 0008 ? Bot API
onboarding front-door", "changedFiles":3, "deletions":376, "headRefName":"feat/track-d-command-serv
ice", "isDraft":false, "mergeStateStatus":"CLEAN", "mergeable":"MERGEABLE", "number":38, "state":"OPE
N", "statusCheckRollup":[], "title":"feat(commands): extract CommandService ? provider-independent
command execution (ADR 0011, Track D)"}

```

## 207. assistant (2026-07-06T08:36:01.742Z)

PR #38 is **Track D** ? extract a provider-independent `CommandService` (+650/?376, 3 files, a refactor). Checkout is on the branch, synced. Let me check base freshness, get the diff, and kick off the gate.

## 208. user (2026-07-06T08:36:10.079Z)

=== files changed ===

```

src/tg_video_filter/command_service.py
src/tg_video_filter/commands.py
tests/test_command_service.py

```

=== branch commits vs origin/main ===

```

22abe72 feat(commands): extract CommandService ? provider-independent command execution (ADR
0011, Track D)

```

=== main ahead of branch? (missing commits) ===  
(empty = branch is current with main)

=== HEAD + tree ===

```

22abe72 feat(commands): extract CommandService ? provider-independent command execution (ADR
0011, Track D)

```

## feat/track-d-command-service...origin/feat/track-d-command-service

```

1113 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr38.diff

```

## 209. assistant (2026-07-06T08:36:23.973Z)

Branch is current with `main` (one commit `22abe72`). Let me start the gate in the background and read the new `command\_service.py`, the refactored `commands.py`, and the old `commands.py` from main for behavioral-equivalence comparison ? the core risk in a refactor.

## 210. user (2026-07-06T08:36:28.006Z)

Command running in background with ID: be6c3gr91. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\be6c3gr91.output. You will be notified when it completes. To check interim output, use Read on that file path.

## 211. user (2026-07-06T08:36:28.081Z)

```

1      """Provider-independent command parsing and execution (ADR 0011, Track D).
2

```

```

3     Extracts command logic from the Telethon adapter (``commands.py``) so the
4     same ``/set``, ``/show`` commands work identically through a Bot API
5     front-door. The service is pure: strings in, ``CommandResult`` out ? no
6     Telethon objects, no network calls.
7
8     See ``docs/adr/0007-inline-config-commands.md`` for the command model and
9     ``docs/adr/0008-bot-api-onboarding-front-door.md`` for the Bot API motivation.
10    """
11
12    from __future__ import annotations
13
14    import logging
15    import re
16    from dataclasses import dataclass
17
18    import aiosqlite
19
20    from tg_video_filter import db
21    from tg_video_filter.models import FilterConfig
22
23    log = logging.getLogger(__name__)
24
25    # Key into the shared engine_holder dict (imported by commands.py's adapter).
26    ENGINE_KEY = "engine"
27
28    # --- regex patterns -----
29
30    _SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmgT]?b?)$", re.IGNORECASE)
31    _LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
32    _MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
33    _MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$",
re.IGNORECASE)
34    _SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
35    _SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
36    _EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$",
re.IGNORECASE)
37    _EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
38    _MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
39
40
41    # --- unit parsing -----
42
43
44    def parse_size_mb(value_str: str, unit: str) -> int:
45        """Parse a size value + unit string into mebibytes."""
46        value = int(value_str)
47        unit = unit.lower().rstrip("b")
48        multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
49        if unit in multipliers:
50            raw = value * multipliers[unit]
51            if unit == "k":
52                raw = max(1, raw // 1024)
53            return raw
54        raise ValueError(f"Unknown size unit: {unit}")
55
56
57    def parse_duration_s(value_str: str, unit: str) -> int:
58        """Parse a duration value + unit string into seconds."""
59        value = int(value_str)
60        unit = unit.lower()
61        multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
62        if unit in multipliers:
63            return value * multipliers[unit]
64        raise ValueError(f"Unknown time unit: {unit}")
65

```

```

66
67 # --- formatting helpers -----
68
69
70 def _format_duration(seconds: int | None) -> str:
71     if seconds is None:
72         return "unlimited"
73     if seconds == 0:
74         return "0s"
75     if seconds < 60:
76         return f"{seconds}s"
77     if seconds < 3600:
78         m, s = divmod(seconds, 60)
79         return f"{m}m{s}s" if s else f"{m}m"
80     h, rem = divmod(seconds, 3600)
81     m, s = divmod(rem, 60)
82     parts = f"{h}h"
83     if m:
84         parts += f"{m}m"
85     if s:
86         parts += f"{s}s"
87     return parts
88
89
90 def _format_size(mb: int | None) -> str:
91     if mb is None:
92         return "unlimited"
93     if mb == 0:
94         return "0MB"
95     if mb >= 1024:
96         gb = mb / 1024
97         if gb == int(gb):
98             return f"{int(gb)}GB"
99         return f"{gb:.1f}GB"
100     return f"{mb}MB"
101
102
103 def format_config(cfg: FilterConfig) -> str:
104     """Format a ``FilterConfig`` as a readable message (Telegram markdown)."""
105     lines = [
106         "? **Current Filter Configuration**",
107         "",
108         f"? **Duration:** {_format_duration(cfg.min_duration_s)} -",
109         f"{_format_duration(cfg.max_duration_s)}",
110         f"? **Min resolution:** {' ' + str(cfg.target_height) + 'p' if",
111         f"cfg.target_height else 'disabled'}",
112         f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
113         f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
114         f"? **Delivery target:** `{cfg.delivery_target}`",
115         "",
116         "? **Content exclusions:**",
117         f"    GIF/animated: {'? excluded' if cfg.exclude_gif else '? allowed'}",
118         f"    Round videos: {'? excluded' if cfg.exclude_round else '? allowed'}",
119         f"    Silent (no audio): {'? excluded' if cfg.exclude_nosound else '? allowed'}",
120     ]
121     return "\n".join(lines)
122
123
124 def format_commands() -> str:
125     """Return a help message listing all available commands."""
126     return "\n".join(
127         [
128             "? **Available Commands**",
129             "",
130             "? **Filter config**",

```

```

129         "\show config` ? Show current filter settings",
130         "\set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
131         "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing
132         `B`",
133         "\set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
134         "    or `/set max length <N><unit>` ? same as above",
135         "    Units: `s` (default), `m`, `h` ? case-insensitive",
136         "\set min length <N><unit>` ? Set min video duration (e.g. `/set min length
137         5s`)",
138         "    Units: `s` (default), `m`, `h` ? case-insensitive",
139         "\set min resolution <N>p` ? Set min video height (e.g. `/set min
140         resolution 720p`)",
141         "    or `/set min res <N>p` ? same as above",
142         "",
143         "? **Content exclusions** (all off by default)",
144         "\set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
145         "\set exclude round on|off` ? Exclude circular video messages",
146         "\set exclude nosound on|off` ? Exclude videos without audio\n"
147         "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs
148         too.",
149         "",
150         "? **Backfill**",
151         "\backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history
152         (default 90d)",
153         "\stop` ? Cancel running backfill",
154         "",
155         "\u2139\u2013 **Help**",
156         "\show commands` ? Show this help message",
157     ]
158 )
159
160 # --- command result -----
161
162 @dataclass
163 class CommandResult:
164     """The output of executing a command ? provider-independent.
165
166     The adapter (Telethon or Bot API) uses ``reply_text`` to send the response
167     and ``config_changed`` to decide whether to hot-reload the filter engine.
168     """
169
170     reply_text: str = ""
171     config_changed: bool = False
172
173 # --- command service -----
174
175 class CommandService:
176     """Provider-independent command execution.
177
178     Parses a command string against an account's current config and DB,
179     returning a ``CommandResult``. No Telethon objects involved ? pure string
180     in, effects out. The Telethon adapter (``commands.py``) handles the
181     transport (checking message.out, sending the reply); the Bot API adapter
182     (future) calls ``handle()`` with the ``from.id`` from the update.
183
184     Usage::
185
186         svc = CommandService()
187         result = await svc.handle(user_id, text, conn, current_cfg)
188         if result.config_changed:
189             engine_holder["engine"] = FilterEngine(current_cfg)

```

```

189         await send_reply(result.reply_text)
190     """
191
192     # -- public API -----
193
194     async def handle(
195         self,
196         user_id: int,
197         text: str,
198         conn: aiosqlite.Connection,
199         current_cfg: FilterConfig,
200     ) -> CommandResult:
201         """Parse and execute a command text, returning the reply + change flag."""
202         text = text.rstrip()
203
204         # --- /set size <value><unit> -----
205         m = _SIZE_RE.match(text)
206         if m:
207             value_str, unit = m.group(1), m.group(2) or ""
208             try:
209                 new_mb = parse_size_mb(value_str, unit)
210             except ValueError:
211                 return CommandResult("? Invalid size format. Use e.g. `/set size
10MB`.")
212             if new_mb < 1:
213                 return CommandResult("? Size must be at least 1 MB (`FilterConfig`
constraint).")
214             cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
215             return CommandResult(
216                 f"? Max file size set to
**[_format_size(new_mb)]**.\n\n{format_config(cfg)}",
217                 config_changed=True,
218             )
219
220         # --- /set length <value><unit> -----
221         m = _LENGTH_RE.match(text)
222         if m:
223             value_str, unit = m.group(1), m.group(2) or ""
224             try:
225                 new_secs = parse_duration_s(value_str, unit)
226             except ValueError:
227                 return CommandResult("? Invalid duration format. Use e.g. `/set length
15m`.")
228             min_secs = current_cfg.min_duration_s or 0
229             if new_secs < min_secs:
230                 return CommandResult(
231                     f"? Max duration ({_format_duration(new_secs)}) must be "
232                     f">= min duration ({_format_duration(min_secs)})."
233                 )
234             cfg = await self._update(conn, user_id, current_cfg,
max_duration_s=new_secs)
235             return CommandResult(
236                 f"? Max duration set to
**[_format_duration(new_secs)]**.\n\n{format_config(cfg)}",
237                 config_changed=True,
238             )
239
240         # --- /set min length <value><unit> -----
241         m = _MIN_LENGTH_RE.match(text)
242         if m:
243             value_str, unit = m.group(1), m.group(2) or ""
244             try:
245                 new_min = parse_duration_s(value_str, unit)
246             except ValueError:
247                 return CommandResult("? Invalid duration format. Use e.g. `/set min

```

```

length 5s`.")
248         max_secs = current_cfg.max_duration_s or 0
249         if max_secs > 0 and new_min > max_secs:
250             return CommandResult(
251                 f"? Min duration ({_format_duration(new_min)}) must be "
252                 f"<= max duration ({_format_duration(max_secs)})."
253             )
254         cfg = await self._update(conn, user_id, current_cfg, min_duration_s=new_min)
255         return CommandResult(
256             f"? Min duration set to
**{_format_duration(new_min)}**.\n\n{format_config(cfg)}",
257             config_changed=True,
258         )
259
260     # --- /set min resolution <N>p -----
261     m = _MIN_RES_RE.match(text)
262     if m:
263         new_height = int(m.group(1))
264         if new_height < 1:
265             return CommandResult("? Resolution must be at least 1p.")
266         cfg = await self._update(conn, user_id, current_cfg,
target_height=new_height)
267         return CommandResult(
268             f"? Min resolution set to **{new_height}p**.\n\n{format_config(cfg)}",
269             config_changed=True,
270         )
271
272     if _MIN_RES_HELP_RE.match(text):
273         return CommandResult(
274             "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)"
275         )
276
277     # --- /set exclude <gif|round|nosound> <on|off> -----
278     m = _EXCLUDE_RE.match(text)
279     if m:
280         what = m.group(1).lower()
281         state = m.group(2).lower()
282         enabled = state == "on"
283         labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent
videos"}
284         field_map = {
285             "gif": "exclude_gif",
286             "round": "exclude_round",
287             "nosound": "exclude_nosound",
288         }
289         cfg = await self._update(conn, user_id, current_cfg, **{field_map[what]:
enabled})
290         return CommandResult(
291             f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
292             f"\n\n{format_config(cfg)}",
293             config_changed=True,
294         )
295
296     if _EXCLUDE_HELP_RE.match(text):
297         return CommandResult(
298             "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
299             "Example: `/set exclude gif on`"
300         )
301
302     # --- /show config -----
303     if _SHOW_CONFIG_RE.match(text):
304         cfg = await db.load_filter_config(conn, user_id)
305         return CommandResult(format_config(cfg))
306
307     # --- /show commands -----

```



```

308         if _SHOW_COMMANDS_RE.match(text):
309             return CommandResult(format_commands())
310
311         # Not a recognised command ? return empty result (noise).
312         return CommandResult()
313
314     # -- internal -----
315
316     async def _update(
317         self,
318         conn: aiosqlite.Connection,
319         user_id: int,
320         current: FilterConfig,
321         max_size_mb: int | None = None,
322         max_duration_s: int | None = None,
323         min_duration_s: int | None = None,
324         target_height: int | None = None,
325         exclude_gif: bool | None = None,
326         exclude_round: bool | None = None,
327         exclude_nosound: bool | None = None,
328     ) -> FilterConfig:
329         """Apply partial updates to the account's config, persist, and return."""
330         updates: dict[str, object] = {}
331         if max_size_mb is not None:
332             updates["max_size_mb"] = max_size_mb
333         if max_duration_s is not None:
334             updates["max_duration_s"] = max_duration_s
335         if min_duration_s is not None:
336             updates["min_duration_s"] = min_duration_s
337         if target_height is not None:
338             updates["target_height"] = target_height
339         if exclude_gif is not None:
340             updates["exclude_gif"] = exclude_gif
341         if exclude_round is not None:
342             updates["exclude_round"] = exclude_round
343         if exclude_nosound is not None:
344             updates["exclude_nosound"] = exclude_nosound
345         cfg = current.model_copy(update=updates) if updates else current
346         await db.save_filter_config(conn, user_id, cfg)
347         log.info(
348             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
349             "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
350             user_id,
351             cfg.max_size_mb,
352             cfg.min_duration_s,
353             cfg.max_duration_s,
354             cfg.target_height,
355             cfg.exclude_gif,
356             cfg.exclude_round,
357             cfg.exclude_nosound,
358         )
359         return cfg
360
361     __all__ = [
362         "ENGINE_KEY",
363         "CommandResult",
364         "CommandService",
365         "_format_duration",
366         "_format_size",
367         "format_config",
368         "format_commands",
369         "parse_duration_s",
370         "parse_size_mb",
371     ]

```

**212. assistant (2026-07-06T08:36:58.306Z)**

`command\_service.py` is a clean extraction. The critical refactor risk is **behavioral equivalence** with the old inline logic ? especially two things I want to check: `/show config` now reloads from DB (was it before?), and `\_update` uses `model\_copy(update=...)` which in Pydantic v2 **skips validators**. Let me diff against the old `commands.py` from main and read the new thin adapter.

**213. user (2026-07-06T08:37:02.351Z)**

```

=== OLD commands.py: _update_config + /show config + model_copy/FilterConfig usage ===
1: """Config commands: /set size, /set length, /show config.
38: _SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
160:     """/show config` ? Show current filter settings",
195:     """Return an async handler for /set and /show config commands.
221:     # reject commands (e.g. "/show config " ? "/show config").
239:         cfg = await _update_config(conn, user.id, max_size_mb=new_mb)
262:         current_cfg = await db.load_filter_config(conn, user.id)
272:         cfg = await _update_config(conn, user.id, max_duration_s=new_secs)
296:         current_cfg = await db.load_filter_config(conn, user.id)
306:         cfg = await _update_config(conn, user.id, min_duration_s=new_min)
324:         cfg = await _update_config(conn, user.id, target_height=new_height)
354:         cfg = await _update_config(conn, user.id, **{field_map[what]: enabled})
374:     # --- /show config -----
375:     if _SHOW_CONFIG_RE.match(text):
376:         cfg = await db.load_filter_config(conn, user.id)
391: async def _update_config(
403:     cfg = await db.load_filter_config(conn, user_id)
405:     cfg = cfg.model_copy(update={"max_size_mb": max_size_mb})
407:     cfg = cfg.model_copy(update={"max_duration_s": max_duration_s})
409:     cfg = cfg.model_copy(update={"min_duration_s": min_duration_s})
411:     cfg = cfg.model_copy(update={"target_height": target_height})
413:     cfg = cfg.model_copy(update={"exclude_gif": exclude_gif})
415:     cfg = cfg.model_copy(update={"exclude_round": exclude_round})
417:     cfg = cfg.model_copy(update={"exclude_nosound": exclude_nosound})
455:     "_SHOW_CONFIG_RE",
459:     "_update_config",

=== OLD _update_config body ===
async def _update_config(
    conn: aiosqlite.Connection,
    user_id: int,
    max_size_mb: int | None = None,
    max_duration_s: int | None = None,
    min_duration_s: int | None = None,
    target_height: int | None = None,
    exclude_gif: bool | None = None,
    exclude_round: bool | None = None,
    exclude_nosound: bool | None = None,
) -> FilterConfig:
    """Load the current config, apply partial updates, persist, and return."""
    cfg = await db.load_filter_config(conn, user_id)
    if max_size_mb is not None:
        cfg = cfg.model_copy(update={"max_size_mb": max_size_mb})
    if max_duration_s is not None:
        cfg = cfg.model_copy(update={"max_duration_s": max_duration_s})
    if min_duration_s is not None:
        cfg = cfg.model_copy(update={"min_duration_s": min_duration_s})
    if target_height is not None:
        cfg = cfg.model_copy(update={"target_height": target_height})
    if exclude_gif is not None:
        cfg = cfg.model_copy(update={"exclude_gif": exclude_gif})
    if exclude_round is not None:

```

```

    cfg = cfg.model_copy(update={"exclude_round": exclude_round})
if exclude_nosound is not None:
    cfg = cfg.model_copy(update={"exclude_nosound": exclude_nosound})
await db.save_filter_config(conn, user_id, cfg)
log.info(
    "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
    "target_height=%s "
    "exclude_gif=%s exclude_round=%s exclude_nosound=%s",
    user_id,
    cfg.max_size_mb,
    cfg.min_duration_s,
    cfg.max_duration_s,
    cfg.target_height,
    cfg.exclude_gif,
    cfg.exclude_round,
    cfg.exclude_nosound,
)
return cfg

```

#### 214. assistant (2026-07-06T08:37:25.619Z)

Confirmed: both old `\_update\_config` and new `\_update` use `model\_copy(update=...)` (validators skipped) ? so that's **\*\*preserved behavior, not a regression\*\***. The one real difference: old `\_update\_config` **\*\*reloaded\*\*** config from DB each call; new `\_update` uses the passed `current\_cfg`. Whether that's equivalent hinges on the adapter loading `current\_cfg` fresh. Let me read the new `commands.py` adapter and the gate results.

#### 215. user (2026-07-06T08:37:25.989Z)

```

1      """Config commands ? Telethon adapter over the provider-independent `CommandService`.
2
3      This module is a thin Telethon adapter. It handles Telethon-specific transport
4      (checking `message.out`, `peer_id`, sending replies) and delegates command
5      parsing and config mutation to `command_service.CommandService` (ADR 0011, Track D).
6
7      Re-exports regex patterns and unit-parsers for backwards-compatibility with
8      existing tests (`tests/test_commands.py`).
9
10     See `docs/adr/0007-inline-config-commands.md` for the command model and
11     `docs/adr/0008-bot-api-onboarding-front-door.md` for the Bot API front-door.
12     """
13
14     from __future__ import annotations
15
16     import logging
17     from collections.abc import Callable
18
19     import aiosqlite
20     from telethon import TelegramClient
21     from telethon.errors import RPCError
22
23     from tg_video_filter import db
24     from tg_video_filter.command_service import ENGINE_KEY, CommandResult, CommandService
25     from tg_video_filter.filters import FilterEngine
26     from tg_video_filter.models import FilterConfig, User
27
28     log = logging.getLogger(__name__)
29
30     from tg_video_filter.command_service import ( # noqa: E402
31         _EXCLUDE_HELP_RE,
32         _EXCLUDE_RE,
33         _LENGTH_RE,
34         _MIN_LENGTH_RE,
35         _MIN_RES_HELP_RE,
36         _MIN_RES_RE,

```

```

37     _SHOW_COMMANDS_RE,
38     _SHOW_CONFIG_RE,
39     _SIZE_RE,
40     _format_duration,
41     _format_size,
42     format_commands,
43     format_config,
44     parse_duration_s,
45     parse_size_mb,
46 )
47
48
49 def make_config_command_handler(
50     user: User,
51     conn: aiosqlite.Connection,
52     engine_holder: dict[str, FilterEngine],
53 ) -> Callable:
54     svc = CommandService()
55
56     async def on_command(event: object) -> None:
57         message = getattr(event, "message", None)
58         if message is None:
59             return
60         if not getattr(message, "out", False):
61             return
62         peer = getattr(message, "peer_id", None)
63         peer_user_id = getattr(peer, "user_id", None) if peer else None
64         if peer_user_id != user.id:
65             return
66
67         text: str = getattr(message, "message", "") or ""
68         client: TelegramClient = getattr(event, "client", None) # type:
ignore[assignment]
69
70         current_cfg = await db.load_filter_config(conn, user.id)
71         result: CommandResult = await svc.handle(user.id, text, conn, current_cfg)
72
73         if result.config_changed:
74             new_cfg = await db.load_filter_config(conn, user.id)
75             engine_holder[ENGINE_KEY] = FilterEngine(new_cfg)
76
77         if result.reply_text:
78             await _reply(client, message, result.reply_text)
79
80     return on_command
81
82
83 # --- backwards-compat helpers (test_commands.py references) -----
84
85
86 async def _update_config(
87     conn: aiosqlite.Connection,
88     user_id: int,
89     max_size_mb: int | None = None,
90     max_duration_s: int | None = None,
91     min_duration_s: int | None = None,
92     target_height: int | None = None,
93     exclude_gif: bool | None = None,
94     exclude_round: bool | None = None,
95     exclude_nosound: bool | None = None,
96 ) -> FilterConfig:
97     """Deprecated wrapper ? delegates to ``CommandService._update``.
98
99     Kept for backwards-compatibility with ``tests/test_commands.py`` which
100     imports ``_update_config`` from here. New code should call

```

```

101         :meth:`CommandService.handle` which returns a :class:`CommandResult`.
102         """
103         svc = CommandService()
104         cfg = await db.load_filter_config(conn, user_id)
105         return await svc._update(
106             conn,
107             user_id,
108             cfg,
109             max_size_mb=max_size_mb,
110             max_duration_s=max_duration_s,
111             min_duration_s=min_duration_s,
112             target_height=target_height,
113             exclude_gif=exclude_gif,
114             exclude_round=exclude_round,
115             exclude_nosound=exclude_nosound,
116         )
117
118
119     async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
120         try:
121             await client.send_message("me", text)
122         except (OSError, TimeoutError, RPCError):
123             log.warning("config_reply_failed", exc_info=True)
124
125
126     __all__ = [
127         "ENGINE_KEY",
128         "_EXCLUDE_RE",
129         "_LENGTH_RE",
130         "_MIN_LENGTH_RE",
131         "_MIN_RES_RE",
132         "_SHOW_CONFIG_RE",
133         "_SHOW_COMMANDS_RE",
134         "_EXCLUDE_HELP_RE",
135         "_MIN_RES_HELP_RE",
136         "_SIZE_RE",
137         "_format_duration",
138         "_format_size",
139         "_update_config",
140         "format_commands",
141         "format_config",
142         "make_config_command_handler",
143         "parse_duration_s",
144         "parse_size_mb",
145     ]
146

```

## 216. assistant (2026-07-06T08:37:41.039Z)

The new adapter loads `current\_cfg` **fresh** before `handle()` and reloads after a change to rebuild the engine ? so the DB-reload difference is equivalent. Now let me verify the text-normalization (old had a "reject trailing whitespace" comment; new does `rstrip()`), the regex patterns are byte-identical, and read the gate.

## 217. user (2026-07-06T08:37:47.896Z)

=== GATE ===

```

-----
TOTAL                                1111    25    278    16    97%
Required test coverage of 80.0% reached. Total coverage: 97.05%
482 passed in 7.35s
exit=0
DONE

```

=== regex patterns: OLD vs NEW (diff) ===

PATTERNS IDENTICAL

=== OLD text normalization + out/peer checks (lines 205-235) ===

```
async def on_command(event: object) -> None:
    message = getattr(event, "message", None)
    if message is None:
        return

    # Only respond to messages the user sent to themselves.
    if not getattr(message, "out", False):
        return

    peer = getattr(message, "peer_id", None)
    peer_user_id = getattr(peer, "user_id", None) if peer else None
    if peer_user_id != user.id:
        return

    text: str = getattr(message, "message", "") or ""
    # Strip trailing whitespace so mobile keyboards don't silently
    # reject commands (e.g. "/show config " ? "/show config").
    text = text.rstrip()
    client: TelegramClient = getattr(event, "client", None) # type: ignore[assignment]

    # --- /set size <value><unit> -----
    m = _SIZE_RE.match(text)
    if m:
        value_str, unit = m.group(1), m.group(2) or ""
        try:
            new_mb = parse_size_mb(value_str, unit)
        except ValueError:
            await _reply(client, message, "? Invalid size format. Use e.g. `/set size
10MB`.")
            return
        if new_mb < 1:
            await _reply(
                client, message, "? Size must be at least 1 MB (`FilterConfig` constraint).")
            return
        cfg = await _update_config(conn, user.id, max_size_mb=new_mb)
        engine_holder[ENGINE_KEY] = FilterEngine(cfg)
```

## 218. assistant (2026-07-06T08:38:38.146Z)

Strong equivalence signals: **\*\*regex patterns byte-identical\*\***, ``rstrip`` preserved (moved into ``handle()``), branch order unchanged, transport checks identical, and **\*\*`test\_commands.py` is untouched yet all 482 pass\*\*** (coverage up to 97.05%). Let me launch a focused equivalence-hunting workflow while I do a differential check myself.

## 219. user (2026-07-06T08:39:27.295Z)

Workflow launched in background. Task ID: wpjqa0myl

Summary: Review telegram-video-filter PR #38 (Track D: extract CommandService) ? focus on behavioral equivalence of the refactor, with adversarial verification

Transcript dir: C:\Users\avidu\claudes\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_634e841e-51b

Script file: C:\Users\avidu\claudes\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr38-track-d-wf\_634e841e-51b.js

(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:

"C:\Users\avidu\claudes\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr38-track-d-wf\_634e841e-51b.js"} to iterate without resending the script.)

Run ID: wf\_634e841e-51b

To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\claudes\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-

pr38-track-d-wf\_634e841e-51b.js", resumeFromRunId: "wf\_634e841e-51b"}}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

## 220. assistant (2026-07-06T08:39:41.025Z)

While the workflow runs, let me do my own differential check: diff every reply string between old and new (catches transcription drift), and run a battery of commands through the new service.

## 221. user (2026-07-06T08:39:44.302Z)

```
=== reply strings: OLD vs NEW (?? messages) ===
2,5c2,5
< "? Max duration set to **{_format_duration(new_secs)}**."
< "? Max file size set to **{_format_size(new_mb)}**."
< "? Min duration set to **{_format_duration(new_min)}**."
< "? Min resolution set to **{new_height}p**."
---
> "? Max duration set to **{_format_duration(new_secs)}**.\n\n{format_config(cfg)}"
> "? Max file size set to **{_format_size(new_mb)}**.\n\n{format_config(cfg)}"
> "? Min duration set to **{_format_duration(new_min)}**.\n\n{format_config(cfg)}"
> "? Min resolution set to **{new_height}p**.\n\n{format_config(cfg)}"
>>> ^^ reply strings differ (investigate)
```

## 222. user (2026-07-06T08:39:51.973Z)

```
/set size 10MB          -> changed=True | '? Max file size set to **10MB**.'
/set size 2GB           -> changed=True | '? Max file size set to **2GB**.'
/set size 0             -> changed=False | '? Size must be at least 1 MB (`FilterConfig`
constra'
/set size abcMB         -> changed=False | '(empty)'
/set size 500KB         -> changed=True | '? Max file size set to **1MB**.'
/set length 15m         -> changed=True | '? Max duration set to **15m**.'
/set length 0           -> changed=False | '? Max duration (0s) must be >= min duration
(5s).'
/set max length 2h      -> changed=True | '? Max duration set to **2h**.'
/set min length 5s      -> changed=True | '? Min duration set to **5s**.'
/set min length 999h    -> changed=False | '? Min duration (999h) must be <= max duration
(2h).'
/set min resolution 720p -> changed=True | '? Min resolution set to **720p**.'
/set min res 1080       -> changed=True | '? Min resolution set to **1080p**.'
/set min resolution 0p   -> changed=False | '? Resolution must be at least 1p.'
/set min resolution abc  -> changed=False | '? Usage: `/set min resolution <N>p` (e.g.
`/set min`
/set exclude gif on     -> changed=True | '? GIF/animated exclusion: **ON**.'
/set exclude round off  -> changed=True | '? round videos exclusion: **OFF**.'
/set exclude bogus on   -> changed=False | '? Usage: `/set exclude <gif|round|nosound>
<on|off>`'
/show config            -> changed=False | '? **Current Filter Configuration**'
/show commands          -> changed=False | '? **Available Commands**'
/SET SIZE 10mb          -> changed=True | '? Max file size set to **10MB**.'
/show config            -> changed=False | '? **Current Filter Configuration**'
random noise            -> changed=False | '(empty)'
/set size 10 MB         -> changed=True | '? Max file size set to **10MB**.'
```

## 223. assistant (2026-07-06T08:40:14.995Z)

The battery looks correct, but the reply-string diff flags a **real difference**: the new success replies append `\n\n{format_config(cfg)}`, while the old strings were just `"? ... set to X."`. I need to check whether the old handler appended the config elsewhere (equivalent) or the new code changed the UX. Let me look at the exact old success-reply construction.

## 224. user (2026-07-06T08:40:18.101Z)

```
=== OLD size-success reply construction (lines 239-260) ===
    cfg = await _update_config(conn, user.id, max_size_mb=new_mb)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Max file size set to **{_format_size(new_mb)}**."
f"\n\n{format_config(cfg)}",
    )
    return

# --- /set length <value><unit> -----
m = _LENGTH_RE.match(text)
if m:
    value_str, unit = m.group(1), m.group(2) or ""
    try:
        new_secs = parse_duration_s(value_str, unit)
    except ValueError:
        await _reply(
            client, message, "? Invalid duration format. Use e.g. `/set length 15m`."
        )
        return
    # Enforce max_duration_s >= min_duration_s so every video isn't
    # silently rejected (and so the FilterConfig validator passes on
    # next startup).
    current_cfg = await db.load_filter_config(conn, user.id)

=== count _reply calls per success branch in OLD (does it send config separately?) ===
134: def format_config(cfg: FilterConfig) -> str:
232:     await _reply(client, message, "? Invalid size format. Use e.g. `/set size
10MB`.")
235:     await _reply(
241:     await _reply(
244:         f"? Max file size set to **{_format_size(new_mb)}**."
f"\n\n{format_config(cfg)}",
255:     await _reply(
265:     await _reply(
274:     await _reply(
278:         f"\n\n{format_config(cfg)}",
289:     await _reply(
299:     await _reply(
308:     await _reply(
312:         f"\n\n{format_config(cfg)}",
322:     await _reply(client, message, "? Resolution must be at least 1p.")
326:     await _reply(
329:         f"? Min resolution set to **{new_height}p**." f"\n\n{format_config(cfg)}",
335:     await _reply(
356:     await _reply(
360:         f"\n\n{format_config(cfg)}",
366:     await _reply(
377:     await _reply(client, message, format_config(cfg))
382:     await _reply(client, message, format_commands())
435: async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
444:     log.warning("config_reply_failed", exc_info=True)
461:     "format_config",
```

## 225. assistant (2026-07-06T08:40:37.534Z)

Resolved ? false alarm from my grep. The OLD code used **implicit string concatenation** of two adjacent f-string literals (`f"? ... **." f"\n\n{format_config(cfg)}"`), which Python joins into the same single string the NEW code produces as one literal. So the success replies are **identical**. My differential check now shows full equivalence (patterns, strings, branch order, battery all match).



Let me confirm the re-export surface covers everything `test\_commands.py` imports, then await the workflow.

## 226. user (2026-07-06T08:40:40.322Z)

```
=== what does (unchanged) test_commands.py import from commands? ===
17:from tg_video_filter.commands import (
```

```
=== multiline import block ===
from tg_video_filter.commands import (
    _EXCLUDE_RE,
    _LENGTH_RE,
    _MIN_LENGTH_RE,
    _MIN_RES_HELP_RE,
    _MIN_RES_RE,
    _SHOW_COMMANDS_RE,
    _SHOW_CONFIG_RE,
    _SIZE_RE,
    ENGINE_KEY,
    _format_duration,
    _format_size,
    _update_config,
    format_commands,
    format_config,
    make_config_command_handler,
    parse_duration_s,
    parse_size_mb,
)
```

## 227. assistant (2026-07-06T08:41:02.758Z)

All re-exports covered ? `test\_commands.py`'s 17 imports all resolve (the suite passing confirms it). My independent equivalence pass finds **\*\*no divergences\*\***: patterns, reply strings, branch order, transport, and the command battery all match the old behavior; re-export surface is complete.

While the workflow finishes its adversarial pass, here's the interim picture:

```
**Gate (branch `22abe72`):** ? `ruff check` + `format` clean . ? **482 passed @ 97.05%** (up from 96.52%, floor 80%).
```

```
**Equivalence (verified myself):**
```

- Regex patterns **\*\*byte-identical\*\***; branch order unchanged; `rstrip` preserved (moved into `handle()`).
- Reply strings **\*\*identical\*\*** ? the old used implicit f-string concatenation (`f"??." f"\n\n{format\_config}"`); the new merges them into one literal ? same output.
- 23-command battery through the new `CommandService` all produce correct replies + `config\_changed` flags.
- Adapter loads `current\_cfg` fresh (equivalent to old's in-`\_update\_config` reload); engine rebuild reloads from DB (equivalent since save?load round-trips).
- All 17 `test\_commands.py` imports satisfied by the re-export shim.

So far this reads as a clean, behavior-preserving refactor. I'll fold in the workflow's findings (it's probing coverage of the moved adapter and any subtle edge divergence) and give you the consolidated review.

## 228. user (2026-07-06T08:44:26.951Z)

```
<task-notification>
<task-id>wpjqa0myl</task-id>
<tool-use-id>toolu_01GXhY7ah9Zus64yZpnaPRgg</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\wpjqa0myl.output</output-file>
<status>completed</status>
```

<summary>Dynamic workflow "Review telegram-video-filter PR #38 (Track D: extract CommandService) ? focus on behavioral equivalence of the refactor, with adversarial verification" completed</summary>

<result>{"counts":{"raw":3,"kept":3,"refuted":0},"findings":[{"dimension":"tests","severity":"medium","verdict":"CONFIRMED","title":"test\_command\_service.py never asserts the persisted config mutation ? a wrong field/value in CommandService.\_update would pass all 20 service tests","where":"tests/test\_command\_service.py:87","summary":"The new service tests only assert result.config\_changed is True plus a substring of reply\_text; none reload the config from the DB to verify the correct FilterConfig field was actually mutated to the correct value."},"failure\_scenario":"Swap the field\_map in command\_service.py so 'gif' maps to 'exclude\_round' (or change \_update to write max\_size\_mb into min\_duration\_s). test\_set\_exclude\_gif\_on still passes because the reply label 'GIF/animated' + 'ON' is derived from labels[what], decoupled from field\_map, and config\_changed stays True; test\_set\_size passes because '100MB' still appears in the reply. The persisted-field regression is invisible to test\_command\_service.py ? it is caught only incidentally by tests/test\_commands.py (e.g. loaded.max\_size\_mb == 512 at line 541), which exercises the same \_update via the adapter. The dedicated service test file, the natural home for that assertion, does not verify its core responsibility (correct persistence).","evidence":"grep for load\_filter\_config / .max\_size\_mb / .exclude\_ across tests/test\_command\_service.py returns zero hits ? no test loads config back. Contrast tests/test\_commands.py:541 `assert loaded.max\_size\_mb == 512`. Confirmed reply label is independent of field\_map, so a swap is not caught by the service tests."},"recommendation":"In the success-path service tests (test\_set\_size, test\_set\_length, test\_set\_min\_length, test\_set\_exclude\_\*, test\_set\_min\_resolution) add an assertion that reloads: `loaded = await db.load\_filter\_config(conn, USER\_ID); assert loaded.max\_size\_mb == 100` etc., so the service file independently verifies the field/value that was persisted."},"verify\_reasoning":"The finding is accurate and I reproduced it. tests/test\_command\_service.py (all 223 lines read) contains no db.load\_filter\_config call and no assertion on any persisted FilterConfig field/value ? its success-path tests assert only result.config\_changed is True plus a reply\_text substring. I empirically confirmed the blind spot: I applied BOTH mutations the finding describes to command\_service.py ? field\_map["gif"] = "exclude\_round" (command\_service.py:284-288) and \_update writing max\_size\_mb into updates["min\_duration\_s"] (command\_service.py:331-332) ? and ran the file: 28 passed, 0 assertion failures (the only "FAIL" is the 80% global coverage floor when running one file in isolation, not a test assertion). So a wrong persisted field/value is invisible to the dedicated service tests.\n\nThe decoupling mechanism is real and verified in source: in the exclude branch the reply uses labels[what] (keyed by the regex group `what`, command\_service.py:283) and ON/OFF from `state`/`enabled` (command\_service.py:281,291) ? both independent of field\_map ? so test\_set\_exclude\_gif\_on's "ON" in reply\_text` passes even when the wrong config field is set. For /set size, the reply prefix contains \_format\_size(new\_mb) = "100MB" regardless of which config field \_update mutates, so test\_set\_size's "100MB" in reply\_text` passes even when max\_size\_mb is not the field written. The regression is caught only incidentally by tests/test\_commands.py:541 (`assert loaded.max\_size\_mb == 512`), exactly as the finding states ? that file reloads and asserts persistence; the new service file does not.\n\nSeverity calibration: this is a test-quality gap, not a behavioral divergence and not a shipping product bug. Because this PR is a REFACTOR and the field\_map/\_update persistence path remains covered end-to-end by the UNCHANGED test\_commands.py adapter tests (which the task confirms still pass), a real persistence regression would still be caught by the suite as a whole ? it just wouldn't be caught by the new dedicated test file that is the natural home for that assertion. That makes it a legitimate weakness in the added tests but keeps it at the lower end. Medium is defensible; low would also be reasonable. I keep medium since the finding, evidence, and recommendation are all correct, but note it sits at the low end of medium given the redundant adapter coverage."}],"dimension":"service\_correctness","severity":"low","verdict":"CONFIRMED","title":"Adapter loads FilterConfig from DB on every self-message, including non-commands and /show config (extra DB read vs. old code)","where":"src/tg\_video\_filter/commands.py:70","summary":"The new Telethon adapter unconditionally calls db.load\_filter\_config(conn, user.id) at line 70 before every handle() call, so plain self-messages (e.g. the user typing 'hello' in Saved Messages) and /show config now trigger a config read that the old code avoided; for config-changing commands it does a second load at line 74 to build the engine, giving 2 DB reads + 1 write per mutating command."},"failure\_scenario":"User sends any non-command text to their own Saved Messages: the old handler matched no branch and never touched the DB; the new adapter issues one db.load\_filter\_config read per such message. And /show config now reads twice (once in the adapter at line 70, once inside handle() at line 304). No user-visible behavior change and no correctness impact -- purely redundant reads on a per-message hot path. Verified: reply text, saved config, engine state, and transport guards are all behaviorally identical to the old code across a 46-input battery; this is only an efficiency/hygiene note, not a

```

divergence.", "evidence": "commands.py:70 `current_cfg = await db.load_filter_config(conn,
user.id)` runs before svc.handle for every message that passes the out/peer guards.
commands.py:74 does a second `new_cfg = await db.load_filter_config(...)` when
result.config_changed. command_service.py:304 loads again for /show config. Old commands.py only
hit the DB inside a matched branch. A realistic single-store harness confirmed loads=2/saves=1
for a mutating command and the engine cfg correctly reflects the saved value (max_size_mb=50),
so this is safe -- just extra reads.", "recommendation": "Optional micro-optimization: skip the
line-70 load for messages that aren't commands, or pass the already-loaded current_cfg into
handle() for /show config instead of reloading inside handle(). Since config reads are cheap
SQLite JSON-blob loads and the full suite passes at 97% coverage, this is not worth blocking the
PR; leave as-is unless the self-message volume is high.", "verify_reasoning": "Verified by reading
both the new code (commands.py:70-75, command_service.py:303-305) and the old code (git show
main:src/tg_video_filter/commands.py).\\n\\nFacts confirmed:\\n1. New adapter commands.py:70
unconditionally calls `current_cfg = await db.load_filter_config(conn, user.id)` for EVERY
message that clears the out/peer guards, BEFORE dispatch. The old handler did NOT: it only
touched the DB inside a matched branch (each _update branch loaded via _update_config; /show
config loaded inside its branch). A non-command self-message (e.g. \"hello\" in Saved Messages)
matched no branch in the old code and never read the DB. In the new code it triggers exactly one
load_filter_config read. Divergence in DB access pattern confirmed.\\n2. /show config double-
reads: adapter loads at line 70, then CommandService.handle re-loads at command_service.py:304
and uses THAT value for the reply. The line-70 value is discarded for this branch. 2 reads
confirmed.\\n3. Mutating command: line 70 load (passed into handle as current_cfg, used as
mutation base and for min/max cross-checks) + line 74 reload (to build the FilterEngine from the
just-saved config) + 1 save. Matches the finding's stated loads=2/saves=1.\\n\\nBehavioral-
equivalence check: the extra reads are pure reads on the same connection against the same
persisted config; they do not
... (truncated 4824 chars, full result in C:\\Users\\avidu\\AppData\\Local\\Temp\\claude\\C--Users-
avidu-Projects-badminton-highlight-
indexer\\7dc3da51-af38-47b6-83f6-fdda83b60664\\tasks\\wpjqa0myl.output)</result>
<usage><agent_count>7</agent_count><subagent_tokens>433737</subagent_tokens><tool_uses>83</tool_
uses><duration_ms>299574</duration_ms></usage>
</task-notification>

```

## 229. assistant (2026-07-06T08:44:38.191Z)

The workflow found **only 3 findings ? zero behavioral divergences** (the equivalence dimension came back clean, corroborating my own pass). Let me read the full result for the third finding.

## 230. user (2026-07-06T08:44:43.096Z)

```

COUNTS: {'raw': 3, 'kept': 3, 'refuted': 0}
=====
[MEDIUM/CONFIRMED] (tests) test_command_service.py never asserts the persisted config mutation ?
a wrong field/value in CommandService._update would pass all 20 service tests
  where: tests/test_command_service.py:87
  rec : In the success-path service tests (test_set_size, test_set_length,
test_set_min_length, test_set_exclude_*, test_set_min_resolution) add an assertion that reloads:
`loaded = await db.load_filter_config(conn, USER_ID); as

[LOW/CONFIRMED] (service_correctness) Adapter loads FilterConfig from DB on every self-message,
including non-commands and /show config (extra DB read vs. old code)
  where: src/tg_video_filter/commands.py:70
  rec : Optional micro-optimization: skip the line-70 load for messages that aren't commands,
or pass the already-loaded current_cfg into handle() for /show config instead of reloading
inside handle(). Since config reads are che

[LOW/CONFIRMED] (tests) test_set_size_invalid_format_returns_empty is misnamed and does not
cover the except ValueError branch it appears to target
  where: tests/test_command_service.py:93
  rec : Either rename the test to reflect that it covers the unrecognized/noise path, or
(better) delete the unreachable except ValueError branches in command_service.py since no regex-
matching input can make parse_size_mb/parse

--- dimensions represented: ['service_correctness', 'tests']
--- refuted: []

```

### 231. assistant (2026-07-06T08:45:02.687Z)

Three findings ? all confirmed, all **test-quality/hygiene**, zero behavioral divergence (the equivalence dimension found nothing across a 46-input battery, matching my own pass). Let me quickly verify the most concrete new claim ? that the `except ValueError`` branches are unreachable dead code ? and glance at the misnamed test.

### 232. user (2026-07-06T08:45:10.133Z)

```
=== can a _SIZE_RE/_LENGTH_RE-matching input EVER make parse_* raise ValueError? ===
parse_size_mb raises for regex-valid units: NEVER -> except branch unreachable
parse_duration_s raises for regex-valid units: NEVER -> except branch unreachable
```

```
=== the misnamed test ===
```

```
async def test_set_size_invalid_format_returns_empty(
    conn: aiomysqlite.Connection, svc: CommandService
) -> None:
    """Unparseable input that doesn't match the regex returns empty result (silent)."""
    result = await svc.handle(USER_ID, "/set size not-a-number", conn, FilterConfig())
    assert result.reply_text == "" # regex doesn't match, no reply
    assert result.config_changed is False
```

```
async def test_set_size_below_minimum(conn: aiomysqlite.Connection, svc: CommandService) -> None:
```

### 233. assistant (2026-07-06T08:45:47.730Z)

Confirmed both hygiene points: the `except ValueError`` branches are provably **unreachable** (no regex-matching input can make the parsers raise), and `test_set_size_invalid_format_returns_empty`` actually exercises the noise path (its input doesn't match `_SIZE_RE``), so the name misleads. Both are pre-existing patterns the refactor faithfully carried over.

Here's the consolidated review.

---

## Code Review ? PR #38: extract `CommandService`` (ADR 0011, Track D)

**Verdict:** ? Clean, behavior-preserving refactor ? mergeable, no blockers. The extraction is faithful: I found **zero behavioral divergences** from the old inline logic, and the independent multi-agent equivalence pass (46-input battery) agreed. Coverage went **up**. The three findings below are all test-quality/hygiene.

**Ramp-up:** synced; branch `feat/track-d-command-service`` (one commit `22abe72``) ? `main``, current with main, `MERGEABLE/CLEAN``.

**Gate (local):** ? `ruff check`` + `format`` clean . ? **482 passed @ 97.05%** (up from 96.52%, floor 80%).

**Behavioral equivalence ? verified two ways:**

- Regex patterns **byte-identical**; branch order unchanged; `rstrip`` preserved (moved into `handle()``).
- All reply strings **identical** ? the "difference" my first grep flagged was the old code's implicit f-string concatenation (`f"??." f"\n\n{format_config}"``); the new single literal produces the same output.
- 23-command battery through the new `CommandService`` ? correct replies + `config_changed`` for every case; the workflow's 46-input battery found no divergence.
- Adapter loads `current_cfg`` fresh and reloads to rebuild the engine ? equivalent to the old in-`_update_config`` reload (save?load round-trips).
- All 17 `test_commands.py`` imports satisfied by the re-export shim (and that unchanged suite passes).

## Findings (all non-blocking)

**\*\*? Medium (low end) ? service tests don't verify persistence\*\* ?**  
``tests/test_command_service.py``  
 The 20 new tests assert only ``config_changed`` + a reply substring; none reload the config to confirm the *\*right field\* got the \*right value\**. Verified by mutation: swapping ``field_map["gif"] ? "exclude_round"`` (or writing ``max_size_mb`` into ``min_duration_s``) still passes all 20 service tests ? the reply label is derived from ``labels[what]``, decoupled from ``field_map``. A real regression is caught only *\*incidentally\** by the unchanged ``test_commands.py`` (``assert loaded.max_size_mb == 512``). The dedicated service file ? the natural home for that assertion ? doesn't check its core responsibility.  
 > **\*\*Fix:\*\*** in the success-path tests add ``loaded = await db.load_filter_config(conn, USER_ID); assert loaded.max_size_mb == 100`` (etc.).

**\*\*? Low ? redundant config reads per self-message\*\* ?** ``commands.py:70``  
 The adapter calls ``db.load_filter_config`` for *\*every\** self-message that clears the out/peer gate ? including plain non-command text (old code never touched the DB there) ? and ``/show config`` reads twice (adapter + inside ``handle()``). No behavioral/correctness impact; just extra SQLite reads on the message hot path. Optional micro-opt (skip the load for non-commands, or pass ``current_cfg`` into the ``/show config`` branch).

**\*\*? Low ? dead `except ValueError` + misnamed test\*\* ?** ``command_service.py` /`tests/test_command_service.py:93``  
 The ``except ValueError`` branches for size/length/min-length are *\*unreachable\** ? the regex ``(\d+)`` + constrained unit group guarantee the parsers never raise (verified by enumerating all capturable units). ``test_set_size_invalid_format_returns_empty`` actually exercises the *\*noise\** path (its input doesn't match ``_SIZE_RE``), so the name misleads. Both are pre-existing patterns carried over. Either delete the dead branches (+rename the test) or leave as-is.

## ? Verified good

- Faithful extraction: identical patterns/strings/order/transport, complete re-exports, clean Telethon-free service ready for the Bot API adapter.
- ``model_copy(update=?)`` skips Pydantic validation in both old and new ? *\*\*preserved\*\**, and all constraint-violating inputs are guarded by explicit pre-checks (size<1, height<1, duration bounds), so no invalid config can persist.
- Coverage rose; ``commands.py`` command paths now fully mapped through the service.

---

**\*\*Bottom line:\*\*** ship it. Worth adding the persistence assertions (medium) since the new test file should own that check, but nothing here blocks. Want me to *\*\*post this to PR #38\*\** (comment, as before)?

**234. user (2026-07-06T08:46:05.062Z)**

yes pls post this as PR comment

**235. assistant (2026-07-06T08:46:11.070Z)**

Posting the review as a comment on PR #38.

**236. assistant (2026-07-06T08:46:30.630Z)**

## Code review ? extract `CommandService` (ADR 0011, Track D)

**\*\*Verdict: ? Clean, behavior-preserving refactor ? mergeable, no blockers.\*\*** The extraction is faithful: *\*\*zero behavioral divergences\*\** from the old inline logic, and coverage went up. Every finding was reproduced locally against ``22abe72``; the three below are all test-quality/hygiene (posting as a comment ? GitHub blocks the author from a formal approve).

### Gate

|                                          |                                                                                  |  |
|------------------------------------------|----------------------------------------------------------------------------------|--|
| Check                                    | Result                                                                           |  |
| --- ---                                  |                                                                                  |  |
| `pytest` (full suite)                    | ? <b>**482 passed**</b> , coverage <b>**97.05%**</b> (up from 96.52%, floor 80%) |  |
| `ruff check .` / `ruff format --check .` | ? clean                                                                          |  |

## Behavioral equivalence ? verified two ways (mine + an independent multi-agent pass)

- Regex patterns `**byte-identical**`; branch order unchanged; ``rstrip`` preserved (moved into ``handle()``).
- All reply strings `**identical**` ? the apparent diff was the old code's implicit f-string concatenation (``f"??. f"\n\n{format_config}"``); the new single literal produces the same output.
- Command battery (23 mine + 46 in the agent pass) ? correct reply + ``config_changed`` for every case; no divergence.
- Adapter loads ``current_cfg`` fresh and reloads to rebuild the engine ? equivalent to the old in-``_update_config`` reload (save?load round-trips).
- All 17 ``test_commands.py`` imports satisfied by the re-export shim (and that unchanged suite passes).

---

## Findings (all non-blocking)

### ? Medium (low end) ? service tests don't verify persistence ? ``tests/test_command_service.py``

The 20 new tests assert only ``config_changed`` + a reply substring; none reload the config to confirm the *\*correct field\** got the *\*correct value\**. Verified by mutation: swapping ``field_map["gif"]`` ? `"exclude_round"`` (or writing ``max_size_mb`` into ``min_duration_s``) still passes all 20 service tests ? the reply label comes from ``labels[what]``, decoupled from ``field_map``. A real persistence regression is caught only *\*incidentally\** by the unchanged ``test_commands.py`` (``assert loaded.max_size_mb == 512``); the dedicated service file ? the natural home for that assertion ? doesn't check its core responsibility.

**\*\*Fix:\*\*** in the success-path tests add a reload assertion, e.g.

```
```python
loaded = await db.load_filter_config(conn, USER_ID)
assert loaded.max_size_mb == 100
```
```

### ? Low ? redundant config reads per self-message ? ``commands.py:70``

The adapter calls ``db.load_filter_config`` for *\*every\** self-message that clears the out/peer gate ? including plain non-command text (the old code never touched the DB there) ? and ``/show config`` reads twice (adapter at line 70 + inside ``handle()`` at ``command_service.py:304``). No behavioral/correctness impact; just extra SQLite reads on the message hot path. Optional micro-opt: skip the line-70 load for non-commands, or pass ``current_cfg`` into the ``/show config`` branch instead of reloading.

### ? Low ? dead ``except ValueError`` + misnamed test ? ``command_service.py`` / ``tests/test_command_service.py:93``

The ``except ValueError`` branches for ``/set size``, ``/set length``, ``/set min length`` are **\*\*unreachable\*\*** ? the regex ``(\d+)`` plus the constrained unit group guarantee ``parse_size_mb``/``parse_duration_s`` never raise (verified by enumerating every capturable unit). ``test_set_size_invalid_format_returns_empty`` actually exercises the *\*noise\** path (its input doesn't match ``_SIZE_RE``), so the name misleads. Both are pre-existing patterns carried over by the refactor. Either delete the dead branches (+ rename the test) or leave as-is.

---

## ? Verified good

- Faithful extraction: identical patterns / strings / branch order / transport, complete re-exports, clean Telethon-free service ready for the Bot API adapter.
- ``model_copy(update=?)`` skips Pydantic validation in **\*\*both\*\*** old and new (preserved), and every constraint-violating input is guarded by an explicit pre-check (`size<1`, `height<1`, `duration` bounds) ? so no invalid ``FilterConfig`` can be persisted.
- Coverage rose; ``commands.py`` command paths now fully mapped through the service.

**\*\*Bottom line:\*\*** ship it. Worth adding the persistence assertions (the new test file should own that check), but nothing here blocks.

<sub>? Reviewed with [Claude Code](https://claude.com/claude-code). Behavioral equivalence verified against ``main``'s pre-refactor ``commands.py``; full suite green at 97.05%. Multi-agent

adversarial pass: 3 findings, 0 behavioral divergences.</sub>

#### 237. user (2026-07-06T08:46:30.659Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr38\_review.md (file state is current in your context ? no need to Read it back)

#### 238. user (2026-07-06T08:46:40.305Z)

avidullu [COMMENTED] 2026-07-06T08:46:39Z  
<https://github.com/avidullu/telegram-video-filter/pull/38#pullrequestreview-4634226485>

#### 239. assistant (2026-07-06T08:46:51.949Z)

? **Posted to PR #38** ? [pullrequestreview-4634226485](<https://github.com/avidullu/telegram-video-filter/pull/38#pullrequestreview-4634226485>)

### Summary

**Verdict:** ? clean, behavior-preserving refactor ? mergeable, no blockers.

- **Gate verified locally:** 482 passed @ 97.05% (up from 96.52%), ruff check + format clean.
- **Behavioral equivalence confirmed two ways** (my differential pass + an independent multi-agent equivalence sweep): identical regex, reply strings, branch order, transport, and complete re-exports ? **zero divergences** across a 69-command combined battery.
- **3 non-blocking findings**, all test-quality/hygiene:
  1. ? (low-end medium) service tests don't assert the **\*persisted\*** config field/value ? a wrong `field_map` would pass all 20, caught only incidentally by the unchanged `test_commands.py`.
  2. ? redundant `load_filter_config` reads per self-message (+ double-read on `/show config`) ? no behavioral impact.
  3. ? unreachable `except ValueError` branches + a misnamed test ? both pre-existing patterns carried over.

That's Track D reviewed. This one needed no fix round from the author ? it's ready to merge as-is, with the persistence-assertion suggestion as an easy follow-up. Ping me when Track E (#30) is up, or if the author pushes changes here and you want a re-verify.

#### 240. user (2026-07-06T09:02:44.430Z)

pls review #39 and also post the comment on the PR.

#### 241. assistant (2026-07-06T09:02:51.009Z)

`/plugin` isn't available in this environment (as noted). Moving on to the review ? let me sync and pull up PR #39.

#### 242. user (2026-07-06T09:02:57.442Z)

---checkout---  
feat/track-e-selective-scan  
**feat/track-e-selective-scan...origin/feat/track-e-selective-scan**

=== PR #39 META ===  
{  
 "additions": 254,  
 "author": {  
 "id": "MDQ6VXNlcjE2NzQ4Nzc=",  
 "is\_bot": false,  
 "login": "avidullu",  
 "name": "Avi"  
 },  
 "baseRefName": "main",  
 "body": "## What\n\nImplements **Track E (#30)** ? selective source scanning with `/watch`, `/unwatch`, `/channels`, `/watchlist` commands and ingestion gating. Users can now opt in to which channels are scanned, making the system manageable (instead of blindly processing every visible channel).\n\n## Commands (command\_service.py)\n\n| Command | Behavior |\n|---|---|\n| `/channels` | List all known sources with ? (watched) / ? (unwatched) indicators |\n| `/watchlist` | List only watched sources |\n| `/watch <@username\\|channel_id>` | Enable scanning for a source |\n| `/unwatch <@username\\|channel_id>` | Disable scanning for a source |\n\n`resolve\_source()` accepts both numeric channel ids (-1001234567890) and

```
`@username`s ? strips the `@` prefix before lookup.\n\n## DB (db.py)\n\n| Function | Purpose
|---|---|\n| `set_source_watched(account_id, source_id, watched)` | Toggle the flag |\n| `list_account_sources(account_id, watched_only=False)` | Enumerate sources, filterable |\n\n## Ingestion gate (telethon_client.py)\n\nAfter extracting `channel_id`, the handler now looks up the source and skips if `watched=False` ? before the three-tier dedup pipeline, saving CPU/DB work.\n\nBackwards-compatible: legacy channels without a `sources` row (possible in edge cases) pass through ? the existing `upsert_channel` call creates them with `watched=True`, matching the ADR 0009 backfill.\n\n## Tests ? 10 new\n\n| Test | What it covers |\n|---|---|\n| `/channels` with mixed watch status | Indicators, both channels listed |\n| `/channels` empty | Help message |\n| `/watchlist` filtered | Only watched shown |\n| `/watchlist` empty | Help message |\n| `/watch` by numeric id | toggles `watched=True` |\n| `/watch` by @username | strips `@` prefix |\n| `/unwatch` | toggles `watched=False` |\n| Unknown source `/watch` | Error message |\n| `set_source_watched` round-trip | Persistence verified |\n| `list_account_sources` filter | `watched_only=True` counts |\n\n## Verification\n\n`n492` passed @ 97.10% (floor 80%)\n\nruff check . ? clean\n\nruff format ? clean\n\n## Scope discipline\n\n- ? Commands: `/watch`, `/unwatch`, `/channels`, `/watchlist` ? pure parsing in `CommandService`, transport in Telethon adapter.\n\n- ? Gate: telethon_client.py skips unwatched sources before dedup/filter/delivery.\n\n- ? DB: `set_source_watched` + `list_account_sources` with filtering.\n\n- ? Backwards-compatible: unwatched sources without a `sources` row pass through.\n\n- ? Not in this track: destination-channel ingestion guard (already covered by watched=0 default), client.get_dialogs() enumeration (Telethon-specific, tracked in a follow-up).\n\n## Closes\n\nCloses #30 (Track E).\n\n## Related\n\n- #25 ? umbrella issue\n\n- #35 ? Track B (landed the `sources.watched` column + backfill)\n\n- ADR 0009 ? selective channel scanning and multiplexed routing", "changedFiles": 5, "deletions": 3, "headRefName": "feat/track-e-selective-scan", "isDraft": false, "mergeStateStatus": "CLEAN", "mergeable": "MERGEABLE", "number": 39, "state": "OPEN", "statusCheckRollup": [], "title": "feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR 0009, Track E)"}]
```

#### 243. assistant (2026-07-06T09:03:19.896Z)

PR #39 is **Track E** ? selective scanning (`/watch`, `/unwatch`, `/channels`, `/watchlist`) + an ingestion gate. +254/?3, 5 files. Things I want to scrutinize: the **ingestion gate ordering** / backwards-compat (do brand-new channels get gated out?), `\_resolve\_source` **@username** handling (sources store numeric ids), and whether the new tests assert persistence. Let me start the gate and read the diff.

#### 244. user (2026-07-06T09:03:29.295Z)

```
gate started (bg pid 626)
=== commits vs main + freshness ===
db87cc0 feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR 0009, Track E)
(second empty=branch current)
=== files ===
src/tg_video_filter/command_service.py
src/tg_video_filter/commands.py
src/tg_video_filter/db.py
src/tg_video_filter/telethon_client.py
tests/test_command_service.py
351 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr39.diff
```

#### 245. assistant (2026-07-06T09:03:33.334Z)

Branch is current with main (one commit `db87cc0`). Let me read the full diff.

#### 246. user (2026-07-06T09:03:34.348Z)

```
1 diff --git a/src/tg_video_filter/command_service.py
b/src/tg_video_filter/command_service.py
2 index 665c0cb..444ac71 100644
3 --- a/src/tg_video_filter/command_service.py
4 +++ b/src/tg_video_filter/command_service.py
5 @@ -18,7 +18,7 @@
```



```

6     import aiosqlite
7
8     from tg_video_filter import db
9     -from tg_video_filter.models import FilterConfig
10    +from tg_video_filter.models import FilterConfig, Source
11
12    log = logging.getLogger(__name__)
13
14    @@ -38,6 +38,13 @@
15    _MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
16
17
18    +# --- source scanning regex (ADR 0009, Track E) -----
19    +
20    +_WATCH_RE = re.compile(r"^/watch\s+(.+)$", re.IGNORECASE)
21    +_UNWATCH_RE = re.compile(r"^/unwatch\s+(.+)$", re.IGNORECASE)
22    +_CHANNELS_RE = re.compile(r"^/channels$", re.IGNORECASE)
23    +_WATCHLIST_RE = re.compile(r"^/watchlist$", re.IGNORECASE)
24    +
25    # --- unit parsing -----
26
27
28    @@ -306,6 +313,22 @@ async def handle(
29        if _SHOW_CONFIG_RE.match(text):
30            return CommandResult(format_config(current_cfg))
31
32    +    # --- source scanning commands (ADR 0009, Track E) -----
33    +
34    +    m = _WATCH_RE.match(text)
35    +    if m:
36    +        return await self._handle_watch(conn, user_id, m.group(1).strip(),
current_cfg)
37    +
38    +    m = _UNWATCH_RE.match(text)
39    +    if m:
40    +        return await self._handle_unwatch(conn, user_id, m.group(1).strip(),
current_cfg)
41    +
42    +    if _CHANNELS_RE.match(text):
43    +        return await self._handle_channels(conn, user_id)
44    +
45    +    if _WATCHLIST_RE.match(text):
46    +        return await self._handle_watchlist(conn, user_id)
47    +
48    +    # --- /show commands -----
49    +    if _SHOW_COMMANDS_RE.match(text):
50    +        return CommandResult(format_commands())
51    @@ -313,6 +336,85 @@ async def handle(
52        # Not a recognised command ? return empty result (noise).
53        return CommandResult()
54
55    +    # -- source scanning handlers (ADR 0009, Track E) -----
56    +
57    +    async def _handle_watch(
58    +        self, conn: aiosqlite.Connection, user_id: int, target: str, _cfg: FilterConfig
59    +    ) -> CommandResult:
60    +        """Parse ``/watch <@username|channel_id>`` and toggle ``watched=1``."""
61    +        src = await self._resolve_source(conn, user_id, target)
62    +        if src is None:
63    +            return CommandResult(
64    +                f"? Source `{target}` not found. Use `/channels` to list "
65    +                f"available sources.\n\nSources are discovered as messages "
66    +                f"arrive; a channel you haven't received traffic from may not "
67    +                f"appear yet."
68    +            )

```

```

69      +      await db.set_source_watched(conn, user_id, src.id, True) # type: ignore[arg-
type]
70      +      label = src.title or src.provider_source_id
71      +      return CommandResult(f"? Now watching **{label}**
({src.provider_source_id}).")
72      +
73      +      async def _handle_unwatch(
74      +          self, conn: aiosqlite.Connection, user_id: int, target: str, _cfg: FilterConfig
75      +      ) -> CommandResult:
76      +          """Parse `/unwatch <username|channel_id>` and toggle `watched=0`."""
77      +          src = await self._resolve_source(conn, user_id, target)
78      +          if src is None:
79      +              return CommandResult(
80      +                  f"? Source `{target}` not found. Use `/channels` to list sources."
81      +              )
82      +          await db.set_source_watched(conn, user_id, src.id, False) # type: ignore[arg-
type]
83      +          label = src.title or src.provider_source_id
84      +          return CommandResult(f"? Stopped watching **{label}**
({src.provider_source_id}).")
85      +
86      +      async def _handle_channels(self, conn: aiosqlite.Connection, user_id: int) ->
CommandResult:
87      +          """List all known sources with watch status."""
88      +          sources = await db.list_account_sources(conn, user_id)
89      +          if not sources:
90      +              return CommandResult(
91      +                  "? **No sources yet.**\n\n"
92      +                  "Sources are discovered when a new message arrives from a "
93      +                  "channel. Run `/backfill` to discover historical channels, "
94      +                  "or wait for new traffic."
95      +              )
96      +          lines = ["? **All sources**", ""]
97      +          for s in sources:
98      +              icon = "?" if s.watched else ""
99      +              label = s.title or s.provider_source_id
100      +              lines.append(f"{icon} `{s.provider_source_id}` ? {label}")
101      +          return CommandResult("\n".join(lines))
102      +
103      +      async def _handle_watchlist(self, conn: aiosqlite.Connection, user_id: int) ->
CommandResult:
104      +          """List only watched sources."""
105      +          sources = await db.list_account_sources(conn, user_id, watched_only=True)
106      +          if not sources:
107      +              return CommandResult(
108      +                  "? **No watched sources.**\n\n"
109      +                  "Use `/channels` to see available sources, then `/watch <id>` "
110      +                  "to start scanning."
111      +              )
112      +          lines = ["? **Watched sources**", ""]
113      +          for s in sources:
114      +              label = s.title or s.provider_source_id
115      +              lines.append(f"? `{s.provider_source_id}` ? {label}")
116      +          return CommandResult("\n".join(lines))
117      +
118      +      async def _resolve_source(
119      +          self, conn: aiosqlite.Connection, user_id: int, target: str
120      +      ) -> Source | None:
121      +          """Resolve a user-provided source identifier to a `Source` row.
122      +
123      +          Accepts `@username` or a numeric channel id string. The `@` prefix
124      +          is stripped before lookup since Telegram channel ids are numeric.
125      +
126      +          target = target.lstrip("@")
127      +          # Try by provider-native id first (the common case).

```

```

128 +         src = await db.get_source_by_provider_id(conn, user_id, "telegram", target)
129 +         if src is not None:
130 +             return src
131 +         # Fall back: try with @ prefix in the target column (just in case).
132 +         return await db.get_source_by_provider_id(conn, user_id, "telegram",
f"@{target}")
133 +
134 +         # -- internal -----
135
136         async def _update(
137             diff --git a/src/tg_video_filter/commands.py b/src/tg_video_filter/commands.py
138             index 974be99..3aea1f6 100644
139             --- a/src/tg_video_filter/commands.py
140             +++ b/src/tg_video_filter/commands.py
141             @@ -27,7 +27,8 @@
142
143             log = logging.getLogger(__name__)
144
145             -from tg_video_filter.command_service import ( # noqa: E402
146             +from tg_video_filter.command_service import ( # noqa: E402, F401
147             +     _CHANNELS_RE,
148             +     _EXCLUDE_HELP_RE,
149             +     _EXCLUDE_RE,
150             +     _LENGTH_RE,
151             @@ -37,6 +38,9 @@
152             +     _SHOW_COMMANDS_RE,
153             +     _SHOW_CONFIG_RE,
154             +     _SIZE_RE,
155             +     _UNWATCH_RE,
156             +     _WATCH_RE,
157             +     _WATCHLIST_RE,
158             +     _format_duration,
159             +     _format_size,
160             +     format_commands,
161             diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
162             index bfaadec..bee716b 100644
163             --- a/src/tg_video_filter/db.py
164             +++ b/src/tg_video_filter/db.py
165             @@ -1534,6 +1534,32 @@ async def get_source_by_id(conn: aiomysqlite.Connection, source_id:
int) -> Source
166                 return _row_to_source(row) if row else None
167
168
169             +async def set_source_watched(
170             +     conn: aiomysqlite.Connection, account_id: int, source_id: int, watched: bool
171             +) -> None:
172             +     """Toggle the ``watched`` flag on a source (ADR 0009, Track E)."""
173             +     await conn.execute(
174             +         "UPDATE sources SET watched = ? WHERE id = ? AND account_id = ?",
175             +         (int(watched), source_id, account_id),
176             +     )
177             +     await conn.commit()
178             +
179
180             +async def list_account_sources(
181             +     conn: aiomysqlite.Connection, account_id: int, *, watched_only: bool = False
182             +) -> list[Source]:
183             +     """List all sources for an account, optionally filtering to watched only."""
184             +     conn.row_factory = aiomysqlite.Row
185             +     query = "SELECT * FROM sources WHERE account_id = ?"
186             +     params: tuple = (account_id,)
187             +     if watched_only:
188             +         query += " AND watched = 1"
189             +     query += " ORDER BY watched DESC, provider_source_id ASC"
190             +     async with conn.execute(query, params) as cur:

```

```

191     +         rows = await cur.fetchall()
192     +         return [_row_to_source(r) for r in rows]
193     +
194     +
195     async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
196         """Deprecated shim: insert into `content_items` (ADR 0011).
197
198     @@ -1717,6 +1743,8 @@ async def list_videos_for_user( # v2: used by on-demand /search;
not called in
199         "upsert_source",
200         "get_source_by_provider_id",
201         "get_source_by_id",
202     +     "set_source_watched",
203     +     "list_account_sources",
204         "insert_content_item",
205         "get_content_item_by_message",
206         "get_content_item_by_dedup_key",
207     diff --git a/src/tg_video_filter/telethon_client.py
b/src/tg_video_filter/telethon_client.py
208     index 883e3f8..ebb7416 100644
209     --- a/src/tg_video_filter/telethon_client.py
210     +++ b/src/tg_video_filter/telethon_client.py
211     @@ -181,6 +181,15 @@ async def on_new_message(event: NewMessage.Event) -> None: #
pragma: no cover
212         message_id = message.id
213         document_id = _extract_document_id(message)
214
215     +     # --- watched gate (ADR 0009, Track E) -----
216     +     # Skip ingestion for unwatched sources. The channel must already be in
217     +     # sources (lazy-created by the /channels enumeration or the legacy
218     +     # upsert_channel below); if absent, we let the legacy path upsert it
219     +     # with watched=True for backwards-compatibility.
220     +     src = await db.get_source_by_provider_id(conn, user.id, "telegram",
str(channel_id))
221     +     if src is not None and not src.watched:
222     +         return # silently skip unwatched source
223     +
224     +     # --- three-tier dedup (cheapest first) ---
225
226     +     # Tier 0: same (user, channel, message) instance ? already processed.
227     diff --git a/tests/test_command_service.py b/tests/test_command_service.py
228     index 0201053..c4d8984 100644
229     --- a/tests/test_command_service.py
230     +++ b/tests/test_command_service.py
231     @@ -17,7 +17,7 @@
232         parse_duration_s,
233         parse_size_mb,
234     )
235     -from tg_video_filter.models import FilterConfig
236     +from tg_video_filter.models import FilterConfig, Source
237
238     USER_ID = 111
239
240     @@ -234,3 +234,111 @@ def test_format_commands_includes_all_sections() -> None:
241         assert "/set size" in text
242         assert "/set exclude" in text
243         assert "/backfill" in text
244     +
245     +
246     + # --- /channels + /watch + /unwatch + /watchlist (ADR 0009, Track E) -----
247     +
248     +
249     + async def _seed_source(
250     +     conn: aiomysqlite.Connection,
251     +     account_id: int,

```

```

252 +     provider_source_id: str,
253 +     title: str = "",
254 +     watched: bool = False,
255 +) -> Source:
256 +     return await db.upsert_source(
257 +         conn,
258 +         Source(
259 +             account_id=account_id,
260 +             provider="telegram",
261 +             provider_source_id=provider_source_id,
262 +             title=title,
263 +             kind="channel",
264 +             watched=watched,
265 +         ),
266 +     )
267 +
268 +
269 +async def test_channels_lists_all_sources_with_watch_status(
270 +     conn: aiomysqlite.Connection, svc: CommandService
271 +) -> None:
272 +     await _seed_source(conn, USER_ID, "-1001", "Channel A", watched=True)
273 +     await _seed_source(conn, USER_ID, "-1002", "Channel B", watched=False)
274 +     result = await svc.handle(USER_ID, "/channels", conn, FilterConfig())
275 +     assert "Channel A" in result.reply_text
276 +     assert "Channel B" in result.reply_text
277 +     assert "?" in result.reply_text # watched indicator
278 +     assert "?" in result.reply_text # unwatched indicator
279 +
280 +
281 +async def test_channels_empty_shows_help(conn: aiomysqlite.Connection, svc:
CommandService) -> None:
282 +     result = await svc.handle(USER_ID, "/channels", conn, FilterConfig())
283 +     assert "No sources yet" in result.reply_text
284 +
285 +
286 +async def test_watchlist_only_watched(conn: aiomysqlite.Connection, svc: CommandService)
-> None:
287 +     await _seed_source(conn, USER_ID, "-1001", "Ch A", watched=True)
288 +     await _seed_source(conn, USER_ID, "-1002", "Ch B", watched=False)
289 +     result = await svc.handle(USER_ID, "/watchlist", conn, FilterConfig())
290 +     assert "Ch A" in result.reply_text
291 +     assert "Ch B" not in result.reply_text
292 +
293 +
294 +async def test_watchlist_empty_shows_help(conn: aiomysqlite.Connection, svc:
CommandService) -> None:
295 +     result = await svc.handle(USER_ID, "/watchlist", conn, FilterConfig())
296 +     assert "No watched sources" in result.reply_text
297 +
298 +
299 +async def test_watch_by_numeric_id(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
300 +     await _seed_source(conn, USER_ID, "-1001", "Target", watched=False)
301 +     result = await svc.handle(USER_ID, "/watch -1001", conn, FilterConfig())
302 +     assert "Now watching" in result.reply_text
303 +     src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1001")
304 +     assert src is not None
305 +     assert src.watched is True
306 +
307 +
308 +async def test_watch_by_username(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
309 +     await _seed_source(conn, USER_ID, "@mychan", "My Channel", watched=False)
310 +     result = await svc.handle(USER_ID, "/watch @mychan", conn, FilterConfig())
311 +     assert "Now watching" in result.reply_text

```

```

312 +     src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "@mychan")
313 +     assert src is not None
314 +     assert src.watched is True
315 +
316 +
317 +async def test_unwatch(conn: aiomysqlite.Connection, svc: CommandService) -> None:
318 +     await _seed_source(conn, USER_ID, "-1001", "Target", watched=True)
319 +     result = await svc.handle(USER_ID, "/unwatch -1001", conn, FilterConfig())
320 +     assert "Stopped watching" in result.reply_text
321 +     src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1001")
322 +     assert src is not None
323 +     assert src.watched is False
324 +
325 +
326 +async def test_watch_unknown_source_shows_error(
327 +     conn: aiomysqlite.Connection, svc: CommandService
328 + ) -> None:
329 +     result = await svc.handle(USER_ID, "/watch -9999", conn, FilterConfig())
330 +     assert "not found" in result.reply_text
331 +
332 +
333 +async def test_set_source_watched_toggle(conn: aiomysqlite.Connection) -> None:
334 +     src = await _seed_source(conn, USER_ID, "-2000", watched=True)
335 +     assert src.id is not None
336 +     await db.set_source_watched(conn, USER_ID, src.id, False)
337 +     reloaded = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-2000")
338 +     assert reloaded is not None
339 +     assert reloaded.watched is False
340 +
341 +
342 +async def test_list_account_sources_watched_only(
343 +     conn: aiomysqlite.Connection,
344 + ) -> None:
345 +     await _seed_source(conn, USER_ID, "-10", watched=True)
346 +     await _seed_source(conn, USER_ID, "-20", watched=False)
347 +     all_srcs = await db.list_account_sources(conn, USER_ID)
348 +     assert len(all_srcs) == 2
349 +     watched = await db.list_account_sources(conn, USER_ID, watched_only=True)
350 +     assert len(watched) == 1
351 +     assert watched[0].provider_source_id == "-10"
352

```

#### 247. assistant (2026-07-06T09:04:51.689Z)

I've read the full diff. Several things to verify ? most notably: `**`/watch @username` looks non-functional for real sources**` (sources store numeric ``provider_source_id``, and ``_resolve_source`` only looks up by that), the `**ingestion gate is inside a `# pragma: no cover` handler**` (untested), and `**`format_commands()` may not list the new commands**`. Let me verify these and check ADR 0009's intended default.

#### 248. user (2026-07-06T09:04:59.279Z)

```

=== does format_commands() list the new Track E commands? ===
43:_WATCH_RE = re.compile(r"^/watch\s+(.+)$", re.IGNORECASE)
44:_UNWATCH_RE = re.compile(r"^/unwatch\s+(.+)$", re.IGNORECASE)
45:_CHANNELS_RE = re.compile(r"^/channels$", re.IGNORECASE)
46:_WATCHLIST_RE = re.compile(r"^/watchlist$", re.IGNORECASE)
344:     """Parse ``/watch <@username|channel_id>`` and toggle ``watched=1``."""
348:         f"? Source `{target}` not found. Use `/channels` to list "
360:     """Parse ``/unwatch <@username|channel_id>`` and toggle ``watched=0``."""
364:         f"? Source `{target}` not found. Use `/channels` to list sources."
393:         "Use `/channels` to see available sources, then `/watch <id>` "
7
--- format_commands body: any watch? ---
    "/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history (default

```

```

=== is on_new_message (the gate) pragma:no cover? ===
110:) -> TelegramClient: # pragma: no cover
167:    async def on_new_message(event: NewMessage.Event) -> None: # pragma: no cover
355: def _channel_title(event: NewMessage.Event) -> str | None: # pragma: no cover

```

```

TOTAL                                1178      26      304      17      97%
Required test coverage of 80.0% reached. Total coverage: 97.10%
492 passed in 7.60s
exit=0
DONE

```

Confirmed: `**format_commands()` doesn't list the new commands\*\* (undiscoverable via ``/show commands``), and the `**gate` lives in a ``# pragma: no cover`` handler\*\* (untested). Gate: 492 passed @ 97.10%. Now let me verify the ``@username`` resolution issue with a realistic repro and check ADR 0009's intended default.

```

=== ruff (head of gate) ===
=== RUFF CHECK ===
All checks passed!
exit=0
=== RUFF FORMAT ===
36 files already formatted
exit=0
=== PYTEST ===

```

```
=== REPRO: /watch @username on a REALISTIC source (numeric id, username in title) ===  
Exception in thread Thread-1 (_connection_worker_thread):  
Traceback (most recent call last):  
File "C:\Users\avidu\Projects\Telegram\.env\Lib\site-packages\aiosqlite\core.py", line 66, in  
_connection_worker_thread  
    future.get_loop().call_soon_threadsafe(set_result, future, result)  
~~~~~^~~~~~  
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line  
878, in call_soon_threadsafe  
    self._check_closed()  
~~~~~^^  
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line  
556, in _check_closed  
    raise RuntimeError('Event loop is closed')  
RuntimeError: Event loop is closed
```

```
Traceback (most recent call last):  
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\threading.py", line 1044, in  
_bootstrap_inner  
    self.run()  
~~~~~^^  
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\threading.py", line 995, in  
run  
    self._target(*self._args, **self._kwargs)  
~~~~~^AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 75, in  
_connection_worker_thread  
    future.get_loop().call_soon_threadsafe(set_exception, future, e)  
~~~~~^AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
878, in call_soon_threadsafe
    self._check_closed()
    ~~~~~^
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
556, in _check_closed
    raise RuntimeError('Event loop is closed')
RuntimeError: Event loop is closed
/watch @mycoolchannel      -> '? Source `@mycoolchannel` not found. Use `/chann'
/watch -1001234567890     -> '? Now watching **My Cool Channel** (`-1001234567'
/watch My Cool Channel    -> '? Source `My Cool Channel` not found. Use `/chan'

```

```

=== ADR 0009: default watched? opt-in vs opt-out? @username? ===
20: Reggae channel, a Rap channel) that subscribers opt into.
25:1. **Selective scanning** ? choose which source channels are watched, rather
35:### 1. Selective scanning (watched-flag)
37:Add a `watched` flag to the `channels` table. The default is locked as
38:**opt-in: `DEFAULT 0`, so a freshly-onboarded user scans nothing until they
44:row and `return` early if `watched` is false, before any dedup or filter work.
48:**Discoverability note.** With `DEFAULT 0` plus the current lazy population
49:(a `channels` row is only inserted when a message is first seen), opt-in is
54:from. The flag default and the enumeration flow are orthogonal ? both are
63: used in `backfill.py`) with their watched state.
64:- `/watch <#name|id>` ? set `watched=1` for a source channel.
65:- `/unwatch <#name|id>` ? set `watched=0`.
66:- `/watchlist` ? list only watched sources.
77: dest_channel_id    INTEGER NOT NULL,    -- a broadcast channel / @username
103:- New schema (`channels.watched`, `routing_rules`) with migrations.
110: user_id = ?)) before the watched-flag check. Issue #25 Track E implements
119:Adding `channels.watched` with `DEFAULT 0` would silently stop every existing
125:ALTER TABLE channels ADD COLUMN watched INTEGER NOT NULL DEFAULT 0;
126:UPDATE channels SET watched = 1 WHERE seen_first_at IS NOT NULL;
130:watched, so pre-migration users observe no change. New channels discovered
131:after the migration default to `watched = 0` and must be explicitly watched ?
137:1. **Selective scanning** (`channels.watched` + `/watch`, `/unwatch`,
150: watched set is per-user and mutable at runtime; re-registering Telethon

```

## 251. assistant (2026-07-06T09:06:07.003Z)

This is the key finding, and ADR 0009 is explicit. Let me capture it:

```

- **ADR 0009 line 37?38:** "The default is locked as opt-in: `DEFAULT 0`, so a freshly-
onboarded user scans nothing until they [opt in]."
- **ADR 0009 line 130?131:** "New channels discovered after the migration default to `watched =
0` and must be explicitly watched."

```

But the repro shows the opposite, and `@username` is confirmed broken:

```

- `/watch @mycoolchannel` ? **not found**; `/watch -1001234567890` ? works. So `@username` (and
name) resolution is non-functional for real sources.

```

Let me confirm the new-channel default by checking `upsert\_channel`'s `watched` value and the gate?upsert ordering ? the crux of the ADR violation.

## 252. user (2026-07-06T09:06:11.693Z)

```

=== upsert_channel: what watched value does the LIVE path create new sources with? ===
16:         watched=True, # back-compat: pre-ADR-0011 channels were all watched

```

```

=== on_new_message ordering: gate (get_source) vs upsert_channel ===
184:         # --- watched gate (ADR 0009, Track E) -----
187:         # upsert_channel below); if absent, we let the legacy path upsert it
189:         src = await db.get_source_by_provider_id(conn, user.id, "telegram", str(channel_id))
191:         return # silently skip unwatched source
193:         # --- three-tier dedup (cheapest first) ---
249:         await db.upsert_channel(

```



=== ADR 0009 lines 44-54 (the intended lazy-create + skip flow) ===  
row and `return` early if `watched` is false, before any dedup or filter work.

This is a small migration plus ~5 lines in `telethon\_client.py`.

**\*\*Discoverability note.\*\*** With `DEFAULT 0` plus the current lazy population (a `channels` row is only inserted when a message is first seen), opt-in is necessary but not sufficient for discoverability: a channel the account has never received a message from has no row to flag. The `/channels` command (step 2 below) therefore upserts candidate rows from `client.get\_dialogs()` so users can browse and watch channels they have not yet received traffic from. The flag default and the enumeration flow are orthogonal ? both are

#### 253. assistant (2026-07-06T09:06:35.506Z)

**\*\*HIGH finding confirmed and ADR-backed.\*\*** The gate only skips when a source row *\*exists\** and is unwatched; a *\*new\** channel (no row) falls through ? gets scanned/delivered ? then `upsert\_channel` creates it `watched=True`. So new channels are *\*auto-watched (opt-out)\** ? directly inverting ADR 0009's locked *\*"opt-in: DEFAULT 0?"* new channels default to *watched=0* and must be explicitly watched." The PR's "matching the ADR 0009 backfill" justification conflates the migration (existing?1) with new-channel behavior (should be ?0).

Let me launch a focused workflow to adversarially verify this + the `@username` break and sweep for more.

#### 254. user (2026-07-06T09:07:16.589Z)

Workflow launched in background. Task ID: wdnf4behx

Summary: Review telegram-video-filter PR #39 (Track E: selective scanning) ? ADR 0009 conformance, gate behavior, command correctness, with adversarial verification

Transcript dir: C:\Users\avidu\claudel\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_3512746c-54e

Script file: C:\Users\avidu\claudel\projects\C--Users-avidu-Projects-

Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr39-track-e-wf\_3512746c-54e.js

(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:

"C:\Users\avidu\claudel\projects\C--Users-avidu-Projects-

Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr39-track-e-

wf\_3512746c-54e.js"} to iterate without resending the script.)

Run ID: wf\_3512746c-54e

To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\claudel\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr39-track-e-wf\_3512746c-54e.js", resumeFromRunId: "wf\_3512746c-54e"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

#### 255. assistant (2026-07-06T09:07:32.083Z)

While the workflow runs, let me confirm two supporting points: ADR 0009's status (accepted?) and whether `/channels` enumerates dialogs (ADR 48?54) or only lists already-seen sources.

#### 256. user (2026-07-06T09:07:35.710Z)

=== ADR 0009 status ===

- **\*\*Status\*\***: Accepted  
- **\*\*Date\*\***: 2026-07-04

=== does \_handle\_channels enumerate get\_dialogs, or just list stored sources? ===

372: sources = await db.list\_account\_sources(conn, user\_id)

389: sources = await db.list\_account\_sources(conn, user\_id, watched\_only=True)

(only list\_account\_sources ? /channels shows only already-seen sources; get\_dialogs enumeration deferred per PR body)

## 257. assistant (2026-07-06T09:07:54.393Z)

Confirmed: ADR 0009 is **Accepted** (so the deviation is real), and `/channels` only lists already-seen sources ? the `get_dialogs()` enumeration is explicitly deferred in the PR body (disclosed scope cut, not a hidden defect, but it compounds the discoverability gap).

While the workflow finishes, here's my interim read:

**Gate** (branch `db87cc0`): **?** ruff check + format clean · **?** **492** passed @ 97.10%**?** (floor 80%).

**Findings** (verified, ADR-backed):

- ?** **High** ? new channels auto-watched, inverting ADR 0009's opt-in. The gate only skips when a source row *exists* and is unwatched; a new channel (no row) falls through ? gets scanned/delivered ? `upsert_channel` creates it `watched=True`. ADR 0009 (Accepted) locks `"opt-in: DEFAULT 0 ? new channels default to watched=0"` and `"return early? before any dedup or filter work."` So a "freshly-onboarded user scans nothing until they opt in" is false ? every new channel is auto-scanned and permanently watched.
- ?** **Medium** ? `/watch @username` non-functional for real sources. `_resolve_source` only looks up by numeric `provider_source_id`; the passing test seeds an unrealistic `@username` source. Reproduced: `/watch @chan` and `/watch <name>` ? "not found"; only `/watch -100?` works. ADR line 64 says `<#name|id>`.
- ?** **Medium** ? the ingestion gate is untested (inside `# pragma: no cover on_new_message`) ? exactly where the auto-watch bug lives.
- ?** **Low** ? `format_commands()` doesn't list the new commands (undiscoverable via `/show commands`).

I'll fold in the workflow's adversarial verification and give you the consolidated review to post.

## 258. user (2026-07-06T09:16:06.667Z)

```
<task-notification>
<task-id>wdnf4behx</task-id>
<tool-use-id>toolu_01Je6u3WMPy4HXNfkjsMNWoy</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\wdnf4behx.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Review telegram-video-filter PR #39 (Track E: selective scanning) ? ADR 0009 conformance, gate behavior, command correctness, with adversarial verification" completed</summary>
<result>{"counts":{"raw":11,"kept":11,"refuted":0},"findings":[{"dimension":"adr_gate","severity":"blocker","verdict":"CONFIRMED","title":"Ingestion gate auto-watches brand-new channels (opt-OUT), inverting ADR 0009's locked opt-in default","where":"src/tg_video_filter/telethon_client.py:190","summary":"The watched gate only returns early when a source row already exists AND is unwatched (`src` is not None and not `src.watched`). A brand-new channel has no source row, so `src` is None, the gate falls through, and the message is deduped, filtered, and delivered. The channel is then lazily recorded via `upsert_channel`, which hardcodes `watched=True` (db.py:1402). Every subsequently seen channel is thus auto-scanned and auto-watched ? the exact opt-out behavior ADR 0009 locks against."},"failure_scenario":"A freshly-onboarded user with zero watched sources receives a video from any channel/group the account is a member of. Trace: on_new_message -> channel_id computed -> `get_source_by_provider_id` returns None -> gate condition `None` is not None and ... is False so no early return -> dedup passes -> `engine.evaluate` runs -> `insert_video` -> the match is DELIVERED to the user's delivery_target. Then `upsert_channel` inserts the source with watched=True, so all future messages from that channel keep flowing. The user opted into nothing yet receives matches from everything ? the opposite of ADR 0009 line 37-39 ('a freshly-onboarded user scans nothing until they explicitly /watch') and line 130-131 ('New channels discovered after the migration default to watched = 0 and must be explicitly watched'). The first message is also delivered rather than skipped, violating line 44 ('return early ... before any dedup or filter work').","evidence":"Empirically reproduced against an in-memory schema (db._apply_schema): BEFORE first message `get_source_by_provider_id(...)` is None; the gate expression `src` is not None and not `src.watched` evaluates False (channel is scanned+delivered); AFTER the lazy `upsert_channel` the source row has watched=True (auto-watched); the 2nd message's gate still evaluates False (keeps scanning). db.py:1402 shows
```

`watched=True` hardcoded in upsert\_channel; the gate comment at telethon\_client.py:186-188 explicitly acknowledges 'if absent, we let the legacy path upsert it with watched=True for backwards-compatibility' ? but that back-compat carve-out was defined by the ADR only for channels already seen PRE-migration (seen\_first\_at IS NOT NULL, ADR line 126), not for channels first seen AFTER the migration. Applying watched=True to newly discovered channels contradicts the ADR's new-channel contract.", "recommendation": "Make the gate skip unwatched sources by default when no row exists: treat `src is None` as unwatched and return early (create the discoverability row with watched=0 rather than processing). Concretely, change the gate to `if src is None or not src.watched: &lt;lazily upsert the source with watched=0 for /channels discoverability, then&gt; return`. The lazy `upsert\_channel` on the ingestion path must not create rows with watched=True for channels first seen after migration ? only the migration backfill (UPDATE ... WHERE seen\_first\_at IS NOT NULL) may set watched=1. This restores ADR 0009's 'scan nothing until opt-in' guarantee and moves the early return before dedup/filter/delivery.", "verify\_reasoning": "Verified against code and ADR 0009. The gate at telethon\_client.py:190 is `if src is not None and not src.watched: return`. When a channel has no source row (brand-new channel never seen ? `src is None`), the condition is False, so there is NO early return: the message proceeds through Tier-0/1/2 dedup, `engine.evaluate` (line 258), `insert\_video`, and delivery (line 308). Only AFTER processing is the channel lazily recorded via `db.upsert\_channel` (telethon\_client.py:249), which hardcodes `watched=True` (db.py:1402). I confirmed the lookup key and upsert key match (`str(channel\_id)` with provider `\"telegram\"` on both sides ? db.py:1149 vs 1399), so every subsequent message from that channel finds a watched=True row and continues to be scanned. This is the exact opt-OUT behavior ADR 0009 locks against: line 37-39 (\"freshly-onboarded user scans nothing until they explicitly /watch\"), line 130-131 (\"New channels discovered after the migration default to watched = 0 and must be explicitly watched\"), and line 44 (\"return early if watched is false, before any dedup or filter work\" ? here the first message is fully processed and delivered before the row is even created). The ADR's watched=1 back-compat carve-out (line 126: `UPDATE ... WHERE seen\_first\_at IS NOT NULL`) applies only to channels seen PRE-migration; applying watched=True to channels first seen AFTER migration contradicts the accepted spec. Additional compounding evidence: `upsert\_source` uses `ON CONFLICT ... watched=excluded.watched` (db.py:1117), and `insert\_video` also upserts with watched=True (db.py:1578), so the ingestion path actively forces sources watched even though the schema default is correctly opt-in (db.py:99, `watched INTEGER NOT NULL DEFAULT 0`). The `# pragma: no cover` on on\_new\_message only excludes it from coverage; the code runs on the live path. This is a genuine defect shipping in THIS PR's diff (telethon\_client.py gate is new in this PR) and a direct inversion of the accepted ADR's central selective-scanning contract.\"}, {\"dimension\": \"tests\_ux\", \"severity\": \"blocker\", \"verdict\": \"CONFIRMED\", \"title\": \"New channels are auto-watched on first message, violating ADR 0009 opt-in default (DEFAULT 0)\", \"where\": \"src/tg\_video\_filter/telethon\_client.py:189\", \"summary\": \"The watched-gate only skips channels that ALREADY have a source row with watched=false; a brand-new channel (no row) falls through the gate and is then lazily created with watched=True by db.upsert\_channel, so its first message is ingested AND delivered.\"}, {\"failure\_scenario\": \"A freshly-onboarded user has watched nothing. A video arrives from channel -1005 that has never been seen before. At telethon\_client.py:189 get\_source\_by\_provider\_id returns None, so `if src is not None and not src.watched` is False and the gate does NOT return. Ingestion proceeds through dedup, then telethon\_client.py:249 calls db.upsert\_channel(...) which hard-codes watched=True (db.py:1402). The very first message is evaluated, inserted, and delivered to the user's target. Every new channel the account is a member of thus gets scanned and delivered exactly once on first traffic and permanently thereafter -- the exact 'scan everything' behavior ADR 0009 set out to eliminate. This directly contradicts ADR 0009 line 37-38 ('DEFAULT 0, so a freshly-onboarded user scans nothing until they explicitly /watch') and line 130-131 ('New channels discovered after the migration default to watched = 0 and must be explicitly watched').\", \"evidence\": \"Repro confirmed: `db.upsert\_channel(c, Channel(user\_id=111, channel\_id=-1005, title='Brand New'))` then reading the source back yields `watched = True`. The shim at db.py:1402 sets `watched=True` # back-compat: pre-ADR-0011 channels were all watched`. The gate at telethon\_client.py:190 (`if src is not None and not src.watched: return`) has no branch for the src-is-None case, so new sources are never blocked. Comment at telethon\_client.py:186-188 explicitly acknowledges 'if absent, we let the legacy path upsert it with watched=True for backwards-compatibility' -- but that back-compat rule applies only to the migration backfill (UPDATE ... WHERE seen\_first\_at IS NOT NULL, ADR line 126), NOT to channels discovered AFTER the migration, which ADR line 130-131 requires to default to watched=0.\"}, {\"recommendation\": \"The gate must treat an absent source row as unwatched (skip), and lazy population must create rows with watched=0. Change the gate to `if src is None or not src.watched: return` (skipping unseen channels), and make the lazy source-creation on the ingest path create the row w ... (truncated 39886 chars, full result in C:\\Users\\avidu\\AppData\\Local\\Temp\\claude\\C--Users-avidu-Projects-badminton-highlight-

```
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\wdnf4behx.output)</result>
<usage><agent_count>14</agent_count><subagent_tokens>714736</subagent_tokens><tool_uses>161</tool_uses><duration_ms>530082</duration_ms></usage>
</task-notification>
```

## 259. assistant (2026-07-06T09:16:11.481Z)

The workflow **\*\*escalated my HIGH to BLOCKER\*\*** (2 agents independently, both CONFIRMED) and found 11 total. Let me parse the full result.

## 260. user (2026-07-06T09:16:15.904Z)

```
COUNTS: {'raw': 11, 'kept': 11, 'refuted': 0}
```

```
=====
[BLOCKER/CONFIRMED] (adr_gate) Ingestion gate auto-watches brand-new channels (opt-OUT),
inverting ADR 0009's locked opt-in default
  where: src/tg_video_filter/telethon_client.py:190
  rec   : Make the gate skip unwatched sources by default when no row exists: treat `src is
None` as unwatched and return early (create the discoverability row with watched=0 rather than
processing). Concretely
```

```
[BLOCKER/CONFIRMED] (tests_ux) New channels are auto-watched on first message, violating ADR
0009 opt-in default (DEFAULT 0)
  where: src/tg_video_filter/telethon_client.py:189
  rec   : The gate must treat an absent source row as unwatched (skip), and lazy population must
create rows with watched=0. Change the gate to `if src is None or not src.watched: return`
(skippping unseen chann
```

```
[HIGH/CONFIRMED] (commands) /watch and /unwatch by @username or channel name are non-functional
for real Telegram sources
  where: src/tg_video_filter/command_service.py:410
  rec   : Resolve by name/username in addition to numeric id: add a title/username lookup in
_resolve_source (e.g. a case-insensitive match on sources.title and/or a dedicated username
column) so '/watch <name>
```

```
[HIGH/CONFIRMED] (commands) /channels does not enumerate candidates from client.get_dialogs(),
defeating the ADR's discoverability design
  where: src/tg_video_filter/command_service.py:352
  rec   : Thread the Telethon client (or a provider-agnostic 'list candidate sources' callback)
into the /channels path and upsert dialogs as watched=0 candidate rows before listing, per ADR
0009 lines 48-54, s
```

```
[MEDIUM/CONFIRMED] (adr_gate) /channels does not upsert candidates from get_dialogs(), so
untrafficked channels are undiscoverable and unwatchable (ADR 0009 lines 48-54)
  where: src/tg_video_filter/command_service.py:370
  rec   : Have /channels (or a shared helper) enumerate `client.get_dialogs()` and upsert each
candidate as a source with watched=0 before listing, per ADR lines 48-54. Since CommandService
is provider-independ
```

```
[MEDIUM/CONFIRMED] (tests_ux) Ingestion gate has zero test coverage (inlined in a # pragma: no
cover handler)
  where: src/tg_video_filter/telethon_client.py:184
  rec   : Extract the gate into a pure, testable helper, e.g. `async def should_ingest(conn,
account_id, channel_id) -> bool` (or a synchronous predicate over a fetched Source), that
returns False for both unwa
```

```
[MEDIUM/CONFIRMED] (tests_ux) /channels does not enumerate candidate channels from
get_dialogs(), so undiscovered channels can't be opted into
  where: src/tg_video_filter/command_service.py:370
  rec   : Provide the CommandService a way to enumerate dialogs (pass the Telethon client or a
dialog-lister callback into /channels), and upsert each dialog as a watched=0 candidate source
before listing, so m
```

```
[MEDIUM/CONFIRMED] (tests_ux) /watch cannot resolve a channel by name/username for real sources
(only numeric id works)
```

```
where: src/tg_video_filter/command_service.py:402
rec : Resolve by title/username in addition to numeric id: query sources by title (case-
insensitive) and, if usernames are to be supported, persist the channel username on the source
row so it can be matched
```

[LOW/CONFIRMED] (adr\_gate) /watch cannot resolve a source by channel name/username ? only numeric id (ADR 0009 line 64)

```
where: src/tg_video_filter/command_service.py:402
rec : Extend _resolve_source to also match by title/username (e.g. a case-insensitive
LIKE/exact match on sources.title, and store/resolve the @username where available) so a user
can /watch by name as the
```

[LOW/CONFIRMED] (commands) lstrip('@') strips multiple leading @ characters, accepting malformed input as valid

```
where: src/tg_video_filter/command_service.py:417
rec : Strip a single leading '@' only, e.g. target = target[1:] if target.startswith('@')
else target, so multi-@ inputs are treated as not-found / invalid rather than silently
normalized.
```

[LOW/CONFIRMED] (tests\_ux) format\_commands() omits /watch, /unwatch, /channels, /watchlist -- undiscoverable via /show commands

```
where: src/tg_video_filter/command_service.py:129
rec : Add a 'Source scanning' section to format_commands() documenting /channels, /watch
<id>, /unwatch <id>, /watchlist, and extend the format_commands test to assert their presence.
```

--- refuted: []

## 261. assistant (2026-07-06T09:17:05.505Z)

The workflow **confirmed** all 11 findings (0 refuted) ? two independent agents escalated the auto-watch issue to **blocker** with their own repros, and confirmed the '@username', 'get\_dialogs', untested-gate, and 'format\_commands' findings. Deduplicated, that's one must-fix + a few mediums/low.

Let me write the review and post it (you pre-authorized posting).

## 262. assistant (2026-07-06T09:17:36.808Z)

### Code review ? selective source scanning (ADR 0009, Track E)

**Verdict:** ? Request changes. The commands and DB layer are clean and well-tested, but the **ingestion gate** inverts ADR 0009's central, explicitly-locked opt-in contract ? new channels are auto-watched, so the feature doesn't actually gate anything for a fresh user. Every finding was reproduced locally against 'db87cc0' (posting as a comment ? GitHub blocks the author from a formal request-changes).

#### Gate

```
| Check | Result |
|---|---|
| `pytest` | ? 492 passed, coverage 97.10% (floor 80%) |
| `ruff check .` / `ruff format --check .` | ? clean |
```

Green ? but the suite passing is misleading here: the core gate logic is uncovered (see #4), and the '@username' test passes against unrealistic data (#2).

---

### ? Blocker ? new channels are auto-watched, inverting ADR 0009 opt-in ? 'telethon\_client.py:189-190' + 'db.up

The gate returns early only when a source row **already exists** and is unwatched:

```
```python
src = await db.get_source_by_provider_id(conn, user.id, "telegram", str(channel_id))
if src is not None and not src.watched:
```

```
    return
...

```

A **brand-new** channel has no row ? ``src` is None` ? the gate falls through ? the message is deduped, filtered, **and delivered** ? then ``upsert_channel`` lazily creates the source with ``watched=True`` (hardcoded). So every new channel is auto-scanned and permanently watched from its first message.

This directly contradicts **ADR 0009 (Accepted)**:

- line 37-38: **"The default is locked as ``opt-in: `DEFAULT 0``, so a freshly-onboarded user scans nothing until they explicitly `/watch``."**
- line 130-131: **"New channels discovered after the migration default to ``watched = 0`` and must be explicitly watched."**
- line 44: **"return early if watched is false, **before** any dedup or filter work."** (here the first message is fully processed and delivered before the row even exists.)

**Failure scenario:** a freshly-onboarded user who has watched nothing receives a video from any channel they're a member of ? it's evaluated and **delivered** to their target, and the channel is now permanently watched. The PR's stated goal ("make the system manageable instead of blindly processing every visible channel") isn't achieved for new channels ? it's opt-out.

The PR frames ``watched=True`` as "matching the ADR 0009 backfill," but the ADR's ``watched=1`` back-compat applies **only to channels seen pre-migration** (``UPDATE ? WHERE seen_first_at IS NOT NULL``, line 126) ? not to channels first seen **after** the migration.

**Fix:** treat an absent row as unwatched and skip before dedup/filter, lazily creating the row ``watched=0`` for ``/channels`` discoverability:

```
```python
if src is None or not src.watched:
    # lazily upsert the source with watched=0 (discoverability), then:
    return
...

```

And the ingest-path lazy creation (``upsert_channel``, and the ``insert_video`` upsert at ``db.py:~1578``) must not force ``watched=True`` for newly-seen channels ? only the migration backfill may set ``watched=1``.

---

## ? Medium

**2.** ``/watch`/`/unwatch`` by ``@username`` or channel name are non-functional for real sources ? ``command_service.py:_resolve_source`` ``_resolve_source`` only looks up by ``provider_source_id``, which for real Telegram sources is always the **numeric** channel id (``str(channel_id)``). So ``/watch @somechannel`` (and by name) returns **"not found"** even for a known channel ? only the numeric id works. Reproduced against a realistic source (numeric id, name in ``title``): ``@name`` and ``<name>`` both fail, ``-100?`` works. The passing ``test_watch_by_username`` seeds an **unrealistic** source (``provider_source_id="@mychan"``), giving false confidence. ADR line 64 specifies ``/watch <#name|id>`` (name **or** id). \*(Impact is bounded because ``/channels`` prints the numeric id users copy ? but half the advertised interface is dead and mis-tested.)\*  
> **Fix:** resolve by ``title`/username` (case-insensitive) in addition to numeric id; fix the test to seed a realistic source.

**3.** Ingestion gate has zero test coverage ? ``telethon_client.py:184`` (inside a ``# pragma: no cover`` handler)  
The core gating behavior (skip unwatched, handle new) is untested ? a test would have caught #1. Extract a pure helper, e.g. ``async def should_ingest(conn, account_id, channel_id) -> bool``, and unit-test the watched / unwatched / **absent** cases.

---

## ? Low

**4.** ``format_commands()`` omits the new commands ? ``/watch``, ``/unwatch``, ``/channels``, ``/watchlist`` are undiscoverable via ``/show commands`` (``command_service.py:129``). Add a "Source

```
scanning" section + extend `test_format_commands_includes_all_sections`.
- **5. `rstrip("@")` strips *multiple* leading `@`** (`command_service.py:~417`) ? `@@foo`
normalizes to `foo`. Use `target[1:]` if `target.startswith("@")` else `target`.
```

## Disclosed scope cut (noting because it compounds #1)

`/channels` lists only already-seen sources; it does **not** enumerate `client.get\_dialogs()` to surface untrafficked channels (ADR lines 48-54). The PR body discloses this as deferred ? fine as scope ? but combined with #1 the intended opt-in discoverability flow (lazy row `watched=0` ? appears in `/channels` ? user opts in) doesn't exist yet in either half.

## ? Verified good

- `set\_source\_watched` / `list\_account\_sources` are clean and **tested with persistence assertions** (nice ? addresses the Track D feedback); command handlers well-structured; `/channels` / `/watchlist` / error paths covered.
- Gate is correctly placed **before** dedup (efficient) ? the placement is right, only the `src` is None` default is wrong.

---

**Bottom line:** the blocker is a small, surgical fix (gate on `src` is None or not `src.watched`, lazy-create `watched=0`) but it's essential ? without it, Track E doesn't actually make scanning selective for new channels. #2/#3 are worth doing in the same pass since they're the exact places a test would have caught #1.

<sub>? Reviewed with [Claude Code](https://claude.com/claude-code). Findings reproduced locally against `feat/track-e-selective-scan`; ADR 0009 conformance checked line-by-line. Multi-agent adversarial pass: 11 findings, 0 refuted (2 agents independently rated the gate issue a blocker).</sub>

### 263. user (2026-07-06T09:17:36.838Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr39\_review.md (file state is current in your context ? no need to Read it back)

### 264. user (2026-07-06T09:17:44.433Z)

avidullu [COMMENTED] 2026-07-06T09:17:43Z  
<https://github.com/avidullu/telegram-video-filter/pull/39#pullrequestreview-4634446993>

### 265. assistant (2026-07-06T09:17:55.823Z)

? **Posted to PR #39** ? [pullrequestreview-4634446993](https://github.com/avidullu/telegram-video-filter/pull/39#pullrequestreview-4634446993)

## Summary

**Verdict:** ? Request changes ? one must-fix that inverts the feature's core contract, plus a few mediums/low.

**Gate:** ? 492 passed @ 97.10%, ruff clean ? but green is misleading here (the gate logic is uncovered, and the `@username` test passes on unrealistic data).

**? Blocker** ? new channels are auto-watched, inverting ADR 0009's locked opt-in. The gate only skips when a source row **already exists** and is unwatched; a brand-new channel (`src` is None) falls through ? gets scanned **and delivered** ? then `upsert\_channel` creates it `watched=True`. ADR 0009 (Accepted) explicitly locks **"opt-in: DEFAULT 0 ? new channels default to watched=0"** and **"return early? before any dedup or filter work."** So a freshly-onboarded user who opted into nothing receives matches from everything. Reproduced by me and two independent workflow agents. Fix is small: `if src is None or not src.watched: return`, lazy-create `watched=0`.

**? Medium:** `/watch @username`/name is non-functional for real sources (only numeric id works;

the test seeds unrealistic data); the ingestion gate has zero test coverage (extract a testable helper).

**\*\*? Low:\*\*** `format_commands()` omits the new commands; `rstrip("@")` strips multiple `@`. Plus a disclosed scope cut (`/channels` doesn't enumerate `get_dialogs()`) that compounds the blocker.

**\*\*? Good:\*\*** the DB functions and command handlers are clean and tested *\*with persistence assertions\** (they carried forward the Track D feedback), and the gate placement (before dedup) is correct ? only the `src is None` default is wrong.

This is the first PR in the series with a genuine must-fix. When the author pushes a fix, ping me for the re-verify pass (like #35/#37) ? I'll confirm the gate skips-and-defaults-to-0 for new channels and that a freshly-onboarded user scans nothing until `/watch`.

#### 266. user (2026-07-06T09:19:56.981Z)

please do another review round in 10 minutes

#### 267. assistant (2026-07-06T09:20:33.136Z)

Got it ? I'll re-review PR #39 in 10 minutes to check whether a fix has landed. Let me set a timer; when it fires I'll fetch the branch, look for a fix commit/response, and re-verify (especially the blocker ? that new channels now skip and default to `watched=0`).

#### 268. user (2026-07-06T09:20:34.211Z)

Command running in background with ID: b3obgueng. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b3obgueng.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 269. assistant (2026-07-06T09:20:40.220Z)

Timer set (10 min, task `b3obgueng`). I'll be re-invoked when it elapses and will run the re-review round on PR #39 then ? syncing the branch, checking for a fix commit / response comment, and re-verifying the blocker (new channels skip ingestion and default to `watched=0`) plus the `@username`, gate-coverage, and `format_commands` items. Sit tight; nothing needed from you until then.

#### 270. user (2026-07-06T09:30:35.007Z)

```
<task-notification>
<task-id>b3obgueng</task-id>
<tool-use-id>toolu_015vgnj5UqGH2n8LcCm76F1m</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b3obgueng.output</output-file>
<status>completed</status>
<summary>Background command "10-minute timer before PR #39 re-review round" completed (exit code 0)</summary>
</task-notification>
```

#### 271. assistant (2026-07-06T09:30:39.426Z)

The timer elapsed ? starting the re-review round. Let me sync and check PR #39 for a fix commit or response.

#### 272. user (2026-07-06T09:30:48.096Z)

---checkout---  
main

main...origin/main

=== PR #39 branch commits since my review (after db87cc0) ===



```
ceabe7f fix(scanning): address PR #39 review ? gate opt-in contract, title lookup, helper
extraction
(empty = no new commits)
```

```
=== branch HEAD + state ===
```

```
ceabe7f fix(scanning): address PR #39 review ? gate opt-in contract, title lookup, helper
extraction
db87cc0 feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR
0009, Track E)
35f4f70 feat(commands): extract CommandService ? provider-independent command execution (ADR
0011, Track D) (#38)
```

```
=== response comment since my review? ===
```

```
---PR state---
```

```
MERGED UNKNOWN UNKNOWN
```

### 273. assistant (2026-07-06T09:31:01.843Z)

PR #39 is now **MERGED**, and the branch got a fix commit `ceabe7f` ? "gate opt-in contract, title lookup, helper extraction." So the author fixed it and merged. Per the gate, I still verify the fix landed correctly (it's in `main` now). Let me confirm main includes it and read the fix diff.

### 274. user (2026-07-06T09:31:06.513Z)

```
=== origin/main recent log (does it include the Track E fix?) ===
```

```
3bfb1da feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR
0009, Track E) (#39)
35f4f70 feat(commands): extract CommandService ? provider-independent command execution (ADR
0011, Track D) (#38)
12d21b9 feat(delivery): destinations, routing_rules, delivery_attempts (ADR 0011, Track C) (#37)
d806b46 Merge pull request #36 from avidullu/chore/format-commands
```

```
=== is ceabe7f's content in main? (check gate code) ===
```

```
167:   async def on_new_message(event: NewMessage.Event) -> None:   # pragma: no cover
184:       # --- watched gate (ADR 0009, Track E) -----
187:       # unwatched and lazily create a watched=0 row for /channels
189:       if not await db.should_ingest_source(conn, user.id, channel_id):
```

```
=== FIX DIFF (db87cc0..ceabe7f) stat ===
```

```
src/tg_video_filter/command_service.py | 27 ++++++-----
src/tg_video_filter/db.py               | 34 ++++++-----
src/tg_video_filter/telethon_client.py | 13 +++++-----
tests/test_command_service.py          | 42 ++++++-----
4 files changed, 97 insertions(+), 19 deletions(-)
```

### 275. assistant (2026-07-06T09:31:20.602Z)

Main is squash-merged as `3bfb1da` (#39) and includes the fix: the gate now calls a new extracted helper `db.should\_ingest\_source(...)` and lazily creates a `watched=0` row. Per the gate, I verify it's actually correct. Let me read the full fix diff and start the gate on main.

### 276. user (2026-07-06T09:31:26.590Z)

```
gate started
```

```
=== FULL FIX DIFF (db87cc0..ceabe7f) ===
```

```
diff --git a/src/tg_video_filter/command_service.py b/src/tg_video_filter/command_service.py
index 444ac71..59b7d81 100644
```

```
--- a/src/tg_video_filter/command_service.py
```

```
+++ b/src/tg_video_filter/command_service.py
```

```
@@ -154,6 +154,12 @@ def format_commands() -> str:
```

```
    """backfill [N][hwd] ? Scan last N hours/days/weeks of channel history (default
    90d)",
```

```
    """stop ? Cancel running backfill",
```

```

        "",
+       "? **Source scanning**",
+       "`/channels` ? List all known sources with watch status",
+       "`/watch <id|@name>` ? Start watching a source for new videos",
+       "`/unwatch <id|@name>` ? Stop watching a source",
+       "`/watchlist` ? Show only watched sources",
+       "",
        "\u2139\u2013 **Help**",
        "`/show commands` ? Show this help message",
    ]
@@ -404,16 +410,25 @@ class CommandService:
    ) -> Source | None:
        """Resolve a user-provided source identifier to a ``Source`` row.

-       Accepts ``@username`` or a numeric channel id string. The ``@`` prefix
-       is stripped before lookup since Telegram channel ids are numeric.
+       Accepts numeric channel id, ``@username``, or channel title.
+       ``@`` prefix is stripped before lookup.
        """
-       target = target.lstrip("@")
-       # Try by provider-native id first (the common case).
+       target = target.removeprefix("@")
+       # Try by provider-native numeric id (the common case).
        src = await db.get_source_by_provider_id(conn, user_id, "telegram", target)
        if src is not None:
            return src
-       # Fall back: try with @ prefix in the target column (just in case).
-       return await db.get_source_by_provider_id(conn, user_id, "telegram", f"@{target}")
+       # Try by @username.
+       src = await db.get_source_by_provider_id(conn, user_id, "telegram", f"@{target}")
+       if src is not None:
+           return src
+       # Try by title (case-insensitive substring match).
+       all_srcs = await db.list_account_sources(conn, user_id)
+       target_lower = target.lower()
+       for s in all_srcs:
+           if s.title and target_lower in s.title.lower():
+               return s
+       return None

    # -- internal -----

diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
index bee716b..f6ef6b3 100644
--- a/src/tg_video_filter/db.py
+++ b/src/tg_video_filter/db.py
@@ -1399,7 +1399,7 @@ async def upsert_channel(conn: aioredis.Connection, channel: Channel) ->
Channe
        provider_source_id=str(channel.channel_id),
        title=channel.title,
        kind="channel",
-       watched=True, # back-compat: pre-ADR-0011 channels were all watched
+       watched=False,
        seen_first_at=channel.seen_first_at,
    ),
)
@@ -1560,6 +1560,35 @@ async def list_account_sources(
    return [_row_to_source(r) for r in rows]

+async def should_ingest_source(
+    conn: aioredis.Connection, account_id: int, channel_id: int
+) -> bool:
+    """Return True if a channel should be ingested (ADR 0009 opt-in gate).
+

```

```

+   - Absent sources row: create it lazily with ``watched=False`` for
+     ``/channels`` discoverability; return False (not yet watched).
+   - Existing row: respect the ``watched`` flag.
+
+ Pure helper extracted from ``telethon_client.py`` for testability.
+ """
+ src = await get_source_by_provider_id(conn, account_id, "telegram", str(channel_id))
+ if src is None:
+     # First-ever message from this channel ? create a discoverable
+     # unwatched row so /channels can list it (ADR 0009 lines 48-54).
+     await upsert_source(
+         conn,
+         Source(
+             account_id=account_id,
+             provider="telegram",
+             provider_source_id=str(channel_id),
+             kind="channel",
+             watched=False,
+         ),
+     )
+     return False
+ return src.watched
+
+
+ async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
+     """Deprecated shim: insert into ``content_items`` (ADR 0011).
+
@@ -1575,7 +1604,7 @@ async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) ->
VideoR
        provider="telegram",
        provider_source_id=str(video.channel_id),
        kind="channel",
-        watched=True,
+        watched=False,
    ),
)
    assert src.id is not None # upsert_source populates the surrogate PK
@@ -1745,6 +1774,7 @@ __all__ = [
    "get_source_by_id",
    "set_source_watched",
    "list_account_sources",
+    "should_ingest_source",
    "insert_content_item",
    "get_content_item_by_message",
    "get_content_item_by_dedup_key",
diff --git a/src/tg_video_filter/telethon_client.py b/src/tg_video_filter/telethon_client.py
index ebb7416..df8b4f9 100644
--- a/src/tg_video_filter/telethon_client.py
+++ b/src/tg_video_filter/telethon_client.py
@@ -182,13 +182,12 @@ def make_new_message_handler(
    document_id = _extract_document_id(message)

    # --- watched gate (ADR 0009, Track E) -----
-    # Skip ingestion for unwatched sources. The channel must already be in
-    # sources (lazy-created by the /channels enumeration or the legacy
-    # upsert_channel below); if absent, we let the legacy path upsert it
-    # with watched=True for backwards-compatibility.
-    src = await db.get_source_by_provider_id(conn, user.id, "telegram", str(channel_id))
-    if src is not None and not src.watched:
-        return # silently skip unwatched source
+    # Opt-in: a source is ingested only when watched=True. An absent
+    # sources row means the channel has never been seen ? treat it as
+    # unwatched and lazily create a watched=0 row for /channels
+    # discoverability (ADR 0009 lines 37-44, 130-131).
+    if not await db.should_ingest_source(conn, user.id, channel_id):

```

```
+      return # skip unwatched / unknown source

# --- three-tier dedup (cheapest first) ---

diff --git a/tests/test_command_service.py b/tests/test_command_service.py
index c4d8984..320f8a5 100644
--- a/tests/test_command_service.py
+++ b/tests/test_command_service.py
@@ -234,6 +234,39 @@ def test_format_commands_includes_all_sections() -> None:
     assert "/set size" in text
     assert "/set exclude" in text
     assert "/backfill" in text
+    assert "/channels" in text
+    assert "/watch " in text
+    assert "/watchlist" in text
+
+# --- should_ingest_source (telethon_client gate, extracted for testability) ---
+
+
+
+async def test_should_ingest_absent_source_creates_unwatched_and_returns_false(
+    conn: aiomysql.Connection,
+) -> None:
+    """An unknown channel gets a lazily-created unwatched source row; ingestion denied."""
+    result = await db.should_ingest_source(conn, USER_ID, "-100888")
+    assert result is False
+    src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-100888")
+    assert src is not None
+    assert src.watched is False
+
+
+
+async def test_should_ingest_existing_watched_returns_true(
+    conn: aiomysql.Connection,
+) -> None:
+    await _seed_source(conn, USER_ID, "-100777", watched=True)
+    result = await db.should_ingest_source(conn, USER_ID, "-100777")
+    assert result is True
+
+
+
+async def test_should_ingest_existing_unwatched_returns_false(
+    conn: aiomysql.Connection,
+) -> None:
+    await _seed_source(conn, USER_ID, "-100777", watched=False)
+    result = await db.should_ingest_source(conn, USER_ID, "-100777")
+    assert result is False
+
+
+
+# --- /channels + /watch + /unwatch + /watchlist (ADR 0009, Track E) -----
@@ -298,11 +298,12 @@ async def test_watch_by_numeric_id(conn: aiomysql.Connection, svc:
CommandServi
    assert src.watched is True


-async def test_watch_by_username(conn: aiomysql.Connection, svc: CommandService) -> None:
-    await _seed_source(conn, USER_ID, "@mychan", "My Channel", watched=False)
-    result = await svc.handle(USER_ID, "/watch @mychan", conn, FilterConfig())
+async def test_watch_by_title(conn: aiomysql.Connection, svc: CommandService) -> None:
+    """Realistic source: numeric provider_source_id, title='My Channel'. /watch 'My' matches by
title."""
+    await _seed_source(conn, USER_ID, "-100999", "My Channel", watched=False)
+    result = await svc.handle(USER_ID, "/watch My", conn, FilterConfig())
+    assert "Now watching" in result.reply_text
+
-    src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "@mychan")
+    src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-100999")
+    assert src is not None
```

```
assert src.watched is True
```

#### 277. assistant (2026-07-06T09:32:01.106Z)

Now I need to scrutinize the fix for a **new regression** it may have introduced. ``upsert_channel`` and ``insert_video`` now create sources with ``watched=False``, but ``upsert_source``'s ``ON CONFLICT`` overwrites ``watched``. If the live handler still calls those for an **already-watched** channel, it would reset ``watched=False`` after the first video ? unwatching it. Let me verify.

#### 278. user (2026-07-06T09:32:05.197Z)

```
=== does upsert_source ON CONFLICT overwrite watched? ===
12:         ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
13:             title=excluded.title,
14:             watched=excluded.watched,
15:             last_seen_at=excluded.last_seen_at"",

=== does the live handler still call upsert_channel / insert_video AFTER the gate? ===
9:     -> db.insert_video(...) + delivery.forward_to_target(..., target=cfg.delivery_target)
146: def make_new_message_handler(
167:     async def on_new_message(event: NewMessage.Event) -> None: # pragma: no cover
189:         if not await db.should_ingest_source(conn, user.id, channel_id):
248:             await db.upsert_channel(
271:                 inserted = await db.insert_video(conn, record)
```

#### 279. assistant (2026-07-06T09:32:20.401Z)

Both conditions hold: ``upsert_source`` ``ON CONFLICT`` **overwrites** ``watched``, and the handler still calls ``upsert_channel`` (line 248) and ``insert_video`` (line 271) ? both now with ``watched=False`` ? **after** the gate. So an already-watched channel would get reset to ``watched=False`` after ingesting one video. Let me reproduce this potential new regression.

#### 280. user (2026-07-06T09:32:29.998Z)

Exception in thread Thread-1 (\_connection\_worker\_thread):

```
self._target(*self._args, **self._kwargs)
after /watch           : watched = True (expect True)
should_ingest (msg 1)  : True (expect True)
after upsert_channel   : watched = False (REGRESSION if False)
after insert_video     : watched = False (REGRESSION if False)
should_ingest (msg 2)  : False (REGRESSION if False -> channel silently un-watched)
```

#### 281. assistant (2026-07-06T09:33:15.522Z)

**Confirmed** ? the fix introduced a new regression, now merged into ``main``. The repro shows: after ``/watch``, the channel is ``watched=True`` and msg 1 ingests ? but ``upsert_channel`` and ``insert_video`` (called during ingestion) upsert the source with ``watched=False``, and ``upsert_source``'s ``ON CONFLICT`` overwrites it. So from **msg 2** onward the channel is silently un-watched. Net: ``/watch`` works for exactly one video, then stops. Let me confirm the suite still passes (i.e., this is untested) and check the gate.

#### 282. user (2026-07-06T09:33:21.430Z)

```
=== gate on main (with fix) ===
=== RUFF ===
All checks passed!
36 files already formatted
=== PYTEST ===
```

```
-----
TOTAL                                1192    27    312    19    97%
Required test coverage of 80.0% reached. Total coverage: 96.94%
```

495 passed in 7.24s  
DONE

```
=== confirm @username still can't match a real (numeric-id) source; only title/numeric work ===
350:      """Parse ``/watch <@username|channel_id>`` and toggle ``watched=1``."""
366:      """Parse ``/unwatch <@username|channel_id>`` and toggle ``watched=0``."""
413:      Accepts numeric channel id, ``@username``, or channel title.
416:      target = target.removeprefix("@")
421:      # Try by @username.
429:      if s.title and target_lower in s.title.lower():
```

### 283. assistant (2026-07-06T09:34:12.985Z)

Gate on main (with fix): ruff clean, \*\*495 passed @ 96.94%\*\* ? so the watched-reset regression ships \*\*green\*\* (untested). Here's the re-review round.

---

## Re-review round ? Track E fix (`ceabe7f`, merged as #39 ? `main` `3bfb1da`)

The author pushed a fix \*\*and merged it\*\*. Good news: the fix addresses all six of my original findings. Bad news: \*\*it introduced a new blocker that's now in `main`\*\* ? I reproduced it.

### ? Original findings ? addressed

| #   | Finding                          | Fix                                                                                                                            | Verified                         |
|-----|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| ? 1 | New channels auto-watched        | Default ? `watched=False`; gate skips absent sources; extracted `should_ingest_source()` that lazily creates a `watched=0` row | intent correct ? (but see below) |
| ? 2 | `/watch @name` broken            | Added title substring lookup (`/watch My` matches title "My Channel")                                                          | works ?                          |
| ? 3 | Gate untested                    | Extracted `should_ingest_source` + 3 unit tests                                                                                | ?                                |
| ? 4 | `format_commands` omits commands | Added "Source scanning" section + test asserts it                                                                              | ?                                |
| ? 5 | `rstrip("@")` strips multiple    | `removeprefix("@")`                                                                                                            | ?                                |

### ? New blocker introduced by the fix (now in `main`) ? a watched channel un-watches itself after one video

The fix flipped `upsert\_channel` and `insert\_video` to upsert sources with `watched=False`. But `upsert\_source`'s `ON CONFLICT` does `watched=excluded.watched`, and the live handler \*\*still calls both after the gate\*\* (`telethon\_client.py:248` `upsert\_channel`, `:271` `insert\_video`). So ingesting a video from a watched channel \*\*overwrites the user's `watched` flag back to `False`\*\*.

Reproduced end-to-end:

```
...
after /watch          : watched = True
should_ingest (msg 1) : True      ? msg 1 ingests
after upsert_channel  : watched = False ? clobbered
after insert_video    : watched = False
should_ingest (msg 2) : False      ? channel silently un-watched
...
```

**Effect:** `/watch <channel>` works for exactly **one** video, then the channel stops being watched. The feature is still broken ? differently, and now silently/intermittently. The suite is green because no test exercises "watch ? ingest ? still watched?" (the exact integration the `# pragma: no cover` handler hides).

**Root cause:** `watched` is user-controlled state; ingestion-path upserts must not clobber it.  
**Fix:** drop `watched=excluded.watched` from `upsert\_source`'s `ON CONFLICT DO UPDATE` so an existing row keeps its `watched` value (only `set\_source\_watched`, a direct `UPDATE`, changes it). New rows still get their INSERT value; the migration backfill still sets `watched=1` via direct insert. I checked ? no caller relies on `upsert\_source` toggling `watched`, so this is safe.

## Minor (carried, low)

- `/watch @username` still can't match a real source (no username column); the title path covers name-based lookup, so acceptable. Title match is a first-substring match, so `/watch a` could hit an unintended channel ? worth an exact-match-first tiebreak later.

---

**\*\*Bottom line:\*\*** the fix resolved the original blocker's *\*intent\** but traded it for an equally-breaking, untested regression that's now merged. This needs a follow-up (one-line `ON CONFLICT` change + a `watch ? ingest ? still watched` regression test).

Since #39 is already merged, want me to:

1. **\*\*Open a GitHub issue\*\*** documenting the regression + fix + repro,
2. **\*\*Post a comment\*\*** on the merged PR #39, and/or
3. **\*\*Draft the follow-up fix\*\*** (branch off `origin/main`: drop `watched=excluded.watched`, add the regression test) ? like the #36 cleanup?

I'll hold for your call on which.

### 284. user (2026-07-06T10:29:42.986Z)

draft the followup fix and send for review

### 285. assistant (2026-07-06T10:30:24.296Z)

On it. First let me verify the fix is safe ? check whether any existing test or caller relies on `upsert\_source` overwriting `watched` (or `title`) on conflict, so I don't break something.

### 286. user (2026-07-06T10:30:27.753Z)

```
=== upsert_source conflict test ? does it assert watched/title updated on conflict? ===
748:         "SELECT account_id, provider, provider_source_id, title, watched " "FROM
sources"
972:async def test_upsert_source_round_trip(conn: aiosqlite.Connection) -> None:
975:     src = await db.upsert_source(
994:async def test_upsert_source_updates_on_conflict(conn: aiosqlite.Connection) -> None:
995:     """Re-upserting the same (account, provider, source_id) updates title/watched."""
997:     await db.upsert_source(
999:         Source(account_id=USER_ID, provider="telegram", provider_source_id="-1",
watched=True),
1001:     updated = await db.upsert_source(
1022:     src = await db.upsert_source(
1056:     src = await db.upsert_source(
1203:         # sources backfilled with watched=1 (ADR 0009 backfill).
1205:         "SELECT provider_source_id, watched FROM sources ORDER BY provider_source_id"
1247:     src = await db.upsert_source(
1279:     src = await db.upsert_source(
1392:         "SELECT title, watched FROM sources WHERE provider_source_id = '-1002'"
1401:         "SELECT title, watched FROM sources WHERE provider_source_id = '-1001'"
1497:     src = await db.upsert_source(
1526:     src = await db.upsert_source(
1560:     src = await db.upsert_source(

--- the conflict-update test body ---
async def test_upsert_source_updates_on_conflict(conn: aiosqlite.Connection) -> None:
    """Re-upserting the same (account, provider, source_id) updates title/watched."""
    await _seed_user(conn)
    await db.upsert_source(
        conn,
        Source(account_id=USER_ID, provider="telegram", provider_source_id="-1", watched=True),
    )
    updated = await db.upsert_source(
        conn,
        Source(
```

```

        account_id=USER_ID,
        provider="telegram",
        provider_source_id="-1",
        title="Renamed",
        watched=False,
    ),
)
assert updated.title == "Renamed"
assert updated.watched is False
# Still one row.
fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1")
assert fetched is not None
assert fetched.watched is False

```

```
async def test_insert_content_item_round_trip(conn: aiosqlite.Connection) -> None:
```

## 287. assistant (2026-07-06T10:32:55.787Z)

The fix is safe but there's a wrinkle: ``test_upsert_source_updates_on_conflict`` (test\_db.py:994) explicitly asserts ``upsert_source`` \*overwrites\* ``watched`` on conflict ? that test \*\*encodes the buggy contract\*\*. The correct model is: ``watched`` is user-state, changed only via ``set_source_watched``, never clobbered by an ingest-path upsert. So the fix is: drop ``watched=excluded.watched`` from the ``ON CONFLICT``, update that test, and add a regression test. I'll also ``COALESCE`` the title (ingest's ``insert_video`` passes ``title=None``, nulling the channel name ? same clause, same root cause).

Let me draft it in an isolated worktree off ``origin/main``.

## 288. user (2026-07-06T10:32:58.375Z)

```

origin/main:
3bfb1da feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR
0009, Track E) (#39)
branch 'fix/upsert-source-preserve-watched' set up to track 'origin/main'.
HEAD is now at 3bfb1da feat(scanning): selective source scanning ? /watch, /unwatch, /channels,
/watchlist (ADR 0009, Track E) (#39)
--- worktree branch ---
3bfb1da feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR
0009, Track E) (#39)

```

```
=== exact upsert_source ON CONFLICT in the worktree ===
```

```
1104:async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
```

## 289. user (2026-07-06T10:33:01.209Z)

```

1104     async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
1105         """Insert a source row, or update title/watched on conflict.
1106
1107         The conflict key is ``(account_id, provider, provider_source_id)`` per the
1108         ADR 0011 schema. Returns the persisted row.
1109         """
1110         await conn.execute(
1111             """INSERT INTO sources
1112                 (account_id, provider, provider_source_id, title, kind, watched,
1113                 seen_first_at, last_seen_at)
1114                 VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1115                 ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
1116                 title=excluded.title,
1117                 watched=excluded.watched,
1118                 last_seen_at=excluded.last_seen_at""",
1119             (
1120                 source.account_id,
1121                 source.provider,

```



```

1122         source.provider_source_id,
1123         source.title,
1124         source.kind,
1125         int(source.watched),
1126         source.seen_first_at.isoformat(),
1127         (source.last_seen_at.isoformat() if source.last_seen_at else None),
1128     ),
1129 )
1130 await conn.commit()
1131 conn.row_factory = aisqlite.Row
1132 async with conn.execute(
1133     """SELECT * FROM sources
1134         WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1135     (source.account_id, source.provider, source.provider_source_id),
1136 ) as cur:
1137     row = await cur.fetchone()
1138     assert row is not None # just upserted by the conflict key
1139     return _row_to_source(row)
1140
1141
1142 async def get_source_by_provider_id(
1143     conn: aisqlite.Connection, account_id: int, provider: str, provider_source_id: str
1144 ) -> Source | None:
1145     """Resolve a source by its provider-native id."""
1146     conn.row_factory = aisqlite.Row
1147     async with conn.execute(
1148         """SELECT * FROM sources

```

#### 290. assistant (2026-07-06T10:33:05.518Z)

Now I'll apply the fix: remove `watched=excluded.watched` (preserve user-controlled `watched`) and `COALESCE` the title so ingest doesn't null it.

#### 291. user (2026-07-06T10:33:13.939Z)

The file C:/Users/avidu/Projects/\_wt-watched-fix/src/tg\_video\_filter/db.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 292. assistant (2026-07-06T10:33:24.895Z)

Now update the conflict test to encode the corrected contract (watched preserved, title still updated), and add a dedicated regression test. Let me read the test region in the worktree.

#### 293. user (2026-07-06T10:33:25.460Z)

```

972 async def test_upsert_source_round_trip(conn: aisqlite.Connection) -> None:
973     """Insert then read back a source via the provider-neutral repo."""
974     await _seed_user(conn)
975     src = await db.upsert_source(
976         conn,
977         Source(
978             account_id=USER_ID,
979             provider="telegram",
980             provider_source_id="-100555",
981             title="Test Channel",
982             kind="channel",
983             watched=True,
984         ),
985     )
986     assert src.id is not None
987     fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-100555")
988     assert fetched is not None
989     assert fetched.title == "Test Channel"
990     assert fetched.watched is True

```

```

991         assert fetched.kind == "channel"
992
993
994     async def test_upsert_source_updates_on_conflict(conn: aiomysqlite.Connection) -> None:
995         """Re-upserting the same (account, provider, source_id) updates title/watched."""
996         await _seed_user(conn)
997         await db.upsert_source(
998             conn,
999             Source(account_id=USER_ID, provider="telegram", provider_source_id="-1",
1000 watched=True),
1001         )
1002         updated = await db.upsert_source(
1003             conn,
1004             Source(
1005                 account_id=USER_ID,
1006                 provider="telegram",
1007                 provider_source_id="-1",
1008                 title="Renamed",
1009                 watched=False,
1010             ),
1011         )
1012         assert updated.title == "Renamed"
1013         assert updated.watched is False
1014         # Still one row.
1015         fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1")
1016         assert fetched is not None
1017         assert fetched.watched is False

```

#### 294. user (2026-07-06T10:33:35.492Z)

The file C:/Users/avidu/Projects/\_wt-watched-fix/tests/test\_db.py has been updated successfully.  
(file state is current in your context ? no need to Read it back)

#### 295. assistant (2026-07-06T10:33:41.076Z)

Now the end-to-end regression test. Let me check test\_db.py's imports for  
'Channel'/'VideoRecord', then add it.

#### 296. user (2026-07-06T10:33:43.562Z)

```

from tg_video_filter.models import (
    DEFAULT_FILTER_CONFIG,
    Account,
    Channel,
    ContentItem,
    DeliveryStatus,
    Destination,
    FilterConfig,
    Policy,
    Source,
    TelegramSession,
    User,
    VideoRecord,
)
--- _seed_user + USER_ID/CHANNEL_ID defs ---
26:USER_ID = 111
27:CHANNEL_ID = -1001234567890
39:async def _seed_user(conn: aiomysqlite.Connection, uid: int = USER_ID) -> User:
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

```

#### 297. assistant (2026-07-06T10:33:57.569Z)

Both imports and helpers are available. Let me add the end-to-end regression test right after  
the conflict test.

### 298. user (2026-07-06T10:33:58.244Z)

```
1020      # Still one row, watched intact.
1021      fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1")
1022      assert fetched is not None
1023      assert fetched.watched is True
1024
1025
1026      async def test_insert_content_item_round_trip(conn: aiosqlite.Connection) -> None:
1027          """Insert a content item and read it back via both natural keys."""
1028          await _seed_user(conn)
1029          src = await db.upsert_source(
1030              conn, Source(account_id=USER_ID, provider="telegram", provider_source_id="-1")
1031          )
```

### 299. user (2026-07-06T10:34:09.841Z)

The file C:/Users/avidu/Projects/\_wt-watched-fix/tests/test\_db.py has been updated successfully.  
(file state is current in your context ? no need to Read it back)

### 300. assistant (2026-07-06T10:34:35.042Z)

Now let me verify: run the full suite against the worktree (with fix), and prove the new tests actually catch the bug by running them with the `db.py` fix temporarily reverted.

### 301. user (2026-07-06T10:34:44.500Z)

=== (A) new/edited tests WITH fix (expect pass) ===

```
-----
TOTAL                                1192    834    312    15    25%
FAIL Required test coverage of 80.0% not reached. Total coverage: 24.93%
2 passed, 78 deselected in 0.88s
```

=== (B) PROOF they catch the bug: revert db.py fix, rerun (expect FAIL) ===

Saved working directory and index state WIP on fix/upsert-source-preserve-watched: 3bfb1da  
feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR 0009,  
Track E) (#39)

FAIL Required test coverage of 80.0% not reached. Total coverage: 24.93%

===== short test summary info =====

FAILED tests/test\_db.py::test\_upsert\_source\_updates\_title\_but\_preserves\_watched\_on\_conflict

FAILED tests/test\_db.py::test\_watched\_source\_survives\_ingestion\_upserts - Ass...

2 failed, 78 deselected in 0.69s

--- restore fix ---

Dropped refs/stash@{0} (7cd607f719751e71c03b5addfcfd87911c83039f)

Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

### 302. assistant (2026-07-06T10:34:53.050Z)

Clean before/after proof: both tests **pass** with the **fix** and **fail** without it (reverting  
`db.py` reproduces the regression). Now the full suite + lint on the worktree.

### 303. user (2026-07-06T10:35:06.383Z)

=== ruff check + format ===

All checks passed!

1 file would be reformatted, 35 files already formatted

=== full suite (worktree, with fix) ===

```
-----
TOTAL                                1192    27    312    19    97%
Required test coverage of 80.0% reached. Total coverage: 96.94%
496 passed in 8.34s
=== changed files ===
```

**fix/upsert-source-preserve-watched...origin/main**

M src/tg\_video\_filter/db.py

M tests/test\_db.py  
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

#### 304. assistant (2026-07-06T10:35:12.640Z)

Full suite \*\*496 passed @ 96.94%\*\* (+1 test), ruff check clean ? but one file needs formatting (my multi-line edits). Let me apply `ruff format` to the changed files and re-verify.

#### 305. user (2026-07-06T10:35:16.049Z)

```
=== which file? ===  
Would reformat: tests\test_db.py  
1 file would be reformatted, 1 file already formatted
```

#### 306. assistant (2026-07-06T10:35:21.207Z)

The `--check` short-circuited the chain (exit 1). Let me format directly.

#### 307. user (2026-07-06T10:35:26.904Z)

```
1 file reformatted, 1 file left unchanged  
--- whole-repo format check ---  
36 files already formatted  
fmt-exit=0  
--- ruff check ---  
All checks passed!  
--- re-run the 2 tests after reformat ---  
FAIL Required test coverage of 80.0% not reached. Total coverage: 24.93%  
2 passed, 78 deselected in 0.76s  
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

#### 308. assistant (2026-07-06T10:35:35.349Z)

Clean: format ?, check ?, tests still pass. Let me review the final diff, then commit and open the PR.

#### 309. user (2026-07-06T10:35:36.804Z)

```
=== FINAL DIFF ===  
diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py  
index f6ef6b3..a059e57 100644  
--- a/src/tg_video_filter/db.py  
+++ b/src/tg_video_filter/db.py  
@@ -1102,10 +1102,20 @@ async def list_enabled_users(conn: aiomysqlite.Connection) -> list[User]:
```

```
    async def upsert_source(conn: aiomysqlite.Connection, source: Source) -> Source:  
-    """Insert a source row, or update title/watched on conflict.  
+    """Insert a source row, or refresh title/last_seen_at on conflict.  
  
    The conflict key is `(account_id, provider, provider_source_id)` per the  
    ADR 0011 schema. Returns the persisted row.  
+  
+    ``watched`` is **user-controlled state** (toggled only via  
+    :func:`set_source_watched` / the ``/watch`` and ``/unwatch`` commands) and is  
+    therefore **never overwritten on conflict** ? the ingestion path  
+    (``upsert_channel`` / ``insert_video``) re-touches a source on every message,  
+    so clobbering ``watched`` here would silently un-watch a channel the user  
+    opted into after its first ingested video (ADR 0009). ``watched`` is only set  
+    on the initial INSERT of a new row. ``title`` is refreshed but never nulled:  
+    the ingestion shims pass ``title=None``, so ``COALESCE`` keeps the existing  
+    title rather than erasing the channel name.  
    """  
    await conn.execute(
```

```

        """INSERT INTO sources
@@ -1113,8 +1123,7 @@ async def upsert_source(conn: aiosqlite.Connection, source: Source) ->
Source:
        seen_first_at, last_seen_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
-         title=excluded.title,
-         watched=excluded.watched,
+         title=COALESCE(excluded.title, sources.title),
        last_seen_at=excluded.last_seen_at""",
        (
            source.account_id,
diff --git a/tests/test_db.py b/tests/test_db.py
index c479c34..8692285 100644
--- a/tests/test_db.py
+++ b/tests/test_db.py
@@ -991,13 +991,20 @@ async def test_upsert_source_round_trip(conn: aiosqlite.Connection) ->
None:
    assert fetched.kind == "channel"

-async def test_upsert_source_updates_on_conflict(conn: aiosqlite.Connection) -> None:
-    """Re-upserting the same (account, provider, source_id) updates title/watched."""
+async def test_upsert_source_updates_title_but_preserves_watched_on_conflict(
+    conn: aiosqlite.Connection,
+) -> None:
+    """On conflict, ``upsert_source`` refreshes the title but must NOT overwrite
+    ``watched`` ? that flag is user-controlled state (set only via
+    ``set_source_watched`` / ``/watch``). Overwriting it here silently un-watched
+    channels after their first ingested video (ADR 0009 regression)."""
    await _seed_user(conn)
    await db.upsert_source(
        conn,
        Source(account_id=USER_ID, provider="telegram", provider_source_id="-1", watched=True),
    )
+    # A subsequent upsert (as the ingestion path does) carries watched=False by
+    # default; it must not clobber the user's opt-in.
    updated = await db.upsert_source(
        conn,
        Source(
@@ -1008,12 +1015,48 @@ async def test_upsert_source_updates_on_conflict(conn:
aiosqlite.Connection) ->
        watched=False,
    ),
)
-    assert updated.title == "Renamed"
-    assert updated.watched is False
-    # Still one row.
+    assert updated.title == "Renamed" # title still refreshed
+    assert updated.watched is True # watched preserved, not clobbered
+    # Still one row, watched intact.
    fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1")
    assert fetched is not None
    assert fetched.watched is False
+    assert fetched.watched is True
+
+
+async def test_watched_source_survives_ingestion_upserts(conn: aiosqlite.Connection) -> None:
+    """Regression (ADR 0009 opt-in): a channel the user ``/watch``-ed must stay
+    watched after ingestion. The live ``on_new_message`` handler re-touches the
+    source via ``upsert_channel`` + ``insert_video`` on every message; before the
+    fix those upserts clobbered ``watched`` back to False, silently un-watching
+    the channel after its first ingested video (``/watch`` worked for exactly one
+    video)."""
+    await _seed_user(conn)

```

```

+     await db.upsert_source(
+         conn,
+         Source(
+             account_id=USER_ID,
+             provider="telegram",
+             provider_source_id=str(CHANNEL_ID),
+             title="My Feed",
+             kind="channel",
+             watched=True, # user opted in via /watch
+         ),
+     )
+     assert await db.should_ingest_source(conn, USER_ID, CHANNEL_ID) is True
+
+     # Simulate the ingestion path re-touching the source (handler lines ~248/271).
+     await db.upsert_channel(conn, Channel(user_id=USER_ID, channel_id=CHANNEL_ID, title="My
Feed"))
+     await db.insert_video(
+         conn,
+         VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=1, document_id=42),
+     )
+
+     src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", str(CHANNEL_ID))
+     assert src is not None
+     assert src.watched is True, "ingestion upserts must not un-watch a user-watched channel"
+     assert src.title == "My Feed", "ingestion (insert_video title=None) must not erase the
title"
+     # The very next message from the channel is still ingested.
+     assert await db.should_ingest_source(conn, USER_ID, CHANNEL_ID) is True

```

```

async def test_insert_content_item_round_trip(conn: aiosqlite.Connection) -> None:
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

```

### 310. assistant (2026-07-06T10:35:50.715Z)

The diff is clean and correct. Let me commit (explicit paths), push, and open the PR.

### 311. user (2026-07-06T10:35:56.632Z)

```

fix/upsert-source-preserve-watched
src/tg_video_filter/db.py
tests/test_db.py
[fix/upsert-source-preserve-watched 5791e29] fix(scanning): don't clobber user-controlled
watched on source upsert
2 files changed, 61 insertions(+), 9 deletions(-)
=== only my commit ahead of origin/main? ===
5791e29 fix(scanning): don't clobber user-controlled watched on source upsert
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

```

### 312. user (2026-07-06T10:36:19.363Z)

```

=== push ===
remote:
branch 'fix/upsert-source-preserve-watched' set up to track 'origin/fix/upsert-source-preserve-
watched'.
To https://github.com/avidullu/telegram-video-filter.git
* [new branch]      fix/upsert-source-preserve-watched -> fix/upsert-source-preserve-watched
=== open PR ===
https://github.com/avidullu/telegram-video-filter/pull/41
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

```

### 313. assistant (2026-07-06T10:36:23.122Z)

PR **41** opened. Let me clean up the worktree and confirm the PR state.

### 314. user (2026-07-06T10:36:27.785Z)

```
worktree removed
C:/Users/avidu/Projects/Telegram 3bfb1da [main]
--- main checkout clean? ---
main
main...origin/main
--- PR #41 ---
#41 OPEN mergeable=MERGEABLE CLEAN (+61/-9)
fix(scanning): don't clobber user-controlled `watched` on source upsert
```

### 315. assistant (2026-07-06T10:36:41.045Z)

Done ? the follow-up fix is drafted, verified, and up for review.

**[PR #41 ? fix(scanning): don't clobber user-controlled `watched` on source upsert](<https://github.com/avidull>)**

```
**State:** OPEN · MERGEABLE · CLEAN (+61/?9) off `origin/main`.

**The regression it fixes** (the one #39's fix introduced, now in `main`): the ingestion path
re-touches the source on every message via `upsert_channel` + `insert_video`, and
`upsert_source`'s `ON CONFLICT` overwrote `watched` with the shims' hardcoded `False` ? so a
`/watch`-ed channel got silently un-watched after its first ingested video.

**The fix:**
```sql
ON CONFLICT(...) DO UPDATE SET
  title=COALESCE(excluded.title, sources.title),    -- was: title=excluded.title (insert_video
passed NULL ? erased the name)
  last_seen_at=excluded.last_seen_at                -- removed: watched=excluded.watched
...

`watched` is user-controlled state (only `set_source_watched` / `/watch`, `/unwatch` change it),
so it's set only on a new row's INSERT and never clobbered on conflict. I confirmed no caller
relies on the old behavior (the commands use a direct `UPDATE`; the migration uses a direct
`INSERT`).

**Verification (all reproduced locally):**
- New `test_watched_source_survives_ingestion_upserts` (watch ? ingest ? still watched + title
intact) and the updated conflict test **both fail against pre-fix `db.py` and pass after** ?
proven by reverting `db.py` in isolation.
- Full suite **496 passed @ 96.94%** (+1 test), `ruff check` + `format` clean.

**Process:** drafted in an isolated worktree off `origin/main` (shared checkout untouched),
staged explicit paths, confirmed only my commit `5791e29` was ahead before pushing; worktree
removed, `main` left clean.

Note it's a **separate PR** (#41) rather than a comment on the merged #39, since the regression
lives in `main` now. When it's reviewed/merged, that closes out the Track E selective-scanning
contract properly (opt-in default **and** `/watch` that sticks). Want me to also drop a short note
on #39 linking to #41 for traceability?
```

### 316. user (2026-07-06T11:22:07.123Z)

Merge the pr

### 317. assistant (2026-07-06T11:22:27.576Z)

Merging PR #41. Let me re-confirm it's still clean against current `main`, then squash-merge (matching the repo's convention) and delete the branch.

### 318. user (2026-07-06T11:22:37.039Z)

=== origin/main + PR #41 state pre-merge ===

```
3bfb1da feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR
0009, Track E) (#39)
state=OPEN mergeable=MERGEABLE CLEAN
```

```
=== squash-merge #41 + delete branch ===
```

### 319. assistant (2026-07-06T11:22:42.249Z)

Let me confirm the merge landed.

### 320. user (2026-07-06T11:22:49.724Z)

```
=== PR #41 state ===
state=MERGED mergedAt=2026-07-06T11:22:35Z mergeCommit=c4c4d187a09e

=== origin/main top 3 (fix present?) ===
c4c4d18 fix(scanning): don't clobber user-controlled watched on source upsert (#41)
3bfb1da feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR
0009, Track E) (#39)
35f4f70 feat(commands): extract CommandService ? provider-independent command execution (ADR
0011, Track D) (#38)

=== the watched fix is in main? ===
1126:          title=COALESCE(excluded.title, sources.title),
```

### 321. assistant (2026-07-06T11:22:59.412Z)

```
? **PR #41 merged** ? squash-merged as [c4c4d18` (#41)](https://github.com/avidullu/telegram-
video-filter/commit/c4c4d18), branch deleted. The fix is confirmed in `main`:
`title=COALESCE(...)` is present and `watched=excluded.watched` is gone.
```

Let me fast-forward the local `main` checkout to match (repo-freshness convention).

### 322. user (2026-07-06T11:23:03.200Z)

```
src/tg_video_filter/db.py | 15 ++++++---
tests/test_db.py          | 55 ++++++-----
2 files changed, 61 insertions(+), 9 deletions(-)
--- local main now ---
c4c4d18 fix(scanning): don't clobber user-controlled watched on source upsert (#41)
main...origin/main
```

### 323. assistant (2026-07-06T11:23:11.841Z)

Done. ?

## Summary

```
**PR #41 is merged** ? squash-merged as [c4c4d18` (#41)](https://github.com/avidullu/telegram-
video-filter/commit/c4c4d18) into `main`, remote branch deleted, local `main` fast-forwarded to
match (clean, synced).
```

The Track E selective-scanning regression is now fully resolved in `main`:

- New channels default to `watched=0` (opt-in) ? from #39's fix.
- **\*\*And\*\*** a `/watch`-ed channel now *stays* watched ? `upsert\_source` no longer clobbers the user's `watched` flag on ingest, and `COALESCE` stops `insert\_video` from erasing the channel title.

Verified before merging: `MERGEABLE/CLEAN`, 496 tests pass @ 96.94%, ruff clean, and the new regression tests provably fail without the fix / pass with it.

That closes the loop on the whole Track B?E series I reviewed this session: #35 (fixed + LGTM), #37 (fixed + LGTM), #38 (LGTM), #39 (blocker ? author fixed but introduced a regression ? my #41



follow-up now merged). If you want, I can drop a one-line note on #39 linking #41 for traceability ? otherwise this is a clean stopping point.

#### 324. user (2026-07-06T11:25:09.714Z)

Yes please drop the comment

#### 325. user (2026-07-06T11:25:20.805Z)

<https://github.com/avidullu/telegram-video-filter/pull/39#issuecomment-4892223894>

#### 326. assistant (2026-07-06T11:25:30.267Z)

? **Comment posted on #39** ? [issuecomment-4892223894](<https://github.com/avidullu/telegram-video-filter/pull/39#pullrequestreview-4634446993#issuecomment-4892223894>)

It links the two for traceability: notes that `ceabe7f`'s fix surfaced the un-watch regression, and points to #41 (`c4c4d18`) as the follow-up that resolved it, with the regression test.

That wraps up the Track B?E review series cleanly:

- **#35** Track B ? blocker + dedup regression ? fixed ? LGTM
- **#37** Track C ? 2 mediums ? fixed ? LGTM
- **#38** Track D ? clean refactor ? LGTM
- **#39** Track E ? blocker ? author fixed, introduced a regression ? **#41** follow-up (my fix) merged, and #39 now annotated

Everything's merged into `main` and the local checkout is clean and synced. Nothing outstanding on my side ? ping me when the next track (or a new PR) is up.

#### 327. user (2026-07-06T19:12:58.320Z)

Can you review the draft PR #40

#### 328. assistant (2026-07-06T19:13:05.787Z)

I'll sync and pull up PR #40 (a draft).

#### 329. user (2026-07-06T19:13:11.922Z)

From <https://github.com/avidullu/telegram-video-filter>

```
- [deleted]          (none)      -> origin/fix/upsert-source-preserve-watched
---checkout---
main
```

**main...origin/main**

=== PR #40 META ===

```
{"additions":261,"author":{"id":"MDQ6VXNlcjE2NzQ4Nzc=","is_bot":false,"login":"avidullu","name":"Avi"},"baseRefName":"main","body":"## What\n\nFinal track of the provider-neutral umbrella (#25) ? Track F (#31) design doc. Docs-only, no code changes.\n\nProposes ADR 0012: Policy pipeline ? a four-tier, ordered, short-circuiting evaluation model that unifies quality gates, topic preferences, content safety, and deep inspection into a single explainable framework.\n\n## The four tiers (cheapest first)\n\n| Tier | Name | Cost | When |\n|---|---|---|---|\n| 1 | Metadata gates | Zero I/O | Always runs |\n| 2 | Topic preferences | Metadata-only | Opt-in |\n| 3 | Content safety | Light I/O | Opt-in |\n| 4 | Deep inspection | Heavy I/O | Opt-in + budget |\n\nEach tier is a `PolicyStage` with a `StageResult(passed, reason, metadata)`. The pipeline short-circuits on first failure ? no expensive work for already-rejected items. The accumulated `StageResult` list is the explanation trace.\n\n## Implementation phasing\n\nSplit into 7 independently-shippable follow-up issues (F1?F7) ? none bundled in this PR. The ADR defines the shared `PolicyStage` protocol they code against.\n\n## Backwards-compatible\n\nNo schema migration needed ? `policies.config_json` already exists; adding optional `topics`, `allowlist`, `security_checks`, `deep_inspect` keys is ignored by the existing `FilterConfig` parser.\n\n## Closes\n\nCloses #31 (Track F).\n\n## Related\n\n- #25 ? umbrella issue (this is the final track)\n- ADR 0003 ? metadata-first probing (Tier 4 explicitly
```

```
opts in to violating)\n- ADR 0011 ? domain schema (the `policies` table this extends)","changedFiles":1,"createdAt":"2026-07-06T10:33:03Z","deletions":0,"headRefName":"docs/track-f-policy-pipeline","isDraft":true,"mergeStateStatus":"UNKNOWN","mergeable":"UNKNOWN","number":40,"state":"OPEN","statusCheckRollup":[],"title":"docs(adr): propose 0012 policy pipeline ? explainable, tiered, user-customizable content gates (Track F)"}

```

### 330. assistant (2026-07-06T19:13:35.196Z)

PR #40 is a **draft**, docs-only PR ? the Track F design doc proposing **ADR 0012 (Policy Pipeline)**, +261/?0. So this is a design/ADR review, not a code review (no gate applies). Let me read the ADR and verify its checkable claims against the codebase.

### 331. user (2026-07-06T19:13:43.526Z)

```
=== changed file ===
docs/adr/0012-policy-pipeline.md

=== branch commits ===
e0f7b13 docs(adr): propose 0012 policy pipeline ? explainable, tiered, user-customizable content gates

=== does FilterConfig IGNORE or FORBID extra keys? (backwards-compat claim) ===
14:from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
44:     model_config = ConfigDict(frozen=True)
65:     model_config = ConfigDict(frozen=True)
88:     model_config = ConfigDict(frozen=True)
104:    model_config = ConfigDict(frozen=True)
131:    model_config = ConfigDict(frozen=True)
159:    model_config = ConfigDict(frozen=True)
196:    model_config = ConfigDict(frozen=True)
230:    model_config = ConfigDict(frozen=True)
253:    model_config = ConfigDict(frozen=True)
--- empirical: does FilterConfig.model_validate_json ignore unknown keys? ---
extra keys IGNORED ok -> max_size_mb= 100 | unknown keys dropped: True

```

### 332. assistant (2026-07-06T19:13:55.208Z)

Backwards-compat claim **verified** ? `FilterConfig` silently ignores unknown keys (Pydantic default `extra='ignore'`), so adding `topics`/`allowlist`/etc.` to `config_json` is dropped by the existing parser. (I'll flag a subtlety: because they're **dropped** not preserved, round-tripping through `save_filter_config` would erase them ? a real design concern.) Now let me read the full ADR.

### 333. user (2026-07-06T19:13:55.392Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\Telegram.

### 334. assistant (2026-07-06T19:14:01.889Z)

Right ? the file is on the branch, not `main`. Let me read it from the branch.

### 335. user (2026-07-06T19:14:03.427Z)

## ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table`)

## Context

The system today evaluates every matched video against a single `FilterConfig`` (per account, via `policies.config_json``). The filter is `flat``: all rules run unconditionally, the output is a boolean (`matched``), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a `policy pipeline`` ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

### What we have today (post-Track-E)

| Concept                                           | Current state                                                                                                        |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>FilterConfig`</code> / <code>Policy`</code> | One JSON blob per account. Flat duration/resolution/size/exclude rules. AND semantics.                               |
| <code>FilterEngine`</code>                        | Evaluates all rules; returns <code>FilterResult(passed, reasons)`</code> .                                           |
| <code>policies`</code> table                      | One row per account ( <code>is_default=1</code> ). <code>config_json`</code> is the <code>FilterConfig`</code> blob. |
| Security / topic / deep-inspect                   | Nothing. Codec/keyword/MIME filters and ffprobe were deferred to v2 in the original plan.                            |

### What we want

A `policy`` should be a pipeline of ordered `stages``, each with a well-defined cost, inputs, and explainable output. The pipeline runs `after`` dedup and `before`` delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view `why`` a specific item was accepted or rejected.

## Decision

### Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?           Policy Pipeline                                     ?
?                                                     ?
? Tier 1 ? Metadata gates  (always runs, zero I/O)         ?
?   duration, resolution, size, codec, mime, GIF/round      ?
?   ? extends today's FilterEngine                         ?
?                                                     ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)       ?
?   source allowlist, keyword match, channel topic tags    ?
?   ? user-expressed interests filter the stream           ?
?                                                     ?
? Tier 3 ? Content safety  (opt-in, metadata + light I/O)  ?
?   URL/domain reputation, file metadata sanity,          ?
?   caption text scanning                                  ?
?   ? cheap checks; no download                            ?
?                                                     ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)             ?
?   ffprobe, perceptual hash, downloaded content scan     ?
?   ? gated behind explicit user opt-in + budget          ?
????????????????????????????????????????????????????????????
...
```

### Stage interface ? `PolicyStage``

Each stage is a pure function (or async function for I/O stages) that

takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
    """Immutable snapshot of what the stage can inspect."""
    content_item: ContentItem          # from ADR 0011
    source: Source                     # the source it came from
    policy: Policy                     # the owning policy config

@dataclass
class StageResult:
    passed: bool
    reason: str                        # human-readable explanation
    stage_name: str                   # e.g. "duration-gate", "url-reputation"
    metadata: dict[str, object] = field(default_factory=dict) # for explainability UI

class PolicyStage(Protocol):
    """A single gate in the pipeline. Stateless; may be async for I/O stages."""
    async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
...```
```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config\_json` under a `topics` key:

```
```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
...```
```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted provider\_source\_ids).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content\_fingerprint` across accounts (already handled by `content\_items.dedup\_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked\_domain": "sketchy-site.com"}`).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated behind explicit opt-in** because it violates ADR 0003's no-download principle. The opt-in is per-policy (``deep_inspect: true`` in ``config_json``) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
```
```

The caller (telethon\_client or future Bot API handler) records the results via ``delivery_attempts.last_error`` (for failed items) or a new ``policy_decisions`` table (for explainability ? deferred to implementation).

## Schema ? minimal changes

**No schema migration** is required for this design. The ``policies`` table already holds a ``config_json`` JSON blob. The pipeline is a code concept:

- ``policies.config_json`` gains optional ``topics``, ``allowlist``, ``security_checks``, ``deep_inspect`` keys ? all backwards-compatible with the existing ``FilterConfig`` Pydantic model (extra fields are ignored).
- Future ``policy_decisions`` table (deferred) would store the explanation trace for debugging: ``(content_item_id, policy_id, stage_name, passed, reason, metadata_json, evaluated_at)``.

## Implementation phasing (future issues)

| Phase     | Scope                   | Dependencies                                                                                             |
|-----------|-------------------------|----------------------------------------------------------------------------------------------------------|
| ---       | ---                     | ---                                                                                                      |
| <b>F1</b> | Policy engine prototype | <code>`PolicyStage`</code> protocol, <code>`evaluate_pipeline()`</code> , wire into live-message handler |
| <b>F2</b> | Topic preferences       | <code>`topics`</code> config parsing, source title/caption keyword matching                              |
| <b>F3</b> | Source allowlist        | <code>`allowlist`</code> config, skip non-allowlisted sources                                            |
| <b>F4</b> | URL/domain reputation   | Domain blocklist, caption link extraction                                                                |
| <b>F5</b> | Deep inspection         | ffprobe/perceptual hash, download budget enforcement                                                     |
| <b>F6</b> | Policy decisions table  | Audit trail storage, explainability UI                                                                   |
| <b>F7</b> | Explainability endpoint | <code>`/why &lt;content_item_id&gt;`</code> command showing the explanation trace                        |

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the **design contract** the phases code against.

## Consequences

#### **\*\*Positive\*\***

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

#### **\*\*Negative\*\***

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

### **Alternatives considered**

- **\*\*Do nothing; keep flat FilterConfig.\*\*** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- **\*\*Inline each stage in the handler\*\*** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- **\*\*Use a rules engine / DSL.\*\*** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- **\*\*Bundle all phases into a single PR.\*\*** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

### **Open questions for the decision owner**

1. **\*\*`StageResult.metadata` shape\*\*** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. **\*\*Policy decisions table\*\*** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. **\*\*Topic matching\*\*** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. **\*\*Deep inspection budget\*\*** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

### **References**

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

### 336. assistant (2026-07-06T19:15:17.629Z)

I've read the full ADR. It's a well-structured design doc (Proposed) with a sound cost-ordered short-circuit model ? but I've found several concrete design gaps, including one verified against the code. Given the mandate, let me run a focused workflow to independently critique the design, cross-check against all existing ADRs, and verify my findings.

### 337. user (2026-07-06T19:16:00.101Z)

Workflow launched in background. Task ID: wrqsbma40

Summary: Design review of telegram-video-filter PR #40 (draft, docs-only): ADR 0012 policy pipeline ? soundness, cross-ADR consistency, claim accuracy, with adversarial verification

Transcript dir: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_e1b943dd-dac

Script file: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-

Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr40-adr0012-wf\_e1b943dd-dac.js

(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:

"C:\Users\avidu\claude\projects\C--Users-avidu-Projects-

Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr40-adr0012-wf\_e1b943dd-dac.js"} to iterate without resending the script.)

Run ID: wf\_e1b943dd-dac

To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\claude\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr40-adr0012-wf\_e1b943dd-dac.js", resumeFromRunId: "wf\_e1b943dd-dac"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

### 338. assistant (2026-07-06T19:16:11.868Z)

While the workflow runs, let me verify two more of my findings: that `content\_item.text` (needed by Tier 2/3 caption matching) is never populated today, and the config round-trip erasure.

### 339. user (2026-07-06T19:18:20.938Z)

Command running in background with ID: b8au8166l. Output is being written to:

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b8au8166l.output. You will be notified when it completes. To check interim output, use Read on that file path.

### 340. user (2026-07-06T19:18:31.388Z)

=== is content\_items.text ever written? (Tier 2/3 caption dependency) ===

```
src/tg_video_filter/db.py:865:         text=row["text"],
```

```
--- does the live path set text on ContentItem? ---
```

```
858:     return ContentItem(
```

```
1620:     item = ContentItem(
```

=== round-trip erasure: a policy config\_json with topics, run through load->save ===

### 341. assistant (2026-07-06T19:18:47.346Z)

`content\_item.text` is only ever \*read\* (`\_row\_to\_content\_item`), never written with a real value ? confirming Tier 2/3 caption matching depends on a future track populating it. The round-trip test output got swallowed; let me re-run it cleanly.

### 342. user (2026-07-06T19:20:53.040Z)

Command running in background with ID: brqu628sa. Output is being written to:

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\brqu628sa.output. You will be notified when it completes. To check interim output, use Read on that file path.

### 343. assistant (2026-07-06T19:20:57.967Z)

The workflow confirmed all 19 findings (0 refuted), corroborating my two headline concerns. Let me parse the full deduplicated set.

### 344. user (2026-07-06T19:21:05.164Z)

=== round-trip result ===

=== workflow findings ===

COUNTS: {'raw': 19, 'kept': 19, 'refuted': 0}

=====

[HIGH/CONFIRMED] (design) "Extra fields are ignored" means DROPPED, not preserved ? the entire config-storage design silently loses topics/allowlist/deep\_inspect

§ Decision ? Schema ? minimal changes; and Open questions (backwards-compat claim)

rec: The ADR must specify HOW Track-F keys are persisted. Either (a) extend the FilterConfig model with typed optional topics/allowlist/security\_checks/deep\_inspect fields (then it's a

[HIGH/CONFIRMED] (design) StageResult recorded via delivery\_attempts.last\_error 'for failed items' is a category error ? rejected items short-circuit before any delivery\_attempts row exists

§ Decision ? Policy evaluation flow (records results via delivery\_attempts.last\_er

rec: Drop the delivery\_attempts.last\_error path for policy rejections. State that the explanation trace for rejected items has NO persistence in F1?F5 (in-memory/log only) and that pers

[HIGH/CONFIRMED] (cross\_adr) "Extra fields are ignored" means topic/security/deep\_inspect config is SILENTLY DROPPED, not preserved ? save round-trip destroys it

§ Schema ? minimal changes; Consequences ("Backwards-compatible")

rec: Change the claim to reflect reality and specify the mechanism: either (a) set model\_config = ConfigDict(extra='allow') on FilterConfig AND stop round-tripping through it in the shi

[HIGH/CONFIRMED] (cross\_adr) Tier 2 and Tier 3 caption-text stages depend on data the pipeline is never given: captions are dropped at ingestion and PolicyContext has no VideoMeta

§ Topic preferences ? Tier 2 ("message caption keyword match"); Security ? Tier 3

rec: Add an explicit precondition/dependency: "Capturing caption text at ingestion (VideoMeta.text or event.message.raw\_text -> ContentItem.text) is a prerequisite for F2/F4 and is not

[HIGH/CONFIRMED] (claims) 'Extra fields are ignored' is presented as purely positive, but the existing save path erases the new config keys ? any /set silently wipes the entire pipeline config

§ Schema ? minimal changes / Consequences (Positive) ? 'backwards-compatible with

rec: Reframe the 'extra fields ignored' bullet as a two-sided fact and add an explicit note under 'Schema ? minimal changes' that F1 must (a) add the new keys to a parseable model (exte

[MEDIUM/CONFIRMED] (design) PolicyContext has no client/message handle ? Tier 3 and Tier 4 cannot perform the I/O the ADR assigns them

§ Decision ? Stage interface (PolicyContext); Tier 3; Tier 4

rec: Add a fetch/client capability to the contract for I/O tiers ? e.g. an optional `fetcher`/`client` handle or a `download(budget)` callable on PolicyContext, or split the protocol so

[MEDIUM/CONFIRMED] (design) Tier 2/3 caption/keyword matching depends on ContentItem.text, which is NULL in the live path ? an unstated hard dependency

§ Decision ? Topic preferences (Tier 2); Security stages (Tier 3, caption text sca

rec: Add an explicit dependency/pre-req to the phasing table: 'F2/F4 require ContentItem.text populated from the message caption in the ingestion hot path (new work, not delivered by Tr

[MEDIUM/CONFIRMED] (design) F1 'wire into live-message handler' understates the work: the live path uses the legacy VideoRecord shim, not ContentItem/Source/Policy that PolicyContext requires

§ Implementation phasing ? F1 (Dependencies: None)

rec: Note in F1 that it depends on obtaining Source + inserted ContentItem + resolved Policy in the handler (or on migrating the handler off the VideoRecord shim), and on how the applic



[MEDIUM/CONFIRMED] (design) AND-only short-circuit pipeline cannot express include/exclude topic semantics or OR/priority rules ? expressiveness gap not acknowledged

§ Decision ? Stage model (AND-only short-circuit) vs Topic preferences (include/ex

rec: Acknowledge the AND-only limitation and specify how within-tier boolean logic (include OR, exclude veto, allowlist bypass) is handled ? either as documented single composite stages

[MEDIUM/CONFIRMED] (design) Tier 3 'Duplicate suppression' stage overlaps/contradicts the already-shipped dedup that runs BEFORE the pipeline

§ Decision ? Security stages (Tier 3) item 3, Duplicate suppression

rec: Either drop Tier-3 duplicate suppression (covered by pre-pipeline dedup\_key) or specify the exact new scope (cross-destination within an account), where it reads that state from (d

[MEDIUM/CONFIRMED] (cross\_adr) Tier 3 "Source allowlist" is dead code ? it duplicates sources.watched (ADR 0009 / Track E) and can never fire

§ Security stages ? Tier 3, item 1 ("Source allowlist"); phasing F3

rec: State the overlap explicitly and resolve it: either remove the Tier-3 source-allowlist stage (watched already is the source gate), or scope it to something orthogonal ? e.g. a per-

[MEDIUM/CONFIRMED] (cross\_adr) Tier 4 references ADR 0003 but does not propose the amendment/waiver it needs; the phasing table cites a nonexistent "ADR 0003 waiver"

§ Deep inspection ? Tier 4; phasing F5 ("Dependencies: F1, ADR 0003 waiver"); Refe

rec: Either add a section to this ADR that formally amends ADR 0003 ("ADR 0003 is amended: ffp/ffprobe/download is permitted for content that passes Tiers 1-3 when the owning policy sets de

[MEDIUM/CONFIRMED] (cross\_adr) References mis-places "topics.allowlist" (Tier 2) while the body puts "Source allowlist" under Tier 3 security ? internally inconsistent

§ References (ADR 0009 line) vs. Security stages ? Tier 3; also the Tier-2 ASCII b

rec: Pick one home and one key name. Given Finding 2 (watched already is the source gate), the cleanest fix is to drop "source allowlist" from both Tier 2 and Tier 3 and correct the Ref

[MEDIUM/CONFIRMED] (cross\_adr) "Record failed policy decisions via delivery\_attempts.last\_error" conflicts with Track C's record\_delivery contract (delivery outcome, not filter outcome)

§ Policy evaluation flow ("records the results via delivery\_attempts.last\_error (f

rec: Drop the delivery\_attempts.last\_error path for policy rejections. State that rejected items are simply not delivered (and, when F6 lands, recorded in policy\_decisions); delivery\_at

[MEDIUM/CONFIRMED] (claims) 'topics' example contradicts the prose (array vs object) and is invalid JSON (contains a // comment)

§ Topic preferences ? Tier 2

rec: Fix the prose to say 'stored as a JSON object' (or restructure to an actual array if that was intended), and remove the // inline comment from the example ? move the mode explanati

[MEDIUM/CONFIRMED] (claims) Tier 2/3 rely on caption text, but ContentItem.text is never populated on the live path ? F2/F4 have an undocumented upstream dependency

§ Topic preferences ? Tier 2 ('message caption keyword match') and Security stages

rec: Add an explicit dependency note: Tier 2 caption keyword match, Tier 3 caption scanning, and F4 link extraction require ContentItem.text to be populated at ingestion, which today it

[LOW/CONFIRMED] (design) F3 'skip non-allowlisted sources' duplicates Track E's watched gate that already runs before the pipeline

§ Decision ? Security stages (Tier 3) item 1; phasing F3; References (ADR 0009 gen

rec: Clarify in F3 that watched (Track E) is the ingestion gate and the Tier-3 allowlist is a narrower per-policy refinement over already-watched sources ? or fold allowlist into routin

[LOW/CONFIRMED] (cross\_adr) Tier 3 "Duplicate suppression" stage overlaps Track B/E dedup that already runs before the pipeline, and its "cross-policy" claim has no supporting schema

§ Security stages ? Tier 3, item 3 ("Duplicate suppression")

rec: Either drop the stage (dedup is a pre-pipeline concern owned by Track B) or specify the cross-destination mechanism concretely (e.g. "suppress if a delivered delivery\_attempts row

distinct findings after de-dupe: 18 of 19

### 345. assistant (2026-07-06T19:22:02.463Z)

The workflow confirmed all 19 findings (0 refuted; they dedupe to ~9 distinct issues),

corroborating my two headline concerns and adding several. Three independent agents empirically verified the config round-trip erasure, so that one's rock-solid. Here's the consolidated design review.

---

## Design review ? PR #40 (draft, docs-only): ADR 0012 Policy Pipeline

**\*\*This is a design doc (Proposed), so no gate applies ? feedback below is on the design contract.\*\*** The core model is sound: a **\*\*cost-ordered, short-circuiting four-tier pipeline\*\*** with a ``PolicyStage`` protocol and a ``StageResult`` explanation trace is a clean, testable generalization of today's flat filter, and the F1?F7 phasing + "alternatives considered" show good discipline. But several load-bearing claims and interfaces have gaps that would break the F-phases if coded as written. Everything below was verified against the code / other ADRs.

### ? High ? resolve before the F-phases

**\*\*1.** The backwards-compat claim is inverted ? "extra fields are ignored" means **\*dropped\***, so the config never persists.

The ADR's central compat mechanism ("adding optional ``topics`/`allowlist`/`security_checks`/`deep_inspect`` keys ? backwards-compatible, extra fields ignored") is the **\*opposite\*** of what's needed. ``FilterConfig`` uses Pydantic's default ``extra='ignore'``, which **\*\*discards\*\*** unknown keys. So the moment a user runs any ``/set``, ``load_filter_config`` parses ``config_json`` ? ``FilterConfig`` (dropping the pipeline keys) and ``save_filter_config`` re-serializes via ``model_dump_json()`` ? **\*\*silently erasing the entire Track-F config\*\***. Verified empirically (``FilterConfig`` drops the keys on round-trip). It's rescuable (``upsert_policy`` writes ``config_json`` verbatim, so a raw-dict path survives), but the ADR directs implementers down the shim path that loses data and specifies no alternative.

> **\*\*Fix:\*\*** state the persistence mechanism explicitly ? either extend ``FilterConfig`` with typed optional fields (a real ``models.py`` change, contradicting "code concept only"), set ``extra='allow'`` and stop round-tripping through it, or parse ``config_json`` as a raw dict. As written it **\*guarantees\*** config loss.

**\*\*2.** Storing rejection reasons in ``delivery_attempts.last_error`` is a category error.

Rejected items short-circuit **\*before\*** delivery, so there's no ``(content_item_id, destination_id)`` pair and no ``delivery_attempts`` row to write to ? ``record_delivery`` (Track C) requires a ``destination_id`` and only runs on the delivery path. It also conflates "delivery transport failed" with "policy rejected." The only viable store is the deferred ``policy_decisions`` table (F6) ? which means **\*\*F6 is a hard dependency of any persisted explainability, and F7's ``/why`` has nothing to read until F6 lands.\*\***

> **\*\*Fix:\*\*** drop the ``last_error`` path; say rejected-item traces are in-memory/log-only until F6, and fix the phasing dependencies.

**\*\*3.** Tier 2/3 depend on caption text that the pipeline is never given.

Tier 2 ("message caption keyword match"), Tier 3 ("caption text scanning"), and F4 (link extraction) all need ``content_item.text`` ? but it's **\*\*never populated on the live path\*\*** (verified: only read, never written; ADR 0011 left it NULL for Phase 1). Capturing captions at ingestion is unstated new work these phases silently depend on.

### ? Medium

**\*\*4.** ``PolicyContext`` can't support the I/O tiers it defines.

It holds only ``(content_item, source, policy)`` ? no client/message handle ? so Tier 3 (light I/O) and Tier 4 (download/ffprobe) can't fetch anything. Add a ``fetcher`/`download(budget)`` capability, or split the protocol for I/O stages.

**\*\*5.** Tier 3 "Source allowlist" duplicates ``sources.watched`` (Track E, just merged).

An unwatched source is never ingested, so a Tier-3 allowlist never sees it ? dead stage. And the References call it ``topics.allowlist`` (Tier 2) while the body puts "Source allowlist" in Tier 3 ? pick one home, or drop it (watched **\*is\*** the source gate).

**\*\*6.** Tier 3 "Duplicate suppression" overlaps the pre-pipeline dedup

(``content_items.dedup_key``, Track B, runs before the pipeline). The "cross-policy/cross-

destination" scope has no supporting schema ? drop it or specify it concretely.

**\*\*7.** F1's "Dependencies: None" understates the work.\*\* The live handler still runs on the legacy `VideoRecord` shim, not the `ContentItem`/`Source`/`Policy` that `PolicyContext` requires ? F1 first has to thread those through the hot path.

**\*\*8.** AND-only short-circuit vs. include/exclude/OR topic semantics.\*\* The pipeline is pure AND across tiers, but topics need include-OR / exclude-veto logic. It's handled \*within\* the Tier-2 stage ? the ADR should say so explicitly rather than leave the tension.

**\*\*9.** Tier 4 references ADR 0003 but never amends it.\*\* The phasing cites a nonexistent "ADR 0003 waiver." Add a formal "ADR 0003 is amended for opt-in deep inspection under budget" section rather than an informal reference.

## ? Low

- **\*\*10.\*\*** The `topics` example contradicts the prose (says "JSON array," shows an object) and contains a `///<` comment ? **\*\*invalid JSON\*\*** for a real `config\_json`.

## ? Worth keeping (design strengths)

Cost-ordered short-circuit is the right instinct; `PolicyStage`/`StageResult` is clean and independently testable; "no migration, `config\_json` already exists" is \*structurally\* true (only the round-trip is broken); phasing and the rejected-alternatives (rules-engine/DSL for <100 users) are well-judged.

---

**\*\*Bottom line for a draft:\*\*** the skeleton is good; the two things that most need fixing before F1 are the **\*\*config persistence mechanism\*\*** (#1) and the **\*\*explainability storage\*\*** (#2), plus wiring the **\*\*caption/context inputs\*\*** (#3/#4) into the contract so F2?F5 have what they read.

Want me to **\*\*post this as a review comment\*\*** on the draft PR (framed as design feedback)? Since it's a draft and you only said "review," I've held off posting ? say the word and I'll drop it.

**346. user (2026-07-06T19:52:28.071Z)**

Please post this as review comment and thanks for the diligent work

**347. assistant (2026-07-06T19:52:34.205Z)**

Posting now ? thank you, this has been a great series to work through.

**348. assistant (2026-07-06T19:52:56.662Z)**

## Design review ? ADR 0012 Policy Pipeline (draft, docs-only)

Reviewing this as a **\*\*design contract\*\*** (no code, so no test gate). The core model is sound ? a **\*\*cost-ordered, short-circuiting four-tier pipeline\*\*** with a `PolicyStage` protocol and a `StageResult` explanation trace is a clean, testable generalization of today's flat filter, and the F1?F7 phasing + "alternatives considered" show good discipline. But several load-bearing claims and interfaces have gaps that would break the F-phases if coded as written. Everything below was verified against the code / the other ADRs.

## ? High ? resolve before the F-phases

**\*\*1.** The backwards-compat claim is inverted ? "extra fields are ignored" means \*dropped\*, so pipeline config never persists.\*\*  
`FilterConfig` uses Pydantic's default `extra='ignore'`, which **\*\*discards\*\*** unknown keys. So the moment a user runs any `/set`, `load\_filter\_config` parses `config\_json` ? `FilterConfig` (dropping `topics`/`allowlist`/`security\_checks`/`deep\_inspect`) and `save\_filter\_config` re-serializes via `model\_dump\_json()` ? **\*\*silently erasing the entire Track-F config\*\***. Verified empirically (the keys don't survive the round-trip). It's rescuable ? `upsert\_policy` writes `config\_json` verbatim, so a raw-dict path survives ? but the ADR directs implementers down the shim path that loses data and specifies no alternative.

> **Fix:** state the persistence mechanism explicitly ? extend `FilterConfig` with typed optional fields (a real `models.py` change, contradicting "code concept only"), set `extra='allow'` and stop round-tripping through it, or parse `config_json` as a raw dict. As written it guarantees config loss.

**2.** Storing rejection reasons in `delivery_attempts.last_error` is a category error.  
Rejected items short-circuit *before* delivery, so there's no `(content_item_id, destination_id)` pair and no `delivery_attempts` row to write to ? `record_delivery` (Track C) requires a `destination_id` and only runs on the delivery path. It also conflates "delivery transport failed" with "policy rejected." The only viable store is the deferred `policy_decisions` table (F6) ? so **F6** is a hard dependency of any persisted explainability, and **F7's** `/why` has nothing to read until **F6** lands.  
> **Fix:** drop the `last_error` path; state that rejected-item traces are in-memory/log-only until **F6**, and fix the phasing dependencies.

**3.** Tier 2/3 depend on caption text the pipeline is never given.  
Tier 2 ("message caption keyword match"), Tier 3 ("caption text scanning"), and **F4** (link extraction) all need `content_item.text` ? but it's *never* populated on the live path (verified: only read, never written; **ADR 0011** left it NULL for Phase 1). Capturing captions at ingestion is unstated new work these phases silently depend on.

## ? Medium

**4.** `PolicyContext` can't support the I/O tiers it defines. It holds only `(content_item, source, policy)` ? no client/message handle ? so Tier 3 (light I/O) and Tier 4 (download/ffprobe) can't fetch anything. Add a `fetcher/download(budget)` capability, or split the protocol for I/O stages.

**5.** Tier 3 "Source allowlist" duplicates `sources.watched` (Track E, just merged). An unwatched source is never ingested, so a Tier-3 allowlist never sees it ? dead stage. And the References call it `topics.allowlist` (Tier 2) while the body puts "Source allowlist" in Tier 3 ? pick one home, or drop it (watched *is* the source gate).

**6.** Tier 3 "Duplicate suppression" overlaps the pre-pipeline dedup (`content_items.dedup_key`, Track B, runs before the pipeline). The "cross-policy/cross-destination" scope has no supporting schema ? drop it or specify it concretely.

**7.** **F1's** "Dependencies: None" understates the work. The live handler still runs on the legacy `VideoRecord` shim, not the `ContentItem/Source/Policy` that `PolicyContext` requires ? **F1** first has to thread those through the hot path.

**8.** AND-only short-circuit vs. include/exclude/OR topic semantics. The pipeline is pure AND across tiers, but topics need include-OR / exclude-veto logic. It's handled *within* the Tier-2 stage ? the **ADR** should say so explicitly rather than leave the tension.

**9.** Tier 4 references **ADR 0003** but never amends it. The phasing cites a nonexistent "ADR 0003 waiver." Add a formal "ADR 0003 is amended for opt-in deep inspection under budget" section rather than an informal reference.

## ? Low

**10.** The `topics` example contradicts the prose (says "JSON array," shows an object) and contains a `///` comment ? **invalid JSON** for a real `config_json`.

## ? Worth keeping

Cost-ordered short-circuit is the right instinct; `PolicyStage/StageResult` is clean and independently testable; "no migration, `config_json` already exists" is *structurally* true (only the round-trip is broken); the phasing and rejected alternatives (rules-engine/DSL for <100 users) are well-judged.

---

**Bottom line for a draft:** the skeleton is good. The two things that most need fixing before **F1** are the **config persistence mechanism** (#1) and the **explainability storage** (#2), plus wiring the **caption/context inputs** (#3/#4) into the contract so **F2?F5** have what they read.

<sub>? Design review with [Claude Code](https://claude.com/claude-code). Claims verified against the code and ADRs 0003/0009/0011. Multi-agent adversarial pass: 19 findings ? ~9 distinct, 0 refuted.</sub>

**349. user (2026-07-06T19:52:56.708Z)**

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr40\_review.md (file state is current in your context ? no need to Read it back)

**350. user (2026-07-06T19:53:03.425Z)**

<https://github.com/avidullu/telegram-video-filter/pull/40#issuecomment-4897017225>

**351. assistant (2026-07-06T19:53:12.736Z)**

? \*\*Posted to draft PR #40\*\* ? [issuecomment-4897017225](https://github.com/avidullu/telegram-video-filter/pull/40#issuecomment-4897017225)

The design review is up ? framed as feedback on the draft, leading with what's sound and ending with the two things to nail down before F1 (config persistence #1, explainability storage #2, plus wiring caption/context inputs #3/#4).

And thank you ? this was a genuinely satisfying series to review end-to-end. For the record, where the whole provider-neutral umbrella landed:

|     | PR   | Track                      | Outcome                                      |
|-----|------|----------------------------|----------------------------------------------|
|     | ---- | -----                      | -----                                        |
| #35 | B    | ? domain schema            | blocker + dedup regression ? fixed ? merged  |
| #37 | C    | ? delivery/routing         | 2 mediums ? fixed ? merged                   |
| #38 | D    | ? CommandService           | clean refactor ? merged                      |
| #39 | E    | ? selective scanning       | blocker ? fixed, but introduced a regression |
| #41 | E    | follow-up (my fix)         | watched-clobber fixed ? **merged**           |
| #40 | F    | ? policy pipeline (design) | draft; design review posted                  |

Every code finding was reproduced locally, and the two fixes I authored (#36 format, #41 watched) were verified before merge. Nothing's left open on my side ? ping me when the F-phases (F1?F7) start landing as real code, or whenever the next PR needs eyes.

**352. user (2026-07-06T20:32:25.775Z)**

Can you take another pass on #40?

**353. assistant (2026-07-06T20:32:35.203Z)**

Let me sync and check whether the ADR was updated since my review.

**354. user (2026-07-06T20:32:44.556Z)**

```
From https://github.com/avidullu/telegram-video-filter
+ e0f7b13...401ec47 docs/track-f-policy-pipeline -> origin/docs/track-f-policy-pipeline
(forced update)
---checkout---
main
main...origin/main
```

```
=== PR #40 state + new commits since my review (e0f7b13) ===
state=OPEN draft=true branch=docs/track-f-policy-pipeline updated=2026-07-06T20:30:05Z
--- commits on branch ---
401ec47 docs(adr): address policy pipeline review gaps
f34a571 docs(adr): propose 0012 policy pipeline ? explainable, tiered, user-customizable content
gates
--- new since e0f7b13? ---
```

```
401ec47 docs(adr): address policy pipeline review gaps
f34a571 docs(adr): propose 0012 policy pipeline ? explainable, tiered, user-customizable content
gates
c4c4d18 fix(scanning): don't clobber user-controlled watched on source upsert (#41)
(empty = no new commits)
```

=== response comments since my review? ===

### 355. assistant (2026-07-06T20:32:52.921Z)

The branch was rebased and got a new commit `**`401ec47` "address policy pipeline review gaps."**`  
So the author revised the ADR in response to my feedback. Let me read exactly what changed and re-verify each finding against the revised text.

### 356. user (2026-07-06T20:32:55.102Z)

```
=== diff of the 'address review gaps' commit (f34a571..401ec47) ===
diff --git a/docs/adr/0012-policy-pipeline.md b/docs/adr/0012-policy-pipeline.md
index a62de58..d5d86d1 100644
--- a/docs/adr/0012-policy-pipeline.md
+++ b/docs/adr/0012-policy-pipeline.md
@@ -25,6 +25,8 @@ them, and defer expensive work until after cheap rejections.
| `FilterConfig` / `Policy` | One JSON blob per account. Flat duration/resolution/size/exclude
rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config_json` is the `FilterConfig`
blob. |
+| Config persistence shims | `load_filter_config` / `save_filter_config` parse and write only
the flat `FilterConfig` shape. Unknown future keys are not preserved by this path. |
+| `content_items.text` / `link_url` | Columns exist from ADR 0011, but the live Telegram path
does not populate captions or extracted links yet. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were
deferred to v2 in the original plan. |

### What we want
@@ -51,14 +53,14 @@ Later stages only run if all earlier stages pass:
? duration, resolution, size, codec, mime, GIF/round ?
? ? extends today's FilterEngine ?
? ?
-? Tier 2 ? Topic preferences (opt-in, metadata-only) ?
-? source allowlist, keyword match, channel topic tags ?
+? Tier 2 ? Topic preferences (opt-in, captured metadata) ?
+? source title, caption keyword match, channel tags ?
? ? user-expressed interests filter the stream ?
? ?
-? Tier 3 ? Content safety (opt-in, metadata + light I/O) ?
+? Tier 3 ? Content safety (opt-in, captured text/links) ?
? URL/domain reputation, file metadata sanity, ?
? caption text scanning ?
-? ? cheap checks; no download ?
+? ? cheap checks; no download or page fetch ?
? ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O) ?
? ffprobe, perceptual hash, downloaded content scan ?
@@ -78,6 +80,8 @@ class PolicyContext:
content_item: ContentItem # from ADR 0011
source: Source # the source it came from
policy: Policy # the owning policy config
+ fetcher: ContentFetcher | None = None # only present for opt-in I/O stages
+ budget: PolicyBudget | None = None # bytes/time/concurrency limits for I/O

@dataclass
class StageResult:
@@ -91,6 +95,11 @@ class PolicyStage(Protocol):
    async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
```

...

+`ContentFetcher` is an implementation detail introduced in F5, not a requirement for metadata-only stages. F1-F4 must work when `ctx.fetcher` is `None`; Tier 4 is the first tier allowed to use it, and only after checking +`ctx.budget`.

+

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is @@ -98,7 +107,7 @@ auditable.

### ### Topic preferences ? Tier 2

-Topic preferences are stored as a JSON array in `policies.config\_json`  
+Topic preferences are stored as an object in `policies.config\_json` under a `topics` key:

```json

@@ -107,29 +116,40 @@ under a `topics` key:

```
"topics": {
  "include": ["AI", "Philosophy", "Soccer"],
  "exclude": ["Politics", "Crypto"],
-  "mode": "any" // "any" = match any include topic; "all" = match all
+  "mode": "any"
}
```

}```

-Topic matching is **metadata-only** in v1: source title substring match, -message caption keyword match, and (future) channel topic tags from -Telegram's channel metadata. No ML classification in the design ? that's -a separate issue.  
+`mode` controls include semantics: `any` means at least one include topic +must match; `all` means every include topic must match. Excludes are always +a veto. The pipeline itself remains AND-only across stages, but the Topic +stage owns its internal include/exclude/OR logic and returns one +`StageResult`.

+

+Topic matching is **captured-metadata-only** in v1: source title substring +match, captured message caption keyword match, and (future) channel topic +tags from Telegram's channel metadata. No ML classification in the design ? +that's a separate issue. Because the current live path does not populate +`content\_items.text`, F2 includes the ingestion work to capture +`message.message`/caption text when inserting the `ContentItem`.

### ### Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design -defines three initial stages (implementation deferred to separate issues): +defines two initial stages (implementation deferred to separate issues):

- 1. **Source allowlist**: reject items from sources not in the `allowlist`  
- (an explicit list of trusted provider\_source\_ids).
- 2. **URL/domain reputation**: check links in captions against a
- +1. **URL/domain reputation**: check links in captured captions against a configurable blocklist. Metadata-only ? no page fetch.
- 3. **Duplicate suppression**: enhanced dedup that compares by  
- `content\_fingerprint` across accounts (already handled by  
- `content\_items.dedup\_key`; this stage adds cross-policy duplicate  
- detection within an account's delivery streams).
- +2. **Caption text scanning**: reject captions containing configured unsafe  
+ terms or patterns.

+

+Source selection is **not** a policy stage: ADR 0009's `sources.watched`

+gate decides whether a source is ingested at all, and `routing\_rules` decide  
+which destination/policy applies after ingestion. Duplicate suppression is  
+also not a policy stage in this ADR: ADR 0011's `content\_items.dedup\_key`  
+runs before the pipeline. A future cross-destination duplicate policy would  
+need its own schema and is out of scope here.

Each security stage records its decision in `StageResult.metadata` for  
explainability (e.g. `{"blocked\_domain": "sketchy-site.com"}`).

@@ -146,6 +166,20 @@ This tier is explicitly deferred to a follow-up issue ? the design only  
\*\*reserves the slot\*\* in the pipeline model so it doesn't require a schema  
migration when implemented.

##### ADR 0003 amendment for Tier 4

+  
+If this ADR is accepted, it amends ADR 0003 only for Tier 4: downloading or  
+probing content is allowed when all of the following are true:

+  
+- The policy explicitly opts in (`deep\_inspect: true`).  
+- The item has already passed Tiers 1-3.  
+- The stage receives a bounded `ContentFetcher` and an enforced  
+ `PolicyBudget`.  
+- The result is recorded as a policy decision, not as a delivery failure.

+  
+All earlier tiers continue to honor ADR 0003's metadata-first/no-download  
+default.

+  
### Policy evaluation flow (post-dedup, pre-delivery)

```
```python
@@ -162,19 +196,31 @@ async def evaluate_pipeline(
    return True, results
```
```

-The caller (telethon\_client or future Bot API handler) records the  
-results via `delivery\_attempts.last\_error` (for failed items) or a new  
-`policy\_decisions` table (for explainability ? deferred to implementation).  
+The caller (Telethon userbot path or future Bot API handler) uses the final  
+boolean to decide whether delivery should run. Rejected items short-circuit  
+before destination resolution and delivery, so they do \*\*not\*\* have a  
+`delivery\_attempts` row and must not be recorded in  
+`delivery\_attempts.last\_error`.

-### Schema ? minimal changes  
+Until F6 lands, rejected-item traces are in-memory/log-only. F6 introduces  
+the persistent `policy\_decisions` table; F7 depends on F6 because `/why`  
+needs that table to read historical decisions.

-\*\*No schema migration\*\* is required for this design. The `policies` table  
-already holds a `config\_json` JSON blob. The pipeline is a code concept:  
+### Schema ? minimal changes

-- `policies.config\_json` gains optional `topics`, `allowlist`,  
- `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with  
- the existing `FilterConfig` Pydantic model (extra fields are ignored).  
-- Future `policy\_decisions` table (deferred) would store the explanation  
+\*\*No migration is required for the policy configuration shape itself.\*\* The  
+`policies` table already holds a `config\_json` JSON blob. The pipeline adds  
+keys inside that blob:  
+  
+- `policies.config\_json` gains optional `topics`, `security\_checks`, and  
+ `deep\_inspect` keys.  
+- The legacy `load\_filter\_config` / `save\_filter\_config` shim is not a safe  
+ persistence path for those keys. It parses through `FilterConfig` and then  
+ serializes `FilterConfig` again, so unknown keys would be dropped. F1 must  
+ add a policy-config persistence path that reads/writes the raw JSON object



- + (or a typed superset model) and preserves unknown keys when legacy ``/set``
- + commands update flat ``FilterConfig`` fields.
- + Future ``policy_decisions`` table (F6) stores the explanation trace for debugging: ``(content_item_id, policy_id, stage_name, passed, reason, metadata_json, evaluated_at)``.

@@ -182,12 +228,12 @@ already holds a ``config_json`` JSON blob. The pipeline is a code concept:

```
| Phase | Scope | Dependencies |
|---|---|---|
-| **F1** ? Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, wire into
live-message handler | None |
-| **F2** ? Topic preferences | `topics` config parsing, source title/caption keyword matching |
F1 |
-| **F3** ? Source allowlist | `allowlist` config, skip non-allowlisted sources | F1 |
-| **F4** ? URL/domain reputation | Domain blacklist, caption link extraction | F1 |
-| **F5** ? Deep inspection | ffprobe/perceptual hash, download budget enforcement | F1, ADR
0003 waiver |
-| **F6** ? Policy decisions table | Audit trail storage, explainability UI | F1 |
+| **F1** ? Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, raw/typed
policy-config persistence, and hot-path wiring from live handler to
`ContentItem`/`Source`/`Policy` context | Tracks B-E landed |
+| **F2** ? Topic preferences | `topics` config parsing, source title matching, caption capture
into `content_items.text`, caption keyword matching | F1 |
+| **F3** ? Caption safety | Unsafe caption term/pattern scanning from captured text | F1, F2
caption capture |
+| **F4** ? URL/domain reputation | Caption link extraction into `content_items.link_url` or
stage metadata, domain blacklist checks with no page fetch | F1, F2 caption capture |
+| **F5** ? Deep inspection | `ContentFetcher`, ffprobe/perceptual hash, download budget
enforcement | F1, accepted ADR 0003 amendment above |
+| **F6** ? Policy decisions table | Persistent audit trail storage for accepted and rejected
items | F1 |
| **F7** ? Explainability endpoint | `/why <content_item_id>` command showing the explanation
trace | F6 |
```

Each phase is a separate, mergeable PR ? none are bundled here. This ADR

@@ -200,10 +246,10 @@ only defines the **design contract** the phases code against.

- framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- ``StageResult`` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: ``policies.config_json`` already exists; adding optional keys requires no migration.
- + ``StageResult`` list provides an in-memory explanation trace at decision time; F6 persists that trace for later ``/why`` lookups.
- + Backwards-compatible at the schema level: ``policies.config_json`` already exists, so adding optional policy keys requires no config-shape migration.
- Each implementation phase is independently shippable behind feature flags.

@@ -211,13 +257,18 @@ only defines the **design contract** the phases code against.

- Four tiers add conceptual complexity vs. today's flat filter.
- ``PolicyStage`` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks
- (metadata-only vs. I/O-heavy). Mitigated by the ``metadata`` dict
- catch-all.
- + (metadata-only vs. I/O-heavy). Mitigated by keeping F1-F4 fetcher-free and making Tier 4's fetcher/budget explicit.
- + Backwards compatibility is not automatic at the repository-shim level:
- + F1 must avoid dropping pipeline keys when legacy config commands rewrite the default policy.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.

- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.
- + Caption and link stages require ingestion changes because the live path does not currently write `content\_items.text` or `content\_items.link\_url`.
- + Deep inspection amends ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

## Alternatives considered

@@ -253,8 +304,8 @@ only defines the **design contract** the phases code against.

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- + ADR 0009 ? selective channel scanning (`sources.watched` stays the ingestion gate, outside the policy pipeline).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).

### 357. assistant (2026-07-06T20:33:33.768Z)

The author's revision is thorough ? at a glance it addresses all 10 of my findings. Per the gate, let me verify the **resulting** ADR is internally consistent and didn't leave dangling references (not just trust the diff).

### 358. user (2026-07-06T20:33:39.964Z)

=== leftover 'allowlist' references (should be gone/reconciled)? ===  
none ?

=== 'delivery\_attempts.last\_error' still claimed as a store? ===  
203: `delivery\_attempts.last\_error`.

=== leftover 'JSON array' / '//' ' comment in JSON / 'waiver'? ===  
none ?

=== config keys mentioned (topics/security\_checks/deep\_inspect consistent?) ===  
116: "topics": {  
161: no-download principle. The opt-in is per-policy (`deep\_inspect: true` in  
174: - The policy explicitly opts in (`deep\_inspect: true`).  
215: - `policies.config\_json` gains optional `topics`, `security\_checks`, and  
216: `deep\_inspect` keys.

### 359. assistant (2026-07-06T20:33:59.452Z)

Let me confirm that line 203 is the **corrected** "must not" phrasing (not a leftover claim), then I'll conclude.

### 360. user (2026-07-06T20:34:01.304Z)

The caller (Telethon userbot path or future Bot API handler) uses the final boolean to decide whether delivery should run. Rejected items short-circuit before destination resolution and delivery, so they do **not** have a `delivery\_attempts` row and must not be recorded in `delivery\_attempts.last\_error`.

Until F6 lands, rejected-item traces are in-memory/log-only. F6 introduces the persistent `policy\_decisions` table; F7 depends on F6 because `why` needs that table to read historical decisions.

**Schema ? minimal changes**

### 361. assistant (2026-07-06T20:34:23.629Z)

Confirmed ? that's the corrected phrasing. The revision (`401ec47`) is a comprehensive, internally-consistent response. Here's the re-review round.

### Re-review ? ADR 0012 revision (`401ec47` "address policy pipeline review gaps")

The author rebased on current `main` and revised the ADR. **All 10 of my findings are addressed**, and I verified the **resulting** doc is consistent (no dangling references, no new issues):

```
| # | My finding | How it's resolved | ? |
|---|-----|-----|---|
| ? 1 | "extra fields ignored" = dropped ? config erased | New Schema text: the
`load_filter_config`/`save_filter_config` shim "is not a safe persistence path? unknown keys
would be dropped. F1 must add a policy-config persistence path that reads/writes the raw JSON
object (or a typed superset)?". Added a current-state row + a two-sided Consequence. `allowlist`
key removed. | ? |
| ? 2 | `delivery_attempts.last_error` category error | Now states rejected items "do not
have a `delivery_attempts` row and must not be recorded in `delivery_attempts.last_error`;
traces are log-only until F6; F7 depends on F6. | ? |
| ? 3 | `content_item.text` never populated | New current-state row; Tier 2: "the current live
path does not populate `content_items.text`, F2 includes the ingestion work to capture caption
text"; F2/F4 phasing updated. | ? |
| ? 4 | PolicyContext can't do I/O | Added `fetcher: ContentFetcher \ None` + `budget:
PolicyBudget \ None`; "F1-F4 must work when `ctx.fetcher` is None"; Tier 4 is the first allowed
to use it." | ? |
| ? 5 | Tier 3 allowlist duplicates `watched` + misplacement | Removed the source-allowlist
stage: "Source selection is not a policy stage: ADR 0009's `sources.watched` gate decides?".
References line corrected. | ? |
| ? 6 | Duplicate-suppression overlaps dedup | Removed: "ADR 0011's `content_items.dedup_key`
runs before the pipeline? out of scope here." | ? |
| ? 7 | F1 "Dependencies: None" understates | Changed to "Tracks B-E landed" + F1 scope now
includes config persistence and hot-path `ContentItem`/`Source`/`Policy` wiring. | ? |
| ? 8 | AND-only vs include/exclude | Tier 2: "The pipeline itself remains AND-only across
stages, but the Topic stage owns its internal include/exclude/OR logic and returns one
`StageResult`." | ? |
| ? 9 | Tier 4 references but doesn't amend ADR 0003 | Added a full "ADR 0003 amendment for
Tier 4" section with 4 conditions; F5 dep ? "accepted ADR 0003 amendment above." | ? |
| ? 10 | `topics` array-vs-object + invalid `//` comment | Prose ? "stored as an object"; the
`//` comment removed. | ? |
```

**Consistency checks** (verified against the revised text): **no** leftover `allowlist`, no stale "record via `last\_error`" claim (only the corrected "must not"), no "JSON array", no `//` comment, no "ADR 0003 waiver"; config keys (`topics`/`security\_checks`/`deep\_inspect`) are consistent throughout.

**One tiny nit** (non-blocking): `ContentFetcher`/`PolicyBudget`` are named in the `PolicyContext`` dataclass but never given even a one-line shape ? deliberately deferred to F5, which is fine for a design contract, though a stub definition would make the interface fully self-contained.

**Verdict**: the revision resolves everything substantive. As a design contract this now reads as **ready to move from Proposed toward Accepted** once you've signed off on the four open questions (which the author sensibly left for the decision owner). Nicely done by the author.

Want me to **post a short follow-up comment** on #40 confirming the revision addresses the review (with that verification table)? Say the word and I'll drop it.

### 362. user (2026-07-06T20:39:10.285Z)

Please post this comment and give an lgtm to proceed further. Also I want you to set up your PR review alarm and wait for the PR hours coming towards this project within a few minutes

### 363. assistant (2026-07-06T20:40:00.230Z)

Both on it. First the LGTM comment on #40, then I'll arm a PR-watch alarm.

### 364. assistant (2026-07-06T20:40:16.551Z)

#### ? LGTM ? revision addresses the review; design contract is ready to proceed

Re-reviewed `401ec47` ("address policy pipeline review gaps") and verified the **resulting** ADR (not just the diff). All 10 findings from the [design review](#) are resolved, and the revised doc is internally consistent ? no dangling references, no new issues.

```
| # | Finding | Resolution |
|---|-----|-----|
| ? 1 | "extra fields ignored" = dropped ? config erased | Schema section now states the
`load_filter_config`/`save_filter_config` shim is not a safe path (keys dropped); F1 must add
a raw-JSON / typed-superset persistence path preserving unknown keys. `allowlist` key removed.
Two-sided Consequence added. |
| ? 2 | `delivery_attempts.last_error` category error | Rejected items "do not have a
`delivery_attempts` row and must not be recorded in `delivery_attempts.last_error`; log-only
until F6; F7 depends on F6. |
| ? 3 | `content_item.text` never populated | New current-state row; F2 now explicitly includes
capturing caption text at ingestion; F2/F4 phasing updated. |
| ? 4 | PolicyContext can't do I/O | Added `fetcher: ContentFetcher` \ None + `budget:
PolicyBudget` \ None; F1?F4 must work when `fetcher` is None. |
| ? 5 | Tier 3 source-allowlist duplicates `watched` | Removed ? "Source selection is not a
policy stage: ADR 0009's `sources.watched` gate decides ingestion." References corrected. |
| ? 6 | Duplicate-suppression overlaps dedup | Removed ? `content_items.dedup_key` runs before
the pipeline; cross-destination dedup is out of scope. |
| ? 7 | F1 "Dependencies: None" understated | ? "Tracks B-E landed"; F1 scope now includes
config persistence + hot-path `ContentItem`/`Source`/`Policy` wiring. |
| ? 8 | AND-only vs include/exclude | "The pipeline remains AND-only across stages, but the
Topic stage owns its internal include/exclude/OR logic." |
| ? 9 | Tier 4 references but doesn't amend ADR 0003 | Added an ADR 0003 amendment for Tier
4** section (4 conditions); F5 dep ? "accepted ADR 0003 amendment." |
| ? 10 | `topics` array-vs-object + `///` comment | Prose ? "object"; comment removed. |
```

**Consistency verified:** no leftover `allowlist`, no stale "record via `last\_error`", no "JSON array", no `///` in JSON, no "ADR 0003 waiver"; `topics`/`security\_checks`/`deep\_inspect` are consistent throughout.

**Non-blocking nit:** `ContentFetcher`/`PolicyBudget` are named in the `PolicyContext` dataclass but never given a one-line shape ? deliberately deferred to F5, which is fine for a design contract (a stub would make the interface fully self-contained).

**LGTM ?** ? as a design contract this is sound and ready to move from **Proposed ? Accepted** once you've signed off on the four open questions (rightly left to the decision owner). Clear to proceed to F1.

<sub>? Verified with [Claude Code](https://claude.com/claude-code) ? resolutions checked against the revised ADR and the referenced code/ADRs.</sub>

### 365. user (2026-07-06T20:40:16.587Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr40\_lgtm.md (file state is current in your context ? no need to Read it back)

### 366. user (2026-07-06T20:40:24.801Z)

=== post LGTM comment ===

<https://github.com/avidullu/telegram-video-filter/pull/40#issuecomment-4897418071>

=== current OPEN PRs (baseline for the watch alarm) ===

#40 draft=true ? docs(adr): propose 0012 policy pipeline ? explainable, tiered, user-customizable content gates (Track F)

### 367. assistant (2026-07-06T20:41:00.592Z)

LGTM posted ? [issuecomment-4897418071](https://github.com/avidullu/telegram-video-filter/pull/40#issuecomment-4897418071). Baseline: only **#40** (draft) is open. Now arming the PR-watch alarm ? a background poll that wakes me the moment a new PR appears (or #40 flips to ready-for-review).

### 368. user (2026-07-06T20:41:06.193Z)

Command running in background with ID: bpsrqryzo. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bpsrqryzo.output. You will be notified when it completes. To check interim output, use Read on that file path.

### 369. assistant (2026-07-06T20:41:15.660Z)

Both done. ?

## Summary

**\*1.** LGTM posted on **#40** ? [issuecomment-4897418071](https://github.com/avidullu/telegram-video-filter/pull/40#issuecomment-4897418071). Re-verified `401ec47` against the *resulting* ADR (not just the diff): all 10 findings resolved, doc internally consistent (no dangling `allowlist`, no stale `last\_error` claim, `topics`/`security\_checks`/`deep\_inspect` consistent). Verdict: sound design contract, **ready to move Proposed ? Accepted** and proceed to F1, pending your sign-off on the four open questions.

**\*2.** PR-watch alarm armed (background task `bpsrqryzo`). Baseline captured: only **#40** (draft) open. It polls every 2 minutes for ~60 minutes and **wakes me the instant** a new PR appears (or #40 flips draft ? ready). When it fires I'll:

- automatically start a full review of the new PR (sync ? gate ? multi-agent adversarial pass ? verify findings ? post the review), same as I did for #35?#40, and
- re-arm the alarm so the watch continues.

It ignores merges/closes (only new PRs and draft?ready transitions trigger it), so no false wake-ups. If the 60-min window elapses with nothing, I'll let you know and can re-arm.

Sit back ? I'll surface the next PR the moment it lands.

### 370. user (2026-07-06T20:49:11.836Z)

```
<task-notification>
<task-id>bpsrqryzo</task-id>
<tool-use-id>toolu_01TyZgX8C358d1Yu6B973xBD</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bpsrqryzo.output</output-file>
<status>completed</status>
<summary>Background command "Arm background PR-watch poll (wakes on new/changed PR)" completed (exit code 0)</summary>
</task-notification>
```

### 371. assistant (2026-07-06T20:49:15.005Z)

The alarm fired. Let me see what changed.

### 372. user (2026-07-06T20:49:17.592Z)

PR-watch armed. Baseline open PRs:  
40:true  
poll 1/30: no change (20:43:08Z)

```
poll 2/30: no change (20:45:09Z)
poll 3/30: no change (20:47:10Z)
=== NEW / CHANGED OPEN PR DETECTED (poll 4, 20:49:11Z) ===
new-or-changed (number:isDraft):
50:false
--- current open PRs ---
#50 draft=false by avidullu: feat(policy): add F1 pipeline primitives
```

### 373. assistant (2026-07-06T20:49:30.253Z)

**\*\*New PR #50 detected\*\*** ? "feat(policy): add F1 pipeline primitives" by avidullu (ready for review). This is the **\*\*F1 implementation\*\*** of the ADR 0012 policy pipeline I just LGTM'd ? so I'll check it especially against the design contract (config persistence that preserves unknown keys, hot-path wiring, `PolicyStage`/`evaluate\_pipeline`). Let me sync and pull it up.

### 374. user (2026-07-06T20:49:35.977Z)

```
c4c4d18..2aad4d6  main -> origin/main
* [new branch]      feat/f1-policy-engine -> origin/feat/f1-policy-engine
---checkout---
main
```

**main...origin/main [behind 1]**

```
=== PR #50 META ===
{"additions":280,"author":{"id":"MDQ6VXNlcjE2NzQ4Nzc=", "is_bot":false, "login":"avidullu", "name":
"Avi"}, "baseRefName":"main", "body":"## Summary\n\nImplements the first ADR 0012/F1 slice:\n\n-
adds `tg_video_filter.policy` with `PolicyContext`, `StageResult`, `PolicyStage`, and
`evaluate_pipeline()`\n-
adds short-circuit/order tests for the policy pipeline primitives\n-
adds raw default-policy config helpers over `policies.config_json`\n-
updates the legacy `save_filter_config` shim to merge flat `FilterConfig` fields into the existing policy JSON
instead of replacing the whole blob\n-
adds a regression test proving future pipeline keys such
as `topics`, `security_checks`, and `deep_inspect` survive legacy config updates\n\n##
Why\n\nADR 0012 identified that `load_filter_config` / `save_filter_config` would otherwise
erase future policy config keys when a user runs existing `/set` commands. F1 establishes the
engine skeleton and fixes that persistence boundary before topic/security stages land.\n\n##
Validation\n\n- `./venv/Scripts/python.exe -m ruff format src tests`\n-
`./venv/Scripts/python.exe -m ruff check src tests`\n- `./venv/Scripts/python.exe -m pytest`
-> 501 passed @ 96.63% coverage\n\nCloses #43\nRefs
#42", "changedFiles":4, "deletions":14, "headRefName":"feat/f1-policy-engine", "isDraft":false, "merg
eStateStatus":"CLEAN", "mergeable":"MERGEABLE", "number":50, "state":"OPEN", "title":"feat(policy):
add F1 pipeline primitives"}
```

### 375. assistant (2026-07-06T20:50:05.781Z)

PR #50 is **\*\*F1\*\*** (+280/?14, 4 files) ? and per the body it directly implements my finding #1 fix: **\*\*`save\_filter\_config` now merges flat fields into the existing policy JSON instead of replacing it\*\***, with a regression test that `topics`/`security\_checks`/`deep\_inspect` survive. That's the critical thing to verify. `main` also moved to `2aad4d6`. Let me get the diff, check freshness, and set up an isolated worktree to run the gate.

### 376. user (2026-07-06T20:50:14.584Z)

```
=== files ===
src/tg_video_filter/db.py
src/tg_video_filter/policy.py
tests/test_db.py
tests/test_policy.py

=== freshness: PR branch vs current origin/main ===
(commits on main not in PR branch ^)
--- what is main's new commit 2aad4d6? ---
2aad4d6 docs(adr): propose 0012 policy pipeline ? explainable, tiered, user-customizable content
gates (Track F) (#40)
```

353 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr50.diff

=== set up worktree + start gate ===  
HEAD is now at 793a8db feat(policy): add F1 pipeline primitives

### 377. assistant (2026-07-06T20:50:24.908Z)

`main` moved because **\*\*#40** (ADR 0012) was merged\*\* (`2aad4d6`), and PR #50 (F1) is current with it. Let me start the gate in the worktree and read the full diff.

### 378. user (2026-07-06T20:50:29.744Z)

gate started

### 379. user (2026-07-06T20:50:29.876Z)

```
1      diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
2      index a059e57..7f7e400 100644
3      --- a/src/tg_video_filter/db.py
4      +++ b/src/tg_video_filter/db.py
5      @@ -11,6 +11,7 @@
6
7          from __future__ import annotations
8
9      +import json
10     from collections.abc import Awaitable, Callable, Iterable
11     from datetime import UTC, datetime
12
13     @@ -1288,6 +1289,56 @@ async def get_default_policy(conn: aiomysqlite.Connection,
account_id: int) -> Pol
14         return _row_to_policy(row) if row else None
15
16
17     +def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
18     +    """Decode a policy config JSON object.
19     +
20     +    ADR 0012 keeps future policy keys in ``policies.config_json``. The value is
21     +    expected to be a JSON object so legacy flat ``FilterConfig`` keys and future
22     +    nested stage config can coexist.
23     +    """
24     +    decoded = json.loads(raw)
25     +    if not isinstance(decoded, dict):
26     +        raise ValueError("policy config_json must decode to a JSON object")
27     +    return decoded
28     +
29     +
30     +def _policy_config_dict_to_json(config: dict[str, object]) -> str:
31     +    """Encode policy config deterministically and compactly."""
32     +    return json.dumps(config, separators=(",", ":"), sort_keys=True)
33     +
34     +
35     +async def load_policy_config(conn: aiomysqlite.Connection, account_id: int) -> dict[str,
object]:
36     +    """Load the raw default policy config as a JSON object.
37     +
38     +    Unlike :func:`load_filter_config`, this preserves future ADR 0012 keys such
39     +    as ``topics`` / ``security_checks`` / ``deep_inspect`` instead of validating
40     +    through the flat ``FilterConfig`` model.
41     +    """
42     +    policy = await get_default_policy(conn, account_id)
43     +    if policy is None:
44     +        return FilterConfig().model_dump(mode="json")
45     +    return _policy_config_json_to_dict(policy.config_json)
```

```

46 +
47 +
48 +async def save_policy_config(
49 +    conn: aiosqlite.Connection, account_id: int, config: dict[str, object]
50 +) -> None:
51 +    """Persist the raw default policy config object."""
52 +    existing = await get_default_policy(conn, account_id)
53 +    await upsert_policy(
54 +        conn,
55 +        Policy(
56 +            id=existing.id if existing is not None else None,
57 +            account_id=account_id,
58 +            name="default",
59 +            config_json=_policy_config_dict_to_json(config),
60 +            is_default=True,
61 +            created_at=(existing.created_at if existing is not None else _now()),
62 +            updated_at=_now(),
63 +        ),
64 +    )
65 +
66 +
67 # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --
68
69
70 @@ -1705,24 +1756,26 @@ async def load_filter_config(conn: aiosqlite.Connection,
user_id: int) -> Filter
71     policy = await get_default_policy(conn, user_id)
72     if policy is None:
73         return FilterConfig()
74     - return FilterConfig.model_validate_json(policy.config_json)
75     + return FilterConfig.model_validate(_policy_config_json_to_dict(policy.config_json))
76
77
78     async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
79     -     """Deprecated shim: upsert the account's default :class:`Policy`."""
80     +     """Deprecated shim: update flat ``FilterConfig`` keys on the default policy.
81     +
82     +     ADR 0012: future pipeline keys live in the same ``policies.config_json``
83     +     object. Do not overwrite the whole blob with ``cfg.model_dump_json``;
84     +     merge the flat config fields into the existing object so unknown policy
85     +     keys survive legacy ``/set`` command updates.
86     +     """
87     existing = await get_default_policy(conn, user_id)
88     - await upsert_policy(
89     -     conn,
90     -     Policy(
91     -         id=existing.id if existing is not None else None,
92     -         account_id=user_id,
93     -         name="default",
94     -         config_json=cfg.model_dump_json(),
95     -         is_default=True,
96     -         created_at=(existing.created_at if existing is not None else _now()),
97     -         updated_at=_now(),
98     -     ),
99     - )
100 + raw_config: dict[str, object] = {}
101 + if existing is not None:
102 +     try:
103 +         raw_config = _policy_config_json_to_dict(existing.config_json)
104 +     except (json.JSONDecodeError, ValueError):
105 +         raw_config = {}
106 +     raw_config.update(cfg.model_dump(mode="json"))
107 +     await save_policy_config(conn, user_id, raw_config)
108

```



```

109
110     # --- bulk helpers (used by the manager on startup) -----
111     @@ -1789,6 +1842,8 @@ async def list_videos_for_user( # v2: used by on-demand /search;
not called in
112         "get_content_item_by_dedup_key",
113         "upsert_policy",
114         "get_default_policy",
115     +     "load_policy_config",
116     +     "save_policy_config",
117         # Destination / Routing / Delivery repos (ADR 0011, Track C)
118         "upsert_destination",
119         "get_destination",
120     diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
121     new file mode 100644
122     index 00000000..e7d0303
123     --- /dev/null
124     +++ b/src/tg_video_filter/policy.py
125     @@ -0,0 +1,82 @@
126     +"""Policy pipeline primitives for ADR 0012.
127     +
128     +The policy layer is deliberately small and framework-free in F1: stages receive
129     +an immutable context, return a structured result, and the evaluator stops on
130     +the first failure. Later Track F phases add concrete stages.
131     +"""
132     +
133     +from __future__ import annotations
134     +
135     +from dataclasses import dataclass, field
136     +from typing import Protocol
137     +
138     +from tg_video_filter.models import ContentItem, Policy, Source
139     +
140     +
141     +@dataclass(frozen=True)
142     +class PolicyContext:
143     +    """Immutable snapshot available to a policy stage."""
144     +
145     +    content_item: ContentItem
146     +    source: Source
147     +    policy: Policy
148     +    fetcher: object | None = None
149     +    budget: object | None = None
150     +
151     +
152     +@dataclass(frozen=True)
153     +class StageResult:
154     +    """Outcome of one policy stage."""
155     +
156     +    passed: bool
157     +    reason: str
158     +    stage_name: str
159     +    metadata: dict[str, object] = field(default_factory=dict)
160     +
161     +    @staticmethod
162     +    def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
163     +        return StageResult(
164     +            passed=True,
165     +            reason=reason,
166     +            stage_name=stage_name,
167     +            metadata=metadata,
168     +        )
169     +
170     +    @staticmethod
171     +    def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
172     +        return StageResult(

```

```

173 +         passed=False,
174 +         reason=reason,
175 +         stage_name=stage_name,
176 +         metadata=metadata,
177 +     )
178 +
179 +
180 +class PolicyStage(Protocol):
181 +    """A single stateless gate in the policy pipeline."""
182 +
183 +    async def evaluate(self, ctx: PolicyContext) -> StageResult:
184 +        """Evaluate ``ctx`` and return a stage result."""
185 +        ...
186 +
187 +
188 +    async def evaluate_pipeline(
189 +        ctx: PolicyContext,
190 +        stages: list[PolicyStage],
191 +    ) -> tuple[bool, list[StageResult]]:
192 +        """Evaluate stages in order and short-circuit on the first failure."""
193 +        results: list[StageResult] = []
194 +        for stage in stages:
195 +            result = await stage.evaluate(ctx)
196 +            results.append(result)
197 +            if not result.passed:
198 +                return False, results
199 +        return True, results
200 +
201 +
202 +    __all__ = [
203 +        "PolicyContext",
204 +        "PolicyStage",
205 +        "StageResult",
206 +        "evaluate_pipeline",
207 +    ]
208
209 diff --git a/tests/test_db.py b/tests/test_db.py
210 index 8692285..7eff4ff 100644
211 --- a/tests/test_db.py
212 +++ b/tests/test_db.py
213 @@ -470,6 +470,34 @@ async def test_save_filter_config_overwrites(conn:
aiosqlite.Connection) -> None
214     assert loaded.max_duration_s == 200
215
216
217 +async def test_save_filter_config_preserves_unknown_policy_keys(
218 +    conn: aiosqlite.Connection,
219 +) -> None:
220 +    """ADR 0012: legacy config updates must not erase pipeline config keys."""
221 +    await _seed_user(conn)
222 +    await db.save_policy_config(
223 +        conn,
224 +        USER_ID,
225 +        {
226 +            "min_duration_s": 5,
227 +            "max_duration_s": 7200,
228 +            "topics": {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"},
229 +            "security_checks": {"blocked_domains": ["sketchy.example"]},
230 +            "deep_inspect": True,
231 +        },
232 +    )
233 +
234 +    await db.save_filter_config(conn, USER_ID, FilterConfig(max_size_mb=512))
235 +
236 +    raw = await db.load_policy_config(conn, USER_ID)
237 +    assert raw["max_size_mb"] == 512

```

```

237 +     assert raw["topics"] == {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"}
238 +     assert raw["security_checks"] == {"blocked_domains": ["sketchy.example"]}
239 +     assert raw["deep_inspect"] is True
240 +     loaded = await db.load_filter_config(conn, USER_ID)
241 +     assert loaded.max_size_mb == 512
242 +
243 +
244     async def test_filter_config_with_null_bounds_round_trips(conn: aiosqlite.Connection)
-> None:
245         """None bounds (rule disabled) must survive the JSON round trip."""
246         await _seed_user(conn)
247         diff --git a/tests/test_policy.py b/tests/test_policy.py
248         new file mode 100644
249         index 00000000..02234a7
250         --- /dev/null
251         +++ b/tests/test_policy.py
252         @@ -0,0 +1,101 @@
253         +"""Tests for ADR 0012 policy pipeline primitives."""
254         +
255         +from __future__ import annotations
256         +
257         +from tg_video_filter.models import ContentItem, Policy, Source
258         +from tg_video_filter.policy import PolicyContext, StageResult, evaluate_pipeline
259         +
260         +
261         +def _ctx() -> PolicyContext:
262         +     return PolicyContext(
263         +         content_item=ContentItem(
264         +             account_id=111,
265         +             source_id=1,
266         +             provider_message_id="42",
267         +             dedup_key="tg:msg:1:42",
268         +         ),
269         +         source=Source(
270         +             id=1,
271         +             account_id=111,
272         +             provider="telegram",
273         +             provider_source_id="-1001",
274         +         ),
275         +         policy=Policy(
276         +             id=1,
277         +             account_id=111,
278         +             name="default",
279         +             config_json="{}",
280         +             is_default=True,
281         +         ),
282         +     )
283         +
284         +
285         +class RecordingStage:
286         +     def __init__(
287         +         self,
288         +         name: str,
289         +         calls: list[str],
290         +         *,
291         +         passed: bool = True,
292         +         reason: str | None = None,
293         +     ) -> None:
294         +         self.name = name
295         +         self.calls = calls
296         +         self.passed = passed
297         +         self.reason = reason or ("passed" if passed else "failed")
298         +
299         +     async def evaluate(self, ctx: PolicyContext) -> StageResult:
300         +         self.calls.append(self.name)

```

```

301 +         return StageResult(
302 +             passed=self.passed,
303 +             reason=self.reason,
304 +             stage_name=self.name,
305 +             metadata={"content_item_id": ctx.content_item.id},
306 +         )
307 +
308 +
309 +async def test_evaluate_pipeline_passes_all_stages_in_order() -> None:
310 +     calls: list[str] = []
311 +     stages = [
312 +         RecordingStage("metadata", calls),
313 +         RecordingStage("topic", calls),
314 +     ]
315 +
316 +     passed, results = await evaluate_pipeline(_ctx(), stages)
317 +
318 +     assert passed is True
319 +     assert calls == ["metadata", "topic"]
320 +     assert [r.stage_name for r in results] == ["metadata", "topic"]
321 +
322 +
323 +async def test_evaluate_pipeline_short_circuits_on_first_failure() -> None:
324 +     calls: list[str] = []
325 +     stages = [
326 +         RecordingStage("metadata", calls),
327 +         RecordingStage("topic", calls, passed=False, reason="excluded topic"),
328 +         RecordingStage("security", calls),
329 +     ]
330 +
331 +     passed, results = await evaluate_pipeline(_ctx(), stages)
332 +
333 +     assert passed is False
334 +     assert calls == ["metadata", "topic"]
335 +     assert [r.stage_name for r in results] == ["metadata", "topic"]
336 +     assert results[-1].reason == "excluded topic"
337 +
338 +
339 +async def test_evaluate_pipeline_with_no_stages_passes_vacuously() -> None:
340 +     passed, results = await evaluate_pipeline(_ctx(), [])
341 +
342 +     assert passed is True
343 +     assert results == []
344 +
345 +
346 +def test_stage_result_helpers_attach_metadata() -> None:
347 +     passed = StageResult.pass_("metadata", "ok", duration_s=60)
348 +     failed = StageResult.fail("topic", "excluded", topic="crypto")
349 +
350 +     assert passed.passed is True
351 +     assert passed.metadata == {"duration_s": 60}
352 +     assert failed.passed is False
353 +     assert failed.metadata == {"topic": "crypto"}
354

```

### 380. assistant (2026-07-06T20:51:27.827Z)

Clean, well-scoped F1 slice. It directly implements my finding #1: `save\_filter\_config` now **\*\*merges\*\*** flat fields into the existing raw config dict (preserving `topics`/`security\_checks`/`deep\_inspect`) via new `load\_policy\_config`/`save\_policy\_config` helpers, plus a regression test. The `policy.py` primitives match the ADR. Let me check the gate and verify the persistence fix + `load\_filter\_config` equivalence empirically.

### 381. user (2026-07-06T20:51:40.136Z)

```

=== gate ===
=== RUFF CHECK ===
All checks passed!
=== RUFF FORMAT ===
36 files already formatted
=== PYTEST ===
-----
TOTAL                                1250      32      322      21      97%
Required test coverage of 80.0% reached. Total coverage: 96.63%
501 passed in 8.86s
DONE

=== verify persistence fix + load_filter_config equivalence (against PR #50 code) ===
after /set: pipeline keys survive = True | max_size_mb = 512
load_filter_config: max_size_mb = 512 | min_duration_s = 5
OVERALL: PASS

```

### 382. assistant (2026-07-06T20:52:00.193Z)

Gate green: ruff clean, \*\*501 passed @ 96.63%\*\*. And the persistence fix is \*\*verified working\*\*  
 ? after a legacy `/set` flow, `topics`/`security\_checks`/`deep\_inspect` survive and the flat  
 field updates correctly. The crux of F1 works. Let me run a compact adversarial pass on the  
 merge edge-cases and the primitives.

### 383. user (2026-07-06T20:52:38.463Z)

Workflow launched in background. Task ID: wjsf683k2  
 Summary: Review telegram-video-filter PR #50 (F1 policy pipeline primitives + config-persistence  
 fix) with adversarial verification  
 Transcript dir: C:\Users\avidu\claudu\projects\C--Users-avidu-Projects-badminton-highlight-  
 indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_6c18d403-385  
 Script file: C:\Users\avidu\claudu\projects\C--Users-avidu-Projects-  
 Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-  
 pr50-f1-wf\_6c18d403-385.js  
 (Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:  
 "C:\Users\avidu\claudu\projects\C--Users-avidu-Projects-  
 Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-  
 pr50-f1-wf\_6c18d403-385.js"} to iterate without resending the script.)  
 Run ID: wf\_6c18d403-385  
 To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\claudu\projects\C--  
 Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-  
 pr50-f1-wf\_6c18d403-385.js", resumeFromRunId: "wf\_6c18d403-385"}) ? completed agents return  
 cached results.

You will be notified when it completes. Use /workflows to watch live progress.

### 384. assistant (2026-07-06T20:52:46.421Z)

While the workflow runs, let me check one edge myself: whether any caller of  
 `load\_filter\_config` catches the old pydantic error type (the exception type changed), and the  
 corrupt-config asymmetry.

### 385. user (2026-07-06T20:52:57.238Z)

```

=== load_filter_config call sites ? any try/except around them? ===
src/tg_video_filter/commands.py:78:         current_cfg = await db.load_filter_config(conn,
src/tg_video_filter/commands.py:82:         new_cfg = await db.load_filter_config(conn,
src/tg_video_filter/commands.py:112:        cfg = await db.load_filter_config(conn, user_id)
src/tg_video_filter/db.py:618:    # (Option 2A). is_default=1 so the legacy
load_filter_config shim reads
src/tg_video_filter/db.py:1809:    "load_filter_config",
src/tg_video_filter/manager.py:140:        cfg = await db.load_filter_config(conn,

```

```
session.account_id)
src/tg_video_filter/models.py:193:  ``load_filter_config`` / ``save_filter_config`` shims
read/write that row.
```

--- any ValidationError/except near those calls? ---

```
src/tg_video_filter/commands.py:78:      current_cfg = await db.load_filter_config(conn,
user.id)
src/tg_video_filter/commands.py:82:      new_cfg = await db.load_filter_config(conn,
user.id)
src/tg_video_filter/commands.py:112:    cfg = await db.load_filter_config(conn, user_id)
src/tg_video_filter/manager.py:140:    cfg = await db.load_filter_config(conn,
session.account_id)
```

=== corrupt-config behavior: load vs save (PR #50 code) ===

```
load_filter_config: RAISES ValidationError
load_policy_config: RAISES ValidationError
save_filter_config over corrupt RAISES ValidationError
```

### 386. assistant (2026-07-06T20:53:26.092Z)

That corrupt-config result is surprising ? `\_policy\_config\_json\_to\_dict('not-json')` should raise `JSONDecodeError`, and `save\_filter\_config` \*catches\* that to degrade gracefully, yet the repro shows it raising `ValidationError`. That contradicts my code-reading. Let me pin down exactly what happens.

### 387. user (2026-07-06T20:53:31.110Z)

```
'not-json'      -> JSONDecodeError: Expecting value: line 1 column 1 (char 0)
'[1,2,3]'      -> ValueError: policy config_json must decode to a JSON object
'"astring"'    -> ValueError: policy config_json must decode to a JSON object
'42'           -> ValueError: policy config_json must decode to a JSON object
'{'            -> dict {}
'{"a":1}'      -> dict {'a': 1}
```

### 388. assistant (2026-07-06T20:53:51.006Z)

Good ? `\_policy\_config\_json\_to\_dict` behaves correctly: `JSONDecodeError` on non-JSON, `ValueError` on non-object JSON, dict otherwise. My earlier combined repro's "ValidationError" was an artifact of running everything on one connection after a raise left it in an aborted state. Let me settle the `save\_filter\_config`-over-corrupt behavior on fresh connections.

### 389. user (2026-07-06T20:53:58.417Z)

```
--- corrupt config_json = 'not-json' (each on a FRESH connection) ---
  load_filter_config : ValidationError: 2 validation errors for Policy
created_a
  load_policy_config : ValidationError: 2 validation errors for Policy
created_a
  save_filter_config : ValidationError: 2 validation errors for Policy
created_a
--- corrupt config_json = '[1,2,3]' (each on a FRESH connection) ---
  load_filter_config : ValidationError: 2 validation errors for Policy
created_a
  load_policy_config : ValidationError: 2 validation errors for Policy
created_a
  save_filter_config : ValidationError: 2 validation errors for Policy
created_a
```

### 390. assistant (2026-07-06T20:54:08.299Z)

The `ValidationError` is a repro artifact ? I used `x` for `created\_at`/`updated\_at`, which fails `Policy` datetime validation before the config path is even reached. Let me fix the timestamps and settle it.

### 391. user (2026-07-06T20:54:15.422Z)

```
--- corrupt config_json = 'not-json' ---
load_filter_config : JSONDecodeError: Expecting value: line 1 column 1 (char 0)
load_policy_config : JSONDecodeError: Expecting value: line 1 column 1 (char 0)
save_filter_config : OK -> degraded, max_size_mb=99
--- corrupt config_json = '[1,2,3]' ---
load_filter_config : ValueError: policy config_json must decode to a JSON obje
load_policy_config : ValueError: policy config_json must decode to a JSON obje
save_filter_config : OK -> degraded, max_size_mb=99
```

### 392. assistant (2026-07-06T20:54:59.878Z)

Now it's clear and correct: ``load_filter_config`/`load_policy_config`` **raise** on corrupt ``config_json`` (as the old code did too ? no regression, exception type just changed), while ``save_filter_config`` **self-heals** (degrades to ``{}``, writes flat fields). That asymmetry is minor and pre-existing on the load side, and corrupt configs aren't reachable via normal writes. So my independent pass finds only low/scope-level items. The persistence fix (the crux) is verified correct.

I also want to note your standing instruction ? let me record it to memory so it persists across sessions.

### 393. user (2026-07-06T20:55:02.303Z)

```
total 77
drwxr-xr-x 1 avidu 197609    0 Jul  7 02:17 .
drwxr-xr-x 1 avidu 197609    0 Jul  6 22:56 ..
-rw-r--r-- 1 avidu 197609 1431 Jul  6 23:17 actual-repo-location.md
-rw-r--r-- 1 avidu 197609 1511 Jul  6 23:01 efficient-timeouts-and-retries.md
-rw-r--r-- 1 avidu 197609 2464 Jul  7 02:17 gpu-vm-setup-gotchass.md
-rw-r--r-- 1 avidu 197609  521 Jul  7 02:17 MEMORY.md
---MEMORY.md?---
```

## Memory index

- [Actual repo location](actual-repo-location.md) ? primary working dir is an empty STUB; real code is under ``khehsutra-guru/badminton-highlight-indexer``.
- [Efficient timeouts, retries & Windows gcloud SSH](efficient-timeouts-and-retries.md) ? no silent retry loops; plain ``gcloud compute ssh`` (plink/pscp reject ``-o``, mishandle ``~``); diagnose-first, run long work detached.
- [GPU VM setup gotchas](gpu-vm-setup-gotchass.md) ? NVDEC libs, apt-lock-at-boot, torch2.x/pynvc; what #501 should encode.

### 394. assistant (2026-07-06T20:55:07.424Z)

The memory dir already has an index. Let me add this as a feedback memory.

### 395. assistant (2026-07-06T20:55:15.171Z)

```
---
name: lgTM-only-when-merge-ready
description: On PR reviews, post LGTM only when genuinely satisfied the PR is merge-ready ?
verified, not a rubber stamp; applies to all PRs.
metadata:
  type: feedback
---
```

On every PR review, do **not** post an LGTM / sign-off until I am genuinely satisfied the PR is ready to merge. Stated by the owner on 2026-07-07 as a standing rule for "this and all upcoming PRs."

**Why:** the owner treats LGTM as a real merge gate, not a courtesy. A premature LGTM (before findings

are resolved or before verification finishes) is worse than no comment. Reinforces the global pre-LGTM gate in `~/.claude/CLAUDE.md` (green suite, lint/type clean, coverage held, findings resolved

? all reproduced locally, never trusting the PR description).

**\*\*How to apply:\*\*** review thoroughly (sync ? gate ? adversarial/multi-agent pass ? verify each finding by

reproducing) and hold the LGTM until every blocker/high is resolved and I'd merge it myself.

Post

interim review comments (findings, request-changes) freely; reserve the LGTM for the verified, merge-ready state. If fixes land, re-verify before signing off. See the badminton-highlight-indexer

Telegram PR-review series (#35?#50) for the cadence.

#### 396. user (2026-07-06T20:55:15.230Z)

File created successfully at: C:/Users/avidu/.claude/projects/C--Users-avidu-Projects-badminton-highlight-indexer/memory/lgtm-only-when-merge-ready.md (file state is current in your context ? no need to Read it back)

#### 397. user (2026-07-06T20:55:20.777Z)

<tool\_use\_error>File has not been read yet. Read it first before writing to it.</tool\_use\_error>

#### 398. user (2026-07-06T20:55:24.455Z)

```
1      # Memory index
2
3      - [Actual repo location](actual-repo-location.md) ? primary working dir is an empty
STUB; real code is under `kheutra-guru/badminton-highlight-indexer`.
4      - [Efficient timeouts, retries & Windows gcloud SSH](efficient-timeouts-and-retries.md)
? no silent retry loops; plain `gcloud compute ssh` (plink/pscp reject `-o`, mishandle `~`);
diagnose-first, run long work detached.
5      - [GPU VM setup gotchas](gpu-vm-setup-gotchas.md) ? NVDEC libs, apt-lock-at-boot,
torch2.x/pynvc; what #501 should encode.
6
```

#### 399. user (2026-07-06T20:55:31.053Z)

The file C:/Users/avidu/.claude/projects/C--Users-avidu-Projects-badminton-highlight-indexer/memory/MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 400. assistant (2026-07-06T20:55:39.170Z)

Memory recorded. Now, per your instruction, I'll hold any LGTM on #50 until the adversarial pass finishes and I'm satisfied. Let me check whether the workflow has completed.

#### 401. user (2026-07-06T20:55:45.929Z)

workflow still running (no result yet)

#### 402. assistant (2026-07-06T20:56:04.722Z)

Understood ? and noted as a standing rule (saved to memory): **\*\*I'll post an LGTM only when I'm genuinely satisfied a PR is merge-ready, for this and every upcoming PR.\*\*** That matches the pre-LGTM gate I've been applying; I'll keep reserving LGTM for the verified, would-merge-it-myself state and use interim comments for findings.

**\*\*Where PR #50 (F1) stands ? I'm holding the LGTM until the adversarial pass finishes:\*\***

- ? **\*\*Gate:\*\*** ruff check + format clean, **\*\*501 passed @ 96.63%\*\*** (matches PR claim).  
- ? **\*\*The crux verified\*\*** ? the persistence fix works: after a legacy `set`,  
`topics`/`security\_checks`/`deep\_inspect` survive and flat fields update (reproduced); solid



regression test included.

- ? **Corrupt-config behavior settled** (after I corrected a bad repro): loads raise on corrupt `config\_json` ? but the **old** code did too, so **no regression**; `save\_filter\_config` self-heals. The exception-type change is immaterial (no caller catches it).  
- My independent pass found only **low/scope** items so far: the `evaluate\_pipeline`/`PolicyStage` primitives aren't wired into the hot path yet (disclosed as "first F1 slice" ? currently exercised only by tests); `fetcher`/`budget` typed as `object` placeholders.

The 2-dimension adversarial workflow is still running its verify pass. I'll fold in its results, and **only if nothing substantive surfaces** will I post the LGTM ? otherwise I'll post findings and hold. Back shortly when it completes.

#### 403. user (2026-07-06T21:01:47.379Z)

Update?

#### 404. assistant (2026-07-06T21:01:49.714Z)

Let me check the workflow status.

#### 405. user (2026-07-06T21:01:58.165Z)

=== workflow DONE ===

COUNTS: {'raw': 7, 'kept': 6, 'refuted': 1}

=====

[LOW/CONFIRMED] (persistence) load\_filter\_config exception type changed from pydantic ValidationError to ValueError/JSONDecodeError on corrupt config\_json

where: src/tg\_video\_filter/db.py:1759

rec: Acceptable as-is since no caller relies on the exception type. If a stable public contract is desired, wrap the decode/validate in load\_filter\_config and re-raise

[LOW/CONFIRMED] (persistence) save\_filter\_config silently drops all pipeline keys on a corrupt config, while load\_policy\_config raises on the same data (read/write asymmetry)

where: src/tg\_video\_filter/db.py:1775

rec: Decide one policy for corrupt configs and apply it on both sides ? either both raise (safest: refuse to silently drop) or both degrade to defaults with a logged

[LOW/CONFIRMED] (persistence) save\_policy\_config hardcodes name='default' but get\_default\_policy is name-agnostic; a default policy named other than 'default' makes save raise IntegrityError and become unsaveable

where: src/tg\_video\_filter/db.py:1333

rec: Make save\_policy\_config target the existing default policy by its actual name (existing.name if existing is not None else 'default') so it updates the row get\_d

[LOW/CONFIRMED] (primitives\_scope) Asymmetric handling of a corrupt/non-object stored config: save heals silently, load crashes

where: src/tg\_video\_filter/db.py:1759

rec: Either make the two paths symmetric (also tolerate a corrupt row on read, falling back to FilterConfig() / defaults) or make save fail loudly too. As-is, document

[LOW/CONFIRMED] (primitives\_scope) StageResult is frozen but its metadata dict is mutable, so the immutability guarantee is only skin-deep

where: src/tg\_video\_filter/policy.py:34

rec: If true immutability is desired, store metadata as an immutable mapping (e.g. types.MappingProxyType wrapping the dict, or a frozen mapping type) or document the

[LOW/PLAUSIBLE] (primitives\_scope) PolicyContext.fetcher/budget typed as 'object | None' instead of the ADR-named ContentFetcher/PolicyBudget

where: src/tg\_video\_filter/policy.py:23

rec: Acceptable as a placeholder for F1. When F5 lands the real types, tighten these annotations (or introduce forward-referenced Protocols now under TYPE\_CHECKING)

refuted: ['policy.py is entirely dead production code in this slice']

#### 406. assistant (2026-07-06T21:02:33.167Z)

Workflow done: **\*\*7 ? 6 kept, 1 refuted\*\*** ? and **\*\*all 6 are LOW\*\*** (the "policy.py is dead code" claim was correctly refuted, since it's a disclosed first slice). One is a new finding I want to verify myself before trusting it: ``save_policy_config`` hardcodes ``name="default"`` while ``get_default_policy`` is name-agnostic ? could break if a default policy is named otherwise. Let me reproduce it.

#### 407. user (2026-07-06T21:02:41.852Z)

```
default policy name = my-custom
save_policy_config RAISES: IntegrityError UNIQUE constraint failed: policies.account_id
```

#### 408. assistant (2026-07-06T21:03:04.344Z)

**\*\*Confirmed\*\*** ? if an account's default policy is named anything other than ``"default"``, ``save_policy_config`` raises ``IntegrityError`` (the single-default partial index blocks a second ``is_default=1`` row). It's **\*\*unreachable today\*\*** (the default policy is always named ``"default"``), but a real footgun once the F-phases add named policies (which ADR 0011/0012 explicitly plan). 1-line fix.

Full picture for #50: **\*\*gate green, crux verified, all 6 findings are LOW\*\*** (none block; 1 dead-code claim refuted). By your standing rule, I'd merge this myself ? so I'm satisfied. Posting the LGTM with the low items as non-blocking follow-ups.

#### 409. assistant (2026-07-06T21:03:22.089Z)

### ? LGTM ? F1 pipeline primitives + config-persistence fix

Reviewed thoroughly and reproduced the key behaviors locally. This is a clean, well-scoped slice that correctly resolves the ADR 0012 review's #1 concern (the config-key erasure), with a solid regression test. Merge-ready. The findings below are all **\*\*low / non-blocking\*\***.

#### Verified

```
| Check | Result |
|---|---|
| `ruff check` / `ruff format` | ? clean |
| `pytest` | ? **501 passed @ 96.63%** |
| **Persistence fix** | ? after a legacy `/set`, `topics`/`security_checks`/`deep_inspect` **survive** and flat fields update (reproduced) ? `save_filter_config` now merges into the raw config dict instead of replacing the blob |
| `evaluate_pipeline` short-circuit/order | ? correct (tests + read) |
```

The ``load_filter_config`` behavior is preserved for valid configs; on a **\*corrupt\*** config it raises as before (only the exception type changed ? see #3).

### ? Low (non-blocking; worth a quick follow-up)

1. **\*\*`save\_policy\_config` hardcodes ``name="default"`` but ``get_default_policy`` is name-agnostic\*\*** (``db.py`:-1333``). If an account's default policy is ever named something other than ``"default"``, ``save_policy_config`` (and thus ``save_filter_config``) raises ``IntegrityError: UNIQUE constraint failed: policies.account_id`` ? the config becomes unsaveable. **\*\*Unreachable today\*\*** (the default policy is always named ``"default"``), but a real footgun once the F-phases add the named/multi-policy support ADR 0011/0012 plan. Reproduced.

> 1-line fix: ``name=existing.name if existing is not None else "default"``.

2. **\*\*Corrupt-config read/write asymmetry\*\*** (``db.py`:-1775`` vs ``:1759/1310``). ``save_filter_config`` catches ``(JSONDecodeError, ValueError)`` and silently resets to ``{}`` (dropping any salvageable pipeline keys, no log), while ``load_policy_config`/`load_filter_config`` **\*\*raise\*\*** on the same blob. Corrupt configs can't arise from this PR's own writes, so it's latent ? but the two sides disagree. Suggest at least a ``log.warning`` in the ``save`` except branch so silent pipeline-key loss is observable.

3. **\*\*`load\_filter\_config` exception type changed\*\*** (``db.py`:1759``) ? pydantic ``ValidationError`` ?

`ValueError`/`JSONDecodeError` on corrupt input. Immaterial (no caller catches it), but note it in the docstring if the type is a contract.

4. `**`StageResult` is `frozen=True` but `metadata` is a mutable `dict`**` (`policy.py:34`) ? immutability is skin-deep. Fine for now; `MappingProxyType` or a note if true immutability is wanted.

5. `**`PolicyContext.fetcher`/`budget` typed `object | None`**` (`policy.py:23`) ? acceptable placeholder for F1; tighten to the ADR-named `ContentFetcher`/`PolicyBudget`` when F5 lands.

6. `**Scope note (disclosed):**` the primitives aren't wired into the live handler yet ? ``evaluate_pipeline`/`PolicyStage`` are exercised only by tests in this slice. Correctly deferred per the PR body; the hot-path wiring is the next F1 slice.

`**LGTM ?**` ? clear to merge. #1 is the one I'd fold in now (trivial, and the multi-policy direction is coming); the rest are polish.

<sub>? Verified with [Claude Code](https://claude.com/claude-code) ? persistence fix, corrupt-config paths, and the name-mismatch footgun all reproduced against the branch. Multi-agent adversarial pass: 6 low findings, 1 refuted.</sub>

#### 410. user (2026-07-06T21:03:22.123Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr50\_lgtn.md (file state is current in your context ? no need to Read it back)

#### 411. user (2026-07-06T21:03:29.977Z)

=== post LGTM on #50 ===

<https://github.com/avidullu/telegram-video-filter/pull/50#issuecomment-4897588383>

=== clean up review worktree ===

C:/Users/avidu/Projects/Telegram c4c4d18 [main]

#### 412. assistant (2026-07-06T21:03:37.995Z)

LGTM posted ? [issuecomment-4897588383](https://github.com/avidullu/telegram-video-filter/pull/50#issuecomment-4897588383). Worktree cleaned up. Now let me refresh the local checkout and `**re-arm the PR-watch alarm**` so I keep catching new PRs.

#### 413. user (2026-07-06T21:03:43.787Z)

Command running in background with ID: bvv3qtkiy. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bvv3qtkiy.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 414. assistant (2026-07-06T21:04:00.231Z)

Done ? here's the update.

### PR #50 (F1) ? reviewed & LGTM'd ?

[LGTM posted](https://github.com/avidullu/telegram-video-filter/pull/50#issuecomment-4897588383). I applied your standing rule: I only signed off after being satisfied it's genuinely merge-ready.

- `**Gate:**` ruff clean . `**501 passed @ 96.63%**`.
- `**Crux verified:**` the config-persistence fix works ? after a legacy ``/set``, ``topics`/`security_checks`/`deep_inspect`` survive and flat fields update (reproduced); good regression test.
- `**All findings LOW, none blocking**` (adversarial pass: 6 low, 1 refuted). The one I flagged to fold in now is a 1-line footgun: ``save_policy_config`` hardcodes ``name="default"`` while ``get_default_policy`` is name-agnostic ? ``IntegrityError`` if a default policy is ever named

otherwise (unreachable today, but the F-phases add named policies). Reproduced. The rest are polish (corrupt-config asymmetry logging, `object` placeholders, skin-deep `frozen`).

## Standing rule ? saved

Recorded to memory: **\*\*post LGTM only when genuinely satisfied a PR is merge-ready (verified), for this and all upcoming PRs.\*\*** I'll keep reserving LGTM for the would-merge-it-myself state and use interim comments for findings/request-changes.

## PR-watch alarm ? re-armed ?

Fast-forwarded local `main` (now includes the merged #40), then re-armed the watch (task `bvv3qtkiy`). Baseline is the current open set (#50); it polls every 2 min for ~60 min and wakes me the instant a new PR appears or one flips draft?ready. When it fires I'll run the same full review-and-verify cycle and only LGTM when it clears the bar.

Nothing needed from you ? I'll surface the next PR when it lands.

### 415. user (2026-07-06T21:21:55.545Z)

```
<task-notification>
<task-id>bvv3qtkiy</task-id>
<tool-use-id>toolu_01Njvt8yr2ajePS9q9kzsrRJ</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bvv3qtkiy.output</output-file>
<status>completed</status>
<summary>Background command "Fast-forward main and re-arm PR-watch alarm" completed (exit code
0)</summary>
</task-notification>
```

### 416. assistant (2026-07-06T21:22:00.120Z)

The alarm fired again. Let me see what changed.

### 417. user (2026-07-06T21:22:02.382Z)

local main now: 2aad4d6 docs(adr): propose 0012 policy pipeline ? explainable, tiered, user-customizable content gates (Track F) (#40)

```
=== PR-watch RE-ARMED. Current open PRs (baseline): ===
50:false
poll 1/30: no change (21:05:47Z)
poll 2/30: no change (21:07:49Z)
poll 3/30: no change (21:09:49Z)
poll 4/30: no change (21:11:50Z)
poll 5/30: no change (21:13:51Z)
poll 6/30: no change (21:15:52Z)
poll 7/30: no change (21:17:53Z)
poll 8/30: no change (21:19:53Z)
=== NEW / CHANGED OPEN PR DETECTED (poll 9, 21:21:54Z) ===
new-or-changed (number:isDraft):
51:false
--- current open PRs ---
#51 draft=false by avidullu: feat(policy): add F2 topic preferences
```

### 418. assistant (2026-07-06T21:22:12.466Z)

**\*\*New PR #51 detected\*\*** ? "feat(policy): add F2 topic preferences" (F2 of the pipeline). Let me sync and pull it up, checking especially against the F2 scope (topic parsing, caption capture into `content\_items.text`, include/exclude logic) and my earlier ADR findings.

### 419. user (2026-07-06T21:22:18.874Z)

- [deleted] (none) -> origin/feat/f1-policy-engine

```
2aad4d6..fb2d2ea  main      -> origin/main
* [new branch]      feat/f2-topic-preferences -> origin/feat/f2-topic-preferences
---checkout---
main
```

## main...origin/main [behind 1]

=== is #50 merged now? recent main ===

```
fb2d2ea feat(policy): add F1 pipeline primitives (#50)
2aad4d6 docs(adr): propose 0012 policy pipeline ? explainable, tiered, user-customizable content
gates (Track F) (#40)
c4c4d18 fix(scanning): don't clobber user-controlled watched on source upsert (#41)
```

=== PR #51 META ===

```
{"additions":417,"author":{"id":"MDQ6VXNlcjE2NzQ4Nzc=","is_bot":false,"login":"avidullu","name":
"Avi"},"baseRefName":"main","body":"## Summary\n\nImplements ADR 0012 F2 topic preferences and
caption capture:\n\n- captures Telegram caption/message text into `VideoRecord.text` and
`content_items.text` for live ingestion and backfill\n- adds `TopicPreferences` parsing from
`policies.config.json`\n- adds `TopicPreferenceStage` with include/exclude semantics and `mode =
any | all`\n- matches topics against source title and captured caption text, case-
insensitively\n- wires the topic stage after the existing metadata `FilterEngine` in live
ingestion and backfill, preserving short-circuit behavior\n- adds tests for topic parsing,
any/all include behavior, exclude vetoes, invalid config, caption storage, and message-text
normalization\n\n## Notes\n\nThis stays metadata/captured-text only. It introduces no ML
classification, page fetches, downloads, or network I/O.\n\n## Validation\n\n-
`./venv/Scripts/python.exe -m ruff check src tests --fix`\n- `./venv/Scripts/python.exe -m
ruff format src tests`\n- `./venv/Scripts/python.exe -m ruff check src tests`\n-
`./venv/Scripts/python.exe -m pytest` -> 513 passed @ 95.58% coverage\n\nCloses #44\nRefs
#42","changedFiles":8,"deletions":16,"headRefName":"feat/f2-topic-preferences","isDraft":false,"
mergeStateStatus":"CLEAN","mergeable":"MERGEABLE","number":51,"state":"OPEN","title":"feat(polic
y): add F2 topic preferences"}
```

## 420. assistant (2026-07-06T21:22:38.315Z)

**\*\*#50 (F1) merged\*\*** ? (`fb2d2ea`). PR #51 is **\*\*F2\*\*** (+417/?16, 8 files) ? and notably it captures caption text into `content\_items.text` (my finding #3), adds `TopicPreferenceStage`, and **\*\*wires the pipeline into the live ingestion path\*\*** (the hot-path wiring F1 deferred). This touches the live path, so I'll scrutinize the wiring and whether topic config is read via the key-preserving `load\_policy\_config`. Let me set up the worktree, start the gate, and get the diff.

## 421. user (2026-07-06T21:22:49.728Z)

```
=== files ===
src/tg_video_filter/backfill.py
src/tg_video_filter/db.py
src/tg_video_filter/models.py
src/tg_video_filter/policy.py
src/tg_video_filter/telethon_client.py
tests/test_db.py
tests/test_policy.py
tests/test_telethon_extraction.py
```

=== freshness: PR vs origin/main ===  
(commits on main not in PR ^ / empty=current)

```
666 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr51.diff
```

```
=== worktree + gate ===
HEAD is now at 78c4f8f feat(policy): add F2 topic preferences
gate started
```

## 422. assistant (2026-07-06T21:22:55.906Z)

PR #51 is current with main (666-line diff, touches the live path). Let me read the full diff.

#### 423. user (2026-07-06T21:22:56.644Z)

```
1      diff --git a/src/tg_video_filter/backfill.py b/src/tg_video_filter/backfill.py
2      index 945132b..7a370f2 100644
3      --- a/src/tg_video_filter/backfill.py
4      +++ b/src/tg_video_filter/backfill.py
5      @@ -31,8 +31,15 @@
6          from tg_video_filter.commands import ENGINE_KEY
7          from tg_video_filter.delivery import build_link_text, deliver
8          from tg_video_filter.filters import FilterEngine
9          -from tg_video_filter.models import DeliveryMode, User, VideoRecord
10         -from tg_video_filter.telethon_client import _extract_document_id, _peer_id,
extract_video_meta
11         +from tg_video_filter.models import Channel, DeliveryMode, User, VideoRecord
12         +from tg_video_filter.telethon_client import (
13         +     _content_item_preview,
14         +     _evaluate_topic_preferences,
15         +     _extract_document_id,
16         +     _message_text,
17         +     _peer_id,
18         +     extract_video_meta,
19         +)
20
21         log = logging.getLogger(__name__)
22
23         @@ -216,22 +223,59 @@ async def backfill_user( # pragma: no cover - requires live
Telethon connection
24             summary["videos_scanned"] += 1
25             dialog_videos += 1
26
27         +         source = None
28         +         try:
29         +             channel = await db.upsert_channel(
30         +                 conn,
31         +                 Channel(user_id=user.id, channel_id=channel_id, title=title),
32         +             )
33         +             if channel.id is not None:
34         +                 source = await db.get_source_by_id(conn, channel.id)
35         +         except Exception:
36         +             log.warning(
37         +                 "failed to upsert channel %s for user %s during backfill",
38         +                 channel_id,
39         +                 user.id,
40         +                 exc_info=True,
41         +             )
42         +
43         +         text = _message_text(message)
44         +         result = engine.evaluate(meta)
45         +         matched = result.matched
46         +         if matched and source is not None:
47         +             topic_result = await _evaluate_topic_preferences(
48         +                 conn,
49         +                 user.id,
50         +                 source,
51         +                 _content_item_preview(
52         +                     user.id,
53         +                     source,
54         +                     message_id,
55         +                     document_id, # type: ignore[arg-type]
56         +                     meta,
57         +                     text,
58         +                     matched,
59         +                 ),
60         +             )
61         +         if topic_result is not None:
```

```

62      +                matched = topic_result.passed
63      +
64          record = VideoRecord(
65              user_id=user.id,
66              channel_id=channel_id,
67              message_id=message_id,
68      +          text=text,
69              document_id=document_id, # type: ignore[arg-type]
70              duration_s=meta.duration_s,
71              width=meta.width,
72              height=meta.height,
73              size_bytes=meta.size_bytes,
74              mime_type=meta.mime_type,
75      -          matched=result.matched,
76      +          matched=matched,
77          )
78      inserted = await db.insert_video(conn, record)
79
80      -          if result.matched:
81      +          if matched:
82              summary["videos_matched"] += 1
83              dialog_matched += 1
84              link_text = build_link_text(
85  diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
86  index 38adc10..e6fca83 100644
87  --- a/src/tg_video_filter/db.py
88  +++ b/src/tg_video_filter/db.py
89  @@ -1573,6 +1573,7 @@ async def get_video_by_content_fingerprint(
90      user_id=user_id,
91      channel_id=_safe_int_cast(row["src_pid"], "provider_source_id"),
92      message_id=_safe_int_cast(row["provider_message_id"], "provider_message_id"),
93      +      text=row["text"],
94      document_id=(
95          _safe_int_cast(row["document_id"], "document_id")
96          if row["document_id"] is not None
97  @@ -1677,6 +1678,7 @@ async def insert_video(conn: aiomysqlite.Connection, video:
VideoRecord) -> VideoR
98      provider="telegram",
99      provider_message_id=str(video.message_id),
100      content_type="video",
101      +      text=video.text,
102      document_id=str(video.document_id) if video.document_id is not None else None,
103      duration_s=video.duration_s,
104      width=video.width,
105  @@ -1730,6 +1732,7 @@ def _content_item_to_video(
106      user_id=user_id,
107      channel_id=channel_id,
108      message_id=_safe_int_cast(item.provider_message_id, "provider_message_id"),
109      +      text=item.text,
110      document_id=(
111          _safe_int_cast(item.document_id, "document_id")
112          if item.document_id is not None
113  @@ -1813,6 +1816,7 @@ async def list_videos_for_user( # v2: used by on-demand /search;
not called in
114      user_id=user_id,
115      channel_id=_safe_int_cast(r["src_pid"], "provider_source_id"),
116      message_id=_safe_int_cast(r["provider_message_id"], "provider_message_id"),
117      +      text=r["text"],
118      document_id=(
119          _safe_int_cast(r["document_id"], "document_id")
120          if r["document_id"] is not None
121  diff --git a/src/tg_video_filter/models.py b/src/tg_video_filter/models.py
122  index 4988efe..70e2a8a 100644
123  --- a/src/tg_video_filter/models.py
124  +++ b/src/tg_video_filter/models.py

```

```

125     @@ -394,6 +394,7 @@ class VideoRecord(BaseModel):
126         user_id: int
127         channel_id: int = Field(description="Origin Telegram channel id")
128         message_id: int = Field(description="Telegram message id within the channel")
129     +     text: str | None = Field(default=None, description="Caption / message text")
130     +     document_id: int | None = Field(
131         +         default=None,
132         +         description="Telegram document.id ? reused across forwards; Tier-1 content
dedup",
133     diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
134     index e7d0303..395435a 100644
135     --- a/src/tg_video_filter/policy.py
136     +++ b/src/tg_video_filter/policy.py
137     @@ -7,6 +7,7 @@
138
139     +     from __future__ import annotations
140
141     +import json
142     +     from dataclasses import dataclass, field
143     +     from typing import Protocol
144
145     @@ -60,6 +61,118 @@ async def evaluate(self, ctx: PolicyContext) -> StageResult:
146     +     ...
147
148
149     +@dataclass(frozen=True)
150     +class TopicPreferences:
151     +     +     """Parsed ADR 0012 topic preference config."""
152     +     +
153     +     +     include: tuple[str, ...] = ()
154     +     +     exclude: tuple[str, ...] = ()
155     +     +     mode: str = "any"
156     +     +
157     +     +     @property
158     +     +     def enabled(self) -> bool:
159     +     +         return bool(self.include or self.exclude)
160     +     +
161     +
162     +def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
163     +     +     """Parse ``topics`` from a raw policy config object."""
164     +     +     raw_topics = config.get("topics")
165     +     +     if raw_topics is None:
166     +     +         return TopicPreferences()
167     +     +     if not isinstance(raw_topics, dict):
168     +     +         raise ValueError("topics must be a JSON object")
169     +     +     mode = raw_topics.get("mode", "any")
170     +     +     if mode not in ("any", "all"):
171     +     +         raise ValueError("topics.mode must be 'any' or 'all'")
172     +     +     return TopicPreferences(
173     +     +         include=_string_tuple(raw_topics.get("include")),
174     +     +         exclude=_string_tuple(raw_topics.get("exclude")),
175     +     +         mode=str(mode),
176     +     +     )
177     +
178     +
179     +def load_topic_preferences(policy: Policy) -> TopicPreferences:
180     +     +     """Load topic preferences from ``policy.config_json``."""
181     +     +     decoded = json.loads(policy.config_json)
182     +     +     if not isinstance(decoded, dict):
183     +     +         raise ValueError("policy config_json must decode to a JSON object")
184     +     +     return parse_topic_preferences(decoded)
185     +
186     +
187     +def _string_tuple(value: object) -> tuple[str, ...]:
188     +     +     if value is None:

```



```

189 +         return ()
190 +     if not isinstance(value, list):
191 +         raise ValueError("topic include/exclude values must be arrays")
192 +     topics: list[str] = []
193 +     for item in value:
194 +         if not isinstance(item, str):
195 +             raise ValueError("topic include/exclude arrays must contain strings")
196 +         topic = item.strip()
197 +         if topic:
198 +             topics.append(topic)
199 +     return tuple(topics)
200 +
201 +
202 +class TopicPreferenceStage:
203 +     """ADR 0012 Tier 2 topic matching over source title and captured caption."""
204 +
205 +     stage_name = "topic-preferences"
206 +
207 +     async def evaluate(self, ctx: PolicyContext) -> StageResult:
208 +         try:
209 +             prefs = load_topic_preferences(ctx.policy)
210 +         except (json.JSONDecodeError, ValueError) as exc:
211 +             return StageResult.fail(self.stage_name, f"invalid topic config: {exc}")
212 +         if not prefs.enabled:
213 +             return StageResult.pass_(self.stage_name, "topic preferences disabled")
214 +
215 +         haystack = _topic_haystack(ctx)
216 +         matched_excludes = _matched_topics(prefs.exclude, haystack)
217 +         if matched_excludes:
218 +             return StageResult.fail(
219 +                 self.stage_name,
220 +                 f"excluded topic matched: {matched_excludes[0]}",
221 +                 matched_exclude_topics=list(matched_excludes),
222 +             )
223 +
224 +         if not prefs.include:
225 +             return StageResult.pass_(self.stage_name, "no include topics configured")
226 +
227 +         matched_includes = _matched_topics(prefs.include, haystack)
228 +         if prefs.mode == "any":
229 +             if matched_includes:
230 +                 return StageResult.pass_(
231 +                     self.stage_name,
232 +                     f"matched include topic: {matched_includes[0]}",
233 +                     matched_include_topics=list(matched_includes),
234 +                 )
235 +             return StageResult.fail(self.stage_name, "no include topics matched")
236 +
237 +         missing = tuple(topic for topic in prefs.include if topic not in
matched_includes)
238 +         if missing:
239 +             return StageResult.fail(
240 +                 self.stage_name,
241 +                 f"missing required topic: {missing[0]}",
242 +                 matched_include_topics=list(matched_includes),
243 +                 missing_include_topics=list(missing),
244 +             )
245 +         return StageResult.pass_(
246 +             self.stage_name,
247 +             "matched all include topics",
248 +             matched_include_topics=list(matched_includes),
249 +         )
250 +
251 +
252 +def _topic_haystack(ctx: PolicyContext) -> str:

```

```

253 + parts = [ctx.source.title or "", ctx.content_item.text or ""]
254 + return "\n".join(parts).casefold()
255 +
256 +
257 +def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
258 + return tuple(topic for topic in topics if topic.casefold() in haystack)
259 +
260 +
261 async def evaluate_pipeline(
262     ctx: PolicyContext,
263     stages: list[PolicyStage],
264 @@ -78,5 +191,9 @@ async def evaluate_pipeline(
265     "PolicyContext",
266     "PolicyStage",
267     "StageResult",
268 + "TopicPreferenceStage",
269 + "TopicPreferences",
270     "evaluate_pipeline",
271 + "load_topic_preferences",
272 + "parse_topic_preferences",
273 ]
274 diff --git a/src/tg_video_filter/telethon_client.py
b/src/tg_video_filter/telethon_client.py
275 index df8b4f9..7b393f8 100644
276 --- a/src/tg_video_filter/telethon_client.py
277 +++ b/src/tg_video_filter/telethon_client.py
278 @@ -35,7 +35,16 @@
279     from tg_video_filter.commands import ENGINE_KEY
280     from tg_video_filter.delivery import build_link_text, deliver
281     from tg_video_filter.filters import FilterEngine, VideoMeta
282     from tg_video_filter.models import Channel, DeliveryMode, TelegramSession, User,
VideoRecord
283 +from tg_video_filter.models import (
284 +    Channel,
285 +    ContentItem,
286 +    DeliveryMode,
287 +    Source,
288 +    TelegramSession,
289 +    User,
290 +    VideoRecord,
291 +)
292 +from tg_video_filter.policy import PolicyContext, StageResult, TopicPreferenceStage
293
294     log = logging.getLogger(__name__)
295
296     @@ -243,29 +252,56 @@ async def on_new_message(event: NewMessage.Event) -> None: #
pragma: no cover
297         )
298         return
299
300 +        source = await db.get_source_by_provider_id(conn, user.id, "telegram",
str(channel_id))
301 +
302         # Lazily record the channel (best-effort; failures are non-fatal).
303         try:
304             await db.upsert_channel(
305 +                channel = await db.upsert_channel(
306                     conn,
307                     Channel(user_id=user.id, channel_id=channel_id,
title=_channel_title(event)),
308                 )
309 +                if channel.id is not None:
310 +                    source = await db.get_source_by_id(conn, channel.id)
311             except Exception:
312                 log.warning(

```

```

313             "failed to upsert channel %s for user %s", channel_id, user.id,
exc_info=True
314         )
315
316     +     text = _message_text(message)
317     result = engine_holder[ENGINE_KEY].evaluate(meta)
318     +     matched = result.matched
319     +     reasons = list(result.reasons)
320     +     if matched and source is not None:
321     +         topic_result = await _evaluate_topic_preferences(
322     +             conn,
323     +             user.id,
324     +             source,
325     +             _content_item_preview(
326     +                 user.id,
327     +                 source,
328     +                 message_id,
329     +                 document_id,
330     +                 meta,
331     +                 text,
332     +                 matched,
333     +             ),
334     +         )
335     +         if topic_result is not None:
336     +             reasons.append(topic_result.reason)
337     +             matched = topic_result.passed
338     +
339     record = VideoRecord(
340         user_id=user.id,
341         channel_id=channel_id,
342         message_id=message_id,
343     +     text=text,
344     document_id=document_id,
345     duration_s=meta.duration_s,
346     width=meta.width,
347     height=meta.height,
348     size_bytes=meta.size_bytes,
349     mime_type=meta.mime_type,
350     -     matched=result.matched,
351     +     matched=matched,
352     )
353     try:
354         inserted = await db.insert_video(conn, record)
355     @@ -288,11 +324,11 @@ async def on_new_message(event: NewMessage.Event) -> None: #
pragma: no cover
356         user.id,
357         channel_id,
358         message_id,
359     -     result.matched,
360     -     result.reasons,
361     +     matched,
362     +     reasons,
363     )
364
365     -     if result.matched:
366     +     if matched:
367         # Build link text for LINK/BOTH modes (None for FORWARD).
368         link_text = None
369         if delivery_mode != DeliveryMode.FORWARD:
370     @@ -335,6 +371,70 @@ def _extract_document_id(message: Message) -> int | None:
371         return None
372
373
374     +def _message_text(message: object) -> str | None:
375     +     """Return caption/message text normalized for storage, or None if empty."""

```

```

376 + text = getattr(message, "message", None)
377 + if text is None:
378 +     text = getattr(message, "raw_text", None)
379 + if not isinstance(text, str):
380 +     return None
381 + text = text.strip()
382 + return text or None
383 +
384 +
385 +def _content_item_preview(
386 +    account_id: int,
387 +    source: Source,
388 +    message_id: int,
389 +    document_id: int | None,
390 +    meta: VideoMeta,
391 +    text: str | None,
392 +    matched: bool,
393 +) -> ContentItem:
394 +    """Build the ContentItem shape used by policy stages before insert."""
395 +    assert source.id is not None
396 +    return ContentItem(
397 +        account_id=account_id,
398 +        source_id=source.id,
399 +        provider=source.provider,
400 +        provider_message_id=str(message_id),
401 +        content_type="video",
402 +        text=text,
403 +        document_id=str(document_id) if document_id is not None else None,
404 +        duration_s=meta.duration_s,
405 +        width=meta.width,
406 +        height=meta.height,
407 +        size_bytes=meta.size_bytes,
408 +        mime_type=meta.mime_type,
409 +        dedup_key=db.compute_dedup_key(
410 +            provider=source.provider,
411 +            document_id=str(document_id) if document_id is not None else None,
412 +            source_id=source.id,
413 +            provider_message_id=str(message_id),
414 +            size_bytes=meta.size_bytes,
415 +            duration_s=meta.duration_s,
416 +            height=meta.height,
417 +            width=meta.width,
418 +        ),
419 +        matched=matched,
420 +    )
421 +
422 +
423 +async def _evaluate_topic_preferences(
424 +    conn: aiosqlite.Connection,
425 +    account_id: int,
426 +    source: Source,
427 +    content_item: ContentItem,
428 +) -> StageResult | None:
429 +    """Evaluate ADR 0012 topic preferences for a candidate item."""
430 +    policy = await db.get_default_policy(conn, account_id)
431 +    if policy is None:
432 +        return None
433 +    return await TopicPreferenceStage().evaluate(
434 +        PolicyContext(content_item=content_item, source=source, policy=policy)
435 +    )
436 +
437 +
438 +def _peer_id(peer: object) -> int:
439 +    """Best-effort numeric id for the peer of a message.
440

```

```

441     @@ -367,4 +467,7 @@ async def __call__(self, event: NewMessage.Event) -> None: ...
442         "build_client",
443         "extract_video_meta",
444         "make_new_message_handler",
445     +     "_content_item_preview",
446     +     "_evaluate_topic_preferences",
447     +     "_message_text",
448     ]
449     diff --git a/tests/test_db.py b/tests/test_db.py
450     index d81347b..54e14bb 100644
451     --- a/tests/test_db.py
452     +++ b/tests/test_db.py
453     @@ -230,6 +230,7 @@ async def test_insert_and_get_video_round_trip(conn:
aiosqlite.Connection) -> No
454         user_id=USER_ID,
455         channel_id=CHANNEL_ID,
456         message_id=42,
457     +     text="AI philosophy caption",
458         duration_s=120,
459         width=1920,
460         height=1080,
461     @@ -244,12 +245,19 @@ async def test_insert_and_get_video_round_trip(conn:
aiosqlite.Connection) -> No
462         fetched = await db.get_video(conn, USER_ID, CHANNEL_ID, 42)
463         assert fetched is not None
464         assert fetched.duration_s == 120
465     +     assert fetched.text == "AI philosophy caption"
466         assert fetched.height == 1080
467         assert fetched.size_bytes == 5_000_000
468         assert fetched.mime_type == "video/mp4"
469         assert fetched.matched is True
470         assert fetched.forwarded is False
471
472     +     src = await db.get_source_by_provider_id(conn, USER_ID, "telegram",
str(CHANNEL_ID))
473     +     assert src is not None and src.id is not None
474     +     item = await db.get_content_item_by_message(conn, USER_ID, src.id, "42")
475     +     assert item is not None
476     +     assert item.text == "AI philosophy caption"
477     +
478
479     async def test_insert_video_dedup_collision(conn: aiosqlite.Connection) -> None:
480         await _seed_user(conn)
481     diff --git a/tests/test_policy.py b/tests/test_policy.py
482     index 02234a7..18e64cf 100644
483     --- a/tests/test_policy.py
484     +++ b/tests/test_policy.py
485     @@ -3,15 +3,28 @@
486         from __future__ import annotations
487
488         from tg_video_filter.models import ContentItem, Policy, Source
489     -from tg_video_filter.policy import PolicyContext, StageResult, evaluate_pipeline
490     -
491     -
492     -def _ctx() -> PolicyContext:
493     +from tg_video_filter.policy import (
494     +    PolicyContext,
495     +    StageResult,
496     +    TopicPreferences,
497     +    TopicPreferenceStage,
498     +    evaluate_pipeline,
499     +    parse_topic_preferences,
500     +)
501     +
502     +

```

```

503 +def _ctx(
504 +     *,
505 +     policy_json: str = "{}",
506 +     source_title: str | None = None,
507 +     text: str | None = None,
508 +) -> PolicyContext:
509     return PolicyContext(
510         content_item=ContentItem(
511             account_id=111,
512             source_id=1,
513             provider_message_id="42",
514 +             text=text,
515             dedup_key="tg:msg:1:42",
516         ),
517         source=Source(
518 @@ -19,12 +32,13 @@ def _ctx() -> PolicyContext:
519             account_id=111,
520             provider="telegram",
521             provider_source_id="-1001",
522 +             title=source_title,
523         ),
524         policy=Policy(
525             id=1,
526             account_id=111,
527             name="default",
528 -             config_json="{}",
529 +             config_json=policy_json,
530             is_default=True,
531         ),
532     )
533 @@ -99,3 +113,89 @@ def test_stage_result_helpers_attach_metadata() -> None:
534     assert passed.metadata == {"duration_s": 60}
535     assert failed.passed is False
536     assert failed.metadata == {"topic": "crypto"}
537 +
538 +
539 +## --- TopicPreferenceStage -----
540 +
541 +
542 +def test_parse_topic_preferences_defaults_to_disabled() -> None:
543 +     assert parse_topic_preferences({}) == TopicPreferences()
544 +
545 +
546 +def test_parse_topic_preferences_rejects_invalid_mode() -> None:
547 +     try:
548 +         parse_topic_preferences({"topics": {"mode": "sometimes"}})
549 +     except ValueError as exc:
550 +         assert "topics.mode" in str(exc)
551 +     else: # pragma: no cover - assertion guard
552 +         raise AssertionError("invalid topic mode should raise")
553 +
554 +
555 +async def test_topic_preferences_disabled_passes() -> None:
556 +     result = await TopicPreferenceStage().evaluate(_ctx(text="anything"))
557 +
558 +     assert result.passed is True
559 +     assert result.reason == "topic preferences disabled"
560 +
561 +
562 +async def test_topic_preferences_include_any_matches_source_title() -> None:
563 +     ctx = _ctx(
564 +         policy_json='{"topics":{"include":["AI","Philosophy"],"mode":"any"}}',
565 +         source_title="Daily AI Clips",
566 +     )
567 +

```

```

568 +     result = await TopicPreferenceStage().evaluate(ctx)
569 +
570 +     assert result.passed is True
571 +     assert result.metadata == {"matched_include_topics": ["AI"]}
572 +
573 +
574 +async def test_topic_preferences_include_all_matches_caption_and_source() -> None:
575 +     ctx = _ctx(
576 +         policy_json='{"topics":{"include":["AI","Philosophy"],"mode":"all"}}',
577 +         source_title="AI Clips",
578 +         text="A philosophy lecture",
579 +     )
580 +
581 +     result = await TopicPreferenceStage().evaluate(ctx)
582 +
583 +     assert result.passed is True
584 +     assert result.reason == "matched all include topics"
585 +     assert result.metadata == {"matched_include_topics": ["AI", "Philosophy"]}
586 +
587 +
588 +async def test_topic_preferences_include_all_fails_when_missing_topic() -> None:
589 +     ctx = _ctx(
590 +         policy_json='{"topics":{"include":["AI","Philosophy"],"mode":"all"}}',
591 +         text="AI news",
592 +     )
593 +
594 +     result = await TopicPreferenceStage().evaluate(ctx)
595 +
596 +     assert result.passed is False
597 +     assert result.metadata == {
598 +         "matched_include_topics": ["AI"],
599 +         "missing_include_topics": ["Philosophy"],
600 +     }
601 +
602 +
603 +async def test_topic_preferences_exclude_vetoes_include_match() -> None:
604 +     ctx = _ctx(
605 +         policy_json='{"topics":{"include":["AI"],"exclude":["Crypto"],"mode":"any"}}',
606 +         text="AI crypto market explainer",
607 +     )
608 +
609 +     result = await TopicPreferenceStage().evaluate(ctx)
610 +
611 +     assert result.passed is False
612 +     assert result.reason == "excluded topic matched: Crypto"
613 +     assert result.metadata == {"matched_exclude_topics": ["Crypto"]}
614 +
615 +
616 +async def test_topic_preferences_invalid_config_fails_explainably() -> None:
617 +     result = await TopicPreferenceStage().evaluate(
618 +         _ctx(policy_json='{"topics":{"include":"AI"}}', text="AI")
619 +     )
620 +
621 +     assert result.passed is False
622 +     assert result.reason.startswith("invalid topic config:")
623 diff --git a/tests/test_telethon_extraction.py b/tests/test_telethon_extraction.py
624 index 49e8cfd..3c5ab20 100644
625 --- a/tests/test_telethon_extraction.py
626 +++ b/tests/test_telethon_extraction.py
627 @@ -23,7 +23,12 @@
628     PeerUser,
629 )
630
631 -from tg_video_filter.telethon_client import _extract_document_id, _peer_id,
extract_video_meta

```

```

632 +from tg_video_filter.telethon_client import (
633 +    _extract_document_id,
634 +    _message_text,
635 +    _peer_id,
636 +    extract_video_meta,
637 +)
638
639 CHANNEL_ID = 1001234567890
640
641 @@ -120,6 +125,25 @@ def test_text_message_returns_none() -> None:
642     assert extract_video_meta(_make_text_message()) is None
643
644
645 +def test_message_text_strips_caption() -> None:
646 +    msg = _make_video_message()
647 +    msg.message = " AI philosophy clip "
648 +    assert _message_text(msg) == "AI philosophy clip"
649 +
650 +
651 +def test_message_text_empty_returns_none() -> None:
652 +    msg = _make_video_message()
653 +    msg.message = " "
654 +    assert _message_text(msg) is None
655 +
656 +
657 +def test_message_text_raw_text_fallback() -> None:
658 +    class Msg:
659 +        raw_text = "Bangalore explorations"
660 +
661 +    assert _message_text(Msg()) == "Bangalore explorations"
662 +
663 +
664     def test_photo_message_returns_none() -> None:
665         msg = _make_text_message()
666         msg.media = MessageMediaPhoto(photo=None) # not a Document
667

```

#### 424. assistant (2026-07-06T21:24:31.582Z)

I've read the full F2 diff. It's substantive and well-structured: caption capture into `content\_items.text` (my finding #3), a `TopicPreferenceStage` with include/exclude/any-all, and hot-path wiring in both live and backfill. Key things I want to verify: the config is read via the raw (key-preserving) path, topic gating is correct AND-after-metadata, and a few edges (fail-closed on bad config, `evaluate\_pipeline` bypass). Let me check the gate and run an end-to-end repro.

#### 425. user (2026-07-06T21:24:47.074Z)

```

=== gate ===
=== RUFF ===
All checks passed!
36 files already formatted
=== PYTEST ===
-----
TOTAL                                1337    45    358    28    96%
Required test coverage of 80.0% reached. Total coverage: 95.58%
513 passed in 9.08s
DONE

=== end-to-end: caption?content_items.text + topics read via raw config + gating ===
caption stored in content_items.text: 'A philosophy lecture on AI'
topics readable after /set: ('AI', 'Philosophy') ('Crypto',) all
topic gate (AI+Philosophy present, all mode): True | matched all include topics
topic gate (crypto excluded): False | excluded topic matched: Crypto

```



#### 426. assistant (2026-07-06T21:25:03.186Z)

Verified end-to-end: **\*\*caption capture works\*\*** (`content\_items.text` populated ? finding #3 fixed), **\*\*topics survive a `/set` and are read via the raw path\*\*** (include/exclude/mode intact), and **\*\*topic gating is correct\*\*** (all-includes ? pass; exclude ? veto). Gate: ruff clean, **\*\*513 passed @ 95.58%\*\*** (dip is the new pragma-no-cover live wiring). The crux works; my findings are low/design-level. Let me run a focused adversarial pass on the live-path wiring (the riskiest, untested part).

#### 427. user (2026-07-06T21:25:50.696Z)

Workflow launched in background. Task ID: w327fvoqx  
Summary: Review telegram-video-filter PR #51 (F2 topic preferences + caption capture + live-path wiring) with adversarial verification  
Transcript dir: C:\Users\avidu\claud\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_c5395511-084  
Script file: C:\Users\avidu\claud\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr51-f2-wf\_c5395511-084.js  
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\claud\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr51-f2-wf\_c5395511-084.js"} to iterate without resending the script.)  
Run ID: wf\_c5395511-084  
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\claud\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr51-f2-wf\_c5395511-084.js", resumeFromRunId: "wf\_c5395511-084"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

#### 428. assistant (2026-07-06T21:26:23.907Z)

While the workflow runs, I confirmed caption propagation is complete across all read shims (`\_content\_item\_to\_video` ? used by `get\_video`/`get\_video\_by\_document\_id` ? plus the two bespoke constructions in `get\_video\_by\_content\_fingerprint`/`list\_videos\_for\_user` all now carry `text`). And the topic gate can only turn a metadata-matched item False (never delivers a topic-rejected one), with delivery correctly gated on the final `matched` in both live and backfill paths.

**\*\*Interim status on #51 (F2):\*\*** gate green (513 @ 95.58%), the crux (caption capture + topic matching + config round-trip) verified working; my findings so far are low/design-level (dormant until a `/set topics` command exists; the wiring bypasses `evaluate\_pipeline`; malformed-config fail-closed; hot-path glue untested). The adversarial pass is running ? I'll fold in its results and, **\*\*per your rule, only LGTM if it clears the bar\*\***; otherwise I'll post findings and hold. Back when it completes.

#### 429. user (2026-07-06T21:34:37.451Z)

```
<task-notification>
<task-id>w327fvoqx</task-id>
<tool-use-id>toolu_017La5myLtsWQZF11CMf38y3</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claud\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\w327fvoqx.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Review telegram-video-filter PR #51 (F2 topic preferences + caption capture + live-path wiring) with adversarial verification" completed</summary>
<result>{"counts":{"raw":14,"kept":12,"refuted":2},"findings":[{"dimension":"stage_config","severity":"medium","verdict":"CONFIRMED","title":"Malformed topics config fails closed and silently drops ALL matched deliveries with no surfaced reason","where":"src/tg_video_filter/policy.py:126","summary":"Any parse/validation error in the topics config makes TopicPreferenceStage.evaluate return StageResult.fail, which the wiring turns into matched=False, so every otherwise-matching video is silently dropped until the config is fixed."},"failure_scenario":"An operator (or a future /set-topics command) writes
```

policies.config\_json with a structurally invalid topics block, e.g. {"topics": {"include": "ai"}} (string instead of array) or {"topics": {"include": [123]}}. load\_topic\_preferences raises ValueError; the stage returns fail; on\_new\_message (telethon\_client.py:291) sets matched=topic\_result.passed=False; every matched video is dropped. Repro confirmed: case 4 -> 'invalid topic config: topic include/exclude values must be arrays', passed=False; case 7 -> 'topic include/exclude arrays must contain strings', passed=False. The only visible signal is the reason appended to the log line (telethon\_client.py:290,322) ? there is no user-facing error and no distinction from a legitimate topic mismatch. Because the F2 diff makes NO command able to set topics safely-validated (only save\_policy\_config/manual edit exist), this is latent today, but the fail-closed choice means the first bad config edit blackholes all delivery.", "evidence": "policy.py:122-126 catches (JSONDecodeError, ValueError) and returns StageResult.fail(); telethon\_client.py:289-291 does `matched = topic\_result.passed`. Repro output: `4 include=string -> DROPPED: False - invalid topic config: topic include/exclude values must be arrays`; `7 include=[123] -> DROPPED: False`; `13 config\_json array -> DROPPED: False`. ADR 0012 does not specify fail-open vs fail-closed for malformed config.", "recommendation": "Decide the failure posture explicitly. Fail-open (treat an unparseable topics block as 'topics disabled' and pass) is safer for a delivery pipeline ? a config typo should not stop all delivery. If fail-closed is intentional for safety, surface the invalidity loudly (dedicated log level/alert, and reject the config at write time in save\_policy\_config rather than silently at evaluate time) so it cannot silently blackhole delivery.", "verify\_reasoning": "The defect ships in this PR's diff and behaves as described. Repro confirms every malformed topics block makes TopicPreferenceStage.evaluate return StageResult.fail: include-as-string, include-of-ints, bad mode, non-object config\_json, and invalid JSON all yield passed=False (policy.py:123-126 catches JSONDecodeError/ValueError and returns fail). The wiring converts that fail directly into a drop: telethon\_client.py:289-291 does `matched = topic\_result.passed`, so a matched video with a malformed default-policy config is dropped. The only surfaced signal is the reason string appended to the log line at telethon\_client.py:290/322-329 ? there is no user-facing error and no way to distinguish a config-invalid drop from a legitimate topic mismatch. So the fail-closed-blackhole behavior is real.\n\nAdversarial checks that keep this at medium rather than high: (1) No command in this PR can write a malformed topics block. The F1 /set path is save\_filter\_config (db.py:1770-1793), which only merges flat FilterConfig keys via cfg.model\_dump(mode='json') and never emits a `topics` key; save\_policy\_config (db.py:1326) does no topic-schema validation but is not wired to any user command in this diff. Reaching the bad state requires a manual DB edit or a hypothetical future command ? the finding admits this. So it is latent, not triggerable by shipped commands today. (2) It is a genuine design-posture inconsistency: F1's own save\_filter\_config (db.py:1781-1791) fails OPEN, logging a warning and resetting a malformed blob to {}, whereas F2's evaluate path fails CLOSED and blackholes delivery for the same malformed blob. ADR 0012 does not specify the posture, so this is an unsettled design choice with a real latent failure mode, not a bug that produces wrong delivery under any shipped flow.\n\nNet: real, in-diff, correctly described behavior, but latent (no shipped write path) and posture-dependent ? consistent with the filed medium severity, not high.", {"dimension": "stage\_config", "severity": "medium", "verdict": "CONFIRMED", "title": "topics.mode is validated eagerly even when no include/exclude is configured, so an invalid mode on an otherwise-disabled block drops all delivery", "where": "src/tg\_video\_filter/policy.py:84", "summary": "parse\_topic\_preferences validates mode before TopicPreferences.enabled is consulted, so a config that is semantically a no-op (no include, no exclude) but carries a bad mode string raises ValueError and fails the stage closed, dropping everything.", "failure\_scenario": "Config {\"topics\": {\"mode\": \"whatever\"}} with no include and no exclude is semantically disabled (no filtering requested), but parse\_topic\_preferences raises 'topics.mode must be \\\"any\\\" or \\\"all\\\"' at policy.py:85-86, before the stage ever checks prefs.enabled (policy.py:127). Result: the stage fails and every matched video is dropped. Repro confirmed (case A): `mode-invalid, empty include/exclude -> DROP ALL?: False - invalid topic config: topics.mode must be 'any' or 'all'`. A leftover/typo'd mode key with the actual topic lists removed should be a no-op, not a full delivery outage.", "evidence": "policy.py:84-86 validates mode unconditionally inside parse\_topic\_preferences; TopicPreferences.enabled (policy.py:72-74) is only checked later at policy.py:127 after load has already raised. Repro r2.py case A output above.", "recommendation": "Only validate/consider mode when include or exclude is non-empty (or default an unknown mode to 'any' rather than raising). At minimum, short-circuit to a disabled TopicPreferences when include and exclude are both empty, before enforcing the mode enum.", "verify\_reasoning": "The defect ships in this PR's diff and produces the described wrong behavior. VERIFIED via git: parse\_topic\_preferences, TopicPreferences, and the mode validation at policy.py:86 do not exist in origin/main ? they are entirely new in this PR (#51). At policy.py:84-86 mode is validated unconditionally inside parse\_topic\_preferences, BEFORE

TopicPreferences.enabled (policy.py:72-74) is ever consulted. enabled is only checked at policy.py:127, after load\_topic\_preferences (policy.py:96) has already called parse and raised. Repro (r2.py) confirms: config {"topics\": {"mode\": \"whatever\"}} with no include/exclude raises 'topics.mode must be \"any\" or \"all\"' rather than parsing to a disabled TopicPreferences.\\n\\nFail-closed wiring confirmed end-to-end: TopicPreferenceStage.evaluate catches the ValueError and returns StageResult.fail (policy.py:125-126). In the live handler, telethon\_client.py:289-291 does `if topic\_result is not None: reasons.append(...); matched = topic\_result.passed` ? so passed=False sets matched=False, dropping the VideoRecord delivery. Repro output: stage passed=False, reason='invalid topic config: topics.mode must be ...'. So a config that is semantically a no-op (no include, no exclude requested) but carries a leftover/typo'd mode string causes every matched video to be dropped.\\n\\nSeverity corrected to medium (matches the finding). Reachability is the only mitigating factor: it requires the specific persisted shape {mode: &lt;bad&gt;} with BOTH include and exclude empty. topics live in the RAW config\_json (not the FilterConfig command surface), so this is not written by the normal /set FilterConfig path; it arises from a partial edit, a typo, or lists being cleared while a bad mode lingers ? an unusual but genuinely reachable state, not a happy-path outage. Note the same eager-validation also fail-closes a bad mode whe ... (truncated 42898 chars, full result in C:\\Users\\avidu\\AppData\\Local\\Temp\\claude\\C--Users-avidu-Projects-badminton-highlight-indexer\\7dc3da51-af38-47b6-83f6-fdda83b60664\\tasks\\w327fvoqx.output)</result>  
<usage><agent\_count>17</agent\_count><subagent\_tokens>814683</subagent\_tokens><tool\_uses>230</tool\_uses><duration\_ms>526683</duration\_ms></usage>  
</task-notification>

#### 430. assistant (2026-07-06T21:34:41.723Z)

The workflow found 14 ? 12 kept, 2 refuted ? both top findings are **medium** (fail-closed config edges), converging with my read. Let me parse the full set.

#### 431. user (2026-07-06T21:34:46.397Z)

COUNTS: {'raw': 14, 'kept': 12, 'refuted': 2}

[MEDIUM/CONFIRMED] (stage\_config) Malformed topics config fails closed and silently drops ALL matched deliveries with no surfaced reason

where: src/tg\_video\_filter/policy.py:126

rec: Decide the failure posture explicitly. Fail-open (treat an unparseable topics block as 'topics disabled' and pass) is safer for a delivery pipeline ? a config t

[MEDIUM/CONFIRMED] (stage\_config) topics.mode is validated eagerly even when no include/exclude is configured, so an invalid mode on an otherwise-disabled block drops all delivery

where: src/tg\_video\_filter/policy.py:84

rec: Only validate/consider mode when include or exclude is non-empty (or default an unknown mode to 'any' rather than raising). At minimum, short-circuit to a disab

[MEDIUM/CONFIRMED] (stage\_config) Config error handling is inconsistent: unknown keys fail open (silently disable filtering) while invalid values fail closed (silently drop all)

where: src/tg\_video\_filter/policy.py:77

rec: Pick one consistent posture. If topics config is meant to be strict, reject unknown keys in parse\_topic\_preferences (raise on any key not in {include, exclude,

[MEDIUM/CONFIRMED] (design\_tests) New topic-wiring helpers \_content\_item\_preview and \_evaluate\_topic\_preferences are entirely untested (happy path missed; coverage regressed 96.63->95.58)

where: src/tg\_video\_filter/telethon\_client.py:385

rec: Add a unit test that seeds a source + default policy with topics and asserts \_evaluate\_topic\_preferences returns pass on include-match, fail on exclude-veto, an

[LOW/CONFIRMED] (wiring) Malformed default-policy config fails closed: silently drops every metadata-matched video (live + backfill)

where: src/tg\_video\_filter/policy.py:126

rec: Decide the intended failure mode explicitly. If topics should fail OPEN on a malformed default policy (don't silently suppress all matched videos), have the sta

[LOW/CONFIRMED] (wiring) Backfill discards the topic decision reason and never logs a topic-

demoted video ? silent non-delivery with zero observability

where: src/tg\_video\_filter/backfill.py:260

rec: Mirror the live path: capture topic\_result.reason and add a log line (at least at DEBUG/INFO) recording matched + reason when the topic stage demotes a video du

[LOW/PLAUSIBLE] (stage\_config) Substring casefold matching yields false-positive topic matches (e.g. include 'AI' matches 'rain'/'again')

where: src/tg\_video\_filter/policy.py:173

rec: Acknowledged as an ADR v1 limitation, so no code change required for this PR. Worth a scope note that short include/exclude terms will over-match; a future stag

[LOW/CONFIRMED] (design\_tests) F1 orchestrator evaluate\_pipeline is dead in production; F2 hand-wires TopicPreferenceStage directly, so no unified StageResult trace is built

where: src/tg\_video\_filter/policy.py:176

rec: Either route topic evaluation through evaluate\_pipeline (wrapping FilterEngine as a PolicyStage) so the StageResult trace is real, or disclose in the PR/ADR tha

[LOW/CONFIRMED] (design\_tests) Topic gating is dormant: no /set topics command exists, so config\_json.topics can only be written via the internal save\_policy\_config

where: src/tg\_video\_filter/policy.py:145

rec: Confirm in the PR description that F2 is intentionally config-plumbing-only and the user-facing /set topics command is a later phase; otherwise the feature is u

[LOW/CONFIRMED] (design\_tests) Malformed topics config fails closed, dropping a filter-matched video with no user-visible recourse

where: src/tg\_video\_filter/policy.py:110

rec: Consider fail-open (treat invalid topic config as disabled, log a warning) so a malformed optional stage never suppresses an otherwise-matched delivery, or surf

[LOW/CONFIRMED] (design\_tests) backfill\_user discards the topic StageResult reason that on\_new\_message logs, creating an observability inconsistency between the two wired paths

where: src/tg\_video\_filter/backfill.py:260

rec: Capture and log the topic reason in backfill\_user for parity with on\_new\_message, or extract the shared gating+logging into one helper so both paths behave iden

[LOW/CONFIRMED] (design\_tests) \_content\_item\_preview computes a dedup\_key that is never consumed (throwaway work on every matched message)

where: src/tg\_video\_filter/telethon\_client.py:409

rec: If ContentItem allows it, pass a sentinel/empty dedup\_key in the preview (it is thrown away) or add a comment noting the preview key is non-authoritative, to av

distinct: 12 of 12

--- refuted: ['Topic matching is substring (not word-boundary), causin', 'When the source row is absent, topic filtering (includi']

#### 432. assistant (2026-07-06T21:35:36.420Z)

The workflow converged with my read: **no highs/blockers**, but a cluster of **medium** config-posture issues (fail-closed) plus an untested-wiring gap. The most concrete is the **eager `mode` validation** ? a semantically-disabled **topics** block with a stray **mode** drops **all** delivery. Per my rule, let me reproduce that one myself before deciding.

#### 433. user (2026-07-06T21:35:44.526Z)

A: empty block + stray bad mode (should be no-op) -> passed=False | invalid topic config: topics.mode must be 'an

B: include as string (malformed) -> passed=False | invalid topic config: topic include/exclude v

C: no topics key at all (disabled) -> passed=True | topic preferences disabled

D: valid include, mismatched caption -> passed=False | no include topics matched

#### 434. assistant (2026-07-06T21:36:08.967Z)

Confirmed the key one ? **Case A**: a semantically-disabled **topics** block (no include/exclude)

with a stray `mode` ? `passed=False` ? **\*\*drops all delivery\*\***. That's a clear over-strictness bug (a no-op config shouldn't stop delivery), plus the broader fail-closed posture and an untested wiring gap.

Per your standing rule, I'm **\*\*not\*\*** giving an unqualified LGTM here ? the substance is solid and verified, but there are small, real medium issues I'd want addressed first. Let me post a review that holds the LGTM.

435. assistant (2026-07-06T21:36:36.445Z)

## Review ? F2 topic preferences + caption capture

Solid, well-structured slice ? the substance is verified working. **\*\*Holding the LGTM on a few small config-posture issues + a wiring test\*\*** (all latent today, but one is a clear bug worth fixing before this lands). Everything was reproduced locally against the branch.

### Verified ?

```
| Check | Result |
|---|---|
| `ruff check` / `format` | ? clean |
| `pytest` | ? **513 passed @ 95.58%** (dip is the new pragma-no-cover live wiring) |
| Caption capture (my ADR review #3) | ? caption reaches `content_items.text`; propagated through **all** read shims |
| Topic config round-trip | ? `topics` survive a legacy `/set` (F1 merge) and are read via the **raw** config path |
| Topic gating | ? include any/all + exclude-veto correct; can only turn a metadata-matched item False (never delivers a rejected one); live + backfill gate delivery consistently |
```

### ? Medium ? worth addressing before merge (all small)

**\*\*1. Disabled `topics` block with a stray `mode` drops ALL delivery\*\*** ? `policy.py:84`  
`parse\_topic\_preferences` validates `mode` **\*before\*** checking whether any include/exclude is configured. So `{ "topics": { "mode": "whatever" } }` (no include, no exclude ? semantically a no-op) raises ? the stage fails ? every matched video is dropped. Reproduced:

```
empty block + stray bad mode -> passed=False ("invalid topic config: topics.mode must be 'any' or 'all'")
no topics key                 -> passed=True (disabled) ?
```

> **\*\*Fix (2 lines):\*\*** short-circuit to a disabled `TopicPreferences` when `include` and `exclude` are both empty, before enforcing the `mode` enum.

**\*\*2. Malformed `topics` config fails \*closed\* ? silently blackholes all matched delivery\*\*** ? `policy.py:126`

Any parse/validation error ? `StageResult.fail` ? `matched=False` ? every otherwise-matching video is dropped, with the only signal being a log-line reason (indistinguishable from a legitimate topic mismatch). This is **\*\*inconsistent\*\*** with F1's own `save\_filter\_config`, which fails **\*open\*** (logs + resets a corrupt blob to `{}`). Latent today (no shipped command writes `topics`; only `save\_policy\_config`/manual edit), but the first bad edit stops all delivery.  
> **\*\*Fix:\*\*** pick a posture and make it consistent ? fail-**\*\*open\*\*** (treat an unparseable `topics` block as disabled + log a warning) is safer for a delivery pipeline; if fail-closed is intentional, surface it loudly and validate at **\*write\*** time.

**\*\*3. The new wiring helpers are untested\*\*** ? `telethon\_client.py:385` (`\_content\_item\_preview`, `\_evaluate\_topic\_preferences`)

The `TopicPreferenceStage` unit is well-tested, but the integration glue that actually decides delivery is behind `# pragma: no cover` and has no test (coverage 96.63 ? 95.58). These helpers take `conn` + models and are unit-testable without a live client.

> **\*\*Fix:\*\*** add a test that seeds a source + default policy with `topics` and asserts `\_evaluate\_topic\_preferences` passes on include-match, fails on exclude-veto ? this pins the gating decision.

### ? Low (polish)

- **\*\*Backfill drops the topic reason\*\*** (`backfill.py:260`) ? `on\_new\_message` appends

`topic\_result.reason` to its log line; `backfill\_user` doesn't, so a topic-demoted video is silently non-delivered in backfill. Mirror the live path (or extract a shared gate+log helper).  
- **`evaluate\_pipeline` is bypassed** (`policy.py:176`) ? F2 hand-wires `TopicPreferenceStage` after `FilterEngine` (which isn't a `PolicyStage`), so F1's orchestrator stays unused and no unified `StageResult` trace is built. Fine as an incremental bridge ? worth a note that a later phase should route both through `evaluate\_pipeline`.  
- **Dormant feature** (scope note) ? there's no `/set topics` command yet, so `topics` is only settable via `save\_policy\_config`/manual edit; the stage is inert until a config command lands. Confirm this is intentional plumbing-first.  
- **`\_content\_item\_preview` computes a `dedup\_key` that's thrown away** (`telethon\_client.py:409`) ? minor per-message waste; a comment or sentinel would clarify it's non-authoritative.  
- Substring `casefold` matching over-matches (e.g. `AI` in `rain`) ? acknowledged ADR v1 limitation; no change needed.

### Refuted during review (no action)

Source-absent skip (source is always non-None past the watched gate) and the substring "word-boundary" framing (an accepted ADR limitation, not a defect).

---

**Holding LGTM** pending #1 (the clear disabled-block bug), a decision on #2's posture, and #3 (a wiring test). All small ? ping me when they're in and I'll re-verify and sign off. The caption-capture + topic-matching core is genuinely good.

<sub>? Reviewed with [Claude Code](https://claude.com/claude-code) ? crux + config edges reproduced against the branch. Multi-agent adversarial pass: 12 findings kept, 2 refuted.</sub>

### 436. user (2026-07-06T21:36:36.477Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr51\_review.md (file state is current in your context ? no need to Read it back)

### 437. user (2026-07-06T21:36:46.394Z)

=== post review (holding LGTM) on #51 ===  
<https://github.com/avidullu/telegram-video-filter/pull/51#issuecomment-4897835327>

=== clean up worktree ===  
removed  
C:/Users/avidu/Projects/Telegram 2aad4d6 [main]

### 438. assistant (2026-07-06T21:36:51.867Z)

Review posted (LGTM held) ? [issuecomment-4897835327](https://github.com/avidullu/telegram-video-filter/pull/51#issuecomment-4897835327). Worktree removed. Now re-fresh `main` and re-arm the watch.

### 439. user (2026-07-06T21:36:57.337Z)

Command running in background with ID: b3dbziwg9. Output is being written to:  
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/tasks/b3dbziwg9.output. You will be notified when it completes. To check interim output, use Read on that file path.

### 440. assistant (2026-07-06T21:37:11.716Z)

PR #51 handled and the watch is re-armed. Summary:

### PR #51 (F2 topic preferences) ? reviewed, **LGTM held**

[Review posted](https://github.com/avidullu/telegram-video-filter/pull/51#issuecomment-4897835327). Applying your standing rule, I did **not** sign off ?

the substance is solid but I'm not fully satisfied yet.

**\*\*Verified good:\*\*** ruff clean · **\*\*513 passed @ 95.58%\*\*** · caption capture reaches `content\_items.text` (my earlier ADR finding #3) · `topics` survive a `/set` and read via the raw path · topic gating correct (include any/all + exclude-veto, never delivers a rejected item).

**\*\*Holding on 3 small mediums (all reproduced, all latent today):\*\***

1. ? **\*\*Clear bug\*\*** ? a disabled `topics` block with a stray `mode` (no include/exclude) drops **\*all\*** delivery (`mode` is validated before checking `enabled`). 2-line fix.
2. ? **\*\*Fail-closed posture\*\*** ? a malformed `topics` config silently blackholes all matched delivery, inconsistent with F1's fail-open `save\_filter\_config`. Needs a posture decision.
3. ? **\*\*Untested wiring\*\*** ? the integration helpers that decide delivery (`\_evaluate\_topic\_preferences`/`\_content\_item\_preview`) have no test.

Plus lows (backfill drops the topic log reason; `evaluate\_pipeline` bypassed; no `/set topics` command yet so the feature's dormant; throwaway `dedup\_key`). Two weak findings were correctly refuted.

I offered to **\*\*re-verify and sign off promptly\*\*** once #1?#3 are addressed (or the author decides the posture) ? same cycle as #35/#39.

### Watch re-armed ?

Fast-forwarded `main` (now has merged #50), re-armed the PR-watch (`b3dbziwg9`). It'll wake me on the next new/changed PR ? including if the author pushes fixes that flip #51, which I'll re-review and only then LGTM if satisfied.

#### 441. user (2026-07-06T21:57:58.818Z)

Update?

#### 442. assistant (2026-07-06T21:58:03.836Z)

Let me check the watch's progress and the current PR state.

#### 443. user (2026-07-06T21:58:11.583Z)

=== PR-watch interim output ===

poll 3/30: no change (21:43:03Z)  
poll 4/30: no change (21:45:03Z)  
poll 5/30: no change (21:47:04Z)  
poll 6/30: no change (21:49:05Z)  
poll 7/30: no change (21:51:06Z)  
poll 8/30: no change (21:53:07Z)  
poll 9/30: no change (21:55:08Z)  
poll 10/30: no change (21:57:09Z)

=== current open PRs ===

#51 draft=false upd=2026-07-06T21:38:59Z ? feat(policy): add F2 topic preferences

=== any new commit on #51 branch since my review (78c4f8f)? ===

7182732 feat(policy): add F2 topic preferences  
(empty = author hasn't pushed a fix yet)

#### 444. assistant (2026-07-06T21:58:32.066Z)

The #51 branch was **\*\*force-pushed\*\*** since my review ? HEAD moved `78c4f8f` ? `7182732` (same commit message, so an amend/rebase). My PR-watch only triggers on new PR **\*numbers\*/draft** flips, not force-pushes within a PR, so it didn't fire. Let me see whether this addresses my findings.

#### 445. user (2026-07-06T21:58:35.000Z)

=== is 7182732 a rebase or a content change? diff 78c4f8f..7182732 ===  
src/tg\_video\_filter/backfill.py | 10 ++++++

```
src/tg_video_filter/policy.py | 22 ++++++--
tests/test_policy.py          | 73 ++++++-----
3 files changed, 98 insertions(+), 7 deletions(-)
```

=== full content diff (what changed vs what I reviewed) ===

```
diff --git a/src/tg_video_filter/backfill.py b/src/tg_video_filter/backfill.py
```

```
index 7a370f2..15a7aa2 100644
```

```
--- a/src/tg_video_filter/backfill.py
```

```
+++ b/src/tg_video_filter/backfill.py
```

```
@@ -242,6 +242,7 @@ async def backfill_user( # pragma: no cover - requires live Telethon
connection
```

```
    text = _message_text(message)
```

```
    result = engine.evaluate(meta)
```

```
    matched = result.matched
```

```
+    reasons = list(result.reasons)
```

```
    if matched and source is not None:
```

```
        topic_result = await _evaluate_topic_preferences(
```

```
            conn,
```

```
@@ -258,6 +259,7 @@ async def backfill_user( # pragma: no cover - requires live Telethon
connection
```

```
        ),
```

```
    )
```

```
    if topic_result is not None:
```

```
+        reasons.append(topic_result.reason)
```

```
        matched = topic_result.passed
```

```
    record = VideoRecord(
```

```
@@ -274,6 +276,14 @@ async def backfill_user( # pragma: no cover - requires live Telethon
connection
```

```
        matched=matched,
```

```
    )
```

```
    inserted = await db.insert_video(conn, record)
```

```
+    log.info(
```

```
+        "backfill_video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
```

```
+        user.id,
```

```
+        channel_id,
```

```
+        message_id,
```

```
+        matched,
```

```
+        reasons,
```

```
+    )
```

```
    if matched:
```

```
        summary["videos_matched"] += 1
```

```
diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
```

```
index 395435a..86db324 100644
```

```
--- a/src/tg_video_filter/policy.py
```

```
+++ b/src/tg_video_filter/policy.py
```

```
@@ -8,11 +8,14 @@ the first failure. Later Track F phases add concrete stages.
```

```
    from __future__ import annotations
```

```
    import json
```

```
+import logging
```

```
    from dataclasses import dataclass, field
```

```
    from typing import Protocol
```

```
    from tg_video_filter.models import ContentItem, Policy, Source
```

```
+log = logging.getLogger(__name__)
```

```
+
```

```
    @dataclass(frozen=True)
```

```
    class PolicyContext:
```

```
@@ -81,12 +84,16 @@ def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
```

```
    return TopicPreferences()
```

```
    if not isinstance(raw_topics, dict):
```



```

        raise ValueError("topics must be a JSON object")
+     include = _string_tuple(raw_topics.get("include"))
+     exclude = _string_tuple(raw_topics.get("exclude"))
+     if not include and not exclude:
+         return TopicPreferences()
    mode = raw_topics.get("mode", "any")
    if mode not in ("any", "all"):
        raise ValueError("topics.mode must be 'any' or 'all'")
    return TopicPreferences(
-         include=_string_tuple(raw_topics.get("include")),
-         exclude=_string_tuple(raw_topics.get("exclude")),
+         include=include,
+         exclude=exclude,
        mode=str(mode),
    )

@@ -123,7 +130,16 @@ class TopicPreferenceStage:
    try:
        prefs = load_topic_preferences(ctx.policy)
    except (json.JSONDecodeError, ValueError) as exc:
-         return StageResult.fail(self.stage_name, f"invalid topic config: {exc}")
+         log.warning(
+             "topic preferences disabled due to invalid config policy_id=%s",
+             ctx.policy.id,
+             exc_info=True,
+         )
+         return StageResult.pass_(
+             self.stage_name,
+             f"invalid topic config ignored: {exc}",
+             invalid_topic_config=True,
+         )
    if not prefs.enabled:
        return StageResult.pass_(self.stage_name, "topic preferences disabled")

diff --git a/tests/test_policy.py b/tests/test_policy.py
index 18e64cf..ca6e424 100644
--- a/tests/test_policy.py
+++ b/tests/test_policy.py
@@ -2,6 +2,9 @@

    from __future__ import annotations

+import aiosqlite
+
+from tg_video_filter import db
+    from tg_video_filter.models import ContentItem, Policy, Source
+    from tg_video_filter.policy import (
+        PolicyContext,
@@ -11,6 +14,7 @@ from tg_video_filter.policy import (
    evaluate_pipeline,
    parse_topic_preferences,
)
+from tg_video_filter.telethon_client import _evaluate_topic_preferences

def _ctx(
@@ -124,13 +128,20 @@ def test_parse_topic_preferences_defaults_to_disabled() -> None:

def test_parse_topic_preferences_rejects_invalid_mode() -> None:
    try:
-         parse_topic_preferences({"topics": {"mode": "sometimes"}})
+         parse_topic_preferences({"topics": {"include": ["AI"], "mode": "sometimes"}})
    except ValueError as exc:
        assert "topics.mode" in str(exc)
    else: # pragma: no cover - assertion guard

```

```

        raise AssertionError("invalid topic mode should raise")

+def test_parse_topic_preferences_empty_block_with_invalid_mode_is_disabled() -> None:
+    assert (
+        parse_topic_preferences({"topics": {"include": [], "exclude": [], "mode":
"sometimes"}})
+        == TopicPreferences()
+    )
+
+
+    async def test_topic_preferences_disabled_passes() -> None:
+        result = await TopicPreferenceStage().evaluate(_ctx(text="anything"))

@@ -192,10 +203,64 @@ async def test_topic_preferences_exclude_vetoes_include_match() -> None:
+        assert result.metadata == {"matched_exclude_topics": ["Crypto"]}

-async def test_topic_preferences_invalid_config_fails_explainably() -> None:
+async def test_topic_preferences_invalid_config_fails_open_with_explanation() -> None:
+    result = await TopicPreferenceStage().evaluate(
+        _ctx(policy_json='{"topics":{"include":"AI"}}', text="AI")
+    )

-    assert result.passed is False
-    assert result.reason.startswith("invalid topic config:")
+    assert result.passed is True
+    assert result.reason.startswith("invalid topic config ignored:")
+    assert result.metadata == {"invalid_topic_config": True}
+
+
+async def test_evaluate_topic_preferences_uses_default_policy_config() -> None:
+    conn = await aiosqlite.connect(":memory:")
+    try:
+        await db._apply_schema(conn)
+        await db.upsert_user(conn, db.User(id=111, phone="+15555550100")) # type: ignore[attr-
defined]
+        src = await db.upsert_source(
+            conn,
+            Source(
+                account_id=111,
+                provider="telegram",
+                provider_source_id="-1001",
+                title="AI Clips",
+            ),
+        )
+        assert src.id is not None
+        await db.save_policy_config(
+            conn,
+            111,
+            {"topics": {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"}},
+        )
+        item = ContentItem(
+            account_id=111,
+            source_id=src.id,
+            provider_message_id="42",
+            text="philosophy lecture",
+            dedup_key="tg:msg:1:42",
+        )
+
+        result = await _evaluate_topic_preferences(conn, 111, src, item)
+
+        assert result is not None
+        assert result.passed is True
+        assert result.metadata == {"matched_include_topics": ["AI"]}

```

```

+
+     await db.save_policy_config(
+         conn,
+         111,
+         {"topics": {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"}},
+     )
+     rejected = await _evaluate_topic_preferences(
+         conn,
+         111,
+         src,
+         item.model_copy(update={"text": "crypto discussion"}),
+     )
+
+     assert rejected is not None
+     assert rejected.passed is False

```

#### 446. assistant (2026-07-06T21:59:02.308Z)

The author **force-pushed** fixes\* (`7182732``) that directly address my review:

- **#1** (disabled-block bug): **parse\_topic\_preferences`** now returns disabled when include+exclude are both empty **before`** validating **mode`** ?
- **#2** (fail-closed): **malformed config** now **fails-open`** (`StageResult.pass_`` + warning log + `invalid_topic_config=True``) instead of dropping delivery ?
- **#3** (untested wiring): **new` test\_evaluate\_topic\_preferences\_uses\_default\_policy\_config`** end-to-end test ?
- **Low** (backfill observability): **added the` backfill\_video\_seen`** log line with reasons ?

Per my rule, I verify before signing off. Let me set up a worktree at the new HEAD, run the gate, and reproduce the two behavior changes.

#### 447. user (2026-07-06T21:59:09.335Z)

HEAD is now at 7182732 feat(policy): add F2 topic preferences  
gate(fix) started

#### 448. user (2026-07-06T21:59:21.176Z)

Traceback (most recent call last):

```

File "C:\Users\avidu\Projects\_wt-pr51b\src\tg_video_filter\policy.py", line 131, in evaluate
    prefs = load_topic_preferences(ctx.policy)
File "C:\Users\avidu\Projects\_wt-pr51b\src\tg_video_filter\policy.py", line 106, in
load_topic_preferences
    return parse_topic_preferences(decoded)
File "C:\Users\avidu\Projects\_wt-pr51b\src\tg_video_filter\policy.py", line 87, in
parse_topic_preferences
    include = _string_tuple(raw_topics.get("include"))
File "C:\Users\avidu\Projects\_wt-pr51b\src\tg_video_filter\policy.py", line 113, in
_string_tuple
    raise ValueError("topic include/exclude values must be arrays")
ValueError: topic include/exclude values must be arrays

```

Traceback (most recent call last):

```

File "C:\Users\avidu\Projects\_wt-pr51b\src\tg_video_filter\policy.py", line 131, in evaluate
    prefs = load_topic_preferences(ctx.policy)
File "C:\Users\avidu\Projects\_wt-pr51b\src\tg_video_filter\policy.py", line 105, in
load_topic_preferences
    raise ValueError("policy config_json must decode to a JSON object")
ValueError: policy config_json must decode to a JSON object

```

```

#1 empty block + stray bad mode (was DROP-ALL)    -> passed=True | topic preferences disabled
#2 malformed include=string (was DROP-ALL)        -> passed=True | invalid topic config ignored:
topic includ
#2 non-object config_json (was DROP-ALL)          -> passed=True | invalid topic config ignored:
policy confi
sanity: real exclude still vetoes                -> passed=False | excluded topic matched:
caption

```

sanity: real include no-match still fails -> passed=False | no include topics matched

#### 449. assistant (2026-07-06T21:59:33.546Z)

The fixes are **verified** (the tracebacks are just the new `log.warning(..., exc_info=True)` emitting ? the functions return `pass_``):

- **#1** empty block + bad mode ? `passed=True`` (disabled, no-op) ?
- **#2** malformed config (both string-include and non-object) ? `passed=True`` (fail-open + warning) ?
- Legitimate rejections still work (exclude veto, include no-match ? `False``) ?

Let me confirm the gate (including the new wiring test) is green.

#### 450. user (2026-07-06T21:59:40.714Z)

```
=== gate (fix) ===
=== RUFF ===
All checks passed!
36 files already formatted
=== PYTEST ===
```

```
-----
TOTAL                                1344    42    360    29    96%
Required test coverage of 80.0% reached. Total coverage: 95.83%
515 passed in 9.68s
DONE
```

#### 451. assistant (2026-07-06T22:00:29.944Z)

Gate on the fix: ruff clean, **515 passed @ 95.83%** (+2 tests, coverage up). All three mediums + the observability low are verified fixed, and legitimate topic rejection still works. Per my standing rule, I'm now **satisfied** ? this is merge-ready. Posting the LGTM.

#### 452. user (2026-07-06T22:00:43.956Z)

```
=== post LGTM ===
https://github.com/avidullu/telegram-video-filter/pull/51#issuecomment-4897994182
=== cleanup worktree ===
removed
```

#### 453. assistant (2026-07-06T22:01:17.889Z)

LGTM posted ? [issuecomment-4897994182](https://github.com/avidullu/telegram-video-filter/pull/51#issuecomment-4897994182). Worktree cleaned up.

Now to your instruction ? **the watcher needs to catch force-pushes and PR-thread comments**, not just new PRs. The current watcher only fingerprints `number:isDraft``, so it missed the #51 force-push (which is exactly how I noticed the gap myself). Let me stop the old watcher and arm an improved one that also tracks each PR's head SHA (force-push) and `updatedAt`` (comment/review activity).

#### 454. user (2026-07-06T22:01:30.239Z)

```
{"message": "Successfully stopped task: b3dbziwg9 (cd \"C:/Users/avidu/Projects/Telegram\" && git pull --ff-only 2>&1 | tail -1 && echo \"local main: $(git log --oneline -1)\" && REPO=\"avidullu/telegram-video-filter\" && BASELINE=$(gh pr list --repo \"$REPO\" --state open --json number,isDraft --jq '[ | \"\\(\\.number):\\(\\.isDraft)\"' | sort) && echo \"=== PR-watch RE-ARMED. Baseline: ===\" && echo \"$BASELINE\" && for i in $(seq 1 30); do\\n sleep 120\\n CUR=$(gh pr list --repo \"$REPO\" --state open --json number,isDraft --jq '[ | \"\\(\\.number):\\(\\.isDraft)\"' | sort)\\n ADDED=$(comm -13 <(echo \"$BASELINE\") <(echo \"$CUR\"))\\n if [ -n \"$ADDED\" ]; then\\n echo \"=== NEW / CHANGED OPEN PR DETECTED (poll $i, $(date -u +%H:%M:%SZ)) ===\"\\n echo \"new-or-changed (number:isDraft):\"; echo \"$ADDED\"\\n echo \"--- current open PRs ---\"\\n gh pr list --repo \"$REPO\" --state open --json number,title,isDraft,author --jq '[ | \"#\\(\\.number) draft=\\(\\.isDraft) by \\(\\.author.login): \\(\\.title)\"'\\n exit 0\\n fi\\n echo \"poll $i/30: no change ($(date -u
```

```
+%H:%M:%SZ))\"\\ndone\\necho \"=== PR-watch window elapsed (~60 min), no new PR
===\")\", \"task_id\": \"b3dbziwg9\", \"task_type\": \"local_bash\", \"command\": \"cd
\\\"C:/Users/avidu/Projects/Telegram\\\" && git pull --ff-only 2>&1 | tail -1 && echo \\\"local main:
$(git log --oneline -1)\\\" && REPO=\\\"avidullu/telegram-video-filter\\\" && BASELINE=$(gh pr list
--repo \\\"$REPO\\\" --state open --json number,isDraft --jq '.[] | \\\"\\\"(.number):\\\"\\\"(.isDraft)\\\"\" |
sort) && echo \\\"=== PR-watch RE-ARMED. Baseline: ===\\\" && echo \\\"$BASELINE\\\" && for i in $(seq 1
30); do\\n sleep 120\\n CUR=$(gh pr list --repo \\\"$REPO\\\" --state open --json number,isDraft
--jq '.[] | \\\"\\\"(.number):\\\"\\\"(.isDraft)\\\"\" | sort)\\n ADDED=$(comm -13 <(echo \\\"$BASELINE\\\")
<(echo \\\"$CUR\\\"))\\n if [ -n \\\"$ADDED\\\" ]; then\\n echo \\\"=== NEW / CHANGED OPEN PR DETECTED
(poll $i, $(date -u +%H:%M:%SZ)) ===\\\"\\n echo \\\"new-or-changed (number:isDraft):\\\"; echo
\\\"$ADDED\\\"\\n echo \\\"--- current open PRs ---\\\"\\n gh pr list --repo \\\"$REPO\\\" --state open
--json number,title,isDraft,author --jq '.[] | \\\"#\\\"\\\"(.number) draft=\\\"\\\"(.isDraft) by
\\\"\\\"(.author.login): \\\"\\\"(.title)\\\"\"\\n exit 0\\n fi\\n echo \\\"poll $i/30: no change ($(date -u
+%H:%M:%SZ))\\\"\\ndone\\necho \"=== PR-watch window elapsed (~60 min), no new PR ===\"}]}
```

#### 455. assistant (2026-07-06T22:01:54.806Z)

Old watcher stopped. Now arming the **improved watcher** ? it fingerprints each open PR as ``number:draft:headSHA:updatedAt`` in a single ``gh`` call per poll, so it wakes me on **new PRs**, draft flips, force-pushes (head SHA), and comment/review-thread activity (`updatedAt`) ? and prints what changed so I know whether to re-review or read a response.

#### 456. user (2026-07-06T22:02:00.658Z)

Command running in background with ID: `ba7m621fq`. Output is being written to:  
`C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\ba7m621fq.output`. You will be notified when it completes. To check interim output, use Read on that file path.

#### 457. assistant (2026-07-06T22:02:16.608Z)

Both done ? #51 finished and the watcher upgraded.

### PR #51 (F2) ? fixes verified, **LGTM'd** ?

The author force-pushed ``7182732`` addressing my review; I re-verified and signed off  
 ([LGTM](https://github.com/avidullu/telegram-video-filter/pull/51#issuecomment-4897994182)):

- **#1** disabled-block-drops-all ? fixed (returns disabled before validating ``mode``) ? reproduced
- **#2** fail-closed ? now **fails open** (pass + warning + ``invalid_topic_config`` flag) ? reproduced
- **#3** untested wiring ? new end-to-end ``_evaluate_topic_preferences`` test
- backfill now logs the topic reason (parity with live)
- Gate: ruff clean, **515 passed @ 95.83%** (+2 tests). Legitimate exclude-veto / include-no-match still reject correctly.

### Watcher upgraded ? (per your instruction)

You were right ? the old watcher only tracked ``number:isDraft``, so it **missed the #51 force-push** (I only caught it by manually re-checking). Fixed:

- **Stopped** the old watcher and **armed v2** (``ba7m621fq``).
- v2 fingerprints each open PR as ``#num:draft:headSHA:updatedAt`` in one ``gh`` call per poll, so it now wakes me on:
  - ? new PRs
  - ? draft?ready
  - **force-pushes** (head SHA change)
  - **comment/review-thread activity** (``updatedAt`` bump ? where the author responds)
- On wake it prints exactly what changed, so I can tell a force-push (? re-review) from a comment (? read the response) from a new PR (? full review) ? and I'll only LGTM once satisfied, every time.

Baseline captured **after** my LGTM, so it won't spuriously fire on my own comment. I'll surface the next activity ? author reply, fix push, or new PR ? as soon as it lands.

#### 458. user (2026-07-06T22:06:06.180Z)

```
<task-notification>
<task-id>ba7m621fq</task-id>
<tool-use-id>toolu_01XWgUSRcwbUpVZKxDNkvTfZ</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\ba7m621fq.output</output-file>
<status>completed</status>
<summary>Background command "Fast-forward main and arm improved PR-watch (force-pushes + comments)" completed (exit code 0)</summary>
</task-notification>
```

#### 459. assistant (2026-07-06T22:06:08.652Z)

The upgraded watcher fired. Let me see what changed.

#### 460. user (2026-07-06T22:06:10.918Z)

```
Already up to date.
local main: fb2d2ea feat(policy): add F1 pipeline primitives (#50)
=== PR-watch v2 ARMED ? tracks: new PRs, draft flips, FORCE-PUSHES (sha), COMMENT/REVIEW
activity (upd) ===
baseline:
#51 draft=false sha=718273201 upd=2026-07-06T22:00:43Z
poll 1/30: no change (22:04:04Z)
=== PR ACTIVITY DETECTED (poll 2, 22:06:05Z) ===
--- baseline ---
#51 draft=false sha=718273201 upd=2026-07-06T22:00:43Z
--- now ---

--- new/changed lines ---

--- open PR details ---
```

#### 461. assistant (2026-07-06T22:06:17.074Z)

The watcher caught a change: the open-PR set went from `#51` to **empty** ? so #51 was merged (or closed) right after my LGTM. Let me confirm.

#### 462. user (2026-07-06T22:06:24.403Z)

```
=== #51 state ===
state=MERGED mergedAt=2026-07-06T22:05:58Z mergeCommit=6d3d7a6f9

=== origin/main recent ===
6d3d7a6 feat(policy): add F2 topic preferences (#51)
fb2d2ea feat(policy): add F1 pipeline primitives (#50)
2aad4d6 docs(adr): propose 0012 policy pipeline ? explainable, tiered, user-customizable content
gates (Track F) (#40)

=== open PRs now ===
(empty = none open)
```

#### 463. assistant (2026-07-06T22:06:30.680Z)

**\*\*#51 (F2) is merged\*\*** ? (`6d3d7a6`) ? the author merged it right after my LGTM. No open PRs now. The watcher fired correctly on the change (open set ? empty). Let me fast-forward `main` and re-arm for the next PR.

#### 464. user (2026-07-06T22:06:37.076Z)

Command running in background with ID: b66200aum. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b66200aum.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 465. assistant (2026-07-06T22:06:46.082Z)

### Update

**\*\*PR #51 (F2 topic preferences) is merged\*\*** ? (`6d3d7a6`) ? the author merged it right after my verified LGTM. No open PRs at the moment.

The upgraded watcher proved itself immediately: it fired on the open-set change (which the old `number:isDraft`-only version would also have caught here, but the new one now *also* catches the force-pushes and comments it used to miss).

**\*\*State:\*\***

- Local `main` fast-forwarded to `6d3d7a6` (includes merged #51), clean and synced.
- **\*\*Watcher v2 re-armed\*\*** (`b66200aum`) with an empty baseline ? it'll wake me the moment the next PR lands (F3/F4?), or on any force-push / comment activity if a PR appears.

The Track F pipeline is progressing cleanly: ADR 0012 (#40) ? F1 (#50) ? F2 (#51), all reviewed and merged. I'll surface the next one as soon as it shows up and run the same verify-then-only-LGTM-when-satisfied cycle.

#### 466. user (2026-07-06T22:12:43.512Z)

```
<task-notification>
<task-id>b66200aum</task-id>
<tool-use-id>toolu_017p7Tk1FNHWkLX9SRpmQVYb</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b66200aum.output</output-file>
<status>completed</status>
<summary>Background command "Fast-forward main and re-arm improved watcher" completed (exit code
0)</summary>
</task-notification>
```

#### 467. assistant (2026-07-06T22:12:45.984Z)

The watcher fired. Let me see what activity it detected.

#### 468. user (2026-07-06T22:12:48.304Z)

```
8 files changed, 506 insertions(+), 14 deletions(-)
local main: 6d3d7a6 feat(policy): add F2 topic preferences (#51)
=== PR-watch v2 RE-ARMED (new PRs, draft flips, force-pushes, comment/review activity) ===
baseline (open PRs now): <none>
poll 1/30: no change (22:08:41Z)
poll 2/30: no change (22:10:41Z)
=== PR ACTIVITY DETECTED (poll 3, 22:12:42Z) ===
--- baseline ---
<none>
--- now ---
#52 draft=false sha=8e7acb66a upd=2026-07-06T22:11:37Z
--- new/changed ---
#52 draft=false sha=8e7acb66a upd=2026-07-06T22:11:37Z
--- open PR details ---
#52 draft=false by avidullu: feat(policy): add F3 caption safety stage
```

#### 469. assistant (2026-07-06T22:12:59.200Z)

**\*\*New PR #52 detected\*\*** ? "feat(policy): add F3 caption safety stage" (F3 = Tier 3 caption safety from ADR 0012). I'll pay special attention to whether it repeats or avoids F2's fail-closed config bug (which I just got fixed) and how it wires/ordering after the topic stage. Let me sync, get the diff, and start the gate.

#### 470. user (2026-07-06T22:13:12.730Z)

```

- [deleted]          (none)          -> origin/feat/f2-topic-preferences
* [new branch]      feat/f3-caption-safety -> origin/feat/f3-caption-safety
=== PR #52 META ===
{"additions":299,"baseRefName":"main","body":"## Summary\n- add ADR 0012 F3 `caption-safety`  
policy stage under `security_checks`\n- support opt-in `unsafe_caption_terms` and  
`unsafe_caption_patterns` scanning over captured captions only\n- fail open with explanation  
metadata on malformed optional safety config\n- run topic preferences and caption safety through  
`evaluate_pipeline()` for live ingestion and backfill\n\n## Config shape\n```\njson\n{\n  \"security_checks\": {\n    \"unsafe_caption_terms\": [\"crypto\"],\n    \"unsafe_caption_patterns\": [\"(?i)free\\\\\\\\s+money\"]\n  }\n}\n```\n\nMissing or empty arrays  
leave caption safety disabled. Missing captions pass explicitly with `caption_present=false`; no  
fetch/download behavior is added.\n\n## Validation\n- `./venv/Scripts/python.exe -m ruff format  
src tests`\n- `./venv/Scripts/python.exe -m ruff check src tests`\n- `./venv/Scripts/python.exe -m pytest` -> 523 passed @ 95.77% coverage\n\nCloses  
#45\", \"changedFiles\":4, \"deletions\":41, \"headRefName\":\"feat/f3-caption-safety\", \"isDraft\":false, \"mer  
geStateStatus\":\"CLEAN\", \"mergeable\":\"MERGEABLE\", \"number\":52, \"title\":\"feat(policy): add F3 caption  
safety stage\"}

=== files ===
src/tg_video_filter/backfill.py
src/tg_video_filter/policy.py
src/tg_video_filter/telethon_client.py
tests/test_policy.py
=== freshness ===
(empty=current)
522 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-  
indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr52.diff
HEAD is now at 8e7acb6 feat(policy): add caption safety stage
gate started

```

#### 471. assistant (2026-07-06T22:13:27.263Z)

PR #52 (F3): +299/?41. Notably the body says it **runs both stages through** `evaluate_pipeline()` **(addressing my F2 "orchestrator bypassed" finding) and fails open on malformed config** (learning from F2). New risk to watch: `unsafe_caption_patterns` are **user-supplied regexes** ? ReDoS / invalid-regex handling. Let me read the full diff.

#### 472. user (2026-07-06T22:13:28.394Z)

```

1      diff --git a/src/tg_video_filter/backfill.py b/src/tg_video_filter/backfill.py
2      index 15a7aa2..64905bf 100644
3      --- a/src/tg_video_filter/backfill.py
4      +++ b/src/tg_video_filter/backfill.py
5      @@ -34,7 +34,7 @@
6      from tg_video_filter.models import Channel, DeliveryMode, User, VideoRecord
7      from tg_video_filter.telethon_client import (
8          _content_item_preview,
9      -    _evaluate_topic_preferences,
10     +    _evaluate_policy_stages,
11         _extract_document_id,
12         _message_text,
13         _peer_id,
14     @@ -244,7 +244,7 @@ async def backfill_user( # pragma: no cover - requires live
Telethon connection
15         matched = result.matched
16         reasons = list(result.reasons)
17         if matched and source is not None:
18     -         topic_result = await _evaluate_topic_preferences(
19     +         policy_results = await _evaluate_policy_stages(
20             conn,
21             user.id,
22             source,
23     @@ -258,9 +258,9 @@ async def backfill_user( # pragma: no cover - requires live
Telethon connection
24         matched,

```



```

25         ),
26     )
27     -         if topic_result is not None:
28     -             reasons.append(topic_result.reason)
29     -             matched = topic_result.passed
30     +         if policy_results:
31     +             reasons.extend(result.reason for result in policy_results)
32     +             matched = policy_results[-1].passed
33
34         record = VideoRecord(
35             user_id=user.id,
36             diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
37             index 86db324..9e2664c 100644
38             --- a/src/tg_video_filter/policy.py
39             +++ b/src/tg_video_filter/policy.py
40             @@ -9,6 +9,7 @@
41
42             import json
43             import logging
44             +import re
45             from dataclasses import dataclass, field
46             from typing import Protocol
47
48             @@ -77,6 +78,18 @@ def enabled(self) -> bool:
49                 return bool(self.include or self.exclude)
50
51
52             +@dataclass(frozen=True)
53             +class CaptionSafetyConfig:
54             +    """Parsed ADR 0012 caption safety config."""
55             +
56             +    unsafe_terms: tuple[str, ...] = ()
57             +    unsafe_patterns: tuple[str, ...] = ()
58             +
59             +    @property
60             +    def enabled(self) -> bool:
61             +        return bool(self.unsafe_terms or self.unsafe_patterns)
62             +
63             +
64             def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
65                 """Parse ``topics`` from a raw policy config object."""
66                 raw_topics = config.get("topics")
67                 @@ -84,8 +97,8 @@ def parse_topic_preferences(config: dict[str, object]) ->
TopicPreferences:
68                     return TopicPreferences()
69                     if not isinstance(raw_topics, dict):
70                         raise ValueError("topics must be a JSON object")
71             - include = _string_tuple(raw_topics.get("include"))
72             - exclude = _string_tuple(raw_topics.get("exclude"))
73             + include = _string_tuple(raw_topics.get("include"), field_name="topics.include")
74             + exclude = _string_tuple(raw_topics.get("exclude"), field_name="topics.exclude")
75             if not include and not exclude:
76                 return TopicPreferences()
77             mode = raw_topics.get("mode", "any")
78             @@ -98,6 +111,40 @@ def parse_topic_preferences(config: dict[str, object]) ->
TopicPreferences:
79                 )
80
81
82             +def parse_caption_safety_config(config: dict[str, object]) -> CaptionSafetyConfig:
83             +    """Parse caption safety settings from ``security_checks``.
84             +
85             +    Expected shape:
86             +    ``{"security_checks":{"unsafe_caption_terms":[],"unsafe_caption_patterns":[]}}``.
87             +    """

```

```

88 +     raw_security = config.get("security_checks")
89 +     if raw_security is None:
90 +         return CaptionSafetyConfig()
91 +     if not isinstance(raw_security, dict):
92 +         raise ValueError("security_checks must be a JSON object")
93 +
94 +     unsafe_terms = _string_tuple(
95 +         raw_security.get("unsafe_caption_terms"),
96 +         field_name="security_checks.unsafe_caption_terms",
97 +     )
98 +     unsafe_patterns = _string_tuple(
99 +         raw_security.get("unsafe_caption_patterns"),
100 +         field_name="security_checks.unsafe_caption_patterns",
101 +     )
102 +     for pattern in unsafe_patterns:
103 +         try:
104 +             re.compile(pattern)
105 +         except re.error as exc:
106 +             raise ValueError(
107 +                 "security_checks.unsafe_caption_patterns contains invalid regex "
108 +                 f"{pattern!r}: {exc}"
109 +             ) from exc
110 +     return CaptionSafetyConfig(
111 +         unsafe_terms=unsafe_terms,
112 +         unsafe_patterns=unsafe_patterns,
113 +     )
114 +
115 +
116 def load_topic_preferences(policy: Policy) -> TopicPreferences:
117     """Load topic preferences from ``policy.config_json``."""
118     decoded = json.loads(policy.config_json)
119 @@ -106,19 +153,27 @@ def load_topic_preferences(policy: Policy) -> TopicPreferences:
120     return parse_topic_preferences(decoded)
121
122
123 -def _string_tuple(value: object) -> tuple[str, ...]:
124 +def load_caption_safety_config(policy: Policy) -> CaptionSafetyConfig:
125 +     """Load caption safety settings from ``policy.config_json``."""
126 +     decoded = json.loads(policy.config_json)
127 +     if not isinstance(decoded, dict):
128 +         raise ValueError("policy config_json must decode to a JSON object")
129 +     return parse_caption_safety_config(decoded)
130 +
131 +
132 +def _string_tuple(value: object, *, field_name: str) -> tuple[str, ...]:
133 +     if value is None:
134 +         return ()
135 +     if not isinstance(value, list):
136 +         raise ValueError("topic include/exclude values must be arrays")
137 +     topics: list[str] = []
138 +     raise ValueError(f"{field_name} must be an array")
139 +     values: list[str] = []
140 +     for item in value:
141 +         if not isinstance(item, str):
142 +             raise ValueError("topic include/exclude arrays must contain strings")
143 +             topic = item.strip()
144 +             if topic:
145 +                 topics.append(topic)
146 +             return tuple(topics)
147 +             raise ValueError(f"{field_name} must contain strings")
148 +             text = item.strip()
149 +             if text:
150 +                 values.append(text)
151 +             return tuple(values)
152

```

```

153
154     class TopicPreferenceStage:
155         @@ -189,6 +244,66 @@ def _matched_topics(topics: tuple[str, ...], haystack: str) ->
tuple[str, ...]:
156             return tuple(topic for topic in topics if topic.casefold() in haystack)
157
158
159     +class CaptionSafetyStage:
160     +    """ADR 0012 Tier 3 captured-caption term and pattern scanning."""
161     +
162     +    stage_name = "caption-safety"
163     +
164     +    async def evaluate(self, ctx: PolicyContext) -> StageResult:
165     +        try:
166     +            config = load_caption_safety_config(ctx.policy)
167     +        except (json.JSONDecodeError, ValueError) as exc:
168     +            log.warning(
169     +                "caption safety disabled due to invalid config policy_id=%s",
170     +                ctx.policy.id,
171     +                exc_info=True,
172     +            )
173     +            return StageResult.pass_(
174     +                self.stage_name,
175     +                f"invalid caption safety config ignored: {exc}",
176     +                invalid_caption_safety_config=True,
177     +            )
178     +        if not config.enabled:
179     +            return StageResult.pass_(self.stage_name, "caption safety disabled")
180     +
181     +        caption = (ctx.content_item.text or "").strip()
182     +        if not caption:
183     +            return StageResult.pass_(
184     +                self.stage_name,
185     +                "no caption to scan",
186     +                caption_present=False,
187     +            )
188     +
189     +        haystack = caption.casefold()
190     +        for term in config.unsafe_terms:
191     +            if term.casefold() in haystack:
192     +                return StageResult.fail(
193     +                    self.stage_name,
194     +                    f"unsafe caption term matched: {term}",
195     +                    caption_present=True,
196     +                    match_type="term",
197     +                    matched_caption_term=term,
198     +                )
199     +
200     +        for pattern in config.unsafe_patterns:
201     +            if re.search(pattern, caption):
202     +                return StageResult.fail(
203     +                    self.stage_name,
204     +                    "unsafe caption pattern matched",
205     +                    caption_present=True,
206     +                    match_type="pattern",
207     +                    matched_caption_pattern=pattern,
208     +                )
209     +
210     +        return StageResult.pass_(
211     +            self.stage_name,
212     +            "caption passed safety checks",
213     +            caption_present=True,
214     +            checked_terms=len(config.unsafe_terms),
215     +            checked_patterns=len(config.unsafe_patterns),
216     +        )

```

```

217 +
218 +
219     async def evaluate_pipeline(
220         ctx: PolicyContext,
221         stages: list[PolicyStage],
222     @@ -204,12 +319,16 @@ async def evaluate_pipeline(
223
224
225     __all__ = [
226 +     "CaptionSafetyConfig",
227 +     "CaptionSafetyStage",
228         "PolicyContext",
229         "PolicyStage",
230         "StageResult",
231         "TopicPreferenceStage",
232         "TopicPreferences",
233         "evaluate_pipeline",
234 +     "load_caption_safety_config",
235         "load_topic_preferences",
236 +     "parse_caption_safety_config",
237         "parse_topic_preferences",
238     ]
239     diff --git a/src/tg_video_filter/telethon_client.py
b/src/tg_video_filter/telethon_client.py
240     index 7b393f8..6dad7c4 100644
241     --- a/src/tg_video_filter/telethon_client.py
242     +++ b/src/tg_video_filter/telethon_client.py
243     @@ -44,7 +44,13 @@
244         User,
245         VideoRecord,
246     )
247     -from tg_video_filter.policy import PolicyContext, StageResult, TopicPreferenceStage
248     +from tg_video_filter.policy import (
249     +    CaptionSafetyStage,
250     +    PolicyContext,
251     +    StageResult,
252     +    TopicPreferenceStage,
253     +    evaluate_pipeline,
254     +)
255
256     log = logging.getLogger(__name__)
257
258     @@ -272,7 +278,7 @@ async def on_new_message(event: NewMessage.Event) -> None: #
pragma: no cover
259         matched = result.matched
260         reasons = list(result.reasons)
261         if matched and source is not None:
262     -        topic_result = await _evaluate_topic_preferences(
263     +        policy_results = await _evaluate_policy_stages(
264             conn,
265             user.id,
266             source,
267     @@ -286,9 +292,9 @@ async def on_new_message(event: NewMessage.Event) -> None: #
pragma: no cover
268             matched,
269         ),
270     )
271     -    if topic_result is not None:
272     -        reasons.append(topic_result.reason)
273     -        matched = topic_result.passed
274     +    if policy_results:
275     +        reasons.extend(result.reason for result in policy_results)
276     +        matched = policy_results[-1].passed
277
278         record = VideoRecord(

```

```

279         user_id=user.id,
280     @@ -420,19 +426,21 @@ def _content_item_preview(
281     )
282
283
284     -async def _evaluate_topic_preferences(
285     +async def _evaluate_policy_stages(
286         conn: aiosqlite.Connection,
287         account_id: int,
288         source: Source,
289         content_item: ContentItem,
290     -) -> StageResult | None:
291     -     """Evaluate ADR 0012 topic preferences for a candidate item."""
292     +) -> list[StageResult]:
293     +     """Evaluate ADR 0012 policy stages for a candidate item."""
294         policy = await db.get_default_policy(conn, account_id)
295         if policy is None:
296             return None
297         return await TopicPreferenceStage().evaluate(
298             PolicyContext(content_item=content_item, source=source, policy=policy)
299     +     return []
300     +     _passed, results = await evaluate_pipeline(
301     +         PolicyContext(content_item=content_item, source=source, policy=policy),
302     +         [TopicPreferenceStage(), CaptionSafetyStage()],
303     +     )
304     +     return results
305
306
307     def _peer_id(peer: object) -> int:
308     @@ -468,6 +476,6 @@ async def __call__(self, event: NewMessage.Event) -> None: ...
309         "extract_video_meta",
310         "make_new_message_handler",
311         "_content_item_preview",
312     -     "_evaluate_topic_preferences",
313     +     "_evaluate_policy_stages",
314         "_message_text",
315     ]
316     diff --git a/tests/test_policy.py b/tests/test_policy.py
317     index ca6e424..8772181 100644
318     --- a/tests/test_policy.py
319     +++ b/tests/test_policy.py
320     @@ -7,14 +7,17 @@
321     from tg_video_filter import db
322     from tg_video_filter.models import ContentItem, Policy, Source
323     from tg_video_filter.policy import (
324     +     CaptionSafetyConfig,
325     +     CaptionSafetyStage,
326         PolicyContext,
327         StageResult,
328         TopicPreferences,
329         TopicPreferenceStage,
330         evaluate_pipeline,
331     +     parse_caption_safety_config,
332         parse_topic_preferences,
333     )
334     -from tg_video_filter.telethon_client import _evaluate_topic_preferences
335     +from tg_video_filter.telethon_client import _evaluate_policy_stages
336
337
338     def _ctx(
339     @@ -213,7 +216,116 @@ async def
340     test_topic_preferences_invalid_config_fails_open_with_explanation() ->
341         assert result.metadata == {"invalid_topic_config": True}
342

```

```

343     -async def test_evaluate_topic_preferences_uses_default_policy_config() -> None:
344     +# --- CaptionSafetyStage -----
345     +
346     +
347     +def test_parse_caption_safety_defaults_to_disabled() -> None:
348     +     assert parse_caption_safety_config({}) == CaptionSafetyConfig()
349     +     assert parse_caption_safety_config({"security_checks": {}}) ==
CaptionSafetyConfig()
350     +
351     +
352     +def test_parse_caption_safety_rejects_invalid_pattern() -> None:
353     +     try:
354     +         parse_caption_safety_config({"security_checks": {"unsafe_caption_patterns":
["("]}})
355     +     except ValueError as exc:
356     +         assert "invalid regex" in str(exc)
357     +     else: # pragma: no cover - assertion guard
358     +         raise AssertionError("invalid caption safety regex should raise")
359     +
360     +
361     +async def test_caption_safety_disabled_passes() -> None:
362     +     result = await CaptionSafetyStage().evaluate(_ctx(text="free crypto airdrop"))
363     +
364     +     assert result.passed is True
365     +     assert result.reason == "caption safety disabled"
366     +
367     +
368     +async def test_caption_safety_clean_caption_passes() -> None:
369     +     result = await CaptionSafetyStage().evaluate(
370     +         _ctx(
371     +             policy_json=(
372     +                 '{"security_checks":'
373     +                 '{"unsafe_caption_terms":["crypto"],"unsafe_caption_patterns":["airdrop"]}'}
374     +             ),
375     +             text="AI philosophy lecture",
376     +         )
377     +     )
378     +
379     +     assert result.passed is True
380     +     assert result.metadata == {
381     +         "caption_present": True,
382     +         "checked_terms": 1,
383     +         "checked_patterns": 1,
384     +     }
385     +
386     +
387     +async def test_caption_safety_unsafe_term_rejects_caption() -> None:
388     +     result = await CaptionSafetyStage().evaluate(
389     +         _ctx(
390     +             policy_json='{"security_checks":{"unsafe_caption_terms":["Crypto"]}}',
391     +             text="Daily crypto market summary",
392     +         )
393     +     )
394     +
395     +     assert result.passed is False
396     +     assert result.reason == "unsafe caption term matched: Crypto"
397     +     assert result.metadata == {
398     +         "caption_present": True,
399     +         "match_type": "term",
400     +         "matched_caption_term": "Crypto",
401     +     }
402     +
403     +
404     +async def test_caption_safety_unsafe_pattern_rejects_caption() -> None:

```

```

405 +     result = await CaptionSafetyStage().evaluate(
406 +         _ctx(
407 +
policy_json='{"security_checks":{"unsafe_caption_patterns":["free\\\\s+money"]}}',
408 +             text="limited FREE money guide",
409 +         )
410 +     )
411 +
412 +     assert result.passed is True
413 +
414 +     result = await CaptionSafetyStage().evaluate(
415 +         _ctx(
416 +
policy_json=('{"security_checks":{"unsafe_caption_patterns":["(?i)free\\\\s+money"]}}'),
417 +             text="limited FREE money guide",
418 +         )
419 +     )
420 +
421 +     assert result.passed is False
422 +     assert result.reason == "unsafe caption pattern matched"
423 +     assert result.metadata == {
424 +         "caption_present": True,
425 +         "match_type": "pattern",
426 +         "matched_caption_pattern": "(?i)free\\\\s+money",
427 +     }
428 +
429 +
430 + async def test_caption_safety_missing_caption_passes_explicitly() -> None:
431 +     result = await CaptionSafetyStage().evaluate(
432 +         _ctx(
433 +             policy_json='{"security_checks":{"unsafe_caption_terms":["crypto"]}}',
434 +             text=None,
435 +         )
436 +     )
437 +
438 +     assert result.passed is True
439 +     assert result.reason == "no caption to scan"
440 +     assert result.metadata == {"caption_present": False}
441 +
442 +
443 + async def test_caption_safety_invalid_config_fails_open_with_explanation() -> None:
444 +     result = await CaptionSafetyStage().evaluate(
445 +         _ctx(policy_json='{"security_checks":{"unsafe_caption_terms":"crypto"}}')
446 +     )
447 +
448 +     assert result.passed is True
449 +     assert result.reason.startswith("invalid caption safety config ignored:")
450 +     assert result.metadata == {"invalid_caption_safety_config": True}
451 +
452 +
453 + async def test_evaluate_policy_stages_uses_default_policy_config() -> None:
454 +     conn = await aiosqlite.connect(":memory:")
455 +     try:
456 +         await db._apply_schema(conn)
457 +     @@ -231,7 +343,10 @@ async def
test_evaluate_topic_preferences_uses_default_policy_config() -> None:
458 +         await db.save_policy_config(
459 +             conn,
460 +             111,
461 +             {"topics": {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"}},
462 +             {
463 +                 "topics": {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"},
464 +                 "security_checks": {"unsafe_caption_terms": ["malware"]},
465 +             },
466 +         )

```

```

467         item = ContentItem(
468             account_id=111,
469             @@ -241,26 +356,42 @@ async def
test_evaluate_topic_preferences_uses_default_policy_config() -> None:
470             dedup_key="tg:msg:1:42",
471         )
472
473     -     result = await _evaluate_topic_preferences(conn, 111, src, item)
474     +     results = await _evaluate_policy_stages(conn, 111, src, item)
475
476     -     assert result is not None
477     -     assert result.passed is True
478     -     assert result.metadata == {"matched_include_topics": ["AI"]}
479     +     assert [result.stage_name for result in results] == [
480     +         "topic-preferences",
481     +         "caption-safety",
482     +     ]
483     +     assert [result.passed for result in results] == [True, True]
484     +     assert results[0].metadata == {"matched_include_topics": ["AI"]}
485
486     -     await db.save_policy_config(
487     +     topic_rejected = await _evaluate_policy_stages(
488         conn,
489         111,
490         -         {"topics": {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"}},
491         +         src,
492         +         item.model_copy(update={"text": "crypto discussion"}),
493     )
494     -     rejected = await _evaluate_topic_preferences(
495     +
496     +     assert [result.stage_name for result in topic_rejected] == ["topic-
preferences"]
497     +     assert topic_rejected[0].passed is False
498     +     assert topic_rejected[0].metadata == {"matched_exclude_topics": ["Crypto"]}
499     +
500     +     caption_rejected = await _evaluate_policy_stages(
501         conn,
502         111,
503         src,
504         -         item.model_copy(update={"text": "crypto discussion"}),
505         +         item.model_copy(update={"text": "malware tutorial"}),
506     )
507
508     -     assert rejected is not None
509     -     assert rejected.passed is False
510     -     assert rejected.metadata == {"matched_exclude_topics": ["Crypto"]}
511     +     assert [result.stage_name for result in caption_rejected] == [
512     +         "topic-preferences",
513     +         "caption-safety",
514     +     ]
515     +     assert caption_rejected[-1].passed is False
516     +     assert caption_rejected[-1].metadata == {
517     +         "caption_present": True,
518     +         "match_type": "term",
519     +         "matched_caption_term": "malware",
520     +     }
521     finally:
522         await conn.close()
523

```

### 473. assistant (2026-07-06T22:14:25.864Z)

I've read the full F3 diff ? it's notably clean and **\*\*incorporated my prior feedback\*\***: fail-open on bad config (like the fixed F2), migrated both stages through `evaluate\_pipeline` (my F2 "orchestrator bypassed" finding), invalid-regex validated at parse time, and the wiring helper



is now tested. My concerns are low/latent ? the standout is **ReDoS**: ``unsafe_caption_patterns`` are user regexes run via ``re.search`` on the shared event loop with no timeout. Let me check the gate and probe that.

#### 474. user (2026-07-06T22:14:43.218Z)

```
=== gate ===
=== RUFF ===
All checks passed!
36 files already formatted
=== PYTEST ===
-----
TOTAL                                1393    44    380    31    96%
Required test coverage of 80.0% reached. Total coverage: 95.77%
523 passed in 9.41s
DONE

=== ReDoS feasibility: pathological user regex on a crafted caption (timeout 5s) ===
exit=1 (124=timed out -> ReDoS hangs the loop)
```

#### 475. assistant (2026-07-06T22:14:56.640Z)

Gate: ruff clean, **523 passed @ 95.77%**. The ReDoS test result is ambiguous (grep ate the exit code). Let me re-run it cleanly to confirm whether a pathological regex actually hangs.

#### 476. user (2026-07-06T22:15:09.071Z)

running with 6s timeout on a 40-char crafted caption...  
rc=124 (124 = hung/timed out = ReDoS confirmed; 0 = completed fine)

#### 477. assistant (2026-07-06T22:15:52.570Z)

**ReDoS confirmed** ? ``rc=124``: the valid-but-pathological pattern ``(a+)+$`` on a 40-char caption hangs for the full timeout. Since ``re.search`` is synchronous, it blocks the shared asyncio event loop ? on a multi-user host, one account's bad pattern would **stall ingestion for everyone**. It's latent (no command sets ``unsafe_caption_patterns`` yet), but it ships in F3's code. That's a real medium. Let me run a focused adversarial pass to pin severity/reachability and check the ``evaluate_pipeline`` migration didn't change F2 behavior.

#### 478. user (2026-07-06T22:16:32.939Z)

Workflow launched in background. Task ID: whug52bvd  
Summary: Review telegram-video-filter PR #52 (F3 caption safety: user-regex ReDoS + evaluate\_pipeline migration) with adversarial verification  
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_ace7351e-c4e  
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr52-f3-wf\_ace7351e-c4e.js  
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr52-f3-wf\_ace7351e-c4e.js"} to iterate without resending the script.)  
Run ID: wf\_ace7351e-c4e  
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr52-f3-wf\_ace7351e-c4e.js", resumeFromRunId: "wf\_ace7351e-c4e"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

#### 479. assistant (2026-07-06T22:17:00.969Z)

While the workflow runs, let me check a subtle posture issue: because

`parse\_caption\_safety\_config` validates *\*all\** patterns and raises on the first bad one, does a *\*\*single typo'd pattern fail-open the entire safety stage\*\** (disabling valid ``unsafe_terms`` too)? For a *\*safety\** stage, fail-open means a config typo silently lets flagged content through ? the opposite of what topics wanted.

#### 480. user (2026-07-06T22:17:12.585Z)

```
re.PatternError: missing ), unterminated subpattern at position 0
```

The above exception was the direct cause of the following exception:

```
ValueError: security_checks.unsafe_caption_patterns contains invalid regex '(: missing ),  
unterminated subpattern at position 0  
one bad pattern + valid term -> passed=True (invalid caption safety config ignored:  
security_checks.)  
term CRYPTO vs caption crypto -> passed=False (casefold, expect False/blocked)  
pattern CRYPTO vs caption crypto -> passed=True (case-SENSITIVE, expect True/not-blocked)
```

#### 481. assistant (2026-07-06T22:17:36.653Z)

Both confirmed (tracebacks are the ``exc_info`` logging; results below them):

1. ? *\*\*Safety stage fails OPEN ? a config typo silently bypasses ALL blocking.\*\** A caption ``"malware tutorial"`` that should be blocked (valid ``unsafe_terms:['malware']``) *\*\*passed\*\** because one typo'd pattern ``(`` disabled the *\*entire\** stage. For a *\*safety\** stage, fail-open defeats the purpose ? the same posture that's right for topic *\*filtering\** is dangerous for content *\*blocking\**.
2. ? *\*\*Case-sensitivity inconsistency:\*\** terms are case-insensitive (casefold), patterns case-sensitive by default (``CRYPTO`` pattern doesn't block ``crypto``).

So F3 has *\*\*two mediums\*\** (ReDoS + safety-fail-open, both latent-but-real) plus lows. Per your standing rule I won't LGTM unqualified. I'll wait for the workflow's independent take, then post a review holding the LGTM.

#### 482. user (2026-07-06T22:21:39.136Z)

```
<task-notification>  
<task-id>whug52bvd</task-id>  
<tool-use-id>toolu_011NTQY2WrNo4b6c9H3J2nTs</tool-use-id>  
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-  
highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\whug52bvd.output</output-file>  
<status>completed</status>  
<summary>Dynamic workflow "Review telegram-video-filter PR #52 (F3 caption safety: user-regex  
ReDoS + evaluate_pipeline migration) with adversarial verification" completed</summary>  
<result>{"counts":{"raw":4,"kept":4,"refuted":0},"findings":[{"dimension":"security","severity":  
"medium","verdict":"CONFIRMED","title":"User-supplied caption regexes run via re.search on the  
shared asyncio event loop with no timeout or complexity bound (ReDoS -&gt; host-wide ingestion  
stall)","where":"src/tg_video_filter/policy.py:294","summary":"CaptionSafetyStage.evaluate calls  
re.search(pattern, caption) for each configured unsafe_caption_pattern with no timeout, length  
cap, or complexity limit; a valid-but-pathological pattern plus an attacker-controlled caption  
causes catastrophic backtracking that blocks the single shared manager event loop, starving  
ingestion for all users on the  
host."},"failure_scenario":"security_checks.unsafe_caption_patterns is set to a valid regex like  
'(a+)+$' (compiles cleanly, so the re.compile validation in parse_caption_safety_config passes).  
An attacker posts a video in any watched channel with a ~29-char caption 'aaaa...a!'.  
CaptionSafetyStage.evaluate runs re.search synchronously inside an async coroutine on the shared  
manager loop; the 29-char caption made re.search run 19.95s in a repro during which a concurrent  
asyncio task received 0 of its expected ~997 ticks (event loop fully starved). A 40-char caption  
did not finish within a 30s timeout (rc=124). Longer captions (Telegram allows ~1024, or 4096  
for premium) make the stall effectively unbounded. Both live (make_new_message_handler -&gt;  
_evaluate_policy_stages) and backfill (backfill_user -&gt; _evaluate_policy_stages) paths are  
affected. All users' new-message ingestion and backfill on that host stall until the regex  
returns."},"evidence":"policy.py:293-294 `for pattern in config.unsafe_patterns: if  
re.search(pattern, caption):` ? no timeout/executor/length guard. re.compile-at-parse
```

(policy.py:135-142) only rejects syntactically invalid regexes; ReDoS patterns are syntactically valid and pass. caption is attacker-controlled: telethon\_client.py:382-388 \_message\_text pulls message.message/raw\_text unbounded; CaptionSafetyStage reads ctx.content\_item.text at policy.py:281 with only .strip() applied (no length cap). Repro (scratchpad/redos2.py): caption len=29 -> regex ran 19.95s, other-task ticks DURING regex = 0. Repro (redos.py): 40-char caption -> rc=124 (killed at 30s). Reachability: save\_policy\_config is only reached via save\_filter\_config (db.py:1770-1793), the legacy /set shim that merges flat FilterConfig keys and never writes security\_checks; no shipped command sets security\_checks today, so the feature is dormant and this is host-wide-DoS-once-configurable rather than exploitable on the default path ? hence medium, not high. Note the danger amplifier: once an operator enables it, the DoS trigger is an attacker's caption from any monitored channel, not the operator's own input.", "recommendation": "Do not run user-supplied regexes synchronously on the event loop. Options, in preference order: (1) use the third-party `regex` module with a per-search timeout (regex.search(..., timeout=...)) and treat timeout as fail-open + warning; (2) offload each re.search to run\_in\_executor with a deadline so the loop keeps servicing other users; (3) at minimum add hard caps ? bound caption length before scanning (e.g. truncate to a few hundred chars), cap the number of patterns, and reject patterns containing obvious nested-quantifier ReDoS shapes at parse time. re.compile alone is insufficient; document that pattern validation does not prevent catastrophic backtracking.", "verify\_reasoning": "Every element of the finding is verified against this PR's diff and reproduces the described behavior.\n\n1. No timeout/executor/length bound. policy.py:288-289 `for pattern in config.unsafe\_patterns: if re.search(pattern, caption):` runs the user-supplied regex directly. evaluate\_pipeline (policy.py:307-318) merely `await stage.evaluate(ctx)` in a loop; CaptionSafetyStage.evaluate (policy.py:252-296) has no run\_in\_executor, no asyncio.wait\_for, no asyncio.to\_thread. re.search is CPU-bound C code with no await points, so it fully blocks the single shared manager event loop.\n\n2. Parse validation is insufficient. policy.py:134-141 validates patterns only via re.compile, which catches syntax errors but not backtracking complexity. Repro confirmed `(a+)+\$` compiles cleanly (passes parse validation), so pathological-but-valid regexes are accepted.\n\n3. Caption is attacker-controlled and unbounded. telethon\_client.py:380-388 (\_message\_text) pulls message.message/raw\_text with only .strip() ? no length cap. It reaches CaptionSafetyStage via ctx.content\_item.text (policy.py:269, again only .strip()).\n\n4. Reachable in ingestion. telethon\_client.py:281 and :439-442 call \_evaluate\_policy\_stages -> evaluate\_pipeline([TopicPreferenceStage(), CaptionSafetyStage()]) on the loop; both live and backfill paths run this.\n\n5. Repro confirms catastrophic backtracking. `(a+)+\$` on a caption of 'a'\*n+'!': len=21 -> 0.076s, len=23 -> 0.305s, len=25 -> 1.277s, len=27 -> 5.088s (~4x per +2 chars, clear exponential). A 40-char caption did not finish within a 40s kill window (rc=124). Telegram captions can be ~1024 (4096 premium) chars, making the stall effectively unbounded.\n\n6. Severity is correctly medium. `security\_checks` appears only in policy.py (parsing) and db.py (F1 raw-config merge preservation), never in a command handler ? no shipped command writes it, so the feature is dormant on the default path. This is host-wide-DoS-once-configurable, not exploitable-on-default, which maps to medium per the ground rules. Once an operator enables it, the trigger is an attacker's caption from any monitored channel, not the operator's own input ? a real amplifier, but does not raise it to high while dormant.\n\nCould not refute on any axis: the code ships in this PR's diff, the wrong behavior reproduces, it is not pre-existing (the stage and re.search call are new in F3), and the reachability caveat is honestly disclosed. The recommendation (regex module timeout / run\_in\_executor with deadline / hard caps on caption length + pattern count + nested-quantifier rejection) is sound.", {"dimension": "security", "severity": "medium", "verdict": "CONFIRMED", "title": "Term matching is case-insensitive but pattern matching is case-sensitive ? silent evasion gap in a safety stage", "where": "src/tg\_video\_filter/policy.py:293", "summary": "unsafe\_caption\_terms are matched via casefold substring (case-insensitive), while unsafe\_caption\_patterns are matched via re.search on the un-casefolded caption (case-sensitive unless the operator remembers to prepend (?i)). An operator configuring a pattern to block content gets silently case-sensitive matching, which an attacker trivially bypasses by changing case.", "failure\_scenario": "Operator sets unsafe\_caption\_patterns to ['badword'] intending to block that word. A caption 'This is a BADWORD here' passes the safety stage (re.search('badword', 'This is a BADWORD here') is None), whereas the equivalent term ['badword'] correctly blocks the same caption. Because this is a safety/blocking stage, the case-sensitive pattern default is a fail-open evasion: an attacker just varies capitalization.", "evidence": "Repro (scratchpad/case.py): TERM 'badword' vs caption 'BADWORD' -> passed=False (blocked); PAT 'badword' vs caption 'BADWORD' -> passed=True (not blocked). Code: terms use haystack=caption.casefold() then `term.casefold()` in haystack` (policy.py:290-292); patterns use `re.search(pattern, caption)` against the original-case caption (policy.py:293-294). Latent because the feature is dormant (no command sets security\_checks) and behavior is operator-configurable, hence medium.", "recommendation": "Make pattern matching case-insensitive by default for consistency with term matching (compile with

```
re.IGNORECASE), or at minimum document the asymmetry prominently and surface it in the config
UI/command when one is added. If preserving raw regex semantics is intentional, kee
... (truncated 9033 chars, full result in C:\Users\avidu\AppData\Local\Temp\claude\C--Users-
avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\whug52bvd.output)</result>
<usage><agent_count>6</agent_count><subagent_tokens>255143</subagent_tokens><tool_uses>60</tool_
uses><duration_ms>306175</duration_ms></usage>
</task-notification>
```

#### 483. assistant (2026-07-06T22:21:43.243Z)

The workflow confirmed all 4 findings (0 refuted) and independently escalated the case-sensitivity gap to **medium** (an evasion vector for a safety stage). Let me get the full set including the 2 I haven't seen.

#### 484. user (2026-07-06T22:21:47.921Z)

```
COUNTS: {'raw': 4, 'kept': 4, 'refuted': 0}
```

```
=====
[MEDIUM/CONFIRMED] (security) User-supplied caption regexes run via re.search on the shared
asyncio event loop with no timeout or complexity bound (ReDoS -> host-wide ingestion stall)
  where: src/tg_video_filter/policy.py:294
  rec: Do not run user-supplied regexes synchronously on the event loop. Options, in preference
order: (1) use the third-party `regex` module with a per-sear
```

```
[MEDIUM/CONFIRMED] (security) Term matching is case-insensitive but pattern matching is case-
sensitive ? silent evasion gap in a safety stage
  where: src/tg_video_filter/policy.py:293
  rec: Make pattern matching case-insensitive by default for consistency with term matching
(compile with re.IGNORECASE), or at minimum document the asymmetr
```

```
[LOW/PLAUSIBLE] (wiring_parse) _evaluate_policy_stages discards the correct aggregate `passed`
and forces callers to reconstruct the AND via `results[-1].passed`, which is correct only while
evaluate_pipeline short-circuits
  where: src/tg_video_filter/telethon_client.py:297
  rec: Have `_evaluate_policy_stages` return the aggregate `passed` alongside results (or a
small result object), and set `matched = passed` in both callers
```

```
[LOW/CONFIRMED] (wiring_parse) Caption safety term matching is unanchored casefold substring, so
short unsafe terms produce false-positive rejections on innocuous captions
  where: src/tg_video_filter/policy.py:281
  rec: Document the substring semantics explicitly for operators, and/or offer word-boundary
matching (e.g. an opt-in `bterm` mode) so short terms don't o
```

```
refuted: []
```

#### 485. assistant (2026-07-06T22:22:11.112Z)

The workflow confirmed 4 (0 refuted) and **escalated the case-sensitivity gap to medium** (an evasion vector). Combined with my independently-verified **fail-open-disables-all-blocking** finding, F3 has a clear theme: **three silent ways to defeat a stage whose entire job is safety.** Per your rule, that's a hold. Let me post the review.

#### 486. assistant (2026-07-06T22:22:36.343Z)

### Review ? F3 caption safety stage

Nicely done on the structure, and thanks for folding in the F2 feedback ? this migrates both stages through `evaluate\_pipeline()`, fails open on malformed config, validates regexes at parse time, and the tests are thorough. **Holding the LGTM** though: for a stage whose whole job is **safety**, there are **three silent ways to defeat it** (all reproduced). All are latent today (no command sets `security\_checks` yet), but they should be resolved before this security feature is relied on.

## Verified good ?

`ruff` clean · **523 passed @ 95.77%** · `evaluate\_pipeline([topic, caption])` migration (my F2 "orchestrator bypassed" finding) · fail-open on bad config · invalid-regex rejected at parse · caption?`content\_items.text` scanning; config read via the raw (key-preserving) path.

## ? Medium ? a safety stage that silently fails to secure

**\*\*1. ReDoS ? user regex on the shared event loop, no timeout ? host-wide ingestion stall\*\* ?**  
`policy.py:293`  
`re.search(pattern, caption)` runs a user-supplied pattern synchronously on the manager's shared asyncio loop with no timeout/length/complexity bound. `re.compile` at parse only catches \*syntax\* errors ? pathological-but-valid patterns pass. Reproduced: `(a+)\$` on a caption `a'n+!` backtracks exponentially (len 25 ? 1.3s, 27 ? 5s, **\*\*40 ? didn't finish in 40s, rc=124\*\***); during the search a concurrent asyncio task got **\*\*0 ticks\*\*** ? the loop is fully starved. Telegram captions reach ~1024 (4096 premium) chars, so the stall is effectively unbounded, and the trigger is an **\*\*attacker's caption in any watched channel\*\***, not the operator's input. Both live and backfill paths.  
> **\*\*Fix:\*\*** don't run user regex synchronously on the loop ? use the `regex` module's `timeout=`, or `run\_in\_executor`/`to\_thread` with a deadline (treat timeout as fail-open + warn); plus hard caps (truncate caption before scanning, cap pattern count, reject nested-quantifier shapes at parse).

**\*\*2. Fail-open granularity ? one typo'd pattern silently disables ALL blocking\*\* ?** `policy.py` (`parse\_caption\_safety\_config` + stage catch)  
`parse\_caption\_safety\_config` validates every pattern and raises on the first bad one ? the stage catches ? **\*\*fails open, disabling the entire stage including valid `unsafe\_terms`\*\***. Reproduced: config `{unsafe\_terms:["malware"], unsafe\_caption\_patterns:[""]}` on caption `malware tutorial` ? **\*\*passed=True\*\*** (the malware caption is delivered). Fail-open is right for \*topic filtering\* (F2) but wrong for a \*safety/blocking\* stage ? a config typo silently lets flagged content through.  
> **\*\*Fix:\*\*** for the safety stage, skip only the invalid pattern (keep valid terms/patterns active), or fail \*closed\*, or at minimum surface the disablement loudly ? don't silently stop blocking.

**\*\*3. Case-sensitivity evasion ? terms are case-insensitive, patterns are case-sensitive\*\* ?**  
`policy.py:293`  
Terms use `casefold` (case-insensitive); patterns use `re.search` on the un-casefolded caption (case-sensitive unless the operator prepends `(?i)`). Reproduced: pattern `badword` does **\*\*not\*\*** block caption `BADWORD` (while the term `badword` does). For a safety stage, an attacker bypasses a pattern by changing capitalization.  
> **\*\*Fix:\*\*** compile patterns with `re.IGNORECASE` by default for parity with terms (or make it an explicit per-pattern opt-in and document loudly).

## ? Low

- **\*\*`matched = policy\_results[-1].passed`\*\*** (`telethon\_client.py:297`) is correct only because `evaluate\_pipeline` short-circuits (last result = first failure or final pass). `evaluate\_pipeline` already returns the aggregate `passed` ? return/use that instead of reconstructing via `[-1]`, so it can't silently break if short-circuit ever changes.
- **\*\*Substring term matching\*\*** (`policy.py:281`) ? short `unsafe\_terms` over-match innocuous captions (same substring semantics as F2 topics). Document, or offer opt-in word-boundary matching.
- **\*\*Scope:\*\*** dormant ? no `/set security` command yet, so `security\_checks` is only settable via `save\_policy\_config`/manual edit. Confirm this is intentional plumbing-first (the mediums become live the moment that command lands).

---

**\*\*Holding LGTM\*\*** pending a plan for the three mediums ? at minimum #1 (ReDoS bound) and #2 (don't silently disable all blocking), since they're in shipped code and the whole point of the stage is to \*block\* unsafe content. They're small, well-contained fixes. Ping me when they're in (or if you want to decide the fail-open-vs-closed posture for safety stages) and I'll re-verify and sign off. The pipeline plumbing and tests are genuinely solid.

<sub>? Reviewed with [Claude Code](https://claude.com/claude-code) ? ReDoS hang, fail-open bypass, and case evasion all reproduced against the branch. Multi-agent adversarial pass: 4

findings, 0 refuted.</sub>

#### 487. user (2026-07-06T22:22:36.379Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr52\_review.md (file state is current in your context ? no need to Read it back)

#### 488. user (2026-07-06T22:22:49.517Z)

Command running in background with ID: bxbv81v6d. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bxbv81v6d.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 489. assistant (2026-07-06T22:22:56.568Z)

Let me confirm the review posted and worktree was removed (I bundled them before the watcher loop).

#### 490. user (2026-07-06T22:23:02.001Z)

```
=== post review (holding LGTM) on #52 ===
https://github.com/avidullu/telegram-video-filter/pull/52#issuecomment-4898150309
=== cleanup worktree ===
removed
=== re-arm watcher v2 ===
baseline: #52 draft=false sha=8e7acb66a upd=2026-07-06T22:22:51Z
watcher re-armed (pid 14661)
```

#### 491. assistant (2026-07-06T22:23:16.725Z)

I see a problem ? I nested the watcher loop in `{ ? } &` inside the background command, so the outer command exited immediately (that's why it "completed" already) and the loop is now orphaned/untracked ? it **won't** re-invoke me **on** detection. Let me re-arm it correctly, with the poll loop as the top-level background command.

#### 492. user (2026-07-06T22:23:23.761Z)

Command running in background with ID: bafyljtpl. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bafyljtpl.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 493. assistant (2026-07-06T22:23:41.778Z)

Fixed ? the watcher is now armed correctly (`bafyljtpl`) as a top-level loop that will actually re-invoke me when it detects activity.

### PR #52 (F3 caption safety) ? reviewed, **\*\*LGTM held\*\***

[Review posted](https://github.com/avidullu/telegram-video-filter/pull/52#issuecomment-4898150309). The plumbing is genuinely good ? it folded in my F2 feedback (both stages now run through `evaluate\_pipeline`, fail-open, regex validated at parse, thorough tests). But I held the LGTM because, for a stage whose whole job is **\*safety\***, there are **\*\*three reproduced ways to silently defeat it\*\***:

- ? **\*\*ReDoS\*\*** ? a user-supplied `(a)+\$` on an attacker's caption hangs `re.search` on the shared event loop (40-char caption ? didn't finish in 40s; concurrent tasks got 0 ticks) ? **\*\*host-wide ingestion stall for all users\*\***.
- ? **\*\*Fail-open granularity\*\*** ? one typo'd pattern makes the stage fail-open and **\*\*silently disable all blocking\*\*** (a `malware` caption passed with a valid `unsafe\_terms:["malware"]`). Fail-open is right for topic **\*filtering\***, wrong for content **\*blocking\***.
- ? **\*\*Case evasion\*\*** ? terms are case-insensitive but patterns aren't, so an attacker bypasses a

pattern by changing capitalization.

Plus 2 lows (`results[-1].passed` fragility; substring term over-matching). All latent (no `/set security` command yet) but shipped in code. I offered to re-verify and sign off once #1/#2 are addressed ? same cycle as F2.

## Watcher status

- Caught my own bug: the previous re-arm nested the loop in `{?} &`, which orphaned it (wouldn't re-invoke me). **\*\*Re-armed correctly\*\*** as a top-level background loop ? baseline `#52`, so it'll wake me on the author's fix push to #52, a review-thread reply, or a new PR.

The Track F series so far: ADR 0012 (#40) ? · F1 (#50) ? · F2 (#51) ? · F3 (#52) ? **\*\*changes requested\*\***. I'll surface the next activity as soon as it lands.

### 494. user (2026-07-06T22:33:29.696Z)

```
<task-notification>
<task-id>bafyljtpl</task-id>
<tool-use-id>toolu_01KXS7QLUV6ZDuwCVMUPzFo1</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bafyljtpl.output</output-file>
<status>completed</status>
<summary>Background command "Re-arm watcher correctly as top-level background loop" completed
(exit code 0)</summary>
</task-notification>
```

### 495. assistant (2026-07-06T22:33:32.875Z)

The watcher fired. Let me see what activity it detected.

### 496. user (2026-07-06T22:33:35.006Z)

```
=== PR-watch v2 ARMED (top-level loop; tracks new PRs, draft flips, force-pushes, comment/review
activity) ===
baseline: #52 draft=false sha=8e7acb66a upd=2026-07-06T22:22:51Z
poll 1/30: no change (22:25:25Z)
poll 2/30: no change (22:27:26Z)
poll 3/30: no change (22:29:27Z)
poll 4/30: no change (22:31:28Z)
=== PR ACTIVITY DETECTED (poll 5, 22:33:28Z) ===
--- baseline ---
#52 draft=false sha=8e7acb66a upd=2026-07-06T22:22:51Z
--- now ---
#52 draft=false sha=7eea8b95c upd=2026-07-06T22:32:44Z
--- new/changed ---
#52 draft=false sha=7eea8b95c upd=2026-07-06T22:32:44Z
--- open PR details ---
#52 draft=false by avidullu: feat(policy): add F3 caption safety stage
```

### 497. assistant (2026-07-06T22:33:43.311Z)

The improved watcher caught a **\*\*force-push on #52\*\*** (`8e7acb66a` ? `7eea8b95c`) ? exactly the case the old watcher missed. The author pushed a fix for my F3 review. Let me fetch and see what changed.

### 498. user (2026-07-06T22:33:48.439Z)

```
=== fix diff (8e7acb6..7eea8b9) ? what the author changed in response to my review ===
pyproject.toml | 1 +
src/tg_video_filter/backfill.py | 4 +-
src/tg_video_filter/policy.py | 137 ++++++-----
src/tg_video_filter/telethon_client.py | 11 +-
tests/test_policy.py | 83 ++++++-----
```





```

+     unsafe_terms = raw_terms[:_MAX_CAPTION_TERMS]
+     unsafe_patterns: list[str] = []
+     invalid_patterns: list[str] = []
+     for pattern in raw_patterns[:_MAX_CAPTION_PATTERNS]:
+         try:
-             re.compile(pattern)
-         except re.error as exc:
-             raise ValueError(
-                 "security_checks.unsafe_caption_patterns contains invalid regex "
-                 f"{pattern!r}: {exc}"
-             ) from exc
+         regex.compile(pattern, regex.IGNORECASE)
+     except regex.error:
+         invalid_patterns.append(pattern)
+     else:
+         unsafe_patterns.append(pattern)
+
+     return CaptionSafetyConfig(
+         unsafe_terms=unsafe_terms,
-         unsafe_patterns=unsafe_patterns,
+         unsafe_patterns=tuple(unsafe_patterns),
+         invalid_patterns=tuple(invalid_patterns),
+         ignored_term_count=max(0, len(raw_terms) - _MAX_CAPTION_TERMS),
+         ignored_pattern_count=max(0, len(raw_patterns) - _MAX_CAPTION_PATTERNS),
+     )

@@ -264,33 +284,83 @@ class CaptionSafetyStage:
    invalid_caption_safety_config=True,
)
    if not config.enabled:
+     metadata = _caption_safety_metadata(config, caption_present=False)
+     if config.has_config_warnings:
+         log.warning(
+             "caption safety has no valid active checks policy_id=%s metadata=%s",
+             ctx.policy.id,
+             metadata,
+         )
+     return StageResult.pass_(
+         self.stage_name,
+         "caption safety has no valid active checks",
+         **metadata,
+     )
    return StageResult.pass_(self.stage_name, "caption safety disabled")

+     if config.has_config_warnings:
+         log.warning(
+             "caption safety ignored invalid or excess config policy_id=%s "
+             "invalid_patterns=%s ignored_terms=%s ignored_patterns=%s",
+             ctx.policy.id,
+             list(config.invalid_patterns),
+             config.ignored_term_count,
+             config.ignored_pattern_count,
+         )
+
    caption = (ctx.content_item.text or "").strip()
    if not caption:
        return StageResult.pass_(
            self.stage_name,
            "no caption to scan",
-            caption_present=False,
+            **_caption_safety_metadata(config, caption_present=False),
        )

-     haystack = caption.casefold()

```

```

+         scan_caption = caption[:_MAX_CAPTION_SCAN_CHARS]
+         caption_truncated = len(caption) > _MAX_CAPTION_SCAN_CHARS
+         haystack = scan_caption.casefold()
+         for term in config.unsafe_terms:
+             if term.casefold() in haystack:
+                 return StageResult.fail(
+                     self.stage_name,
+                     f"unsafe caption term matched: {term}",
-                     caption_present=True,
+                     **_caption_safety_metadata(
+                         config,
+                         caption_present=True,
+                         caption_truncated=caption_truncated,
+                     ),
+                     match_type="term",
+                     matched_caption_term=term,
+                 )
+
+         timed_out_patterns: list[str] = []
+         for pattern in config.unsafe_patterns:
-             if re.search(pattern, caption):
+             try:
+                 matched_pattern = regex.search(
+                     pattern,
+                     scan_caption,
+                     flags=regex.IGNORECASE,
+                     timeout=_CAPTION_REGEX_TIMEOUT_SECONDS,
+                 )
+             except TimeoutError:
+                 timed_out_patterns.append(pattern)
+                 log.warning(
+                     "caption safety regex timed out policy_id=%s pattern=%r",
+                     ctx.policy.id,
+                     pattern,
+                 )
+                 continue
+
+             if matched_pattern:
+                 return StageResult.fail(
+                     self.stage_name,
+                     "unsafe caption pattern matched",
-                     caption_present=True,
+                     **_caption_safety_metadata(
+                         config,
+                         caption_present=True,
+                         caption_truncated=caption_truncated,
+                         timed_out_patterns=tuple(timed_out_patterns),
+                     ),
+                     match_type="pattern",
+                     matched_caption_pattern=pattern,
+                 )
+
@@ -298,12 +368,41 @@ class CaptionSafetyStage:
+         return StageResult.pass_(
+             self.stage_name,
+             "caption passed safety checks",
-             caption_present=True,
-             checked_terms=len(config.unsafe_terms),
-             checked_patterns=len(config.unsafe_patterns),
+             **_caption_safety_metadata(
+                 config,
+                 caption_present=True,
+                 caption_truncated=caption_truncated,
+                 timed_out_patterns=tuple(timed_out_patterns),
+             ),
+         ),
+     )

```

```

+def _caption_safety_metadata(
+    config: CaptionSafetyConfig,
+    *,
+    caption_present: bool,
+    caption_truncated: bool = False,
+    timed_out_patterns: tuple[str, ...] = (),
+) -> dict[str, object]:
+    metadata: dict[str, object] = {
+        "caption_present": caption_present,
+        "checked_terms": len(config.unsafe_terms),
+        "checked_patterns": len(config.unsafe_patterns),
+    }
+    if caption_truncated:
+        metadata["caption_truncated"] = True
+        metadata["max_caption_scan_chars"] = _MAX_CAPTION_SCAN_CHARS
+    if config.invalid_patterns:
+        metadata["ignored_invalid_caption_patterns"] = list(config.invalid_patterns)
+    if config.ignored_term_count:
+        metadata["ignored_caption_term_count"] = config.ignored_term_count
+    if config.ignored_pattern_count:
+        metadata["ignored_caption_pattern_count"] = config.ignored_pattern_count
+    if timed_out_patterns:
+        metadata["timed_out_caption_patterns"] = list(timed_out_patterns)
+    return metadata
+
+
+    async def evaluate_pipeline(
+        ctx: PolicyContext,
+        stages: list[PolicyStage],

```

#### 499. assistant (2026-07-06T22:34:16.813Z)

Thorough fix ? addresses all three mediums:

- **ReDoS:** switched to the `regex` module with a **0.01s** per-search timeout + caption length cap (4096) + pattern/term count caps.
- **Case-sensitivity:** patterns now use `regex.IGNORECASE` (consistent with terms).
- **Fail-open granularity:** invalid patterns are now **skipped individually** (kept in `invalid\_patterns`, surfaced + logged) instead of disabling the whole stage ? valid terms/patterns still enforce.

New dependency `regex` was added. Let me check the wiring changes, confirm `regex` is installed, and verify the fixes empirically.

#### 500. user (2026-07-06T22:34:20.556Z)

Exit code 1

=== telethon\_client + backfill changes (the [-1].passed low) ===

```
diff --git a/src/tg_video_filter/backfill.py b/src/tg_video_filter/backfill.py
```

```
index 64905bf..ff5d8a3 100644
```

```
--- a/src/tg_video_filter/backfill.py
```

```
+++ b/src/tg_video_filter/backfill.py
```

```
@@ -244,7 +244,7 @@ async def backfill_user( # pragma: no cover - requires live Telethon
connection
```

```
    matched = result.matched
```

```
    reasons = list(result.reasons)
```

```
    if matched and source is not None:
```

```
-         policy_results = await _evaluate_policy_stages(
```

```
+         policy_passed, policy_results = await _evaluate_policy_stages(
```

```
            conn,
```

```
            user.id,
```

```
            source,
```

```
@@ -260,7 +260,7 @@ async def backfill_user( # pragma: no cover - requires live Telethon
connection
```

```
    )
```

```
    if policy_results:
```

```

        reasons.extend(result.reason for result in policy_results)
-       matched = policy_results[-1].passed
+       matched = policy_passed

        record = VideoRecord(
            user_id=user.id,
diff --git a/src/tg_video_filter/telethon_client.py b/src/tg_video_filter/telethon_client.py
index 6dad7c4..011669b 100644
--- a/src/tg_video_filter/telethon_client.py
+++ b/src/tg_video_filter/telethon_client.py
@@ -278,7 +278,7 @@ def make_new_message_handler(
    matched = result.matched
    reasons = list(result.reasons)
    if matched and source is not None:
-       policy_results = await _evaluate_policy_stages(
+       policy_passed, policy_results = await _evaluate_policy_stages(
            conn,
            user.id,
            source,
@@ -294,7 +294,7 @@ def make_new_message_handler(
    )
    if policy_results:
        reasons.extend(result.reason for result in policy_results)
-       matched = policy_results[-1].passed
+       matched = policy_passed

        record = VideoRecord(
            user_id=user.id,
@@ -431,16 +431,15 @@ async def _evaluate_policy_stages(
    account_id: int,
    source: Source,
    content_item: ContentItem,
- ) -> list[StageResult]:
+ ) -> tuple[bool, list[StageResult]]:
    """Evaluate ADR 0012 policy stages for a candidate item."""
    policy = await db.get_default_policy(conn, account_id)
    if policy is None:
-       return []
-       _passed, results = await evaluate_pipeline(
+       return True, []
+       return await evaluate_pipeline(
            PolicyContext(content_item=content_item, source=source, policy=policy),
            [TopicPreferenceStage(), CaptionSafetyStage()],
        )
-       return results

```

```
def _peer_id(peer: object) -> int:
```

```
=== is 'regex' installed in the venv? ===
```

```
Traceback (most recent call last):
```

```
File "<string>", line 1, in <module>
```

```
import regex; print('regex', regex.__version__)
```

```
^^^^^^^^^^^^^^^^
```

```
ModuleNotFoundError: No module named 'regex'
```

## 501. assistant (2026-07-06T22:34:32.036Z)

Two findings:

1. ? \*\*The `[-1].passed` low is also fixed\*\* ? `\_evaluate\_policy\_stages` now returns `(policy\_passed, results)` (the aggregate from `evaluate\_pipeline`) and callers use `matched = policy\_passed`.
2. ?? \*\*`regex` isn't installed in the venv\*\* ? the fix depends on it. Per my gate ("install any dependency needed so the suite runs"), let me install it, then verify.

#### 502. user (2026-07-06T22:34:44.351Z)

```
=== install the new 'regex' dependency ===  
regex installed: 2026.6.28
```

```
=== worktree at fixed HEAD + start gate ===  
HEAD is now at 7eea8b9 feat(policy): add caption safety stage  
gate started
```

#### 503. assistant (2026-07-06T22:34:51.031Z)

```
`regex 2026.6.28` installed. Let me verify all three mediums are fixed empirically.
```

#### 504. user (2026-07-06T22:35:02.621Z)

```
running (10s hard cap to catch any residual hang)...  
File "C:\Users\avidu\AppData\Local\Temp\verify_f3.py", line 12  
    print(f'#1 ReDoS: completed in {time.perf_counter()-t:.3f}s, passed={r.passed},  
timed_out={r.metadata.get("timed_out_caption_patterns\")==}')  
    ^  
SyntaxError: unexpected character after line continuation character  
rc=0 (0=fine, 124=still hangs)
```

#### 505. assistant (2026-07-06T22:35:10.186Z)

```
Quote-escaping broke the heredoc. Let me write the repro cleanly via the Write tool.
```

#### 506. assistant (2026-07-06T22:35:20.301Z)

```
import asyncio, json, time  
from tg_video_filter.models import ContentItem, Policy, Source  
from tg_video_filter.policy import CaptionSafetyStage, PolicyContext  
  
def ctx(cfg, text):  
    return PolicyContext(  
        content_item=ContentItem(account_id=1, source_id=1, provider_message_id="5", text=text,  
dedup_key="k"),  
        source=Source(id=1, account_id=1, provider="telegram", provider_source_id="-1"),  
        policy=Policy(id=1, account_id=1, name="default", config_json=json.dumps(cfg),  
is_default=True),  
    )  
  
async def main():  
    # 1. ReDoS: pathological pattern + long caption must NOT hang (per-search timeout -> skip)  
    t = time.perf_counter()  
    r = await CaptionSafetyStage().evaluate(ctx({"security_checks": {"unsafe_caption_patterns":  
["(a+)+$"]}, "a" * 60 + "X"}))  
    dt = time.perf_counter() - t  
    print("1 ReDoS: completed in %.3fs passed=%s timed_out=%s" % (dt, r.passed,  
r.metadata.get("timed_out_caption_patterns")))  
  
    # 2. Fail-open granularity: one invalid pattern must NOT disable a valid term  
    r2 = await CaptionSafetyStage().evaluate(ctx({"security_checks": {"unsafe_caption_terms":  
["malware"], "unsafe_caption_patterns": [""], "malware tutorial"}))  
    print("2 granularity: malware caption passed=%s (expect False=BLOCKED) ignored_invalid=%s" %  
(r2.passed, r2.metadata.get("ignored_invalid_caption_patterns")))  
  
    # 3. Case-insensitive patterns now (IGNORECASE)  
    r3 = await CaptionSafetyStage().evaluate(ctx({"security_checks": {"unsafe_caption_patterns":  
["CRYPTO"]}, "daily crypto news"}))  
    print("3 case: pattern CRYPTO vs caption crypto passed=%s (expect False=BLOCKED)" %  
r3.passed)
```

```

# sanity: clean caption still passes
r4 = await CaptionSafetyStage().evaluate(ctx({"security_checks": {"unsafe_caption_terms":
["scam"]}}, "nice video"))
print("sanity clean: passed=%s (expect True)" % r4.passed)

asyncio.run(main())

```

#### 507. user (2026-07-06T22:35:20.332Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/verify\_f3.py (file state is current in your context ? no need to Read it back)

#### 508. user (2026-07-06T22:35:28.157Z)

```

=== F3 fix verification (10s hard cap) ===
1 ReDoS: completed in 0.002s passed=True timed_out=None
2 granularity: malware caption passed=False (expect False=BLOCKED) ignored_invalid=['(']
3 case: pattern CRYPTO vs caption crypto passed=False (expect False=BLOCKED)
sanity clean: passed=True (expect True)
rc=0 (0=fine, 124=still hangs)

=== gate (fix) ===
=== RUFF ===
All checks passed!
36 files already formatted
=== PYTEST ===
-----
TOTAL                                1435      48    394      34    96%
Required test coverage of 80.0% reached. Total coverage: 95.52%
525 passed in 9.19s
DONE

```

#### 509. assistant (2026-07-06T22:35:50.720Z)

All three mediums **verified fixed**:

- ReDoS** ? ``(a)+$`` on a 60-char caption now completes in **0.002s** (was 40s+ hang), no loop stall ?
- Fail-open granularity** ? a ``malware`` caption with a bad ``(`` pattern is now **blocked** (``passed=False``, invalid pattern skipped, valid term enforced) ?
- Case-insensitive patterns** ? ``CRYPTO`` pattern now blocks ``crypto`` ?

Gate: ruff clean, **525 passed @ 95.52%**. One correctness check before I sign off ? ``regex`` is a `*runtime*` import (``policy.py`` top-level), so let me confirm the ``pyproject.toml`` change puts it in runtime ``dependencies``, not dev.

#### 510. user (2026-07-06T22:35:54.289Z)

```

=== pyproject.toml change ===
diff --git a/pyproject.toml b/pyproject.toml
index 250828a..14ea202 100644
--- a/pyproject.toml
+++ b/pyproject.toml
@@ -26,6 +26,7 @@ dependencies = [
     "pydantic>=2.7,<3.0",
     "pydantic-settings>=2.3,<3.0",
     "python-dotenv>=1.0,<2.0",
+    "regex>=2024.0,<2027.0",
     # Native crypto acceleration for Telethon (optional but strongly recommended;
     # without it Telethon falls back to slower pure-Python encryption).
     "cryptg>=0.4,<1.0",

=== is 'regex' under [project].dependencies (runtime) not dev? ===

```

```
2: "telethon>=1.36,<2.0",
3: "aiosqlite>=0.20,<1.0",
7: "regex>=2024.0,<2027.0",
```

#### 511. assistant (2026-07-06T22:36:03.321Z)

`regex>=2024.0,<2027.0` is correctly in runtime `[project].dependencies`. Everything checks out ? all 3 mediums + the low fixed and verified, dependency properly declared, gate green. Per your rule, I'm satisfied ? this is merge-ready. Posting the LGTM.

#### 512. user (2026-07-06T22:36:18.769Z)

```
=== post LGTM ===
https://github.com/avidullu/telegram-video-filter/pull/52#issuecomment-4898232133
=== cleanup worktree ===
removed
```

#### 513. assistant (2026-07-06T22:36:26.292Z)

LGTM posted ? [issuecomment-4898232133](https://github.com/avidullu/telegram-video-filter/pull/52#issuecomment-4898232133). Worktree cleaned up. Re-arming the watcher (top-level loop this time).

#### 514. user (2026-07-06T22:36:33.346Z)

Command running in background with ID: buierxrttd. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\buierxrttd.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 515. assistant (2026-07-06T22:36:47.788Z)

### PR #52 (F3 caption safety) ? fixes verified, \*\*LGTM'd\*\* ?

The upgraded watcher caught the author's **force-push** (`8e7acb6` ? `7eea8b9`) ? exactly the case that used to slip through. I re-reviewed and verified every fix  
([LGTM](https://github.com/avidullu/telegram-video-filter/pull/52#issuecomment-4898232133)):

```
| Finding | Fix | Reproduced |
|---|---|---|
| ? ReDoS | `regex` module + 0.01s per-search timeout + 4096-char/count caps | `(a+)$` on 60
chars now **0.002s** (was 40s+ hang) ? |
| ? Fail-open bypass | invalid patterns skipped individually, valid checks still enforce |
`malware` caption now **blocked** despite a typo'd pattern ? |
| ? Case evasion | patterns use `regex.IGNORECASE` | `CRYPTO` pattern blocks `crypto` ? |
| ? `[1].passed` | returns aggregate `policy_passed` | ? |
```

Also: `regex` correctly added to runtime `[project].dependencies` (I `pip install`ed it so the suite ran, per the gate). **\*\*525 passed @ 95.52%\*\***, ruff clean. Genuinely well-handled ? the timeout + per-pattern-skip + caps is the right shape for a user-configurable safety stage.

### Status

- **\*\*Track F:\*\*** ADR 0012 (#40) ? · F1 (#50) ? · F2 (#51) ? · F3 (#52) ? ? all reviewed, verified, LGTM'd.
- **\*\*Watcher re-armed\*\*** (`buierxrttd`, top-level loop) ? baseline `#52`, so it'll wake me on the merge, the next F-phase PR, or any force-push/comment.

Two notes for my own hygiene: I had to install the new `regex` dep into the shared `.venv` to run the gate (declared in pyproject, so deploys are fine), and my earlier nested-`&` watcher bug is fixed. I'll surface the next activity when it lands.

#### 516. user (2026-07-06T22:38:37.774Z)

```
<task-notification>
<task-id>buierxrt</task-id>
<tool-use-id>toolu_0113u1JHF8CNnjJhvS3HfHXU</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\buierxrt.output</output-file>
<status>completed</status>
<summary>Background command "Re-arm watcher v2 as top-level loop" completed (exit code 0)</summary>
</task-notification>
```

#### 517. assistant (2026-07-06T22:38:40.122Z)

The watcher fired. Let me see what activity it caught.

#### 518. user (2026-07-06T22:38:42.464Z)

```
=== PR-watch v2 RE-ARMED (new PRs, draft flips, force-pushes, comment/review activity) ===
baseline: #52 draft=false sha=7eea8b95c upd=2026-07-06T22:36:18Z
=== PR ACTIVITY DETECTED (poll 1, 22:38:36Z) ===
--- baseline ---
#52 draft=false sha=7eea8b95c upd=2026-07-06T22:36:18Z
--- now ---
<none>
--- new/changed ---

--- open PR details ---
```

#### 519. assistant (2026-07-06T22:38:45.735Z)

Open set went `#52` ? empty ? #52 was merged right after my LGTM. Let me confirm and check for the next one.

#### 520. user (2026-07-06T22:38:53.010Z)

```
=== #52 state ===
state=MERGED merged=2026-07-06T22:38:09Z commit=5cd65d11b
=== origin/main recent ===
5cd65d1 feat(policy): add caption safety stage (#52)
6d3d7a6 feat(policy): add F2 topic preferences (#51)
fb2d2ea feat(policy): add F1 pipeline primitives (#50)
2aad4d6 docs(adr): propose 0012 policy pipeline ? explainable, tiered, user-customizable content gates (Track F) (#40)
=== open PRs now ===
(empty=none)
```

#### 521. assistant (2026-07-06T22:38:57.518Z)

\*\*#52 (F3) merged\*\* ? (`5cd65d1`) ? merged right after my verified LGTM. No open PRs. Let me fast-forward `main` and re-arm the watcher for the next one.

#### 522. user (2026-07-06T22:39:03.851Z)

Command running in background with ID: bt1v3mzfv. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bt1v3mzfv.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 523. assistant (2026-07-06T22:39:12.293Z)

### Update

\*\*PR #52 (F3 caption safety) is merged\*\* ? (`5cd65d1`) ? merged right after my verified LGTM. No



open PRs at the moment.

**\*\*Track F progress ? all reviewed, verified, and merged:\*\***

| PR  | Phase                                       | Outcome                                                                 |
|-----|---------------------------------------------|-------------------------------------------------------------------------|
| #40 | ADR 0012 design                             | design-reviewed ? revised ? LGTM ? merged                               |
| #50 | F1 pipeline primitives + config-persistence | LGTM ? merged                                                           |
| #51 | F2 topic preferences                        | 3 mediums ? author fixed ? LGTM ? merged                                |
| #52 | F3 caption safety                           | 3 mediums (ReDoS/fail-open/case-evasion) ? author fixed ? LGTM ? merged |

The cadence is working well: every finding reproduced locally, LGTM held until fixes verified, and the author has been fixing-and-merging each round quickly. The upgraded watcher earned its keep by catching the #52 force-push.

**\*\*State:\*\*** local `main` fast-forwarded to `5cd65d1` (all F-phases in), clean and synced.

**\*\*Watcher re-armed\*\*** (`bt1v3mzfv`) with an empty baseline ? it'll wake me on the next PR (F4 URL/domain reputation, or F6/F7 explainability, or the `/set` command phases) or any force-push/comment activity.

I'll surface the next one as soon as it lands.

#### 524. user (2026-07-06T22:45:10.214Z)

```
<task-notification>
<task-id>bt1v3mzfv</task-id>
<tool-use-id>toolu_01GYmivfCAvxe8qviMFS7nja</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bt1v3mzfv.output</output-file>
<status>completed</status>
<summary>Background command "Fast-forward main and re-arm watcher" completed (exit code 0)</summary>
</task-notification>
```

#### 525. assistant (2026-07-06T22:45:14.183Z)

The watcher fired. Let me see what it caught.

#### 526. user (2026-07-06T22:45:16.512Z)

```
5 files changed, 441 insertions(+), 41 deletions(-)
local main: 5cd65d1 feat(policy): add caption safety stage (#52)
=== PR-watch v2 RE-ARMED ===
baseline: <none>
poll 1/30: no change (22:41:07Z)
poll 2/30: no change (22:43:08Z)
=== PR ACTIVITY DETECTED (poll 3, 22:45:09Z) ===
--- baseline ---
<none>
--- now ---
#53 draft=false sha=6d563f823 upd=2026-07-06T22:43:15Z
--- new/changed ---
#53 draft=false sha=6d563f823 upd=2026-07-06T22:43:15Z
--- open PR details ---
#53 draft=false by avidullu: feat(policy): add F4 URL reputation stage
```

#### 527. assistant (2026-07-06T22:45:27.062Z)

**\*\*New PR #53 detected\*\*** ? "feat(policy): add F4 URL reputation stage" (F4 = URL/domain reputation, metadata-only per ADR). I'll check whether it reuses F3's now-good patterns (fail-open granularity, regex safety, case handling) and scrutinize URL/domain parsing (subdomain matching, IDN/obfuscation, and the "no page fetch" claim). Let me sync and start.

## 528. user (2026-07-06T22:45:39.526Z)

```
* [new branch]      feat/f4-url-reputation -> origin/feat/f4-url-reputation
=== PR #53 META ===
{"additions":357,"body":"## Summary\n- add ADR 0012 F4 `url-reputation` policy stage under  
`security_checks.blocked_domains`\n- extract plain URLs from captured captions/stored `link_url`\ninto stage metadata without network I/O\n- normalize domains predictably:  
scheme/port/case/trailing-dot cleanup, leading `www.` stripped, IDNA normalized, IP literals  
supported\n- reject exact blocklist domains and their subdomains; skip invalid blocklist entries  
without disabling valid ones\n- wire stage order as topic preferences -> URL reputation ->  
caption safety\n\n## Config shape\n```\njson\n{\n  \"security_checks\": {\n    \"blocked_domains\": [\"bad.example\"]\n  }\n}\n```\n\nThis PR keeps extraction metadata-only  
for F4; it does not fetch pages, dereference redirects, or download content.\n\n## Validation\n\n`./venv/Scripts/python.exe -m ruff format src tests`\n- `./venv/Scripts/python.exe -m ruff  
check src tests`\n- `./venv/Scripts/python.exe -m pytest tests/test_policy.py --no-cov` -> 32  
passed\n- `./venv/Scripts/python.exe -m pytest` -> 533 passed @ 94.87% coverage\n\nCloses  
#46\nr\n\", \"changedFiles\":3, \"deletions\":3, \"headRefName\":\"feat/f4-url-reputation\", \"isDraft\":false, \"  
mergeStateStatus\":\"CLEAN\", \"mergeable\":\"MERGEABLE\", \"number\":53, \"title\":\"feat(policy): add F4 URL  
reputation stage\"}
```

```
=== files + freshness ===
src/tg_video_filter/policy.py
src/tg_video_filter/telethon_client.py
tests/test_policy.py
(empty=current)
496 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-  
indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr53.diff
HEAD is now at 6d563f8 feat(policy): add URL reputation stage
gate started
```

## 529. assistant (2026-07-06T22:45:59.984Z)

PR #53 (F4): +357/?3. The body says it **\*\*skips invalid blocklist entries without disabling valid ones\*\*** (F3's lesson applied) and is **\*\*metadata-only (no fetch)\*\***. Top things to scrutinize: the **\*\*URL-extraction regex\*\*** (new ReDoS surface ? does it reuse F3's timeout, or a raw regex?), and **\*\*subdomain-matching correctness\*\*** (naive suffix match would block `notbad.example` for `bad.example`). Let me read the diff.

## 530. user (2026-07-06T22:46:00.939Z)

```
1      diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
2      index e65789c..a90cf50 100644
3      --- a/src/tg_video_filter/policy.py
4      +++ b/src/tg_video_filter/policy.py
5      @@ -9,8 +9,11 @@
6
7      import json
8      import logging
9      +import re
10     from dataclasses import dataclass, field
11     +from ipaddress import ip_address
12     from typing import Protocol
13     +from urllib.parse import urlsplit
14
15     import regex
16
17     @@ -22,6 +25,18 @@
18     _MAX_CAPTION_TERMS = 100
19     _MAX_CAPTION_PATTERNS = 25
20     _CAPTION_REGEX_TIMEOUT_SECONDS = 0.01
21     +_URL_RE = re.compile(
22     +    r"""(?ix)
23     +    (?<![@\\w.-])
24     +    (
25     +        (?:https?://|www\\.)([^\s<>'"]+)
```

```

26 +         |
27 +         (?:[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?\.)+[a-z]{2,}
28 +         (?:/[^\s<>"']*)?
29 +     )
30 +     """
31 +)
32 +_TRAILING_URL_PUNCTUATION = ".,!?;:"
33
34
35     @dataclass(frozen=True)
36     @@ -103,6 +118,26 @@ def has_config_warnings(self) -> bool:
37         return bool(self.invalid_patterns or self.ignored_term_count or
38 self.ignored_pattern_count)
39
40 +@dataclass(frozen=True)
41 +class DomainReputationConfig:
42 +     """Parsed ADR 0012 URL/domain reputation config."""
43 +
44 +     blocked_domains: tuple[str, ...] = ()
45 +     invalid_domains: tuple[str, ...] = ()
46 +
47 +     @property
48 +     def enabled(self) -> bool:
49 +         return bool(self.blocked_domains)
50 +
51 +
52 +@dataclass(frozen=True)
53 +class ExtractedUrl:
54 +     """URL discovered from captured text with its normalized host domain."""
55 +
56 +     original_url: str
57 +     domain: str
58 +
59 +
60     def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
61         """Parse ``topics`` from a raw policy config object."""
62         raw_topics = config.get("topics")
63     @@ -165,6 +200,35 @@ def parse_caption_safety_config(config: dict[str, object]) ->
CaptionSafetyConfi
64         )
65
66
67     +def parse_domain_reputation_config(config: dict[str, object]) ->
DomainReputationConfig:
68     +     """Parse URL/domain reputation settings from ``security_checks``."""
69     +     raw_security = config.get("security_checks")
70     +     if raw_security is None:
71     +         return DomainReputationConfig()
72     +     if not isinstance(raw_security, dict):
73     +         raise ValueError("security_checks must be a JSON object")
74     +
75     +     raw_domains = _string_tuple(
76     +         raw_security.get("blocked_domains"),
77     +         field_name="security_checks.blocked_domains",
78     +     )
79     +     blocked_domains: list[str] = []
80     +     invalid_domains: list[str] = []
81     +     seen: set[str] = set()
82     +     for raw_domain in raw_domains:
83     +         domain = normalize_domain(raw_domain)
84     +         if domain is None:
85     +             invalid_domains.append(raw_domain)
86     +             continue
87     +         if domain not in seen:

```

```

88     +         blocked_domains.append(domain)
89     +         seen.add(domain)
90     +     return DomainReputationConfig(
91     +         blocked_domains=tuple(blocked_domains),
92     +         invalid_domains=tuple(invalid_domains),
93     +     )
94     +
95     +
96     def load_topic_preferences(policy: Policy) -> TopicPreferences:
97         """Load topic preferences from ``policy.config_json``."""
98         decoded = json.loads(policy.config_json)
99     @@ -181,6 +245,14 @@ def load_caption_safety_config(policy: Policy) ->
CaptionSafetyConfig:
100         return parse_caption_safety_config(decoded)
101
102
103     +def load_domain_reputation_config(policy: Policy) -> DomainReputationConfig:
104     +     """Load URL/domain reputation settings from ``policy.config_json``."""
105     +     decoded = json.loads(policy.config_json)
106     +     if not isinstance(decoded, dict):
107     +         raise ValueError("policy config_json must decode to a JSON object")
108     +     return parse_domain_reputation_config(decoded)
109     +
110     +
111     def _string_tuple(value: object, *, field_name: str) -> tuple[str, ...]:
112         if value is None:
113             return ()
114     @@ -264,6 +336,146 @@ def _matched_topics(topics: tuple[str, ...], haystack: str) ->
tuple[str, ...]:
115         return tuple(topic for topic in topics if topic.casefold() in haystack)
116
117
118     +def normalize_domain(value: str) -> str | None:
119     +     """Normalize a configured or extracted domain for blocklist matching."""
120     +     text = value.strip()
121     +     if not text:
122     +         return None
123     +
124     +     parsed = urlsplit(text if "://" in text else f"//{text}")
125     +     host = parsed.hostname
126     +     if host is None:
127     +         return None
128     +
129     +     host = host.strip().strip("[]").rstrip(".").casefold()
130     +     if not host:
131     +         return None
132     +
133     +     try:
134     +         return ip_address(host).compressed.casefold()
135     +     except ValueError:
136     +         pass
137     +
138     +     try:
139     +         domain = host.encode("idna").decode("ascii").casefold()
140     +     except UnicodeError:
141     +         return None
142     +     if domain.startswith("www."):
143     +         domain = domain[4:]
144     +     if not _valid_domain_name(domain):
145     +         return None
146     +     return domain
147     +
148     +
149     +def extract_caption_urls(text: str | None) -> tuple[ExtractedUrl, ...]:
150     +     """Extract plain URLs from captured caption text without network I/O."""

```

```

151 +     if not text:
152 +         return ()
153 +
154 +     urls: list[ExtractedUrl] = []
155 +     for match in _URL_RE.finditer(text):
156 +         raw_url = match.group(1).rstrip(_TRAILING_URL_PUNCTUATION)
157 +         while raw_url.endswith("(") and raw_url.count("(") < raw_url.count(")"):
158 +             raw_url = raw_url[:-1]
159 +         domain = normalize_domain(raw_url)
160 +         if domain is not None:
161 +             urls.append(ExtractedUrl(original_url=raw_url, domain=domain))
162 +     return tuple(urls)
163 +
164 +
165 +def _valid_domain_name(domain: str) -> bool:
166 +    if len(domain) > 253 or "." not in domain:
167 +        return False
168 +    labels = domain.split(".")
169 +    return all(
170 +        re.fullmatch(r"[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?", label) is not None for
label in labels
171 +    )
172 +
173 +
174 +def _domain_matches_blocked(domain: str, blocked_domain: str) -> bool:
175 +    return domain == blocked_domain or domain.endswith(f".{blocked_domain}")
176 +
177 +
178 +class UrlReputationStage:
179 +    """ADR 0012 Tier 3 captured-URL domain blocklist checks."""
180 +
181 +    stage_name = "url-reputation"
182 +
183 +    async def evaluate(self, ctx: PolicyContext) -> StageResult:
184 +        try:
185 +            config = load_domain_reputation_config(ctx.policy)
186 +        except (json.JSONDecodeError, ValueError) as exc:
187 +            log.warning(
188 +                "domain reputation disabled due to invalid config policy_id=%s",
189 +                ctx.policy.id,
190 +                exc_info=True,
191 +            )
192 +            return StageResult.pass_(
193 +                self.stage_name,
194 +                f"invalid domain reputation config ignored: {exc}",
195 +                invalid_domain_reputation_config=True,
196 +            )
197 +
198 +            if config.invalid_domains:
199 +                log.warning(
200 +                    "domain reputation ignored invalid domains policy_id=%s
invalid_domains=%s",
201 +                    ctx.policy.id,
202 +                    list(config.invalid_domains),
203 +                )
204 +            if not config.enabled:
205 +                metadata = _url_reputation_metadata((), config)
206 +                if config.invalid_domains:
207 +                    return StageResult.pass_(
208 +                        self.stage_name,
209 +                        "domain reputation has no valid blocked domains",
210 +                        **metadata,
211 +                    )
212 +            return StageResult.pass_(self.stage_name, "domain reputation disabled")
213 +

```

```

214 +         urls = extract_caption_urls(ctx.content_item.link_url) + extract_caption_urls(
215 +             ctx.content_item.text
216 +         )
217 +         if not urls:
218 +             return StageResult.pass_(
219 +                 self.stage_name,
220 +                 "no links to scan",
221 +                 **_url_reputation_metadata(urls, config),
222 +             )
223 +
224 +         for url in urls:
225 +             for blocked_domain in config.blocked_domains:
226 +                 if _domain_matches_blocked(url.domain, blocked_domain):
227 +                     return StageResult.fail(
228 +                         self.stage_name,
229 +                         f"blocked domain matched: {blocked_domain}",
230 +                         **_url_reputation_metadata(urls, config),
231 +                         blocked_domain=url.domain,
232 +                         matched_blocklist_domain=blocked_domain,
233 +                         matched_url=url.original_url,
234 +                     )
235 +
236 +             return StageResult.pass_(
237 +                 self.stage_name,
238 +                 "links passed domain reputation checks",
239 +                 **_url_reputation_metadata(urls, config),
240 +             )
241 +
242 +
243 +def _url_reputation_metadata(
244 +    urls: tuple[ExtractedUrl, ...],
245 +    config: DomainReputationConfig,
246 +) -> dict[str, object]:
247 +    domains = sorted({url.domain for url in urls})
248 +    metadata: dict[str, object] = {
249 +        "link_count": len(urls),
250 +        "domains": domains,
251 +        "blocked_domain_count": len(config.blocked_domains),
252 +    }
253 +    if config.invalid_domains:
254 +        metadata["ignored_invalid_domains"] = list(config.invalid_domains)
255 +    return metadata
256 +
257 +
258 +class CaptionSafetyStage:
259 +    """ADR 0012 Tier 3 captured-caption term and pattern scanning."""
260 +
261 +@@ -420,14 +632,21 @@ async def evaluate_pipeline(
262 +    __all__ = [
263 +        "CaptionSafetyConfig",
264 +        "CaptionSafetyStage",
265 +        "DomainReputationConfig",
266 +        "ExtractedUrl",
267 +        "PolicyContext",
268 +        "PolicyStage",
269 +        "StageResult",
270 +        "TopicPreferenceStage",
271 +        "TopicPreferences",
272 +        "UrlReputationStage",
273 +        "evaluate_pipeline",
274 +        "extract_caption_urls",
275 +        "load_caption_safety_config",
276 +        "load_domain_reputation_config",
277 +        "load_topic_preferences",
278 +        "normalize_domain",

```

```

279     "parse_caption_safety_config",
280 +   "parse_domain_reputation_config",
281     "parse_topic_preferences",
282 ]
283 diff --git a/src/tg_video_filter/telethon_client.py
b/src/tg_video_filter/telethon_client.py
284 index 011669b..3c7d534 100644
285 --- a/src/tg_video_filter/telethon_client.py
286 +++ b/src/tg_video_filter/telethon_client.py
287 @@ -49,6 +49,7 @@
288     PolicyContext,
289     StageResult,
290     TopicPreferenceStage,
291 +   UrlReputationStage,
292     evaluate_pipeline,
293 )
294
295 @@ -438,7 +439,7 @@ async def _evaluate_policy_stages(
296     return True, []
297     return await evaluate_pipeline(
298         PolicyContext(content_item=content_item, source=source, policy=policy),
299 -       [TopicPreferenceStage(), CaptionSafetyStage()],
300 +       [TopicPreferenceStage(), UrlReputationStage(), CaptionSafetyStage()],
301     )
302
303
304 diff --git a/tests/test_policy.py b/tests/test_policy.py
305 index 042c37b..e792ef7 100644
306 --- a/tests/test_policy.py
307 +++ b/tests/test_policy.py
308 @@ -9,12 +9,18 @@
309     from tg_video_filter.policy import (
310         CaptionSafetyConfig,
311         CaptionSafetyStage,
312 +   DomainReputationConfig,
313 +   ExtractedUrl,
314         PolicyContext,
315         StageResult,
316         TopicPreferences,
317         TopicPreferenceStage,
318 +   UrlReputationStage,
319         evaluate_pipeline,
320 +   extract_caption_urls,
321 +   normalize_domain,
322         parse_caption_safety_config,
323 +   parse_domain_reputation_config,
324         parse_topic_preferences,
325     )
326     from tg_video_filter.telethon_client import _evaluate_policy_stages
327 @@ -216,6 +222,115 @@ async def
test_topic_preferences_invalid_config_fails_open_with_explanation() ->
328     assert result.metadata == {"invalid_topic_config": True}
329
330
331 +# --- UrlReputationStage -----
332 +
333 +
334 +def test_normalize_domain_strips_scheme_port_www_case_and_trailing_dot() -> None:
335 +   assert normalize_domain("HTTPS://WWW.Example.COM:443/path") == "example.com"
336 +   assert normalize_domain("sub.Example.com.") == "sub.example.com"
337 +   assert normalize_domain("https://[2001:db8::1]:443/path") == "2001:db8::1"
338 +   assert normalize_domain("not a domain") is None
339 +
340 +
341 +def test_extract_caption_urls_handles_plain_urls() -> None:

```

```

342 +     assert extract_caption_urls(
343 +         "See example.com/path, https://www.Bad.Example/a?b=1. and mail me@example.com"
344 +     ) == (
345 +         ExtractedUrl(original_url="example.com/path", domain="example.com"),
346 +         ExtractedUrl(original_url="https://www.Bad.Example/a?b=1",
domain="bad.example"),
347 +     )
348 +
349 +
350 +def test_parse_domain_reputation_config_normalizes_and_records_invalid_domains() ->
None:
351 +     assert parse_domain_reputation_config(
352 +         {"security_checks": {"blocked_domains": ["www.Bad.Example", "bad.example",
"nope"]}})
353 +     ) == DomainReputationConfig(
354 +         blocked_domains=("bad.example",),
355 +         invalid_domains=("nope",),
356 +     )
357 +
358 +
359 +async def test_url_reputation_disabled_passes() -> None:
360 +     result = await UrlReputationStage().evaluate(_ctx(text="https://bad.example/a"))
361 +
362 +     assert result.passed is True
363 +     assert result.reason == "domain reputation disabled"
364 +
365 +
366 +async def test_url_reputation_blocked_domain_rejects_subdomain() -> None:
367 +     result = await UrlReputationStage().evaluate(
368 +         _ctx(
369 +             policy_json='{"security_checks":{"blocked_domains":["bad.example"]}}',
370 +             text="Watch https://cdn.bad.example/video now",
371 +         )
372 +     )
373 +
374 +     assert result.passed is False
375 +     assert result.reason == "blocked domain matched: bad.example"
376 +     assert result.metadata == {
377 +         "link_count": 1,
378 +         "domains": ["cdn.bad.example"],
379 +         "blocked_domain_count": 1,
380 +         "blocked_domain": "cdn.bad.example",
381 +         "matched_blocklist_domain": "bad.example",
382 +         "matched_url": "https://cdn.bad.example/video",
383 +     }
384 +
385 +
386 +async def test_url_reputation_allowed_domain_passes() -> None:
387 +     result = await UrlReputationStage().evaluate(
388 +         _ctx(
389 +             policy_json='{"security_checks":{"blocked_domains":["bad.example"]}}',
390 +             text="Watch https://good.example/video",
391 +         )
392 +     )
393 +
394 +     assert result.passed is True
395 +     assert result.reason == "links passed domain reputation checks"
396 +     assert result.metadata == {
397 +         "link_count": 1,
398 +         "domains": ["good.example"],
399 +         "blocked_domain_count": 1,
400 +     }
401 +
402 +
403 +async def test_url_reputation_no_link_caption_passes() -> None:

```



```

404 +     result = await UrlReputationStage().evaluate(
405 +         _ctx(
406 +             policy_json='{"security_checks":{"blocked_domains":["bad.example"]}}',
407 +             text="plain caption with no link",
408 +         )
409 +     )
410 +
411 +     assert result.passed is True
412 +     assert result.reason == "no links to scan"
413 +     assert result.metadata == {
414 +         "link_count": 0,
415 +         "domains": [],
416 +         "blocked_domain_count": 1,
417 +     }
418 +
419 +
420 + async def test_url_reputation_invalid_domain_keeps_valid_blocklist_active() -> None:
421 +     result = await UrlReputationStage().evaluate(
422 +         _ctx(
423 +             policy_json=('{"security_checks":{"blocked_domains":["bad.example","not a
domain"]}}'),
424 +             text="https://bad.example/path",
425 +         )
426 +     )
427 +
428 +     assert result.passed is False
429 +     assert result.metadata == {
430 +         "link_count": 1,
431 +         "domains": ["bad.example"],
432 +         "blocked_domain_count": 1,
433 +         "ignored_invalid_domains": ["not a domain"],
434 +         "blocked_domain": "bad.example",
435 +         "matched_blocklist_domain": "bad.example",
436 +         "matched_url": "https://bad.example/path",
437 +     }
438 +
439 +
440 + # --- CaptionSafetyStage -----
441 +
442 +
443 @@ -383,7 +498,10 @@ async def test_evaluate_policy_stages_uses_default_policy_config()
-> None:
444 +         111,
445 +         {
446 +             "topics": {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"},
447 +             "security_checks": {"unsafe_caption_terms": ["malware"]},
448 +             "security_checks": {
449 +                 "blocked_domains": ["bad.example"],
450 +                 "unsafe_caption_terms": ["malware"],
451 +             },
452 +         },
453 +     )
454 +     item = ContentItem(
455 @@ -398,10 +516,11 @@ async def test_evaluate_policy_stages_uses_default_policy_config()
-> None:
456 +
457 +     assert [result.stage_name for result in results] == [
458 +         "topic-preferences",
459 +         "url-reputation",
460 +         "caption-safety",
461 +     ]
462 +     assert policy_passed is True
463 +     assert [result.passed for result in results] == [True, True]
464 +     assert [result.passed for result in results] == [True, True, True]
465 +     assert results[0].metadata == {"matched_include_topics": ["AI"]}

```

```

466
467         topic_passed, topic_rejected = await _evaluate_policy_stages(
468     @@ -416,6 +535,20 @@ async def test_evaluate_policy_stages_uses_default_policy_config()
-> None:
469         assert topic_rejected[0].passed is False
470         assert topic_rejected[0].metadata == {"matched_exclude_topics": ["Crypto"]}
471
472     +         url_passed, url_rejected = await _evaluate_policy_stages(
473     +             conn,
474     +             111,
475     +             src,
476     +             item.model_copy(update={"text": "AI https://bad.example/path"}),
477     +         )
478     +
479     +         assert url_passed is False
480     +         assert [result.stage_name for result in url_rejected] == [
481     +             "topic-preferences",
482     +             "url-reputation",
483     +         ]
484     +         assert url_rejected[-1].metadata["matched_blocklist_domain"] == "bad.example"
485     +
486         caption_passed, caption_rejected = await _evaluate_policy_stages(
487             conn,
488             111,
489     @@ -426,6 +559,7 @@ async def test_evaluate_policy_stages_uses_default_policy_config()
-> None:
490         assert caption_passed is False
491         assert [result.stage_name for result in caption_rejected] == [
492             "topic-preferences",
493     +         "url-reputation",
494             "caption-safety",
495         ]
496         assert caption_rejected[-1].passed is False
497

```

### 531. assistant (2026-07-06T22:46:46.263Z)

F4 continues the good patterns (fail-open, per-entry invalid-skip, metadata-only), and ? importantly ? the **\*\*subdomain match is correct\*\*** (``domain == blocked or domain.endswith("."+blocked)``, so ``notbad.example`` won't match ``bad.example``). But the new **\*\*`\_URL\_RE` is a stdlib `re` regex run on the full caption with no timeout and no length cap\*\*** ? unlike F3's protected ``regex`` scans. That's the #1 thing to test. Let me probe it for ReDoS and check the gate.

### 532. user (2026-07-06T22:46:59.639Z)

=== gate ===

=== RUFF ===

All checks passed!

36 files already formatted

=== PYTEST ===

```

-----
TOTAL                                1547      60    442      42    95%

```

Required test coverage of 80.0% reached. Total coverage: 94.87%

533 passed in 9.22s

DONE

=== ReDoS probe on `_URL_RE` (stdlib re, no timeout) with adversarial captions (8s cap each) ===

subdomain correctness: `notbad.example` vs `bad.example` = False (expect False)

`cdn.bad.example` vs `bad.example` = True (expect True)

```

many dotted single labels no TLD    len= 8001 -> 0.001s (0 urls)
long hyphen label no dot            len= 8002 -> 0.000s (0 urls)
long a.a.a chain                    len= 8000 -> 0.001s (0 urls)
long non-space after www.           len= 20004 -> 0.004s (0 urls)
giant caption realistic              len= 56000 -> 0.022s (2000 urls)

```

rc=0 (124=a probe hung the 40s budget)

### 533. assistant (2026-07-06T22:47:51.822Z)

Two key results:

- **``\_URL\_RE` is ReDoS-safe** ? adversarial inputs up to 56K chars all extract in <0.03s (the mandatory ``\.` separators + bounded `{0,61}` prevent catastrophic backtracking). And unlike F3's \*user\*-supplied patterns, `\_URL\_RE` is developer-controlled and fixed, so no timeout is needed.
- **Subdomain match is correct** ? `notbad.example` is NOT blocked by `bad.example`;  
`cdn.bad.example` is.
- Gate: ruff clean, **533 passed @ 94.87%**.

This is clean and reuses F3's proven patterns. Remaining concerns are low (IDN/obfuscation evasion ? an acceptable v1 metadata limitation). Let me run a focused adversarial pass on domain-parsing evasion and the wiring, then decide.

### 534. user (2026-07-06T22:48:37.031Z)

Workflow launched in background. Task ID: wxxxwpgn5

Summary: Review telegram-video-filter PR #53 (F4 URL/domain reputation) ? regex/domain-parsing security & wiring, with adversarial verification

Transcript dir: C:\Users\avidu\claud\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_65bdc027-8a2

Script file: C:\Users\avidu\claud\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr53-f4-wf\_65bdc027-8a2.js

(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:

"C:\Users\avidu\claud\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr53-f4-wf\_65bdc027-8a2.js"} to iterate without resending the script.)

Run ID: wf\_65bdc027-8a2

To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\claud\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr53-f4-wf\_65bdc027-8a2.js", resumeFromRunId: "wf\_65bdc027-8a2"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

### 535. assistant (2026-07-06T22:48:56.298Z)

While the workflow runs, let me verify the highest-severity risk myself ? **false positives** (wrongly blocking a legit domain), which for a blocklist is worse than evasion. And confirm `normalize\_domain` never throws.

### 536. user (2026-07-06T22:49:06.517Z)

```
<tool_use_error>InputValidationError: [
  {
    "code": "custom",
    "path": [
      "command"
    ],
    "message": "command contains control characters that would be hidden in the approval dialog"
  }
]</tool_use_error>
```

### 537. assistant (2026-07-06T22:49:10.492Z)

The inline command tripped a control-character check. Let me write the battery via the Write tool.

### 538. assistant (2026-07-06T22:49:19.160Z)

```
from tg_video_filter.policy import extract_caption_urls, _domain_matches_blocked,
```

```

normalize_domain

BLOCK = "bad.example"

def blocked(text):
    for u in extract_caption_urls(text):
        if _domain_matches_blocked(u.domain, BLOCK):
            return u.domain
    return None

print("=== FALSE-POSITIVE battery (blocklist=bad.example; None = correctly NOT blocked) ===")
cases = [
    ("legit good.com, bad.example in query", "visit good.com/r?u=bad.example"),
    ("bad.example.evil.com (evil subdomain)", "http://bad.example.evil.com/x"),
    ("userinfo bad.example@good.com", "http://bad.example@good.com/x"),
    ("lookalike notbad.example", "http://notbad.example/x"),
    ("lookalike xbad.example", "http://xbad.example/x"),
    ("suffix bad.example.co", "http://bad.example.co/x"),
]
for name, t in cases:
    print(" %40s -> blocked=%s" % (name, blocked(t)))

print(" --- true positives (SHOULD block) ---")
for name, t in [
    ("exact bad.example", "go bad.example now"),
    ("sub cdn.bad.example", "http://cdn.bad.example/x"),
    ("userinfo host = bad.example", "http://good.com@bad.example/x"),
]:
    print(" %40s -> blocked=%s" % (name, blocked(t)))

print("=== normalize_domain robustness (must never THROW; return str|None) ===")
weird = ["", " ", "http://", "://", "http://[", "http://:99", "." * 300, "a..b",
    "http://xn--", " bad", "http://999.999.999.999", "http://0x7f.1",
    "http://user:p@ss@h", "HTTP://K.com"]
for w in weird:
    try:
        print(" %24r -> %r" % (w[:22], normalize_domain(w)))
    except Exception as e:
        print(" %24r -> THREW %s: %s" % (w[:22], type(e).__name__, e))

```

### 539. user (2026-07-06T22:49:19.191Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/fp\_battery.py (file state is current in your context ? no need to Read it back)

### 540. user (2026-07-06T22:49:25.058Z)

```

=== FALSE-POSITIVE battery (blocklist=bad.example; None = correctly NOT blocked) ===
legit good.com, bad.example in query      -> blocked=None
bad.example.evil.com (evil subdomain)     -> blocked=None
userinfo bad.example@good.com              -> blocked=None
lookalike notbad.example                  -> blocked=None
lookalike xbad.example                    -> blocked=None
suffix bad.example.co                     -> blocked=None
--- true positives (SHOULD block) ---
exact bad.example                         -> blocked=bad.example
sub cdn.bad.example                       -> blocked=cdn.bad.example
userinfo host = bad.example                 -> blocked=bad.example
=== normalize_domain robustness (must never THROW; return str|None) ===
''                                         -> None
' '                                       -> None
'http://'                               -> None

```

```

':://' -> None
'http://[' -> THREW ValueError: Invalid IPv6 URL
'http://:99' -> None
'.....' -> None
'a..b' -> None
'http://xn--' -> None
' bad' -> None
'http://999.999.999.999' -> '999.999.999.999'
'http://0x7f.1' -> '0x7f.1'
'http://user:p@ss@h' -> None
'HTTP://K.com' -> 'k.com'

```

#### 541. assistant (2026-07-06T22:50:01.200Z)

**\*\*Found a real issue.\*\*** `normalize_domain("http://[")` throws an uncaught `ValueError: Invalid IPv6 URL` (from `urlsplit`) ? it's only expected to return `None` for bad input. The false-positive battery is clean (no legit domain wrongly blocked, all true-positives blocked), but this throw is concerning because `extract_caption_urls` runs `normalize_domain` on **attacker-controlled caption URLs**, and that call is *outside* the stage's try/except. Let me verify it propagates as an uncaught exception through the stage.

#### 542. assistant (2026-07-06T22:50:10.417Z)

```

import asyncio, json
from tg_video_filter.models import ContentItem, Policy, Source
from tg_video_filter.policy import extract_caption_urls, UrlReputationStage, PolicyContext,
parse_domain_reputation_config

def ctx(cfg, text):
    return PolicyContext(
        content_item=ContentItem(account_id=1, source_id=1, provider_message_id="5", text=text,
dedup_key="k"),
        source=Source(id=1, account_id=1, provider="telegram", provider_source_id="-1"),
        policy=Policy(id=1, account_id=1, name="default", config_json=json.dumps(cfg),
is_default=True),
    )

async def main():
    # 1. extract_caption_urls on an attacker caption containing 'http://['
    for cap in ["see http://[ here", "http://[bad", "visit http://[:1 now"]]:
        try:
            r = extract_caption_urls(cap)
            print("extract_caption_urls(%r) -> %s urls (ok)" % (cap, len(r)))
        except Exception as e:
            print("extract_caption_urls(%r) -> THREW %s: %s" % (cap, type(e).__name__, e))

    # 2. full stage, domain-rep ENABLED, attacker caption -> does evaluate throw (uncaught)?
    try:
        res = await UrlReputationStage().evaluate(ctx({"security_checks": {"blocked_domains":
["bad.example"]}}, "clip http://[ nice"))
        print("STAGE evaluate -> passed=%s (handled)" % res.passed)
    except Exception as e:
        print("STAGE evaluate -> THREW %s: %s <== uncaught in ingestion path" %
(type(e).__name__, e))

    # 3. config side: a blocked_domains entry with '[' -> parse throws -> whole stage fail-open?
    try:
        c = parse_domain_reputation_config({"security_checks": {"blocked_domains":
["bad.example", "http://[ ]}})
        print("parse config -> blocked=%s invalid=%s (per-entry handled)" % (c.blocked_domains,
c.invalid_domains))
    except Exception as e:
        print("parse config -> THREW %s: %s <== one bad entry disables whole stage (fail-open)"

```

```
% (type(e).__name__, e))
```

```
asyncio.run(main())
```

#### 543. user (2026-07-06T22:50:10.453Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/throw\_probe.py (file state is current in your context ? no need to Read it back)

#### 544. user (2026-07-06T22:50:17.201Z)

```
extract_caption_urls('see http://[ here') -> THREW ValueError: Invalid IPv6 URL
extract_caption_urls('http://[bad') -> THREW ValueError: Invalid IPv6 URL
extract_caption_urls('visit http://[::1 now') -> THREW ValueError: Invalid IPv6 URL
STAGE evaluate -> THREW ValueError: Invalid IPv6 URL  <== uncaught in ingestion path
parse config -> THREW ValueError: Invalid IPv6 URL  <== one bad entry disables whole stage
(fail-open)
```

#### 545. assistant (2026-07-06T22:52:14.937Z)

**\*\*Confirmed ? a real medium bug.\*\*** `normalize\_domain` violates its `str | None` contract by **\*\*throwing `ValueError`\*\*** (from `urlsplit` on an unterminated IPv6 `[`) instead of returning `None`. Both callers assume None, so:

- An **\*\*attacker-controlled caption\*\*** containing `http://[` throws an **\*\*uncaught exception out of the URL stage\*\*** in the live ingestion path (the extraction is outside the stage's try/except).
- One `[`-containing config entry **\*\*disables the whole stage\*\*** (parse throws ? fail-open), defeating the per-entry granularity the author correctly built for other bad domains.

Both latent (no config command yet) but real; trivial fix (catch `ValueError` from `urlsplit`). The false-positive battery was otherwise clean (no legit domain wrongly blocked). I'll wait for the workflow's independent take, then post a review **\*\*holding the LGTM\*\*** on this medium.

#### 546. user (2026-07-06T22:52:17.327Z)

```
<task-notification>
<task-id>wxxxwpgn5</task-id>
<tool-use-id>toolu_01MRUfEGdW8c3RSWAtCjC5wr</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\wxxxwpgn5.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Review telegram-video-filter PR #53 (F4 URL/domain reputation) ? regex/domain-parsing security & wiring, with adversarial verification" completed</summary>
<result>{"counts":{"raw":0,"kept":0,"refuted":0},"findings":[],"refuted":[]}</result>
<failures>[review:domain_parsing] failed: API Error: Connection closed mid-response. The response above may be incomplete.
[review:wiring_config] failed: API Error: Connection closed mid-response. The response above may be incomplete.</failures>
<usage><agent_count>2</agent_count><subagent_tokens>106426</subagent_tokens><tool_uses>20</tool_uses><duration_ms>170606</duration_ms></usage>
</task-notification>
```

#### 547. assistant (2026-07-06T22:52:52.370Z)

The workflow died on transient API errors (both agents hit "Connection closed mid-response") ? no findings from it this round. But my manual verification was thorough and deterministic, so I have a solid picture. Given I'm already holding the LGTM on the confirmed medium, let me do one quick cross-contamination check the workflow would have covered, then post.

#### 548. user (2026-07-06T22:53:01.343Z)

Traceback (most recent call last):

```
File "C:\Users\avidu\Projects\wt-pr53\src\tg_video_filter\policy.py", line 486, in evaluate
```

```

    config = load_caption_safety_config(ctx.policy)
    File "C:\Users\avidu\Projects\wt-pr53\src\tg_video_filter\policy.py", line 245, in
load_caption_safety_config
    return parse_caption_safety_config(decoded)
    File "C:\Users\avidu\Projects\wt-pr53\src\tg_video_filter\policy.py", line 178, in
parse_caption_safety_config
    raw_patterns = _string_tuple(
        raw_security.get("unsafe_caption_patterns"),
        field_name="security_checks.unsafe_caption_patterns",
    )
    File "C:\Users\avidu\Projects\wt-pr53\src\tg_video_filter\policy.py", line 260, in
_string_tuple
    raise ValueError(f"{field_name} must be an array")
ValueError: security_checks.unsafe_caption_patterns must be an array
URL stage w/ bad sibling caption cfg -> passed=False (expect False=still blocks)
Caption stage w/ its own bad cfg -> passed=True (fail-open, expect True)

```

#### 549. assistant (2026-07-06T22:53:14.712Z)

Cross-contamination is clean ? the two security stages parse `security\_checks` independently (the URL stage still blocks correctly despite a bad sibling caption config). I have a complete, verified picture. Per your rule, the confirmed medium means I hold the LGTM. Posting the review.

#### 550. assistant (2026-07-06T22:53:37.696Z)

### Review ? F4 URL/domain reputation stage

Strong PR ? it reuses the F3 patterns well (fail-open, per-entry invalid-skip, metadata-only) and the domain matching is precise. **\*\*Holding the LGTM on one confirmed medium\*\*** (a contract violation in `normalize\_domain` that surfaces as an uncaught exception on attacker-controlled captions). Trivial fix. Everything below was reproduced locally.

#### Verified good ?

- **\*\*No false positives\*\*** (the important risk for a blocklist): `notbad.example`, `xbad.example`, `bad.example.co`, `bad.example.evil.com`, and `bad.example` in a query string / userinfo are all correctly **\*\*not\*\*** blocked; exact/subdomain/real-host are correctly blocked. `domain\_matches\_blocked` uses `domain == blocked or endswith(".") + blocked` ? no naive-suffix bug.
- **\*\*`\_URL\_RE` is ReDoS-safe\*\*** ? adversarial inputs to 56K chars extract in <0.03s (mandatory `.` separators + bounded `{0,61}`); and it's developer-controlled/fixed, so no timeout needed.
- **\*\*Metadata-only\*\*** ? no fetch/DNS (`ip\_address()` doesn't resolve, `encode('idna')` is local).
- Per-entry invalid-domain granularity (invalid ? `invalid\_domains`, valid stay active); independent parsing (a bad `unsafe\_caption\_patterns` sibling doesn't break the URL stage); correct wiring `topic ? url ? caption` with short-circuit. **\*\*533 passed @ 94.87%\*\***, ruff clean.

#### ? Medium ? `normalize\_domain` throws instead of returning `None`

`normalize\_domain(value) -> str | None` is expected to return `None` for bad input, but `urlsplit` raises `ValueError: Invalid IPv6 URL` on an unterminated `[` and that's **\*\*not caught\*\*** (only `ip\_address`'s `ValueError` and `encode`'s `UnicodeError` are). Reproduced: `normalize\_domain("http://[")` throws. Two consequences, both from the same root:

1. **\*\*Attacker-controlled caption crashes the stage (uncaught, in the live ingestion path).\*\*** `extract\_caption\_urls(ctx.content\_item.text)` runs `normalize\_domain` on caption-derived URLs, and that call is **\*\*outside\*\*** the stage's try/except. Reproduced: a caption `clip http://[ nice` makes `UrlReputationStage.evaluate` throw `ValueError` ? propagates through `evaluate\_pipeline` ? `evaluate\_policy\_stages` ? `on\_new\_message`. An attacker posts a matching video with such a caption and policy evaluation for that message throws.
2. **\*\*One `[`-containing blocklist entry disables the whole stage.\*\*** `parse\_domain\_reputation\_config(["bad.example", "http://[")` throws (instead of routing `http://[` to `invalid\_domains`) ? the stage fails open, disabling **\*\*all\*\*** domain blocking ? defeating the per-entry granularity you correctly built for other bad entries.

Both are latent today (no command sets `blocked\_domains`), but they ship in the stage's code.  
> **\*\*Fix (trivial):\*\*** make `normalize\_domain` total ? wrap the `urlsplit(...)` (and ideally the

```

whole body) so any parse error returns `None`:
> ```python
> try:
>     parsed = urlsplit(text if "://" in text else f"//{text}")
> except ValueError:
>     return None
> ```
> Add a test: `normalize_domain("http://[")` is None, and a caption `"http://["` passes the
stage (0 links) rather than throwing.

```

#### **? Low (acceptable v1 limitations ? false-negatives only, per the ADR's "metadata-only" framing)**

- **\*\*Evasion:\*\*** IDN homographs and domains that fail Python's IDNA2003 `encode('idna')` `normalize` to `None` ? not blocked; IP obfuscation (`http://0x7f.1`, decimal IPs) is treated as a bare domain (not resolved), so it evades an IP blocklist. Fine for a v1 reputation stage ? worth a note for future hardening.
- **\*\*Scope:\*\*** dormant ? no `/set security blocked_domains` command yet (the medium becomes user-reachable once it lands, so worth fixing before then regardless).

---

**\*\*Holding LGTM\*\*** on the medium (make `normalize_domain` total). It's a couple of lines + a test, and it closes an attacker-controllable uncaught-exception path in ingestion. Ping me when it's in and I'll re-verify and sign off. Everything else is genuinely clean ? the false-positive precision and ReDoS-safety in particular.

<sub>? Reviewed with [Claude Code](https://claude.com/claude-code) ? false-positive battery, ReDoS probes, the `normalize_domain` throw, and cross-stage independence all reproduced against the branch. (The multi-agent pass hit transient API errors this round; findings are from thorough manual verification.)</sub>

#### **551. user (2026-07-06T22:53:37.727Z)**

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr53\_review.md (file state is current in your context ? no need to Read it back)

#### **552. user (2026-07-06T22:53:48.880Z)**

```

=== post review (holding LGTM) on #53 ===
https://github.com/avidullu/telegram-video-filter/pull/53#issuecomment-4898337290
=== cleanup worktree ===
removed

```

#### **553. assistant (2026-07-06T22:53:55.618Z)**

Review posted (LGTM held) ? [issuecomment-4898337290](https://github.com/avidullu/telegram-video-filter/pull/53#issuecomment-4898337290). Worktree cleaned up. Re-arming the watcher.

#### **554. user (2026-07-06T22:54:01.922Z)**

Command running in background with ID: bmpa2wfsa. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bmpa2wfsa.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### **555. assistant (2026-07-06T22:54:13.908Z)**

### **PR #53 (F4 URL/domain reputation) ? reviewed, \*\*LGTM held\*\***

[Review posted](https://github.com/avidullu/telegram-video-filter/pull/53#issuecomment-4898337290). Per your standing rule, I held the LGTM on one confirmed **\*\*medium\*\***.

**\*\*Verified good\*\*** (all reproduced): **\*\*no false positives\*\*** ? the blocklist is precise



(`notbad.example`, `bad.example.evil.com`, query-string/userinfo `bad.example` all correctly \*not\* blocked; subdomains correctly blocked); \*\*`\_URL\_RE` is ReDoS-safe\*\* (56K-char adversarial inputs <0.03s); metadata-only (no fetch); per-entry invalid-domain granularity; independent stage parsing; correct `topic?url?caption` wiring. Gate: 533 passed @ 94.87%, ruff clean.

\*\*\*? Medium held:\*\* `normalize\_domain` breaks its `str | None` contract ? `urlsplit` throws `ValueError` on an unterminated `[`, uncaught. So (1) an \*\*attacker caption\*\* `http://[` throws an \*\*uncaught exception out of the URL stage\*\* in the live ingestion path, and (2) one `[`-containing config entry disables the whole stage. Latent (no config command) but shipped; trivial fix (catch `ValueError` from `urlsplit` ? return `None`). Plus low v1-evasion notes (IDN/IP obfuscation).

Transparency note: the multi-agent workflow hit transient API errors this round (both agents dropped mid-response), so this review rests on my own thorough manual verification ? false-positive battery, ReDoS probes, the throw repro, and cross-stage independence all reproduced directly.

## Status

- \*\*Track F:\*\* #40 ? · F1 #50 ? · F2 #51 ? · F3 #52 ? · \*\*F4 #53 ? changes requested\*\*.
- \*\*Watcher re-armed\*\* (`bmpa2wfsa`, baseline #53) ? it'll wake me on the author's fix-push to #53 (which it'll catch via the head-SHA change), a reply, or the next PR.

I'll surface the next activity when it lands, and re-verify #53 the moment a fix is pushed.

### 556. user (2026-07-06T22:56:04.907Z)

```
<task-notification>
<task-id>bmpa2wfsa</task-id>
<tool-use-id>toolu_01QXfopAUYeBsqqTRh6LE7Ma</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bmpa2wfsa.output</output-file>
<status>completed</status>
<summary>Background command "Re-arm watcher v2 (baseline #53)" completed (exit code 0)</summary>
</task-notification>
```

### 557. assistant (2026-07-06T22:56:07.933Z)

The watcher fired. Let me see what changed on #53.

### 558. user (2026-07-06T22:56:10.142Z)

```
=== PR-watch v2 RE-ARMED ===
baseline: #53 draft=false sha=6d563f823 upd=2026-07-06T22:53:48Z
=== PR ACTIVITY DETECTED (poll 1, 22:56:03Z) ===
--- baseline ---
#53 draft=false sha=6d563f823 upd=2026-07-06T22:53:48Z
--- now ---
#53 draft=false sha=6d563f823 upd=2026-07-06T22:54:12Z
--- new/changed ---
#53 draft=false sha=6d563f823 upd=2026-07-06T22:54:12Z
--- open PR details ---
#53 draft=false by avidullu: feat(policy): add F4 URL reputation stage
```

### 559. assistant (2026-07-06T22:56:18.134Z)

The watcher caught \*\*comment activity\*\* on #53 ? SHA unchanged (`6d563f823`), but `updatedAt` bumped. That's a review-thread reply, not a force-push ? exactly the signal you asked me to watch for (the old watcher would've missed it). Let me read what the author said.

### 560. user (2026-07-06T22:56:23.215Z)

```
=== latest issue comments on #53 (after my review) ===
? avidullu 2026-07-06T22:53:48Z
```

## Review ? F4 URL/domain reputation stage

Strong PR ? it reuses the F3 patterns well (fail-open, per-entry invalid-skip, metadata-only) and the domain matching is precise. **\*\*Holding the LGTM on one confirmed medium\*\*** (a contract violation in ``normalize_domain`` that surfaces as an uncaught exception on attacker-controlled captions). Trivial fix. Everything below was reproduced locally.

### Verified good ?

- **\*\*No false positives\*\*** (the important risk for a blocklist): ``notbad.example``, ``xbad.example``, ``bad.example.co``, ``bad.example.evil.com``, and ``bad.example`` in a query string / userinfo are all correctly **\*\*not\*\*** blocked; exact/subdomain/real-host are correctly blocked. ``_domain_matches_blocked`` uses ``domain == blocked or endswith("." + blocked)`` ? no naive-suffix bug.
- **\*\*`\_URL\_RE` is ReDoS-safe\*\*** ? adversarial inputs to 56K chars extract in <0.03s (mandatory ``.`` separators + bounded ``{0,61}``); and it's developer-controlled/fixed, so no timeout needed.
- **\*\*Metadata-only\*\*** ? no fetch/DNS (``ip_address()`` doesn't resolve, ``encode('idna')`` is local).
- Per-entry invalid-domain granularity (invalid ? ``invalid_domains``, valid stay active); independent parsing (a bad ``unsafe_caption_patterns`` sibling doesn't break the URL stage); correct wiring ``topic ? url ? caption`` with short-circuit. **\*\*533 passed @ 94.87%\*\***, ruff clean.

### ? Medium ? ``normalize_domain`` throws instead of returning ``None``

``normalize_domain(value) -> str | None`` is expected to return ``None`` for bad input, but ``urlsplit`` raises ``ValueError: Invalid IPV6 URL`` on an unterminated ``[`` and that's **\*\*not caught\*\*** (only ``ip_address``'s ``ValueError`` and ``encode``'s ``UnicodeError`` are). Reproduced: ``normalize_domain("http://[")`` throws. Two consequences, both from the same root:

1. **\*\*Attacker-controlled caption crashes the stage (uncaught, in the live ingestion path).\*\*** ``extract_caption_urls(ctx.content_item.text)`` runs ``normalize_domain`` on caption-derived URLs, and that call is **\*\*outside\*\*** the stage's try/except. Reproduced: a caption ``"clip http://[ nice]"`` makes ``UrlReputationStage.evaluate`` throw ``ValueError`` ? propagates through ``evaluate_pipeline`` ? ``_evaluate_policy_stages`` ? ``on_new_message``. An attacker posts a matching video with such a caption and policy evaluation for that message throws.
2. **\*\*One ``[``-containing blocklist entry disables the whole stage.\*\*** ``parse_domain_reputation_config(["bad.example", "http://[")`` throws (instead of routing ``http://[`` to ``invalid_domains``) ? the stage fails open, disabling **\*\*all\*\*** domain blocking ? defeating the per-entry granularity you correctly built for other bad entries.

Both are latent today (no command sets ``blocked_domains``), but they ship in the stage's code.

> **\*\*Fix (trivial):\*\*** make ``normalize_domain`` total ? wrap the ``urlsplit(...)`` (and ideally the whole body) so any parse error returns ``None``:

> ``python``

> try:

> `parsed = urlsplit(text if "://" in text else f"//{text}")`

> except ``ValueError``:

> `return None`

> ```

> Add a test: ``normalize_domain("http://[")`` is ``None``, and a caption ``"http://["`` passes the stage (0 links) rather than throwing.

### ? Low (acceptable v1 limitations ? false-negatives only, per the ADR's "metadata-only" framing)

- **\*\*Evasion:\*\*** IDN homographs and domains that fail Python's IDNA2003 ``encode('idna')`` normalize to ``None`` ? not blocked; IP obfuscation (``http://0x7f.1``, decimal IPs) is treated as a bare domain (not resolved), so it evades an IP blocklist. Fine for a v1 reputation stage ? worth a note for future hardening.
- **\*\*Scope:\*\*** dormant ? no ``set security blocked_domains`` command yet (the medium becomes user-reachable once it lands, so worth fixing before then regardless).

---

**\*\*Holding LGTM\*\*** on the medium (make ``normalize_domain`` total). It's a couple of lines + a test, and it closes an attacker-controllable uncaught-exception path in ingestion. Ping me when it's in and I'll re-verify and sign off. Everything else is genuinely clean ? the false-positive precision and ReDoS-safety in particular.

<sub>? Reviewed with [Claude Code](https://claude.com/claude-code) ? false-positive battery, ReDoS probes, the `normalize\_domain` throw, and cross-stage independence all reproduced against the branch. (The multi-agent pass hit transient API errors this round; findings are from thorough manual verification.)</sub>

? avidullu 2026-07-06T22:54:12Z

Ready for review on F4. Scope is metadata-only URL extraction from captured captions/stored `link\_url`, `security\_checks.blocked\_domains` matching, no fetch/redirect/download behavior, and stage order `topic-preferences -> url-reputation -> caption-safety`. Validation is clean: ruff format/check and pytest -> 533 passed @ 94.87% coverage.

=== any review-thread (inline) comments? ===  
(empty = none)

#### 561. assistant (2026-07-06T22:56:45.602Z)

The activity was **not a fix** ? the SHA is unchanged (`6d563f823`, no code change), and the comment (`22:54:12Z`) is a generic **"Ready for review on F4"** re-announcement posted ~24 s after my review (too fast to be a substantive response; it doesn't touch the `normalize\_domain` finding). The earlier `22:53:48Z` comment is my own review.

So there's **nothing new to re-review** ? the code is identical and my held medium stands. Per your rule, I'm not re-LGTM'ing on an unchanged PR with an open finding. Let me re-arm the watcher for the actual fix.

#### 562. user (2026-07-06T22:56:53.338Z)

Command running in background with ID: b7p0yh0ep. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b7p0yh0ep.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 563. assistant (2026-07-06T22:57:03.776Z)

### Update ? #53 activity was a re-announce, not a fix

The upgraded watcher did its job (caught comment activity the old one would've missed), but on inspection it was a non-actionable **"Ready for review on F4"** comment ? posted ~24 s after my review, SHA unchanged (`6d563f823`), and it doesn't address the `normalize\_domain` medium. So:

- **No code change** ? nothing to re-review. My held-LGTM review stands; the medium (`normalize\_domain` throwing on `http://[` ? uncaught exception in ingestion / whole-stage-disable) is still open.
- **I did not re-LGTM** ? per your rule, I only sign off when the PR is genuinely merge-ready, and it hasn't changed.

One small watcher-tuning note for transparency: an unchanged-SHA + bumped-`updatedAt` wake is a "comment happened" signal ? useful (that's how I'll catch the author's reply), but it also fires on canned/automated comments like this one. When it wakes me I check the SHA first: **SHA changed** ? re-review the fix; **SHA same** ? read the comment and act only if substantive (as I did here).

**Status:** Track F ? #40/#50/#51/#52 merged ?; **#53** still awaiting the `normalize\_domain` fix. Watcher re-armed (`b7p0yh0ep`); it'll wake me on the real fix-push (SHA change) or the next PR, and I'll re-verify then.

#### 564. user (2026-07-06T23:00:56.861Z)

<task-notification>  
<task-id>b7p0yh0ep</task-id>  
<tool-use-id>toolu\_017sXof96tsqt3wLYSqHMMB</tool-use-id>  
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b7p0yh0ep.output</output-file>

```
<status>completed</status>
<summary>Background command "Re-arm watcher after non-substantive comment" completed (exit code 0)</summary>
</task-notification>
```

#### 565. assistant (2026-07-06T23:00:59.275Z)

The watcher fired. Let me check what changed on #53.

#### 566. user (2026-07-06T23:01:01.705Z)

```
=== PR-watch v2 RE-ARMED (baseline reflects the re-announce comment; next wake = real change)
===
baseline: #53 draft=false sha=6d563f823 upd=2026-07-06T22:54:12Z
poll 1/30: no change (22:58:55Z)
=== PR ACTIVITY DETECTED (poll 2, 23:00:56Z) ===
--- baseline ---
#53 draft=false sha=6d563f823 upd=2026-07-06T22:54:12Z
--- now ---
#53 draft=false sha=f24beef15 upd=2026-07-06T23:00:20Z
--- new/changed (note: sha change=force-push/fix; only upd change=comment) ---
#53 draft=false sha=f24beef15 upd=2026-07-06T23:00:20Z
--- open PR details ---
#53 draft=false by avidullu: feat(policy): add F4 URL reputation stage
```

#### 567. assistant (2026-07-06T23:01:07.530Z)

SHA changed (`6d563f8` ? `f24beef`) ? a real **fix push** for the `normalize\_domain` medium. Let me fetch and see what changed.

#### 568. user (2026-07-06T23:01:11.655Z)

```
=== fix diff (6d563f8..f24beef) ===
src/tg_video_filter/policy.py | 5 +++-
tests/test_policy.py          | 18 ++++++
2 files changed, 22 insertions(+), 1 deletion(-)

diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
index a90cf50..be60459 100644
--- a/src/tg_video_filter/policy.py
+++ b/src/tg_video_filter/policy.py
@@ -342,7 +342,10 @@ def normalize_domain(value: str) -> str | None:
     if not text:
         return None

-    parsed = urlsplit(text if "://" in text else f"//{text}")
+    try:
+        parsed = urlsplit(text if "://" in text else f"//{text}")
+    except ValueError:
+        return None
     host = parsed.hostname
     if host is None:
         return None

diff --git a/tests/test_policy.py b/tests/test_policy.py
index e792ef7..1dd660a 100644
--- a/tests/test_policy.py
+++ b/tests/test_policy.py
@@ -229,6 +229,7 @@ def test_normalize_domain_strips_scheme_port_www_case_and_trailing_dot() ->
None
    assert normalize_domain("HTTPS://www.Example.COM:443/path") == "example.com"
    assert normalize_domain("sub.Example.com.") == "sub.example.com"
    assert normalize_domain("https://[2001:db8::1]:443/path") == "2001:db8::1"
+    assert normalize_domain("http://[") is None
```



Required test coverage of 80.0% reached. Total coverage: 94.93%  
534 passed in 9.12s  
DONE

#### 573. assistant (2026-07-06T23:01:56.685Z)

Gate green: ruff clean, \*\*534 passed @ 94.93%\*\* (+1 test, coverage up). The medium is fixed and verified, no new issues. Per your rule, I'm satisfied ? merge-ready. Posting the LGTM.

#### 574. user (2026-07-06T23:02:08.677Z)

```
=== post LGTM ===  
https://github.com/avidullu/telegram-video-filter/pull/53#issuecomment-4898384190  
=== cleanup worktree ===  
removed
```

#### 575. assistant (2026-07-06T23:02:13.228Z)

LGTM posted ? [issuecomment-4898384190](https://github.com/avidullu/telegram-video-filter/pull/53#issuecomment-4898384190). Worktree cleaned. Re-arming the watcher.

#### 576. user (2026-07-06T23:02:20.141Z)

Command running in background with ID: bxs7jboj. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bxs7jboj.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 577. assistant (2026-07-06T23:02:30.567Z)

### PR #53 (F4) ? fix verified, \*\*LGTM'd\*\* ?

The watcher's SHA-vs-`updatedAt` distinction paid off: the earlier wake was a canned comment (SHA unchanged ? I held), and this one was the real \*\*fix push\*\* (SHA `6d563f8` ? `f24beef` ? I re-reviewed). [LGTM posted](https://github.com/avidullu/telegram-video-filter/pull/53#issuecomment-4898384190).

The fix is exactly what I recommended ? `normalize\_domain` wrapped in `try/except ValueError: return None`. Verified against my own repro:

- Attacker caption `http://["` ? 0 links, stage passes (no uncaught exception) ?
- Config `[ "bad.example", "http://[" ]` ? `bad.example` stays blocked, `http://["` ? `invalid\_domains` (per-entry, whole stage not disabled) ?
- Plus two targeted tests. Gate: ruff clean, \*\*534 passed @ 94.93%\*\*.

### Status

- \*\*Track F:\*\* #40 ? · F1 #50 ? · F2 #51 ? · F3 #52 ? · \*\*F4 #53 ?\*\* ? all reviewed, verified, LGTM'd (F2/F3/F4 each had findings the author fixed before I signed off).
- \*\*Watcher re-armed\*\* (`bxs7jboj`, baseline fixed-#53) ? wakes me on the #53 merge, the next F-phase (F5 deep-inspect, F6 decisions table, F7 `why`, or the `set` command phases), or any force-push/comment.

The review cadence is humming: every finding reproduced, LGTM strictly gated on verified fixes, and the author turning them around fast. I'll surface the next activity when it lands.

#### 578. user (2026-07-06T23:06:23.690Z)

```
<task-notification>  
<task-id>bxs7jboj</task-id>  
<tool-use-id>toolu_01XiHGuvFCapQRRTcnoCihH</tool-use-id>  
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bxs7jboj.output</output-file>  
<status>completed</status>  
<summary>Background command "Re-arm watcher v2 (baseline fixed #53)" completed (exit code
```

0)</summary>  
</task-notification>

579. assistant (2026-07-06T23:06:26.922Z)

The watcher fired. Let me check what changed.

580. user (2026-07-06T23:06:29.168Z)

```
=== PR-watch v2 RE-ARMED ===
baseline: #53 draft=false sha=f24beef15 upd=2026-07-06T23:02:08Z
poll 1/30: no change (23:04:22Z)
=== PR ACTIVITY DETECTED (poll 2, 23:06:22Z) ===
--- baseline ---
#53 draft=false sha=f24beef15 upd=2026-07-06T23:02:08Z
--- now ---
<none>
--- new/changed (sha change=force-push/fix; only upd change=comment) ---

--- open PR details ---
```

581. assistant (2026-07-06T23:06:32.618Z)

Open set ? empty ? #53 merged after my LGTM. Let me confirm and check for the next PR.

582. user (2026-07-06T23:06:39.382Z)

```
=== #53 state ===
state=MERGED merged=2026-07-06T23:05:48Z commit=a7cbbda79
=== origin/main recent ===
a7cbbda feat(policy): add URL reputation stage (#53)
5cd65d1 feat(policy): add caption safety stage (#52)
6d3d7a6 feat(policy): add F2 topic preferences (#51)
=== open PRs now ===
(empty=none)
```

583. assistant (2026-07-06T23:06:43.626Z)

\*\*#53 (F4) merged\*\* ? (`a7cbbda`). No open PRs. Fast-forwarding `main` and re-arming for the next one.

584. user (2026-07-06T23:06:50.226Z)

Command running in background with ID: bgnuea2ay. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bgnuea2ay.output. You will be notified when it completes. To check interim output, use Read on that file path.

585. assistant (2026-07-06T23:06:58.840Z)

Update

\*\*PR #53 (F4 URL/domain reputation) is merged\*\* ? (`a7cbbda`) ? merged right after my verified LGTM. No open PRs at the moment.

```
**Track F ? all reviewed, verified, merged:**
| PR | Phase | Findings ? outcome |
|----|-----|-----|
| #40 | ADR 0012 design | 10 design gaps ? revised ? LGTM |
| #50 | F1 primitives + persistence | clean ? LGTM |
| #51 | F2 topic preferences | 3 mediums ? fixed ? LGTM |
| #52 | F3 caption safety | 3 mediums (ReDoS/fail-open/case) ? fixed ? LGTM |
```

| #53 | F4 URL reputation | 1 medium (`normalize\_domain` throw) ? fixed ? LGTM |

Every finding reproduced locally, LGTM strictly gated on verified fixes, and the author has turned each round around within minutes.

**\*\*State:\*\*** local `main` fast-forwarded to `a7cbbda` (all F1?F4 in), clean and synced. **\*\*Watcher re-armed\*\*** (`bgnuea2ay`, empty baseline) ? it'll wake me on the next PR (F5 deep-inspect, F6 policy-decisions table, F7 `/why`, or the `/set` config commands) or any force-push/comment, and I'll run the same verify-then-only-LGTM-when-satisfied cycle.

I'll surface the next one as soon as it lands.

#### 586. user (2026-07-06T23:12:56.277Z)

```
<task-notification>
<task-id>bgnuea2ay</task-id>
<tool-use-id>toolu_01YWGkMjpsssjfVcgiv8YHcS</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bgnuea2ay.output</output-file>
<status>completed</status>
<summary>Background command "Fast-forward main and re-arm watcher" completed (exit code
0)</summary>
</task-notification>
```

#### 587. assistant (2026-07-06T23:13:00.583Z)

The watcher fired. Let me check what it caught.

#### 588. user (2026-07-06T23:13:02.836Z)

```
3 files changed, 378 insertions(+), 3 deletions(-)
local main: a7cbbda feat(policy): add URL reputation stage (#53)
=== PR-watch v2 RE-ARMED ===
baseline: <none>
poll 1/30: no change (23:08:53Z)
poll 2/30: no change (23:10:54Z)
=== PR ACTIVITY DETECTED (poll 3, 23:12:55Z) ===
--- baseline ---
<none>
--- now ---
#54 draft=false sha=49f15a8a9 upd=2026-07-06T23:11:34Z
--- new/changed (sha change=force-push/fix; only upd change=comment) ---
#54 draft=false sha=49f15a8a9 upd=2026-07-06T23:11:34Z
--- open PR details ---
#54 draft=false by avidullu: feat(policy): add F5 deep inspection budget boundary
```

#### 589. assistant (2026-07-06T23:13:19.808Z)

**\*\*New PR #54 detected\*\*** ? "feat(policy): add F5 deep inspection budget boundary" (F5 = Tier 4, the tier that per ADR 0012 \*amends\* ADR 0003 to allow opt-in bounded download/ffprobe). Critical to scrutinize: whether it actually does network I/O (SSRF/resource risks) or just establishes the budget/fetcher boundary, and that the ADR-0003-amendment guardrails (opt-in + passed Tiers 1-3 + enforced budget) hold. Let me sync and start.

#### 590. user (2026-07-06T23:13:31.852Z)

```
* [new branch]      feat/f5-deep-inspection-budget -> origin/feat/f5-deep-inspection-budget
=== PR #54 META ===
{"additions":364,"body":"## Summary\n- add ADR 0012 F5 `PolicyBudget` and `ContentFetcher`\nshapes for opt-in Tier 4 work\n- add `DeepInspectionStage`, enabled only by `deep_inspect: true`\nor `{ \"enabled\": true }`\n- refuse missing/invalid budgets, missing size metadata, and size\nbudget exceedance as `StageResult` failures\n- enforce timeout and max-concurrency around\n`fetcher.inspect(...)` before any heavy work can run\n- wire the stage after F1-F4: topic
```



```
preferences -> URL reputation -> caption safety -> deep inspection\n\n## Scope\nNo downloader,
ffprobe, perceptual hash, or content scan implementation is bundled here. This PR only lands the
execution boundary so later deep-inspection work has an explicit opt-in and enforceable budget
contract.\n\n## Validation\n- `./venv/Scripts/python.exe -m ruff format src tests`\n-
`./venv/Scripts/python.exe -m ruff check src tests`\n- `./venv/Scripts/python.exe -m pytest
tests/test_policy.py --no-cov` -> 42 passed\n- `./venv/Scripts/python.exe -m pytest` -> 543
passed @ 94.57% coverage\n\nCloses
#47\r\n", "changedFiles": 3, "deletions": 4, "headRefName": "feat/f5-deep-inspection-budget", "isDraft"
: false, "mergeStateStatus": "CLEAN", "mergeable": "MERGEABLE", "number": 54, "title": "feat(policy): add
F5 deep inspection budget boundary"}
```

```
=== files + freshness ===
src/tg_video_filter/policy.py
src/tg_video_filter/telethon_client.py
tests/test_policy.py
fatal: ambiguous argument 'origin/feat/f5-deep-inspection..origin/main': unknown revision or
path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
gate started
```

#### 591. assistant (2026-07-06T23:13:45.207Z)

The branch is `feat/f5-deep-inspection-budget` (I used the wrong name ? the worktree/gate ran against a bad path). Also good news from the body: **no downloader/ffprobe** ? this PR only lands the **execution boundary + budget contract** (opt-in, timeout, max-concurrency, size checks), no actual network I/O. Let me fix the worktree/gate and read the diff.

#### 592. user (2026-07-06T23:13:53.015Z)

```
HEAD is now at 49f15a8 feat(policy): add deep inspection budget boundary
freshness:
(empty=current)
gate restarted
```

#### 593. user (2026-07-06T23:13:57.395Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\Telegram.

#### 594. assistant (2026-07-06T23:14:06.171Z)

The diff wasn't saved (the earlier `&&` chain broke on the bad branch name). Let me re-save and read it.

#### 595. user (2026-07-06T23:14:09.953Z)

```
499 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr54.diff
```

#### 596. user (2026-07-06T23:14:17.751Z)

```
1      diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
2      index be60459..879b28d 100644
3      --- a/src/tg_video_filter/policy.py
4      +++ b/src/tg_video_filter/policy.py
5      @@ -7,6 +7,7 @@
6
7      from __future__ import annotations
8
9      +import asyncio
10     import json
11     import logging
12     import re
13     @@ -39,6 +40,43 @@
```

```

14     _TRAILING_URL_PUNCTUATION = "., !?;:"
15
16
17     +@dataclass(frozen=True)
18     +class PolicyBudget:
19     +     """Bounded execution budget for opt-in deep inspection."""
20     +
21     +     max_bytes_per_item: int
22     +     timeout_s: float
23     +     max_concurrent: int
24     +     _semaphore: asyncio.Semaphore | None = field(
25     +         default=None,
26     +         init=False,
27     +         repr=False,
28     +         compare=False,
29     +     )
30     +
31     +     def __post_init__(self) -> None:
32     +         if self.max_concurrent > 0:
33     +             object.__setattr__(self, "_semaphore",
34     +                 asyncio.Semaphore(self.max_concurrent))
35     +     async def inspect(self, fetcher: ContentFetcher, content_item: ContentItem) ->
36     +         StageResult:
37     +             """Run ``fetcher`` under the timeout and concurrency budget."""
38     +             if self._semaphore is None:
39     +                 raise ValueError("max_concurrent must be positive")
40     +             async with self._semaphore:
41     +                 return await asyncio.wait_for(
42     +                     fetcher.inspect(content_item, self),
43     +                     timeout=self.timeout_s,
44     +                 )
45     +
46     +class ContentFetcher(Protocol):
47     +     """Fetcher used only by opt-in Tier 4 policy stages."""
48     +
49     +     async def inspect(self, content_item: ContentItem, budget: PolicyBudget) ->
50     +         StageResult:
51     +             """Inspect content under ``budget`` and return a policy result."""
52     +             ...
53     +
54     @dataclass(frozen=True)
55     class PolicyContext:
56         """Immutable snapshot available to a policy stage."""
57     @@ -46,8 +84,8 @@ class PolicyContext:
58         content_item: ContentItem
59         source: Source
60         policy: Policy
61         - fetcher: object | None = None
62         - budget: object | None = None
63         + fetcher: ContentFetcher | None = None
64         + budget: PolicyBudget | None = None
65
66
67     @dataclass(frozen=True)
68     @@ -253,6 +291,21 @@ def load_domain_reputation_config(policy: Policy) ->
69     DomainReputationConfig:
70         return parse_domain_reputation_config(decoded)
71
72
73     +def load_deep_inspection_enabled(policy: Policy) -> bool:
74     +     """Return whether Tier 4 deep inspection is explicitly enabled."""
75     +     decoded = json.loads(policy.config_json)

```

```

75     +     if not isinstance(decoded, dict):
76     +         raise ValueError("policy config_json must decode to a JSON object")
77     +     raw_deep_inspect = decoded.get("deep_inspect", False)
78     +     if isinstance(raw_deep_inspect, bool):
79     +         return raw_deep_inspect
80     +     if isinstance(raw_deep_inspect, dict):
81     +         enabled = raw_deep_inspect.get("enabled", False)
82     +         if isinstance(enabled, bool):
83     +             return enabled
84     +     raise ValueError("deep_inspect must be a boolean or an object with boolean
enabled")
85     +
86     +
87     def _string_tuple(value: object, *, field_name: str) -> tuple[str, ...]:
88     +     if value is None:
89     +         return ()
90     @@ -618,6 +671,89 @@ def _caption_safety_metadata(
91     +     return metadata
92
93
94     +class DeepInspectionStage:
95     +     """ADR 0012 Tier 4 opt-in boundary for budgeted heavy inspection."""
96     +
97     +     stage_name = "deep-inspection"
98     +
99     +     async def evaluate(self, ctx: PolicyContext) -> StageResult:
100    +         try:
101    +             enabled = load_deep_inspection_enabled(ctx.policy)
102    +             except (json.JSONDecodeError, ValueError) as exc:
103    +                 log.warning(
104    +                     "deep inspection disabled due to invalid config policy_id=%s",
105    +                     ctx.policy.id,
106    +                     exc_info=True,
107    +                 )
108    +                 return StageResult.pass_(
109    +                     self.stage_name,
110    +                     f"invalid deep inspection config ignored: {exc}",
111    +                     invalid_deep_inspection_config=True,
112    +                 )
113    +             if not enabled:
114    +                 return StageResult.pass_(self.stage_name, "deep inspection disabled")
115    +
116    +             budget_result = _deep_inspection_budget_result(ctx)
117    +             if budget_result is not None:
118    +                 return budget_result
119    +
120    +             if ctx.fetcher is None:
121    +                 return StageResult.fail(
122    +                     self.stage_name,
123    +                     "deep inspection fetcher missing",
124    +                     missing_fetcher=True,
125    +                 )
126    +
127    +             assert ctx.budget is not None
128    +             try:
129    +                 return await ctx.budget.inspect(ctx.fetcher, ctx.content_item)
130    +             except TimeoutError:
131    +                 return StageResult.fail(
132    +                     self.stage_name,
133    +                     "deep inspection budget exceeded",
134    +                     budget_exceeded=True,
135    +                     exceeded_budget="timeout_s",
136    +                     timeout_s=ctx.budget.timeout_s,
137    +                 )
138    +

```

```

139 +
140 +def _deep_inspection_budget_result(ctx: PolicyContext) -> StageResult | None:
141 +    budget = ctx.budget
142 +    if budget is None:
143 +        return StageResult.fail(
144 +            DeepInspectionStage.stage_name,
145 +            "deep inspection budget missing",
146 +            missing_budget=True,
147 +        )
148 +    if budget.max_bytes_per_item <= 0 or budget.timeout_s <= 0 or budget.max_concurrent
<= 0:
149 +        return StageResult.fail(
150 +            DeepInspectionStage.stage_name,
151 +            "deep inspection budget invalid",
152 +            invalid_budget=True,
153 +            max_bytes_per_item=budget.max_bytes_per_item,
154 +            timeout_s=budget.timeout_s,
155 +            max_concurrent=budget.max_concurrent,
156 +        )
157 +    size_bytes = ctx.content_item.size_bytes
158 +    if size_bytes is None:
159 +        return StageResult.fail(
160 +            DeepInspectionStage.stage_name,
161 +            "deep inspection budget cannot be enforced without content size",
162 +            missing_size_bytes=True,
163 +            max_bytes_per_item=budget.max_bytes_per_item,
164 +        )
165 +    if size_bytes > budget.max_bytes_per_item:
166 +        return StageResult.fail(
167 +            DeepInspectionStage.stage_name,
168 +            "deep inspection budget exceeded",
169 +            budget_exceeded=True,
170 +            exceeded_budget="max_bytes_per_item",
171 +            size_bytes=size_bytes,
172 +            max_bytes_per_item=budget.max_bytes_per_item,
173 +        )
174 +    return None
175 +
176 +
177 +    async def evaluate_pipeline(
178 +        ctx: PolicyContext,
179 +        stages: list[PolicyStage],
180 @@ -635,9 +771,12 @@ async def evaluate_pipeline(
181 +    __all__ = [
182 +        "CaptionSafetyConfig",
183 +        "CaptionSafetyStage",
184 +        "ContentFetcher",
185 +        "DeepInspectionStage",
186 +        "DomainReputationConfig",
187 +        "ExtractedUrl",
188 +        "PolicyContext",
189 +        "PolicyBudget",
190 +        "PolicyStage",
191 +        "StageResult",
192 +        "TopicPreferenceStage",
193 @@ -646,6 +785,7 @@ async def evaluate_pipeline(
194 +        "evaluate_pipeline",
195 +        "extract_caption_urls",
196 +        "load_caption_safety_config",
197 +        "load_deep_inspection_enabled",
198 +        "load_domain_reputation_config",
199 +        "load_topic_preferences",
200 +        "normalize_domain",
201 +    diff --git a/src/tg_video_filter/telethon_client.py
b/src/tg_video_filter/telethon_client.py

```

```

202 index 3c7d534..2d5cbb1 100644
203 --- a/src/tg_video_filter/telethon_client.py
204 +++ b/src/tg_video_filter/telethon_client.py
205 @@ -46,6 +46,7 @@
206 )
207 from tg_video_filter.policy import (
208     CaptionSafetyStage,
209 +     DeepInspectionStage,
210     PolicyContext,
211     StageResult,
212     TopicPreferenceStage,
213 @@ -439,7 +440,12 @@ async def _evaluate_policy_stages(
214     return True, []
215     return await evaluate_pipeline(
216         PolicyContext(content_item=content_item, source=source, policy=policy),
217 -        [TopicPreferenceStage(), UrlReputationStage(), CaptionSafetyStage()],
218 +        [
219 +            TopicPreferenceStage(),
220 +            UrlReputationStage(),
221 +            CaptionSafetyStage(),
222 +            DeepInspectionStage(),
223 +        ],
224     )
225
226
227 diff --git a/tests/test_policy.py b/tests/test_policy.py
228 index 1dd660a..2ca3f22 100644
229 --- a/tests/test_policy.py
230 +++ b/tests/test_policy.py
231 @@ -2,6 +2,8 @@
232
233 from __future__ import annotations
234
235 +import asyncio
236 +
237 import aiosqlite
238
239 from tg_video_filter import db
240 @@ -9,8 +11,10 @@
241 from tg_video_filter.policy import (
242     CaptionSafetyConfig,
243     CaptionSafetyStage,
244 +     DeepInspectionStage,
245     DomainReputationConfig,
246     ExtractedUrl,
247 +     PolicyBudget,
248     PolicyContext,
249     StageResult,
250     TopicPreferences,
251 @@ -31,6 +35,9 @@ def _ctx(
252     policy_json: str = "{}",
253     source_title: str | None = None,
254     text: str | None = None,
255 +     size_bytes: int | None = None,
256 +     fetcher: object | None = None,
257 +     budget: PolicyBudget | None = None,
258 ) -> PolicyContext:
259     return PolicyContext(
260         content_item=ContentItem(
261 @@ -38,6 +45,7 @@ def _ctx(
262         source_id=1,
263         provider_message_id="42",
264         text=text,
265 +     size_bytes=size_bytes,
266 +     dedup_key="tg:msg:1:42",

```

```

267         ),
268         source=Source(
269     @@ -54,6 +62,8 @@ def _ctx(
270         config_json=policy_json,
271         is_default=True,
272     ),
273     +         fetcher=fetcher, # type: ignore[arg-type]
274     +         budget=budget,
275     )
276
277
278     @@ -496,6 +506,209 @@ async def
test_caption_safety_invalid_config_fails_open_with_explanation() -> No
279         assert result.metadata == {"invalid_caption_safety_config": True}
280
281
282     +# --- DeepInspectionStage -----
283     +
284     +
285     +class _FakeFetcher:
286     +    def __init__(self) -> None:
287     +        self.calls: list[tuple[ContentItem, PolicyBudget]] = []
288     +
289     +    async def inspect(self, content_item: ContentItem, budget: PolicyBudget) ->
StageResult:
290     +        self.calls.append((content_item, budget))
291     +        return StageResult.pass_("deep-inspection", "deep inspection passed",
inspected=True)
292     +
293     +
294     +class _SlowFetcher:
295     +    async def inspect(self, content_item: ContentItem, budget: PolicyBudget) ->
StageResult:
296     +        await asyncio.sleep(1)
297     +        return StageResult.pass_("deep-inspection", "too late")
298     +
299     +
300     +class _ConcurrencyFetcher:
301     +    def __init__(self) -> None:
302     +        self.active = 0
303     +        self.max_active = 0
304     +
305     +    async def inspect(self, content_item: ContentItem, budget: PolicyBudget) ->
StageResult:
306     +        self.active += 1
307     +        self.max_active = max(self.max_active, self.active)
308     +        try:
309     +            await asyncio.sleep(0.01)
310     +            return StageResult.pass_("deep-inspection", "deep inspection passed")
311     +        finally:
312     +            self.active -= 1
313     +
314     +
315     +async def test_deep_inspection_opt_out_passes_without_fetcher_or_budget() -> None:
316     +    fetcher = _FakeFetcher()
317     +
318     +    result = await DeepInspectionStage().evaluate(_ctx(fetcher=fetcher,
size_bytes=1_000))
319     +
320     +    assert result.passed is True
321     +    assert result.reason == "deep inspection disabled"
322     +    assert fetcher.calls == []
323     +
324     +
325     +async def test_deep_inspection_missing_budget_fails_before_fetcher() -> None:

```

```

326 +     fetcher = _FakeFetcher()
327 +
328 +     result = await DeepInspectionStage().evaluate(
329 +         _ctx(policy_json='{"deep_inspect":true}', fetcher=fetcher, size_bytes=1_000)
330 +     )
331 +
332 +     assert result.passed is False
333 +     assert result.reason == "deep inspection budget missing"
334 +     assert result.metadata == {"missing_budget": True}
335 +     assert fetcher.calls == []
336 +
337 +
338 +async def test_deep_inspection_invalid_budget_fails_before_fetcher() -> None:
339 +     fetcher = _FakeFetcher()
340 +     budget = PolicyBudget(max_bytes_per_item=0, timeout_s=1.0, max_concurrent=1)
341 +
342 +     result = await DeepInspectionStage().evaluate(
343 +         _ctx(
344 +             policy_json='{"deep_inspect":true}',
345 +             fetcher=fetcher,
346 +             budget=budget,
347 +             size_bytes=1_000,
348 +         )
349 +     )
350 +
351 +     assert result.passed is False
352 +     assert result.reason == "deep inspection budget invalid"
353 +     assert result.metadata == {
354 +         "invalid_budget": True,
355 +         "max_bytes_per_item": 0,
356 +         "timeout_s": 1.0,
357 +         "max_concurrent": 1,
358 +     }
359 +     assert fetcher.calls == []
360 +
361 +
362 +async def test_deep_inspection_missing_size_fails_before_fetcher() -> None:
363 +     fetcher = _FakeFetcher()
364 +     budget = PolicyBudget(max_bytes_per_item=2_000, timeout_s=1.0, max_concurrent=1)
365 +
366 +     result = await DeepInspectionStage().evaluate(
367 +         _ctx(policy_json='{"deep_inspect":true}', fetcher=fetcher, budget=budget)
368 +     )
369 +
370 +     assert result.passed is False
371 +     assert result.reason == "deep inspection budget cannot be enforced without content
size"
372 +     assert result.metadata == {
373 +         "missing_size_bytes": True,
374 +         "max_bytes_per_item": 2_000,
375 +     }
376 +     assert fetcher.calls == []
377 +
378 +
379 +async def test_deep_inspection_budget_exceeded_fails_before_fetcher() -> None:
380 +     fetcher = _FakeFetcher()
381 +     budget = PolicyBudget(max_bytes_per_item=999, timeout_s=1.0, max_concurrent=1)
382 +
383 +     result = await DeepInspectionStage().evaluate(
384 +         _ctx(
385 +             policy_json='{"deep_inspect":true}',
386 +             fetcher=fetcher,
387 +             budget=budget,
388 +             size_bytes=1_000,
389 +         )

```

```

390 + )
391 +
392 + assert result.passed is False
393 + assert result.reason == "deep inspection budget exceeded"
394 + assert result.metadata == {
395 +     "budget_exceeded": True,
396 +     "exceeded_budget": "max_bytes_per_item",
397 +     "size_bytes": 1_000,
398 +     "max_bytes_per_item": 999,
399 + }
400 + assert fetcher.calls == []
401 +
402 +
403 +async def test_deep_inspection_timeout_budget_fails_as_stage_result() -> None:
404 +     budget = PolicyBudget(max_bytes_per_item=2_000, timeout_s=0.001, max_concurrent=1)
405 +
406 +     result = await DeepInspectionStage().evaluate(
407 +         _ctx(
408 +             policy_json='{"deep_inspect":true}',
409 +             fetcher=_SlowFetcher(),
410 +             budget=budget,
411 +             size_bytes=1_000,
412 +         )
413 +     )
414 +
415 +     assert result.passed is False
416 +     assert result.reason == "deep inspection budget exceeded"
417 +     assert result.metadata == {
418 +         "budget_exceeded": True,
419 +         "exceeded_budget": "timeout_s",
420 +         "timeout_s": 0.001,
421 +     }
422 +
423 +
424 +async def test_deep_inspection_budget_limits_concurrency() -> None:
425 +     budget = PolicyBudget(max_bytes_per_item=2_000, timeout_s=1.0, max_concurrent=1)
426 +     fetcher = _ConcurrencyFetcher()
427 +     stage = DeepInspectionStage()
428 +
429 +     results = await asyncio.gather(
430 +         stage.evaluate(
431 +             _ctx(
432 +                 policy_json='{"deep_inspect":true}',
433 +                 fetcher=fetcher,
434 +                 budget=budget,
435 +                 size_bytes=1_000,
436 +             )
437 +         ),
438 +         stage.evaluate(
439 +             _ctx(
440 +                 policy_json='{"deep_inspect":true}',
441 +                 fetcher=fetcher,
442 +                 budget=budget,
443 +                 size_bytes=1_000,
444 +             )
445 +         ),
446 +     )
447 +
448 +     assert [result.passed for result in results] == [True, True]
449 +     assert fetcher.max_active == 1
450 +
451 +
452 +async def test_deep_inspection_missing_fetcher_fails_after_budget_passes() -> None:
453 +     budget = PolicyBudget(max_bytes_per_item=2_000, timeout_s=1.0, max_concurrent=1)
454 +

```



```

455 +     result = await DeepInspectionStage().evaluate(
456 +         _ctx(policy_json='{"deep_inspect":true}', budget=budget, size_bytes=1_000)
457 +     )
458 +
459 +     assert result.passed is False
460 +     assert result.reason == "deep inspection fetcher missing"
461 +     assert result.metadata == {"missing_fetcher": True}
462 +
463 +
464 + async def test_deep_inspection_valid_budget_delegates_to_fetcher() -> None:
465 +     fetcher = _FakeFetcher()
466 +     budget = PolicyBudget(max_bytes_per_item=2_000, timeout_s=1.0, max_concurrent=1)
467 +
468 +     result = await DeepInspectionStage().evaluate(
469 +         _ctx(
470 +             policy_json='{"deep_inspect":{"enabled":true}}',
471 +             fetcher=fetcher,
472 +             budget=budget,
473 +             size_bytes=1_000,
474 +         )
475 +     )
476 +
477 +     assert result.passed is True
478 +     assert result.reason == "deep inspection passed"
479 +     assert result.metadata == {"inspected": True}
480 +     assert [(item.size_bytes, call_budget) for item, call_budget in fetcher.calls] == [
481 +         (1_000, budget)
482 +     ]
483 +
484 +
485 + async def test_evaluate_policy_stages_uses_default_policy_config() -> None:
486 +     conn = await aiosqlite.connect(":memory:")
487 +     try:
488 +         @@ -536,9 +749,10 @@ async def test_evaluate_policy_stages_uses_default_policy_config()
-> None:
489 +             "topic-preferences",
490 +             "url-reputation",
491 +             "caption-safety",
492 +         +             "deep-inspection",
493 +         ]
494 +         assert policy_passed is True
495 +         - assert [result.passed for result in results] == [True, True, True]
496 +         + assert [result.passed for result in results] == [True, True, True, True]
497 +         assert results[0].metadata == {"matched_include_topics": ["AI"]}
498 +
499 +         topic_passed, topic_rejected = await _evaluate_policy_stages(
500 +

```

### 597. assistant (2026-07-06T23:15:23.680Z)

I've read the full F5 diff ? it's a clean **\*\*boundary-only\*\*** PR (no downloader/ffprobe): concrete ``PolicyBudget`/`ContentFetcher`` types (finally replacing the F1 ``object`` placeholders ? my old low finding), a ``DeepInspectionStage`` that enforces opt-in + budget (size-from-metadata, timeout, semaphore concurrency) **\*before\*** any fetcher call, comprehensive tests. One thing I want to confirm: since F5 doesn't wire a fetcher/budget into the live path, does enabling ``deep_inspect: true`` today **\*\*drop all matched videos\*\*** (fail-closed)? Let me check the gate and that behavior.

### 598. user (2026-07-06T23:15:36.748Z)

```

=== gate ===
=== RUFF ===
All checks passed!
36 files already formatted
=== PYTEST ===

```

```
-----
TOTAL                                1612    67    468    46    95%
Required test coverage of 80.0% reached. Total coverage: 94.57%
543 passed in 9.37s
DONE
```

```
=== live-path behavior: deep_inspect enabled but no fetcher/budget wired (F5) ===
deep_inspect=true, no fetcher/budget in live path -> policy_passed=False
  stages: [('topic-preferences', True, 'topic preferences disabled'), ('url-reputation', True,
'domain reputation disabled'), ('caption-safety', True, 'caption safety disabled'), ('deep-
inspection', False, 'deep inspection budget missing')]
deep_inspect unset (default) -> policy_passed=True (expect True)
```

#### 599. assistant (2026-07-06T23:16:11.680Z)

Confirmed the behavior ? and it's the **\*\*safe (fail-closed) direction\*\***: ``deep_inspect: true`` with no fetcher/budget wired ? the item is **\*\*dropped\*\*** (``policy_passed=False``, "budget missing"); ``deep_inspect`` unset ? unaffected. That's intentional (tests assert it) and dormant (no command), just an operational note. Gate: ruff clean, **\*\*543 passed @ 94.57%\*\*** (+9 tests, comprehensive). This is clean ? the budget enforcement runs **\*before\*** any fetcher call. Let me run a focused adversarial pass on the budget-bypass surface to be sure, then decide.

#### 600. user (2026-07-06T23:16:54.878Z)

Workflow launched in background. Task ID: wh85h1wao  
Summary: Review telegram-video-filter PR #54 (F5 deep-inspection budget boundary) ?  
budget/concurrency/timeout enforcement, with adversarial verification  
Transcript dir: C:\Users\avidu\claudu\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_58a19e3e-3ce  
Script file: C:\Users\avidu\claudu\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr54-f5-wf\_58a19e3e-3ce.js  
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\claudu\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr54-f5-wf\_58a19e3e-3ce.js"} to iterate without resending the script.)  
Run ID: wf\_58a19e3e-3ce  
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\claudu\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr54-f5-wf\_58a19e3e-3ce.js", resumeFromRunId: "wf\_58a19e3e-3ce"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

#### 601. assistant (2026-07-06T23:17:02.460Z)

While the workflow runs, let me verify the semaphore concurrency semantics myself ? specifically that it only limits globally when the ``PolicyBudget`` is shared (the future-wiring note), and that a timeout releases the permit (no leak).

#### 602. assistant (2026-07-06T23:17:14.569Z)

```
import asyncio
from tg_video_filter.models import ContentItem, Policy, Source
from tg_video_filter.policy import DeepInspectionStage, PolicyBudget, PolicyContext, StageResult

class Tracker:
    def __init__(self):
        self.active = 0
        self.max_active = 0

    async def inspect(self, content_item, budget):
        self.active += 1
```

```

        self.max_active = max(self.max_active, self.active)
    try:
        await asyncio.sleep(0.02)
        return StageResult.pass_("deep-inspection", "ok")
    finally:
        self.active -= 1

def ctx(fetcher, budget):
    return PolicyContext(
        content_item=ContentItem(account_id=1, source_id=1, provider_message_id="5",
size_bytes=1000, dedup_key="k"),
        source=Source(id=1, account_id=1, provider="telegram", provider_source_id="-1"),
        policy=Policy(id=1, account_id=1, name="default", config_json='{"deep_inspect":true}',
is_default=True),
        fetcher=fetcher, budget=budget,
    )

async def main():
    stage = DeepInspectionStage()

    # A) SHARED budget across 5 concurrent inspects, max_concurrent=2 -> max_active should be 2
    t = Tracker()
    shared = PolicyBudget(max_bytes_per_item=2000, timeout_s=1.0, max_concurrent=2)
    await asyncio.gather(*[stage.evaluate(ctx(t, shared)) for _ in range(5)])
    print("A) shared budget, max_concurrent=2, 5 tasks -> max_active=%d (expect 2 = limited)" %
t.max_active)

    # B) SEPARATE budget per task (what a per-message wiring would do) -> concurrency NOT
limited
    t2 = Tracker()
    await asyncio.gather(*[stage.evaluate(ctx(t2, PolicyBudget(max_bytes_per_item=2000,
timeout_s=1.0, max_concurrent=2))) for _ in range(5)])
    print("B) separate budgets, 5 tasks -> max_active=%d (expect 5 = NOT limited; singleton
required)" % t2.max_active)

    # C) timeout must release the permit (no leak): a slow fetcher times out, then a fast one
still runs
    class Slow:
        async def inspect(self, content_item, budget):
            await asyncio.sleep(1)
            return StageResult.pass_("deep-inspection", "late")
    b = PolicyBudget(max_bytes_per_item=2000, timeout_s=0.01, max_concurrent=1)
    r1 = await stage.evaluate(ctx(Slow(), b))
    tt = Tracker()
    r2 = await stage.evaluate(ctx(tt, b)) # reuse same budget -> permit must have been released
    print("C) after timeout, reuse budget -> timed_out=%s then next passed=%s (permit
released=%s)" % (
        not r1.passed, r2.passed, r2.passed))

asyncio.run(main())

```

### 603. user (2026-07-06T23:17:14.637Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/f5\_conc.py (file state is current in your context ? no need to Read it back)

### 604. user (2026-07-06T23:17:22.010Z)

A) shared budget, max\_concurrent=2, 5 tasks -> max\_active=2 (expect 2 = limited)  
B) separate budgets, 5 tasks -> max\_active=5 (expect 5 = NOT limited; singleton required)  
C) after timeout, reuse budget -> timed\_out=True then next passed=False (permit released=False)

#### 605. assistant (2026-07-06T23:17:45.797Z)

Wait ? test C's "passed=False" is likely my **test bug**: I reused a budget with ``timeout_s=0.01`` but the second Tracker sleeps ``0.02s``, so it **\*times out\*** (not a permit leak). Let me re-test the permit-release cleanly with correct timing ? a real permit leak on timeout would be a **\*\*high\*\*** finding (exhausts the semaphore ? deadlock).

#### 606. user (2026-07-06T23:17:56.303Z)

after 3 timeouts on `max_concurrent=1` budget...  
fast call after timeouts -> `passed=True` (PERMIT RELEASED - ok)

#### 607. assistant (2026-07-06T23:18:10.931Z)

**\*\*No permit leak\*\*** ? after 3 timeouts on a ``max_concurrent=1`` budget, a subsequent call passes (the ``async`` with `self._semaphore`` releases correctly on ``TimeoutError``). My earlier "passed=False" was a test-timing bug on my part. So the enforcement is solid:

- Shared budget ? concurrency limited ? · separate budgets ? not limited (singleton requirement ? a note for the future live wiring) ?
- Timeout releases the permit (no deadlock) ? · budget checks run before any fetcher call ?

F5 is clean ? findings are low/notes only. I'll fold in the workflow's adversarial pass (still running) before deciding, per your rule. Awaiting it now.

#### 608. user (2026-07-06T23:22:36.938Z)

```
<task-notification>
<task-id>wh85h1wao</task-id>
<tool-use-id>toolu_01EkE1ZDHRT61hB5Rh5Hoxrm</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\wh85h1wao.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Review telegram-video-filter PR #54 (F5 deep-inspection budget
boundary) ? budget/concurrency/timeout enforcement, with adversarial verification"
completed</summary>
<result>{"counts":{"raw":7,"kept":6,"refuted":1},"findings":[{"dimension":"budget_enforcement",
severity":"medium","verdict":"CONFIRMED","title":"Fetcher exceptions other than TimeoutError
propagate uncaught out of DeepInspectionStage and evaluate_pipeline","where":"src/tg_video_filte
r/policy.py:708","summary":"DeepInspectionStage.evaluate wraps ctx.budget.inspect(...) in a
try/except that catches only TimeoutError. Any other exception raised by the delegated
fetcher.inspect (network error, ffprobe/subprocess crash, ValueError, OSError, etc.) escapes the
stage and evaluate_pipeline entirely, unlike every other policy stage which fails-open by
catching and returning a pass StageResult.","failure_scenario":"A wired fetcher raises
RuntimeError('ffprobe exploded') (or any non-timeout error) during deep inspection. The
exception propagates out of evaluate() -> evaluate_pipeline() ->
_evaluate_policy_stages(); the caller at telethon_client.py:283 is not wrapped in try/except, so
the message handler aborts mid-processing (the video is never recorded/delivered and the record
insert at line 301-315 is skipped) rather than producing a clean fail-closed
StageResult.","evidence":"Repro confirmed: with a fetcher raising RuntimeError, both
`DeepInspectionStage().evaluate(ctx)` and `evaluate_pipeline(ctx,[stage])` print 'RAISED
UNCAUGHT: RuntimeError ffprobe exploded'. policy.py:708-717 catches only TimeoutError. No test
in the PR (tests/test_policy.py) exercises a non-TimeoutError fetcher exception. Dormant today
because live wiring passes fetcher=None/budget=None (stage fails at 'budget missing' first), but
active as soon as a real fetcher is added ? which is the whole point of this
boundary.","recommendation":"Decide the failure direction explicitly: either catch Exception
around ctx.budget.inspect and return StageResult.fail(...) (fail-closed, consistent with
dropping suspicious items) or StageResult.pass(...) with an error flag (fail-open, consistent
with the other stages' config-error handling). Add a test asserting the chosen behavior so the
contract is pinned before a fetcher lands.","verify_reasoning":"Code facts verified.
policy.py:708-717 (DeepInspectionStage.evaluate) wraps `await ctx.budget.inspect(ctx.fetcher,
ctx.content_item)` in a try/except that catches ONLY TimeoutError; any other exception from the
delegated fetcher.inspect escapes. Repro (PYTHONPATH=.../_wt-pr54/src) with a fetcher raising
RuntimeError('ffprobe exploded') printed \"RAISED UNCAUGHT (stage): RuntimeError ffprobe
exploded\" AND \"RAISED UNCAUGHT (pipeline): RuntimeError ffprobe exploded\", while the
```

TimeoutError control returned a clean fail StageResult (passed=False, reason='deep inspection budget exceeded'). Confirmed asaCo.wait\_for re-raises the inner exception verbatim, so it is not converted.

Caller path verified: telethon\_client.py:283 calls \_evaluate\_policy\_stages, which (lines 431-449) calls evaluate\_pipeline with no try/except; on\_new\_message (184) has no top-level guard either, so an uncaught fetcher exception aborts handler processing before the VideoRecord insert at 301-315 ? the item is silently not recorded (propagates to Telethon's dispatcher; not a process crash, and the finding does not claim one). The finding's characterization ("aborts mid-processing", insert skipped) is accurate.

Fail-open asymmetry is real but slightly overstated: the other stages catch only config-parse errors (json.JSONDecodeError, ValueError) because they do NO delegated I/O; this is the only stage doing a delegated call, and it converts exactly one exception type. The core claim ? that non-timeout fetcher exceptions have no defined failure direction and leak out, unlike the timeout path ? holds.

Dormancy correctly acknowledged by the finding: live wiring passes fetcher=None/budget=None, so \_deep\_inspection\_budget\_result returns 'budget missing' fail before the fetcher call; the exception path is unreachable in production until a real fetcher+budget is wired ? which is the explicit purpose of this boundary PR. No PR test exercises a non-TimeoutError fetcher exception.

Refutation attempts failed: not a budget/timeout/concurrency bypass (when the fetcher raises, the heavy work has already failed ? nothing runs unbounded, and semaphore+wait\_for remain in force), so high is unwarranted; but it is a genuine latent design defect in the exact code this PR introduces, inevitable the moment a fetcher lands, so it is above mere hygiene. Severity medium is correct.

{ "dimension": "config\_wiring", "severity": "medium", "verdict": "CONFIRMED", "title": "Real TimeoutError from fetcher internals is misclassified as a budget-timeout event", "where": "src/tg\_video\_filter/policy.py:709", "summary": "DeepInspectionStage.evaluate wraps budget.inspect in `except TimeoutError`, so any genuine TimeoutError raised inside the fetcher (unrelated to the budget) is reported as budget\_exceeded/exceeded\_budget=timeout\_s with the budget's timeout value, not the actual failure.", "failure\_scenario": "With budget timeout\_s=10.0 and a fetcher that raises TimeoutError immediately (e.g. an inner network/socket timeout that completes in milliseconds), the stage returns FAIL reason='deep inspection budget exceeded', metadata={'budget\_exceeded': True, 'exceeded\_budget': 'timeout\_s', 'timeout\_s': 10.0}. Operators/telemetry see a fabricated 10s budget overrun for an error that was not a budget overrun. Verified by repro: fetcher raising TimeoutError well within a 10s budget produced exactly that mislabeled StageResult.", "evidence": "policy.py evaluate(): `try: return await ctx.budget.inspect(ctx.fetcher, ctx.content\_item) except TimeoutError: return StageResult.fail(... budget\_exceeded=True, exceeded\_budget='timeout\_s', timeout\_s=ctx.budget.timeout\_s)`. asyncio.wait\_for raises asyncio.TimeoutError (== builtin TimeoutError in 3.11+) on real budget expiry, but a builtin TimeoutError from fetcher.inspect propagates through wait\_for unchanged and is caught by the same handler, indistinguishable from a true budget timeout. Repro output: 'passed= False | reason= deep inspection budget exceeded | meta= {budget\_exceeded: True, exceeded\_budget: timeout\_s, timeout\_s: 10.0}' for a fetcher that raised TimeoutError instantly.", "recommendation": "Distinguish the two: let PolicyBudget.inspect catch the wait\_for timeout itself and re-raise a dedicated sentinel (e.g. a BudgetExceeded exception), or check elapsed time, so the stage only reports budget\_exceeded when the budget actually expired. A fetcher-internal TimeoutError should surface as a distinct failure reason. Direction stays fail-closed either way, so this is correctness/observability, not a bypass.", "verify\_reasoning": "The finding is confirmed against this PR's diff. The entire DeepInspectionStage class (policy.py:674-717), including the `except TimeoutError` handler at line 710, is new in this PR. PolicyBudget.inspect (policy.py:66-69) uses `asyncio.wait\_for(..., timeout=self.timeout\_s)`. In this environment (Python 3.13; repro printed `asyncio.TimeoutError is TimeoutError: True`), a builtin TimeoutError raised inside fetcher.inspect propagates through wait\_for unchanged and is caught by the same handler, indistinguishable from a genuine wait\_for budget-expiry timeout.

Repro (real DeepInspectionStage.evaluate with a fetcher raising `TimeoutError` immediately, budget timeout\_s=10.0): elapsed=0.0001s yet the stage returned `passed=False, reason='deep inspection budget exceeded', meta={budget\_exceeded: True, exceeded\_budget: 'timeout\_s', timeout\_s: 10.0}`. So an inner network/socket TimeoutError (socket.timeout is a builtin-TimeoutError alias since 3.10) that completes in milliseconds is reported as a fabricated 10-second budget overrun. Operators/telemetry are misled about the failure cause.

Adversarial refutation attempts fail: (a) the handler is genuinely new PR code; (b) wait\_for does let an inner exception raised before the timeout propagate as-is, and the alias identity makes

... (truncated 18778 chars, full result in C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\wh85h1wao.output)

<usage><agent\_count>9</agent\_count><subagent\_tokens>439301</subagent\_tokens><tool\_uses>131</tool\_uses><duration\_ms>342034</duration\_ms></usage>

</task-notification>

#### 609. assistant (2026-07-06T23:22:55.411Z)

The workflow caught **two mediums I missed** ? both in the fetcher-exception handling (a genuine gap in my analysis, which focused on budget **\*enforcement\*** but not fetcher **\*error\*** handling). Let me parse the full set and verify the exception-leak claim myself.

#### 610. user (2026-07-06T23:23:07.947Z)

COUNTS: {'raw': 7, 'kept': 6, 'refuted': 1}

=====

[MEDIUM/CONFIRMED] (budget\_enforcement) Fetcher exceptions other than TimeoutError propagate uncaught out of DeepInspectionStage and evaluate\_pipeline

where: src/tg\_video\_filter/policy.py:708

rec: Decide the failure direction explicitly: either catch Exception around ctx.budget.inspect and return StageResult.fail(...) (fail-closed, consistent wi

[MEDIUM/CONFIRMED] (config\_wiring) Real TimeoutError from fetcher internals is misclassified as a budget-timeout event

where: src/tg\_video\_filter/policy.py:709

rec: Distinguish the two: let PolicyBudget.inspect catch the wait\_for timeout itself and re-raise a dedicated sentinel (e.g. a BudgetExceeded exception), o

[LOW/CONFIRMED] (budget\_enforcement) Negative or zero-ish content size\_bytes passes the max\_bytes budget check and delegates to the fetcher

where: src/tg\_video\_filter/policy.py:745

rec: Treat size\_bytes <= 0 the same as missing size (fail with missing/invalid size), so only a positive, in-range size delegates to the fetcher. Low sever

[LOW/CONFIRMED] (config\_wiring) Live wiring drops ALL matched videos if deep\_inspect is enabled; footgun is undocumented at the wiring site

where: src/tg\_video\_filter/telethon\_client.py:441

rec: Add a comment at the wiring site (and/or an ADR note) stating that DeepInspectionStage is intentionally wired with no fetcher/budget and therefore dro

[LOW/CONFIRMED] (config\_wiring) Stringly-typed enable (deep\_inspect={'enabled':'true'}) silently fails OPEN instead of enabling

where: src/tg\_video\_filter/policy.py:306

rec: Acceptable if intentional, but consider either coercing common truthy strings, or (better) logging/surfacing the invalid\_deep\_inspection\_config at the

[LOW/CONFIRMED] (config\_wiring) Semaphore acquisition is not covered by the budget timeout; time-to-decision is unbounded under contention

where: src/tg\_video\_filter/policy.py:63

rec: If total time-to-decision must be bounded, wrap the semaphore acquisition inside the timeout as well (e.g. wait\_for over an inner coroutine that acqui

refuted: ['Per-instance \_semaphore does not bound global deep-insp']

=== independently verify: non-timeout fetcher exception leaks uncaught ===

non-timeout fetcher error -> UNCAUGHT RuntimeError: ffmpeg exploded (crashes handler)

inner TimeoutError (instant) -> reason='deep inspection budget exceeded'

meta={'budget\_exceeded': True, 'exceeded\_budget': 'timeout\_s', 'timeout\_s': 1.0} (misabeled as budget)

#### 611. assistant (2026-07-06T23:23:36.475Z)

Both mediums **\*\*independently reproduced\*\***:

- Non-timeout fetcher exception (RuntimeError) ? **\*\*uncaught\*\***, propagates out of the stage/pipeline ? crashes the message handler.

- An inner TimeoutError (instant) ? **\*\*misabeled\*\*** as "deep inspection budget exceeded (timeout\_s=1.0)" ? a socket timeout gets reported as a fabricated budget overrun.

These are genuine gaps in the *\*boundary contract\** this PR exists to define ? latent (no fetcher wired yet) but they'll bite the moment the fetcher phase codes against this contract. Plus 4 lows. Per your rule, I hold the LGTM. Let me post.

612. assistant (2026-07-06T23:24:05.713Z)

## Review ? F5 deep-inspection budget boundary

The budget *\*enforcement\** is solid and comprehensively tested ? but since this PR's whole job is to define the *\*\*execution-boundary contract\*\** that the future fetcher will code against, two gaps in how *\*fetcher exceptions\** are handled are worth closing now. *\*\*Holding the LGTM\*\** on those two mediums. Everything reproduced locally.

### Verified good ?

- *\*\*No budget bypass\*\** ? ``_deep_inspection_budget_result`` runs all checks (budget present/valid, size present, size ? max, from *\*metadata\**) *\*\*before\*\** any fetcher call; the fetcher is only reached when all pass.
- *\*\*Concurrency correct\*\** ? a shared ``PolicyBudget`` limits to ``max_concurrent`` (repro: 5 tasks, ``max_concurrent=2`` ? ``max_active=2``); *\*\*timeout releases the permit\*\** (3 timeouts on a ``max_concurrent=1`` budget, then a fast call still passes ? no leak/deadlock).
- *\*\*Timeout cancels the work\*\** via ``wait_for``; size budget enforced from metadata (ADR-0003-safe); ``deep_inspect`` unset is a no-op pass (no impact on existing behavior); concrete ``PolicyBudget`/`ContentFetcher`` types replace the F1 ``object`` placeholders (closes an old low).

*\*\*543 passed @ 94.57%\*\**, ruff clean.

### ? Medium ? pin the fetcher-exception contract now (this PR defines it)

*\*\*1. Non-timeout fetcher exceptions propagate uncaught\*\** ? ``policy.py:708``  
``DeepInspectionStage.evaluate`` wraps ``ctx.budget.inspect(...)`` in ``except TimeoutError`` *\*\*only\*\**. Any other error from ``fetcher.inspect`` (network, ``ffprobe``/subprocess crash, ``OSError``, ``ValueError``, ?) escapes ``evaluate`` ? ``evaluate_pipeline`` ? ``_evaluate_policy_stages`` ? ``on_new_message`` (no guard) and *\*\*aborts message processing\*\** (the video is never recorded/delivered). Every other stage fails-open on error; this one has no defined direction for fetcher errors. Reproduced: a fetcher raising ``RuntimeError("ffprobe exploded")`` propagates uncaught out of ``evaluate_pipeline``. Dormant today (live wiring passes ``fetcher=None`` ? fails at "budget missing" first), but active the instant a real fetcher lands ? which is this boundary's entire purpose.

> *\*\*Fix:\*\** catch ``Exception`` around ``ctx.budget.inspect`` and return a defined ``StageResult`` ? fail-closed (drop un-inspected/suspicious content) is the natural choice for a security gate; add a test pinning it.

*\*\*2. A real `TimeoutError` from fetcher internals is misclassified as a budget-timeout\*\** ? ``policy.py:709``  
``asyncio.wait_for`` raises ``asyncio.TimeoutError`` (? builtin ``TimeoutError`` in 3.11+) on budget expiry ? but a builtin ``TimeoutError`` raised *\*inside\** ``fetcher.inspect`` (e.g. a socket timeout, which is a ``TimeoutError`` alias) propagates through ``wait_for`` unchanged and is caught by the same handler. Reproduced: a fetcher raising ``TimeoutError`` *\*\*instantly\*\** (elapsed ? 0.0001s) under a ``timeout_s=10.0`` budget reports ``reason="deep inspection budget exceeded", exceeded_budget="timeout_s", timeout_s=10.0`` ? a fabricated 10 s budget overrun. Misleads operators/telemetry.

> *\*\*Fix:\*\** distinguish the two ? have ``PolicyBudget.inspect`` catch the ``wait_for`` timeout itself and re-raise a dedicated sentinel (e.g. ``BudgetExceeded``), so the stage only reports ``budget_exceeded`` on a genuine budget expiry; a fetcher-internal timeout surfaces as its own reason. (Direction stays fail-closed either way ? this is correctness/observability.)

### ? Low

- *\*\*Non-positive size passes the budget check\*\** (``policy.py:745``): ``size_bytes <= 0`` (bogus ``0``/negative metadata) passes ``size_bytes > max_bytes`` and delegates to the fetcher. Treat ``<= 0`` like missing/invalid size.
- *\*\*Undocumented footgun at the wiring site\*\** (``telethon_client.py:441``): enabling ``deep_inspect: true`` today drops *\*\*all\*\** matched videos (no fetcher/budget wired ? "budget missing" ? fail-closed). It's intentional/safe and dormant, but add a comment/ADR note that the ``/set deep_inspect`` command must land *\*with/after\** the fetcher wiring, not before.
- *\*\*Stringly-typed enable fails open\*\** (``policy.py:306``): ``deep_inspect: {"enabled": "true"}``

(string, not bool) ? `ValueError` ? fail-open (silently \*not\* enabled). Consistent with the config-error posture, but a user's near-miss silently no-ops; consider surfacing it.

- **Semaphore acquisition isn't covered by the timeout** (`policy.py:63`): the `timeout\_s` wraps `fetcher.inspect` but not the `async with self.\_semaphore` acquire, so time-to-decision is unbounded under contention. Wrap the acquire in the timeout if total latency must be bounded.

#### Forward-looking note (not a defect in this PR)

The `\_semaphore` is per-`PolicyBudget` instance, so it only bounds **global** concurrency if the future live wiring creates the budget **once** (singleton), not per-message (repro: separate budgets ? `max\_active=5`, not limited). Worth encoding when the fetcher/budget get wired.

---

**Holding LGTM** on #1 and #2 ? small changes that pin down the boundary contract before the fetcher phase inherits an undefined error path. Ping me when they're in and I'll re-verify and sign off. The enforcement core (bypass-free, correct concurrency/timeout, no permit leak) is genuinely solid.

<sub>? Reviewed with [Claude Code](https://claude.com/claude-code) ? budget-bypass, concurrency/permit-release, the uncaught fetcher exception, and the timeout misclassification all reproduced on the branch. Multi-agent adversarial pass: 6 findings kept, 1 refuted.</sub>

#### 613. user (2026-07-06T23:24:05.747Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr54\_review.md (file state is current in your context ? no need to Read it back)

#### 614. user (2026-07-06T23:24:15.867Z)

=== post review (holding LGTM) on #54 ===  
https://github.com/avidullu/telegram-video-filter/pull/54#issuecomment-4898514947  
=== cleanup worktree ===  
removed  
=== re-arm watcher v2 (baseline #54) ===  
baseline: #54 draft=false sha=49f15a8a9 upd=2026-07-06T23:24:14Z

#### 615. assistant (2026-07-06T23:24:20.104Z)

Review posted (LGTM held) ? [issuecomment-4898514947](https://github.com/avidullu/telegram-video-filter/pull/54#issuecomment-4898514947). Worktree cleaned. Arming the watcher (baseline #54).

#### 616. user (2026-07-06T23:24:26.284Z)

Command running in background with ID: b8nvbrd6r. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b8nvbrd6r.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 617. assistant (2026-07-06T23:24:39.310Z)

### PR #54 (F5 deep-inspection budget boundary) ? reviewed, **LGTM held**

[Review posted](https://github.com/avidullu/telegram-video-filter/pull/54#issuecomment-4898514947). Per your rule, held on two mediums ? and notably, **the workflow caught these two, not me**: I'd verified the budget/concurrency/timeout **enforcement** was correct (it is ? no bypass, concurrency limited, permits released, no leak), but under-scrutinized the fetcher **error** handling. Good example of the adversarial pass earning its place.

**? Held mediums** (both reproduced, both in the boundary contract this PR exists to define):  
1. **Non-timeout fetcher exceptions propagate uncaught** ? only `TimeoutError` is caught, so a real fetcher's `RuntimeError`/`OSError`/ffprobe-crash would escape and abort message processing.



No defined fail direction.

2. **\*\*Inner `TimeoutError` misclassified as budget-timeout\*\*** ? a socket timeout that completes instantly is reported as a fabricated 10 s "budget exceeded."

Both latent (fetcher not wired yet) but they'll bite the moment the next phase codes against this contract. Plus 4 lows (non-positive size passes the check; the drop-all footgun undocumented at the wiring site; string-enable fails open; semaphore-acquire not covered by the timeout). One finding correctly refuted (per-instance semaphore ? a forward-looking wiring note, not a defect here).

**\*\*Verified good:\*\*** budget checks run before any fetcher call, shared-budget concurrency correct, timeout releases permits (no deadlock), size-from-metadata (ADR-0003-safe), and it closes my old F1 `object`-placeholder finding. 543 passed @ 94.57%.

## Status

- **\*\*Track F:\*\*** #40/#50/#51/#52/#53 merged ? . **\*\*F5 #54 ? changes requested\*\***.  
- **\*\*Watcher armed\*\*** (`b8nvbrd6r`, baseline #54) ? SHA change ? re-review the fix;  
`updatedAt`-only ? read the reply. I'll re-verify #54 the moment a fix lands.

### 618. user (2026-07-06T23:30:30.596Z)

```
<task-notification>
<task-id>b8nvbrd6r</task-id>
<tool-use-id>toolu_01F7hyxaBGcyyFzt3NtdbN7b</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users\avidu-Projects-badminton-
highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b8nvbrd6r.output</output-file>
<status>completed</status>
<summary>Background command "Arm watcher v2 (baseline #54)" completed (exit code 0)</summary>
</task-notification>
```

### 619. assistant (2026-07-06T23:30:33.864Z)

The watcher fired. Let me check what changed on #54.

### 620. user (2026-07-06T23:30:36.905Z)

```
=== PR-watch v2 ARMED ===
baseline: #54 draft=false sha=49f15a8a9 upd=2026-07-06T23:24:14Z
poll 1/30: no change (23:26:28Z)
poll 2/30: no change (23:28:29Z)
=== PR ACTIVITY DETECTED (poll 3, 23:30:29Z) ===
--- baseline ---
#54 draft=false sha=49f15a8a9 upd=2026-07-06T23:24:14Z
--- now ---
#54 draft=false sha=d2d44aef6 upd=2026-07-06T23:29:28Z
--- new/changed (sha change=force-push/fix; only upd change=comment) ---
#54 draft=false sha=d2d44aef6 upd=2026-07-06T23:29:28Z
--- open PR details ---
#54 draft=false by avidullu: feat(policy): add F5 deep inspection budget boundary
```

### 621. assistant (2026-07-06T23:30:40.994Z)

SHA changed (`49f15a8` ? `d2d44ae`) ? a **\*\*fix push\*\*** for #54. Let me fetch and see what the author changed.

### 622. user (2026-07-06T23:30:45.694Z)

```
=== fix diff (49f15a8..d2d44ae) ===
src/tg_video_filter/policy.py          | 79 ++++++-----
src/tg_video_filter/telethon_client.py |  2 +
tests/test_policy.py                   | 74 ++++++
3 files changed, 149 insertions(+), 6 deletions(-)
```

```

diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
index 879b28d..ab0c029 100644
--- a/src/tg_video_filter/policy.py
+++ b/src/tg_video_filter/policy.py
@@ -8,9 +8,11 @@ the first failure. Later Track F phases add concrete stages.
     from __future__ import annotations

     import asyncio
+import contextlib
     import json
     import logging
     import re
+from collections.abc import Awaitable
     from dataclasses import dataclass, field
     from ipaddress import ip_address
     from typing import Protocol
@@ -40,6 +42,15 @@ _URL_RE = re.compile(
     _TRAILING_URL_PUNCTUATION = ".!?:;"

+class PolicyBudgetExceeded(Exception):
+    """Raised when a deep-inspection budget is exhausted."""
+
+    def __init__(self, exceeded_budget: str, **metadata: object) -> None:
+        super().__init__(exceeded_budget)
+        self.exceeded_budget = exceeded_budget
+        self.metadata = metadata
+
+    @dataclass(frozen=True)
+    class PolicyBudget:
+        """Bounded execution budget for opt-in deep inspection."""
+
@@ -62,11 +73,47 @@ class PolicyBudget:
     """Run ``fetcher`` under the timeout and concurrency budget."""
     if self._semaphore is None:
         raise ValueError("max_concurrent must be positive")
-    async with self._semaphore:
-        return await asyncio.wait_for(
+
+    loop = asyncio.get_running_loop()
+    deadline = loop.time() + self.timeout_s
+    await self._acquire_before(deadline)
+    try:
+        remaining = deadline - loop.time()
+        if remaining <= 0:
+            raise PolicyBudgetExceeded("timeout_s", timeout_s=self.timeout_s)
+        return await _await_with_budget_timeout(
+            fetcher.inspect(content_item, self),
+            timeout=self.timeout_s,
+            timeout_s=remaining,
+            configured_timeout_s=self.timeout_s,
+        )
+    finally:
+        self._semaphore.release()
+
+    async def _acquire_before(self, deadline: float) -> None:
+        assert self._semaphore is not None
+        remaining = deadline - asyncio.get_running_loop().time()
+        if remaining <= 0:
+            raise PolicyBudgetExceeded("timeout_s", timeout_s=self.timeout_s)
+        try:
+            await asyncio.wait_for(self._semaphore.acquire(), timeout=remaining)
+        except TimeoutError as exc:
+            raise PolicyBudgetExceeded("timeout_s", timeout_s=self.timeout_s) from exc

```

```

+
+async def _await_with_budget_timeout(
+    awaitable: Awaitable[StageResult],
+    *,
+    timeout_s: float,
+    configured_timeout_s: float,
+) -> StageResult:
+    task = asyncio.ensure_future(awaitable)
+    done, _pending = await asyncio.wait({task}, timeout=timeout_s)
+    if task not in done:
+        task.cancel()
+        with contextlib.suppress(asyncio.CancelledError):
+            await task
+        raise PolicyBudgetExceeded("timeout_s", timeout_s=configured_timeout_s)
+    return await task

class ContentFetcher(Protocol):
@@ -707,13 +754,25 @@ class DeepInspectionStage:
    assert ctx.budget is not None
    try:
        return await ctx.budget.inspect(ctx.fetcher, ctx.content_item)
-    except TimeoutError:
+    except PolicyBudgetExceeded as exc:
        return StageResult.fail(
            self.stage_name,
            "deep inspection budget exceeded",
            budget_exceeded=True,
            exceeded_budget="timeout_s",
            timeout_s=ctx.budget.timeout_s,
            exceeded_budget=exc.exceeded_budget,
            **exc.metadata,
        )
+    except Exception as exc:
+        log.warning(
+            "deep inspection fetcher failed policy_id=%s",
+            ctx.policy.id,
+            exc_info=True,
+        )
+        return StageResult.fail(
+            self.stage_name,
+            "deep inspection fetcher failed",
+            fetcher_error_type=type(exc).__name__,
+            fetcher_error=str(exc),
+        )

@@ -742,6 +801,13 @@ def _deep_inspection_budget_result(ctx: PolicyContext) -> StageResult |
None:
    missing_size_bytes=True,
    max_bytes_per_item=budget.max_bytes_per_item,
)
+    if size_bytes <= 0:
+        return StageResult.fail(
+            DeepInspectionStage.stage_name,
+            "deep inspection budget cannot be enforced with non-positive content size",
+            invalid_size_bytes=size_bytes,
+            max_bytes_per_item=budget.max_bytes_per_item,
+        )
+    if size_bytes > budget.max_bytes_per_item:
+        return StageResult.fail(
+            DeepInspectionStage.stage_name,
@@ -777,6 +843,7 @@ __all__ = [
    "ExtractedUrl",
    "PolicyContext",

```

```

    "PolicyBudget",
+   "PolicyBudgetExceeded",
    "PolicyStage",
    "StageResult",
    "TopicPreferenceStage",
diff --git a/src/tg_video_filter/telethon_client.py b/src/tg_video_filter/telethon_client.py
index 2d5cbb1..c5f760f 100644
--- a/src/tg_video_filter/telethon_client.py
+++ b/src/tg_video_filter/telethon_client.py
@@ -438,6 +438,8 @@ async def _evaluate_policy_stages(
    policy = await db.get_default_policy(conn, account_id)
    if policy is None:
        return True, []
+   # DeepInspectionStage is dormant unless ``deep_inspect`` is enabled. Any
+   # user-facing enable command must land with fetcher/budget wiring.
    return await evaluate_pipeline(
        PolicyContext(content_item=content_item, source=source, policy=policy),
        [

```

### 623. assistant (2026-07-06T23:31:21.242Z)

The author fixed **both mediums + 3 of 4 lows** with a proper redesign: a dedicated `PolicyBudgetExceeded` exception (only raised on *genuine* budget expiry), an `except Exception` clause that turns fetcher errors into a fail-closed `StageResult`, deadline-bounded semaphore acquisition, and the `size_bytes <= 0` check + wiring-site comment. This is the most intricate fix in the series (manual acquire/release + task-cancel), so careful verification is warranted ? especially the permit-release under the new logic. Let me set up a worktree, gate, and verify comprehensively.

### 624. user (2026-07-06T23:31:28.046Z)

HEAD is now at d2d44ae feat(policy): add deep inspection budget boundary  
gate started

### 625. assistant (2026-07-06T23:31:52.195Z)

```

import asyncio
from tg_video_filter.models import ContentItem, Policy, Source
from tg_video_filter.policy import DeepInspectionStage, PolicyBudget, PolicyContext,
StageResult, evaluate_pipeline

def ctx(fetcher, budget, size=1000):
    return PolicyContext(
        content_item=ContentItem(account_id=1, source_id=1, provider_message_id="5",
size_bytes=size, dedup_key="k"),
        source=Source(id=1, account_id=1, provider="telegram", provider_source_id="-1"),
        policy=Policy(id=1, account_id=1, name="default", config_json='{"deep_inspect":true}',
is_default=True),
        fetcher=fetcher, budget=budget,
    )

class Boom:
    async def inspect(self, ci, b):
        raise RuntimeError("ffprobe exploded")

class InnerTimeout:
    async def inspect(self, ci, b):
        raise TimeoutError("socket timeout")

class Slow:

```

```

    async def inspect(self, ci, b):
        await asyncio.sleep(0.5)
        return StageResult.pass_("deep-inspection", "late")

class Fast:
    async def inspect(self, ci, b):
        await asyncio.sleep(0.001)
        return StageResult.pass_("deep-inspection", "ok")

async def main():
    stage = DeepInspectionStage()
    b = PolicyBudget(max_bytes_per_item=2000, timeout_s=1.0, max_concurrent=1)

    # Medium #1: non-timeout fetcher exception -> fail-closed StageResult, NOT uncaught
    try:
        p, res = await evaluate_pipeline(ctx(Boom(), b), [stage])
        r = res[-1]
        print("#1 fetcher RuntimeError -> passed=%s reason=%r type=%s (fail-closed, not
uncaught)" % (
            r.passed, r.reason, r.metadata.get("fetcher_error_type")))
    except Exception as e:
        print("#1 STILL UNCAUGHT: %s" % type(e).__name__)

    # Medium #2: inner TimeoutError -> 'fetcher failed', NOT 'budget exceeded'
    r2 = await stage.evaluate(ctx(InnerTimeout(), b))
    print("#2 inner TimeoutError -> reason=%r budget_exceeded=%s (must NOT be budget)" % (
        r2.reason, r2.metadata.get("budget_exceeded")))

    # Genuine budget timeout (slow fetcher, short budget) -> 'budget exceeded'
    r3 = await stage.evaluate(ctx(Slow(), PolicyBudget(max_bytes_per_item=2000, timeout_s=0.05,
max_concurrent=1)))
    print("genuine budget timeout -> reason=%r meta=%s" % (r3.reason, r3.metadata))

    # size<=0 -> fail
    r4 = await stage.evaluate(ctx(Fast(), b, size=0))
    print("size=0 -> passed=%s reason=%r" % (r4.passed, r4.reason))
    r4b = await stage.evaluate(ctx(Fast(), b, size=-5))
    print("size=-5 -> passed=%s reason=%r" % (r4b.passed, r4b.reason))

    # Permit-release stress: many budget-timeouts on max_concurrent=1, then fast call must pass
    bshort = PolicyBudget(max_bytes_per_item=2000, timeout_s=0.03, max_concurrent=1)
    for _ in range(5):
        rr = await stage.evaluate(ctx(Slow(), bshort))
        assert not rr.passed
    fast_ok = await asyncio.wait_for(stage.evaluate(ctx(Fast(), bshort)), timeout=2.0)
    print("after 5 budget-timeouts, fast call -> passed=%s (permit released, no leak)" %
fast_ok.passed)

    # Concurrency still limited with shared budget (2 permits, 5 tasks)
    class Tk:
        def __init__(s): s.active = 0; s.max = 0
        async def inspect(s, ci, bb):
            s.active += 1; s.max = max(s.max, s.active)
            try:
                await asyncio.sleep(0.02); return StageResult.pass_("deep-inspection", "ok")
            finally:
                s.active -= 1
    tk = Tk(); shared = PolicyBudget(max_bytes_per_item=2000, timeout_s=1.0, max_concurrent=2)
    await asyncio.gather(*[stage.evaluate(ctx(tk, shared)) for _ in range(5)])
    print("shared budget max_concurrent=2, 5 tasks -> max_active=%d (expect 2)" % tk.max)

    # Acquire-timeout permit-leak check: max_concurrent=1, hold it with a slow task while others
time out on ACQUIRE

```



```

True, 'exceeded_budget': 'timeout_s', 'timeout_s': 0.05}
size=0 -> passed=False reason='deep inspection budget cannot be enforced with non-positive
content size'
size=-5 -> passed=False reason='deep inspection budget cannot be enforced with non-positive
content size'
after 5 budget-timeouts, fast call -> passed=True (permit released, no leak)
shared budget max_concurrent=2, 5 tasks -> max_active=2 (expect 2)
6 contending on 1 permit (0.05s budget) -> passed=[False, False, False, False, False, False]
post-contention fast call -> passed=True (no acquire-leak deadlock)
rc=0 (124=hung=acquire-permit-leak)

=== gate ===
=== RUFF ===
All checks passed!
36 files already formatted
=== PYTEST ===
-----
TOTAL                                1649    71    476    48    94%
Required test coverage of 80.0% reached. Total coverage: 94.40%
546 passed in 9.42s
DONE

```

#### 628. assistant (2026-07-06T23:32:32.686Z)

All fixes **verified** (the tracebacks are the new ``exc_info`` warning logs, not crashes ? results below them confirm clean handling):

| Item   | Before                                                                                                                                                                                 | After                                                                                                                             |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Med #1 | fetcher <code>`RuntimeError`</code>   uncaught ? crashes handler   <code>`passed=False, reason="deep inspection fetcher failed"`</code> (fail-closed) ?                                | inner <code>`TimeoutError`</code>   mislabeled "budget exceeded"   <code>`reason="fetcher failed", budget_exceeded=None`</code> ? |
| Med #2 | Genuine budget timeout   ?   still <code>`"budget exceeded"`</code> ? (correctly distinguished)   <code>`size &lt;= 0`</code>   passed the check   fails "non-positive content size" ? | Semaphore acquire   unbounded   deadline-bounded ?                                                                                |

Critically, the new manual acquire/release logic is **leak-free**: after 5 budget-timeouts and a 6-way contention storm on a 1-permit budget, a subsequent call still passes (no deadlock); shared-budget concurrency still caps at ``max_concurrent=2``. Gate: ruff clean, **546 passed @ 94.40%** (+3 tests).

Both mediums + 3 lows resolved, new logic sound. Per your rule, I'm satisfied ? merge-ready. Posting the LGTM.

#### 629. user (2026-07-06T23:32:47.356Z)

```

=== post LGTM ===
https://github.com/avidullu/telegram-video-filter/pull/54#issuecomment-4898568147
=== cleanup ===
removed

```

#### 630. assistant (2026-07-06T23:32:51.499Z)

LGTM posted ? [issuecomment-4898568147](https://github.com/avidullu/telegram-video-filter/pull/54#issuecomment-4898568147). Worktree cleaned. Re-arming the watcher.

#### 631. user (2026-07-06T23:32:58.289Z)

Command running in background with ID: bht1do8aq. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bht1do8aq.output. You will be notified when it completes. To check interim output, use Read on that file path.

632. assistant (2026-07-06T23:33:09.172Z)

### PR #54 (F5 deep-inspection budget boundary) ? fix verified, \*\*LGTM'd\*\* ?

The watcher caught the fix-push (SHA `49f15a8` ? `d2d44ae`); I re-reviewed and signed off ([LGTM](https://github.com/avidullu/telegram-video-filter/pull/54#issuecomment-4898568147)).

The author resolved **both mediums + 3 lows** with a proper redesign ? a dedicated `PolicyBudgetExceeded`` (raised only on genuine budget expiry), an `except Exception`` fetcher-error clause (fail-closed), deadline-bounded semaphore acquisition, and the `size <= 0`` guard. I verified the intricate new async logic is **leak-free**: fetcher errors become clean fail-closed results, inner timeouts are no longer mislabeled as budget overruns, and after a 5-timeout + 6-way-contention storm on a 1-permit budget nothing deadlocks. Gate: 546 passed @ 94.40%, ruff clean.

### Status

- **Track F:** #40 · F1 #50 · F2 #51 · F3 #52 · F4 #53 merged ? · **F5 #54 LGTM'd** (awaiting merge).
- F2/F3/F4/F5 each had real findings the author fixed before I signed off ? and F5's two mediums came from the adversarial workflow, not my first pass, which is exactly why I run it.
- **Watcher armed** (`bht1do8aq``, baseline #54) ? wakes me on the #54 merge, the next phase (F6 policy-decisions table / F7 `/why`` / the `/set`` config commands), or any force-push/comment.

I'll surface the next activity when it lands and run the same verify-then-only-LGTM-when-satisfied cycle.

633. user (2026-07-06T23:35:01.420Z)

```
<task-notification>
<task-id>bht1do8aq</task-id>
<tool-use-id>toolu_01CqEYsVL25gA17kRJxVaeKq</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bht1do8aq.output</output-file>
<status>completed</status>
<summary>Background command "Arm watcher v2 (baseline fixed #54)" completed (exit code 0)</summary>
</task-notification>
```

634. assistant (2026-07-06T23:35:03.944Z)

The watcher fired. Let me check.

635. user (2026-07-06T23:35:05.901Z)

```
=== PR-watch v2 ARMED ===
baseline: #54 draft=false sha=d2d44aef6 upd=2026-07-06T23:32:46Z
=== PR ACTIVITY DETECTED (poll 1, 23:35:00Z) ===
--- baseline ---
#54 draft=false sha=d2d44aef6 upd=2026-07-06T23:32:46Z
--- now ---
<none>
--- new/changed (sha change=force-push/fix; only upd change=comment) ---

--- open PR details ---
```

636. assistant (2026-07-06T23:35:09.905Z)

Open set ? empty ? #54 merged after my LGTM. Let me confirm and check for the next one.

637. user (2026-07-06T23:35:16.845Z)

```
=== #54 state ===
```



```
MERGED da0ab4711
=== origin/main recent ===
da0ab47 feat(policy): add deep inspection budget boundary (#54)
a7cbbda feat(policy): add URL reputation stage (#53)
5cd65d1 feat(policy): add caption safety stage (#52)
=== open PRs now ===
(empty=none)
```

#### 638. assistant (2026-07-06T23:35:21.277Z)

**\*\*#54 (F5) merged\*\*** ? (`da0ab47`). No open PRs. Fast-forwarding `main` and re-arming.

#### 639. user (2026-07-06T23:35:27.796Z)

Command running in background with ID: b8ia33sc0. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b8ia33sc0.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 640. assistant (2026-07-06T23:35:37.642Z)

### Update

**\*\*PR #54 (F5 deep-inspection budget boundary) is merged\*\*** ? (`da0ab47`) ? merged right after my verified LGTM. No open PRs at the moment.

**\*\*Track F is now complete through F5 ? all reviewed, verified, merged:\*\***

| PR   | Phase                       | Findings ?                                        | outcome        |
|------|-----------------------------|---------------------------------------------------|----------------|
| ---- | -----                       | -----                                             |                |
| #40  | ADR 0012 design             | 10 gaps ?                                         | revised ? LGTM |
| #50  | F1 primitives + persistence | clean ?                                           | LGTM           |
| #51  | F2 topic preferences        | 3 mediums ?                                       | fixed ? LGTM   |
| #52  | F3 caption safety           | 3 mediums (ReDoS/fail-open/case) ?                | fixed ? LGTM   |
| #53  | F4 URL reputation           | 1 medium (`normalize_domain` throw) ?             | fixed ? LGTM   |
| #54  | F5 deep-inspection budget   | 2 mediums (fetcher-error leak/timeout misclass) ? | fixed ? LGTM   |

**\*\*State:\*\*** local `main` fast-forwarded to `da0ab47`, clean and synced. **\*\*Watcher re-armed\*\*** (`b8ia33sc0`, empty baseline).

What's likely still to come per ADR 0012: **\*\*F6\*\*** (policy-decisions table / audit trail), **\*\*F7\*\*** (`/why` explainability command), and the **\*\*`/set` config commands\*\*** that make F2?F5's dormant stages user-reachable (the phase where several of my "latent until a command lands" notes become live ? I'll be watching for those interactions closely).

I'll surface the next PR the moment it lands and run the same cycle. The setup is humming: watcher distinguishes fix-pushes from comments, every finding gets reproduced, and LGTM stays strictly gated on verified fixes.

#### 641. user (2026-07-06T23:45:35.430Z)

```
<task-notification>
<task-id>b8ia33sc0</task-id>
<tool-use-id>toolu_01XZwU9FNEN9EcWmyRuyS1NJ</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b8ia33sc0.output</output-file>
<status>completed</status>
<summary>Background command "Fast-forward main and re-arm watcher" completed (exit code 0)</summary>
</task-notification>
```

#### 642. assistant (2026-07-06T23:45:41.156Z)

The watcher fired. Let me check what it caught.

#### 643. user (2026-07-06T23:45:42.561Z)

```
=== PR-watch v2 RE-ARMED ===
baseline: <none>
poll 1/30: no change (23:37:31Z)
poll 2/30: no change (23:39:32Z)
poll 3/30: no change (23:41:33Z)
poll 4/30: no change (23:43:33Z)
=== PR ACTIVITY DETECTED (poll 5, 23:45:34Z) ===
--- baseline ---
<none>
--- now ---
#55 draft=false sha=42b563765 upd=2026-07-06T23:43:40Z
--- new/changed (sha change=force-push/fix; only upd change=comment) ---
#55 draft=false sha=42b563765 upd=2026-07-06T23:43:40Z
--- open PR details ---
#55 draft=false by avidullu: F6: persist policy decision traces
```

#### 644. assistant (2026-07-06T23:45:59.612Z)

**\*\*New PR #55 detected\*\*** ? "F6: persist policy decision traces" (the `policy_decisions` audit-trail table + persistence). This is a **\*\*new DB migration + write path\*\*** ? historically the highest-risk category in this repo (I found blockers in the Track B/C migrations). I'll scrutinize migration idempotency/schema, the persistence wiring (accepted *and* rejected items, FK validity, `metadata_json` serialization), and any TTL/dedup. Let me sync and start.

#### 645. user (2026-07-06T23:46:08.183Z)

```
* [new branch]      feat/f6-policy-decisions -> origin/feat/f6-policy-decisions
=== PR #55 META ===
{"additions":387,"body":"## Summary\n\n- Add the `policy_decisions` table plus idempotent migration for ADR 0012 audit traces.\n- Persist one decision row per content item/policy with final pass/fail and an ordered JSON stage trace.\n- Wire live ingestion and backfill to store accepted and rejected policy decisions after content insert; policy rejections remain separate from delivery attempts.\n- Add repository and helper tests for accepted traces, rejected traces, upsert behavior, and migration coverage.\n\nCloses #48\n\n## Validation\n\n- `\\\\.\\.\\.venv\\Scripts\\python.exe -m ruff check src tests`\n- `\\\\.\\.\\.venv\\Scripts\\python.exe -m pytest` (`551 passed`),\n  \"changedFiles\":7,\"deletions\":11,\"headRefName\":\"feat/f6-policy-decisions\", \"isDraft\":false,\"mergeStateStatus\":\"CLEAN\", \"mergeable\":\"MERGEABLE\", \"number\":55,\"title\":\"F6: persist policy decision traces\"}

=== files ===
src/tg_video_filter/backfill.py
src/tg_video_filter/db.py
src/tg_video_filter/models.py
src/tg_video_filter/policy.py
src/tg_video_filter/telethon_client.py
tests/test_db.py
tests/test_policy.py
=== HEAD/branch ===
feat/f6-policy-decisions
```

#### 646. assistant (2026-07-06T23:46:16.709Z)

PR #55 (F6): +387/?11, 7 files ? a migration + write path. It correctly separates policy decisions from `delivery_attempts` (addressing my Track C finding) and stores accepted *and* rejected. Let me set up the worktree, start the gate, and save the diff.

#### 647. user (2026-07-06T23:46:23.840Z)

```
(empty=current)
643 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr55.diff
```

HEAD is now at 42b5637 feat(policy): persist policy decision traces  
gate started

#### 648. user (2026-07-06T23:46:28.185Z)

```
1      diff --git a/src/tg_video_filter/backfill.py b/src/tg_video_filter/backfill.py
2      index ff5d8a3..624d332 100644
3      --- a/src/tg_video_filter/backfill.py
4      +++ b/src/tg_video_filter/backfill.py
5      @@ -38,6 +38,7 @@
6          _extract_document_id,
7          _message_text,
8          _peer_id,
9      +    _record_policy_decision,
10         extract_video_meta,
11     )
12
13     @@ -243,8 +244,11 @@ async def backfill_user( # pragma: no cover - requires live
Telethon connection
14         result = engine.evaluate(meta)
15         matched = result.matched
16         reasons = list(result.reasons)
17     +    policy_passed = True
18     +    policy_results = []
19     +    policy_id = None
20         if matched and source is not None:
21     -        policy_passed, policy_results = await _evaluate_policy_stages(
22     +        policy_passed, policy_results, policy_id = await
_evaluate_policy_stages(
23             conn,
24             user.id,
25             source,
26     @@ -284,6 +288,13 @@ async def backfill_user( # pragma: no cover - requires live
Telethon connection
27         matched,
28         reasons,
29     )
30     +    await _record_policy_decision(
31     +        conn,
32     +        inserted=inserted,
33     +        policy_id=policy_id,
34     +        policy_passed=policy_passed,
35     +        policy_results=policy_results,
36     +    )
37
38         if matched:
39             summary["videos_matched"] += 1
40     diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
41     index e6fca83..c76747a 100644
42     --- a/src/tg_video_filter/db.py
43     +++ b/src/tg_video_filter/db.py
44     @@ -27,6 +27,7 @@
45         Destination,
46         FilterConfig,
47         Policy,
48     +    PolicyDecision,
49         RoutingRule,
50         Source,
51         TelegramSession,
52     @@ -181,6 +182,16 @@
53         next_retry_at    TEXT,
54         UNIQUE (content_item_id, destination_id)
55     );
56     +
57     +CREATE TABLE IF NOT EXISTS policy_decisions (
```

```

58 + id INTEGER PRIMARY KEY AUTOINCREMENT,
59 + content_item_id INTEGER NOT NULL REFERENCES content_items(id),
60 + policy_id INTEGER NOT NULL REFERENCES policies(id),
61 + passed INTEGER NOT NULL,
62 + trace_json TEXT NOT NULL,
63 + evaluated_at TEXT NOT NULL,
64 + UNIQUE (content_item_id, policy_id)
65 +);
66 +"""
67
68 # Note: the ``idx_videos_user_document`` partial unique index is created by
69 @@ -764,6 +775,24 @@ async def _migration_004_destinations_and_routing(conn:
aiosqlite.Connection) ->
70 )
71
72
73 +@_migration(5)
74 +async def _migration_005_policy_decisions(conn: aiosqlite.Connection) -> None:
75 + """Create ADR 0012 policy decision audit table."""
76 + await conn.execute(
77 + """
78 + CREATE TABLE IF NOT EXISTS policy_decisions (
79 + id INTEGER PRIMARY KEY AUTOINCREMENT,
80 + content_item_id INTEGER NOT NULL REFERENCES content_items(id),
81 + policy_id INTEGER NOT NULL REFERENCES policies(id),
82 + passed INTEGER NOT NULL,
83 + trace_json TEXT NOT NULL,
84 + evaluated_at TEXT NOT NULL,
85 + UNIQUE (content_item_id, policy_id)
86 + )
87 + """
88 + )
89 +
90 +
91 + async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
92 + """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
93 + async with conn.execute(
94 @@ -893,6 +922,20 @@ def _row_to_policy(row: aiosqlite.Row) -> Policy:
95 )
96
97
98 +def _row_to_policy_decision(row: aiosqlite.Row) -> PolicyDecision:
99 + trace = json.loads(row["trace_json"])
100 + if not isinstance(trace, list):
101 + raise ValueError("policy_decisions.trace_json must decode to a JSON array")
102 + return PolicyDecision(
103 + id=row["id"],
104 + content_item_id=row["content_item_id"],
105 + policy_id=row["policy_id"],
106 + passed=bool(row["passed"]),
107 + trace=trace,
108 + evaluated_at=row["evaluated_at"],
109 + )
110 +
111 +
112 +def _row_to_destination(row: aiosqlite.Row) -> Destination:
113 + return Destination(
114 + id=row["id"],
115 @@ -1342,6 +1385,80 @@ async def save_policy_config(
116 )
117
118
119 +# --- PolicyDecision repos (ADR 0012, Track F) -----
120 +
121 +

```

```

122 +def _policy_trace_json(trace: list[dict[str, object]]) -> str:
123 +     """Encode a policy stage trace deterministically."""
124 +     return json.dumps(trace, separators=(",", ":"), sort_keys=True)
125 +
126 +
127 +async def record_policy_decision(
128 +     conn: aiosqlite.Connection,
129 +     *,
130 +     content_item_id: int,
131 +     policy_id: int,
132 +     passed: bool,
133 +     trace: list[dict[str, object]],
134 +     evaluated_at: datetime | None = None,
135 +) -> PolicyDecision:
136 +     """Persist or replace the policy evaluation trace for a content item."""
137 +     evaluated_at = evaluated_at or _now()
138 +     trace_json = _policy_trace_json(trace)
139 +     await conn.execute(
140 +         """INSERT INTO policy_decisions
141 +             (content_item_id, policy_id, passed, trace_json, evaluated_at)
142 +             VALUES (?, ?, ?, ?, ?)
143 +             ON CONFLICT(content_item_id, policy_id) DO UPDATE SET
144 +                 passed=excluded.passed,
145 +                 trace_json=excluded.trace_json,
146 +                 evaluated_at=excluded.evaluated_at""",
147 +         (
148 +             content_item_id,
149 +             policy_id,
150 +             int(passed),
151 +             trace_json,
152 +             evaluated_at.isoformat(),
153 +         ),
154 +     )
155 +     await conn.commit()
156 +     decision = await get_policy_decision(conn, content_item_id, policy_id)
157 +     assert decision is not None
158 +     return decision
159 +
160 +
161 +async def get_policy_decision(
162 +     conn: aiosqlite.Connection,
163 +     content_item_id: int,
164 +     policy_id: int,
165 +) -> PolicyDecision | None:
166 +     """Read a policy decision by its natural key."""
167 +     conn.row_factory = aiosqlite.Row
168 +     async with conn.execute(
169 +         """SELECT * FROM policy_decisions
170 +             WHERE content_item_id = ? AND policy_id = ?""",
171 +         (content_item_id, policy_id),
172 +     ) as cur:
173 +         row = await cur.fetchone()
174 +         return _row_to_policy_decision(row) if row else None
175 +
176 +
177 +async def list_policy_decisions_for_content(
178 +     conn: aiosqlite.Connection,
179 +     content_item_id: int,
180 +) -> list[PolicyDecision]:
181 +     """Return all policy decisions recorded for a content item."""
182 +     conn.row_factory = aiosqlite.Row
183 +     async with conn.execute(
184 +         """SELECT * FROM policy_decisions
185 +             WHERE content_item_id = ?
186 +             ORDER BY evaluated_at, id""",

```

```

187 +         (content_item_id,),
188 +     ) as cur:
189 +         rows = await cur.fetchall()
190 +         return [_row_to_policy_decision(row) for row in rows]
191 +
192 +
193 # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --
194
195
196 @@ -1860,6 +1977,9 @@ async def list_videos_for_user( # v2: used by on-demand /search;
not called in
197     "get_default_policy",
198     "load_policy_config",
199     "save_policy_config",
200 +     "record_policy_decision",
201 +     "get_policy_decision",
202 +     "list_policy_decisions_for_content",
203     # Destination / Routing / Delivery repos (ADR 0011, Track C)
204     "upsert_destination",
205     "get_destination",
206 diff --git a/src/tg_video_filter/models.py b/src/tg_video_filter/models.py
207 index 70e2a8a..fd4c85e 100644
208 --- a/src/tg_video_filter/models.py
209 +++ b/src/tg_video_filter/models.py
210 @@ -296,6 +296,22 @@ class DeliveryAttempt(BaseModel):
211     next_retry_at: datetime | None = None
212
213
214 +class PolicyDecision(BaseModel):
215 +     """Persisted ADR 0012 policy evaluation trace for one content item."""
216 +
217 +     model_config = ConfigDict(frozen=True)
218 +
219 +     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
220 +     content_item_id: int = Field(description="FK -> content_items.id")
221 +     policy_id: int = Field(description="FK -> policies.id")
222 +     passed: bool
223 +     trace: list[dict[str, object]] = Field(
224 +         default_factory=list,
225 +         description="Ordered StageResult trace serialized in
policy_decisions.trace_json",
226 +     )
227 +     evaluated_at: datetime = Field(default_factory=_utcnow)
228 +
229 +
230     class FilterConfig(BaseModel):
231         """Per-user filter configuration.
232
233 diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
234 index ab0c029..b9ec862 100644
235 --- a/src/tg_video_filter/policy.py
236 +++ b/src/tg_video_filter/policy.py
237 @@ -834,6 +834,19 @@ async def evaluate_pipeline(
238     return True, results
239
240
241 +def stage_results_to_trace(results: list[StageResult]) -> list[dict[str, object]]:
242 +     """Convert in-memory stage results to a JSON-serializable trace."""
243 +     return [
244 +         {
245 +             "stage_name": result.stage_name,
246 +             "passed": result.passed,
247 +             "reason": result.reason,
248 +             "metadata": result.metadata,
249 +         }

```

```

250 +         for result in results
251 +     ]
252 +
253 +
254 +     __all__ = [
255 +         "CaptionSafetyConfig",
256 +         "CaptionSafetyStage",
257 +         @@ -859,4 +872,5 @@ async def evaluate_pipeline(
258 +             "parse_caption_safety_config",
259 +             "parse_domain_reputation_config",
260 +             "parse_topic_preferences",
261 +             +         "stage_results_to_trace",
262 +         ]
263 +     diff --git a/src/tg_video_filter/telethon_client.py
b/src/tg_video_filter/telethon_client.py
264 +     index c5f760f..37089ae 100644
265 +     --- a/src/tg_video_filter/telethon_client.py
266 +     +++ b/src/tg_video_filter/telethon_client.py
267 +     @@ -52,6 +52,7 @@
268 +         TopicPreferenceStage,
269 +         UrlReputationStage,
270 +         evaluate_pipeline,
271 +         +     stage_results_to_trace,
272 +     )
273 +
274 +     log = logging.getLogger(__name__)
275 +     @@ -279,8 +280,11 @@ async def on_new_message(event: NewMessage.Event) -> None: #
pragma: no cover
276 +         result = engine_holder[ENGINE_KEY].evaluate(meta)
277 +         matched = result.matched
278 +         reasons = list(result.reasons)
279 +         policy_passed = True
280 +         policy_results: list[StageResult] = []
281 +         policy_id: int | None = None
282 +         if matched and source is not None:
283 +             -         policy_passed, policy_results = await _evaluate_policy_stages(
284 +             +         policy_passed, policy_results, policy_id = await _evaluate_policy_stages(
285 +                 conn,
286 +                 user.id,
287 +                 source,
288 +             @@ -335,6 +339,13 @@ async def on_new_message(event: NewMessage.Event) -> None: #
pragma: no cover
289 +                 matched,
290 +                 reasons,
291 +             )
292 +             +         await _record_policy_decision(
293 +             +             conn,
294 +             +             inserted=inserted,
295 +             +             policy_id=policy_id,
296 +             +             policy_passed=policy_passed,
297 +             +             policy_results=policy_results,
298 +             +         )
299 +
300 +             if matched:
301 +                 # Build link text for LINK/BOTH modes (None for FORWARD).
302 +             @@ -433,14 +444,14 @@ async def _evaluate_policy_stages(
303 +                 account_id: int,
304 +                 source: Source,
305 +                 content_item: ContentItem,
306 +             -) -> tuple[bool, list[StageResult]]:
307 +             +) -> tuple[bool, list[StageResult], int | None]:
308 +                 """Evaluate ADR 0012 policy stages for a candidate item."""
309 +                 policy = await db.get_default_policy(conn, account_id)
310 +                 if policy is None:
311 +                     -         return True, []

```

```

312 +         return True, [], None
313         # DeepInspectionStage is dormant unless ``deep_inspect`` is enabled. Any
314         # user-facing enable command must land with fetcher/budget wiring.
315     - return await evaluate_pipeline(
316 +     passed, results = await evaluate_pipeline(
317         PolicyContext(content_item=content_item, source=source, policy=policy),
318         [
319             TopicPreferenceStage(),
320 @@ -449,6 +460,27 @@ async def _evaluate_policy_stages(
321             DeepInspectionStage(),
322         ],
323     )
324 +     return passed, results, policy.id
325 +
326 +
327 +async def _record_policy_decision(
328 +    conn: aiomysqlite.Connection,
329 +    *,
330 +    inserted: VideoRecord,
331 +    policy_id: int | None,
332 +    policy_passed: bool,
333 +    policy_results: list[StageResult],
334 +) -> None:
335 +    """Persist a policy trace after the content item has been inserted."""
336 +    if not policy_results or policy_id is None or inserted.id is None:
337 +        return
338 +    await db.record_policy_decision(
339 +        conn,
340 +        content_item_id=inserted.id,
341 +        policy_id=policy_id,
342 +        passed=policy_passed,
343 +        trace=stage_results_to_trace(policy_results),
344 +    )
345
346
347     def _peer_id(peer: object) -> int:
348 @@ -485,5 +517,6 @@ async def __call__(self, event: NewMessage.Event) -> None: ...
349         "make_new_message_handler",
350         "_content_item_preview",
351         "_evaluate_policy_stages",
352 +     + "_record_policy_decision",
353         "_message_text",
354     ]
355 diff --git a/tests/test_db.py b/tests/test_db.py
356 index 54e14bb..7b357ea 100644
357 --- a/tests/test_db.py
358 +++ b/tests/test_db.py
359 @@ -65,6 +65,7 @@ async def test_init_db_creates_tables(tmp_path: object) -> None:
360         "destinations",
361         "routing_rules",
362         "delivery_attempts",
363 +     + "policy_decisions",
364     }
365
366
367 @@ -694,6 +695,14 @@ async def test_fresh_db_user_version_is_latest(conn:
aiosqlite.Connection) -> No
368     assert version >= 1
369
370
371 +async def test_migration_005_policy_decisions_is_idempotent(
372 +    conn: aiomysqlite.Connection,
373 +) -> None:
374 +    await db._migration_005_policy_decisions(conn)
375 +    await db._migration_005_policy_decisions(conn)

```



```

376 +     assert await db._table_exists(conn, "policy_decisions")
377 +
378 +
379 + async def test_migration_adds_document_id_to_legacy_db(tmp_path: object) -> None:
380 +     """A v1 DB (no document_id, user_version=0) must be migrated on open."""
381 +     db_path = str(tmp_path) + "/legacy.db" # type: ignore[operator]
382 +     @@ -1205,6 +1214,124 @@ async def test_policy_upsert_and_default_lookup(conn:
aiosqlite.Connection) -> N
383 +         assert fetched.is_default is True
384 +
385 +
386 + async def _seed_policy_decision_subject(
387 +     conn: aiosqlite.Connection,
388 + ) -> tuple[ContentItem, Policy]:
389 +     await _seed_user(conn)
390 +     src = await db.upsert_source(
391 +         conn, Source(account_id=USER_ID, provider="telegram", provider_source_id="-1")
392 +     )
393 +     assert src.id is not None
394 +     item = await db.insert_content_item(
395 +         conn,
396 +         ContentItem(
397 +             account_id=USER_ID,
398 +             source_id=src.id,
399 +             provider="telegram",
400 +             provider_message_id="42",
401 +             dedup_key="tg:msg:1:42",
402 +             matched=False,
403 +         ),
404 +     )
405 +     policy = await db.upsert_policy(
406 +         conn,
407 +         Policy(
408 +             account_id=USER_ID,
409 +             name="default",
410 +             config_json="{}",
411 +             is_default=True,
412 +         ),
413 +     )
414 +     assert item.id is not None
415 +     assert policy.id is not None
416 +     return item, policy
417 +
418 +
419 + async def test_record_policy_decision_accepted_trace(conn: aiosqlite.Connection) ->
None:
420 +     item, policy = await _seed_policy_decision_subject(conn)
421 +     assert item.id is not None
422 +     assert policy.id is not None
423 +     trace = [
424 +         {
425 +             "stage_name": "topic-preferences",
426 +             "passed": True,
427 +             "reason": "topic preferences disabled",
428 +             "metadata": {},
429 +         }
430 +     ]
431 +
432 +     decision = await db.record_policy_decision(
433 +         conn,
434 +         content_item_id=item.id,
435 +         policy_id=policy.id,
436 +         passed=True,
437 +         trace=trace,
438 +     )

```

```

439 +
440 +     assert decision.content_item_id == item.id
441 +     assert decision.policy_id == policy.id
442 +     assert decision.passed is True
443 +     assert decision.trace == trace
444 +     fetched = await db.get_policy_decision(conn, item.id, policy.id)
445 +     assert fetched == decision
446 +
447 +
448 + async def test_record_policy_decision_rejected_without_delivery_attempt(
449 +     conn: aiosqlite.Connection,
450 + ) -> None:
451 +     item, policy = await _seed_policy_decision_subject(conn)
452 +     assert item.id is not None
453 +     assert policy.id is not None
454 +     trace = [
455 +         {
456 +             "stage_name": "url-reputation",
457 +             "passed": False,
458 +             "reason": "blocked domain matched: bad.example",
459 +             "metadata": {"blocked_domain": "bad.example"},
460 +         }
461 +     ]
462 +
463 +     decision = await db.record_policy_decision(
464 +         conn,
465 +         content_item_id=item.id,
466 +         policy_id=policy.id,
467 +         passed=False,
468 +         trace=trace,
469 +     )
470 +
471 +     assert decision.passed is False
472 +     assert decision.trace == trace
473 +     async with conn.execute("SELECT COUNT(*) FROM delivery_attempts") as cur:
474 +         assert (await cur.fetchone())[0] == 0
475 +
476 +
477 + async def test_record_policy_decision_upserts_trace(conn: aiosqlite.Connection) ->
None:
478 +     item, policy = await _seed_policy_decision_subject(conn)
479 +     assert item.id is not None
480 +     assert policy.id is not None
481 +
482 +     await db.record_policy_decision(
483 +         conn,
484 +         content_item_id=item.id,
485 +         policy_id=policy.id,
486 +         passed=False,
487 +         trace=[{"stage_name": "topic", "passed": False, "reason": "old", "metadata":
{}}],
488 +     )
489 +     updated = await db.record_policy_decision(
490 +         conn,
491 +         content_item_id=item.id,
492 +         policy_id=policy.id,
493 +         passed=True,
494 +         trace=[{"stage_name": "topic", "passed": True, "reason": "new", "metadata":
{}}],
495 +     )
496 +     listed = await db.list_policy_decisions_for_content(conn, item.id)
497 +
498 +     assert updated.passed is True
499 +     assert updated.trace[0]["reason"] == "new"
500 +     assert len(listed) == 1

```

```

501     +     assert listed[0] == updated
502     +
503     +
504     async def test_filter_config_shim_uses_default_policy(conn: aisqlite.Connection) ->
None:
505         """save_filter_config ? load_filter_config round-trips through policies."""
506         await _seed_user(conn)
507     diff --git a/tests/test_policy.py b/tests/test_policy.py
508     index 8b4a60c..3dfd222 100644
509     --- a/tests/test_policy.py
510     +++ b/tests/test_policy.py
511     @@ -7,7 +7,7 @@
512     import aisqlite
513
514     from tg_video_filter import db
515     -from tg_video_filter.models import ContentItem, Policy, Source
516     +from tg_video_filter.models import ContentItem, Policy, Source, VideoRecord
517     from tg_video_filter.policy import (
518         CaptionSafetyConfig,
519         CaptionSafetyStage,
520     @@ -27,7 +27,7 @@
521         parse_domain_reputation_config,
522         parse_topic_preferences,
523     )
524     -from tg_video_filter.telethon_client import _evaluate_policy_stages
525     +from tg_video_filter.telethon_client import _evaluate_policy_stages,
    _record_policy_decision
526
527
528     def _ctx(
529     @@ -817,7 +817,7 @@ async def test_evaluate_policy_stages_uses_default_policy_config()
-> None:
530         dedup_key="tg:msg:1:42",
531     )
532
533     -     policy_passed, results = await _evaluate_policy_stages(conn, 111, src, item)
534     +     policy_passed, results, policy_id = await _evaluate_policy_stages(conn, 111,
src, item)
535
536         assert [result.stage_name for result in results] == [
537             "topic-preferences",
538     @@ -826,10 +826,11 @@ async def test_evaluate_policy_stages_uses_default_policy_config()
-> None:
539             "deep-inspection",
540         ]
541         assert policy_passed is True
542     +     assert policy_id is not None
543         assert [result.passed for result in results] == [True, True, True, True]
544         assert results[0].metadata == {"matched_include_topics": ["AI"]}
545
546     -     topic_passed, topic_rejected = await _evaluate_policy_stages(
547     +     topic_passed, topic_rejected, topic_policy_id = await _evaluate_policy_stages(
548         conn,
549         111,
550         src,
551     @@ -837,11 +838,12 @@ async def test_evaluate_policy_stages_uses_default_policy_config()
-> None:
552         )
553
554         assert topic_passed is False
555     +     assert topic_policy_id == policy_id
556         assert [result.stage_name for result in topic_rejected] == ["topic-
preferences"]
557         assert topic_rejected[0].passed is False
558         assert topic_rejected[0].metadata == {"matched_exclude_topics": ["Crypto"]}

```

```

559
560 -         url_passed, url_rejected = await _evaluate_policy_stages(
561 +         url_passed, url_rejected, url_policy_id = await _evaluate_policy_stages(
562             conn,
563             111,
564             src,
565     @@ -849,13 +851,14 @@ async def test_evaluate_policy_stages_uses_default_policy_config()
-> None:
566         )
567
568         assert url_passed is False
569 +         assert url_policy_id == policy_id
570         assert [result.stage_name for result in url_rejected] == [
571             "topic-preferences",
572             "url-reputation",
573         ]
574         assert url_rejected[-1].metadata["matched_blocklist_domain"] == "bad.example"
575
576 -         caption_passed, caption_rejected = await _evaluate_policy_stages(
577 +         caption_passed, caption_rejected, caption_policy_id = await
578 _evaluate_policy_stages(
579             conn,
580             111,
581             src,
582     @@ -863,6 +866,7 @@ async def test_evaluate_policy_stages_uses_default_policy_config()
-> None:
583         )
584
585         assert caption_passed is False
586 +         assert caption_policy_id == policy_id
587         assert [result.stage_name for result in caption_rejected] == [
588             "topic-preferences",
589             "url-reputation",
590     @@ -878,3 +882,54 @@ async def test_evaluate_policy_stages_uses_default_policy_config()
-> None:
591         }
592         finally:
593             await conn.close()
594
595 +async def test_record_policy_decision_helper_persists_stage_trace() -> None:
596 +    conn = await aioredis.connect(":memory:")
597 +    try:
598 +        await db._apply_schema(conn)
599 +        await db.upsert_user(conn, db.User(id=111, phone="+15555550100")) # type:
600 ignore[attr-defined]
601 +        policy = await db.upsert_policy(
602 +            conn,
603 +            Policy(account_id=111, name="default", config_json={}, is_default=True),
604 +        )
605 +        assert policy.id is not None
606 +        inserted = await db.insert_video(
607 +            conn,
608 +            VideoRecord(
609 +                user_id=111,
610 +                channel_id=-1001,
611 +                message_id=42,
612 +                matched=False,
613 +            ),
614 +        )
615 +        assert inserted.id is not None
616 +        results = [
617 +            StageResult.fail(
618 +                "url-reputation",
619 +                "blocked domain matched: bad.example",

```

```

619 +         blocked_domain="bad.example",
620 +     )
621 + ]
622 +
623 +     await _record_policy_decision(
624 +         conn,
625 +         inserted=inserted,
626 +         policy_id=policy.id,
627 +         policy_passed=False,
628 +         policy_results=results,
629 +     )
630 +
631 +     decision = await db.get_policy_decision(conn, inserted.id, policy.id)
632 +     assert decision is not None
633 +     assert decision.passed is False
634 +     assert decision.trace == [
635 +         {
636 +             "stage_name": "url-reputation",
637 +             "passed": False,
638 +             "reason": "blocked domain matched: bad.example",
639 +             "metadata": {"blocked_domain": "bad.example"},
640 +         }
641 +     ]
642 +     finally:
643 +         await conn.close()
644

```

#### 649. assistant (2026-07-06T23:47:29.103Z)

I've read the full F6 diff ? it's clean and addresses several prior concerns (policy decisions are stored **\*\*separately\*\*** from `delivery\_attempts` ? my Track C finding; the wiring helper is directly tested). The `policy\_decisions` table uses one-row-per-decision with a JSON trace (a reasonable deviation from the ADR's per-stage rows), migration 005 is in both `\_SCHEMA` and the ladder, and the write is FK-valid (after insert, idempotent upsert). Let me check the gate and verify the trace serialization + legacy-DB migration.

#### 650. user (2026-07-06T23:47:50.343Z)

```

=== gate ===
=== RUFF ===
All checks passed!
36 files already formatted
=== PYTEST ===
-----
TOTAL                                1692    73    480    50    94%
Required test coverage of 80.0% reached. Total coverage: 94.34%
551 passed in 9.69s
DONE

=== verify: full pipeline trace serializes + FK-valid persist + legacy-DB migration to v5 ===
1 full trace persisted: passed=False, stages=['topic-preferences', 'url-reputation']
  after re-record: rows=1 (expect 1, upsert), passed=True
2 legacy DB migrated: user_version=5 (expect 5) policy_decisions_exists=True

```

#### 651. assistant (2026-07-06T23:48:06.513Z)

All verified: full-pipeline trace **\*\*serializes + persists\*\***, **\*\*idempotent upsert\*\*** (1 row on re-record), and a **\*\*legacy v1 DB migrates cleanly to `user\_version=5`\*\*** with `policy\_decisions` created (idempotent). Gate: ruff clean, **\*\*551 passed @ 94.34%\*\***. This is clean ? findings so far are low/notes. Given migrations are the highest-risk category here, let me run a focused adversarial pass on the migration + persistence.

#### 652. user (2026-07-06T23:48:46.028Z)

Workflow launched in background. Task ID: w2d4z0pnx  
Summary: Review telegram-video-filter PR #55 (F6 policy\_decisions table + persistence) ? migration & write-path correctness, with adversarial verification  
Transcript dir: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_a6bedd49-683  
Script file: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr55-f6-wf\_a6bedd49-683.js  
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\claude\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr55-f6-wf\_a6bedd49-683.js"} to iterate without resending the script.)  
Run ID: wf\_a6bedd49-683  
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\claude\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr55-f6-wf\_a6bedd49-683.js", resumeFromRunId: "wf\_a6bedd49-683"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

### 653. assistant (2026-07-06T23:48:52.301Z)

While the workflow runs, let me verify the serialization-crash concern (following F5's fetcher-error lesson): does a non-JSON-serializable `StageResult.metadata` value crash `record\_policy\_decision` uncaught?

### 654. user (2026-07-06T23:49:02.597Z)

non-serializable metadata -> RAISES TypeError: Object of type object is not JSON serializable (would crash handler)

### 655. assistant (2026-07-06T23:49:31.224Z)

**\*\*Confirmed\*\*** ? a non-JSON-serializable `StageResult.metadata` value makes `record\_policy\_decision` raise `TypeError` (from `json.dumps`), which would propagate uncaught ? crash the handler. But it's **\*\*latent\*\***: all shipped F1?F5 stages produce JSON-safe metadata; only a future `ContentFetcher` returning weird metadata could trigger it. So it's a low (defensive-hardening suggestion, mirroring F5's fetcher-error fail-safe), not a current bug.

My pass finds only low/notes so far (serialization-hardening, no TTL, Tier-1-rejections-not-recorded, schema deviation) ? no current bugs; migration verified correct on legacy+fresh. But given the workflow caught F5's two mediums that I'd missed, I'll **\*\*wait for its adversarial pass before deciding\*\*** on the LGTM, per your rule. Awaiting it now.

### 656. user (2026-07-06T23:58:03.844Z)

```
<task-notification>
<task-id>w2d4z0pnx</task-id>
<tool-use-id>toolu_01EJy3PFS4h2hLBdqMAZ11M5</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\w2d4z0pnx.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Review telegram-video-filter PR #55 (F6 policy_decisions table + persistence) ? migration & write-path correctness, with adversarial verification" completed</summary>
<result>{"counts":{"raw":3,"kept":3,"refuted":0},"findings":[{"dimension":"persist_wiring","severity":"low","verdict":"PLAUSIBLE","title":"Non-serializable stage metadata crashes ingestion via uncaught json.dumps TypeError on the policy-decision write path","where":"src/tg_video_filter/db.py:1391","summary":"record_policy_decision -&gt; _policy_trace_json does json.dumps(trace) with no error handling; any StageResult whose metadata contains a non-JSON-serializable value raises TypeError that propagates uncaught out of _record_policy_decision and out of both call sites (on_new_message and the backfill message loop have no try/except around it), crashing per-message ingestion / aborting the whole backfill."},"failure_scenario":"A future ContentFetcher (F5) or budget path returns/raises
```

metadata carrying a non-JSON value (e.g. a datetime, set, bytes, or arbitrary object) -- the most concrete exposed path is DeepInspectionStage at policy.py:763 which splats `\*\*exc.metadata` from a caller-supplied PolicyBudgetExceeded straight into the StageResult. stage\_results\_to\_trace copies result.metadata verbatim into the trace, then record\_policy\_decision calls json.dumps and raises `TypeError: Object of type X is not JSON serializable`. In on\_new\_message (telethon\_client.py:342) there is no surrounding try/except, so the handler dies; in backfill the try at backfill.py:147 only catches FloodWaitError, so the TypeError bubbles to the outer `except Exception` (backfill.py:464) and aborts the entire user backfill. Reproduced: stage\_results\_to\_trace([StageResult.pass\_('s','ok',obj=object())]) then db.record\_policy\_decision(...) raises TypeError uncaught. NOTE: not triggerable by current F1-F5 stages -- every present metadata builder emits only str/int/bool/list-of-str (topic.py list(...), url\_reputation\_url\_reputation\_metadata, caption\_caption\_safety\_metadata), so this is latent/defensive, same class as the F5 fetcher-error lesson.", "evidence": "db.py:1387-1390 `\_policy\_trace\_json` = json.dumps(trace, separators, sort\_keys) with no try/except; policy.py:838-845 stage\_results\_to\_trace copies result.metadata unchanged; policy.py:757-764 DeepInspectionStage splats untrusted `\*\*exc.metadata`; telethon\_client.py:342-348 and 464-475 call the helper with no try/except; backfill.py:147 try only pairs with `except FloodWaitError` at :327. Repro output: `record\_policy\_decision` raised `TypeError: Object of type object is not JSON serializable`.", "recommendation": "Make the trace write defensive so an audit-log serialization failure can never crash the ingestion/backfill main path: either wrap the json.dumps in `\_policy\_trace\_json` with a fallback (json.dumps(..., default=str)) or wrap the db.record\_policy\_decision call inside `\_record\_policy\_decision` in try/except that logs and swallows (best-effort audit, matching the F5 fetcher-error handling), so a bad-metadata stage degrades to a missing/repr'd trace row rather than a dropped message or aborted backfill.", "verify\_reasoning": "The serialization mechanism is real and reproducible. db.py:1391-1393 `\_policy\_trace\_json` calls `json.dumps(trace, ...)` with no error handling; policy.py:837-847 `stage\_results\_to\_trace` copies `result.metadata` verbatim; both call sites (telethon\_client.py:342-348 in on\_new\_message, backfill.py:291-297) invoke `\_record\_policy\_decision` with no try/except, and backfill's only inner catch is `except FloodWaitError` (327) so a TypeError bubbles to the outer `except Exception` (464) and aborts the whole user backfill. Repro (PYTHONPATH=../src) confirms: a StageResult with metadata `obj=object()` makes `db.\_policy\_trace\_json(trace)` raise `TypeError: Object of type object is not JSON serializable`, uncaught. However, the described WRONG BEHAVIOR (crashing ingestion / aborting backfill) CANNOT occur with this PR's reachable code, and the finding's own NOTE concedes this. Verified: (1) every present `PolicyBudgetExceeded` raise (policy.py:83,96,100,115) carries only `timeout\_s=<float>` ? JSON-serializable. (2) The only untrusted-splat path, `\*\*exc.metadata` at policy.py:763, is inside DeepInspectionStage's `ctx.budget.inspect(...)` branch, which is unreachable from the wired call site: `\_evaluate\_policy\_stages` (telethon\_client.py:454-462) builds `PolicyContext(content\_item, source, policy)` with NO fetcher/budget, so DeepInspectionStage.evaluate returns early via `\_deep\_inspection\_budget\_result` (missing/invalid budget) or `ctx.fetcher is None` (752) before line 756 ever runs. (3) All other current stage metadata builders emit only str/int/bool/list-of-str. So no non-serializable value can reach `json.dumps` today; the crash is purely latent/defensive, triggerable only if future F5 fetcher/budget wiring lands with a caller-supplied non-JSON metadata value. Net: mechanism CONFIRMED, but it is a defensive-hardening gap on a newly added write path, not a defect that produces wrong behavior in this PR's live paths ? hence PLAUSIBLE rather than CONFIRMED. The recommendation (json.dumps(..., default=str) fallback, or wrap the record\_policy\_decision call in try/except to make the audit write best-effort, matching the F5 fetcher-error lesson) is sound and worth doing before fetcher wiring lands.", {"dimension": "persist\_wiring", "severity": "low", "verdict": "CONFIRMED", "title": "Metadata-rejected (Tier-1 FilterEngine) items persist no policy\_decision row", "where": "src/tg\_video\_filter/telethon\_client.py:464", "summary": "When FilterEngine returns matched=False, policy stages are skipped so policy\_results is empty and the guard `if not policy\_results ...` return records nothing; these rejections leave no audit trace, so a future F7 /why cannot explain metadata-based rejections (only Tier 2-4 policy rejections are explainable).", "failure\_scenario": "A video is rejected by a Tier-1 metadata filter (duration/size/resolution/content-type). matched=False, so `if matched and source is not None` (telethon\_client.py:286) is skipped, policy\_results stays []; `\_record\_policy\_decision`'s guard early-returns; no policy\_decisions row exists. A later /why query for that item finds no persisted decision and cannot report why it was filtered.", "evidence": "telethon\_client.py:283-303 (policy stages only run when matched); telethon\_client.py:472-473 guard `if not policy\_results or policy\_id is None or inserted.id is None: return`. Confirmed as a known deviation in the PR description; recording it as an explicit F7 explainability gap.", "recommendation": "Deviation/growth note only -- acceptable for F6 scope. Flag for F7: metadata-rejection reasons live only in content\_items/logs, not in

policy\_decisions, so /why must fall back to FilterEngine reasons for Tier-1 rejects.", "verify\_reasoning": "The finding accurately describes real code behavior that ships in this PR's diff. Verified at telethon\_client.py:286 ? policy stages only run inside `if matched and source is not None`, so when the Tier-1 FilterEngine returns matched=False the block is skipped and `policy\_results` stays `[]` (initialized line 284). The new `\_record\_policy\_decision` helper (added in this diff, lines 466-483) guards with `if not policy\_results or policy\_id is None or inserted.id is None: return` (line 475), so an empty `policy\_results` causes an early return and no policy\_decisions row is written. The same wiring is added in on\_new\_message (lines 342-348, per the diff) and backfill. Thus metadata-rejected items get a content\_items row with matched=False and logged reasons (lines 334-341) but no policy\_decisions audit row ? exactly as stated. This is an observability/explainability gap, not a fault: no crash, no FK violation, no data corruption, no wrong data persisted. The finding itself frames it as a deferred F7 note (`"acceptable for F6 scope"`), and the PR description already discloses it as a known deviation. Verdict CONFIRMED because the described behavior genuinely ships in this diff and is real; severity is correctly low since it is a deviation/growth note (per rubric: ... (truncated 2598 chars, full result in C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\w2d4z0pnx.output)</result><usage><agent\_count>5</agent\_count><subagent\_tokens>258423</subagent\_tokens><tool\_uses>76</tool\_uses><duration\_ms>557725</duration\_ms></usage></task-notification>

#### 657. assistant (2026-07-06T23:58:20.968Z)

The workflow found \*\*3 findings, all LOW\*\* (0 refuted) ? corroborating my read (serialization-hardening is even downgraded to PLAUSIBLE/latent since it can't trigger with current stages). No mediums, no blockers. Let me grab the third finding, then decide.

#### 658. user (2026-07-06T23:58:25.850Z)

COUNTS: {'raw': 3, 'kept': 3, 'refuted': 0}

[LOW/PLAUSIBLE] (persist\_wiring) Non-serializable stage metadata crashes ingestion via uncaught json.dumps TypeError on the policy-decision write path

where: src/tg\_video\_filter/db.py:1391

rec: Make the trace write defensive so an audit-log serialization failure can never crash the ingestion/backfill main path: either wrap the json.

[LOW/CONFIRMED] (persist\_wiring) Metadata-rejected (Tier-1 FilterEngine) items persist no policy\_decision row

where: src/tg\_video\_filter/telethon\_client.py:464

rec: Deviation/growth note only -- acceptable for F6 scope. Flag for F7: metadata-rejection reasons live only in content\_items/logs, not in polic

[LOW/CONFIRMED] (persist\_wiring) policy\_decisions has no TTL/cleanup; table grows unbounded

where: src/tg\_video\_filter/db.py:185

rec: Growth note only. Track a retention/cleanup job (or size cap) as follow-up; not blocking for F6.

refuted: []

#### 659. assistant (2026-07-06T23:58:52.334Z)

All 3 findings are \*\*LOW\*\* (matching my own pass) ? no current bugs. The migration is verified correct (legacy+fresh, idempotent, `user\_version=5`, FK-valid), persistence is correct (idempotent upsert, separate from `delivery\_attempts`, comprehensively tested), and the only "crash" path (non-serializable metadata) is \*\*unreachable today\*\* (every shipped stage emits JSON-safe metadata). Unlike F2?F5, F6 has no real mediums ? just latent-defensive + disclosed-deferral notes.

Per your rule, I'm satisfied this is merge-ready. Posting the LGTM with the low items as non-blocking recommendations.



#### 660. user (2026-07-06T23:59:12.866Z)

```
=== post LGTM ===
https://github.com/avidullu/telegram-video-filter/pull/55#issuecomment-4898711436
=== cleanup ===
removed
```

#### 661. assistant (2026-07-06T23:59:16.952Z)

LGTM posted ? [issuecomment-4898711436](https://github.com/avidullu/telegram-video-filter/pull/55#issuecomment-4898711436). Worktree cleaned. Re-arming the watcher.

#### 662. user (2026-07-06T23:59:23.867Z)

Command running in background with ID: b37pyc2it. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b37pyc2it.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 663. assistant (2026-07-06T23:59:36.387Z)

### PR #55 (F6 policy-decision persistence) ? reviewed, **\*\*LGTM'd\*\*** ?

[LGTM posted](https://github.com/avidullu/telegram-video-filter/pull/55#issuecomment-4898711436). This one cleared the bar on the first pass ? no fix round needed.

**\*\*Verified:\*\*** migration 005 correct (legacy v1 DB ? `user\_version=5`, idempotent, FK-valid on both fresh + migrated paths); persistence idempotent-upsert and FK-valid (written after insert); tz-aware timestamps; and it stores decisions **\*\*separately from `delivery\_attempts`\*\*** ? directly fixing the "wrong home for decisions" point from my Track C review. The wiring helper is unit-tested, not just the pragma-no-cover handler. 551 passed @ 94.34%.

**\*\*3 low, non-blocking\*\*** (my pass + the workflow agreed): harden the trace `json.dumps` before the F5 fetcher lands (a future fetcher's non-serializable metadata would crash ingestion ? same class as F5's lesson, but unreachable today); no TTL/cleanup (ADR open question); Tier-1 metadata rejections aren't recorded (an F7 `why` note). None are current bugs ? so per your rule, merge-ready.

### Status

- **\*\*Track F:\*\*** #40 · F1 #50 · F2 #51 · F3 #52 · F4 #53 · F5 #54 merged ? · **\*\*F6 #55 LGTM'd\*\*** (awaiting merge).
- Interesting contrast: F6 is the first F-phase with *\*no\** mediums ? a correct, well-tested migration where the only notes are defensive/deferred. The author's quality has been trending up across the series.
- **\*\*Watcher armed\*\*** (`b37pyc2it`, baseline #55) ? likely next is **\*\*F7\*\*** (`why` explainability ? where my F6 Tier-1 note becomes relevant) or the **\*\*`set` config commands\*\*** (where the F2?F5 dormant stages, and several of my "latent until a command lands" findings, go live). I'll be watching those interactions closely.

I'll surface the next activity when it lands.

#### 664. user (2026-07-07T00:01:26.730Z)

```
<task-notification>
<task-id>b37pyc2it</task-id>
<tool-use-id>toolu_01FMgJE2QCLRxT5JFAVCo46L</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b37pyc2it.output</output-file>
<status>completed</status>
<summary>Background command "Arm watcher v2 (baseline #55)" completed (exit code 0)</summary>
</task-notification>
```

#### 665. assistant (2026-07-07T00:01:29.389Z)

The watcher fired. Let me check what changed.

#### 666. user (2026-07-07T00:01:31.437Z)

```
=== PR-watch v2 ARMED ===
baseline: #55 draft=false sha=42b563765 upd=2026-07-06T23:59:12Z
=== PR ACTIVITY DETECTED (poll 1, 00:01:25Z) ===
--- baseline ---
#55 draft=false sha=42b563765 upd=2026-07-06T23:59:12Z
--- now ---
#55 draft=false sha=6d004ba38 upd=2026-07-07T00:00:57Z
--- new/changed (sha change=force-push/fix; only upd change=comment) ---
#55 draft=false sha=6d004ba38 upd=2026-07-07T00:00:57Z
--- open PR details ---
#55 draft=false by avidullu: F6: persist policy decision traces
```

#### 667. assistant (2026-07-07T00:01:36.639Z)

SHA changed on #55 (`42b5637` ? `6d004ba`) ? a force-push \*after\* my LGTM, likely applying one of my low recommendations. Let me see what changed and re-confirm.

#### 668. user (2026-07-07T00:01:41.238Z)

```
=== diff since my LGTM (42b5637..6d004ba) ===
src/tg_video_filter/db.py | 2 +-
tests/test_db.py          | 25 ++++++
2 files changed, 26 insertions(+), 1 deletion(-)

diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
index c76747a..6cdd65f 100644
--- a/src/tg_video_filter/db.py
+++ b/src/tg_video_filter/db.py
@@ -1390,7 +1390,7 @@ async def save_policy_config(

def _policy_trace_json(trace: list[dict[str, object]]) -> str:
    """Encode a policy stage trace deterministically."""
-    return json.dumps(trace, separators=(",", ":"), sort_keys=True)
+    return json.dumps(trace, default=str, separators=(",", ":"), sort_keys=True)

async def record_policy_decision(
diff --git a/tests/test_db.py b/tests/test_db.py
index 7b357ea..30ed312 100644
--- a/tests/test_db.py
+++ b/tests/test_db.py
@@ -1305,6 +1305,31 @@ async def test_record_policy_decision_rejected_without_delivery_attempt(
    assert (await cur.fetchone())[0] == 0

+async def test_record_policy_decision_coerces_non_json_trace_metadata(
+    conn: aiosqlite.Connection,
+) -> None:
+    item, policy = await _seed_policy_decision_subject(conn)
+    assert item.id is not None
+    assert policy.id is not None
+
+    decision = await db.record_policy_decision(
+        conn,
+        content_item_id=item.id,
+        policy_id=policy.id,
+        passed=True,
+        trace=[
+            {
```

```

+         "stage_name": "deep-inspection",
+         "passed": True,
+         "reason": "inspection returned metadata",
+         "metadata": {"raw_payload": b"\x00\x01"},
+     }
+ ],
+ )
+
+ assert decision.trace[0]["metadata"] == {"raw_payload": "b'\\x00\\x01'"}
+
+
+ async def test_record_policy_decision_upserts_trace(conn: aiosqlite.Connection) -> None:
+     item, policy = await _seed_policy_decision_subject(conn)
+     assert item.id is not None

```

#### 669. assistant (2026-07-07T00:01:52.443Z)

The author proactively applied my **\*\*#1 recommendation\*\*** (even post-LGTM) ? added ``default=str`` to ``_policy_trace_json`'s `json.dumps`` so non-serializable metadata coerces to its string repr instead of crashing, plus a ``bytes`-metadata` test. Let me verify it (re-run my crash repro ? should now coerce) and confirm the gate.

#### 670. user (2026-07-07T00:02:15.143Z)

```

HEAD is now at 6d004ba feat(policy): persist policy decision traces
=== ruff + full suite ===
All checks passed!
36 files already formatted
TOTAL                                1692      73    480      50    94%
Required test coverage of 80.0% reached. Total coverage: 94.34%
552 passed in 9.70s
=== re-run the crash repro against the hardened code (must now COERCE, not crash) ===
non-serializable metadata -> COERCED + persisted (no crash). meta= {'blob': 'str', 'probe':
'str', 'tags': 'str'}

```

#### 671. assistant (2026-07-07T00:02:26.790Z)

Verified: the hardening works ? previously-crashing metadata (``object``, ``set``, ``bytes``) now **\*\*coerces to string reprs and persists, no crash\*\***. Gate green: ruff clean, **\*\*552 passed @ 94.34%\*\*** (+1 test). The author addressed my top recommendation proactively. Let me post a brief confirmation (closing the loop on my LGTM), clean up, and re-arm.

#### 672. user (2026-07-07T00:02:37.503Z)

```

=== confirm comment ===
https://github.com/avidullu/telegram-video-filter/pull/55#issuecomment-4898729190
=== cleanup ===
removed
=== re-arm watcher (baseline #55 hardened) ===
baseline: #55 draft=false sha=6d004ba38 upd=2026-07-07T00:02:36Z

```

#### 673. assistant (2026-07-07T00:02:42.200Z)

Confirmation posted ? [issuecomment-4898729190](https://github.com/avidullu/telegram-video-filter/pull/55#issuecomment-4898729190). Worktree cleaned. Arming the watcher (baseline = hardened #55).

#### 674. user (2026-07-07T00:02:48.595Z)

Command running in background with ID: b8kxdn3r1. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b8kxdn3r1.output. You will be notified when it completes. To check interim output, use Read on that file path.

675. assistant (2026-07-07T00:02:57.856Z)

## PR #55 (F6) ? hardening applied & verified, **\*\*LGTM stands\*\*** ?

Nice follow-through: after my LGTM, the author pushed the **\*\*serialization-hardening\*\*** I'd recommended for "before the fetcher lands" ? ``_policy_trace_json`` now uses ``json.dumps(..., default=str)``, plus a ``bytes`-metadata` test. I re-verified: previously-crashing metadata (``object`/`set`/`bytes``) now coerces to string reprs and persists cleanly. [Confirmation posted](https://github.com/avidullu/telegram-video-filter/pull/55#issuecomment-4898729190). 552 passed @ 94.34%, ruff clean.

That closes my one actionable F6 note; the remaining items (TTL, Tier-1 rejection recording) are genuine deferrals to later phases.

## Status

- **\*\*Track F:\*\*** #40 · F1 #50 · F2 #51 · F3 #52 · F4 #53 · F5 #54 merged ? · **\*\*F6 #55 LGTM'd + hardened\*\*** (awaiting merge).
- The watcher's SHA-tracking has now proven itself twice on post-LGTM pushes (#53 canned-comment vs real-fix; #55 proactive hardening) ? exactly the force-push visibility you asked for.
- **\*\*Watcher armed\*\*** (``b8kxdn3r1``, baseline hardened #55) ? next likely **\*\*F7 `/why`\*\*** or the **\*\*`/set` config commands\*\*** (where the dormant F2?F5 stages and my "latent until a command lands" notes go live).

I'll surface the next activity when it lands.

676. user (2026-07-07T00:06:52.297Z)

```
<task-notification>
<task-id>b8kxdn3r1</task-id>
<tool-use-id>toolu_018MT8fYi61Ykjq3XUTpmvSY</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b8kxdn3r1.output</output-file>
<status>completed</status>
<summary>Background command "Arm watcher v2 (baseline hardened #55)" completed (exit code 0)</summary>
</task-notification>
```

677. assistant (2026-07-07T00:06:55.072Z)

The watcher fired. Let me check.

678. user (2026-07-07T00:06:57.159Z)

```
=== PR-watch v2 ARMED ===
baseline: #55 draft=false sha=6d004ba38 upd=2026-07-07T00:02:36Z
poll 1/30: no change (00:04:50Z)
=== PR ACTIVITY DETECTED (poll 2, 00:06:51Z) ===
--- baseline ---
#55 draft=false sha=6d004ba38 upd=2026-07-07T00:02:36Z
--- now ---
<none>
--- new/changed (sha change=force-push/fix; only upd change=comment) ---

--- open PR details ---
```

679. assistant (2026-07-07T00:07:00.714Z)

Open set ? empty ? #55 merged. Let me confirm, fast-forward ``main``, and re-arm.

680. user (2026-07-07T00:07:09.005Z)

Command running in background with ID: blz9rvsun. Output is being written to:

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\blz9rvsun.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 681. assistant (2026-07-07T00:07:28.515Z)

#### Update ? F6 merged; watching for F7

**\*\*PR #55 (F6 policy-decision persistence) merged\*\*** ? (the open set went empty right after my confirmation). Local `main` fast-forwarded, watcher re-armed (`blz9rvsun`).

**\*\*Track F scoreboard ? all reviewed, verified, merged:\*\***

| PR  | Phase                       | Findings ?  | outcome                  |
|-----|-----------------------------|-------------|--------------------------|
| #40 | ADR 0012 design             | 10 gaps ?   | revised ? LGTM           |
| #50 | F1 primitives + persistence | clean ?     | LGTM                     |
| #51 | F2 topic preferences        | 3 mediums ? | fixed ? LGTM             |
| #52 | F3 caption safety           | 3 mediums ? | fixed ? LGTM             |
| #53 | F4 URL reputation           | 1 medium ?  | fixed ? LGTM             |
| #54 | F5 deep-inspection budget   | 2 mediums ? | fixed ? LGTM             |
| #55 | F6 policy decisions         | 3 lows ?    | hardening applied ? LGTM |

That's the full ADR 0012 pipeline (F1?F6) plus the design doc ? every finding reproduced locally, LGTM strictly gated on verification, and the author fixing each round in minutes.

**\*\*Still likely to come:\*\*** **\*\*F7\*\*** (`/why` explainability ? where my F6 "Tier-1 rejections aren't recorded" note becomes directly relevant), and the **\*\*`/set` config commands\*\*** that flip the dormant F2?F5 stages live (where a cluster of my "latent until a command lands" findings ? malformed-config postures, the deep-inspect drop-all footgun ? become user-reachable, so I'll scrutinize those command?stage interactions hard).

Watcher's armed and distinguishing fix-pushes from comments. I'll surface the next PR the moment it lands and run the same cycle.

#### 682. user (2026-07-07T00:13:19.050Z)

```
<task-notification>
<task-id>blz9rvsun</task-id>
<tool-use-id>toolu_01YUuBwnBwcOWYkg7x3mUjT4</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\blz9rvsun.output</output-file>
<status>completed</status>
<summary>Background command "Confirm #55 merge, ff main, re-arm watcher" completed (exit code 0)</summary>
</task-notification>
```

#### 683. assistant (2026-07-07T00:13:24.882Z)

The watcher fired. Let me check what it caught.

#### 684. user (2026-07-07T00:13:27.039Z)

```
local main: 6149c38 feat(policy): persist policy decision traces (#55)
open PRs: []
=== PR-watch v2 RE-ARMED ===
baseline: <none>
poll 1/30: no change (00:09:16Z)
poll 2/30: no change (00:11:17Z)
=== PR ACTIVITY DETECTED (poll 3, 00:13:18Z) ===
--- baseline ---
<none>
--- now ---
#57 draft=false sha=12b4442e5 upd=2026-07-07T00:13:02Z
--- new/changed (sha change=force-push/fix; only upd change=comment) ---
```

```
#57 draft=false sha=12b4442e5 upd=2026-07-07T00:13:02Z
--- open PR details ---
#57 draft=false by avidullu: F7: add /why explainability command
```

#### 685. assistant (2026-07-07T00:13:39.021Z)

Confirmed #55 merged. **\*\*New PR #57 detected\*\*** ? "F7: add /why explainability command" (the final F-phase). This is a **\*\*command\*\*** reading F6's `policy\_decisions` by `content\_item\_id` ? so my top concern is **\*\*authorization/IDOR\*\***: does `/why <id>` scope the lookup to the requesting account, or can a user read another account's traces? Plus the F6 "metadata-rejections have no trace" note. Let me sync and start.

#### 686. user (2026-07-07T00:13:48.588Z)

```
* [new branch]      feat/f7-why-command -> origin/feat/f7-why-command
=== PR #57 META ===
{"additions":295,"body":"## Summary\n\n- Add `/why <content_item_id>` to the provider-independent `CommandService` and advertise it in `/show commands`. \n- Render persisted ADR 0012 policy decisions stage by stage, including final pass/fail, policy id, evaluated timestamp, reason text, and trace metadata. \n- Add an account-scoped content-item lookup so users can only inspect their own content. \n- Add a clear fallback for items without `policy_decisions` rows: recorded item status plus current metadata-filter replay with a config-change caveat. \n\nCloses #49\n\n## Validation\n\n- `\\\\.\\.\\.venv\\Scripts\\python.exe -m ruff check src tests`\n- `\\\\.\\.\\.venv\\Scripts\\python.exe -m pytest` (`560 passed`)", "changedFiles":5, "deletions":2, "headRefName":"feat/f7-why-command", "isDraft":false, "mergeable":"MERGEABLE", "number":57, "title":"F7: add /why explainability command"}
```

```
=== files ===
src/tg_video_filter/command_service.py
src/tg_video_filter/commands.py
src/tg_video_filter/db.py
tests/test_command_service.py
tests/test_db.py
(freshness; ambiguous=branch name differs)
feat/f7-why-command
```

#### 687. assistant (2026-07-07T00:14:00.383Z)

PR #57 (F7): +295/?2, 5 files. Good signs in the body ? it claims an **\*\*account-scoped content-item lookup\*\*** (addressing the IDOR I was worried about) and a **\*\*fallback\*\*** for items without `policy\_decisions` (addressing my F6 note). Per the gate, I verify the scoping is actually correct. Let me set up the worktree, gate, and read the diff.

#### 688. user (2026-07-07T00:14:07.443Z)

```
(empty=current)
425 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr57.diff
HEAD is now at 12b4442 feat(commands): add why explainability command
gate started
```

#### 689. user (2026-07-07T00:14:11.606Z)

```
1      diff --git a/src/tg_video_filter/command_service.py
b/src/tg_video_filter/command_service.py
2      index 59b7d81..0e3a390 100644
3      --- a/src/tg_video_filter/command_service.py
4      +++ b/src/tg_video_filter/command_service.py
5      @@ -11,6 +11,7 @@
6
7      from __future__ import annotations
8
```

```

9     +import json
10     import logging
11     import re
12     from dataclasses import dataclass
13     @@ -18,7 +19,8 @@
14     import aiosqlite
15
16     from tg_video_filter import db
17     -from tg_video_filter.models import FilterConfig, Source
18     +from tg_video_filter.filters import FilterEngine, VideoMeta
19     +from tg_video_filter.models import ContentItem, FilterConfig, PolicyDecision, Source
20
21     log = logging.getLogger(__name__)
22
23     @@ -33,6 +35,7 @@
24     _MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$",
re.IGNORECASE)
25     _SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
26     _SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
27     +_WHY_RE = re.compile(r"^/why(?:\s+(.))*?$", re.IGNORECASE)
28     _EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$",
re.IGNORECASE)
29     _EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
30     _MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
31     @@ -160,12 +163,97 @@ def format_commands() -> str:
32         "/unwatch <id|@name>` ? Stop watching a source",
33         "/watchlist` ? Show only watched sources",
34         "",
35         +        "? **Explainability**",
36         +        "/why <content_item_id>` ? Explain why an item passed or failed",
37         +        "",
38         +        "\u2139\u2013 **Help**",
39         +        "/show commands` ? Show this help message",
40     ]
41     )
42
43
44     +def _format_json_value(value: object, max_len: int = 500) -> str:
45     +    text = json.dumps(value, default=str, ensure_ascii=False, sort_keys=True)
46     +    if len(text) <= max_len:
47     +        return text
48     +    return f"{text[: max_len - 1]}?"
49     +
50     +
51     +def _content_item_to_video_meta(item: ContentItem) -> VideoMeta:
52     +    """Build a metadata replay input from fields persisted on content_items."""
53     +    return VideoMeta(
54     +        duration_s=item.duration_s,
55     +        width=item.width,
56     +        height=item.height,
57     +        size_bytes=item.size_bytes,
58     +        mime_type=item.mime_type,
59     +    )
60     +
61     +
62     +def _format_policy_decisions(item: ContentItem, decisions: list[PolicyDecision]) ->
str:
63     +    status = "PASSED" if decisions[-1].passed else "REJECTED"
64     +    lines = [
65     +        f"? **Why content item `{item.id}`?",
66     +        "",
67     +        f"Final policy result: **{status}**",
68     +        f"Provider message: `{item.provider}:{item.provider_message_id}`",
69     +    ]
70     +    if item.link_url:

```

```

71 +         lines.append(f"Link: {item.link_url}")
72 +     if item.text:
73 +         lines.append(f"Text: {_format_json_value(item.text, max_len=160)}")
74 +
75 +     for decision in decisions:
76 +         decision_status = "PASSED" if decision.passed else "REJECTED"
77 +         lines.extend(
78 +             [
79 +                 "",
80 +                 f"Policy `{decision.policy_id}`: **{decision_status}**",
81 +                 f"Evaluated: `{decision.evaluated_at.isoformat()}`",
82 +             ]
83 +         )
84 +         if not decision.trace:
85 +             lines.append("- No stage trace recorded.")
86 +             continue
87 +         for stage in decision.trace:
88 +             stage_name = str(stage.get("stage_name", "unknown"))
89 +             stage_passed = bool(stage.get("passed"))
90 +             reason = str(stage.get("reason", "no reason recorded"))
91 +             icon = "?" if stage_passed else ""
92 +             lines.append(f"- {icon} `{stage_name}` ? {reason}")
93 +             metadata = stage.get("metadata")
94 +             if isinstance(metadata, dict) and metadata:
95 +                 lines.append(f"  metadata: {_format_json_value(metadata)}")
96 +     return "\n".join(lines)
97 +
98 +
99 +def _format_missing_policy_decision(item: ContentItem, current_cfg: FilterConfig) ->
str:
100 +     recorded_status = "accepted" if item.matched else "rejected"
101 +     replay = FilterEngine(current_cfg).evaluate(_content_item_to_video_meta(item))
102 +     replay_status = "accepted" if replay.matched else "rejected"
103 +     lines = [
104 +         f"? **Why content item `{item.id}`?***",
105 +         "",
106 +         "No policy-stage decision trace was recorded for this item.",
107 +         f"Recorded item status: **{recorded_status}**.",
108 +         "",
109 +         "Current metadata-filter replay:",
110 +         f"- Result with current config: **{replay_status}**",
111 +     ]
112 +     if replay.reasons:
113 +         lines.extend(f"- {reason}" for reason in replay.reasons)
114 +     else:
115 +         lines.append("- No metadata filters are currently configured.")
116 +     lines.extend(
117 +         [
118 +             "",
119 +             "This fallback uses the current filter config and persisted metadata only;
120 +             "historical config changes can make it differ from the original decision.",
121 +         ]
122 +     )
123 +     return "\n".join(lines)
124 +
125 +
126 + # --- command result -----
127 +
128 +
129 +@@ -335,6 +423,11 @@ async def handle(
130 +     if _WATCHLIST_RE.match(text):
131 +         return await self._handle_watchlist(conn, user_id)
132 +
133 + # --- /why <content_item_id> -----

```



```

134 +         m = _WHY_RE.match(text)
135 +         if m:
136 +             return await self._handle_why(conn, user_id, m.group(1), current_cfg)
137 +
138 +         # --- /show commands -----
139 +         if _SHOW_COMMANDS_RE.match(text):
140 +             return CommandResult(format_commands())
141 @@ -405,6 +498,30 @@ async def _handle_watchlist(self, conn: aiomysqlite.Connection,
user_id: int) -> C
142         lines.append(f"? {s.provider_source_id} ? {label}")
143         return CommandResult("\n".join(lines))
144
145 +     async def _handle_why(
146 +         self,
147 +         conn: aiomysqlite.Connection,
148 +         user_id: int,
149 +         item_id_text: str | None,
150 +         current_cfg: FilterConfig,
151 +     ) -> CommandResult:
152 +         """Explain a recorded content item's policy decision."""
153 +         if item_id_text is None or not item_id_text.strip().isdecimal():
154 +             return CommandResult("? Usage: `/why <content_item_id>`")
155 +         item_id = int(item_id_text.strip())
156 +         if item_id < 1:
157 +             return CommandResult("? Usage: `/why <content_item_id>`")
158 +
159 +         item = await db.get_content_item_for_account(conn, user_id, item_id)
160 +         if item is None:
161 +             return CommandResult(f"? Content item `{item_id}` was not found for your
account.")
162 +
163 +         assert item.id is not None
164 +         decisions = await db.list_policy_decisions_for_content(conn, item.id)
165 +         if decisions:
166 +             return CommandResult(_format_policy_decisions(item, decisions))
167 +         return CommandResult(_format_missing_policy_decision(item, current_cfg))
168 +
169 +     async def _resolve_source(
170 +         self, conn: aiomysqlite.Connection, user_id: int, target: str
171 +     ) -> Source | None:
172 @@ -482,6 +599,7 @@ async def _update(
173     "ENGINE_KEY",
174     "CommandResult",
175     "CommandService",
176 +     "_WHY_RE",
177     "_format_duration",
178     "_format_size",
179     "format_config",
180 diff --git a/src/tg_video_filter/commands.py b/src/tg_video_filter/commands.py
181 index 3aea1f6..06db189 100644
182 --- a/src/tg_video_filter/commands.py
183 +++ b/src/tg_video_filter/commands.py
184 @@ -41,6 +41,7 @@
185     _UNWATCH_RE,
186     _WATCH_RE,
187     _WATCHLIST_RE,
188 +     _WHY_RE,
189     _format_duration,
190     _format_size,
191     format_commands,
192 @@ -142,6 +143,7 @@ async def _reply(client: TelegramClient, to_message: object, text:
str) -> None:
193     "_EXCLUDE_HELP_RE",
194     "_MIN_RES_HELP_RE",
195     "_SIZE_RE",

```

```

196 +     "_WHY_RE",
197     "_format_duration",
198     "_format_size",
199     "_update_config",
200 diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
201 index 6cdd65f..0d1069d 100644
202 --- a/src/tg_video_filter/db.py
203 +++ b/src/tg_video_filter/db.py
204 @@ -1285,6 +1285,21 @@ async def get_content_item_by_dedup_key(
205     return _row_to_content_item(row) if row else None
206
207
208 +async def get_content_item_for_account(
209 +    conn: aiosqlite.Connection,
210 +    account_id: int,
211 +    content_item_id: int,
212 +) -> ContentItem | None:
213 +    """Read a content item by id, scoped to an account for user-facing lookups."""
214 +    conn.row_factory = aiosqlite.Row
215 +    async with conn.execute(
216 +        "SELECT * FROM content_items WHERE id = ? AND account_id = ?",
217 +        (content_item_id, account_id),
218 +    ) as cur:
219 +        row = await cur.fetchone()
220 +        return _row_to_content_item(row) if row else None
221 +
222 +
223     # --- Policy repos (ADR 0011, Option 2A) -----
224
225
226     @@ -1973,6 +1988,7 @@ async def list_videos_for_user( # v2: used by on-demand /search;
not called in
227         "insert_content_item",
228         "get_content_item_by_message",
229         "get_content_item_by_dedup_key",
230     +     "get_content_item_for_account",
231         "upsert_policy",
232         "get_default_policy",
233         "load_policy_config",
234 diff --git a/tests/test_command_service.py b/tests/test_command_service.py
235 index 320f8a5..11a2be8 100644
236 --- a/tests/test_command_service.py
237 +++ b/tests/test_command_service.py
238 @@ -17,7 +17,7 @@
239     parse_duration_s,
240     parse_size_mb,
241 )
242 -from tg_video_filter.models import FilterConfig, Source
243 +from tg_video_filter.models import ContentItem, FilterConfig, Policy, Source
244
245     USER_ID = 111
246
247     @@ -237,6 +237,7 @@ def test_format_commands_includes_all_sections() -> None:
248         assert "/channels" in text
249         assert "/watch " in text
250         assert "/watchlist" in text
251     +     assert "/why" in text
252
253
254     # --- should_ingest_source (telethon_client gate, extracted for testability) ---
255     @@ -376,3 +377,146 @@ async def test_list_account_sources_watched_only(
256         watched = await db.list_account_sources(conn, USER_ID, watched_only=True)
257         assert len(watched) == 1
258         assert watched[0].provider_source_id == "-10"
259     +

```

```

260 +
261 +# --- /why explainability (ADR 0012, Track F7) -----
262 +
263 +
264 +async def _seed_content_item(
265 +     conn: aiomysqlite.Connection,
266 +     *,
267 +     account_id: int = USER_ID,
268 +     provider_message_id: str = "42",
269 +     matched: bool = False,
270 +     height: int | None = 720,
271 + ) -> ContentItem:
272 +     src = await _seed_source(conn, account_id, f"-100{provider_message_id}", "Trace
Source")
273 +     assert src.id is not None
274 +     item = await db.insert_content_item(
275 +         conn,
276 +         ContentItem(
277 +             account_id=account_id,
278 +             source_id=src.id,
279 +             provider="telegram",
280 +             provider_message_id=provider_message_id,
281 +             duration_s=120,
282 +             height=height,
283 +             size_bytes=10 * 1024 * 1024,
284 +             dedup_key=f"tg:msg:{account_id}:{provider_message_id}",
285 +             matched=matched,
286 +         ),
287 +     )
288 +     assert item.id is not None
289 +     return item
290 +
291 +
292 +async def _seed_policy(conn: aiomysqlite.Connection, account_id: int = USER_ID) ->
Policy:
293 +     policy = await db.upsert_policy(
294 +         conn,
295 +         Policy(
296 +             account_id=account_id,
297 +             name="default",
298 +             config_json="{}",
299 +             is_default=True,
300 +         ),
301 +     )
302 +     assert policy.id is not None
303 +     return policy
304 +
305 +
306 +async def test_why_renders_policy_decision_trace(
307 +     conn: aiomysqlite.Connection, svc: CommandService
308 + ) -> None:
309 +     item = await _seed_content_item(conn, matched=False)
310 +     policy = await _seed_policy(conn)
311 +     assert item.id is not None
312 +     assert policy.id is not None
313 +     await db.record_policy_decision(
314 +         conn,
315 +         content_item_id=item.id,
316 +         policy_id=policy.id,
317 +         passed=False,
318 +         trace=[
319 +             {
320 +                 "stage_name": "topic-preferences",
321 +                 "passed": True,
322 +                 "reason": "topic preferences disabled",

```

```

323 +         "metadata": {},
324 +     },
325 +     {
326 +         "stage_name": "url-reputation",
327 +         "passed": False,
328 +         "reason": "blocked domain matched: bad.example",
329 +         "metadata": {"matched_blocklist_domain": "bad.example"},
330 +     },
331 + ],
332 + )
333 +
334 + result = await svc.handle(USER_ID, f"/why {item.id}", conn, FilterConfig())
335 +
336 + assert "Final policy result: **REJECTED**" in result.reply_text
337 + assert "`url-reputation`" in result.reply_text
338 + assert "blocked domain matched: bad.example" in result.reply_text
339 + assert "matched_blocklist_domain" in result.reply_text
340 + assert result.config_changed is False
341 +
342 +
343 + async def test_why_missing_policy_decision_uses_metadata_fallback(
344 +     conn: aiomysqlite.Connection, svc: CommandService
345 + ) -> None:
346 +     item = await _seed_content_item(conn, matched=False, height=720)
347 +     assert item.id is not None
348 +
349 +     result = await svc.handle(
350 +         USER_ID,
351 +         f"/why {item.id}",
352 +         conn,
353 +         FilterConfig(target_height=1080),
354 +     )
355 +
356 +     assert "No policy-stage decision trace was recorded" in result.reply_text
357 +     assert "Recorded item status: **rejected**" in result.reply_text
358 +     assert "height 720 < target 1080" in result.reply_text
359 +     assert "current filter config" in result.reply_text
360 +
361 +
362 + @pytest.mark.parametrize("text", ["/why", "/why abc", "/why -1"])
363 + async def test_why_malformed_input_shows_usage(
364 +     conn: aiomysqlite.Connection, svc: CommandService, text: str
365 + ) -> None:
366 +     result = await svc.handle(USER_ID, text, conn, FilterConfig())
367 +
368 +     assert "Usage: `/why <content_item_id>`" in result.reply_text
369 +     assert result.config_changed is False
370 +
371 +
372 + async def test_why_missing_item_shows_clear_response(
373 +     conn: aiomysqlite.Connection, svc: CommandService
374 + ) -> None:
375 +     result = await svc.handle(USER_ID, "/why 9999", conn, FilterConfig())
376 +
377 +     assert "not found for your account" in result.reply_text
378 +     assert result.config_changed is False
379 +
380 +
381 + async def test_why_rejects_cross_account_access(
382 +     conn: aiomysqlite.Connection, svc: CommandService
383 + ) -> None:
384 +     other_user = 222
385 +     await db.upsert_user(
386 +         conn,
387 +         db.User(id=other_user, phone="+15555550200"), # type: ignore[attr-defined]

```

```

388     + )
389     + await db.save_filter_config(conn, other_user, FilterConfig())
390     + item = await _seed_content_item(
391     +         conn,
392     +         account_id=other_user,
393     +         provider_message_id="77",
394     +         matched=True,
395     +     )
396     + assert item.id is not None
397     +
398     + result = await svc.handle(USER_ID, f"/why {item.id}", conn, FilterConfig())
399     +
400     + assert "not found for your account" in result.reply_text
401     + assert "Trace Source" not in result.reply_text
402     diff --git a/tests/test_db.py b/tests/test_db.py
403     index 30ed312..4cc34a3 100644
404     --- a/tests/test_db.py
405     +++ b/tests/test_db.py
406     @@ -1276,6 +1276,19 @@ async def test_record_policy_decision_accepted_trace(conn:
aiosqlite.Connection)
407         assert fetched == decision
408
409
410     +async def test_get_content_item_for_account_scopes_by_owner(
411     +     conn: aiosqlite.Connection,
412     +) -> None:
413     +     item, _policy = await _seed_policy_decision_subject(conn)
414     +     assert item.id is not None
415     +
416     +     fetched = await db.get_content_item_for_account(conn, USER_ID, item.id)
417     +     other_account = await db.get_content_item_for_account(conn, 222, item.id)
418     +
419     +     assert fetched == item
420     +     assert other_account is None
421     +
422     +
423     async def test_record_policy_decision_rejected_without_delivery_attempt(
424     conn: aiosqlite.Connection,
425     ) -> None:
426

```

#### 690. assistant (2026-07-07T00:14:50.111Z)

I've read the full F7 diff ? and the **\*\*IDOR concern is correctly handled\*\***: ``_handle_why`` uses ``get_content_item_for_account(conn, user_id, item_id)`` which does ``WHERE id = ? AND account_id = ?``, gating access *\*before\** the decisions query. There's even a ``test_why_rejects_cross_account_access``. Command parsing is robust (``isdecimal`` + ``< 1`` ? usage), and the fallback for no-trace items (metadata replay + caveat) addresses my F6 note. Let me check the gate and reproduce the account-scoping myself.

#### 691. user (2026-07-07T00:15:05.610Z)

=== gate ===

=== RUFF ===

All checks passed!

36 files already formatted

=== PYTEST ===

```

-----
TOTAL                                1757      80    506     56    94%
Required test coverage of 80.0% reached. Total coverage: 93.99%
560 passed in 10.14s
DONE

```

=== IDOR verify: user A cannot /why user B's item; + parsing + fallback ===

A /why B-item -> reply: '? Content item `1` was not found for your account.'

```
IDOR leak (B data visible to A): False (expect False)
B /why own-item -> shows trace: True (expect True)
/why          -> 'usage'
/why abc      -> 'usage'
/why -1       -> 'usage'
/why 0        -> 'usage'
```

#### 692. assistant (2026-07-07T00:15:26.553Z)

IDOR **\*\*verified safe\*\***: user A's `/why <B's item>`` ? "not found for your account" with **\*\*zero leak\*\*** of B's channel/caption/URL/domain; B sees its own trace; all malformed inputs (`/why``, ``abc``, ``-1``, ``0``) ? usage. Gate: ruff clean, **\*\*560 passed @ 93.99%\*\***. The main security risk is correctly handled and tested. Given `/why`` reads by id, let me run a focused adversarial pass on access-control edges + rendering, then decide.

#### 693. user (2026-07-07T00:16:04.260Z)

```
<tool_use_error>Invalid workflow script: Script parse error: Unexpected token (41:50). Workflow
scripts must be plain JavaScript ? TypeScript syntax (type annotations like `: string[]`,
interfaces, generics) fails to parse.</tool_use_error>
```

#### 694. assistant (2026-07-07T00:16:38.224Z)

The workflow script hit a JS parse error (a raw regex in the template literal confused the parser). I've already verified the crux (IDOR safe, gate green, parsing robust), so let me re-issue a clean version with the regex described in words.

#### 695. user (2026-07-07T00:17:09.248Z)

Workflow launched in background. Task ID: woadsh7sn  
Summary: Review telegram-video-filter PR #57 (F7 /why explainability) ? access control (IDOR) and rendering edges, adversarial verification  
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_56dfabe6-f8c  
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr57-f7-wf\_56dfabe6-f8c.js  
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr57-f7-wf\_56dfabe6-f8c.js"} to iterate without resending the script.)  
Run ID: wf\_56dfabe6-f8c  
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr57-f7-wf\_56dfabe6-f8c.js", resumeFromRunId: "wf\_56dfabe6-f8c"}) ? completed agents return cached results.

You will be notified when it completes. Use `/workflows` to watch live progress.

#### 696. assistant (2026-07-07T00:17:30.900Z)

While the workflow runs, let me check two ``parse_render`` edges myself: the unicode-digit ``isdecimal`` behavior and whether attacker-controlled caption/URL backticks break the markdown code span in the reply.

#### 697. user (2026-07-07T00:17:38.106Z)

```
isdecimal('42')=True int=42
isdecimal('??')=True int=42
isdecimal('???')=True int=42
isdecimal('²')=False int=None
---
caption with backtick -> "evil `code` here"
metadata with backtick -> {"matched_url": "http://ev`il.example"}
```

## 698. assistant (2026-07-07T00:18:00.372Z)

Both are low:

- **Unicode digits**: ``??`/`??`` pass ``isdecimal()`` and ``int()`` accepts them (? 42) ? no crash, still account-scoped (mild surprise, non-issue). Superscript ``2`` correctly rejected.
- **Backtick injection**: ``_format_json_value`` preserves backticks (JSON doesn't escape them), so an attacker-controlled caption/URL/domain containing a backtick would break the markdown code span in the reply ? a **display-formatting glitch** in the user's own ``/why`` output\*\* (not a cross-account leak). Worth escaping.

So my pass finds only lows (no IDOR issue, no crash). I'll await the workflow's adversarial IDOR check before deciding ? per your rule, and because the workflow has caught things I missed before (F5's mediums). Awaiting it.

## 699. user (2026-07-07T00:22:46.998Z)

```
<task-notification>
<task-id>woadsh7sn</task-id>
<tool-use-id>toolu_01BPV4Nddkpum7yoVAALux51</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\woadsh7sn.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Review telegram-video-filter PR #57 (F7 /why explainability) ? access control (IDOR) and rendering edges, adversarial verification" completed</summary>
<result>{"counts":{"raw":3,"kept":3,"refuted":0},"findings":[{"dimension":"parse_render","severity":"medium","verdict":"CONFIRMED","title":"Oversized /why reply exceeds Telegram 4096-char limit and is silently dropped","where":"src/tg_video_filter/command_service.py:223","summary":"The /why output is unbounded: per-stage `reason` is rendered with no length cap (line 223) and the overall message has no cap, so an item with several policy decisions x multi-stage traces (or any long reason string) produces a message well over Telegram's 4096-character limit. `send_message` then raises MessageTooLongError (an RPCError subclass) which `_reply` (commands.py:131) catches and swallows, so the user receives NO response at all.","failure_scenario":"A video re-evaluated a few times (e.g. 3 decisions) each with a 6-stage trace whose stages carry a long `reason` (e.g. a URL-reputation or error message ~600 chars) produces a 17,781-char reply (reproduced). Telethon's send_message raises MessageTooLongError; _reply logs a warning and returns; the /why command silently produces nothing. The item that most needs explaining (many stages, long reasons) is exactly the one that yields no answer.","evidence":"Reproduced with the real formatter: `_format_policy_decisions` on a valid ContentItem + 3 PolicyDecisions of 6 stages each (reason len 627, untruncated) returned MESSAGE LENGTH: 17781, EXCEEDS 4096: True. Note the author truncated `metadata` to 500 (line 226) and `item.text` to 160 (line 204) but left `reason` (line 221/223) untruncated and applied no overall message cap. commands.py:131 catches RPCError and only logs 'config_reply_failed'.","recommendation":"Cap the total assembled message below the Telegram limit (e.g. truncate to ~4000 chars with a trailing '?' (truncated) marker), and/or truncate `reason` like the other fields. Alternatively split into multiple messages. At minimum, guarantee the user always gets a bounded, sendable reply.","verify_reasoning":"Verified all three load-bearing claims in the PR diff. (1) In `_format_policy_decisions` (command_service.py:193-227), per-stage `reason` is rendered untruncated at line 223 (only `metadata` is capped at 500 via line 226 and `item.text` at 160 via line 204), and there is no overall message-length cap. (2) The send path is confirmed: /why returns CommandResult.reply_text, which commands.py:87 passes to _reply, which calls client.send_message(`me`, text) inside a try/except catching (OSError, TimeoutError, RPCError) and only logging log.warning(`config_reply_failed`) (commands.py:128-132). (3) I verified telethon.errors.MessageTooLongError IS a subclass of RPCError, so an oversized send raises it and it is swallowed ? the user receives no reply.\n\nReproduced the finding's exact scenario with the real formatter: a valid ContentItem + 3 PolicyDecisions x 6-stage traces with 625-char reasons produced a 11,871-char message (finding claimed 17,781 with slightly longer reasons ? same order, and my extreme variant exceeded 17k); EXCEEDS 4096 = True. Note: pydantic field types required policy_id to be int and provider_message_id str, but that did not affect the result.\n\nAdversarial counter-check: realistic modest cases stay well under 4096 ? a single decision with a 6-stage trace of 122-char reasons is 991 chars, and 10 short-trace decisions is 2587 chars. So crossing Telegram's limit requires genuinely long `reason` strings (~600 chars) or a high count of accumulated decisions/stages. This is not routinely triggered, which keeps it an edge bug rather than a systematic failure. However, engine reasons (policy.py:143/843) are persisted verbatim with no upstream cap and can embed source-derived URLs/hostnames/error text,
```

so long reasons are plausible, especially in the error/re-evaluation cases /why is most useful for. The failure is silent (user gets nothing) precisely for the most complex items. Genuine defect shipping in this diff.\n\nSeverity: this is a rendering/limit bug on user input with a silent-drop outcome ? not an access-control bypass, cross-account leak, or crash. Medium is the correct rating per the rubric and matches the finding.},{\"dimension\":\"parse\_render\",\"severity\":\"medium\",\"verdict\":\"CONFIRMED\",\"title\":\"Markdown injection into the /why report via attacker-controlled caption / link\_url\",\"where\":\"src/tg\_video\_filter/command\_service.py:204\",\"summary\":\"`item.text` (the caption of a watched channel's post, attacker-controlled) is echoed into the markdown-parsed /why reply at line 204, and `item.link\_url` raw at line 202. Telethon sends these with markdown parsing enabled (send\_message with no parse\_mode, messages use `\*\*bold\*\*`/backticks). The JSON-encoding at line 176 escapes quotes/backslashes but does NOT neutralize markdown metacharacters ([ ] ( ) \* \_ `), so an attacker's caption can inject active formatting ? most notably a clickable [label](url) hyperlink ? into the bot's own trusted explainability output.\"},{\"failure\_scenario\":\"An attacker who controls a post in a channel the victim watches sets the caption to `Nice video [CLICK FOR PRIZE](http://phish.evil)`. That lands as item.text. When the victim runs `/why &id&`, line 204 renders `Text: \\\"Nice video [CLICK FOR PRIZE](http://phish.evil)\\\"`; Telethon's markdown parser turns it into a real MessageEntityTextUrl ? the phishing link appears as trusted-looking text inside the bot's report. link\_url (line 202, currently often NULL) is even more exposed as it is rendered with no encoding at all.\"},{\"evidence\":\"item.text is populated from real message captions (\_message\_text at telethon\_client.py:279 / backfill.py:243) and persisted (db.py:1813,1867), so it is live attacker data. Reproduced through telethon.extensions.markdown.parse on the actual `\_format\_json\_value(caption)` output: the caption `[CLICK FOR PRIZE](http://phish.evil)` produced a MessageEntityTextUrl entity in the final message; `[pwn](http://x)` in a link\_url likewise produced a MessageEntityTextUrl. JSON encoding left `[ ] ( )` intact.\"},{\"recommendation\":\"Escape markdown metacharacters in attacker-controlled values before interpolation (or send this reply with parse\_mode disabled and format entities explicitly, or wrap echoed text as inline code with the backtick content itself escaped). Apply to item.text and item.link\_url, and to any stage `reason` that can carry echoed content.\"},{\"verify\_reasoning\":\"Confirmed and reproduced. The vulnerable sinks are new in this PR: git diff HEAD~1 shows the entire \_format\_policy\_decisions function is added, including line 202 `lines.append(f\"Link: {item.link\_url}\")` and line 204 `lines.append(f\"Text: {\_format\_json\_value(item.text, max\_len=160)}\\\")` (command\_service.py:202,204).\\n\\nSend path: /why -& CommandResult.reply\_text -& commands.py:87 \_reply -& commands.py:130 `client.send\_message(\\\"me\\\", text)` with NO parse\_mode override. Verified Telethon 1.44.0 defaults `self.\_parse\_mode = markdown` in TelegramBaseClient.\_\_init\_\_, so the reply is markdown-parsed.\\n\\nAttacker data: item.text is populated from the raw watched-channel post caption via \_message\_text (telethon\_client.py:279/393) and persisted (db.py:1813,1867). It is live attacker-controlled input. \_format\_json\_value uses json.dumps which escapes quotes/backslashes but leaves markdown metacharacters [ ] ( ) \* \_ ` intact.\\n\\nReproduced end-to-end through the real code: built a ContentItem with text='Nice video [CLICK FOR PRIZE](http://phish.evil)' and link\_url='http://real[pwn](http://x.evil)', ran \_format\_policy\_decisions, then telethon.extensions.markdown.parse on the output. Result: two MessageEntityTextUrl entities were produced ? url=http://phish.evil covering \\\"CLICK FOR PRIZE\\\" (from item.text, line 204) and url=http://x.evil covering \\\"pwn\\\" (from item.link\_url, line 202). Both become real clickable hyperlinks inside the bot's trusted /why report.\\n\\nRefutation attempts all failed: (1) not neutralized ? JSON encoding leaves brackets/parens intact, verified; (2) genuinely at ... (truncated 4721 chars, full result in C:\\Users\\avidu\\AppData\\Local\\Temp\\claude\\C--Users-avidu-Projects-badminton-highlight-indexer\\7dc3da51-af38-47b6-83f6-fdda83b60664\\tasks\\woadsh7sn.output)</result>  
<usage><agent\_count>5</agent\_count><subagent\_tokens>248378</subagent\_tokens><tool\_uses>83</tool\_uses><duration\_ms>337714</duration\_ms></usage>  
</task-notification>

## 700. assistant (2026-07-07T00:23:42.201Z)

The workflow found \*\*2 mediums I under-rated\*\* (I had both as low) ? and one is more serious than my backtick note: attacker-controlled captions render as \*\*active markdown\*\* (clickable phishing links) in the `/why` report. Let me get the full set and verify the injection myself.

## 701. user (2026-07-07T00:23:56.624Z)

COUNTS: {'raw': 3, 'kept': 3, 'refuted': 0}  
[MEDIUM/CONFIRMED] (parse\_render) Oversized /why reply exceeds Telegram 4096-char limit and is



silently dropped  
 where: src/tg\_video\_filter/command\_service.py:223  
 rec: Cap the total assembled message below the Telegram limit (e.g. truncate to ~4000 chars with a trailing '?' (truncated)' marker), an

[MEDIUM/CONFIRMED] (parse\_render) Markdown injection into the /why report via attacker-controlled caption / link\_url  
 where: src/tg\_video\_filter/command\_service.py:204  
 rec: Escape markdown metacharacters in attacker-controlled values before interpolation (or send this reply with parse\_mode disabled and

[LOW/CONFIRMED] (parse\_render) Replay-fallback caveat omits that content-type filters (gif/round/nosound) are invisible to the replay  
 where: src/tg\_video\_filter/command\_service.py:250  
 rec: Extend the caveat to state that content-type exclusion filters (gif/round/nosound) are not captured by the metadata replay, so the

refuted: []

=== independently verify markdown injection: caption -> clickable link in /why (via telethon markdown parse) ===  
 markdown entities in /why reply: [('MessageEntityBold', None), ('MessageEntityCode', None), ('MessageEntityBold', None), ('MessageEntityCode', None), ('MessageEntityTextUrl', 'http://phish.evil'), ('MessageEntityCode', None), ('MessageEntityBold', None), ('MessageEntityCode', None), ('MessageEntityCode', None)]  
 -> PHISHING LINK INJECTED: True

## 702. assistant (2026-07-07T00:24:26.035Z)

**Markdown injection confirmed independently** ? an attacker's watched-channel caption `[CLAIM PRIZE](http://phish.evil)` renders as a real `MessageEntityTextUrl` (clickable **phishing link**) inside the bot's trusted `/why` report. This is live (captions are populated by F2). The access-control dimension found **nothing** ? the IDOR is genuinely clean, corroborating my check. So F7 has **2 mediums** (both rendering), and I under-rated the injection (I'd caught the backtick variant but missed the worse `[label](url)` hyperlink). Per your rule, that's a hold. Let me post.

## 703. assistant (2026-07-07T00:24:52.456Z)

### Review ? F7 `/why` explainability command

The security fundamental is right ? **access control is correctly scoped** (no IDOR), which is the thing I was most worried about for a lookup-by-id command. **Holding the LGTM** on two rendering mediums (one is a real stored-injection / phishing vector). Everything reproduced locally.

#### Verified good ?

- **No IDOR / cross-account leak** ? `\_handle\_why` gates on `db.get\_content\_item\_for\_account` (`WHERE id=? AND account\_id=?`) **before** the decisions query; the "not found for your account" message is identical for not-exists vs not-owned (no enumeration oracle). Reproduced: account A's `/why` <B's item> ? "not found", with **zero** leak of B's channel/caption/URL/domain; B sees its own trace. Tested (`test\_why\_rejects\_cross\_account\_access` + the DB scoping test). `user\_id` is the requester's own account (adapter gates on `message.out` + `peer\_id==user.id`).
- **Robust parsing** ? `isdecimal` + `<1` ? Usage; unicode digits (`??/??`) parse to their value without crashing (still account-scoped); huge ids ? no overflow, no match. Malformed (`/why`, `/why abc`, `-1`, `0`) ? Usage.
- **Fallback for no-trace items** (`\_format\_missing\_policy\_decision`) nicely addresses my F6 note ? metadata-filter replay + a config-drift caveat. **560 passed @ 93.99%**, ruff clean.

#### ? Medium

**1. Markdown injection via attacker-controlled caption / `link\_url` ? phishing link in the bot's own report** ? `command\_service.py:204` (`item.text`) / `:202` (`item.link\_url`)  
 The `/why` reply is sent through `\_reply` ? `client.send\_message("me", text)` with Telethon's

**\*\*default markdown parse\_mode\*\*.** ``item.text`` (a watched channel's post caption ? attacker-controlled) is echoed via ``_format_json_value``, which JSON-escapes quotes/backslashes but **\*\*leaves markdown metacharacters ``[]()*_`` intact\*\***; ``item.link_url`` is echoed **\*\*raw\*\***. So an attacker who posts in a channel the victim watches can inject active formatting into the bot's trusted explainability output. **\*\*Reproduced end-to-end:\*\*** a caption ``Nice clip [CLAIM PRIZE](http://phish.evil)`` produces a real ``MessageEntityTextUrl`` (``url=http://phish.evil``) ? a clickable phishing link ? inside the ``/why`` reply. ``item.text`` is live (populated by F2); ``link_url`` is currently NULL (even more exposed once populated). The stage ``reason`/`metadata`` values (e.g. ``matched_url``, ``matched_caption_term``) are also attacker-derived sinks.

> **\*\*Fix:\*\*** escape markdown metacharacters in every attacker-controlled interpolated value (``item.text``, ``item.link_url``, and content-bearing ``reason`/`metadata``), or send the ``/why`` reply with ``parse_mode`` disabled and build entities explicitly. This is the first command that echoes untrusted content, so the sink is new in F7.

**\*\*2. Oversized ``/why`` reply exceeds Telegram's 4096-char limit and is silently dropped\*\*** ?

``command_service.py:223``

Per-stage ``reason`` is rendered **\*\*untruncated\*\*** (only ``metadata`` is capped at 500 and ``item.text`` at 160), and there's **\*\*no overall message cap\*\***. A large trace ? ``MessageTooLongError`` (an ``RPCError``) ? caught + swallowed by ``_reply`` ? the user gets **\*\*no reply at all\*\***, precisely for the most complex item they wanted explained. Reproduced ~17k chars with an extreme trace.

**\*Reachability caveat:** with the current single-default-policy + 4-stage pipeline and short reasons, a realistic ``/why`` stays well under 4096 (~2.7k worst-case), so this isn't routinely triggered today ? but the untruncated ``reason`` + no total cap is a real robustness gap with a silent-failure outcome.

> **\*\*Fix:\*\*** cap the total assembled message (~4000 chars + "? (truncated)"), and/or truncate ``reason`` like the other fields, so a bounded, sendable reply is guaranteed.

## ? Low

- The replay-fallback caveat (``command_service.py:250``) should also state that **\*\*content-type exclusion filters (gif/round/nosound) aren't captured\*\*** by the metadata replay (those flags aren't persisted on ``content_items``), so the replay can differ from the original decision for those rules.

---

**\*\*Holding LGTM\*\*** on #1 (markdown/phishing injection ? the priority; it's live via captions) and #2 (bounded, always-sendable reply). Both are contained fixes. The access-control design is genuinely solid ? nice to see the account-scoping done right the first time. Ping me when the escaping + cap are in and I'll re-verify.

<sub>> Reviewed with [Claude Code](https://claude.com/claude-code) ? IDOR-safety (no leak), the caption?clickable-link injection, and the oversized-reply drop all reproduced on the branch. Multi-agent adversarial pass: 3 findings, 0 refuted; access-control dimension found no leak.</sub>

## 704. user (2026-07-07T00:24:52.493Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr57\_review.md (file state is current in your context ? no need to Read it back)

## 705. user (2026-07-07T00:25:06.274Z)

=== post review (holding LGTM) on #57 ===  
https://github.com/avidullu/telegram-video-filter/pull/57#issuecomment-4898858467  
=== cleanup ===  
removed  
=== arm watcher (baseline #57) ===  
baseline: #57 draft=false sha=12b4442e5 upd=2026-07-07T00:25:05Z

## 706. assistant (2026-07-07T00:25:11.006Z)

Review posted (LGTM held) ? [issuecomment-4898858467](https://github.com/avidullu/telegram-video-filter/pull/57#issuecomment-4898858467). Worktree cleaned. Arming the watcher.

## 707. user (2026-07-07T00:25:17.752Z)

Command running in background with ID: b6os4fwyr. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b6os4fwyr.output. You will be notified when it completes. To check interim output, use Read on that file path.

## 708. assistant (2026-07-07T00:25:31.216Z)

### PR #57 (F7 `/why` explainability) ? reviewed, \*\*LGTM held\*\*

[Review posted](https://github.com/avidullu/telegram-video-filter/pull/57#issuecomment-4898858467). The one thing that could have been a real security bug ? \*\*IDOR\*\* ? is done correctly (account-scoped lookup + tested; I reproduced zero cross-account leak). But I held on two rendering \*\*mediums\*\*, and once again the workflow caught what I under-rated:

- ? \*\*Markdown/phishing injection\*\* ? an attacker's watched-channel caption `[CLAIM PRIZE](http://phish.evill)` renders as a \*\*clickable link inside the bot's own `/why` report\*\* (reply is markdown-parsed; JSON-encoding doesn't neutralize `[]()`). Reproduced end-to-end. Live via captions. Fix: escape markdown in echoed untrusted values (or disable `parse\_mode`). \*I'd flagged only the milder backtick variant; the workflow found the worse hyperlink injection.\*
- ? \*\*Oversized reply silently dropped\*\* ? no total message cap + untruncated `reason` ? `MessageTooLongError` swallowed ? user gets nothing. Reachability-limited today (single policy, 4 stages), but a real robustness gap; fix with a total cap.

Plus one low (replay caveat should mention content-type filters). Access-control dimension found \*\*nothing\*\* ? IDOR clean.

### Status

- \*\*Track F:\*\* #40 · F1 #50 · F2 #51 · F3 #52 · F4 #53 · F5 #54 · F6 #55 merged ? · \*\*F7 #57 ? changes requested\*\* (final phase, so close to done).
- \*\*Watcher armed\*\* (`b6os4fwyr`, baseline #57) ? I'll re-verify the moment a fix-push lands (SHA change) and only LGTM when the injection-escaping + reply-cap check out.

Once #57 lands, that's the complete ADR 0012 policy pipeline (design ? F1?F7) reviewed end-to-end. I'll surface the fix or the next PR when it arrives.

## 709. user (2026-07-07T00:31:22.025Z)

```
<task-notification>
<task-id>b6os4fwyr</task-id>
<tool-use-id>toolu_01NTqfxmtp5sohzgbB7GW6HF</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b6os4fwyr.output</output-file>
<status>completed</status>
<summary>Background command "Arm watcher v2 (baseline #57)" completed (exit code 0)</summary>
</task-notification>
```

## 710. assistant (2026-07-07T00:31:28.403Z)

The watcher fired. Let me check what changed on #57.

## 711. user (2026-07-07T00:31:30.625Z)

```
=== PR-watch v2 ARMED ===
baseline: #57 draft=false sha=12b4442e5 upd=2026-07-07T00:25:05Z
poll 1/30: no change (00:27:19Z)
poll 2/30: no change (00:29:20Z)
=== PR ACTIVITY DETECTED (poll 3, 00:31:21Z) ===
--- baseline ---
#57 draft=false sha=12b4442e5 upd=2026-07-07T00:25:05Z
--- now ---
#57 draft=false sha=fe3c48349 upd=2026-07-07T00:31:16Z
```

```
--- new/changed (sha change=force-push/fix; only upd change=comment) ---
#57 draft=false sha=fe3c48349 upd=2026-07-07T00:31:16Z
--- open PR details ---
#57 draft=false by avidullu: F7: add /why explainability command
```

## 712. assistant (2026-07-07T00:31:36.197Z)

SHA changed (`12b4442` ? `fe3c483`) ? a **fix push** for the two mediums. Let me see what changed.

## 713. user (2026-07-07T00:31:40.887Z)

```
=== fix diff (12b4442..fe3c483) ===
src/tg_video_filter/command_service.py | 42 ++++++-----
tests/test_command_service.py          | 73 ++++++-----
2 files changed, 105 insertions(+), 10 deletions(-)

diff --git a/src/tg_video_filter/command_service.py b/src/tg_video_filter/command_service.py
index 0e3a390..0ab172a 100644
--- a/src/tg_video_filter/command_service.py
+++ b/src/tg_video_filter/command_service.py
@@ -47,6 +47,9 @@ _WATCH_RE = re.compile(r"^/watch\s+(.+)$", re.IGNORECASE)
 _UNWATCH_RE = re.compile(r"^/unwatch\s+(.+)$", re.IGNORECASE)
 _CHANNELS_RE = re.compile(r"^/channels$", re.IGNORECASE)
 _WATCHLIST_RE = re.compile(r"^/watchlist$", re.IGNORECASE)
+_MARKDOWN_ESCAPE_RE = re.compile(r"([\`*_\[\]\(\)])")
+_AUTOLINK_RE = re.compile(r"\b(https?)://" , re.IGNORECASE)
+_REPLY_CHAR_LIMIT = 4000

# --- unit parsing -----

@@ -172,11 +175,29 @@ def format_commands() -> str:
)

+def _truncate_text(text: str, max_len: int) -> str:
+    if len(text) <= max_len:
+        return text
+    return f"{text[: max_len - 3]}..."
+
+def _safe_markdown_text(text: str, max_len: int) -> str:
+    text = _truncate_text(text, max_len)
+    text = _AUTOLINK_RE.sub(r"\1: //" , text)
+    return _MARKDOWN_ESCAPE_RE.sub(r"\\1" , text)
+
+def _format_json_value(value: object, max_len: int = 500) -> str:
+    text = json.dumps(value, default=str, ensure_ascii=False, sort_keys=True)
+    if len(text) <= max_len:
+        return _safe_markdown_text(text, max_len)
+
+def _format_reply(lines: list[str]) -> str:
+    text = "\n".join(lines)
+    if len(text) <= _REPLY_CHAR_LIMIT:
+        return text
+    return f"{text[: max_len - 1]}?"
+    suffix = "\n... (truncated)"
+    return f"{text[: _REPLY_CHAR_LIMIT - len(suffix)].rstrip()}{suffix}"

def _content_item_to_video_meta(item: ContentItem) -> VideoMeta:
@@ -196,10 +217,11 @@ def _format_policy_decisions(item: ContentItem, decisions:
```

```

list[PolicyDecision])
    f"? **Why content item `{item.id}`?***",
    "",
    f"Final policy result: **{status}***",
-   f"Provider message: `{item.provider}:{item.provider_message_id}`",
+   "Provider message: "
+   f"`{_safe_markdown_text(f'{item.provider}:{item.provider_message_id}', 120)}`",
    ]
    if item.link_url:
-       lines.append(f"Link: {item.link_url}")
+       lines.append(f"Link: `{_safe_markdown_text(item.link_url, 300)}`")
    if item.text:
        lines.append(f"Text: {_format_json_value(item.text, max_len=160)}")

@@ -218,13 +240,13 @@ def _format_policy_decisions(item: ContentItem, decisions:
list[PolicyDecision])
    for stage in decision.trace:
        stage_name = str(stage.get("stage_name", "unknown"))
        stage_passed = bool(stage.get("passed"))
-       reason = str(stage.get("reason", "no reason recorded"))
+       reason = _safe_markdown_text(str(stage.get("reason", "no reason recorded")), 300)
        icon = "?" if stage_passed else ""
-       lines.append(f"- {icon} `{stage_name}` ? {reason}")
+       lines.append(f"- {icon} `{_safe_markdown_text(stage_name, 80)}` ? {reason}")
        metadata = stage.get("metadata")
        if isinstance(metadata, dict) and metadata:
            lines.append(f"  metadata: `{_format_json_value(metadata)}`")
-       return "\n".join(lines)
+       return _format_reply(lines)

def _format_missing_policy_decision(item: ContentItem, current_cfg: FilterConfig) -> str:
@@ -248,10 +270,12 @@ def _format_missing_policy_decision(item: ContentItem, current_cfg:
FilterConfig
    [
        "",
        "This fallback uses the current filter config and persisted metadata only; "
-       "historical config changes can make it differ from the original decision.",
+       "historical config changes can make it differ from the original decision. "
+       "Content-type exclusion flags such as GIF, round video, and nosound are not "
+       "persisted for replay.",
    ]
)
-   return "\n".join(lines)
+   return _format_reply(lines)

```

# --- command result -----

#### 714. assistant (2026-07-07T00:32:05.251Z)

The author fixed both mediums + the low comprehensively:

- **\*\*#1 injection:\*\*** `\_safe\_markdown\_text` escapes ` \ ` \* \_ [ ] ( ) ` and de-fangs `http://` autolinks ? applied to `item.text`, `link\_url`, `reason`, `stage\_name`, provider msg, and metadata.
- **\*\*#2 oversized:\*\*** `\_REPLY\_CHAR\_LIMIT = 4000` total cap via `\_format\_reply` (+ per-field truncation) on both render paths.
- **\*\*Low:\*\*** caveat now mentions gif/round/nosound.

Per the gate, I verify. Let me set up a worktree, run the gate, and re-run my injection repro (should now show **\*\*no\*\*** clickable link) + a length check.

#### 715. user (2026-07-07T00:32:23.122Z)

HEAD is now at fe3c483 feat(commands): add why explainability command

```
gate started
=== verify injection neutralized + reply cap (against fixed code) ===
1 injection: phishing link present = False (expect False) | texturl entities: []
2 oversized: reply len = 1060 (<=4096 expect True: True ) truncated marker: False
```

#### 716. assistant (2026-07-07T00:32:37.374Z)

```
Both mediums **verified fixed**:
1. **Injection neutralized** ? the `[CLAIM PRIZE](http://phish.evil)` caption produces **no**
clickable link (`texturl` entities: []). Escaping works.
2. **Reply bounded** ? a 20,000-char reason yields a **1060-char** reply (per-field truncation +
the 4000 total cap keep it well under 4096).
```

Let me confirm the gate is green.

#### 717. user (2026-07-07T00:32:45.147Z)

```
=== RUFF ===
All checks passed!
36 files already formatted
=== PYTEST ===
TOTAL                                1772      77    508    53    94%
Required test coverage of 80.0% reached. Total coverage: 94.30%
562 passed in 10.17s
DONE
```

#### 718. assistant (2026-07-07T00:32:59.076Z)

Gate green: ruff clean, \*\*562 passed @ 94.30%\*\* (+2 tests, coverage up). Both mediums + the low are fixed and verified, no new issues. Per your rule, I'm satisfied ? merge-ready. Posting the LGTM.

#### 719. user (2026-07-07T00:33:11.751Z)

```
=== post LGTM ===
https://github.com/avidullu/telegram-video-filter/pull/57#issuecomment-4898900238
=== cleanup ===
removed
```

#### 720. assistant (2026-07-07T00:33:15.653Z)

LGTM posted ? [issuecomment-4898900238](https://github.com/avidullu/telegram-video-filter/pull/57#issuecomment-4898900238). Worktree cleaned. Arming the watcher.

#### 721. user (2026-07-07T00:33:22.436Z)

Command running in background with ID: bjtemdyru. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bjtemdyru.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 722. assistant (2026-07-07T00:33:36.248Z)

### PR #57 (F7 `why`) ? fix verified, \*\*LGTM'd\*\* ?

The watcher caught the fix-push; I re-reviewed and signed off ([LGTM](https://github.com/avidullu/telegram-video-filter/pull/57#issuecomment-4898900238)). The author fixed \*\*both mediums + the low\*\*, verified against my own exploits:

- \*\*Markdown injection\*\* ? `\_safe\_markdown\_text` escapes ` \ ` \* \_ [ ] ( ) ` and de-fangs `http(s)://`, applied to every echoed untrusted value. My phishing caption now yields \*\*zero\*\* clickable-link entities.
- \*\*Oversized reply\*\* ? per-field truncation + a 4000-char total cap; a 20k-char reason now

renders a 1060-char (always-sendable) reply.

- Content-type replay caveat added.

Gate: \*\*562 passed @ 94.30%\*\* , ruff clean.

## ? The full ADR 0012 policy pipeline is now reviewed end-to-end

| PR   | Phase                       | Findings ?                         | outcome        |
|------|-----------------------------|------------------------------------|----------------|
| ---- | -----                       | -----                              | -----          |
| #40  | ADR 0012 design             | 10 gaps ?                          | revised ? LGTM |
| #50  | F1 primitives + persistence | clean ?                            | LGTM           |
| #51  | F2 topic preferences        | 3 mediums ?                        | fixed ? LGTM   |
| #52  | F3 caption safety           | 3 mediums (ReDoS/fail-open/case) ? | fixed ? LGTM   |
| #53  | F4 URL reputation           | 1 medium ?                         | fixed ? LGTM   |
| #54  | F5 deep-inspection budget   | 2 mediums ?                        | fixed ? LGTM   |
| #55  | F6 policy decisions         | 3 lows + proactive hardening ?     | LGTM           |
| #57  | F7 `/why`                   | 2 mediums (injection/limit) ?      | fixed ? LGTM   |

Every finding reproduced locally, LGTM strictly gated on verified fixes, and the workflow repeatedly caught mediums I under-rated (F5's fetcher-error handling, F7's markdown-injection) ? which is exactly why the adversarial pass runs on each.

**\*\*State:\*\*** watcher armed (`bjtemdyru`, baseline fixed #57) ? it'll wake me on the #57 merge or the next PR. This whole Track (B?F, plus the earlier fixes) has been a genuinely clean series; I'll keep watching and running the same cycle for whatever lands next.

### 723. user (2026-07-07T00:37:26.034Z)

```
<task-notification>
<task-id>bjtemdyru</task-id>
<tool-use-id>toolu_012e9ZcfzWN61vDg38hCdjmMH</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bjtemdyru.output</output-file>
<status>completed</status>
<summary>Background command "Arm watcher v2 (baseline fixed #57)" completed (exit code 0)</summary>
</task-notification>
```

### 724. assistant (2026-07-07T00:37:30.806Z)

The watcher fired. Let me check.

### 725. user (2026-07-07T00:37:32.936Z)

```
=== PR-watch v2 ARMED ===
baseline: #57 draft=false sha=fe3c48349 upd=2026-07-07T00:33:11Z
poll 1/30: no change (00:35:24Z)
=== PR ACTIVITY DETECTED (poll 2, 00:37:25Z) ===
--- baseline ---
#57 draft=false sha=fe3c48349 upd=2026-07-07T00:33:11Z
--- now ---
<none>
--- new/changed (sha change=force-push/fix; only upd change=comment) ---

--- open PR details ---
```

### 726. assistant (2026-07-07T00:37:37.567Z)

Open set ? empty ? #57 merged, **\*\*completing the ADR 0012 pipeline\*\***. Let me confirm, fast-forward `main`, and re-arm.

### 727. user (2026-07-07T00:37:45.662Z)

Command running in background with ID: bxh7un3vj. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bxh7un3vj.output. You will be notified when it completes. To check interim output, use Read on that file path.

**728. assistant (2026-07-07T00:38:00.329Z)**

## **PR #57 (F7 `/why`) merged ? \*\*ADR 0012 policy pipeline complete\*\* ?**

The open set went empty right after my LGTM, so #57 is merged. That closes out the entire umbrella:

```
**Provider-neutral umbrella (Tracks B?F), all reviewed & merged:**  
- **B** #35 (domain schema) . **C** #37 (delivery/routing) . **D** #38 (CommandService) . **E**  
#39 + #41 (selective scanning + my follow-up fix)  
- **ADR 0012** #40 (design) . **F1** #50 . **F2** #51 . **F3** #52 . **F4** #53 . **F5** #54 .  
**F6** #55 . **F7** #57
```

Across the F-series alone: \*\*12 mediums\*\* the author fixed before I'd sign off (topic fail-closed, ReDoS, URL `normalize\_domain` throw, deep-inspection fetcher-error handling, `/why` markdown injection?), every one reproduced locally, and LGTM held until each fix verified. The adversarial workflow repeatedly earned its keep by catching mediums I under-rated (F5, F7).

**\*\*State:\*\*** local `main` fast-forwarded to the merged tip, clean and synced. **\*\*Watcher re-armed\*\*** (`bxh7un3vj`) ? it'll wake me on the next PR or any force-push/comment, distinguishing fix-pushes (re-review) from comments (read + judge).

With the pipeline done, the next PRs are likely a different theme (maybe wiring the `/set` config commands that flip F2?F5's dormant stages live ? where a cluster of my "latent until a command lands" notes go from theoretical to real, so I'll scrutinize those command?stage interactions hard). I'll surface whatever lands next and run the same cycle.

**729. user (2026-07-07T01:38:14.591Z)**

```
<task-notification>  
<task-id>bxh7un3vj</task-id>  
<tool-use-id>toolu_017dfirvjDB6AqKwtV5hxvrw</tool-use-id>  
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bxh7un3vj.output</output-file>  
<status>completed</status>  
<summary>Background command "Confirm #57 merge, ff main, re-arm watcher" completed (exit code 0)</summary>  
</task-notification>
```

**730. assistant (2026-07-07T01:38:28.322Z)**

The watcher fired. Let me check.

**731. user (2026-07-07T01:38:30.493Z)**

```
poll 14/30: no change (01:06:02Z)  
poll 15/30: no change (01:08:02Z)  
poll 16/30: no change (01:10:03Z)  
poll 17/30: no change (01:12:04Z)  
poll 18/30: no change (01:14:05Z)  
poll 19/30: no change (01:16:05Z)  
poll 20/30: no change (01:18:06Z)  
poll 21/30: no change (01:20:07Z)  
poll 22/30: no change (01:22:08Z)  
poll 23/30: no change (01:24:09Z)  
poll 24/30: no change (01:26:09Z)  
poll 25/30: no change (01:28:10Z)  
poll 26/30: no change (01:30:11Z)  
poll 27/30: no change (01:32:12Z)
```



poll 28/30: no change (01:34:12Z)  
poll 29/30: no change (01:36:13Z)  
poll 30/30: no change (01:38:14Z)  
=== window elapsed (~60 min), no activity ===

### 732. assistant (2026-07-07T01:38:50.025Z)

First quiet window ? the watcher ran its full ~60 min (30 polls) with **\*\*no activity\*\***: no new PR, no force-push, no comment since #57 merged. The author has paused (it's ~01:38 UTC after a long, productive session).

Per your standing instruction I'll keep the vigil going, but I wanted to flag the state change since it's the first idle stretch. Let me re-arm.

### 733. user (2026-07-07T01:38:58.041Z)

Command running in background with ID: bur02c0dk. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claudc\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bur02c0dk.output. You will be notified when it completes. To check interim output, use Read on that file path.

### 734. assistant (2026-07-07T01:39:10.920Z)

## Status ? quiet for the first time; watch continues

The author has gone idle: a full ~60-minute window (30 polls) passed with **\*\*no PR, no push, no comment\*\*** since #57 (F7) merged. After a marathon session that took the entire provider-neutral umbrella (Tracks B?F + ADR 0012's F1?F7) from open PRs to merged, that's a natural stopping point.

**\*\*Watcher re-armed\*\*** (`bur02c0dk`) ? I'll keep the vigil per your standing instruction, waking on the next PR / force-push / comment whenever the author resumes.

Two small things worth your call:

- **\*\*If you're wrapping up for now\*\***, just say the word and I'll stop the watch (no point polling an idle repo indefinitely). Otherwise it keeps running in ~60-min windows and I re-arm each time.
- **\*\*Open follow-ups I noted but that are the author's to act on\*\*** (none block what's merged): the F5 deep-inspection *\*fetcher/budget wiring\** (where my "enabling `deep\_inspect` drops all videos" and serialization notes go live), and the *`/set`* config commands that make F2?F5's dormant stages user-reachable (where the malformed-config postures matter). I'll scrutinize those hard whenever they show up.

Everything's merged, *`main`* is clean and synced, and the session's LGTMs were all verification-gated. I'll surface the next activity the moment it lands ? or stand down on your signal.

### 735. user (2026-07-07T02:39:24.413Z)

```
<task-notification>
<task-id>bur02c0dk</task-id>
<tool-use-id>toolu_01H2oyYN9vQ8Wd9y14Atqh8K</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claudc\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bur02c0dk.output</output-file>
<status>completed</status>
<summary>Background command "Re-arm watcher after first idle window" completed (exit code 0)</summary>
</task-notification>
```

### 736. assistant (2026-07-07T02:39:38.113Z)

The watcher fired. Let me check whether the author resumed or it's another idle window.

### 737. user (2026-07-07T02:39:40.114Z)

```
poll 24/30: no change (02:27:19Z)
poll 25/30: no change (02:29:20Z)
poll 26/30: no change (02:31:21Z)
poll 27/30: no change (02:33:22Z)
poll 28/30: no change (02:35:22Z)
poll 29/30: no change (02:37:23Z)
poll 30/30: no change (02:39:24Z)
=== window elapsed (~60 min), no activity ===
```

#### 738. assistant (2026-07-07T02:40:09.464Z)

Second full idle window ? ~2 hours now with no activity since #57 merged. The author has clearly wrapped for the session. Rather than keep pinging you every hour with "still idle," let me widen the watch: same responsive 120s poll (so a new PR is caught within ~2 min), but a much longer window so idle-window-end doesn't wake me hourly. It still fires immediately on any real activity.

#### 739. user (2026-07-07T02:40:17.629Z)

Command running in background with ID: b491b9qb2. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claudu\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b491b9qb2.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 740. assistant (2026-07-07T02:40:30.378Z)

### Status ? repo quiet ~2h; watch continues quietly

Second idle window elapsed with no activity, so the author has wrapped for the session. I've re-armed the watcher (`b491b9qb2`) with an **extended ~8-hour window** ? it still catches any new PR / force-push / comment within ~2 minutes, but it won't wake me hourly just to report "still idle." So you'll only hear from me again when there's **real activity** (or after a long quiet stretch).

**Where things stand ? everything's landed and clean:**

- The entire provider-neutral umbrella (Tracks B?F) and the full ADR 0012 policy pipeline (design ? F1?F7) are reviewed, verified, and merged.
- `main` is synced; no open PRs; no outstanding review actions on my side.
- Session tally: ~13 PRs reviewed, every finding reproduced locally, LGTM strictly gated on verified fixes, and my two authored fixes (#36 format, #41 watched-clobber) merged.

If you're done for now, you can leave the watch running (it's cheap and silent) or tell me to **stand down** and I'll stop it. Either way, I'll surface the next PR the moment the author resumes ? and go straight into the sync ? gate ? adversarial-pass ? verify ? only-LGTM-when-satisfied cycle. ?

#### 741. user (2026-07-07T03:32:39.872Z)

```
<task-notification>
<task-id>b491b9qb2</task-id>
<tool-use-id>toolu_01W29mXUdaz4PJX4WdFU1Bw1</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claudu\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b491b9qb2.output</output-file>
<status>completed</status>
<summary>Background command "Re-arm watcher with extended ~8h window (repo idle)" completed (exit code 0)</summary>
</task-notification>
```

#### 742. user (2026-07-07T03:32:45.684Z)

```
=== PR-watch v2 ARMED ? LONG window (~8h; repo idle 2h). Responsive to activity within ~2min; won't wake me hourly for idle. ===
baseline: <none> | main: 7fadffb feat(commands): add why explainability command (#57)
=== PR ACTIVITY DETECTED (poll 26, 03:32:39Z) ===
```

```

--- baseline ---
<none>
--- now ---
#58 draft=false sha=3069f1801 upd=2026-07-07T03:31:38Z
--- new/changed (sha change=force-push/fix; only upd change=comment) ---
#58 draft=false sha=3069f1801 upd=2026-07-07T03:31:38Z
--- open PR details ---
#58 draft=false by avidullu: docs: refresh current project documentation

```

#### 743. assistant (2026-07-07T03:32:56.866Z)

The author resumed ? \*\*new PR #58: "docs: refresh current project documentation"\*\*\* (docs-only, updating the docs to reflect the completed Track B?F work). Since I've reviewed all of B?F, I'm well-placed to check the docs \*\*against the actual merged code\*\* ? accuracy is the main risk for a docs refresh. Let me pull it up.

#### 744. user (2026-07-07T03:33:05.882Z)

```

* [new branch]          agent/docs-sweep -> origin/agent/docs-sweep
=== PR #58 META ===
{"additions":782,"body":"## Summary\n\n- Refresh README, design, plan, and runbook around the current post-F7 codebase.\n- Document opt-in live source scanning, current command surface, delivery modes/targets, policy stages, `policy_decisions`, and `/why` behavior.\n- Add implementation-status notes to stale ADRs for inline commands, selective scanning, identity/session split, provider-neutral schema, and the ADR 0012 F1-F7 pipeline.\n- Fix stale source-facing docs/help text: exclusion defaults in `/show commands`, `FilterConfig` storage docstring, Dockerfile entrypoint comment, and package description.\n\n## Why\n\nThe docs still described the early v1 flow in several places, including automatic all-channel scanning, old tables, future policy phases, and outdated exclusion defaults. This sweep makes the public docs, runbook, ADR status notes, and source-level documentation line up with the merged implementation.\n\n## Validation\n\n- `\\.\.\.venv\\Scripts\\python.exe -m ruff check src tests`\n- `\\.\.\.venv\\Scripts\\python.exe -m pytest` (`562 passed`)", "changedFiles":14, "deletions":787, "headRefName":"agent/docs-sweep", "isDraft":false, "mergeable":"MERGEABLE", "number":58, "title":"docs: refresh current project documentation"}

```

```

=== files ===
Dockerfile
README.md
docs/adr/0007-inline-config-commands.md
docs/adr/0009-selective-channel-scanning.md
docs/adr/0010-identity-and-session-separation.md
docs/adr/0011-provider-neutral-domain-schema.md
docs/adr/0012-policy-pipeline.md
docs/design.md
docs/plan.md
docs/runbook.md
pyproject.toml
src/tg_video_filter/command_service.py
src/tg_video_filter/models.py
tests/test_commands.py
1904 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr58.diff

```

#### 745. assistant (2026-07-07T03:33:30.344Z)

PR #58 (docs sweep): +782/?787, 14 files ? mostly docs, but with \*\*source changes\*\* (help text in `command\_service.py`, a `models.py` docstring, `Dockerfile`, `pyproject`, and a `test\_commands.py` update). Accuracy vs the merged code is the main risk ? and I can check that directly from my B?F review. Notably it claims to fix "exclusion defaults in `/show commands`" (the old text said "all off by default", but `exclude\_gif`/`exclude\_round` default \*\*True\*\*). Let me start the gate and read the source changes + key doc claims.

## 746. user (2026-07-07T03:33:37.160Z)

```
HEAD is now at 3069f18 docs: refresh current project documentation
gate started
=== SOURCE + config changes (non-docs) ===
diff --git a/Dockerfile b/Dockerfile
index 9f4ffa3..a0c882b 100644
--- a/Dockerfile
+++ b/Dockerfile
@@ -25,6 +25,5 @@ ENV DB_PATH=/data/app.db \
    SESSIONS_DIR=/data/sessions \
    LOG_LEVEL=INFO

-# Run the manager entrypoint (Phase 4). For now this will fail fast until
-# __main__.py lands; that's expected at the Phase 0 stage.
+# Run the manager entrypoint.
CMD ["python", "-m", "tg_video_filter"]
diff --git a/pyproject.toml b/pyproject.toml
index 14ea202..83c4aad 100644
--- a/pyproject.toml
+++ b/pyproject.toml
@@ -5,7 +5,7 @@ build-backend = "hatchling.build"
[project]
    name = "tg-video-filter"
    version = "0.1.0"
-    description = "Telethon userbot that filters Telegram channel videos by duration, resolution,
+    description = "Telethon userbot that filters selected Telegram video sources, delivers matches,
    and forwards matches to Saved Messages."
+    description = "Telethon userbot that filters selected Telegram video sources, delivers matches,
    and explains policy decisions."
    readme = "README.md"
    requires-python = ">=3.11"
    license = { text = "TBD" }
diff --git a/src/tg_video_filter/command_service.py b/src/tg_video_filter/command_service.py
index 0ab172a..79fef65 100644
--- a/src/tg_video_filter/command_service.py
+++ b/src/tg_video_filter/command_service.py
@@ -150,11 +150,11 @@ def format_commands() -> str:
    "`/set min resolution <N>p` ? Set min video height (e.g. `/set min resolution
    720p`)",
    "    or `/set min res <N>p` ? same as above",
    "",
-    "? **Content exclusions** (all off by default)",
+    "? **Content exclusions**",
    "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
    "`/set exclude round on|off` ? Exclude circular video messages",
    "`/set exclude nosound on|off` ? Exclude videos without audio\n"
-    "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs too.",
+    "    Defaults: GIF/round excluded, nosound allowed.",
    "",
    "? **Backfill**",
    "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history (default
    90d)",
diff --git a/src/tg_video_filter/models.py b/src/tg_video_filter/models.py
index fd4c85e..5c36b88 100644
--- a/src/tg_video_filter/models.py
+++ b/src/tg_video_filter/models.py
@@ -149,9 +149,10 @@ class Source(BaseModel):
    class ContentItem(BaseModel):
        """A normalized content record discovered from any :class:`Source` (ADR 0011).

-        Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
-        (`content_type='video'`); `text` / `link_url` are landed now but stay
-        NULL until later tracks populate them (avoids a future `ALTER TABLE`).
+        Generalizes the legacy :class:`VideoRecord`. The Telegram path stores videos
+        today (`content_type='video'`) and captures caption text; `link_url` is
```

```

+ available for future link/article content while URL policy stages currently
+ extract links from captured text at evaluation time.
  ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
  wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
  """
@@ -198,9 +199,7 @@ class Policy(BaseModel):
    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    account_id: int = Field(description="FK ? accounts.id")
    name: str = Field(description="Policy name; 'default' for the account default policy")
-   config_json: str = Field(
-       description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
-   )
+   config_json: str = Field(description="FilterConfig JSON blob plus ADR 0012 policy keys")
    is_default: bool = Field(
        default=False, description="Exactly one per account (UNIQUE partial index)"
    )
@@ -313,11 +312,12 @@ class PolicyDecision(BaseModel):

class FilterConfig(BaseModel):
-   """Per-user filter configuration.
+   """Flat per-account filter configuration.

-   v1 rules: duration bounds, minimum height (resolution), and max file size.
-   Stored in the DB as a JSON blob (``filter_config.config_json``) so adding a
-   new field later is backwards-compatible.
+   Stored inside the account's default ``policies.config_json`` object through
+   compatibility shims. Legacy ``filter_config`` callers still work, but new
+   policy keys can live alongside these flat fields without being erased by
+   ``/set`` command updates.
    """

    # Duration bounds in seconds. None means unbounded on that side.
diff --git a/tests/test_commands.py b/tests/test_commands.py
index 162c97b..7ff316d 100644
--- a/tests/test_commands.py
+++ b/tests/test_commands.py
@@ -1101,7 +1101,7 @@ def test_format_commands_excludes_exclude() -> None:
    assert "/set exclude gif" in result
    assert "/set exclude round" in result
    assert "/set exclude nosound" in result
-   assert "off by default" in result
+   assert "GIF/round excluded, nosound allowed" in result

# --- regex: /set min resolution -----

```

#### 747. assistant (2026-07-07T03:33:50.638Z)

The source changes are all **doc-accuracy fixes** ? and the key one is real: the old `/show commands` said "Content exclusions (all off by default)", but `exclude_gif/exclude_round` default **True**. The new text "GIF/round excluded, nosound allowed" is correct, and `test_commands.py` is updated to match. Let me confirm the gate and verify the defaults claim, then read the doc sections for accuracy.

#### 748. user (2026-07-07T03:33:57.568Z)

```

=== gate ===
=== RUFF ===
All checks passed!
36 files already formatted
=== PYTEST ===
TOTAL                                1772      77    508     53    94%
Required test coverage of 80.0% reached. Total coverage: 94.30%
562 passed in 9.56s

```

DONE

```
=== verify FilterConfig defaults match the new help text ===
exclude_gif=True exclude_round=True exclude_nosound=False
help says "GIF/round excluded, nosound allowed" -> matches: True
```

#### 749. assistant (2026-07-07T03:34:11.266Z)

Verified: gate green (562 @ 94.30%), and the exclusion-defaults fix is **accurate** (``gif=True, round=True, nosound=False`` ? "GIF/round excluded, nosound allowed" ?; the old "all off by default" was a real doc bug). Now the docs themselves ? accuracy is the risk, and I can cross-check against the merged code. Given the breadth (README + design + plan + runbook + 5 ADR status notes), let me fan out a workflow to adversarially verify doc claims against the code, while I read the README myself.

#### 750. user (2026-07-07T03:34:12.986Z)

```
=== README diff ===
diff --git a/README.md b/README.md
index 80de756..d4c6ecc 100644
--- a/README.md
+++ b/README.md
@@ -1,75 +1,148 @@
 # telegram-video-filter

-A Telethon-based userbot that logs in as your own Telegram account, watches
-new video messages across all the channels/groups you subscribe to, applies
-configurable filters (duration, resolution, file size), and forwards matching
-videos to your Saved Messages in real time.
-
-> ?? This project automates a user account (a "userbot") via MTProto. Keep
-> volumes reasonable and respect Telegram's Terms of Service. You are
-> responsible for your own account. See [`docs/adr/0001-userbot-over-botapi.md`](docs/adr/0001-userbot-over-botapi.md).
-
-## Status
-
-? v1 complete ? all 6 build phases merged via PR. The app runs end-to-end:
-bootstrap a user account ? manager watches their channels ? filters videos ?
-forwards matches to Saved Messages. See [`docs/plan.md`](docs/plan.md) for the
-build history and [`docs/runbook.md`](docs/runbook.md) to deploy.
-phased build plan and current status.
-
-## What it does (v1)
-
-- Watches every channel/group your account is a member of ? no per-channel setup.
-- Reads video metadata without downloading the file (duration, width/height,
- size, MIME).
-- Applies an AND of your filters, e.g.:
- - duration between 5s and 2h,
- - height ? 1080 (1080p or better),
- - size ? 2 GiB.
-- Forwards each match to your Saved Messages in real time, or to a
- channel you own if you set `delivery_target` (per-user).
-- Never forwards the same video twice ? three-tier content dedup
- (message instance ? Telegram `document.id` ? metadata fingerprint) means
- a video cross-posted to 5 channels is delivered once. See
- [`docs/adr/0006-content-dedup.md`](docs/adr/0006-content-dedup.md).
-- Supports multiple users (<100) on a single host, each with its own session,
- filters, delivery mode/target, and dedup state.
-
-## What it does NOT do yet (v2+)
-
-- Public-channel `/add @channel` join, manual-add ingestion.
-- Scheduled digest delivery and on-demand `/list` / `/search`.
```

```

-- ffpb-based deep inspection (`/inspect`).
-- Codec / keyword / MIME filters.
+A Telethon-based Telegram **userbot** that logs in as your own account, filters
+video messages from selected sources, and delivers matches to Saved Messages or
+another Telegram destination you control.
+
+The app is metadata-first: it reads Telegram message metadata and captions
+without downloading media for the normal path. It is built for a small
+multi-user deployment on one host, with SQLite state and file-backed Telethon
+sessions under `data/`.
+
+> This project automates a **user** account via MTProto. Keep volumes
+> reasonable and respect Telegram's Terms of Service. Session files are
+> credentials. See [ADR 0001](docs/adr/0001-userbot-over-botapi.md).
+
+### Current Status
+
+The main userbot path is implemented end to end:
+
+- Bootstrap one or more Telegram user sessions.
+- Discover Telegram channels/groups as sources.
+- Opt in live ingestion per source with `/watch`.
+- Filter video metadata with duration, resolution, size, GIF/round/nosound
+  exclusions, and the ADR 0012 policy pipeline.
+- Deliver accepted videos by forward, link, or both.
+- Persist content, delivery state, policy decisions, and explanation traces.
+- Explain historical decisions with `/why <content_item_id>`.
+
+The original v1 plan and the ADR 0012 F1-F7 policy pipeline are complete. The
+current follow-up is policy-decision retention cleanup; see issue #56.
+
+### Core Features
+
+- **Selective live scanning**: unknown sources are discovered as `watched=false`.
+  Use `/channels`, `/watch <id|@name>`, `/unwatch <id|@name>`, and `/watchlist`
+  from Saved Messages to control live ingestion.
+- **Metadata filters**: duration bounds, minimum height, maximum size, GIF,
+  round-video, and nosound exclusions.
+- **Policy stages**: topic preferences, captured-caption safety checks,
+  captured URL/domain reputation checks, and an opt-in deep-inspection boundary.
+  Deep inspection is budgeted but no production fetcher is wired by default.
+- **Deduplication**: provider-neutral dedup keys catch same message, same
+  Telegram `document.id`, and same metadata fingerprint.
+- **Delivery modes**: `forward`, `link`, or `both`, targeting `"me"`,
+  `@username`, or a numeric Telegram chat id.
+- **Backfill**: `/backfill [N][h|d|w]` scans historical channel/group messages.
+  Backfill is intentionally broad and should be run carefully on large accounts.
+- **Explainability**: `/why <content_item_id>` renders the stored policy trace
+  when available, or a clearly caveated current metadata replay for older
+  items without a policy-decision row.
+
+### What Is Not Built
+
+- Bot API onboarding/front-door commands from ADR 0008.
+- Public-channel auto-join or manual `/add @channel` ingestion.
+- Scheduled digests and on-demand search/list commands.
+- A production deep-inspection fetcher/downloader.
+- Policy-decision retention cleanup.
+- Horizontal scaling across multiple hosts.
+
+### Quick Start
+
+## Documentation
+```bash
+cp .env.example .env

```

```

+# edit .env with TG_API_ID and TG_API_HASH from https://my.telegram.org
+
+docker compose up -d --build
+python -m tg_video_filter.bootstrap
+docker compose restart app
+```
+
+After startup, send commands to your own Saved Messages:
+

```

#### 751. assistant (2026-07-07T03:34:34.886Z)

The README rewrite is **accurate and high-quality** ? it correctly captures the nuanced current state: opt-in scanning (``watched=false`` default, post-#41), the policy stages with "no production fetcher wired by default" (my F5 note), and "policy-decision retention cleanup" as not-built (my F6 TTL note, tracked as #56). That's a good sign. Given the breadth of remaining docs (design, plan, runbook, 5 ADR status notes), let me fan out a workflow to cross-check every doc file's claims against the merged code.

#### 752. user (2026-07-07T03:35:16.464Z)

Workflow launched in background. Task ID: w3glb28kq  
Summary: Review telegram-video-filter PR #58 (docs refresh) ? verify every doc claim against the merged code, with adversarial verification  
Transcript dir: C:\Users\avidu\claudes\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_0febe894-ff9  
Script file: C:\Users\avidu\claudes\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr58-docs-wf\_0febe894-ff9.js  
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\claudes\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr58-docs-wf\_0febe894-ff9.js"} to iterate without resending the script.)  
Run ID: wf\_0febe894-ff9  
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\claudes\projects\C--Users-avidu-Projects-Telegram\7dc3da51-af38-47b6-83f6-fdda83b60664\workflows\scripts\review-pr58-docs-wf\_0febe894-ff9.js", resumeFromRunId: "wf\_0febe894-ff9"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

#### 753. assistant (2026-07-07T03:35:22.450Z)

While the workflow verifies the doc claims, let me spot-check the runbook myself ? a wrong deploy/operational step is the highest-severity risk in a docs refresh.

#### 754. user (2026-07-07T03:35:29.370Z)

```

=== runbook diff (deploy/ops steps) ===
+- Docker + Docker Compose, or Python 3.11+.
+- Optional local virtualenv for scripts/tests:
+python -m venv .venv
+.\venv\Scripts\python.exe -m pip install -e .[dev]
+# edit .env: TG_API_ID, TG_API_HASH, DB_PATH, SESSIONS_DIR
+Expected first log state: the manager starts with no runnable sessions until a
+user is bootstrapped.
+Run the bootstrap command in the same environment that can write to `data/`:
+- `telegram_sessions` row,
+- `data/sessions/user_{id}.session`.
+not ingested until you watch them. Send commands to your own Saved Messages:
+/watch <id|@name|title-fragment>
+/watchlist
+1. Let new traffic arrive, or run `/backfill` to discover historical dialogs.
+3. Run `/watch -1001234567890` or `/watch Some Channel`.

```



```

+4. Confirm with `/watchlist`.
+## 5. Filter Commands
+These commands persist config changes and hot-reload the in-memory filter
+/show commands
+/set size 10MB
+/set length 15m
+/set max length 15m
+/set min length 5s
+/set min resolution 720p
+/set min res 720p
+/set exclude gif on
+/set exclude round off
+/set exclude nosound on
+Saved-Messages command for changing them yet, so update deliberately and restart
+.\.venv\Scripts\python.exe scripts\set_delivery_target.py <account_id> <target>
+.\.venv\Scripts\python.exe scripts\resolve_target.py <account_id> <target>
+docker compose restart app
+/backfill
+/backfill 8h
+/backfill 7d
+/backfill 2w
+Flat `/set` commands update `FilterConfig` fields inside the default policy
+Policy-stage config currently has no friendly command UI. Edit
+Use `/why <content_item_id>` from Saved Messages.
+`/why` behavior:

=== reality check: entrypoints + bootstrap + env exist? ===
C:/Users/avidu/Projects/_wt-pr58/src/tg_video_filter/__main__.py:25:def main() -> int: #
pragma: no cover - entrypoint glue (asyncio.run + logging)
C:/Users/avidu/Projects/_wt-pr58/src/tg_video_filter/__main__.py:45:if __name__ == "__main__":
C:/Users/avidu/Projects/_wt-pr58/src/tg_video_filter/bootstrap.py:152:def main() -> int: #
pragma: no cover - interactive entrypoint
C:/Users/avidu/Projects/_wt-pr58/src/tg_video_filter/bootstrap.py:171:if __name__ == "__main__":
--- bootstrap module runnable? ---
171:if __name__ == "__main__":
--- env vars in config ---
19: All fields map 1:1 to env vars (case-insensitive). ``TG_API_ID`` is the
20: Telegram application id; ``TG_API_HASH`` its secret. Both come from

```

#### 755. assistant (2026-07-07T03:35:41.497Z)

The runbook's entrypoints (`python -m tg\_video\_filter.bootstrap`, `python -m tg\_video\_filter`), env vars, and command surface (`/watch`, `/why`, `/backfill`, `/set ?`) all match the code. One thing to verify ? the runbook tells operators to run `scripts/set\_delivery\_target.py` and `scripts/resolve\_target.py`; a referenced-but-missing script would be a high-severity broken step.

#### 756. user (2026-07-07T03:35:45.571Z)

```

=== do the runbook-referenced scripts exist? ===
total 12
drwxr-xr-x 1 avidu 197609  0 Jul  7 09:03 .
drwxr-xr-x 1 avidu 197609  0 Jul  7 09:03 ..
-rw-r--r-- 1 avidu 197609 4014 Jul  7 09:03 resolve_target.py
-rw-r--r-- 1 avidu 197609 2698 Jul  7 09:03 set_delivery_target.py

=== docker-compose service name 'app' (runbook says 'docker compose restart app')? ===
3:services:
4:  app:
5:    build: .
6:    image: tg-video-filter:latest
7:    container_name: tg-video-filter
8:    restart: unless-stopped
9:    env_file:
11:   volumes:

```

17: environment:

=== .env.example has the env vars the runbook lists? ===

3: TG\_API\_ID=123456

4: TG\_API\_HASH=0123456789abcdef0123456789abcdef

8: DB\_PATH=data/app.db

11: SESSIONS\_DIR=data/sessions

### 757. assistant (2026-07-07T03:35:57.645Z)

All runbook operational references are accurate: `scripts/set\_delivery\_target.py` + `resolve\_target.py` **exist**, the compose service is named `app` (so `docker compose restart app` is valid), and `.env.example` has the listed vars. So my spot-checks ? README, source edits, exclusion-defaults fix, runbook deploy steps ? are **all accurate**. This is a high-quality docs refresh.

The workflow is cross-checking the remainder (design.md, plan.md, all 5 ADR status notes). I'll fold in its results before deciding, per your rule. Awaiting it.

### 758. user (2026-07-07T03:44:29.256Z)

```
<task-notification>
<task-id>w3glb28kq</task-id>
<tool-use-id>toolu_01Y9EEanvevVAEG1dcVjF4X1</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claudex\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\w3glb28kq.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Review telegram-video-filter PR #58 (docs refresh) ? verify every doc claim against the merged code, with adversarial verification" completed</summary>
<result>{"counts":{"raw":3,"kept":3,"refuted":0},"findings":[{"dimension":"plan_runbook","severity":"high","verdict":"CONFIRMED","title":"Runbook §6 recommends set_delivery_target.py, which errors on every live DB (writes to dropped filter_config table)","where":"docs/runbook.md:151","summary":"The runbook's §6 'Delivery Mode and Target' section tells operators to change the delivery target with `scripts\\set_delivery_target.py &lt;account_id&gt; &lt;target&gt;`, but that script issues `UPDATE filter_config ... WHERE user_id = ?` against a table that no longer exists on any migrated database, so the documented step fails hard.","why_wrong":"Under ADR 0011 (migration 002/003) the legacy `filter_config` table is renamed to `filter_config_deprecated` and init_db() explicitly DROPS any recreated `filter_config` (db.py:230); delivery config now lives in `policies.config_json` and is read via the load_filter_config shim (db.py:1887-1899). Every live DB is at user_version=5, so `filter_config` is gone. The helper script (scripts/set_delivery_target.py:47-51, 64) still runs raw SQL against `filter_config`. An operator following the runbook to change delivery mode/target cannot do so through the documented path.","evidence":"Reproduced against a fresh migrated DB (user_version=5): the script's exact SQL `UPDATE filter_config SET config_json=json_set(..., '$.delivery_target',?) WHERE user_id=?` raises `sqlite3.OperationalError: no such table: filter_config`. Table listing after init_db: filter_config present? False; filter_config_deprecated present? True; policies present? True; user_version: 5. Script source: scripts/set_delivery_target.py:47-51. Runbook instruction: docs/runbook.md:151-154 ('Use the helper script for targets: ...set_delivery_target.py &lt;account_id&gt; &lt;target&gt;').","recommendation":"Fix the helper script to update `policies.config_json` for the default policy (matching db.save_filter_config: get_default_policy + save_policy_config, WHERE account_id=? AND is_default=1) instead of `filter_config`. Until the script is fixed, the runbook should not present it as the way to change the delivery target; document the working `UPDATE policies SET config_json=json_set(..., '$.delivery_target',?) WHERE account_id=? AND is_default=1` SQL (parallel to the §8 policy-editing examples) instead. Note: `resolve_target.py` is unaffected (it uses only session files), and the `&lt;account_id&gt;` vs script's `&lt;user_id&gt;` label difference is harmless since accounts.id == telegram user id == session-file id (bootstrap.py:65).","verify_reasoning":"The PR's new §6 \"Delivery Mode and Target\" (docs/runbook.md:151-154) tells operators to change the delivery target with `\\.\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py &lt;account_id&gt; &lt;target&gt;`. That script (scripts/set_delivery_target.py:47-51) executes `UPDATE filter_config SET config_json=json_set(config_json, '$.delivery_target',?) WHERE user_id=?`. Under ADR 0011 the legacy `filter_config` table is renamed to `filter_config_deprecated` (db.py:679) and init_db() drops any recreated `filter_config` once migration 003 has run
```

(db.py:227-230). Delivery config now lives in policies.config\_json; save\_filter\_config merges flat keys into the default policy via get\_default\_policy/save\_policy\_config (db.py:1902-1925). Reproduced against a freshly initialized DB (user\_version=5): filter\_config exists? False; filter\_config\_deprecated exists? True; policies exists? True; and the script's exact SQL raises `sqlite3.OperationalError: no such table: filter\_config`. So the documented operator step fails hard on every migrated/current DB. The claim genuinely ships in this diff ? the PR re-presents the broken script as the way to change the delivery target in the newly written \$6 (diff line 212 \"Use the helper script for targets\"). The resolve\_target.py reference is unaffected (session-file only), and the <account\_id> vs script's <user\_id> label difference is harmless (accounts.id == telegram user id). This is a wrong actionable operational step an operator would follow and be broken by: severity high is correct.\"},{\"dimension\":\"readme\_design\", \"severity\":\"low\", \"verdict\":\"CONFIRMED\", \"title\":\"design.md lists a log event `delivery\_record\_failed` that does not exist in the code\", \"where\":\"docs/design.md:219\", \"summary\":\"The new Observability section in design.md (§10) lists `delivery\_record\_failed` as an \\\"important event\\\" the app logs, but no such log string exists anywhere in src/tg\_video\_filter.\"}, {\"why\_wrong\":\"An operator relying on this doc would grep logs or build alerting on the literal token `delivery\_record\_failed` and never match anything, because the delivery layer never emits it. The real delivery-failure events are `link\_delivery\_failed\_in\_both\_mode` and `forward\_delivery\_failed\_in\_both\_mode` (delivery.py:266,268), plus `flood\_wait` (delivery.py:111) and the fallback notice `link\_mode\_fallback\_to\_forward` (delivery.py:273). Every other event named in the §10 list is real and grep-matches (`manager\_started`, `client\_connected`, `video\_seen`, `video\_forwarded`, `flood\_wait`, `config\_updated`, `client\_disconnected`) ? this one is the sole invented token.\"}, {\"evidence\":\"docs/design.md §10 bullet - delivery\_record\_failed`. Grep for `delivery\_record\_failed` across src/tg\_video\_filter returns no matches. Actual delivery-failure logs: delivery.py:266 `log.warning(\\\"link\_delivery\_failed\_in\_both\_mode video=%s\\\", ...) ` and delivery.py:268 `log.warning(\\\"forward\_delivery\_failed\_in\_both\_mode video=%s\\\", ...)`.\", \"recommendation\":\"Replace `delivery\_record\_failed` with the events the code actually emits (e.g. `link\_delivery\_failed\_in\_both\_mode`, `forward\_delivery\_failed\_in\_both\_mode`, `link\_mode\_fallback\_to\_forward`, `link\_sent`) or drop the line, so the observability list matches grep-able log tokens.\"}, {\"verify\_reasoning\":\"The PR diff adds `+- delivery\_record\_failed` to the new §10 Observability list (docs/design.md:218 in the checked-out copy; the finding's line 219 is off by one but the token is present and clearly in-diff). Grepping `delivery\_record\_failed` across src/tg\_video\_filter returns zero matches ? the delivery layer never emits this token. Every other event in the §10 list grep-matches in the source: manager\_started (manager.py:73), client\_connected (manager.py:177), client\_disconnected (manager.py:181), video\_forwarded (delivery.py:159), video\_seen (telethon\_client.py:335), config\_updated (command\_service.py:608), and flood\_wait. So delivery\_record\_failed is the sole invented token. The actual delivery-failure events are link\_delivery\_failed\_in\_both\_mode (delivery.py:266) and forward\_delivery\_failed\_in\_both\_mode (delivery.py:268), plus the fallback notice link\_mode\_fallback\_to\_forward (delivery.py:273). An operator grepping logs or building alerting on the literal token would never match. Could not refute: the claim ships in this diff and is factually wrong vs merged code.\"}, {\"dimension\":\"adr\_notes\", \"severity\":\"low\", \"verdict\":\"CONFIRMED\", \"title\":\"design.md observability list cites a nonexistent log event `delivery\_record\_failed`\", \"where\":\"docs/design.md:214\", \"summary\":\"The new Observability section (§10) lists `delivery\_record\_failed` as an \\\"important event\\\", but no such log event exists anywhere in the codebase.\"}, {\"why\_wrong\":\"grep of src/tg\_video\_filter/\*.py finds no `delivery\_record\_failed`. The only delivery-failure log lines are `delivery\_failed` and `delivery\_retry\_failed` (delivery.py:116,119) plus `link\_delivery\_failed\_in\_both\_mode`/`forward\_delivery\_failed\_in\_both\_mode` (delivery.py:266,268). The other events in the list (`manager\_started`, `client\_connected`, `video\_seen`, `video\_forwarded`, `flood\_wait`, `config\_updated`, `client\_disconnected`) all exist; this one is fabricated. An operator following the design doc and grepping logs for `delivery\_record\_failed` would find nothing.\"}, {\"evidence\":\"src/tg\_video\_filter/delivery.py:119 `log.exception(\\\"delivery\_failed video=%s\\\", video\_i ... (truncated 1706 chars, full result in C:\\Users\\avidu\\AppData\\Local\\Temp\\claude\\C--Users-avidu-Projects-badminton-highlight-indexer\\7dc3da51-af38-47b6-83f6-fdda83b60664\\tasks\\w3glb28kq.output)</result> <usage><agent\_count>6</agent\_count><subagent\_tokens>402429</subagent\_tokens><tool\_uses>125</tool\_uses><duration\_ms>552776</duration\_ms></usage> </task-notification>

```

total 196
drwxr-xr-x 1 avidu 197609      0 Jul  6 10:47 .
drwxr-xr-x 1 avidu 197609      0 Jul  5 22:04 ..
-rw-r--r-- 1 avidu 197609 122880 Jul  6 10:47 .coverage
-rw-r--r-- 1 avidu 197609   733 Jul  3 23:33 .env
-rw-r--r-- 1 avidu 197609   730 Jul  3 22:46 .env.example
drwxr-xr-x 1 avidu 197609      0 Jul  6 10:49 .git
-rw-r--r-- 1 avidu 197609   827 Jul  3 22:36 .gitignore
drwxr-xr-x 1 avidu 197609      0 Jul  4 08:36 .mypy_cache
-rw-r--r-- 1 avidu 197609   645 Jul  3 22:46 .pre-commit-config.yaml
drwxr-xr-x 1 avidu 197609      0 Jul  3 22:43 .pytest_cache
drwxr-xr-x 1 avidu 197609      0 Jul  4 08:36 .ruff_cache
drwxr-xr-x 1 avidu 197609      0 Jul  3 22:41 .venv
drwxr-xr-x 1 avidu 197609      0 Jul  4 15:06 data
-rw-r--r-- 1 avidu 197609   662 Jul  3 22:46 docker-compose.yml
-rw-r--r-- 1 avidu 197609  1005 Jul  4 00:39 Dockerfile
drwxr-xr-x 1 avidu 197609      0 Jul  6 10:23 docs
-rw-r--r-- 1 avidu 197609   3373 Jul  4 00:47 pyproject.toml
-rw-r--r-- 1 avidu 197609   3354 Jul  4 14:26 README.md
drwxr-xr-x 1 avidu 197609      0 Jul  4 14:26 scripts
drwxr-xr-x 1 avidu 197609      0 Jul  3 22:40 src
drwxr-xr-x 1 avidu 197609      0 Jul  6 10:23 tests
---TELEGRAM GIT---
origin https://github.com/avidullu/telegram-video-filter.git (fetch)
origin https://github.com/avidullu/telegram-video-filter.git (push)
---BADMINTON---
total 13
drwxr-xr-x 1 avidu 197609      0 Jul  2 11:41 .
drwxr-xr-x 1 avidu 197609      0 Jul  5 22:04 ..
drwxr-xr-x 1 avidu 197609      0 Jul  2 22:08 .claude
-rw-r--r-- 1 avidu 197609   600 Jul  2 11:41 AGENTS.md
drwxr-xr-x 1 avidu 197609      0 Jun 30 03:00 output

```

#### 760. user (2026-07-06T05:54:37.852Z)

```

---PROJECTS DIR---
_p3_scratch
Agent Sessions
badminton-highlight-indexer
khelsutra-guru
muneem-ai
Telegram
Vairagi
---gh version---
gh version 2.92.0 (2026-04-28)
---gh auth---
github.com
? Logged in to github.com account avidullu (keyring)
- Active account: true
- Git operations protocol: https
- Token: gho_*****
- Token scopes: 'gist', 'read:org', 'repo', 'workflow'

```

#### 761. user (2026-07-06T05:54:55.878Z)

```

---Telegram memory dir---
---handoff root---
---any project memory dirs---

```

#### 762. user (2026-07-06T05:55:32.455Z)

```

===PR #35 META===
{"additions":1299,"body":"## What\n\nImplements **Track B (#27)** of the provider-neutral umbrella (#25), following ADR 0011 (Option 1A: tiered `dedup_key`; Option 2A: `policies`

```

generalizes `filter\_config`), accepted via PR #34.\n\nThis is the implementation PR promised in #34's body once the two decisions landed.\n\n## Schema (6 new tables)\n\n| Table | Populated by Track B? | Purpose |\n|---|---|---|\n| `sources` | ? backfilled from `channels` | Generalizes channels (`provider`, `provider\_source\_id`, `watched`) |\n| `content\_items` | ? backfilled from `videos` | Generalizes videos (`dedup\_key`, `provider\_message\_id`) |\n| `policies` | ? backfilled from `filter\_config` | Generalizes filter\_config (`name`, `is\_default`, `config\_json`) |\n| `destinations` | ? empty | Track C (#28) populates |\n| `routing\_rules` | ? empty | Track C populates |\n| `delivery\_attempts` | ? empty | Track C populates |\n\nThe three empty tables are the \*\*contract Track C codes against\*\* ? landed now so the ADR is complete and Track C doesn't need a migration.\n\n## Dedup (Option 1A ? tiered lookup)\n\nPure `compute\_dedup\_key(provider, \*\*metadata)` implements the decision tree:\n\n1. `document\_id` set ? `\"tg:doc:{id}\"` (Tier-1)\n2. all four fingerprint fields set ? `\"tg:fp:{size:duration:width}\"` (Tier-2)\n3. else ? `\"tg:msg:{source\_id}:{message\_id}\"` (Tier-0 fallback)\n\nEach branch is \*\*guarded\*\* so partial metadata never collapses to a degenerate key (e.g. `tg:fp:0:0:0:0`) ? the ADR 0011 review fix. `UNIQUE(account\_id, dedup\_key)` replaces the old Tier-1 `document\_id` index.\n\n## Migration `\_003\_domain\_schema` (idempotent, integer-preserving)\n\n1. Backfill `sources` from `channels` (`watched=1` per ADR 0009 backfill).\n2. Backfill `content\_items` from `videos`, computing `dedup\_key` in SQL via the 3-branch `CASE`.\n3. Backfill `policies` from `filter\_config` (one `default` policy per account, `is\_default=1`).\n4. Rename `channels`/`videos`/`filter\_config` ? `\*\_deprecated` (one-release rollback net, ADR 0010 pattern).\n\n\*\*Idempotency fix:\*\* `\_apply\_schema` drops empty legacy-table recreations on subsequent `init\_db` runs (the `\_SCHEMA` `CREATE TABLE IF NOT EXISTS` recreates them after `user\_version` is bumped, so the migration never re-runs to clean them up).\n\n## Backwards-compat shims (Track-A pattern)\n\nLegacy repos rewritten as shims over the new tables ? `telethon\_client.py`, `backfill.py`, `commands.py` keep working without a flag day:\n\n| Legacy function | Now reads/writes |\n|---|---|\n| `upsert\_channel` / `get\_channel` | `sources` |\n| `get\_video` / `get\_video\_by\_\*` / `insert\_video` | `content\_items` (+ `compute\_dedup\_key`) |\n| `load\_filter\_config` / `save\_filter\_config` | `policies` (default policy) |\n| `mark\_forwarded` | no-op (delivery state ? Track C) |\n\n`mark\_forwarded` is a no-op because `content\_items` has no `forwarded` column ? per-destination delivery state moves to `delivery\_attempts` in Track C. For Phase 1 this is safe: forwards are idempotent, and `matched` (set at insert time) is the surviving signal.\n\n## Verification\n\n`n434` passed @ 96.63% coverage (gate: 80%)\n\nruff check . ? clean\n\nruff format --check ? clean (on changed files)\n\n`n17` new tests:\n\n- \*\*`compute\_dedup\_key` matrix (9 cases):\*\* all three branches, partial-metadata guard (missing `width` ? Tier-0, not degenerate `tg:fp`), empty-string `document\_id`, case-insensitive provider prefix, future-provider prefix, missing-required-fields `ValueError`.\n- \*\*Source/ContentItem/Policy repos:\*\* round-trip, conflict-update, dedup\_key uniqueness enforcement, default-policy lookup.\n- \*\*Migration 003 backfill:\*\* seeds rows hitting all three dedup branches on a legacy DB, asserts no two rows collapse to the same `dedup\_key`.\n- \*\*Idempotency:\*\* second `init\_db` is a no-op.\n\n## Scope discipline\n\n- ? \*\*Creates:\*\* `sources`, `content\_items`, `policies` (+ backfill).\n- ? \*\*Creates empty:\*\* `destinations`, `routing\_rules`, `delivery\_attempts`.\n- ? \*\*Shims:\*\* legacy `channels`/`videos`/`filter\_config` repos keep working.\n- ? \*\*Not in this track:\*\* delivery logic, routing logic, command changes (Tracks C/D/E).\n\n## Closes\n\nCloses #27 (Track B). Unblocks #28 (Track C), #29 (Track D), #30 (Track E).\n\n## Related\n\n- ADR 0011 ? the accepted decision (PR #34).\n- ADR 0010 ? identity/session split (this PR's `account\_id` FKs point at `accounts.id`).\n- ADR 0006 ? content dedup (generalized by Option 1A).\n- ADR 0009 ? selective scanning + routing (`sources.watched`, `routing\_rules`).\n\n\"changedFiles\":4,\"deletions\":150,\"mergeStateStatus\":\"CLEAN\",\"mergeable\":\"MERGEABLE\",\"number\":35,\"reviewDecision\":\"\",\"statusCheckRollup\":[],\"title\":\"feat(domain): provider-neutral schema ? sources/content\_items/policies (ADR 0011, Track B)\"}

```
===FILES CHANGED===
src/tg_video_filter/db.py
src/tg_video_filter/models.py
tests/test_db.py
tests/test_delivery.py
```

763. user (2026-07-06T05:55:34.616Z)

===README===

## telegram-video-filter

A Telethon-based \*\*userbot\*\* that logs in as your own Telegram account, watches new video messages across all the channels/groups you subscribe to, applies

configurable filters (duration, resolution, file size), and **forwards** matching videos to your **Saved Messages** in real time.

> ?? This project automates a **user** account (a "userbot") via MTProto. Keep volumes reasonable and respect Telegram's Terms of Service. You are responsible for your own account. See [docs/adr/0001-userbot-over-botapi.md](docs/adr/0001-userbot-over-botapi.md).

## Status

? **v1 complete** ? all 6 build phases merged via PR. The app runs end-to-end: bootstrap a user account ? manager watches their channels ? filters videos ? forwards matches to Saved Messages. See [docs/plan.md](docs/plan.md) for the build history and [docs/runbook.md](docs/runbook.md) to deploy. phased build plan and current status.

## What it does (v1)

- Watches every channel/group your account is a member of ? no per-channel setup.
- Reads video metadata **without downloading** the file (duration, width/height, size, MIME).
- Applies an **AND** of your filters, e.g.:
  - duration between 5s and 2h,
  - height ? 1080 (1080p or better),
  - size ? 2 GiB.
- Forwards each match to your **Saved Messages** in real time, or to a **channel you own** if you set `delivery_target` (per-user).
- **Never forwards the same video twice** ? three-tier content dedup (message instance ? Telegram `document.id` ? metadata fingerprint) means a video cross-posted to 5 channels is delivered once. See [docs/adr/0006-content-dedup.md](docs/adr/0006-content-dedup.md).
- Supports multiple users (<100) on a single host, each with its own session, filters, delivery mode/target, and dedup state.

## What it does NOT do yet (v2+)

- Public-channel `/add @channel` join, manual-add ingestion.
- Scheduled digest delivery and on-demand `/list` / `/search`.
- ffpprobe-based deep inspection (`/inspect`).
- Codec / keyword / MIME filters.

## Documentation

- [docs/plan.md](docs/plan.md) ? phased build plan, scope, verification.
- [docs/design.md](docs/design.md) ? architecture, data model, control flow.
- [docs/runbook.md](docs/runbook.md) ? deploy, onboard users, rotate sessions, handle flood waits.
- [docs/adr/](docs/adr/) ? Architecture Decision Records:
  - [0001 ? Userbot over Bot API](docs/adr/0001-userbot-over-botapi.md)
  - [0002 ? SQLite over PostgreSQL](docs/adr/0002-sqlite-over-postgres.md)
  - [0003 ? Metadata-first probing](docs/adr/0003-metadata-first-probing.md)
  - [0004 ? Pluggable delivery modes](docs/adr/0004-pluggable-delivery-modes.md)
  - [0005 ? Per-user delivery target](docs/adr/0005-per-user-delivery-target.md)

## Quick start

See the [runbook](docs/runbook.md) for the full guide. In short:

```
```bash
cp .env.example .env          # set TG_API_ID, TG_API_HASH from my.telegram.org
docker compose up -d --build
python -m tg_video_filter.bootstrap # interactive one-time login per user
docker compose restart app
```

## Tech stack

Python 3.11+ · [Telethon](https://docs.telethon.dev/) · aiosqlite (SQLite) · pydantic / pydantic-settings · Docker · pytest · ruff.

## License

TBD.

```
===repo CLAUDE.md / AGENTS.md===
ls: cannot access 'CLAUDE.md': No such file or directory
ls: cannot access 'AGENTS.md': No such file or directory
ls: cannot access 'CONTRIBUTING.md': No such file or directory
---
```

### 764. user (2026-07-06T05:56:04.578Z)

```
Exit code 2
1767 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr35.diff
---CI workflows---
ls: cannot access '.github/workflows/': No such file or directory
```

### 765. user (2026-07-06T05:56:25.812Z)

```
---pre-commit---
```

**See <https://pre-commit.com> for hook docs.**

**Install once: pre-commit install**

**Run manually: pre-commit run --all-files**

```
repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
    - id: trailing-whitespace
    - id: end-of-file-fixer
    - id: check-yaml
    - id: check-toml
    - id: check-added-large-files
      args: ["--maxkb=512"]
    - id: check-merge-conflict
    - id: detect-private-key

- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.5.7
  hooks:
    - id: ruff
      args: [--fix, --exit-non-zero-on-fix]
    - id: ruff-format

---any CI anywhere---
```

```
ls: cannot access '.github': No such file or directory
---mypy config?---
```

### 766. user (2026-07-06T05:56:50.205Z)

Command running in background with ID: bzc5er02v. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-  
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bzc5er02v.output. You will be notified when  
it completes. To check interim output, use Read on that file path.

#### 767. user (2026-07-06T05:59:26.344Z)

```
=== ruff check ===
All checks passed!
ruff-check-exit=0
=== ruff format --check ===
Would reformat: src\tg_video_filter\commands.py
1 file would be reformatted, 33 files already formatted
fmt-exit=1
```

#### 768. user (2026-07-06T06:00:07.657Z)

```
=== is commands.py changed by PR? ===
(empty above = unchanged by PR ? format issue is pre-existing on main)

=== confirm on main directly ===
Would reformat: .commands_main_check.py
1 file would be reformatted
```

#### 769. user (2026-07-06T06:00:08.343Z)

```
=== backfill.py dedup section (155-235) ===
    if meta is None:
        continue

    channel_id = _peer_id(getattr(message, "peer_id", None) or 0)
    message_id = message.id
    document_id = _extract_document_id(message) # type: ignore[arg-type]

    # --- three-tier dedup (cheapest first) ---

    # Tier 0: same (user, channel, message) instance.
    existing = await db.get_video(conn, user.id, channel_id, message_id)
    if existing is not None:
        continue

    # Tier 1: same Telegram document.id (forwarded / cross-posted file).
    if document_id is not None:
        existing = await db.get_video_by_document_id(conn, user.id, document_id)
        if existing is not None:
            log.info(
                "backfill_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
                "original_channel=%s original_msg=%s",
                user.id,
                document_id,
                channel_id,
                message_id,
                existing.channel_id,
                existing.message_id,
            )
            continue

    # Tier 2: metadata fingerprint (re-uploaded identical file).
    if (
        meta.size_bytes is not None
        and meta.duration_s is not None
        and meta.height is not None
        and meta.width is not None
    ):
        existing = await db.get_video_by_content_fingerprint(
            conn,
            user.id,
            meta.size_bytes,
            meta.duration_s,
            meta.height,
```



```

        meta.width,
    )
    if existing is not None:
        log.info(
            "backfill_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
            "channel=%s msg=%s original_channel=%s original_msg=%s",
            user.id,
            meta.size_bytes,
            meta.duration_s,
            meta.height,
            meta.width,
            channel_id,
            message_id,
            existing.channel_id,
            existing.message_id,
        )
        continue

    summary["videos_scanned"] += 1
    dialog_videos += 1

    result = engine.evaluate(meta)
    record = VideoRecord(
        user_id=user.id,
        channel_id=channel_id,
        message_id=message_id,
        document_id=document_id, # type: ignore[arg-type]
        duration_s=meta.duration_s,
        width=meta.width,
        height=meta.height,
        size_bytes=meta.size_bytes,
        mime_type=meta.mime_type,
        matched=result.matched,
    )
    inserted = await db.insert_video(conn, record)

    if result.matched:
        summary["videos_matched"] += 1

```

#### 770. assistant (2026-07-06T10:32:55.787Z)

The fix is safe but there's a wrinkle: `test\_upsert\_source\_updates\_on\_conflict` (test\_db.py:994) explicitly asserts `upsert\_source` \*overwrites\* `watched` on conflict? that test \*\*encodes the buggy contract\*\*. The correct model is: `watched` is user-state, changed only via `set\_source\_watched`, never clobbered by an ingest-path upsert. So the fix is: drop `watched=excluded.watched` from the `ON CONFLICT`, update that test, and add a regression test. I'll also `COALESCE` the title (ingest's `insert\_video` passes `title=None`, nulling the channel name? same clause, same root cause).

Let me draft it in an isolated worktree off `origin/main`.

#### 771. user (2026-07-06T10:32:58.375Z)

```

origin/main:
3bfb1da feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR
0009, Track E) (#39)
branch 'fix/upsert-source-preserve-watched' set up to track 'origin/main'.
HEAD is now at 3bfb1da feat(scanning): selective source scanning ? /watch, /unwatch, /channels,
/watchlist (ADR 0009, Track E) (#39)
--- worktree branch ---
3bfb1da feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR
0009, Track E) (#39)

```

=== exact upsert\_source ON CONFLICT in the worktree ===

1104:async def upsert\_source(conn: aiosqlite.Connection, source: Source) -> Source:

#### 772. user (2026-07-06T10:33:01.209Z)

```
1104     async def upsert_source(conn: aiomysqlite.Connection, source: Source) -> Source:
1105         """Insert a source row, or update title/watched on conflict.
1106
1107         The conflict key is `` (account_id, provider, provider_source_id) `` per the
1108         ADR 0011 schema. Returns the persisted row.
1109         """
1110         await conn.execute(
1111             """INSERT INTO sources
1112                 (account_id, provider, provider_source_id, title, kind, watched,
1113                 seen_first_at, last_seen_at)
1114             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1115             ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
1116                 title=excluded.title,
1117                 watched=excluded.watched,
1118                 last_seen_at=excluded.last_seen_at""",
1119             (
1120                 source.account_id,
1121                 source.provider,
1122                 source.provider_source_id,
1123                 source.title,
1124                 source.kind,
1125                 int(source.watched),
1126                 source.seen_first_at.isoformat(),
1127                 (source.last_seen_at.isoformat() if source.last_seen_at else None),
1128             ),
1129         )
1130         await conn.commit()
1131         conn.row_factory = aiomysqlite.Row
1132         async with conn.execute(
1133             """SELECT * FROM sources
1134                 WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1135             (source.account_id, source.provider, source.provider_source_id),
1136         ) as cur:
1137             row = await cur.fetchone()
1138             assert row is not None # just upserted by the conflict key
1139             return _row_to_source(row)
1140
1141
1142     async def get_source_by_provider_id(
1143         conn: aiomysqlite.Connection, account_id: int, provider: str, provider_source_id: str
1144     ) -> Source | None:
1145         """Resolve a source by its provider-native id."""
1146         conn.row_factory = aiomysqlite.Row
1147         async with conn.execute(
1148             """SELECT * FROM sources
```

#### 773. assistant (2026-07-06T10:33:05.518Z)

Now I'll apply the fix: remove `watched=excluded.watched` (preserve user-controlled `watched`) and `COALESCE` the title so ingest doesn't null it.

#### 774. user (2026-07-06T10:33:13.939Z)

The file C:/Users/avidu/Projects/\_wt-watched-fix/src/tg\_video\_filter/db.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 775. assistant (2026-07-06T10:33:24.895Z)

Now update the conflict test to encode the corrected contract (watched preserved, title still updated), and add a dedicated regression test. Let me read the test region in the worktree.

#### 776. user (2026-07-06T10:33:25.460Z)

```

972     async def test_upsert_source_round_trip(conn: aiosqlite.Connection) -> None:
973         """Insert then read back a source via the provider-neutral repo."""
974         await _seed_user(conn)
975         src = await db.upsert_source(
976             conn,
977             Source(
978                 account_id=USER_ID,
979                 provider="telegram",
980                 provider_source_id="-100555",
981                 title="Test Channel",
982                 kind="channel",
983                 watched=True,
984             ),
985         )
986         assert src.id is not None
987         fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-100555")
988         assert fetched is not None
989         assert fetched.title == "Test Channel"
990         assert fetched.watched is True
991         assert fetched.kind == "channel"
992
993
994     async def test_upsert_source_updates_on_conflict(conn: aiosqlite.Connection) -> None:
995         """Re-upserting the same (account, provider, source_id) updates title/watched."""
996         await _seed_user(conn)
997         await db.upsert_source(
998             conn,
999             Source(account_id=USER_ID, provider="telegram", provider_source_id="-1",
1000 watched=True),
1001         )
1002         updated = await db.upsert_source(
1003             conn,
1004             Source(
1005                 account_id=USER_ID,
1006                 provider="telegram",
1007                 provider_source_id="-1",
1008                 title="Renamed",
1009                 watched=False,
1010             ),
1011         )
1012         assert updated.title == "Renamed"
1013         assert updated.watched is False
1014         # Still one row.
1015         fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1")
1016         assert fetched is not None
1017         assert fetched.watched is False

```

#### 777. user (2026-07-06T10:33:35.492Z)

The file C:/Users/avidu/Projects/\_wt-watched-fix/tests/test\_db.py has been updated successfully.  
(file state is current in your context ? no need to Read it back)

#### 778. assistant (2026-07-06T10:33:41.076Z)

Now the end-to-end regression test. Let me check test\_db.py's imports for  
`Channel`/`VideoRecord`, then add it.

#### 779. user (2026-07-06T10:33:43.562Z)

```

from tg_video_filter.models import (
    DEFAULT_FILTER_CONFIG,
    Account,
    Channel,
    ContentItem,

```

```

    DeliveryStatus,
    Destination,
    FilterConfig,
    Policy,
    Source,
    TelegramSession,
    User,
    VideoRecord,
)
--- _seed_user + USER_ID/CHANNEL_ID defs ---
26:USER_ID = 111
27:CHANNEL_ID = -1001234567890
39:async def _seed_user(conn: aiosqlite.Connection, uid: int = USER_ID) -> User:
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

```

#### 780. assistant (2026-07-06T10:33:57.569Z)

Both imports and helpers are available. Let me add the end-to-end regression test right after the conflict test.

#### 781. user (2026-07-06T10:33:58.244Z)

```

1020         # Still one row, watched intact.
1021         fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1")
1022         assert fetched is not None
1023         assert fetched.watched is True
1024
1025
1026     async def test_insert_content_item_round_trip(conn: aiosqlite.Connection) -> None:
1027         """Insert a content item and read it back via both natural keys."""
1028         await _seed_user(conn)
1029         src = await db.upsert_source(
1030             conn, Source(account_id=USER_ID, provider="telegram", provider_source_id="-1")
1031         )

```

#### 782. user (2026-07-06T10:34:09.841Z)

The file C:/Users/avidu/Projects/\_wt-watched-fix/tests/test\_db.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 783. assistant (2026-07-06T10:34:35.042Z)

Now let me verify: run the full suite against the worktree (with fix), and prove the new tests actually catch the bug by running them with the `db.py` fix temporarily reverted.

#### 784. user (2026-07-06T10:34:44.500Z)

=== (A) new/edited tests WITH fix (expect pass) ===

```

-----
TOTAL                                1192    834    312    15    25%
FAIL Required test coverage of 80.0% not reached. Total coverage: 24.93%
2 passed, 78 deselected in 0.88s

```

=== (B) PROOF they catch the bug: revert db.py fix, rerun (expect FAIL) ===

Saved working directory and index state WIP on fix/upsert-source-preserve-watched: 3bfb1da  
feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR 0009, Track E) (#39)

FAIL Required test coverage of 80.0% not reached. Total coverage: 24.93%

===== short test summary info =====

FAILED tests/test\_db.py::test\_upsert\_source\_updates\_title\_but\_preserves\_watched\_on\_conflict

FAILED tests/test\_db.py::test\_watched\_source\_survives\_ingestion\_upserts - Ass...

2 failed, 78 deselected in 0.69s

--- restore fix ---

Dropped refs/stash@{0} (7cd607f719751e71c03b5addfcfd87911c83039f)

Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

**785. assistant (2026-07-06T10:34:53.050Z)**

Clean before/after proof: both tests **\*\*pass with the fix\*\*** and **\*\*fail without it\*\*** (reverting `db.py` reproduces the regression). Now the full suite + lint on the worktree.

**786. user (2026-07-06T10:35:06.383Z)**

=== ruff check + format ===

All checks passed!

1 file would be reformatted, 35 files already formatted

=== full suite (worktree, with fix) ===

-----  
TOTAL 1192 27 312 19 97%

Required test coverage of 80.0% reached. Total coverage: 96.94%

496 passed in 8.34s

=== changed files ===

**fix/upsert-source-preserve-watched...origin/main**

M src/tg\_video\_filter/db.py

M tests/test\_db.py

Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

**787. assistant (2026-07-06T10:35:12.640Z)**

Full suite **\*\*496 passed @ 96.94%\*\*** (+1 test), ruff check clean ? but one file needs formatting (my multi-line edits). Let me apply `ruff format` to the changed files and re-verify.

**788. user (2026-07-06T10:35:16.049Z)**

=== which file? ===

Would reformat: tests\test\_db.py

1 file would be reformatted, 1 file already formatted

**789. assistant (2026-07-06T10:35:21.207Z)**

The `--check` short-circuited the chain (exit 1). Let me format directly.

**790. user (2026-07-06T10:35:26.904Z)**

1 file reformatted, 1 file left unchanged

--- whole-repo format check ---

36 files already formatted

fmt-exit=0

--- ruff check ---

All checks passed!

--- re-run the 2 tests after reformat ---

FAIL Required test coverage of 80.0% not reached. Total coverage: 24.93%

2 passed, 78 deselected in 0.76s

Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

**791. assistant (2026-07-06T10:35:35.349Z)**

Clean: format ?, check ?, tests still pass. Let me review the final diff, then commit and open the PR.

**792. user (2026-07-06T10:35:36.804Z)**

=== FINAL DIFF ===

diff --git a/src/tg\_video\_filter/db.py b/src/tg\_video\_filter/db.py

index f6ef6b3..a059e57 100644

--- a/src/tg\_video\_filter/db.py

+++ b/src/tg\_video\_filter/db.py

```
@@ -1102,10 +1102,20 @@ async def list_enabled_users(conn: aiosqlite.Connection) -> list[User]:
```

```

    async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
-        """Insert a source row, or update title/watched on conflict.
+        """Insert a source row, or refresh title/last_seen_at on conflict.

        The conflict key is ``(account_id, provider, provider_source_id)`` per the
        ADR 0011 schema. Returns the persisted row.
+
+        ``watched`` is **user-controlled state** (toggled only via
+        :func:`set_source_watched` / the ``/watch`` and ``/unwatch`` commands) and is
+        therefore **never overwritten on conflict** ? the ingestion path
+        (``upsert_channel`` / ``insert_video``) re-touches a source on every message,
+        so clobbering ``watched`` here would silently un-watch a channel the user
+        opted into after its first ingested video (ADR 0009). ``watched`` is only set
+        on the initial INSERT of a new row. ``title`` is refreshed but never nulled:
+        the ingestion shims pass ``title=None``, so ``COALESCE`` keeps the existing
+        title rather than erasing the channel name.
        """
        await conn.execute(
            """INSERT INTO sources

```

```
@@ -1113,8 +1123,7 @@ async def upsert_source(conn: aiosqlite.Connection, source: Source) ->
Source:
```

```

        seen_first_at, last_seen_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
-        title=excluded.title,
-        watched=excluded.watched,
+        title=COALESCE(excluded.title, sources.title),
        last_seen_at=excluded.last_seen_at""",
        (

```

```

            source.account_id,
diff --git a/tests/test_db.py b/tests/test_db.py
index c479c34..8692285 100644
--- a/tests/test_db.py
+++ b/tests/test_db.py

```

```

@@ -991,13 +991,20 @@ async def test_upsert_source_round_trip(conn: aiosqlite.Connection) ->
None:
    assert fetched.kind == "channel"

```

```

-async def test_upsert_source_updates_on_conflict(conn: aiosqlite.Connection) -> None:
-    """Re-upserting the same (account, provider, source_id) updates title/watched."""
+async def test_upsert_source_updates_title_but_preserves_watched_on_conflict(
+    conn: aiosqlite.Connection,
+) -> None:
+    """On conflict, ``upsert_source`` refreshes the title but must NOT overwrite
+    ``watched`` ? that flag is user-controlled state (set only via
+    ``set_source_watched`` / ``/watch``). Overwriting it here silently un-watched
+    channels after their first ingested video (ADR 0009 regression)."""
    await _seed_user(conn)
    await db.upsert_source(
        conn,
        Source(account_id=USER_ID, provider="telegram", provider_source_id="-1", watched=True),
    )
+    # A subsequent upsert (as the ingestion path does) carries watched=False by
+    # default; it must not clobber the user's opt-in.
    updated = await db.upsert_source(
        conn,
        Source(
@@ -1008,12 +1015,48 @@ async def test_upsert_source_updates_on_conflict(conn:
aiosqlite.Connection) ->
        watched=False,
    ),

```

```

    )
-   assert updated.title == "Renamed"
-   assert updated.watched is False
-   # Still one row.
+   assert updated.title == "Renamed" # title still refreshed
+   assert updated.watched is True # watched preserved, not clobbered
+   # Still one row, watched intact.
    fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1")
    assert fetched is not None
-   assert fetched.watched is False
+   assert fetched.watched is True
+
+
+async def test_watched_source_survives_ingestion_upserts(conn: aiomysqlite.Connection) -> None:
+    """Regression (ADR 0009 opt-in): a channel the user ``/watch``-ed must stay
+    watched after ingestion. The live ``on_new_message`` handler re-touches the
+    source via ``upsert_channel`` + ``insert_video`` on every message; before the
+    fix those upserts clobbered ``watched`` back to False, silently un-watching
+    the channel after its first ingested video (``/watch`` worked for exactly one
+    video)."""
+    await _seed_user(conn)
+    await db.upsert_source(
+        conn,
+        Source(
+            account_id=USER_ID,
+            provider="telegram",
+            provider_source_id=str(CHANNEL_ID),
+            title="My Feed",
+            kind="channel",
+            watched=True, # user opted in via /watch
+        ),
+    )
+    assert await db.should_ingest_source(conn, USER_ID, CHANNEL_ID) is True
+
+    # Simulate the ingestion path re-touching the source (handler lines ~248/271).
+    await db.upsert_channel(conn, Channel(user_id=USER_ID, channel_id=CHANNEL_ID, title="My
Feed"))
+    await db.insert_video(
+        conn,
+        VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=1, document_id=42),
+    )
+
+    src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", str(CHANNEL_ID))
+    assert src is not None
+    assert src.watched is True, "ingestion upserts must not un-watch a user-watched channel"
+    assert src.title == "My Feed", "ingestion (insert_video title=None) must not erase the
title"
+    # The very next message from the channel is still ingested.
+    assert await db.should_ingest_source(conn, USER_ID, CHANNEL_ID) is True


    async def test_insert_content_item_round_trip(conn: aiomysqlite.Connection) -> None:
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

```

### 793. assistant (2026-07-06T10:35:50.715Z)

The diff is clean and correct. Let me commit (explicit paths), push, and open the PR.

### 794. user (2026-07-06T10:35:56.632Z)

```

fix/upsert-source-preserve-watched
src/tg_video_filter/db.py
tests/test_db.py
[fix/upsert-source-preserve-watched 5791e29] fix(scanning): don't clobber user-controlled
watched on source upsert

```

```
2 files changed, 61 insertions(+), 9 deletions(-)
=== only my commit ahead of origin/main? ===
5791e29 fix(scanning): don't clobber user-controlled watched on source upsert
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

#### 795. user (2026-07-06T10:36:19.363Z)

```
=== push ===
remote:
branch 'fix/upsert-source-preserve-watched' set up to track 'origin/fix/upsert-source-preserve-watched'.
To https://github.com/avidullu/telegram-video-filter.git
 * [new branch]      fix/upsert-source-preserve-watched -> fix/upsert-source-preserve-watched
=== open PR ===
https://github.com/avidullu/telegram-video-filter/pull/41
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

#### 796. assistant (2026-07-06T10:36:23.122Z)

PR **#41** opened. Let me clean up the worktree and confirm the PR state.

#### 797. user (2026-07-06T10:36:27.785Z)

```
worktree removed
C:/Users/avidu/Projects/Telegram 3bfb1da [main]
--- main checkout clean? ---
main
main...origin/main
--- PR #41 ---
#41 OPEN mergeable=MERGEABLE CLEAN (+61/-9)
fix(scanning): don't clobber user-controlled `watched` on source upsert
```

#### 798. assistant (2026-07-06T10:36:41.045Z)

Done ? the follow-up fix is drafted, verified, and up for review.

**[PR #41 ? fix(scanning): don't clobber user-controlled `watched` on source upsert](<https://github.com/avidullu/telegram-video-filter/pull/41>)**

**\*\*State:\*\*** OPEN · MERGEABLE · CLEAN (+61/?9) off `origin/main`.

**\*\*The regression it fixes\*\*** (the one #39's fix introduced, now in `main`): the ingestion path re-touches the source on every message via `upsert\_channel` + `insert\_video`, and `upsert\_source`'s `ON CONFLICT` overwrote `watched` with the shims' hardcoded `False` ? so a `/watch`-ed channel got silently un-watched after its first ingested video.

**\*\*The fix:\*\***

```
```sql
ON CONFLICT(...) DO UPDATE SET
  title=COALESCE(excluded.title, sources.title),    -- was: title=excluded.title (insert_video
passed NULL ? erased the name)
  last_seen_at=excluded.last_seen_at                -- removed: watched=excluded.watched
```

`watched` is user-controlled state (only `set_source_watched` / `/watch`, `/unwatch` change it),
so it's set only on a new row's INSERT and never clobbered on conflict. I confirmed no caller
relies on the old behavior (the commands use a direct `UPDATE`; the migration uses a direct
`INSERT`).
```

**\*\*Verification (all reproduced locally):\*\***

- New `test\_watched\_source\_survives\_ingestion\_upserts` (watch ? ingest ? still watched + title intact) and the updated conflict test **\*\*both fail against pre-fix `db.py` and pass after\*\*** ? proven by reverting `db.py` in isolation.
- Full suite **\*\*496 passed @ 96.94%\*\*** (+1 test), `ruff check` + `format` clean.



**\*\*Process:\*\*** drafted in an isolated worktree off `origin/main` (shared checkout untouched), staged explicit paths, confirmed only my commit `5791e29` was ahead before pushing; worktree removed, `main` left clean.

Note it's a **\*\*separate PR\*\*** (#41) rather than a comment on the merged #39, since the regression lives in `main` now. When it's reviewed/merged, that closes out the Track E selective-scanning contract properly (opt-in default **\*\*and\*\*** `/watch` that sticks). Want me to also drop a short note on #39 linking to #41 for traceability?

### 799. assistant (2026-07-06T22:52:14.937Z)

**\*\*Confirmed ? a real medium bug.\*\*** `normalize\_domain` violates its `str | None` contract by **\*\*throwing `ValueError`\*\*** (from `urlsplit` on an unterminated IPv6 `[`) instead of returning `None`. Both callers assume None, so:

- An **\*\*attacker-controlled caption\*\*** containing `http://[` throws an **\*\*uncaught exception out of the URL stage\*\*** in the live ingestion path (the extraction is outside the stage's try/except).
- One `[`-containing config entry **\*\*disables the whole stage\*\*** (parse throws ? fail-open), defeating the per-entry granularity the author correctly built for other bad domains.

Both latent (no config command yet) but real; trivial fix (catch `ValueError` from `urlsplit`). The false-positive battery was otherwise clean (no legit domain wrongly blocked). I'll wait for the workflow's independent take, then post a review **\*\*holding the LGTM\*\*** on this medium.

### 800. user (2026-07-07T03:47:41.605Z)

This session is being continued from a previous conversation that ran out of context. The summary below covers the earlier portion of the conversation.

Summary:

#### 1. Primary Request and Intent:

- Initial: "Hi. Can you ramp up and review <https://github.com/avidullu/telegram-video-filter/pulls> thoroughly. Make sure to sync the repo to remote head before executing."
- Subsequently: review a sequence of PRs (#35, #37, #38, #39, #40, #50, #51, #52, #53, #54, #55, #57, #58), post review comments to GitHub, author fixes to my own findings when asked (#36, #41), and re-verify after fixes.
- Standing rule (explicit, for all PRs): "post LGTM only when you are satisfied with the merger of PRs" ? i.e., LGTM is strictly gated on locally-reproduced verification, never a rubber stamp.
- Set up a background "PR review alarm" to watch for new PRs "within a few minutes" and review them as they arrive.
- "Please update your watcher to watch force pushes and updates to PR review threads because that is where the author will comment" ? watcher must detect force-pushes (SHA change) and comments (updatedAt change), not just new PR numbers.

#### 2. Key Technical Concepts:

- Telethon userbot, ADR-driven design (ADRs 0001?0012), SQLite via aiosqlite, Pydantic v2 models, `user\_version` migration ladder.
- Provider-neutral domain schema (Track B): `sources`, `content\_items`, `policies`, `destinations`, `routing\_rules`, `delivery\_attempts`, `policy\_decisions` tables (+ `\*\_deprecated`).
- Three-tier content dedup via `content\_items.dedup\_key` (`tg:doc:`, `tg:fp:`, `tg:msg:`); `compute\_dedup\_key`.
- ADR 0012 policy pipeline (F1?F7): `PolicyStage` protocol, `StageResult`, `evaluate\_pipeline` (short-circuit); stages topic-preferences ? url-reputation ? caption-safety ? deep-inspection.
- Selective scanning (Track E, ADR 0009): opt-in `watched` flag, `should\_ingest\_source`, `/watch`/`/unwatch`/`/channels`/`/watchlist`.
- CommandService (Track D): provider-independent command parsing; Telethon adapter in `commands.py`.
- Config persistence: `policies.config\_json` raw dict; `save\_filter\_config` merges flat `FilterConfig` keys to preserve ADR 0012 keys.
- ReDoS mitigation via `regex` module `timeout=`; markdown-injection escaping; IDOR account-scoping.
- My workflow pattern: multi-agent adversarial review (dimension reviewers ? parallel verify ? synthesis), all findings reproduced locally, LGTM gated on verification.

- Verification gate (no CI, no mypy): ``ruff check` + `ruff format --check` + `pytest`` (coverage floor 80%).
- Concurrent-checkout safety: git worktrees for isolated fix work; branch from ``origin/main``; explicit-path staging.
- PR-watcher: background bash poll of ``gh pr list --json number,isDraft,headRefOID,updatedAt``; SHA change = force-push (re-review), updatedAt-only = comment (judge).

### 3. Files and Code Sections:

- ``src/tg_video_filter/db.py`` ? heart of migrations/repos. #41 fix I authored: ``upsert_source`` ON CONFLICT changed to ``title=COALESCE(excluded.title, sources.title), last_seen_at=excluded.last_seen_at`` (removed ``watched=excluded.watched``) so ingestion doesn't clobber user-controlled ``watched``. F6 added ``policy_decisions`` table + migration 005 + ``record_policy_decision`` (UPSER on ``(content_item_id, policy_id)``), ``_policy_trace_json`` (later hardened with ``default=str``).
- ``src/tg_video_filter/policy.py`` ? pipeline. F5 ``PolicyBudget`` (semaphore + ``wait_for`` timeout, fixed to raise dedicated ``PolicyBudgetExceeded`` and ``except Exception`` for fetcher errors). F3 ``CaptionSafetyStage`` using ``regex`` module with ``timeout=_CAPTION_REGEX_TIMEOUT_SECONDS``. F4 ``normalize_domain`` (fixed to wrap ``urlsplit`` in ``try/except ValueError: return None``), ``_domain_matches_blocked(domain, blocked) = domain==blocked or domain.endswith('.'+blocked)``.
- ``src/tg_video_filter/command_service.py`` ? F7 ``why`` command with ``get_content_item_for_account`` (account-scoped, IDOR-safe); markdown-injection fix added ``_MARKDOWN_ESCAPE_RE = re.compile(r"([\\`*_\\[\\]()])"``, ``_AUTOLINK_RE``, ``_safe_markdown_text``, ``_REPLY_CHAR_LIMIT=4000``, ``_format_reply``. #58 doc-fix: ``format_commands`` exclusion header changed from "(all off by default)" to "" and nosound note to "Defaults: GIF/round excluded, nosound allowed."
- ``src/tg_video_filter/models.py`` ? #58 docstring fixes: ``ContentItem`` (caption text now captured), ``Policy.config_json`` ("FilterConfig JSON blob plus ADR 0012 policy keys"), ``FilterConfig`` (stored in ``policies.config_json`` via shims). FilterConfig defaults verified: ``exclude_gif=True, exclude_round=True, exclude_nosound=False``.
- ``tests/test_commands.py`` ? #58 assertion updated: ``assert "GIF/round excluded, nosound allowed" in result``.
- ``Dockerfile``, ``pyproject.toml`` ? #58 stale-comment/description fixes.
- README.md ? #58 rewrite verified accurate (opt-in scanning watched=false, policy stages with "no production fetcher wired by default", "what is not built" includes retention cleanup / issue #56).
- Memory: ``C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\lgtm-only-when-merge-ready.md`` (feedback type) + MEMORY.md pointer.

### 4. Errors and fixes:

- Watcher nested in ``{...}`` & inside ``run_in_background`` ? orphaned/untracked. Fixed by making the poll loop the top-level background command.
- Workflow script parse error (raw regex in JS template literal) ? rewrote CONTEXT using ``[...].join('\n')`` and describing regexes in words.
- Heredoc/inline-bash quote-escaping and a control-character rejection ? switched to Write tool for repro scripts.
- F5 permit-release repro had a timeout-mismatch bug (my test used a slow fetcher exceeding the tiny budget) ? re-ran with correct timing; confirmed no leak.
- Wrong branch name for #54/#58 worktree adds ? redid with correct branch (``feat/f5-deep-inspection-budget``, ``agent/docs-sweep``).
- User feedback that mattered: "post LGTM only when you are satisfied" (saved to memory); "update your watcher to watch force pushes and updates to PR review threads" (upgraded watcher fingerprint).

### 5. Problem Solving:

- Reproduced every finding locally (blockers, regressions, ReDoS hang, IDOR-safety, markdown injection). The adversarial workflow repeatedly caught mediums I under-rated (F5 fetcher-error handling, F7 markdown injection). Whole Track B?F umbrella + ADR 0012 F1?F7 reviewed, verified, merged. Two of my own fix PRs (#36 format, #41 watched-clobber) merged. Watcher distinguishes fix-pushes from comments; extended to ~8h window during idle.

### 6. All user messages:

- "Hi. Can you ramp up and review <https://github.com/avidullu/telegram-video-filter/pulls> thoroughly. Make sure to sync the repo to remote head before executing"

- "please review PR comments" (re-review #35 fix)
- "yes, please post these comments on the github and also an LGTM if your analysis governs this is mergeable and wel tested"
- "Can you review PR #37 thoroughly" / "please post this as PR comment"
- "pls take another pass." (re-verify #37 fix) / "please post the comment and cleanup the stray file"
- "can you review #37"; "please do the house keeping work!"
- "Can you review the draft PR #40"; "Please post this as review comment and thanks for the diligent work"
- "Can you take another pass on #40?"; "Please post this comment and give an lgTM to proceed further. Also I want you to set up your PR review alarm and wait for the PR hours coming towards this project within a few minutes"
- "Update?" (multiple times)
- "pls review #39 and also post the comment on the PR." (this was actually the /plugin lines + review #39)
- "please review #37" / "please post the comments on the github" (various)
- "Please continue this review and post LGTM only when you are satisfied with the merger of PRs. Please follow this for this and all upcoming PRs"
- "Please update your watcher to watch force pushes and updates to PR review threads because that is where the author will comment"
- Security-relevant constraints (from global CLAUDE.md, must persist): never clobber sibling-session uncommitted work; branch from origin explicitly; stage explicit paths (never `git add -A`); never trust PR descriptions (reproduce locally); session files are credentials (from repo README, do not expose). No secrets/credentials were ever printed.

#### 7. Pending Tasks:

- Complete the PR #58 docs-accuracy review (workflow `w3glb28kq` running), post the review (LGTM only if satisfied), clean up worktree `\_wt-pr58`, and re-arm the watcher.
- Continue watching for new PRs / force-pushes / comments and review each with the same cycle.

#### 8. Current Work:

Reviewing \*\*PR #58 "docs: refresh current project documentation"\*\* (branch `agent/docs-sweep`, +782/-787, 14 files). I set up worktree `C:/Users/avidu/Projects/\_wt-pr58`, ran the gate (verified: ruff clean, \*\*562 passed @ 94.30%\*\*), and verified the key doc-accuracy source fix: FilterConfig defaults are `exclude\_gif=True, exclude\_round=True, exclude\_nosound=False`, so the new `/show commands` text "GIF/round excluded, nosound allowed" is ACCURATE (the old "all off by default" was a real bug). I read the README diff and confirmed it accurately reflects the merged state (opt-in scanning, policy stages, deep-inspection with no fetcher wired, retention-cleanup not built/issue #56). I then launched a 3-dimension workflow (task ID `w3glb28kq`, run `wf\_0febe894-ff9`) to cross-check all doc files against the code: dim1 README+design, dim2 plan+runbook, dim3 ADR status notes (0007/0009/0010/0011/0012). The workflow was still running when the summary was requested.

#### 9. Optional Next Step:

Wait for the docs workflow `w3glb28kq` to complete, then parse its findings, and ? per the standing rule "post LGTM only when you are satisfied" ? either LGTM #58 (if only low/no accuracy issues, since the source changes + README are verified accurate and gate is green) or post a review holding LGTM (if the workflow confirms any factually-wrong doc claims). Then clean up worktree `\_wt-pr58` and re-arm the PR-watcher (baseline #58, top-level loop tracking `#num draft=X sha=Y upd=Z`). Direct quote of where I left off: I "launched a workflow ... to cross-check all doc files against the code" and stated I would "fold in the workflow's ... findings" before deciding ? consistent with the user's instruction to "post LGTM only when you are satisfied with the merger of PRs."

If you need specific details from before compaction (like exact code snippets, error messages, or content you generated), read the full transcript at: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664.jsonl Continue the conversation from where it left off without asking the user any further questions. Resume directly ? do not acknowledge the summary, do not recap what was happening, do not preface with "I'll continue" or similar. Pick up the last task as if the break never happened.

### 801. assistant (2026-07-07T03:48:15.429Z)

The workflow returned 3 CONFIRMED findings (0 refuted): one \*\*HIGH\*\* (runbook §6 points

operators at `set\_delivery\_target.py`, which writes to the dropped `filter\_config` table and errors on every migrated DB) and two LOW that are the same issue (design.md §10 lists a nonexistent log event `delivery\_record\_failed`). Per my rule I won't trust even my own subagents ? let me reproduce both independently, and check whether #58 actually introduced/touched the runbook §6 section.

## 802. user (2026-07-07T03:48:21.158Z)

```
=== [1] Did #58 add/modify runbook §6 'Delivery Mode and Target'? ===
81:-# ? inserts a users row (enabled=1) + default filter_config
156:-Filters are stored per-user as JSON in `filter_config`. Default v1 config:
161:@@ -75,224 +118,219 @@ Filters are stored per-user as JSON in `filter_config`. Default v1
config:
169:  "delivery_target": "me"
174:+++ 6. Delivery Mode and Target
181:-UPDATE filter_config
212:+Use the helper script for targets:
215:+.\.venv\Scripts\python.exe scripts\set_delivery_target.py <account_id> <target>
246:+`delivery_target`.
302:-UPDATE filter_config
312:+`policies.config_json` carefully when needed. Example:
316:-By default matches go to your **Saved Messages**. Set ``delivery_target`` to
319:+UPDATE policies
337:-UPDATE filter_config
338:-  SET config_json = json_set(config_json, '$.delivery_target', '@my-movie-feed')
340:+UPDATE policies
365:-  videos/links honor ``delivery_target``.
366:-  Old configs without ``delivery_target`` default to ``"me"`` (backwards-compatible).
369:-Instead of hand-writing SQL, use the helper script (run with the **venv** python,
374:-  \\.venv\Scripts\python.exe scripts\set_delivery_target.py <user_id> <target>
```

```
=== [2] What table does set_delivery_target.py write? ===
30:def main() -> int:
31:     if len(sys.argv) != 3:
36:         user_id = int(sys.argv[1])
37:         target = sys.argv[2].strip()
48:         "UPDATE filter_config "
49:         "SET config_json = json_set(config_json, '$.delivery_target', ?) "
50:         "WHERE user_id = ?",
55:         print(f"ERROR: no filter_config row for user_id={user_id}.", file=sys.stderr)
64:         "SELECT config_json FROM filter_config WHERE user_id = ?", (user_id,)
```

```
=== [4] delivery_record_failed present in src? (expect NO matches) ===
exit=0
--- design.md §10 other events grep-check ---
manager_started -> 2 matches in src
client_connected -> 2 matches in src
video_seen -> 2 matches in src
video_forwarded -> 2 matches in src
flood_wait -> 1 matches in src
config_updated -> 2 matches in src
client_disconnected -> 2 matches in src
delivery_record_failed -> 0 matches in src
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

## 803. assistant (2026-07-07T03:48:41.748Z)

Key nuance from the diff: #58 \*\*rewrote §6\*\* ? it correctly fixed the inline SQL examples from `UPDATE filter\_config` to `UPDATE policies ... WHERE account\_id=? AND is\_default=1` (lines 319, 340), but \*\*still recommends\*\* the helper script `set\_delivery\_target.py` (line 212/215), and that script \*internally\* still does `UPDATE filter\_config ... WHERE user\_id=?` (script lines 48-50, 64). So the same rewritten section now contradicts itself: correct manual SQL alongside a recommended script that hits a dropped table. That's squarely in this PR's scope. And the LOW is clean (`delivery\_record\_failed`: 0 matches, every other §10 event: 2 matches).

Let me reproduce the actual SQL failure myself against a freshly-migrated DB ? this is the LGTM-gating claim, so I verify it, not the subagent.

#### 804. user (2026-07-07T03:48:51.005Z)

Traceback (most recent call last):

```
File "C:\Users\avidu\Projects\wt-pr58\repro_delivery.py", line 3, in <module>
    from tg_video_filter.db import init_db
```

```
File "C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter\db.py", line 19, in <module>
    import aiosqlite
```

ModuleNotFoundError: No module named 'aiosqlite'

Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

#### 805. assistant (2026-07-07T03:48:55.195Z)

No venv in the worktree. Let me find the env I used for the gate run (562 passed) ? likely in the main checkout.

#### 806. user (2026-07-07T03:49:00.314Z)

FOUND: C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe

deps OK

--- any python with aiosqlite? ---

python:

Traceback (most recent call last):

```
File "<string>", line 1, in <module>
```

python3:

Traceback (most recent call last):

```
File "<string>", line 1, in <module>
```

py:

Traceback (most recent call last):

```
File "<string>", line 1, in <module>
```

#### 807. assistant (2026-07-07T03:49:04.057Z)

The Telegram `.venv` has the deps. Let me run the repro with it against the worktree's source.

#### 808. user (2026-07-07T03:49:12.617Z)

user\_version: 5

filter\_config present?: False | deprecated present?: True | policies present?: True

script SQL (UPDATE filter\_config): OperationalError -> no such table: filter\_config

runbook manual SQL (UPDATE policies): SUCCEEDED (as documented)

Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer

#### 809. assistant (2026-07-07T03:49:40.931Z)

Fully reproduced, independently, exactly as claimed:

- Fresh migrated DB ? `user\_version=5`, `filter\_config` **\*\*dropped\*\*** (gone),

`filter\_config\_deprecated` + `policies` present.

- The script's exact SQL `UPDATE filter\_config ? WHERE user\_id=?` ? **\*\*`OperationalError: no such table: filter\_config`\*\*** (hard failure).

- The runbook's *\*new\** manual SQL `UPDATE policies ? WHERE account\_id=? AND is\_default=1` ?

**\*\*succeeds\*\***, as documented.

So #58's rewritten \$6 is self-contradicting: it shows the correct manual `UPDATE policies` SQL *\*and\**, in the same section, recommends a helper script that errors on every migrated DB. Let me grab exact line citations for the review, then write it up.

#### 810. user (2026-07-07T03:49:46.198Z)

=== runbook \$6 helper-script recommendation (exact lines) ===

129:## 6. Delivery Mode and Target

151:Use the helper script for targets:

```

154:.\.venv\Scripts\python.exe scripts\set_delivery_target.py <account_id> <target>
160:.\.venv\Scripts\python.exe scripts\resolve_target.py <account_id> <target>
209:UPDATE policies
222:UPDATE policies
333:| Delivery target cannot resolve | Telethon session cache lacks entity | Run
`scripts/resolve_target.py`, then restart. |

=== set_delivery_target.py broken lines ===
30:def main() -> int:
48:     "UPDATE filter_config "
50:     "WHERE user_id = ?",
55:     print(f"ERROR: no filter_config row for user_id={user_id}.", file=sys.stderr)
64:     "SELECT config_json FROM filter_config WHERE user_id = ?", (user_id,)

=== design.md nonexistent event line ===
218:- `delivery_record_failed`

=== real delivery-failure log tokens in code ===
116:     log.exception("delivery_retry_failed video=%s", video_id)
119:     log.exception("delivery_failed video=%s", video_id)
266:     log.warning("link_delivery_failed_in_both_mode video=%s", video_id)
268:     log.warning("forward_delivery_failed_in_both_mode video=%s", video_id)
273:     "link_mode_fallback_to_forward video=%s mode=%s link_text_missing=%s",

```

## 811. user (2026-07-07T03:49:51.238Z)

```

129    ## 6. Delivery Mode and Target
130
131    Delivery mode and target are stored in policy/config JSON. There is no
132    Saved-Messages command for changing them yet, so update deliberately and restart
133    the manager.
134
135    Modes:
136
137    | Mode | Behavior |
138    |---|---|
139    | `forward` | Forward the Telegram video message. |
140    | `link` | Send a `t.me` link plus metadata summary; falls back to forward if no link
exists. |
141    | `both` | Forward and send the link summary. |
142
143    Targets:
144
145    | Target | Meaning |
146    |---|---|
147    | `me` | Saved Messages. |
148    | `@your-feed` | Public channel or chat username. |
149    | `-1001234567890` | Numeric Telegram chat/channel id. |
150
151    Use the helper script for targets:
152
153    ```powershell
154    .\.venv\Scripts\python.exe scripts\set_delivery_target.py <account_id> <target>
155    ```
156
157    If Telethon cannot resolve a target, refresh the entity cache:
158
159    ```powershell
160    .\.venv\Scripts\python.exe scripts\resolve_target.py <account_id> <target>
161    ```
162
163    Then restart:
164
165    ```bash
166    docker compose restart app

```

167     ` `` `

168

#### 812. user (2026-07-07T03:49:54.435Z)

198     2. URL/domain reputation,  
199     3. caption safety,  
200     4. deep inspection boundary.  
201  
202     Flat `/set` commands update `FilterConfig` fields inside the default policy  
203     without erasing unknown policy keys.  
204  
205     Policy-stage config currently has no friendly command UI. Edit  
206     `policies.config\_json` carefully when needed. Example:  
207  
208     ```sql  
209     UPDATE policies  
210         SET config\_json = json\_set(  
211             config\_json,  
212             '\$.topics',  
213             json('{"include":["ai"],"exclude":["crypto"],"mode":"any"}')  
214         )  
215     WHERE account\_id = <account\_id>  
216     AND is\_default = 1;  
217     ```\n

218  
219     Caption and domain checks:  
220  
221     ```sql  
222     UPDATE policies  
223         SET config\_json = json\_set(  
224             config\_json,  
225             '\$.security\_checks',  
226             json('{  
227                 "blocked\_domains": ["bad.example"],  
228                 "unsafe\_caption\_terms": ["malware"],  
229                 "unsafe\_caption\_patterns": ["free\\\\\\\\s+money"]

#### 813. assistant (2026-07-07T03:50:03.278Z)

Let me check whether the runbook offers \*any\* working manual path for `delivery\_target` (the \$6 script is the only method shown so far; the \$7 `UPDATE policies` examples are for other keys).

#### 814. user (2026-07-07T03:50:06.519Z)

=== all mentions of delivery\_target in runbook (checked-out) ===  
124: "delivery\_mode": "forward",  
125: "delivery\_target": "me"  
154: .\\.venv\\Scripts\\python.exe scripts\\set\_delivery\_target.py <account\_id> <target>  
170: `delivery\_target`.

=== is there a manual UPDATE policies ... \$.delivery\_target example anywhere in runbook? ===  
NONE ? no manual SQL example targets \$.delivery\_target

#### 815. assistant (2026-07-07T03:50:28.000Z)

Decisive: \$6 offers the broken `set\_delivery\_target.py` script as the **\*\*sole\*\*** documented way to change the delivery target ? there is **\*\*no\*\*** working manual `\$.delivery\_target` SQL anywhere in the runbook (the \$7 `UPDATE policies` examples cover `\$.topics`/`\$.security\_checks` only). So an operator doing a core task (route deliveries to a channel) is hard-blocked with no documented recourse. And the diff shows #58 correctly migrated the **\*inline\*** SQL from `filter\_config`?`policies` but left the recommended helper script ? which has the identical bug ? in place.

Both findings reproduced. Everything else in #58 is verified accurate, gate is green (562 @ 94.30%, ruff clean). Per my standing rule I hold LGTM on the broken operational path. Let me write and post the review.

**816. assistant (2026-07-07T03:50:52.372Z)**

## Review ? docs refresh (`agent/docs-sweep`)

Strong, mostly-accurate sweep ? it fixes several real doc bugs (the `/show commands` "all off by default" claim, the inline `UPDATE filter\_config` ? `UPDATE policies` migration, the README's opt-in-scanning / deep-inspection-with-no-fetcher / retention-not-built state). Gate is green on the branch: **562 passed @ 94.30%**, `ruff check` + `ruff format --check` clean. **Holding the LGTM** on one operational-accuracy defect (a documented step that hard-fails), plus one low. Both reproduced locally.

### Verified accurate ?

- **\*/show commands` exclusion text** ? new "GIF/round excluded, nosound allowed" matches `FilterConfig` defaults (`exclude\_gif=True, exclude\_round=True, exclude\_nosound=False`). The old "all off by default" was a genuine bug; good catch.
- **Runbook deploy path** ? `python -m tg\_video\_filter.bootstrap` and `python -m tg\_video\_filter` both have `main()` + `\_\_main\_\_` guards; `.env.example` has `TG\_API\_ID/TG\_API\_HASH/DB\_PATH/SESSIONS\_DIR`; compose service is named `app` (so `docker compose restart app` is valid); `scripts/resolve\_target.py` exists and is session-file-only (unaffected).
- **README** ? opt-in scanning (`watched=false`), policy stages "no production fetcher wired by default", "what is not built" (retention cleanup / #56) all match merged code.
- **Model/docstring fixes** (`ContentItem` caption, `Policy.config\_json`, `FilterConfig` shims) match the implementation.

### ? Blocking ? `docs/runbook.md:151` §6 documents a delivery-target step that errors on every live DB

§6 "Delivery Mode and Target" gives exactly **one** way to change the target:

```
...  
.\.venv\Scripts\python.exe scripts\set_delivery_target.py <account_id> <target>  
...
```

But `scripts/set\_delivery\_target.py:48-50,64` still runs `UPDATE filter\_config ? WHERE user\_id = ?` ? and under ADR 0011 (migrations 002/003) `filter\_config` is renamed to `filter\_config\_deprecated` and `init\_db()` drops any recreated `filter\_config` (`db.py`). Delivery config now lives in `policies.config\_json`. **Reproduced against a fresh migrated DB (`user\_version=5`):**

```
...  
filter_config present? False | filter_config_deprecated present? True | policies present? True  
script SQL (UPDATE filter_config ?): OperationalError -> no such table: filter_config  
manual SQL (UPDATE policies ? WHERE account_id=? AND is_default=1): SUCCEEDED  
...
```

So the sole documented path for a core operation hard-fails, and ? unlike the policy-stage keys ? there's **no** manual `\$.delivery\_target` SQL anywhere in the runbook to fall back to. Note this PR **did** correctly migrate the neighboring inline SQL from `filter\_config` to `policies`; it just left the recommended helper script (identical bug) in place.

```
> Fix (either):  
> (a) fix the script to update the default policy, mirroring `db.save_filter_config` ?  
> `UPDATE policies SET config_json = json_set(config_json, '$.delivery_target', ?) WHERE  
account_id = ? AND is_default = 1`; or  
> (b) drop the script recommendation and show the equivalent manual `UPDATE policies`  
`$.delivery_target` ?` SQL in §6 (parallel to the §7 policy-stage examples ? I verified that  
statement succeeds on a migrated DB).
```

### ? Low ? `docs/design.md:218` observability list cites a nonexistent log event



\$10 lists `` `delivery\_record\_failed` `` as an "important event", but that token appears **\*\*nowhere\*\*** in `src/` (0 matches; every other event in the list ? `manager\_started`, `client\_connected`, `video\_seen`, `video\_forwarded`, `flood\_wait`, `config\_updated`, `client\_disconnected` ? grep-matches). An operator building log alerting on it would never match. The real delivery-failure tokens are `delivery\_failed` / `delivery\_retry\_failed` (`delivery.py:116,119`) and `link\_delivery\_failed\_in\_both\_mode` / `forward\_delivery\_failed\_in\_both\_mode` (`delivery.py:266,268`).

> **\*\*Fix:\*\*** swap in a real token (or drop the line).

---

**\*\*Holding LGTM\*\*** on the \$6 blocker ? the runbook documents a broken, and sole, path for changing the delivery target. It's a one-line fix (point the script, or the doc, at `policies.config.json`). Everything else is verified accurate and the gate is green; ping me when \$6 is corrected and I'll re-verify and sign off.

<sub>? Reviewed with [Claude Code](https://claude.com/claude-code) ? the `no such table: filter\_config` failure and the missing `delivery\_record\_failed` token both reproduced on the branch against a freshly-migrated (`user\_version=5`) DB. Multi-agent adversarial pass: 3 findings kept, 0 refuted (the two design.md hits are the same issue).</sub>

#### 817. user (2026-07-07T03:50:52.413Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr58\_review.md (file state is current in your context ? no need to Read it back)

#### 818. assistant (2026-07-07T03:50:54.335Z)

Now let me post it to GitHub as a PR review comment.

#### 819. user (2026-07-07T03:51:00.216Z)

<https://github.com/avidullu/telegram-video-filter/pull/58#issuecomment-4899934073>

--- posted ---

docs: refresh current project documentation | agent/docs-sweep |

<https://github.com/avidullu/telegram-video-filter/pull/58>

#### 820. assistant (2026-07-07T03:51:04.737Z)

Review posted to [PR #58](https://github.com/avidullu/telegram-video-filter/pull/58#issuecomment-4899934073). Now let me clean up the worktree and re-arm the watcher.

#### 821. user (2026-07-07T03:51:09.295Z)

=== remove PR#58 worktree ===

removed

C:/Users/avidu/Projects/Telegram 7fadffb [main]

=== current open PR baseline for watcher ===

#58 draft=false sha=3069f1801 upd=2026-07-07T03:50:59Z

#### 822. assistant (2026-07-07T03:51:35.572Z)

Worktree removed cleanly; `main` is the only checkout again. Only open PR is **\*\*#58\*\*** at `sha=3069f1801`, `upd=2026-07-07T03:50:59Z` (already includes my just-posted comment, so the watcher won't false-fire on it). Re-arming the watcher ? it fingerprints every open PR and exits (re-invoking me) on any new PR, **\*\*SHA change\*\*** (force-push ? re-review) or **\*\*updatedAt change\*\*** (comment ? judge & respond).

#### 823. user (2026-07-07T03:51:40.684Z)

Command running in background with ID: bit0xilej. Output is being written to:

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bit0xilej.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 824. assistant (2026-07-07T03:51:54.196Z)

Done with the #58 cycle. Summary:

#### PR #58 ? docs refresh ? reviewed, \*\*LGTM held\*\*

Posted [review](https://github.com/avidullu/telegram-video-filter/pull/58#issuecomment-4899934073). It's a strong, mostly-accurate sweep (gate green: \*\*562 passed @ 94.30%\*\* , ruff clean) that fixes several real doc bugs ? but it ships one broken operational path, so I held the stamp per your standing rule.

**Blocking** ? [docs/runbook.md:151](https://github.com/avidullu/telegram-video-filter) \$6 documents `set\_delivery\_target.py` as the **only** way to change the delivery target, but that script still does `UPDATE filter\_config ? WHERE user\_id=?` against a table that ADR 0011 renamed to `filter\_config\_deprecated` and `init\_db()` drops. **Reproduced** on a fresh migrated DB (`user\_version=5`): script SQL ? `OperationalError: no such table: filter\_config`; the equivalent `UPDATE policies ? WHERE account\_id=? AND is\_default=1` ? succeeds. The PR correctly migrated the **inline** SQL to `policies` but left the recommended helper script (same bug) in place, and there's no manual `\$.delivery\_target` fallback anywhere in the runbook. One-line fix (point the script ? or the doc ? at `policies.config\_json`).

**Low** ? `docs/design.md:218` observability list cites `delivery\_record\_failed`, which exists nowhere in `src/` (every other event in the list grep-matches); real tokens are `delivery\_failed`/`delivery\_retry\_failed` + the `\*\_in\_both\_mode` pair.

Both findings independently reproduced ? not taken on the subagents' word.

#### Housekeeping

- Worktree `\_wt-pr58` removed; `main` is the only checkout again.
- **Watcher re-armed** (bg `bit0xilej`, baseline `#58 sha=3069f1801`). It'll re-invoke me on a force-push (SHA change ? re-review), an author comment/thread update (updatedAt change ? judge & respond), or a new PR. I'll pick up #58's fix the same way when it lands.

#### 825. user (2026-07-07T03:57:44.144Z)

```
<task-notification>
<task-id>bit0xilej</task-id>
<tool-use-id>toolu_01JJSNNBpKgFgbiu9SwmPAWm</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bit0xilej.output</output-file>
<status>completed</status>
<summary>Background command "Re-arm background PR watcher (force-push + comment + new-PR aware)"
completed (exit code 0)</summary>
</task-notification>
```

#### 826. user (2026-07-07T03:57:47.373Z)

```
1      WATCHER ARMED 2026-07-07T03:51:42Z
2      baseline:
3      #58 draft=false sha=3069f1801 upd=2026-07-07T03:50:59Z
4      === PR CHANGE DETECTED 2026-07-07T03:57:44Z ===
5      --- was ---
6      #58 draft=false sha=3069f1801 upd=2026-07-07T03:50:59Z
7      --- now ---
8      #58 draft=false sha=fda42c241 upd=2026-07-07T03:56:24Z
9      (sha change=force-push?re-review; upd-only=comment?judge; new #=new PR?review)
10
```

#### 827. assistant (2026-07-07T03:57:54.709Z)

Force-push detected on **\*\*#58\*\***: SHA `3069f1801` ? `fda42c241` (~6 min after my review). The author pushed a fix. Re-review cycle. Let me fetch and see exactly what changed since the commit I reviewed.

## 828. user (2026-07-07T03:58:00.082Z)

```
* branch          agent/docs-sweep -> FETCH_HEAD
+ 3069f18...fda42c2 agent/docs-sweep -> origin/agent/docs-sweep (forced update)
=== commits on branch since the sha I reviewed (3069f1801) ===
fda42c2 docs: refresh current project documentation
```

=== files changed in the force-push ===

```
docs/design.md          | 3 +-
docs/runbook.md         | 3 +-
scripts/set_delivery_target.py | 29 ++++++-----
tests/test_scripts.py    | 58 ++++++-----
4 files changed, 80 insertions(+), 13 deletions(-)
```

=== full diff of the fix ===

```
diff --git a/docs/design.md b/docs/design.md
index 1c112c4..07028cf 100644
--- a/docs/design.md
+++ b/docs/design.md
@@ -215,7 +215,8 @@ The app logs to stdout. Important events include:
- `client_connected`
- `video_seen`
- `video_forwarded`
-- `delivery_record_failed`
+- `delivery_failed`
+- `delivery_retry_failed`
- `flood_wait`
- `config_updated`
- policy-stage warnings for invalid config or failed deep inspection
diff --git a/docs/runbook.md b/docs/runbook.md
index 030c318..678892b 100644
--- a/docs/runbook.md
+++ b/docs/runbook.md
@@ -148,7 +148,8 @@ Targets:
| `@your-feed` | Public channel or chat username. |
| `-1001234567890` | Numeric Telegram chat/channel id. |
```

-Use the helper script for targets:

+Use the helper script for targets. It updates `delivery\_target` on the  
+account's default policy in `policies.config.json`:

```
```powershell
.\.venv\Scripts\python.exe scripts\set_delivery_target.py <account_id> <target>
diff --git a/scripts/set_delivery_target.py b/scripts/set_delivery_target.py
index 0026735..2d6b726 100644
--- a/scripts/set_delivery_target.py
+++ b/scripts/set_delivery_target.py
@@ -1,11 +1,11 @@
-"""One-shot helper: set delivery_target for a user.
+"""One-shot helper: set delivery_target on an account's default policy.
```

Usage (run from the repo root with the VENV python)::

```
# Windows (PowerShell)
- .\\.venv\Scripts\python.exe scripts\set_delivery_target.py <user_id> <target>
+ .\\.venv\Scripts\python.exe scripts\set_delivery_target.py <account_id> <target>
# macOS / Linux
- .venv/bin/python scripts/set_delivery_target.py <user_id> <target>
+ .venv/bin/python scripts/set_delivery_target.py <account_id> <target>
```

Examples::

```

@@ -29,11 +29,14 @@ import sys

def main() -> int:
    if len(sys.argv) != 3:
        print("Usage: python scripts/set_delivery_target.py <user_id> <target>",
file=sys.stderr)
+    print(
+        "Usage: python scripts/set_delivery_target.py <account_id> <target>",
+        file=sys.stderr,
+    )
    print(" target: 'me', '@username', or a chat id like -1001234567890", file=sys.stderr)
    return 2

-    user_id = int(sys.argv[1])
+    account_id = int(sys.argv[1])
    target = sys.argv[2].strip()

    # Read db_path from settings so .env / env vars are honoured.
@@ -45,14 +48,17 @@ def main() -> int:
    conn = sqlite3.connect(db_path)
    try:
        cur = conn.execute(
-            "UPDATE filter_config "
+            "UPDATE policies "
            "SET config_json = json_set(config_json, '$.delivery_target', ?) "
-            "WHERE user_id = ?",
-            (target, user_id),
+            "WHERE account_id = ? AND is_default = 1",
+            (target, account_id),
        )
        conn.commit()
        if cur.rowcount == 0:
-            print(f"ERROR: no filter_config row for user_id={user_id}.", file=sys.stderr)
+            print(
+                f"ERROR: no default policy row for account_id={account_id}.",
+                file=sys.stderr,
+            )
        print("Run `python -m tg_video_filter.bootstrap` first.", file=sys.stderr)
        return 1

    finally:
@@ -61,11 +67,12 @@ def main() -> int:
    # Show the resulting config so you can verify.
    conn = sqlite3.connect(db_path)
    row = conn.execute(
-        "SELECT config_json FROM filter_config WHERE user_id = ?", (user_id,)
+        "SELECT config_json FROM policies WHERE account_id = ? AND is_default = 1",
+        (account_id,),
    ).fetchone()
    conn.close()

-    print(f"? delivery_target for user {user_id} set to: {target}")
+    print(f"OK: delivery_target for account {account_id} set to: {target}")
    print(f" current config: {row[0] if row else '(none)'}")
    print("\nRestart the manager for the change to take effect:")
    print(" python -m tg_video_filter          # or")
diff --git a/tests/test_scripts.py b/tests/test_scripts.py
new file mode 100644
index 0000000..1b39b29
--- /dev/null
+++ b/tests/test_scripts.py
@@ -0,0 +1,58 @@
+"""Tests for operational helper scripts."""
+
+from __future__ import annotations
+

```

```

+import json
+import os
+import sqlite3
+import subprocess
+import sys
+from pathlib import Path
+
+import aiosqlite
+
+from tg_video_filter import db
+from tg_video_filter.models import FilterConfig, User
+
+
+async def test_set_delivery_target_updates_default_policy(tmp_path: Path) -> None:
+    db_path = tmp_path / "app.db"
+    await db.init_db(str(db_path))
+    async with aiosqlite.connect(db_path) as conn:
+        await db.upsert_user(conn, User(id=111, phone="+15555550100"))
+        await db.save_filter_config(conn, 111, FilterConfig(delivery_target="me"))
+
+    env = os.environ.copy()
+    env.update(
+        {
+            "DB_PATH": str(db_path),
+            "TG_API_ID": "1",
+            "TG_API_HASH": "abcdefgh",
+            "ADMIN_USER_IDS": "",
+        }
+    )
+    result = subprocess.run(
+        [
+            sys.executable,
+            "scripts/set_delivery_target.py",
+            "111",
+            "@curated-feed",
+        ],
+        cwd=Path(__file__).resolve().parents[1],
+        env=env,
+        check=False,
+        capture_output=True,
+        text=True,
+    )
+
+    assert result.returncode == 0, result.stderr
+    assert "OK: delivery_target for account 111 set to: @curated-feed" in result.stdout
+    conn = sqlite3.connect(db_path)
+    try:
+        row = conn.execute(
+            "SELECT config_json FROM policies WHERE account_id = 111 AND is_default = 1"
+        ).fetchone()
+    finally:
+        conn.close()
+    assert row is not None
+    assert json.loads(row[0])["delivery_target"] == "@curated-feed"

```

## 829. assistant (2026-07-07T03:58:38.628Z)

The author's force-push fixes **both** findings at the root, cleanly:

- **Blocker:** `set_delivery_target.py` now does `UPDATE policies ? WHERE account_id=? AND is_default=1` (my recommended fix (a)), renames `user_id` to `account_id` throughout, updates the read-back + error messages, and **adds a subprocess test** (`tests/test_scripts.py`) that runs the script against a migrated DB and asserts the policy row updates. Runbook §6 clarified too.
- **Low:** `delivery_record_failed` ? `delivery_failed` + `delivery_retry_failed` (the real tokens).

Now I verify it myself. First a caution: I must ensure the gate actually tests the `**worktree's**` code, not the venv's editable install pointing at the main checkout. Let me re-add the worktree and confirm import resolution before trusting any numbers.

#### 830. user (2026-07-07T03:58:44.172Z)

```
Preparing worktree (detached HEAD fda42c2)
HEAD is now at fda42c2 docs: refresh current project documentation
=== checked-out sha ===
fda42c241a493d20fa0350b8ffb91b2794707480
=== import resolution WITHOUT forcing (cwd=worktree) ===
imports from: C:\Users\avidu\Projects\Telegram\src\tg_video_filter
=== import resolution WITH PYTHONPATH=worktree/src ===
imports from: C:\Users\avidu\Projects\wt-pr58\src\tg_video_filter
=== does the venv have an editable/install pin? ===
default resolves to: C:\Users\avidu\Projects\Telegram\src\tg_video_filter
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

#### 831. assistant (2026-07-07T03:59:08.337Z)

Confirmed my caution was warranted: by default the venv imports ``tg_video_filter`` from `**Telegram/src`**` (main, not the PR branch) ? only ``PYTHONPATH=$WT/src`` points it at the worktree. The script fix runs via subprocess against the worktree's file regardless, but I'll force ``PYTHONPATH`` for the whole gate so the package code and ``test_scripts.py`` under test are the branch's. First, an independent standalone repro of the blocker fix (my own eyes, not just the bundled test):

#### 832. assistant (2026-07-07T04:00:20.374Z)

API Error: Connection closed mid-response. The response above may be incomplete.